Cursus

De programmeertaal C

Frank B. Brokken

01 11

De programmeertaal C

Frank B. Brokken en Karel Kubat

Zetwerk en redactie: ICCE, Groningen

Alle rechten voorbehouden. Niets uit deze uitgave mag worden verveelvoudigd, opgeslagen in een geautomatiseerd gegevensbestand of openbaar worden gemaakt in enige vorm of op enige wijze, hetzij elektronisch, mechanisch, door fotokopieën, opnamen of enige andere manier, zonder voorafgaande schriftelijke toestemming van de uitgever. De enige uitzondering die hierop bestaat is dat eventuele programma's en door de gebruiker in te typen voorbeelden mogen worden ingevoerd, opgeslagen en uitgevoerd op een computersysteem, zolang deze voor privé-doeleinden worden gebruikt, en niet bestemd zijn voor reproduktie of publikatie.

Correspondentie inzake overnemen of reproduktie kunt u richten aan:
ICCE, Rijks Universiteit Groningen, Postbus 335, 9700 AH Groningen.

All rights reserved. No part of this book may be reproduced, stored in a database or retrieval system, or published, in any form or in any way, electronically, mechanically, by print, photoprint, microfilm or any other means without prior written permission from the publisher.

All correspondence regarding copyrights, translations or any other matter can be directed to:
ICCE, Rijks Universiteit Groningen, Postbus 335, 9700 AH Groningen.

Ondanks alle aan de samenstelling van de tekst bestede zorg, kan noch de redactie noch de uitgever aansprakelijkheid aanvaarden voor eventuele schade, die zou kunnen voortvloeien uit enige fout, die in deze uitgave zou kunnen voorkomen.

(c) Copyright 1995 ICCE, Rijks Universiteit Groningen, Groningen

Voorwoord

C *1. (nat.) het teken voor de honderddelige schaal van de thermometer van Celsius; — 2. (elektr.)* coulomb; *— 3. (scheik.)* carboneum, *koolstof; — 4. (handel)* courant, *kapitaal,* conto; *— 5. (op auto's)* Cuba.
symbool voor de lichtsnelheid.

Naast bovengenoemde betekenissen van de letter C, die afkomstig zijn uit de tiende druk van Van Dale's Groot Woordenboek der Nederlandse Taal (Kruyskamp, 1976), is **C** ook de naam van de programmeertaal die in 1978 door Brian Kernighan en Dennis Ritchie is ontwikkeld. Door een aardige samenloop van omstandigheden is de letter C het symbool voor de lichtsnelheid en zijn programma's die in de programmeertaal **C** zijn ontwikkeld doorgaans snel. Maar, zoals opgemerkt, dat is louter een coïncidentie ....

De programmeertaal **C** is het centrale thema van dit boek. Het boek bestaat uit twee delen: in het eerste deel worden de eigenschappen van de taal zelf behandeld, in het tweede deel worden gevorderde onderwerpen behandeld. In het eerste deel van het boek staat de grammatica (de syntaxis) van **C** centraal. Het eerste deel kan dan ook goed worden gebruikt bij een eerste bestudering van de taal **C**. In het tweede deel van het boek wordt gebruik gemaakt van de mogelijkheden die de programmeertaal **C** biedt. In dit deel worden onderwerpen als complexe datatypen, grammatica's en codegeneratoren behandeld. De programmeertaal **C** wordt in dit deel gebruikt als "implementatievehikel", waarbij we bij de lezer een grondige bekendheid met de programmeertaal **C** veronderstellen.

De programmeertaal **C** is op veel computers beschikbaar. We hebben er daarom naar gestreefd de inhoud van de hoofdstukken "machine onafhankelijk" te laten zijn in de zin dat de behandelde stof relevant is voor software-ontwikkelaars die gebruik maken van de programmeertaal **C**, in plaats van relevant voor personen die specifiek over machines van het merk *X* beschikken. Wijzelf werken momenteel voornamelijk met IBM-PS/2 computers, waarop versie 6.00-A van de Microsoft **C** vertaler beschikbaar is. In een paar hoofdstukken (zoals hoofdstuk 6 over de preprocessor en hoofdstuk 12 over geheugenmodellen) wordt specifiek ingegaan op de Microsoft vertaler en/of de architectuur van de IBM computers. Deze hoofdstukken bevatten uiteraard vrij veel "machine-afhankelijke" informatie. De Microsoft vertaler wordt ook op andere plaatsen in het boek expliciet genoemd. Wie gebruik maakt van andere vertalers (zoals Turbo-C van Borland) zal doorgaans zonder problemen in staat zijn de Microsoft-specifieke opmerkingen te relateren aan de eigen situatie.

In zekere zin is dit boek een lustrumuitgave. Het studiejaar 1991-1992 is het vijfde jaar dat wij in Groningen cursussen over de programmeertaal **C** geven. De hoofdstukken van dit boek zijn dan ook gebaseerd

op collegedictaten en syllabussen die we de afgelopen jaren hebben samengesteld ten behoeve van de studenten die onze cursussen volgen. Wat betreft deze groep: zelden zagen we een meer gevarieerde groep dan 'onze' **C**-cursisten: studenten informatica, wiskunde, natuurkunde, letteren, sociale wetenschappen, en geneeskunde zijn standaard aanwezig bij onze **C**-cursussen.

De stijl die we bij het onderwijs hanteren wordt in dit boek ook teruggevonden. Bij de ontwikkeling van een programma wordt een verbaal gesteld programmeerprobleem in een aantal 'slagen' geherformuleerd tot een stukje proza waarin in de gebiedende wijs wordt gesteld wat er moet gebeuren. Daarna wordt dit proza 'ingebed' in een zogenaamd 'code-frame' waarin al kenmerken van de programmeertaal **C** worden herkend. Tenslotte wordt het code-frame omgezet naar pure **C**-brontekst. Bij complexere problemen wordt bij de herformulering expliciet en uitgebreid aandacht geschonken aan de organisatie van de data die door het programma zullen worden bewerkt. De ervaring leert dat een goede data-organisatie de ontwikkeling van het programma sterk vereenvoudigt. Om te voorkomen dat het wiel opnieuw moet worden uitgevonden bieden we (in deel twee) *blauwdrukfuncties* aan voor de implementatie van complexe datastructuren zoals *queues, stacks, lijsten* en *bomen*. De blauwdrukfuncties kunnen worden aangepast aan concrete situaties zonder dat de manier om de datastructuren te bewerken opnieuw hoeft te worden bedacht. In onze cursussen besteden we uitgebreid aandacht aan het veel gehoorde probleem dat **C** een *write-only* taal zou zijn: de brontekst van eenmaal ontwikkelde **C** programma's zou niet meer begrijpelijk zijn, zelfs niet voor de ontwikkelaar ervan. Ons inziens is het pure nonsense om deze generalisatie over de programmeertaal **C** te maken. In bijlage D geven we regels voor de layout van **C** bronteksten waarmee de onleesbaarheid wordt voorkomen. In onze cursussen *eisen* we dat de layout van gemaakte opgaven aan deze conventies voldoen. Hier kunnen we er uiteraard alleen de nadruk op leggen dat diegenen die serieus programmeren in **C** goede layoutregels volgen. De regels die in dit boek worden gegeven zijn standaard layoutregels en we raden de lezers dan ook aan programma's volgens deze regels op te stellen. In de hoofdstukken van dit boek worden opgaven gegeven waarbij het vaak de bedoeling is dat stukken brontekst worden ontwikkeld. Hierbij dient niet alleen aandacht te worden geschonken aan de oplossing van een probleem, maar vooral ook aan de layout van de geschreven brontekst.

Tot een aantal jaren geleden waren we super-enthousiaste gebruikers van de taal **pascal.** Op dit moment is er evenwel bij de computers van het Interdisciplinair Centrum voor de ontwikkeling van Computercoaches en Expertsystemen (ICCE) geen pascalvertaler meer in gebruik: **C** heeft als productietaal de taal **pascal** volledig verdrongen. Misschien zijn we nu **C**-goeroes geworden? De ongeveer vijf personen die zich bij het ICCE met software-ontwikkeling bezig houden hebben in ieder geval in een paar

jaar tijd een indrukwekkende ervaring en productiecapaciteit opgebouwd, waarbij de programmeertaal **C**, incidenteel in combinatie met functies die in een assembleertaal zijn geschreven, steeds de gebruikte ontwikkeltaal was (en is). We hopen dat de lezers net zoveel plezier beleven aan het "werken met **C**" als wij. Als dat het geval is, dan is onze opzet volledig geslaagd.

Uiteraard verdienen een aantal personen met name te worden genoemd. Wij zijn met name dank verschuldigd aan:

- Klaas (prof. dr.) B. Koster, voor de fiattering van de aanschaf van de LaserJet printer waarmee we de zetsels van dit boek hebben kunnen afdrukken.
- Piet ("PeeWee") W. Plomp, voor de nauwgezetheid waarmee hij de computersystemen van het ICCE beheert en aan de praat weet te houden, ondanks de "tube-wars" die regelmatig uitbreken en waaronder hij toch ook zal lijden.
- Erik-Jan ("Vunzo") Vens, voor de indrukwekkende hulp die hij ons heeft geboden bij het gebruik van de typesettingsystemen TEX en LATEX.
- Fabian ("Febojan") F. M. Vermeer, voor de eerste opzet van de expressiekraker uit hoofdstuk 16, maar vooral voor het beschikbaar stellen van de ijskast waarin we het bier in de hete zomer van 1991 koud hebben kunnen houden.
- Ad Weterings, voor het feit dat hij ons een aantal jaren geleden heeft geleerd te "werken met C".

Groningen, zomer 1991

F.B.B.
K.K.

Inhoudsopgave

1 **Inleiding** *15*
1.1 *De geschiedenis van C* *16*
1.1.1 De filosofie van C *16*
1.1.2 Standaard-C en ANSI-C *18*
1.2 *Layout van C brontekst* *18*
1.2.1 Commentaar in bronteksten *19*
1.3 *Code en data* *19*
1.3.1 Constanten *21*
1.4 *Talstelsels, bits, nibbles, bytes, words* *21*
1.4.1 Rekenen in verschillende talstelsels *22*
1.4.2 De rekeneenheden van de computer *22*
1.4.3 Negatieve getallen *23*
1.5 *Gebruikte software* *25*
1.5.1 Een software-ontwikkelomgeving *25*
1.5.2 Organisatie van de software *30*
1.6 *Het 'Hello World' programma* *33*
1.7 *Het 'Repeat' programma* *35*

I DE PROGRAMMEERTAAL C *37*

2 **Variabelen** *39*
2.1 *Naamgeving van identifiers* *40*
2.2 *Types in C* *41*
2.2.1 Standaard data types *41*
2.2.2 Type and access modifiers *42*
2.2.3 Het default data type *43*
2.3 *Definiëren van variabelen* *43*
2.3.1 Lokale en globale variabelen *45*
2.3.2 Static variabelen *48*
2.3.3 Register variabelen *49*
2.3.4 Declaratie van variabelen: externe variabelen *50*
2.3.5 Initialisatie bij definitie *50*
2.4 *Arrays en strings* *52*
2.4.1 Initialisatie bij definitie *54*

3 **Waardetoekenning en operatoren** *57*
3.1 *Waarvoor dienen assignments* *58*
3.2 *Unaire operatoren* *58*
3.3 *Binaire operatoren* *62*
3.4 *Assignment operatoren* *64*
3.5 *Overige operatoren* *66*
3.6 *Type checking en type conversions* *67*

3.7 *Logische expressies en logische operatoren* *69*
3.7.1 Nul en niet-nul *70*
3.7.2 Volgorde van evaluatie bij logische expressies *70*
3.7.3 De wetten van De Morgan *72*

4 **Flow control** ***73***
4.1 *Selectie statements* *74*
4.1.1 De enkelvoudige if-else *74*
4.1.2 De geneste if-else *75*
4.1.3 if-else ladders *76*
4.1.4 De ternaire operator ?: *77*
4.1.5 De switch *79*
4.2 *Repeteer statements* *81*
4.2.1 De while *81*
4.2.2 De do-while *83*
4.2.3 De for *83*
4.3 *Gerelateerde statements* *85*
4.3.1 De break *85*
4.3.2 De continue *86*
4.3.3 De functie exit() *87*
4.3.4 De return *87*
4.3.5 goto en labels *88*

5 **Functies** ***91***
5.1 *Inleiding* *92*
5.2 *Definiëren van functies* *92*
5.2.1 Parameters van functies *95*
5.2.2 Declareren van functies: prototyping *97*
5.2.3 Het functietype: de return-waarde *99*
5.2.4 Functies met variabel aantal argumenten *100*
5.2.5 De functie main() *102*
5.3 *Functies en programma-opbouw* *106*
5.3.1 Het probleem van het toiletpapier *112*

6 **De preprocessor** ***119***
6.1 *De include directive* *120*
6.2 *De define directive* *121*
6.2.1 Defines zonder argumenten *121*
6.2.2 Defines met argumenten *122*
6.3 *Overige directives* *124*
6.3.1 Ifdef *124*
6.3.2 Ifndef *125*
6.4 *Uitvoer van de preprocessor* *125*

7 **Input/output en Files** ***127***
7.1 *Inleiding* *128*

7.1.1 Input en output *128*
7.2 *Buffered I/O en unbuffered I/O* *128*
7.3 *Formated I/O en unformated I/O* *131*
7.3.1 Functies voor formated I/O *131*
7.3.2 De format string *132*
7.3.3 Een applicatie: dechex *133*
7.4 *Het openen van een stream* *137*
7.4.1 De FILE pointer *138*
7.4.2 De functie fopen() *138*
7.4.3 Tijdelijke streams *141*
7.5 *Overige functies en opmerkingen* *141*
7.5.1 Het gebruik van niet-gebufferde I/O *141*
7.5.2 Het positioneren van een stream *143*
7.5.3 Het gebruik van de functie open() *147*

8 **Pointers** ***149***
8.1 *Pointeroperatoren* *150*
8.1.1 Definitie van pointervariabelen *150*
8.1.2 Waardetoekenning *150*
8.1.3 Rekenen met pointers *155*
8.2 *Allocatie van geheugen* *157*
8.2.1 De functies malloc() en free() *157*
8.2.2 De grootte van het te alloceren geheugen *159*
8.2.3 De functie realloc() *159*
8.3 *Multiple indirection* *161*
8.3.1 Arrays van pointers *162*
8.3.2 Pointers naar functies *166*
8.4 *Frequente fouten* *171*
8.4.1 Wilde pointers *172*
8.4.2 Beschrijven van niet gealloceerd geheugen *172*
8.5 *De Variable Definer: VD* *174*
8.5.1 De ontwikkeling van VD *175*

9 **Nieuwe types** ***183***
9.1 *De typedef* *184*
9.2 *De struct* *185*
9.2.1 Bitvelden bij structs *188*
9.3 *De union* *189*
9.4 *Het gebruik van structs en unions* *189*
9.4.1 Geneste structs en unions *190*
9.4.2 Initialisatie van structs en unions *192*
9.4.3 Allocatie van structs en unions *193*
9.5 *De enum* *194*
9.6 *Een database applicatie* *197*
9.6.1 De functie main() *199*

9.6.2 De functie readbase() *201*
9.6.3 De functie mainmenu() *202*
9.6.4 De functie listbase() *203*
9.6.5 De hulpfuncties *205*

10 **Werken met bits** ***209***
10.1 *Het werken met bits* *210*
10.2 *Het genereren van priemgetallen* *210*
10.2.1 Een eenvoudig algoritme *210*
10.2.2 De Zeef van Aristothenes *212*

11 **Code splitsen** ***223***
11.1 *Inleiding* *224*
11.2 *Hoe te splitsen* *224*
11.3 *Bibliotheken en de library manager* *226*
11.4 *Bibliotheken en de linker* *227*
11.5 *Een applicatie: movelib* *228*
11.5.1 De hoofdtaak van movelib *229*
11.5.2 Het verwerken van objectmodules *231*
11.5.3 Het uitvoeren van een DOS-opdracht *234*
11.6 *Een help-systeem applicatie* *235*
11.6.1 Inleiding *235*
11.6.2 Organisatie van het databestand *236*
11.6.3 Organisatie van het indexbestand *237*
11.6.4 Probleemanalyse en implementatie *239*
11.6.5 Het tonen van help-informatie *253*

II GEVORDERD GEBRUIK VAN C *257*

12 **Geheugenmodellen** ***259***
12.1 *Inleiding* *260*
12.2 *Segment Registers* *261*
12.2.1 Code *261*
12.2.2 Data *262*
12.2.3 Stack *263*
12.2.4 Nomenclatuur bij adresseringen *264*
12.3 *Modellen* *265*
12.3.1 Small geheugenmodel: optie: /As *266*
12.3.2 Medium geheugenmodel: optie: /Am *266*
12.3.3 Compact geheugenmodel: optie: /Ac *267*
12.3.4 Large geheugenmodel: optie: /Al *267*
12.3.5 Huge geheugenmodel: optie: /Ah *268*
12.4 *Gemengde modellen* *268*
12.4.1 Data *269*

12.4.2 Code *273*
12.4.3 Return-waarden van functies *275*

13 **Iteratieve processen** ***279***
13.1 *Inleiding 280*
13.1.1 Annuïteitsberekening *280*
13.1.2 Een ecologisch evenwicht *281*
13.2 *De zin die zichzelf beschrijft 285*
13.2.1 Waarom iteratie *286*
13.2.2 Organisatie van de data en eerste herschrijving *286*
13.2.3 De functie main() *288*
13.2.4 De functie count_sentence() *288*
13.2.5 Een uitbreiding *290*

14 **Transitiematrixen en Eindige Toestandsautomaten** ***291***
14.1 *De eindige toestandsautomaat 292*
14.2 *De transitiematrix 295*
14.3 *Een toepassing: de string grabber 297*
14.3.1 Het transitiediagram van de string grabber *298*
14.3.2 Toestanden en herkenningstekens *302*
14.3.3 Acties via een transitiematrix *303*
14.3.4 Implementatie van de string grabber *304*

15 **Recursieve processen** ***313***
15.1 *Inleiding 314*
15.2 *Recursieve faculteitberekening 317*
15.3 *Verzamelingen 319*
15.3.1 Implementatie *321*
15.4 *De paardesprongpuzzel 322*
15.4.1 Analyse van de puzzel *324*
15.4.2 Implementatie *325*
15.5 *Toepassingen: Quicksort en Binair zoeken 331*

16 **Grammatica's en grammaticaherkenners** ***335***
16.1 *Inleiding 336*
16.1.1 Het nut van grammatica's *336*
16.2 *Een grammaticavoorbeeld: taal 336*
16.3 *Bouwstenen van grammatica's 339*
16.3.1 Alternatieven *339*
16.3.2 Herhalingen van één of meer elementen *339*
16.3.3 Herhalingen van nul of meer elementen *341*
16.3.4 Herhalingen met scheidingstekens *341*
16.3.5 Nestingen van niet-herhaalde elementen *342*
16.3.6 Nestingen van herhaalde elementen *342*
16.4 *Conflicten in een grammatica 343*
16.4.1 Onvoltooide regelconflicten *343*

16.4.2 Keuzeconflicten *344*
16.5 *Een eenvoudige rekenmachine* *345*
16.5.1 Herkenning van de invoer *346*
16.5.2 Implementatie *353*
16.5.3 Een uitbreiding *357*

17 **Gevorderde datatypen** ***359***
17.1 *Inleiding* *360*
17.2 *Stacks* *360*
17.2.1 Het rangeerprobleem *363*
17.2.2 Analyse van het probleem *363*
17.2.3 Implementatie *365*
17.3 *Queues* *369*
17.3.1 Een wachtkamer als voorbeeld *372*
17.4 *Lijsten* *377*
17.4.1 Het vorige lijstelement *383*
17.4.2 De functie lagged_listsearch() *385*
17.4.3 Een lijst van namen als voorbeeld *387*
17.5 *Bomen* *390*
17.5.1 Huffman compressie *396*
17.6 *Hashing* *410*
17.6.1 Collisies *413*
17.6.2 Vullen, zoeken en verwijderen *415*
17.6.3 Het tellen van woorden in een bestand *417*
17.6.4 Het sorteren van de hash-tabel *419*

18 **Codegeneratoren bison en flex** ***421***
18.1 *Inleiding* *422*
18.2 *Flex en lexical analyzers* *424*
18.2.1 De specificatie van een lexical analyzer *424*
18.2.2 Reguliere expressies in de flex-specificatie *427*
18.2.3 Hoe yylex() reguliere expressies vindt *429*
18.2.4 Miniscanners *430*
18.2.5 De opdracht flex *433*
18.3 *Bison en parsers* *434*
18.3.1 De specificatie van een parser *434*
18.3.2 De acties in een bison-specificatie *436*
18.3.3 Hoe yyparse() met yylex() communiceert *437*
18.3.4 ASCII-waarden als terminale symbolen *439*
18.3.5 Bison en expressies *440*
18.3.6 Fouten bij het gebruik van bison *442*
18.4 *Een applicatie: rasic* *443*
18.4.1 Implementatie van rasic *447*
18.4.2 De datatypen en de data van rasic *447*
18.4.3 Het inlezen van een robotprogramma *450*

18.4.4 Het uitvoeren van een robotprogramma *460*
18.5 *Speciale kenmerken van bison en flex* *472*
18.5.1 De functie yylex() en meerdere invoerbestanden *472*
18.5.2 De bison-specificatie en het aangeven van fouten *473*
18.5.3 Het herstel van de parser na foutcondities *474*
18.6 *Een fout in flex* *476*

19 **Heuristieken** ***479***
19.1 *Inleiding* *480*
19.2 *De invoer voor het roosterprogramma* *482*
19.3 *Decompositie van het probleem* *484*
19.3.1 Opstellen van tweetallen *484*
19.3.2 De constructie van de roosters *487*
19.4 *Implementatie van het roosterprogramma* *490*
19.4.1 De header file *490*
19.4.2 De hulpfuncties *492*
19.4.3 De functie main() *495*
19.4.4 De functie schedule_day() *497*
19.4.5 De functie solve() *500*
19.4.6 De functie solveday() *501*
19.4.7 De functie addsol() *503*

III BIJLAGEN *507*

A **Syntaxis van C** ***509***
A.1 *De grammatica van C* *509*
A.2 *De keywords van C* *515*
A.3 *Escape-sequences* *515*
A.4 *Operatoren, associativiteit en prioriteit* *516*

B **Standaard-C versus ANSI-C** ***519***

C **Variabele aantallen argumenten** ***521***

D **Layout van C brontekst** ***525***

E **Unix System V Flex Description** ***529***

F **Antwoorden op opgaven** ***535***

G **Literatuurverwijzingen** ***573***

Lijst van figuren

1.1 One's complement: 4 naar -4. *24*
1.2 Anatomie van de **C** vertaler. *27*
1.3 Van een **C** brontekst naar een uitvoerbaar programma. *29*
1.4 Overzicht van de voorgestelde directory-structuur. *31*

2.1 Globale en lokale variabelen in het datasegment. *48*
2.2 Arrays en hun elementen. *53*

3.1 De bitwise and bewerking: 5 & 6 == 4. *63*
3.2 De bitwise exclusive or bewerking: 5 ^ 6 == 3. *64*
3.3 Compound assignment operatoren. *65*
3.4 Logische operatoren in volgorde van prioriteit. *70*
3.5 Notaties voor waarde nul. *71*
3.6 Waarheidstabel bij *!(a && b)*. *72*

5.1 De toiletpapier-simulatie. *113*

7.1 Het bufferen van i/o. *130*
7.2 Functies voor gebufferde i/o. *130*
7.3 Formated i/o functies. *132*
7.4 Vaak gebruikte stuurcodes in format strings. *133*
7.5 Drie buffers bij gebufferd kopiëren. *142*
7.6 Constanten gebruikt bij de functie *open()*. *147*

8.1 Waardetoekenning aan *value_ptr*. *152*
8.2 Een pointer naar een array-element. *156*
8.3 *argv* bij *main():* multipele indirectie. *162*
8.4 Een array van pointers en een tweedimensionaal array. *163*
8.5 Toegestane variabele definities. *175*

10.1 Getallen ontleed in *ints* en bits. *216*

11.1 Verwijzingen in het indexbestand naar het databestand. *238*
11.2 De tabel *SYMTAB_*. *245*

12.1 Near en far calls bij functies. *262*
12.2 Near en far pointers naar near en far data. *271*
12.3 De definitie van near en far pointers. *272*
12.4 Zichtbaarheid van hulp- en interface-functies. *274*
12.5 Prototypes van near/far functies die near/far pointers retourneren. *276*

13.1 Resultaten van het ecologische model. *285*

14.1 De toestanden van de koffiezetter. *294*
14.2 Een transitiediagram voor een grottenstelsel. *296*
14.3 De gevulde transitiematrix. *297*
14.4 De string grabber: transitiediagram voor de toestand *start*. *300*

15.1 Deelverzamelingen van drie elementen. *321*
15.2 De paardesprong op een 5 x 5 bord. *323*
15.3 Een leeg 2 x 2 bord bij *horse.c*. *327*

16.1 Een eenvoudige grammatica van taal. *337*
16.2 Ontleding van een zin. *338*
16.3 De ontleedboom bij een links-recursieve regel. *340*
16.4 De ontleedboom bij een rechts-recursieve regel. *341*
16.5 Ontleedbomen van het 'dangling-else' conflict. *344*
16.6 De evaluatieboom van expressie 12 – 5 – 2. *347*
16.7 De ontleedboom van expressie 12 – 5 – 2. *347*
16.8 De ontleedboom van expressie 12 – 3 ∗ 2. *348*
16.9 Grammaticale regels voor een expressie. *349*
16.10 Ontleding van 12 – 3 ∗ –2 (eerste deel). *350*
16.11 Ontleding van 12 – 3 ∗ –2 (tweede deel). *351*
16.12 Ontleding van 12 – 3 ∗ –2 (derde deel). *352*

17.1 Het rangeeremplacement bij Oostum. *364*
17.2 De circulaire buffer (queue). *371*
17.3 De lijststructuur. *378*
17.4 De constructie van een lijst. *381*
17.5 Het verwijderen uit een lijst. *382*
17.6 Een boom met zeven knopen. *391*
17.7 Een boom voor de expressie 3 ∗ 5 + 2 ∗ 7. *393*
17.8 De constructie van de binaire boom. *399*
17.9 De uiteindelijke bitpatronen. *399*
17.10 De bit-representatie van een binaire boom. *404*
17.11 Het verband tussen zoektijden en zoekalgoritmes. *413*

18.1 Een programma met functies *yyparse()* en *yylex()*. *423*
18.2 Speciale symbolen in reguliere expressies. *429*
18.3 De quote-miniscanner als eindige toestandsautomaat. *432*
18.4 Het aardappelveld bij het *rasic* spel. *448*

19.1 Voorbeeld van de invoer. *483*
19.2 Kruistabel voor paarvorming. *484*
19.3 De beste paren voor vijf dagen. *486*
19.4 Wegen langs paren. *488*
19.5 De eerste negen pogingen om een oplossing te vinden. *489*
19.6 Poging om een oplossing te vinden, uitgaande van dag 1. *490*
19.7 De organisatie van de data. *493*
19.8 De organisatie van het array *solution*. *503*

C.1 Twee *int* argumenten op de stack. *522*

Hoofdstuk 1

Inleiding

1.1 De geschiedenis van C

In 1066 veroverde Willem de Veroveraar Engeland. Het zou echter nog ruim negen eeuwen duren voordat Brian W. Kernighan en Dennis M. Ritchie in 1978 de programmeertaal C formuleerden. De wortels van **C** liggen in de '**B** Computer Programming Language' (**bcpl**), een taal die in 1969 ontwikkeld werd. De **C** taal evolueerde uit **bcpl** via een verbeterde versie, die **B** genoemd werd.

Al deze talen hebben de eigenschappen van een hogere programmeertaal: meerdere statements kunnen in blokken gegroepeerd worden, onder andere zijn er constructies voor conditionele uitvoering (*if*) en constructies voor herhaalde uitvoering (*while*). Bovendien kennen al deze talen het begrip pointer[1], staan het rekenen met pointers toe ('pointer arithmetic') en kennen het samenstellen van grote eenheden uit kleinere datatypen (structs, unions, arrays, enzovoort).

De talen **bcpl** en **B** zijn echter op één basis-data element gericht: het machine-woord van de computer[2]. **C** daarentegen staat ook andere elementen toe: characters (letters, zoals 'a'), floating point values (getallen met cijfers achter de komma), enzovoort.

1.1.1 De filosofie van C

B.W. Kernighan en D.M. Ritchie hebben **C** ontwikkeld met een aantal duidelijke doelstellingen. In de eerste plaats moest **C** een snelle taal worden, geschikt voor een breed scala van toepassingen en met name gericht op *produktie*. Dit lijkt een triviale opmerking, maar gezien de achtergrond van een aantal talen is deze design-overweging niet onbelangrijk. De programmeertaal **pascal** werd primair ontwikkeld voor onderwijsdoeleinden, daarom maken de 'ingebakken' faciliteiten als boundary checking'[3] programma's die in **pascal** geschreven zijn in het algemeen trager dan programma's in **C**. **Basic** werd ontwikkeld om programmeren voor een breed publiek toegankelijk te maken – kleine programma's zijn in **basic** snel te implementeren, maar voor grote projecten is **basic** ongeschikt.

C is een taal geworden die alle aspecten van een 'low level language' als **assembler** in zich heeft, en zich dus uitstekend leent voor taken die zeer dicht bij de architectuur ('hardware') van de computer liggen. Daarnaast heeft **C** alle mogelijheden van een hogere programmeertaal als **fortran** en **pascal**. Vermoedelijk heeft deze 'best of two worlds'-aanpak **C** zeer populair gemaakt. In het UNIX bedrijfsssysteem heeft **C** **assembler** zelfs bijna geheel verdreven, alleen de eerste paar instructies van UNIX (bij het laden

1. Een pointer is een verwijzing naar een variabele of functie.
2. Een machine-woord is een geheugeneenheid van een computer. Op de PC is dit een 16-bits waarde. Deze eenheid komt overeen met het *int* type van **C**.
3. Boundary checking van **pascal** komt erop neer dat bij een variabele een bereik opgegeven mag worden. Als de waarde van die variabele buiten dit bereik komt, dan breekt het programma met een foutmelding af.

van het bedrijfssysteem zelf, de 'bootstrapping') zijn in **assembler** geschreven.

Een tweede overweging bij de opzet van **C** is de splitsing van de **C** vertaler en van de **C** functiebibliotheek. De **C** vertaler *sec* 'kan niets' behalve het omzetten van een brontekst naar instructies. **C** wordt daarom ook een 'kale' taal genoemd. Alle handelingen van een programma die voor een gebruiker zichtbaar zijn zoals: het verschijnen van tekst op het beeldscherm, het invoeren van een getal, het bewerken van een bestand, zijn geen eigenschappen van de **C** vertaler, maar van de meegeleverde functies in een **C** functiebibliotheek.

Deze scheiding is niet in alle talen aanwezig. Zo splitst de **pascal** vertaler een *write* statement waarmee een integer[4] en een string[5] worden weggeschreven op in twee elementen (een *writeint* deel en een *writestring* deel); het *write* statement is in deze context dus deel van de taal **pascal**. **C** daarentegen zal alle argumenten van een *printf* [6] statement doorgeven aan een functie uit een bibliotheek; de functie zelf moet dan uitzoeken of het om een string of integer gaat. De verantwoordelijkheid voor het juist functioneren van een functie ligt daarmee niet bij de constructeurs van de **C** vertaler, maar bij de schrijvers van bibliotheekfuncties.

De scheiding tussen vertaler en functiebibliotheek heeft in het begin van het bestaan van **C** tot verhitte, bijna theologische, discussies geleid. Aan de ene kant worden programma's in **C** door deze scheiding bijzonder goed 'portable' (dat wil zeggen overdraagbaar naar andere computers); de **C** vertaler is immers 'kaal' en kan daarmee snel op verschillende computers geïmplementeerd worden. Na de implementatie van een **C** vertaler kunnen ook andere programma's worden overgezet. Aan de andere kant kan een **C** programma niet veel als de functiebibliotheken op die andere computer niet aanwezig zijn. Bovendien zal een **C** programmeur een goede kennis moeten hebben van de functies die in de **C** bibliotheek aanwezig zijn; het gebruikersgemak van een programma valt of staat immers met deze input/output functies. Een voorstander van **C** schreef in 1983 het volgende commentaar[7].

> *"**C** is not a perfect language, but it certainly beats what's in second place. Consequently, my enthousiasm about **C** makes me very chauvinistic about misplaced and invalid criticisms. I have a couple of minor complaints about some aspects of **C**, but I bite my tongue when I think about the dark ages. After several happy years with **algol** in the 1960s, I was sentenced to over ten years of **fortran** purgatory before being born again with **C**. I guard this language jealously, and you had better be careful of what you write or I'll curse you to a task of debugging 10,000 lines of **basic** code."*

4. Een integer is een variabele die een geheel getal bevat.
5. Een string is data bestaande uit tekst.
6. De *printf()* functie schrijft data naar het beeldscherm.
7. Uit: Dr. Dobb's Toolbook of **C**

De filosofie van scheidingen en van modulaire opbouw is ook in de **C** bibliotheken terug te vinden. Zo bevatten de meeste functiebibliotheken een aantal low level input/output functies[8], die dichtbij het besturingssysteem van de computer staan. Deze low level functies worden in een tussenlaag gebruikt, bij voorbeeld in een functie die een string wegschrijft[9]. De tussenlaag wordt wederom bij een 'high level' functie als *printf()* gebruikt.

1.1.2 Standaard-C en ANSI-C

De eerste versie van **C**, door B.W. Kernighan en D.M. Ritchie in 1978 geformuleerd, staat bekend als standaard-**C**. Sinds deze versie is **C** zeer populair geworden, en daarmee zijn **C** vertalers naar vele computers overgebracht. Met deze overdracht verschenen al snel verschillende **C** dialecten; versies die op standaard-**C** gebaseerd waren, maar op kleine punten afweken.

In 1983 besloot het 'American National Standards Institute' (ANSI) een commissie in te stellen die de **C** taal "eenduidig en machine-onafhankelijk" zou definiëren. Het resultaat van deze definitie is ANSI-**C**.

De meeste moderne vertalers voldoen aan de ANSI definitie. Er zijn echter op sommige computers (met name grotere mainframes en UNIX-machines) **C** vertalers te vinden die alleen de oorspronkelijke standaard-**C** ondersteunen. Hoewel dit boek gericht is op de Microsoft **C** vertaler, die aan de ANSI standaard voldoet, verdient het daarom aanbeveling om van deze verschillen tussen standaard-**C** en ANSI-**C** op de hoogte te zijn. Een aantal van die verschillen is in bijlage B beschreven.

1.2 Layout van C brontekst

Layout van **C** brontekst is geen triviaal punt; het succes van een programma is onder meer afhankelijk van de leesbaarheid van de bronteksten. De eis van een duidelijke layout heeft zowel voor de schrijver van het programma als voor anderen voordelen. De schrijver moet in staat zijn een fout snel op te kunnen sporen, en moet de oorspronkelijke code kunnen gebruiken om verbeterde en uitgebreidere versies van een programma op te stellen.

Ook andere software-ontwikkelaars moeten in staat zijn de bronteksten te kunnen begrijpen. Vaak worden bronteksten van nuttige programma's via elektronische netwerken verspreid; de leesbaarheid van de code vergroot dan de levensvatbaarheid van een programma. De ontvanger zal niet geneigd zijn onleesbare bronteksten naar een computer van zijn/haar keuze over te zetten.

We suggereren daarom de in dit boek aangehouden layout-conventies nauwkeurig op te volgen. Deze conventies zijn in bijlage D weergegeven.

---

8. b.v. functies *write()*, *read()*, *open()*
9. b.v. functie *fputs()*

1.2.1 Commentaar in bronteksten

Een aanverwant onderwerp, waarmee de leesbaarheid van bronteksten vergroot kan worden, is de aanwezigheid van commentaar. Een goede beschrijving van een brontekst in termen als *"wat doet deze code"* of *"waarvoor dient deze variabele"* kan essentieel zijn als een programma op fouten nagekeken of uitgebreid wordt.

C ondersteunt twee types commentaar, die gewoonlijk aangeduid worden als standaardcommentaar en end-of-line commentaar. Standaardcommentaar is gedefinieerd in de door B.W. Kernighan en D.M. Ritchie opgestelde versie van **C**. Dit commentaar wordt begonnen door '/*', kan meerdere regels beslaan, en wordt afgesloten door '*/'. End-of-line commentaar is gedefinieerd in de ANSI **C** versie, begint met twee schuine strepen ('//'), en loopt door tot het einde van de regel.

In de Microsoft implementatie van **C** zijn beide types commentaar beschikbaar.

1.3 Code en data

Elk programma bestaat, globaal gezien, uit twee delen: code en data. De code is dat deel van een programma dat iets 'doet': informatie op het beeldscherm zet, een bestand aanmaakt, en dergelijke. De data zijn het 'substraat' van de code; hetgene dat bewerkt wordt. In deze context is het Deense woord voor computer een aardige wetenswaardigheid: 'datamaskine', oftewel gegevensverwerker.

Als voorbeeld noemen we hier een editor: de code van zo'n programma zorgt ervoor dat een letter tussengevoegd wordt, dat een paragraaf uitgelijnd wordt, enzovoort. De data is hier de bewerkte tekst. Bij zo'n programma worden de data op een specifieke manier georganiseerd; een editor zal gewoonlijk de bewerkte tekst regelgewijs opslaan en verwerken. Bij andere programma's, zoals bij de code die door de DOS opdracht *copy* geactiveerd wordt, kunnen de data minder specifiek georganiseerd zijn: *copy* zal informatie kopiëren zonder dat de aard van de informatie relevant is.

Veel software-ontwikkelaars besteden veel tijd aan de opzet van de code van een programma. Toch moet gesteld worden dat de organisatie van de data van een programma minstens evenzo belangrijk is. Optimalisatie (sneller en efficiënter maken) van een programma waarbij alleen de code verfraaid wordt, zal weinig uitmaken als de organisatie van de data niet goed zit. Zelfs kan gezegd worden dat de constructie van een gecompliceerd programma bij de opzet en organisatie van de data begint.

De terminologie 'code en data' introduceert voor de **C** programmeur een aantal nieuwe termen:

Variabelen zijn data-elementen die tijdens het verloop van een programma verschillende waarden kunnen aannemen. Zo kan een variabele die in een editor gebruikt wordt en het aantal regels in een bestand telt, oplopen van nul tot het aantal ingelezen regels.

Constanten zijn data-elementen die tijdens het verloop van een programma dezelfde waarde houden. Strikt gezien zijn alle strings, getallen, enzovoort, die letterlijk in een brontekst voorkomen, constanten. In het statement:

```
a = 12;
```

(lees: *a* wordt *12*) is symbool *a* een variabele die van een waarde voorzien wordt. Getal *12* is een constante. Verschillende soorten constanten worden in 1.3.1 besproken.
De term 'constante' wordt tevens voor symbolische namen gebruikt die door de preprocessor opdracht *#define* aangemaakt worden. Dit is in 1.5.1 toegelicht.

Statements zijn elementen van de code die òf wel, òf niet uitgevoerd worden. Een statement is daarmee een niet onder te verdelen stukje code; het 'atoom' van het codegedeelte van een programma.

Compound statements zijn gegroepeerde losse statements, en worden ook wel *blokken* genoemd. Door statements in blokken te groeperen kan de programmeur uit meerdere, op zichzelf ondeelbare eenheden, één 'cluster' maken dat op zijn beurt weer ondeelbaar is. Een compound statement wordt dus òf wel, òf niet uitgevoerd. Binnen een compound statement kunnen de afzonderlijke statements op hun beurt weer òf wel, òf niet uitgevoerd worden, afhankelijk van de situatie.
Een compound statement wordt in **C** aangeduid door een '{', gevolgd door statements en afgesloten door '}'. Elk statement binnen het blok kan op zijn beurt weer een blok zijn; dit wordt het *nesten* van blokken genoemd.

Functies bestaan uit compound statements die binnen een programma onder een symbolische naam bekend staan. Samen vervullen de afzonderlijke statements binnen zo'n compound statement gewoonlijk één bepaalde taak, die ook door de functienaam gesuggereerd wordt. Een software-ontwikkelaar kan bij voorbeeld het opzoeken en openen van bestanden die voor een programma relevant zijn, groeperen in een functie met de toepasselijke naam *open_files*.
Het opdelen van de code in 'logische' delen is aan de software-ontwikkelaar overgelaten.
Functies 'doen iets' met variabelen die door andere functies weer gebruikt worden. Bovendien kunnen functies aan elkaar waarden doorgeven: deze doorgegeven data-elementen worden 'return' waarden en argumenten genoemd.

Lokale data kunnen door functies als een kladgebied gebruikt worden. **C**

staat hiermee toe dat tijdelijke variabelen wanneer nodig aangemaakt worden en later weer verdwijnen. In tegenstelling tot lokale data staan de globale data; dit zijn variabelen die altijd aanwezig zijn en die te allen tijde gebruikt kunnen worden.

1.3.1 Constanten

De **C** vertaler ondersteunt verschillende notaties voor constanten. Hieronder worden deze notaties toegelicht.

Numerieke constanten zijn getallen in de brontekst. De vertaler ondersteunt decimale en hexadecimale notatie.

- Bij decimale notatie bestaat een getal uit de cijfers 0 tot en met 9. Bij een reëel getal (een getal dat een breuk representeert) kan een punt met een serie cijfers volgen. Tevens kan bij reële getallen van de wetenschappelijke notatie gebruik gemaakt worden, waarbij het getal door de letter 'e' of 'E' en het exponent gevolgd wordt.
- Naast de decimale notatie kunnen gehele getallen in hexadecimale notatie gerepresenteerd worden. Hierbij wordt *0x* vóór het getal geplaatst, het getal zelf bestaat uit de cijfers 0 tot en met 9 en a tot en met f. Cijfers a tot en met f kunnen ook als hoofdletters voorkomen. (Zie 1.4 voor een beschrijving van het hexadecimale stelsel.)

Character constanten zijn lettertekens in de brontekst. Een character constante bestaat uit een letterteken tussen enkele aanhalingstekens. De vertaler zet een character constante om naar de bijbehorende ASCII waarde; het is voor de vertaler daarom onbelangrijk of een letterteken als letter (bij voorbeeld als 'a') of als getal (bij voorbeeld als 97 of 0x61) opgegeven wordt.
Tussen de enkele aanhalingstekens kunnen in plaats van lettertekens *stuurcodes* of 'escape-sequences' opgegeven worden. Deze escape-sequences zijn in bijlage A.3 toegelicht. Een voorbeeld van de escape-sequence *n* is op pagina 35 gegeven.

String constanten zijn stukken tekst in de brontekst. Strings zijn series lettertekens tussen dubbele aanhalingstekens; zie het *hello* programma op pagina 34.

1.4 Talstelsels, bits, nibbles, bytes, words

In **C** kunnen gehele getallen in verschillende talstelsels gerepresenteerd worden. In dit onderdeel worden talstelsels toegelicht en geheugeneenheden ter opslag van getallen besproken.

1.4.1 Rekenen in verschillende talstelsels

Elk talstelsel wordt gekenmerkt door een grondtal. Dit getal specificeert hoe de afzonderlijke cijfers van een getal geïnterpreteerd worden en hoeveel mogelijke cijfers het talstelsel kent. De mens rekent gewoonlijk in het decimale (tientallige) stelsel: elk cijfer is daarmee een macht van tien, en er zijn tien cijfers (0 tot en met 9). Het grondtal van dit talstelsel zal wellicht met het aantal vingers van de mens te maken hebben [10].

Bij elk getal in een willekeurig talstelsel wordt het meest rechtse cijfer beschouwd als het aantal eenheden, oftewel een aantal dat met grondtal tot de macht 0 overeenkomt. Het cijfer links ernaast staat voor een aantal dat overeenkomt met het grondtal tot de macht 1 (de tientallen in het decimale stelsel). Deze regel is voor alle overige cijfers toepasbaar. Het meest linkse cijfer wordt hierbij het *meest significante* cijfer genoemd, aangezien dit cijfer de meeste informatie aan het totaal bijdraagt.

Het decimale getal 1962 levert zo de volgende waarde op:

Cijfer	Levert decimaal op	
1	$* 10^3 =$	1000
9	$* 10^2 =$	900
6	$* 10^1 =$	60
2	$* 10^0 =$	2
totaal		1962

Bij veel programmeertalen, waaronder **C**, worden naast het decimale stelsel hexadecimale (zestientallige), octale (achttallige) en binaire (tweetallige) stelsels gebruikt. Bij het binaire stelsel worden twee cijfers (0 en 1) gebruikt, bij het octale stelsels zijn cijfers 0 tot en met 7 beschikbaar. Bij het hexadecimale stelsel worden zestien cijfers gebruikt, die gewoonlijk met 0 tot en met 9 en vervolgens a tot en met f aangeduid worden (cijfers a tot en met f kunnen ook als hoofdletter voorkomen). Hierbij staat bijvoorbeeld cijfer c voor decimaal 12. Het hexadecimale getal a12b levert zo het volgende decimale getal op:

Cijfer	Levert decimaal op	
a	$* 16^3 =$	40960
1	$* 16^2 =$	256
2	$* 16^1 =$	32
b	$* 16^0 =$	11
totaal		41259

1.4.2 De rekeneenheden van de computer

In tegenstelling tot de mens heeft de computer niet zoveel aan het decimale stelsel. Dit heeft met de basale architectuur van elke chip te maken: deze

10. Wie de plaatjes van Marten Toonder nauwkeurig bestudeert zal tot de conclusie komen dat Olivier B. Bommel in het octale (achttallige) stelsel rekent.

bestaat uit schakelaars, die òf aan, òf uit kunnen zijn. Computers worden daarom ook 'binaire rekenapparaten' genoemd[11]. Elk van deze schakelaars representeert daarmee òfwel cijfer 0, òfwel cijfer 1.

Deze eenheden worden 'bits' genoemd. Deze term is een samentrekking van binary digit. Aan bits heeft de programmeur gewoonlijk weinig: meestal moeten elementen worden opgeslagen die meer dan twee waarden kunnen aannemen. Het ligt daarom voor de hand om bits te groeperen in veelvouden. Zo kunnen in een eenheid van twee bits vier verschillende waarden (00, 01, 10, en 11, oftewel 2^2 mogelijkheden) worden opgeslagen; in drie bits acht verschillende waarden (2^3), etc..

Groepen van twee of drie bits worden echter gewoonlijk niet gebruikt. De eerste vaak gebruikte veelvoud van bits is de nibble, een groep van vier bits. In één nibble kunnen 16 mogelijke waarden (2^4) opgeslagen worden, bijvoorbeeld getallen 0 t/m 15. Eén nibble kan als één hexadecimaal cijfer (0 t/m f) gerepresenteerd worden.

De volgende opslageenheid is het byte. Een byte bestaat uit twee nibbles oftewel acht bits. In één byte kunnen 256 verschillende waarden (2^8) opgeslagen worden, bijvoorbeeld getallen 0 t/m 255. Bytes worden vaak gebruikt om letters en andere tekens uit de ASCII[12] tabel op te slaan. Een byte bestaat dus uit twee nibbles, en kan als twee hexadecimale cijfers genoteerd worden. Analoog aan het meest significante cijfer kan bij een byte van het meest significante nibble worden gesproken.

De term 'word', ofwel machinewoord, wordt gebruikt om aan te geven met welke opslageenheid de computer gewoonlijk werkt. Op computers die op de Intel 8088 chip[13] gebaseerd zijn, is een word een zestienbits waarde. Daardoor kunnen op de PC in één word 65536 verschillende waarden opgeslagen worden. Naast word is nog een double word of dword beschikbaar. Deze eenheid is twee maal zo groot als word.

1.4.3 Negatieve getallen

Variabelen hebben in computers altijd een bepaald waardebereik. Dit bereik wordt bepaald door de hoeveelheid geheugen die beschikbaar is voor die variabelen. Zo bestaan er in **C** variabelen die in een byte (dus 8-bits) zijn opgeslagen. De grootste waarde die in een byte kan worden opgeslagen is 255, welke waarde is opgeslagen wanneer alle 8 bits van het byte zijn gezet.

Wanneer bij een variabele een rekenkundige bewerking wordt uitgevoerd, verandert de hoeveelheid geheugen die beschikbaar is voor die

11. Een aardigheidje hierbij is dat in het begin van de zestiger jaren aan de RijksUniversiteit Groningen met een toen zeer moderne computer, de ZEBRA, gewerkt werd. Deze afkorting staat voor 'zeer eenvoudige binaire rekenautomaat'.
12. American Standard Code for Information Interchange. Volgens deze afspraak wordt aan elk letterteken een bepaald nummer toegekend.
13. Deze chip is in de IBM XT gebruikt. De ervan afgeleide chips (de 8086, 80286, 80386 en 80486) zijn in nieuwere XT's, in AT's en in de PS/2 serie gebruikt.

4	0	0	0	0	0	1	0	0
-4	1	1	1	1	1	0	1	1

one's complement

Figuur 1.1: One's complement: 4 naar -4.

variabele natuurlijk niet: een 8-bits byte blijft een 8-bits byte. Wanneer bij een byte-waarde van 255 de waarde 1 wordt opgeteld is het resultaat 0. Deze berekening is vergelijkbaar met de berekening $99+1=0$, wanneer we uitgaan van het gegeven dat er slechts twee decimale getallen beschikbaar zijn: 100 past niet meer in twee decimale getallen, en op de twee decimale getalsposities die beschikbaar zijn staat 0, zodat $99+1=0$. Evenzo bij bytes: $255+1=256$, maar 256 is (binair) 100000000. Aangezien voor een byte 8 bits beschikbaar zijn, past de 1 niet meer in het byte, en is $255+1=0$.

Wanneer de waarde van een variabele ook negatief mag zijn, verandert dat wederom niets aan de hoeveelheid geheugenruimte die door die variabele in beslag wordt genomen. Toch moet de informatie dat het getal negatief is ook worden opgeslagen. Wanneer de waarde van een variabele ook negatief mag zijn, wordt een bit gebruikt om aan te geven of de waarde positief of negatief is. Dit bit, dat *altijd* het meest significante bit van een variabele is, heet het *teken*-bit of sign-bit.

De eenvoudigste manier om een getal van teken te doen wisselen is wellicht het omzetten van alle bits: alle 1-bitjes worden 0, alle 0-bitjes worden 1. Dit wordt de one's complement methode genoemd. In een byte met waarde 4 wordt de waarde -4 bereikt zoals aangegeven in figuur 1.1. We kunnen een waarde in een byte waarin het hoogste bit is gezet beschouwen als een negatieve waarde en als een positieve waarde. Het ligt er maar aan wat we willen. Wanneer we besluiten het sign-bit te beschouwen als een gewoon numeriek bit, representeert dit bit de waarde 2^7. De waarde -4 zoals gerepresenteerd in figuur 1.1 kan dus ook gerepresenteerd als de (positieve) waarde 251. Wanneer we aan een byte een waarde toekennen tussen 128 en 255 bereiken we daarmee dat het hoogste (meest significante) bit wordt gezet. Of we vervolgens die waarde als positief of als negatief willen beschouwen is een *keuze,* en geen *noodzaak*. We kunnen een variabele een negatieve waarde geven door er een zo grote positieve waarde aan toe te kennen dat het sign bit wordt gezet.

Wat is nu de negatieve waarde volgens het one's complement van 255? Wanneer alle bits worden omgezet krijgen we 0. Er zijn in het one's complement dus twee manieren om de waarde 0 te representeren: 0 zelf en de situatie waarin alle bits zijn gezet, een waarde die ook wel als -0 wordt

aangeduid.

Een andere manier om negatieve waarden te representeren bestaat uit het two's complement. De two's complement methode wordt gebruikt door de INTEL 80x86 processors. Om een getal volgens de two's complement methode van teken te laten wisselen worden de volgende twee stappen uitgevoerd:

- Het one's complement van de waarde wordt bepaald,
- De waarde 1 wordt bij de one's complement waarde opgeteld.

Wanneer volgens de two's complement methode de representatie van -4 wordt gezocht gegeven de waarde 4, gaat dat dus als volgt:

- De one's complement waarde van 4 is (positief decimaal) 251
- De two's complement waarde -4 is (positief decimaal) 252

Wanneer op de waarde -4 (two's complement) de methode nogmaals wordt losgelaten, ontstaat wederom de waarde 4:

- De one's complement waarde van -4 (positief decimaal 252) is 3.
- De two's complement waarde van -4 is $3 + 1 = 4$.

Bij de two's complement methode zijn er geen twee representaties meer voor 0: -0 bereiken we als 255 (one's complement) $+1 = 0$. Op dezelfde wijze kunnen we ook het teken wisselen bij een getal waarin *alleen* het sign-bit staat (wederom uitgaande van het byte): de two's complement waarde van -128 is 127 (one's complement) $+1 = -128$. Kennelijk kan de waarde -128 niet van teken wisselen: doordat één representatie van 0 is vervallen, is er ruimte voor een extra waarde ontstaan. Deze extra waarde is -128: een negatieve waarde omdat het sign-bit is gezet.

Samenvattend is de waarde -0 in one's complement notatie -1 in two's complement notatie. De meest negatieve waarde in two's complement wordt bereikt wanneer alleen nog het sign-bit is gezet. Die negatieve waarde kan niet van teken wisselen.

Opgave 1.

Bereken (uitgaande van 8-bits bytes) de volgende waarden als hexadecimaal, als decimaal, en als positief decimaal getal (gebruik hierbij de two's complement methode):

1. Het meest positieve getal
2. Het meest negatieve getal
3. Het tegengestelde van -128
4. 120
5. 135
6. -1.

1.5 Gebruikte software

1.5.1 Een software-ontwikkelomgeving

Voor het ontwikkelen van software in **C** is een aantal zaken, zoals een **C** vertaler, onontbeerlijk. Er zijn echter programma's ontwikkeld die aan een

'vriendelijke' ontwikkelomgeving bijdragen en hiermee het schrijven en onderhouden van software vergemakkelijken.

In dit onderdeel wordt de werking van noodzakelijke en handige hulpmiddelen van een software-ontwikkelomgeving toegelicht.

De C vertaler

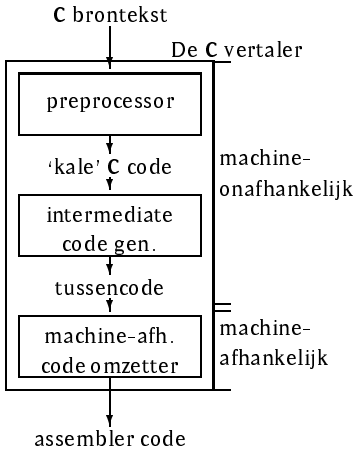
De **C** vertaler is de eerste stap bij de omzetting van een voor mensen leesbare brontekst[14] naar een programma dat door de computer uitgevoerd kan worden. Hierbij zet de vertaler een brontekst om naar een tussenformaat, dat door andere programma's die bij het onstaan van een programma betrokken zijn, gelezen kan worden. De **C** vertaler bestaat uit drie onderdelen: de preprocessor, de intermediate codegenerator en de machine-afhankelijke code-omzetter. Deze onderdelen worden hieronder toegelicht.

De Preprocessor. Het vertalen van een **C** brontekst begint bij de **C** preprocessor. Deze 'voor-vertaler' leest de **C** brontekst en zorgt er onder andere voor dat macro's (constanten) in de brontekst omgezet worden. Zo kan in een brontekst een macro *MAX* gedefiniëerd zijn als het getal 20. Na de voorvertaling staat op alle plaatsen waar *MAX* stond nu letterlijk het getal 20.

De preprocessor is een relatief eenvoudig, maar niet onbelangrijk deel van de **C** vertaler. Dankzij de preprocessor wordt het mogelijk in een **C** brontekst symbolische namen op te nemen, waardoor aanpassingen van een programma eenvoudig doorgevoerd kunnen worden. In het bovenstaande voorbeeld kan de definitie van *MAX* gewijzigd worden (bij voorbeeld in 30); hierna kan het programma opnieuw vertaald worden en functioneert dan gebaseerd op *MAX* is 30. Door de aanwezigheid van de preprocessor is de ontwikkelaar niet genoodzaakt de gehele brontekst te bewerken om '20' in '30' te wijzigen.

De intermediate codegenerator. De uitvoer van de preprocessor is 'kale' **C** code, ontdaan van macronamen. Deze uitvoer wordt door de intermediate codegenerator gelezen en omgezet naar (grotendeels) machine-onafhankelijke tussencode, waarbij de intermediate codegenerator tevens de **C** brontekst op fouten nakijkt. Een wetenswaardigheid hierbij is dat 75% van de code van de PCC vertaler (portable **C** compiler), een vertaler die in de vroege dagen van **C** zeer populair was, gemoeid was met machine-onafhankelijke omzettingen. Slechts 25% van de code behandelde de machine-afhankelijke code. Hierdoor kon de PCC vertaler snel overgebracht worden naar nieuwe computers, wat de populariteit ten goede kwam.

14. Bestanden met bronteksten hebben in **C** gewoonlijk een extensie *.c*.



Figuur 1.2: Anatomie van de **C** vertaler.

De tussencode lijkt al meer op de uiteindelijke machinetaal die zal ontstaan. Zo wordt een statement in een **C** brontekst opgesplitst in kleinere, functionele stukjes. Een **C** statement als:

```
a = b + 3;
```

(lees: *a* wordt *b* + *3*) wordt door de intermediate codegenerator vertaald naar pseudo-opdrachten als:

```
haal de waarde van b op
zet die waarde in een tijdelijk register
tel bij dit register 3 op
stop de waarde van dit register in variabele a
```

De machine-afhankelijke code-omzetter. De tussencode wordt door de machine-afhankelijke code-omzetter vertaald naar assemblercode. De assembler-code bevat een voor mensen leesbare representatie van de machinetaal die gegenereerd zal worden. Het bovenbeschreven **C** statement kan door een **C**-vertaler op de PC worden omgezet naar de assembler-instructies:

```
mov ax, b
add ax, 3
mov a, ax
```

De assemblercode kan bij de Microsoft **C** vertaler naar een bestand worden geschreven. Gewoonlijk heeft dit bestand dezelfde naam als de

brontekst, behalve de extensie die *.asm* is. De werking van een **C** vertaler is in figuur 1.2 nog eens geïllustreerd.

De assembler

De assembler heeft tot doel de uitvoer van de **C** vertaler in object-code om te zetten. Bij deze vertaalslag verliest de code zijn leesbaarheid voor mensen: object-code is in een 'binaire' representatie opgeslagen. De Microsoft vertaler schrijft de object-code naar een bestand met dezelfde naam als de brontekst, maar met de extensie *.obj*.

De uitvoer van de assembler kan door de computer nog niet geëxecuteerd worden. De assembler beschikt namelijk niet over alle informatie die nodig is om een programma te maken. In de code die de assembler omzet kan informatie staan die specificeert dat een stuk code, dat bekend staat als functie *printf()*, geactiveerd moet worden. Wat die functie doet, en waar de code van die functie staat, weet de assembler echter niet.

De uitvoer van de assembler wordt daarom ook wel 'relocatable object code' genoemd: de uitvoer is één van de objecten waaruit het uiteindelijke programma gebouwd wordt. De code bevat verder alleen verwijzingen naar andere objecten; deze referenties moeten nog ingevuld worden. De plaats die het object in het uiteindelijke programma zal innemen ligt nog niet vast; het object is relocatable (verplaatsbaar).

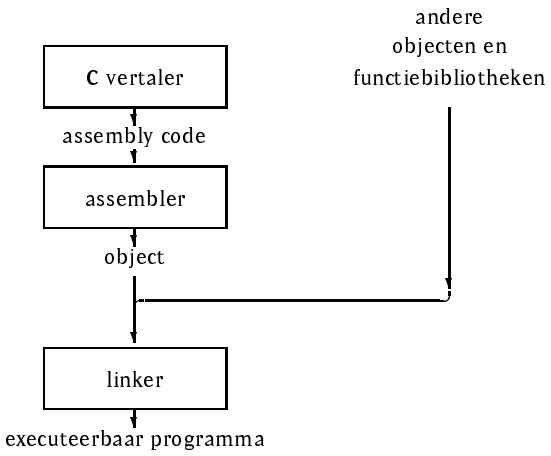
Opgemerkt dient te worden dat de assembler vaak een niet zichtbaar onderdeel van de **C** vertaler is: de opdracht die de **C** vertaler aanroept, roept (als bij de vertaling geen fouten optreden) ook vaak de assembler aan. Dit is ook bij de Microsoft vertaler het geval.

De linker

De invoer voor de linker zijn relocatable object modules van de assembler. De taak van de linker is het aaneenknopen van objects tot één uitvoerbaar programma. De linker moet hierbij alle objects langs lopen, en vaststellen welke functies, welke data, enzovoort gebruikt worden.

Vaak moet de linker de informatie in functiebibliotheken afzoeken om aangeroepen code in objects te kunnen vinden. Wanneer een **C** programma de **C** functie *printf()* gebruikt, dan zal de linker bij het verwerken van dit programma de code van de functie *printf()* uit een functiebibliotheek moeten halen om het uiteindelijke programma compleet te kunnen maken. De uitvoer van de Microsoft linker wordt naar een bestand geschreven dat dezelfde naam heeft als het eerste opgegeven object, maar dat de extensie *.exe* heeft.

Opgemerkt kan worden dat het linker proces kan ontbreken, zoals blijkt uit het volgende. Bij het NOS/VE besturingssysteem, gebruikt bij de CYBER mainframes van Control Data, heeft de gebruiker de mogelijkheid een object uit te laten voeren. Het besturingssysteem voert dan de link-



Figuur 1.3: Van een C brontekst naar een uitvoerbaar programma.

slag pas tijdens de uitvoering van het programma uit; het èchte programma wordt pas in het geheugen van de computer aangemaakt.

Het proces van de conversie van een C brontekst naar een uitvoerbaar programma is in figuur 1.3 nog eens geïllustreerd.

Overige software

Bij de ontwikkeling van programma's kan andere software, die niet noodzakelijk is voor het vertaalproces *sec*, uitermate handig zijn. We noemen hier de symbolische debugger, de library manager voor functiebibliotheken, een goede editor, de profiler, de project manager en de code-analyzer. Daarnaast kunnen codegeneratoren van groot belang zijn.

De symbolische debugger. Van oudsher waren debuggers alleen beschikbaar voor code op machinetaal-niveau. Met zulke debuggers kon de waarde van een register, de inhoud van een geheugenlocatie, de volgende uit te voeren instructie, enzovoort opgevraagd worden.

Met de komst van hogere programmeertalen verschenen symbolische debuggers. Deze programma's onderscheiden zich van de gewone debuggers doordat elementen van een programma bij symbolische naam genoemd kunnen worden. Een C programmeur heeft weinig boodschap aan het feit dat een variabele *a* zich bij voorbeeld op geheugenlocatie 1287 bevindt: eerder wil hij of zij deze locatie met de symbolische naam *a* kunnen aanduiden om op te vragen wat voor waarde de variabele heeft.

De library manager. De taak van een library manager is het bewerken van functiebibliotheken. Een functiebibliotheek is een combinatie van vele afzonderlijke verplaatsbare objecten in een formaat dat voor de linker gemakkelijk te bewerken is.

Met name bij grote software-pakketten zijn library managers handig: de ontwikkelaars hoeven zich niet bezig te houden met grote aantallen losse objecten die aan de linker ter bewerking doorgegeven moeten worden. In plaats daarvan worden de objecten in één functiebibliotheek geplaatst.

Andere hulpmiddelen. Voor een programmeertaal als **C** kan een toegespitste editor zeer handig zijn. Een voorbeeld hiervan is de EMACS editor van de GNU Software Foundation, en de hiervan afgeleide editor JOVE. Beide editors zijn in staat specifieke delen van een **C** brontekst te herkennen: als een sluitaccolade wordt ingetypt, zoekt de editor de daarbij behorende open accolade op en wijst die aan. De bij het ICCE ontwikkelde editor OE! is een verder voorbeeld.

Een ander handig hulpmiddel is de profiler. Dit programma voert een klaar **C** programma uit en bepaalt in welke delen van het programma de meeste tijd verloren gaat. Een goede profiler is onmisbaar als de softwareontwikkelaar een programma sneller probeert te maken: als de ontwikkelaar eenmaal de kritische delen van een programma gelokaliseerd heeft, kan hij of zij trachten deze onderdelen sneller te maken (te optimaliseren).

Voor grote projecten kan een project manager handig zijn. Een voorbeeld hiervan is het UNIX-programma *make*. Dit programma gaat ervan uit dat tussen bestanden afhankelijkheden bestaan. Wanneer een **C** brontekst gewijzigd wordt, zodat het bijbehorende bestand jonger is dan een al bestaand object met dezelfde naam, dan roept *make* de **C** vertaler aan om een recentere versie van het object te maken.

Veel programma's hebben gemeenschappelijke kenmerken, zoals het herkennen van invoer en het ondernemen van acties aan de hand van die invoer. Uit de vertalerbouw afkomstige codegeneratoren kunnen eveneens zeer nuttige gereedschappen zijn. Van deze gereedschappen komen *bison* en *flex* in dit boek uitgebreid aan de orde.

Tenslotte noemen we in dit rijtje de code analyzer zoals het UNIX-programma *lint*. Dit programma heeft veel gemeenschappelijk met de **C** vertaler: een **C** brontekst wordt gelezen en geanalyseerd. *Lint* probeert te achterhalen welke code-elementen in de **C** brontekst potentieel fouten kunnen opleveren.

1.5.2 Organisatie van de software

De installatie van een **C** vertaler gaat gewoonlijk gepaard met het aanbrengen van een aantal bestanden in verschillende directories op een harde

schijf. In dit deel beschrijven we een mogelijke directory-structuur voor de installatie van de Microsoft **C** vertaler onder DOS. We gaan hierbij uit van een opzet waarbij alle bestanden die bij de vertaler horen onder een directory *c* geïnstalleerd worden.

In de eerste plaats bestaat het vertalerpakket uit een aantal programma's, waaronder de **C** vertaler zelf, de library manager, de linker, enzovoort. Voorgesteld wordt om deze bestanden onder te brengen in directory *c**bin*. Deze directory moet door de systeemvariabele[15] *PATH* aangeduid worden, om ervoor te zorgen dat DOS bij aanroepen de programma's vindt.

In de tweede plaats horen bij een **C** vertaler functiebibliotheken. Het installatieprogramma van Microsoft's vertaler geeft de mogelijkheid slechts een subset van mogelijke bibliotheken te installeren. We raden aan om, in verband met de stof die in hoofdstuk 12 behandeld wordt (het werken met 'memory models'), bibliotheken voor alle geheugenmodellen te installeren. Voorgesteld wordt om functiebibliotheken (bestanden met de extensie *.lib*) een plaats te geven in directory *c**lib*. De functiebibliotheken moeten na een vertaling door de linker gelezen kunnen worden. Systeemvariabele *LIB* moet hiertoe deze directory aanwijzen.

In de derde plaats maakt de preprocessor van de vertaler gebruik van een aantal header files waarin externe functies gedeclareerd zijn. Deze bestanden, die gewoonlijk de extensie *.h* hebben, kunnen in een directory *c**inc* geplaatst worden. De preprocessor moet door de systeemvariabele *INCLUDE* over de plaats van de header files geïnformeerd worden.

Naast bovengenoemde directories raden we aan bronteksten van **C** programma's in een directory *c**source* te plaatsen (eventueel kunnen onder deze directory verdere subdirectories worden aangemaakt). Bovendien is het handig om een aantal batch-bestanden ten behoeve van de aanroep van de vertaler in *c**bat* te installeren. Deze batch-bestanden worden in sectie 1.5.2 verder besproken.

Directory	Bestanden	Aangewezen door systeemvariabele
\c\bat	*.bat	*PATH*
\c\bin	programma's	*PATH*
\c\inc	*.h	*INCLUDE*
\c\lib	*.lib	*LIB*
\c\source	*.c	

Figuur 1.4: Overzicht van de voorgestelde directory-structuur.

De voorgestelde directory-structuur is ook in figuur 1.4 geïllustreerd.

Voor een goede werking van de **C** vertaler moet een aantal systeem-

15. Systeemvariabelen zijn specifiek bepaalde waarden ten behoeve van het bedrijfssysteem. Zie voor verdere informatie de handboeken van DOS.

variabelen gezet worden. Onderstaande listing geeft een uitbreiding op het *autoexec.bat* bestand:

```
set PATH=\c\bat;\c\bin; .....
set INCLUDE=\c\inc
set LIB=\c\lib
set MM=S
```

Hier wordt op de plaats van de puntjes in de definitie van de *PATH* variabele het eigen pad gedefinieerd (zoals *\dos*). De laatste systeemvariabele in dit rijtje (*MM*) is het geheugenmodel (memory model) waarin gewerkt wordt. Voorlopig wordt deze variabele op 'S' ('small') gezet.

Overzicht en implementatie van opdrachten

We geven hieronder een overzicht van veel gebruikte opdrachten. Hoewel deze opdrachten op vele andere manieren gerealiseerd kunnen worden, raden we aan de hieronder voorgestelde naamgeving aan te houden; in de rest van dit boek wordt aan deze namen gerefereerd.

c file.c Deze opdracht roept de **C** vertaler aan om *file.c* te vertalen naar een object-module. Hierin staat 'file.c' voor een nader in te vullen naam van een bestand; de extensie *.c* is echter verplicht. De vertaler (*cl.exe*) wordt via een batch-bestand in directory *\c\bat* aangeroepen. Het batch-bestand ziet er als volgt uit:

```
@echo off
rem C.BAT -- calls the C compiler "cl"
rem DOS 3.3 and up
if "%1"==""" goto helpme
    cl /c /W2 /J /A%MM% %1 %2 %3 %4 %5 %6 %7 %8 %9
    goto done
:helpme
    echo Usage: C file.c [file.c ....]
    echo Activates C compiler to compile the file(s).
:done
```

In dit batch-bestand wordt de **C** vertaler met een aantal opties geactiveerd. Optie '/A' specificeert het gebruikte geheugenmodel, en wordt ingevuld aan de hand van de systeemvariabele *MM*. Het werken met verschillende geheugenmodellen komt in hoofdstuk 12 ter sprake. Overige opties staan in de met de vertaler meegeleverde documentatie beschreven.

cc file.c Deze opdracht werkt analoog aan opdracht *c*, maar verzoekt de vertaler om informatie voor de symbolische debugger *codeview*[16] in het gegenereerde object op te nemen. De opdracht is via een batchbestand *cc.bat* in directory *\c\bat* geïmplementeerd:

```
@echo off
rem CC.BAT -- calls the C compiler "cl", compiles for codeview
```

16. De symbolische debugger *codeview* wordt met de Microsoft vertaler meegeleverd.

```
rem DOS 3.3 and up
if "%1"=="" goto helpme
    cl /c /Zi /Od /W2 /J /A%MM% %1 %2 %3 %4 %5 %6 %7 %8 %9
    goto done
:helpme
    echo Usage: CC file.c [file.c ....]
    echo Activates compiler to compile the file(s), includes
    echo information for codeview in the object files.
:done
```

l file(s) Deze opdracht roept de linker aan om *file(s)* aaneen te knopen tot één programma. Bij de specificatie van de bestanden hoeft de extensie *.obj* niet gespecificeerd te worden. De opdracht is geïmplementeerd door middel van batch-bestand *l.bat* in directory *\c\bat*:

```
@echo off
rem L.BAT -- calls the linker to make an exe file
rem DOS 3.3 and up
if "%1"=="" goto helpme
    link /E /NOI %1 %2 %3 %4 %5 %6 %7 %8 %9 ;
    goto done
:helpme
    echo Usage: L file(s)
    echo Links files together to one executable.
:done
```

lc file(s) Deze opdracht werkt analoog aan *l*, maar verzoekt de linker om informatie voor de symbolische debugger *codeview* in de gegenereerde programma op te nemen. De implementatie is via batch-bestand *lc.bat* in directory *\c\bat*:

```
@echo off
rem LC.BAT -- calls the linker to make an .exe file, including
rem codeview information in the executable
rem DOS 3.3 and up
if "%1"=="" goto helpme
    link /CO /NOI %1 %2 %3 %4 %5 %6 %7 %8 %9 ;
    goto done
:helpme
    echo Usage: LC file(s)
    echo Links files together to one executable, including
    echo information for codeview into the .exe file.
:done
```

1.6 Het 'Hello World' programma

De volgende opgaven gaan over een eenvoudig **C** programma, dat de boodschap 'Hello World' naar het scherm schrijft. Onder **C** programmeurs is dit klassieke code: het eerste boek over **C** van B.W. Kernighan en D.M. Ritchie

```
1     #include <stdio.h>
2
3     void main ()
4     {
5         printf ("Hello World!\n");
6     }
```

Listing 1: Het 'Hello World' programma.

begint met dit voorbeeld. De brontekst van het 'Hello World' programma is als listing 1 weergegeven.

In regel 1 van dit programma is een preprocessor directive terug te vinden[17]. De preprocessor zal bij het verwerken van deze opdracht het bestand *stdio.h* lezen, en de inhoud ervan op de plaats van de directive zetten. Deze header file wordt door de preprocessor in eerste instantie in de huidige directory gezocht. Als het bestand daar niet gevonden kan worden, probeert de preprocessor het bestand in de directory te vinden die door de systeemvariabele *INCLUDE* wordt aangewezen.

In deze header file staat een aantal functies van de bij de compiler behorende functiebibliotheek gedeclareerd. Dankzij de informatie in dit bestand weet de vertaler nu dat er een functie *printf()* bestaat, en zal bij de aanroep van de functie geen waarschuwende melding worden gegenereerd.

Let ook op de layout van de brontekst; conform de layout-regels begint de preprocessor directive op de eerste kolom van de tekst.

Op de derde regel van de brontekst wordt een functie *main()* aangemaakt (gedefinieerd). De naam *main* is in **C** een gereserveerde term: elk programma moet één functie met deze naam hebben. Bij die functie begint de uitvoering van het programma; *main()* hoeft echter geenszins de eerste functie in een **C** brontekst te zijn.

Vóór *main* staat het keyword *void*. Deze term specificeert dat de functie *main()* aan de aanroeper niets teruggeeft, oftewel geen return-waarde kent (return waarden komen later aan de orde). Als laatste staan op regel 3 twee haakjes. Deze haakjes betekenen dat *main* een functie is, hoewel niet voorzien van argumenten (argumenten van functies worden later behandeld).

Regels 4 en 6 bevatten een open-accolade en een sluit-accolade. Deze tekens markeren in **C** het begin resp. einde van een blok. Een blok is een eenheid van code; tussen de accolades 'moet iets gedaan worden'.

De code van *main()* tenslotte staat op regel 5. Een functie *printf()* wordt geactiveerd. Aan de haakjes achter *printf* is te zien dat het om een functie gaat. Bij de aanroep krijgt *printf()* een argument mee. Het argument is in dit geval een string *Hello World!*, afgesloten door *\n*. Strings

17. Overigens kunnen alle preprocessor directives herkend worden aan het feit dat zij met een '#'-teken beginnen.

worden in **C** aangeduid door dubbele aanhalingstekens. In deze string is het laatste deel, de *n*, een speciaal symbool (een escape sequence): *printf()* interpreteert dit deel van de string als een signaal om een nieuwe regel te beginnen.

In overeenstemming met de layout-regels staan de accolades op de eerste kolom van de tekst, elk op een aparte regel. De inspringing van de accolades is even diep als die van het keyword *void* erboven. De code tussen de accolades is over één tabulatiestop extra geïndenteerd.

Opgave 2.
Voer de brontekst van listing 1 in als bestand *hello.c*. Vertaal de brontekst met de opdracht *c hello.c*.
Link de ontstane objectmodule vervolgens met de opdracht *l hello*.
Welke bestanden zijn bij de vertaling ontstaan? Roep het programma aan door opdracht *hello*.

Opgave 3.
Wijzig de string *Hello World* in *Hello\nWorld*. Vertaal en link het programma opnieuw. Welke melding verschijnt nu op het scherm als het programma aangeroepen wordt?

Opgave 4.
Noteer de omvang van het bestand *hello.exe* en vertaal het programma nu m.b.v. de *cc* opdracht. Link vervolgens het programma m.b.v. de *lc* opdracht. Wat is nu de omvang van *hello.exe*? Verklaar het verschil. Start het programma. Werkt het nog net zoals de met de *c* en *l* opdrachten aangemaakte *hello.exe*?

Opgave 5.
Geef nu de opdracht *cv hello*.
Deze opdracht activeert de symbolische debugger *codeview*. Het programma is nu met behulp van de F8 toets regel voor regel te traceren. Door op F4 te drukken kan het 'gewone' scherm bekeken worden. Verlaat *codeview* door de 'q' toets in te drukken als de melding 'program terminated normally' verschijnt.

1.7 Het 'Repeat' programma

Het volgende programma *repeat.exe* is een eenvoudig **C** programma, dat de input/output faciliteiten van **C** demonstreert. De brontekst van dit programma is in listing 2 gegeven.

Let bij de listing op de aangehouden layout. Wederom wordt de code tussen twee accolades over één extra tabulatiestop ingesprongen Bovendien is de aanroep van functie *putchar()* op regel 9 één tabulatiestop verder ingesprongen dan de *while* erboven. Dit is gedaan om aan te duiden dat de *putchar()* aanroep 'conditioneel' is aan de *while*-lus erboven (de *while*-lus constructie wordt in hoofdstuk 4 behandeld).

```
1     #include <stdio.h>
2
3     void main ()
4     {
5         int
6             ch;
7
8         while ( (ch = getchar ()) != EOF )
9             putchar (ch);
10
11        printf ("\nEnd of file.\n");
12    }
```

Listing 2: Het repeat programma.

Opgave 6.

Voer de brontekst van listing 2 in. Noem de brontekst *repeat.c*. Vertaal en link het programma. Beantwoord de volgende vragen:
1. Waarvoor is de opdracht *#include <stdio.h>* nodig?
2. Wat betekent de regel *void main()*?
3. Wat doen functies *getchar()* en *putchar()* (zie de informatie over standaardfuncties, meegeleverd bij de vertaler)?
4. Wat doet regel *printf ("\nEnd of file.\n")*? Waarvoor staat de *\n*?
5. Wat doet het programma bij een aanroep als *repeat < repeat.c*?

DEEL I

DE PROGRAMMEERTAAL C

Hoofdstuk 2

Variabelen

2.1 Naamgeving van identifiers

Zoals de meeste programmeertalen kent ook de programmeertaal **C** namen die in de brontekst van programma's worden gebruikt om variabelen, functies, datatypes of constanten aan te duiden. Dergelijke namen zijn symbolen die in plaats van de waarden van constanten, of in plaats van geheugenlocaties van variabelen of functies, worden gebruikt en die voor mensen veel begrijpelijker zijn dan getallen. Deze symbolische namen worden identifiers genoemd. Identifiers mogen door de programmeur zelf worden gekozen. Bij voorkeur worden namen gebruikt die suggestief zijn voor het doel dat zij vervullen. Wanneer een variabele wordt gebruikt waarmee het aantal spaties wordt geteld, kan die variabele bij voorbeeld de symbolische naam *n_spaces* krijgen.

Terwijl constanten symbolisch mogen worden aangeduid *moet* elke variabele en elke functie een symbolische naam hebben. De regels waaraan namen moeten voldoen om door de vertaler te worden geaccepteerd zijn eenvoudig:

- Een naam moet met een letter of met de underscore (_) beginnen. Sommige **C** vertalers staan de underscore als eerste character van een naam niet toe, maar ANSI-vertalers (waaronder de Microsoft vertaler) wel[1].
- De overige tekens van een naam mogen uit letters, cijfers of underscores bestaan.
- Hoofd- en kleine letters in identifiers worden van elkaar onderscheiden. De programmeertaal **C** is case-sensitive.
- De naam mag een onbepaalde lengte hebben. Echter, veel **C** vertalers zullen alleen de eerste zes tekens van een naam interpreteren en zullen de overige negeren. De Microsoft vertaler en linker kennen echter geen praktische beperking (de eerste 127 tekens van namen worden geïnterpreteerd).
- De naam mag niet *gereserveerd* zijn. De keywords van de taal (zoals *while, if, case*) zijn gereserveerde namen en mogen niet als identifier worden gebruikt (zie bijlage A.2 voor de keywords van **C**).

1. Interne functies en variabelen die door Microsoft zijn gedefinieerd of specifieke functies voor de IBM PC familie beginnen vaak met een underscore (zoals de functie *_bios_keybrd()*). Aangeraden wordt daarom om zelf geen identifiers met een underscore te laten beginnen.

Opgave 7.

Welke van de volgende namen kunnen als identifier worden gebruikt? Beredeneer waarom de overige symbolen geen identifier zijn.

1. *_while*
2. *_18*
3. *18_*
4. *counter*
5. *_bios_keybrd*
6. *is_dit_nog_wel_een_identifier_of_is_dit_geen_identifier_?*

2.2 Types in C

2.2.1 Standaard data types

Hieronder worden de vijf standaard datatypes en hun waardebereik (de minimale en maximale waarde van dat type) genoemd, zoals die gelden voor een computer die is gebaseerd op een INTEL 80x86 processor. Bij andere computers kunnen andere bereiken gelden.

char: Dit datatype is een 8-bits waarde (een byte genoemd), doorgaans gebruikt voor opslag van ASCII waarden (dus lettertekens). Tenzij anders gespecificeerd, is het bereik van 0 tot en met 255. Een *char* variabele heeft dus gewoonlijk geen negatieve waarde.
Merk op dat de Microsoft **C** vertaler bij verstek een *char* type hanteert waarbij waarden vanaf -128 tot en met 127 kunnen worden aangenomen. Het Microsoft *char* datatype kent dus wél negatieve waarden. Dat is echter eenvoudig te wijzigen in een *char* datatype met alleen positieve waarden door gebruik te maken van de `/J` vertaleroptie (zie ook de listing van bestand `c.bat` op pagina 32).

int: Dit datatype staat voor integer. Het is een 16-bits waarde (ook wel 'woord' (*word*) genoemd), voor de opslag van gehele getallen. Het bereik is van -32768 tot en met 32767.

float: Het *float* data type is een 32-bits waarde voor opslag van floating point waarden. Het bereik van het type *float* loopt van ongeveer $3,4 * 10^{-34}$ tot $3,4 * 10^{34}$, de precisie is ongeveer 6 cijfers. Het *float* datatype wordt gebruikt om reële getalswaarden te representeren.

double: Het *double* datatype beslaat twee maal zoveel ruimte als de *float*, dus 64 bits. Het bereik van het type *double* loopt van ongeveer $1,7 * 10^{-308}$ tot $1,7 * 10^{308}$, de precisie is ongeveer 12 cijfers.

void: Met het *void* datatype wordt eigenlijk *geen* datatype gespecificeerd. Dit keyword wordt vooral gebruikt om aan te geven dat een functie niets retourneert[2].

2. Een *void* functie is vergelijkbaar met de *procedure* bij **pascal**.

2.2.2 Type and access modifiers

Elke typenaam behalve *void* kan voorafgegaan worden door een type modifier. Daarnaast bestaan er access modifiers, die veel minder vaak worden gebruikt dan de type modifiers. Er zijn vier type modifiers, en twee access modifiers.
De type modifiers zijn:

signed: Dit keyword forceert bij op *char* en *int* gebaseerde variabelen de mogelijkheid om een negatieve waarde te kunnen geven. Zo kan een 8-bits *signed char* waarden van -128 tot en met 127 aannemen. Door *signed* te gebruiken, wordt het bereik dus niet groter, maar anders verdeeld dan het bereik van het *unsigned* type. Anders gezegd: het gebruik van *signed* bewerkstelligt een nulpuntsverschuiving in het waardenbereik, maar laat de breedte van het bereik onaangetast.

unsigned: Dit keyword forceert bij op *char* en *int* gebaseerde variabelen het bereik naar alleen niet-negatieve waarden. Zo heeft een 16-bits *unsigned int* een bereik van 0 tot en met 65535.

long: Voor *int* variabelen die met de type modifier *long* zijn gedefinieerd wordt twee keer zo veel geheugen gereserveerd dan voor *int* variabelen die zonder de *long* type modifier zijn gedefinieerd. Daarmee gepaard gaat een groter waardenbereik. Een *long int* wordt opgeslagen in 32-bits. Het bereik van *long ints* loopt daarmee van de waarde $-2.147.483.649$ tot en met de waarde $2.147.483.648$. Constanten voor het type *long int* zijn *int* constanten waaraan een *L* wordt toegevoegd (de *L* wordt een *suffix* genoemd). Voorbeelden zijn *12L* en *-34L*. Een alternatief voor de *L*-suffix is de aanduiding *(long)* vóór het getal (zie hoofdstuk 3): *12L* is hetzelfde als *(long)12*.

short: Dit is het tegenovergestelde van *long*: het aantal bits wordt gehalveerd. Bij de Microsoft **C** vertaler wordt de aanduiding *short* echter genegeerd, of leidt tot een foutmelding bij de vertaling.

Een typenaam mag door meerdere modifiers worden voorafgegaan. Zo is een variabele van het type *unsigned long int* een 32-bits variabele die geen negatieve waarden kan aannemen. Het bereik loopt dan van 0 tot en met 4.294.967.296. Het is niet toegestaan twee modifiers die het tegengestelde specificeren (*long short* of *signed unsigned*) bij één variabeledefinitie op te geven.

De access modifiers worden hier voor de volledigheid ook vermeld, hoewel deze gewoonlijk niet worden gebruikt. Door middel van een access modifier kan de programmeur de vertaler aangeven dat een variabele in het verloop van een programma al dan niet een andere waarde *mag* krijgen. Er zijn twee access modifiers: *const* en *volatile*.

Wanneer geen access modifier wordt opgegeven, wordt *volatile* door de vertaler verondersteld. Variabelen die met de access modifier *volatile* worden gedefinieerd mogen een andere waarde krijgen in het verloop van

het programma. Een variabele die als *const* gedefinieerd is, mag niet gewijzigd worden. Zo wordt met:

```
const int
    total = 100;
```

een variabele *total* gedefinieerd met een initiële waarde 100. De waarde in die variabele mag niet gewijzigd worden.

2.2.3 Het default data type

C beschouwt het type *int* als het default datatype. Dat betekent dat het keyword *int* in definities kan worden weggelaten, in welk geval de vertaler aanneemt dat het type *int* wordt bedoeld. Het type *int* zou dus kunnen worden weggelaten wanneer wèl een *int* variabele moet worden gedefinieerd. Dit impliciete gebruik van het type *int* wordt echter sterk afgeraden, omdat de duidelijkheid van de brontekst waarin dat gebeurt vermindert. Het kan natuurlijk ook per ongeluk gebeuren: wanneer in een variabele definitie een puntkomma wordt getypt waar een komma bedoeld was, gaat de vertaler over op het default type *int,* hetgeen tot verwarrende fouten kan leiden. Een voorbeeld van zo'n situatie geven we op pagina 44.

In combinatie met type- of access modifiers kan het default datatype echter goed worden weggelaten. Als typenaam wordt dan alleen een modifier, zoals *long* of *unsigned* gebruikt, terwijl dan *long int* respectievelijk *unsigned int* wordt bedoeld. In de praktijk worden op deze wijze *short, long* en *unsigned* als typenaam gebruikt zonder de vermelding *int* erachter.

2.3 Definiëren van variabelen

Variabelen moeten, voordat ze worden gebruikt, bekend zijn aan de vertaler. Daartoe worden variabelen gedefinieerd of gedeclareerd. Hier wordt de variabele*definitie* behandeld. Het *declareren* wordt behandeld in 2.3.4.

Bij de definitie van variabelen vertellen we de vertaler drie belangrijke zaken over de variabele.

- Bij de definitie wordt de *naam* van de variabele gegeven zoals die verderop door de programmeur in het programma zal worden gebruikt.
- Bij de definitie wordt het *type* van de variabele vastgelegd, waardoor de vertaler weet *welke* waarden in de variabele kunnen worden opgeslagen en *hoe* die waarden worden opgeslagen.
- Bij de definitie wordt de vertaler verteld dat in het *geheugen* ruimte moet worden gereserveerd voor de opslag van de waarden van de variabele.

Merk op dat het laatste punt de definitie van de declaratie onderscheid: alleen bij de definitie wordt geheugen gereserveerd voor de variabele, terwijl bij de declaratie alleen een naam en een type worden vastgelegd. Meer hierover in 2.3.4.

Variabelen worden gedefinieerd door de typenaam te vermelden, gevolgd door de naam (identifier) van de variabele. Het geheel wordt afgesloten door een puntkomma ;. In het volgende voorbeeld wordt een variabele *length* van het type *double* gedefinieerd[3]:

```
double
    length;
```

De typenaam mag voorafgegaan worden door type modifiers of access modifiers. In één definitie mogen meerdere variabelen aangemaakt worden. De variabelenamen zijn dan door komma's gescheiden, zoals in het volgende voorbeeld wordt geïllustreerd:

```
double
    lenght,
    total;
```

Opgave 8.

1. **Definieer twee variabelen *n_spaces* en *n_letters* die geen negatieve waarden krijgen, ten hoogste de waarde 200 zullen bevatten en waarvoor zo weinig mogelijk geheugen wordt gereserveerd.**
2. **Definieer twee variabelen *n_spaces* en *n_letters* die geen negatieve waarden krijgen, ten hoogste de waarde 40.000 zullen bevatten en waarvoor zo weinig mogelijk geheugen wordt gereserveerd.**
3. **Definieer twee variabelen *n_spaces* en *n_letters* die geen negatieve waarden krijgen, ten hoogste de waarde 4.000.000 zullen bevatten en waarvoor zo weinig mogelijk geheugen wordt gereserveerd.**

Tenslotte nog een voorbeeld van een definite van variabelen die tot verwarring kan leiden. In het volgende voorbeeld was het de bedoeling van de programmeur om vijf *float* variabelen te definiëren, maar door een typefout werd een puntkomma in plaats van een komma gebruikt na *float_3*. De vertaler interpreteert de puntkomma als het einde van een variabeledefinitie (dus van *floats)* en gaat weer over op het *int* datatype. De variabelen *float_4* en *float_5* zijn daardoor van het type *int* in plaats van het type *float:*

```
float
    float_1,
```

3. Let bij de voorbeelden van definities van variabelen op de typografische suggestie om achter de typenaam een nieuwe regel te beginnen en variabelenamen te indenteren ten opzichte van de typenaam.

```
        float_2,
        float_3;
        float_4,
        float_5;
```

2.3.1 Lokale en globale variabelen

De plaats waar een variabele is gedefinieerd legt de levensduur en zichtbaarheid van die variabele vast.

Wanneer een variabele wordt gedefinieerd, wordt er weliswaar geheugen voor die variabele gereserveerd, maar dat geheugen hoeft niet gedurende de volledige verwerkingstijd van dat programma voor die variabele beschikbaar te zijn. De levensduur van zo'n variabele kan daarmee korter of langer zijn.

Daarnaast kunnen variabelen wel of niet zichtbaar zijn voor onderdelen van het programma. Een variabele kan voor het hele programma, voor delen van het programma, of een zeer beperkt deel van het programma zichtbaar zijn.

Variabelen kunnen binnen een functie direct na een open-accolade worden gedefinieerd. Dergelijke variabelen heten *lokaal*. Lokale variabelen bestaan zolang als de functie actief is (dus: zolang de functie wordt uitgevoerd). Lokale variabelen zijn zichtbaar tussen de open-accolade waarachter ze zijn gedefinieerd tot aan de bijbehorende sluit-accolade. Aangezien de code tussen een open-accolade en de bijbehorende sluit-accolade een blok wordt genoemd, zijn lokale variabelen zichtbaar in het blok waarin ze zijn gedefinieerd. Wanneer een blok genest is binnen een blok waarin lokale variabelen zijn aangemaakt, dan zijn de lokale variabelen ook zichtbaar in dat geneste blok. De zichtbaarheid wordt dus niet beperkt *binnen* het blok. *Buiten* het blok is de variabele onzichtbaar. Als voorbeeld geven we de volgende code, waarin een functie wordt gedefinieerd die *func()* heet. In de functie worden twee variabelen op verschillende plaatsen gedefinieerd:

```
func()
{
    int
        x;

    // eventuele 'statements'
    {
        int
            y;

        // eventuele 'statements'
        y = 4;
    }
    y = 6;                          // fout !
}
```

De variabele *x* kan overal in de functie *func()* worden gebruikt. De variabele *y* kan alleen in het binnenste blok van de functie worden gebruikt.

Wanneer de functie *func()* door de vertaler wordt bewerkt, zal het statement *y = 6;*[4] tot een fout bij de vertaling leiden: *y* bestaat niet meer buiten het binnenste blok van de functie *func()*.

Stel nu dat in het buitenste blok van de functie ook een variabele *y* was gedefinieerd, zodat op twee plaatsen sprake is van de definitie van een variabele *y*. In die situatie geldt de dichtstbijzijnde definitie: het statement *y = 4;* heeft dan betrekking op de variabele *y* in het binnenste blok, het statement *y = 6;* heeft dan betrekking op de variabele *y* die in het buitenste blok is gedefinieerd. Om de dichtstbijzijnde definitie te vinden, zoeken we naar een definitie waarbij we in de brontekst in de richting van het begin van de brontekst op zoek gaan naar een definitie van de variabele.

Variabelen kunnen ook buiten een functie worden gedefinieerd. Dergelijke variabelen heten *globale* variabelen. Een globale variabele is alle code volgend op de definitie bereikbaar. In het volgende voorbeeld is *total* een globale *int,* die bereikbaar is voor code in de functies *func1()* en *func2():*

```
int
    total;

func1 ()
{
    total = 10;
}

func2 ()
{
    total = 20;
}
```

Doordat globale variabelen gedurende het gehele programma blijven bestaan, kunnen we ook iets zeggen over hun waarde bij binnenkomst en bij het verlaten van een functie. Wanneer in het bovenstaande voorbeeld functie *func2()* onmiddellijk na *func1()* wordt uitgevoerd heeft *total* bij binnenkomst in functie *func2()* de waarde 10 en bij het verlaten van *func2()* de waarde 20. In tegenstelling tot globale variabelen, die hun waarde behouden, *verliezen* lokale variabelen hun waarde wanneer de functie waarin ze zijn gedefinieerd niet meer wordt geëxecuteerd.

Merk op dat ook bij globale variabelen de regel van de dichtstbijzijnde definitie geldt: wanneer in functie *func2()* een lokale variabele *total* zou zijn gedefinieerd, dan had het statement *total = 20;* betrekking op de *lokale* variabele *total,* en niet op de *globale* variabele *total*.

Enigszins vooruitlopend op de beschrijving van de 'storage class specifiers' kan hier al worden opgemerkt dat de definitie van niet-statische lokale variabelen eventueel het keyword *auto* kan bevatten. Het keyword *auto* geeft aan dat de variabele automatisch, bij elke activatie van de

4. Het statement *y = 6;* is een waardetoekenning: aan de variabele *y* wordt de waarde zes toegekend (zie 3).

functie waarbinnen de variabele is gedefinieerd, wordt aangemaakt. In de praktijk wordt *auto* niet gebruikt.

Waar bevinden zich de variabelen

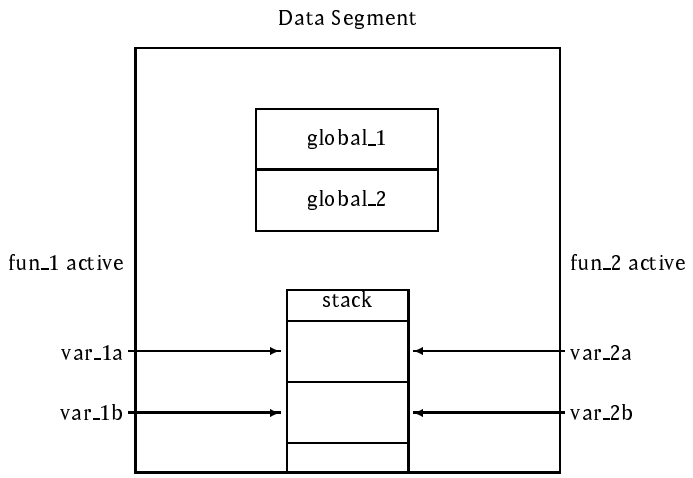
Hoe komt het dat lokale variabelen zich zo anders gedragen dan globale variabelen? We hebben gezien dat globale variabelen door het hele programma heen beschikbaar zijn en dat lokale variabelen slechts binnen een functie (of zelfs slechts binnen een blok) bestaan. Daarnaast hebben we gezien dat globale variabelen over verschillende functies hun waarde behouden, terwijl lokale variabelen elke keer dat een functie opnieuw wordt aangeroepen een andere waarde kunnen hebben.

Bij de vertaling van C bronteksten onderscheidt de vertaler onder andere de *code* die wordt uitgevoerd en de *data* die door het programma worden gewijzigd. Niet alleen worden code en data onderscheiden, maar ook worden code en data van elkaar *gescheiden* in een *codesegment* en een *datasegment*. Dergelijke segmenten zijn, wanneer het programma eenmaal klaar is en wordt uitgevoerd, gescheiden delen van het werkgeheugen van de computer.

Voor de definitie van variabelen is het datasegment van belang. In het datasegment wordt voor alle globale variabelen een plaats gereserveerd. Gesproken wordt van het *adres* van variabelen.

In het datasegment bevindt zich echter nog een andere belangrijke datastructuur: de *stapel* of 'stack'[5]. Met behulp van de stack wordt onder andere bijgehouden wat er moet gebeuren wanneer een functie is afgelopen. Ook wordt de stack gebruikt om alle lokale variabelen van een functie een tijdelijke plaats te geven. Wanneer een functie wordt gestart, dan is bekend *hoeveel* geheugenruimte de lokale variabelen van die functie in beslag zullen nemen en die geheugenruimte wordt op de stack beschikbaar gemaakt (*gealloceerd*). Zolang de functie actief is blijft de gealloceerde ruimte op de stack beschikbaar. Zodra de uitvoering van de functie is voltooid wordt de gealloceerde ruimte op de stack weer vrijgegeven. De volgende functie die wordt gestart heeft wellicht ook weer lokale variabelen en zal de ruimte op de stack toegewezen krijgen die daarvoor aan een eerdere functie was toegekend. Dezelfde plaats op de stack kan dus herhaaldelijk voor verschillende doeleinden worden gebruikt. Vergelijk dat met globale variabelen: het geheugen dat voor globale variabelen is gereserveerd, wordt voor eens en altijd gereserveerd voor die globale variabelen en die ruimte in het geheugen wordt nooit gedeeld met andere variabelen, waardoor globale variabelen hun waarde behouden wanneer de uitvoering van het programma van de ene functie overgaat naar de andere functie.

5. Vergelijk de stack met de *bordenbak* in zelfbedieningsrestaurants: het bord dat het laatst op de stapel wordt gezet, drukt de overige borden naar beneden en is bovendien zelf het eerste bord dat weer van de stapel wordt verwijderd.



Figuur 2.1: Globale en lokale variabelen in het datasegment.

Figuur 2.1 illustreert het één en ander nog eens. De globale variabelen *global_1* en *global_2* zijn opgenomen in het datasegment, de lokale variabelen *var_1a* en *var_1b* zijn op de stack aanwezig wanneer *fun_1()* wordt uitgevoerd, terwijl de lokale variabelen *var_2a* en *var_2b* op dezelfde plaats op de stack aanwezig zijn wanneer *fun_2()* wordt uitgevoerd.

2.3.2 Static variabelen

De storage class specifier *static* geeft aan dat een variabele een status heeft die tussen globaal en lokaal in zit. *Static* variabelen zijn slechts zichtbaar in de brontekst of in de functie waarin ze zijn gedefinieerd en nooit daarbuiten. Daarnaast houden ze hun laatst gekregen waarde, ook al wordt de functie, waarin een *static* variabele is gedefinieerd, beëindigd. Het laatste aspect maakt dat een *static* variabele doet denken aan een globale variabele, het eerste aspect (de beperkte zichtbaarheid) maakt dat een *static* variabele doet denken aan een lokale variabele.

Als een *static* variabele in een functie is gedefinieerd, dan wordt de variabele *niet* vernietigd als de functie afloopt. Dit wordt geïllustreerd door de volgende code:

```
func1 ()
{
    int
        a;

    a = a + 1;
}
```

```
func2 ()
{
    static int
        a;

    a = a + 1;
}
```

In functie *func1()* is *a* een niet-statische variabele. Als *func1()* tweemaal achtereen wordt aangeroepen, dan wordt variabele *a* elke keer opnieuw aangemaakt en heeft *a* bij elke nieuwe aanroep opnieuw een ongedefinieerde waarde, waarbij de waarde 1 wordt opgeteld. Dan wordt functie *func1()* beëindigd en bestaat de variabele *a* niet meer.

In functie *func2()* daarentegen wordt *a* slechts één keer aangemaakt en (zoals toegelicht in 2.3.5) slechts één keer geïnitialiseerd op de waarde 0. De eerste aanroep van *func2()* resulteert in waarde 1 in variabele *a*, de tweede aanroep resulteert in waarde 2, enzovoorts. De variabele *a* in functie *func2()* blijft dus bestaan, ook al is de functie zelf niet actief.

Wanneer een variabele *buiten* functiecode als *static* wordt gedefinieerd, dan heeft dit een speciale betekenis: de variabele is dan alleen globaal voor *deze* brontekst. In **C** kunnen programma's over meerdere bronteksten verdeeld zijn. Door gebruik te maken van *static* globale variabelen kan het bestaan van een variabele eenvoudig verborgen blijven voor functies die niets met die variabele te maken hebben.

2.3.3 Register variabelen

Een tweede storage class specifier is *register*. Register variabelen kunnen alleen binnen functiecode gebruikt worden (het zijn dus lokale variabelen). Als de **C** vertaler voor een INTEL 80x86 processor een registervariabele aanmaakt, dan wordt getracht die variabele aan één van de registers van de 80x86 processor toe te kennen in plaats van aan een locatie in het werkgeheugen van de computer. Dit resulteert in enorm snelle berekeningen[6]. Nu zou het een misverstand zijn om dan maar aan alle lokale variabelen de storage class *register* toe te kennen: in de Microsoft implementatie zijn er twee 16-bits registers beschikbaar voor registervariabelen. Daarom moeten *register* variabelen van een 16-bits datatype zijn.

Als registervariabelen kunnen in feite alleen *int*s (evt. *unsigned*) gebruikt worden[7]. Bij alle andere types heeft de *register* storage class geen effect, en leidt het gebruik ervan ook niet tot een waarschuwing of fout bij de vertaling. Dus, in het volgende voorbeeld wordt *length* een gewone lokale (niet-register) variabele:

6. Voor de snelheidsmaniakken: een *increment* operatie van een registervariabele duurt drie klokcycli, dezelfde operatie van een geheugenvariabele duurt minstens 15 cycli.
7. In small en medium memory models kunnen ook pointers als registervariabelen gebruikt worden. Memory models worden behandeld in 12, pointers in 8.

```
func ()
{
    register int
        total;
    register double
        length;
}
```

2.3.4 Declaratie van variabelen: externe variabelen

De derde storage class specifier is *extern*. Een externe variabele is (nog) niet in de huidige brontekst gedefinieerd. Deze situatie doet zich bij programma's die slechts uit één brontekst bestaan zelden voor. Echter, veel **C** programma's bestaan uit meerdere bronteksten en in die situatie komt het juist vaak voor dat een variabele die in de ene brontekst is gedefinieerd (ook) in een andere brontekst moet worden gebruikt. In dergelijke gevallen wordt de vertaler op de hoogte gesteld van het bestaan van een variabele door middel van de *extern* storage class. Zo is in het volgende voorbeeld *offset* een *long int* variabele, die in een andere brontekst als een globale, niet-statische variabele moet zijn gedefinieerd:

```
extern long int
    offset;
```

De storage class *extern* leidt niet tot het reserveren van geheugen voor de externe variabelen. Het gebruik ervan is louter bedoeld als informatie voor de vertaler dat een bepaalde variabele al bestaat. In dit verband wordt onderscheid gemaakt tussen een *definitie* en een *declaratie* . Gesproken wordt van een variabele*definitie* wanneer daadwerkelijk geheugen wordt gereserveerd voor de variabele. Een definitie kost dus geheugen. Wanneer het bestaan van een variabele alleen maar wordt aangekondigd zoals bij de *extern* storage class wordt gesproken van een variabele*declaratie* . Declaraties kosten geen geheugen en zijn geïmpliceerd door het gebruik van het keyword *extern*.

2.3.5 Initialisatie bij definitie

Variabelen kunnen bij hun definitie een initiële waarde krijgen. Voor de initialisatie van variabelen (voorbeelden van initialisaties zijn gegeven in listing 3) gelden de volgende regels:

- Globale of statische (*static)* variabelen worden geïnitialiseerd op de waarde 0, tenzij de initialisatie expliciet door de programmeur wordt aangegeven.
- Globale of statische (*static)* variabelen hebben hun initiële waarde zodra het programma is gestart, maar houden hun initiële waarde slechts tot de eerste waardetoekenning.

- Initialisaties van globale of statische variabelen moeten door de vertaler volledig kunnen worden berekend. Dat betekent dat de initiële waarde weliswaar uit een expressie mag bestaan, maar dat alle onderdelen van die expressies al bij de vertaler bekend moeten zijn. De vertaler moet de initialisatie-expressie tot een getal kunnen evalueren. Dit heet compile-time evaluation van expressies. In tegenstelling tot compile-time evaluation staat run-time evaluation. Run-time evaluation heeft betrekking op de evaluatie van expressies door het programma zelf.
  Expressies die uit constanten en **C** operatoren bestaan kunnen compile-time worden geëvalueerd. Waarden van variabelen zijn geen constanten en kunnen dus niet worden gebruikt bij initialisatie-expressies van globale of statische variabelen.
- Lokale- en registervariabelen hebben een ongedefinieerde initiële waarde tenzij de initialisatie expliciet is. Wanneer de initialisatie expliciet is, wordt de initiële waarde elke keer opnieuw toegekend wanneer de functie wordt gestart waarin de lokale- of registervariabele is gedefinieerd. In tegenstelling tot de initialisatie van globale of statische variabelen die compile-time moeten kunnen worden uitgevoerd, worden initialisaties van lokale- en registervariabelen niet compile-time, maar run-time uitgevoerd. Daardoor is de initialisatie van een lokale- of registervariabele eigenlijk een statement dat is opgenomen in een variabeledefinitie.

```
1           int
2               index = 5 * 33;
3           char
4               first_ch = 'a',
5               beyond_alphabet = 'z' + 1;
6
7           func()
8           {
9               int
10                  local_1 = index,
11                  index = 0;
12              static int
13                  times_called = 0;
14          }
```

Listing 3: Initialisatie van variabelen.

In listing 3 worden de volgende initialisaties gedefinieerd:

- De variabele *index* op regel 2 heeft bij de start van het programma de waarde 165.
- De variabele *first_ch* heeft bij de start van het programma de ASCII-waarde van de letter `'a'`.

- De variabele *beyond_alphabet* heeft bij de start van het programma de ASCII-waarde van het teken volgend op de letter 'z' (dat is het teken '{').
- De variabele *local_1* krijgt elke keer dat de functie *func* wordt gestart de waarde die de variabele *index* die op regel 2 is gedefinieerd dan heeft.
- De variabele *index* op regel 11 krijgt elke keer dat de functie *func* wordt gestart de waarde 0.
- De variabele *times_called* heeft bij de start van het programma de waarde 0.

Opgave 9.

Initialiseer bij definitie de volgende variabelen of geef aan waarom de initialisatie niet mogelijk is (alle variabelen zijn *ints*).

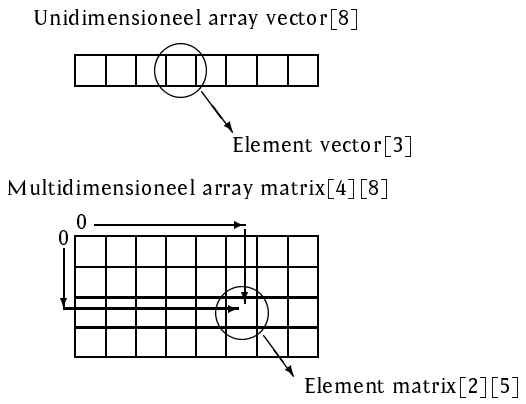
1. een globale variabele *index:* initialiseer op $3 * 331$.
2. een statische globale variabele *statvar:* initialiseer op 0.
3. een statische globale variabele *statglob:* initialiseer op de waarde van de globale variabele *index*.
4. een lokale variabele *locin:* initialiseer op $3 * 331$.
5. een statische lokale variabele *statloc:* initialiseer op 0.
6. een lokale registervariabele *locreg:* initialiseer op de waarde van de globale variabele *index*.

2.4 Arrays en strings

Als eerste introductie op complexe datastructuren worden hier strings en arrays geïntroduceerd. Arrays bestaan uit een serie elementen, die alle van hetzelfde type zijn, bij voorbeeld *ints* of *chars*. Bij elk element hoort een nummer, dat de *index* wordt genoemd. Het *eerste* element van een array heeft in **C** indexwaarde 0, het volgende element indexwaarde 1, enzovoort. Er zijn dus geen negatieve indices, en de telling begint ook niet bij 1 (wat wordt gesuggereerd door het *eerste* element) maar bij 0.

Arrays kunnen ook uit arrays bestaan, of uit arrays van arrays van arrays. Er wordt dan gesproken van meerdimensionale arrays. Terwijl een enkelvoudig array kan worden voorgesteld als een rij (of *vector)* van elementen, kan een tweedimensionaal array worden voorgesteld als een *vlak* (zoals een schaakbord) en kan een driedimensionaal array worden voorgesteld als een *balk*. Langs elke *dimensie* is een *schaalverdeling* waarop de index waarden aanneemt. Door een indexwaarde voor elke dimensie te specificeren, wordt een specifiek element van het array bereikt, zoals aangegeven in figuur 2.2.

Een veel voorkomend ééndimensionaal array is dat van *chars*. De elementen van dit array zijn dus *chars*. Wanneer het laatste *char* ASCII teken 0 (*ascii-z*) is, is er sprake van een string.



Figuur 2.2: Arrays en hun elementen.

In **C** worden stukken tekst praktisch altijd opgeslagen als strings, en zijn *per definitie* afgesloten door een ascii-z teken. De ascii-z afsluiting van strings is een *conventie* van **C**. Veel functies die met strings omgaan (zoals de *printf()* functie) doen hun werk zolang als ze geen ascii-z tegenkomen en gaan dus uit van de veronderstelling dat de ascii-z inderdaad aan het einde van de te verwerken string staat.

Een array wordt in **C** gedefinieerd als typenaam, gevolgd door de variabelenaam, gevolgd door de omvang van het array in rechte haken. Zo is in onderstaand voorbeeld een array *row* gedefinieerd dat uit 100 *ints* bestaat, een tweedimensionaal array *board* dat uit 8 rijen en 10 kolommen van *ints* bestaat en ruimte voor een string *line* van 80 *chars*. Merk op dat in **C** het laatste teken in een string een ascii-z moet zijn, zodat in *line* hoogstens een string van 79 tekst-tekens zal kunnen staan, omdat aan het einde van de tekst immers nog de ascii-z moet volgen. De *lengte* van een string is altijd ten hoogste het aantal tekens dat in het *char* array kan worden opgeslagen minus 1 (voor de ascii-z):

```
int
    row [100],
    board[8][10];
char
    line[80];
```

Let bij het array *board* op de manier waarop de dubbele index wordt aangegeven. Elke index krijgt een *eigen* stel rechte haken, de laatste index heeft betrekking op de unidimensionale vector van elementen. Bij het array *board* zijn er dus 8 vectoren van elk 10 *ints*.

2.4.1 Initialisatie bij definitie

Net zoals variabelen kunnen ook arrays bij de definitie worden geïnitialiseerd. Een uitzondering op deze regel wordt gevormd door arrays die lokaal zijn gedefinieerd. Globale, statische, en statisch-lokale arrays kunnen wel worden geïnitialiseerd. Zodra het programma wordt gestart, zijn de geïnitialiseerde array-elementen voorzien van de aangeduide waarden, alle overige (niet-geïnitialiseerde) elementen zijn nul.

Een aardigheidje van de vertaler is dat het aantal array-elementen niet hoeft te worden gespecificeerd, wanneer dat door de initialisatie al blijkt. Aan de andere kant mogen minder elementen worden geïnitialiseerd dan er zijn gedefinieerd. In dat geval worden net zoveel array elementen geïnitialiseerd als er initialisatie-expressies zijn. De resterende elementen zijn dan 0. Bij de initialisatie van een array kan de initialisatie van een (sub)array door accolades worden omgeven. Wanneer die procedure wordt gevolgd, kunnen bij voorbeeld delen van meerdimensionale arrays worden geïnitialiseerd. Het is ook mogelijk om de elementen van een array te initialiseren met behulp van een enkele lijst van initialisatie-expressies. In dat geval worden de elementen geïnitialiseerd in de volgorde waarin ze in het geheugen zijn opgeslagen: eerst alle elementen van de eerste rij, dan de elementen van de tweede rij, enzovoort, totdat alle rijen van een tweedimensionaal array zijn geïnitialiseerd. Bij een driedimensionaal array worden eerst de elementen van het eerste tweedimensionale array geïnitialiseerd, dan de elementen van het tweede tweedimensionale array, enzovoort. Bij arrays van een hogere dimensie verloopt de initialisatie analoog.

```
1     int
2         een[3]  = {1, 2, 3},
3         twee[3] = {1},
4         drie[2][3] =
5         {
6             {1, 2, 3},
7             {4, 5, 6},
8         },
9         vier[2][3] = {1, 2, 3, 4, 5, 6},
10        vijf[][3] =
11        {
12            {1, 2, 3},
13            {4, 5, 6},
14        },
15        zes[][3] =
16        {
17            {1, 2, 3},
18            0
19        },
20        zeven[][3] =
21        {
22            {1, 2, 3},
23        },
24        acht[2][3] =
```

```
25        {
26            {1},
27            {2}
28        };
29
30    char
31        negen[]   = "tekst\n",
32        tien[]    = {'t', 'e', 'k', 's', 't', '\n', 0},
33        elf[]     = {116, 101, 107, 115, 111, 10, 0},
34        twaalf[7] = {116, 101, 107, 115, 111, 10};
35
```

Listing 4: Initialisatie van arrays.

In listing 4 zijn een aantal arrays geïnitialiseerd. Hierbij is de layout conventie enigszins flexibel toegepast: de initialisatie van de unidimensionale arrays staat telkens op één regel, inclusief de bijbehorende accolades. Bij de initialisatie van de arrays maken we de volgende opmerkingen:

- Array *een* is een ééndimensionaal array, dat bij de start van het programma waarde 1 in het eerste element, 2 in het tweede element, en 3 in het laatste element heeft staan.
- Array *twee* heeft ook drie elementen, maar die zijn 0, behalve het eerste element, dat 1 is.
- Array *drie* heeft twee rijen van elk drie kolommen. Merk op dat na de initialisatie van elke rij een komma staat. De komma na de laatste rij zou ook mogen vervallen.
- Array *vier* heeft dezelfde omvang en dezelfde initialisatie als array *drie*. Alleen de wijze van initialisatie verschilt.
- Array *vijf* heeft ook dezelfde omvang en initialisatie als array *drie*. De vertaler ontdekt dat er twee rijen zijn op grond van de initialisatie.
- Array *zes* bestaat ook uit twee rijen. De eerste rij is geïnitialiseerd op specifieke waarden, de tweede rij is geïnitialiseerd op 0. De 0 op regel 18 vertelt de vertaler dat er inderdaad twee rijen zijn bedoeld.
- Array *zeven* is een tweedimensionaal array, maar heeft toch maar één rij. De rij-index is dus bij dit array altijd 0.
- Array *acht* is een tweedimensionaal array waarvan alleen het eerste element van elke rij is geïnitialiseerd.
- Array *negen* is een array waarin een string is opgeslagen. Bij deze initialisatie voegt de vertaler zelf een extra ASCII-z teken toe aan de string. Het array heeft dus 7 elementen, alle van het type *char*. Let ook op het laatste teken van de string: $\backslash n$ is een voorbeeld van een escape-sequence , en representeert het newline teken. In appendix A.3 worden de escape-sequences nader toegelicht.
- Array *tien* is identiek geïnitialiseerd als array *negen*. In plaats van een string zijn *character constanten* gebruikt. Let op de expliciete 0

na het laatste teken van de string, dat nu niet automatisch door de vertaler wordt toegevoegd.

- Array *elf* is ook identiek geïnitialiseerd als array *negen,* maar nu zijn de numerieke ASCII-waarden gebruikt in plaats van de ASCII-tekens zelf. Opnieuw moet de ASCII-Z door de programmeur expliciet worden toegevoegd.
- Array *twaalf* is identiek geïnitialiseerd als array *negen.* De initialisatie maakt gebruik van het feit dat niet-geïnitialiseerde elementen automatisch op nul worden geïnitialiseerd. Bij de definitie wordt aangegeven dat er zeven elementen moeten zijn, maar er zijn er slechts zes geïnitialiseerd. Het zevende element is dus 0, en daarmee gelijk aan de ASCII-Z.

Hoofdstuk 3

Waardetoekenning en operatoren

3.1 Waarvoor dienen assignments

Toekenningen, oftewel assignments, zijn de code-elementen waarmee een programmeur aan een variabele een waarde toekent. De algemene vorm van een toekenning is al eerder geïllustreerd en volgt de syntaxis:

variable = *expression* ;

waarbij *variable* de naam van de variabele is, die een waarde gaat krijgen. *Expression* is daarbij de waarde. De meest eenvoudige vorm van een expressie is een constante, zoals in:

```
a = 5;
```

Expressies hoeven echter niet zo eenvoudig te zijn. De taal **C** kent een groot aantal operatoren waarmee complexe expressies gevormd kunnen worden. Bij het uitvoeren van een toekenning wordt de expressie geëvalueerd volgens *voorrangsregels*, die bepalen welke operatoren het eerst geëvalueerd worden. Zo wordt vermenigvuldiging eerder geëvalueerd dan optelling. Deze voorrangsregels worden gewoonlijk aangeduid met de term *prioriteitsregels*; de maal-operator heeft een hogere prioriteit dan de plus-operator.

Naast prioriteit hebben operatoren ook een *associativiteit*. De associativiteit bepaalt hoe een serie van dezelfde operatoren geëvalueerd wordt. De volgorde van evaluatie kan van links naar rechts of van rechts naar links plaatsvinden. De associativiteit van de – operator is bij voorbeeld van links naar rechts; zo wordt 12 – 2 – 3 opgevat als (12 – 2) – 3 en niet als 12 – (2 – 3). De meeste operatoren in **C** zijn links-associatief.

De operatoren van **C** worden globaal in vier groepen verdeeld. De eerste groep, de unaire[1] operatoren, bewerkt één argument (bij voorbeeld de – in –34). De argumenten van operatoren worden gewoonlijk operand genoemd; deze terminologie wordt in de hierop volgende beschijving ook aangehouden. De tweede groep, de binaire operatoren, werkt op twee operanden (bij voorbeeld de + in a + b). De derde groep, de assignment-operatoren, draagt een waarde over. Een voorbeeld hiervan, de = operator, is al eerder genoemd. De laatste groep bestaat uit twee operatoren die in geen van de bovenstaande groepen ingedeeld kunnen worden. Voor alle operatoren geldt dat elke operator een zekere prioriteit en associativiteit heeft.

De operatoren worden in de hierop volgende onderdelen besproken. Alle operatoren zijn nogmaals in bijlage A.4 weergegeven.

3.2 Unaire operatoren

De unaire operatoren van **C** worden hieronder beschreven. Wij raden een layout aan waarbij unaire operatoren *niet* van hun operanden door spaties

1. De term unair is een samentrekking van 'uno', en van 'ariteit' hetgeen 'het aantal argumenten' betekent.

gescheiden zijn.

Haakjes ( ) hebben in **C** altijd de hoogste prioriteit. Haakjes worden gebruikt om in een expressie de volgorde van evaluatie te forceren.
Bijvoorbeeld, in het codefragment *(a + 3) ∗ 4* wordt de sub-expressie *a + 3* het eerst geëvalueerd. Dit tussenresultaat wordt vervolgens met *4* vermenigvuldigd.

Vierkante haken [ ] worden bij indiceren van elementen van reeksen gebruikt. Deze operatoren staan achter de array-naam, en ertussen staat het indexnummer. Bijvoorbeeld, in expressie *a[12]* wordt het dertiende element van reeks *a* aangeduid, het element met index 0 is immers het eerste element in de reeks.
Het indexnummer moet een geheel getal zijn, van het type *int* of *char* (eventueel *signed* of *unsigned*, *long* of *short*). Constructies als *a[12.34]* zijn niet toegestaan.

Struct en union operatoren . en –> hebben met samengestelde types te maken, en worden in onderdeel 9.2 behandeld.

De unaire-min operator – wijzigt het teken van de operand die erachter staat. Als bijvoorbeeld variabele *a* waarde –12 bevat, dan is *–a* gelijk aan 12.

De bitwise not operator ˜ zet alle bits om van de erop volgende operand. Alle bits met waarde 0 worden 1 en vice versa. Bijvoorbeeld is *˜3* gelijk aan *–4*. De bitwise not operator voert dus een one's complement bewerking uit (zie 1.4.3).
De bitwise not is alleen toegestaan bij operanden van het type *int*, en *char*; met andere woorden alleen bij types die een duidelijke bitwise representatie hebben.

De logical not operator ! wijzigt de waarde van de erop volgende operand, maar op een iets andere wijze dan de bitwise not. Als de erop volgende waarde 0 is, dan levert de *!*-operator de waarde 1 op. Als de erop volgende waarde niet 0 is, dan levert de *!*-operator de waarde 0 op.
De logical not wordt in 3.7 verder behandeld.

Pointer operatoren ∗ en & worden bij pointers gebruikt en komen in hoofdstuk 8 ter sprake.

De increment operator ++ verhoogt de waarde van de operand waar de operator bij hoort. De plaats van de operator kan of vóór of achter de bijbehorende operand zijn; men spreekt daarom ook wel van een prefix- of postfix-notatie.
Als van de increment operator in prefix-notatie gegeven is, dan wordt de bijbehorende operand met één verhoogd, en wordt vervolgens voor een verdere rekenkundige bewerking gebruikt.
Bijvoorbeeld, in de expressie *++a + 6* wordt variabele *a* eerst met één verhoogd. Die waarde wordt vervolgens bij 6 opgeteld, en het resultaat hiervan wordt voor verdere berekening gebruikt.

Bij de postfix-notatie wordt de waarde van de operand van de ++ operator voor verdere berekening gebruikt; daarna wordt de operand met één verhoogd. Zo wordt in de expressie *a*++ + *6* de waarde van *a* + *6* voor verdere berekening gebruikt. Pas daarna wordt *a* met één verhoogd.

De ++ operator introduceert de term 'lvalue'. Deze term is uit de vertalerbouw afkomstig, en duidt aan dat een operand van een expressie een 'left value' is, oftewel dat deze operand van een waarde voorzien mag worden. Bij een operand die een lvalue is, kan de vertaler een waarde 'in die operand plaatsen', de operand is dus een variabele. De naam lvalue staat voor left value, omdat lvalues gewoonlijk links van het = teken staan.

De ++ operator kan alleen met lvalues worden gebruikt, aangezien de operand van de ++ een waarde moet kunnen krijgen. Daarom zijn constructies als ++*12* of *(–a)*++ niet geoorloofd; in het eerste geval is de operand van de ++ een constante; in het tweede geval is de operand van de ++ een tussenresultaat.

De increment-operator is alleen bij datatypes toegestaan die een bitwise representatie hebben; dit zijn de types *int* en *char*.

De decrement-operator – – werkt analoog aan de ++ operator, maar verlaagt de waarde van een operand met één. Ook bij de – – operator is sprake van een prefix- en postfixvorm.

De sizeof operator geeft het aantal bytes dat de erop volgende operand als representatie in het geheugen inneemt. De operand van *sizeof* moet ofwel een typenaam ofwel een variabele zijn.

Bijvoorbeeld is onder de Microsoft implementatie van **C** *sizeof int* gelijk aan 2, aangezien een *int* in het geheugen als twee bytes gerepresenteerd wordt.

Bij de *sizeof* operator dient het volgende opgemerkt te worden:

- De *sizeof* operator levert een constante op. Dat betekent dat de *sizeof* operator bij compile-time initialisaties gebruikt kan worden. Een voorbeeld hiervan is in het volgende codefragment weergegeven:

```
static int
    doublesize = sizeof (double);
```

- Als de operand van de *sizeof* operator een naam van een variabele is, dan levert *sizeof* de omvang van het type van die variabele. Bijvoorbeeld, als variabele *counter* van type *int* is, dan levert *sizeof(counter)* twee op.

  De variabelenaam mag ook op een array duiden. In dat geval levert *sizeof* de omvang van het gehele array, wederom in bytes. In het volgende voorbeeld is *row* een array van tien *int*'s. De omvang van dit array is daarom tien maal twee bytes:

```
int
    row [10],
    rowsize;

...
rowsize = sizeof (row);
```

- Als de operand van *sizeof* een string-variabele is, dan levert *sizeof* het aantal bytes op dat die string in het geheugen beslaat. Bij dit aantal wordt de afsluitende \0 meegeteld. In onderstaand voorbeeld is *sizeof(str)* gelijk aan 5:

```
char
    str [] = "test";
int
    strsize;

...
strsize = sizeof (str);
```

  Als echter de operand van *sizeof* geen string-*variabele* maar een string zelf is, dan levert *sizeof* het aantal bytes in die string *uitgezonderd* een \0-byte. Zo is *sizeof ("test")* gelijk aan 4.

Opgave 10.

Wat zijn de waarden van *i* en *j* na de volgende statements?

```
int
    i = 10,
    j;

j = i++;
```

Opgave 11.

Waarom is, in de volgende twee statements, het eerste statement niet juist en het laatste wel?

```
int
    i = 6,
    j = 5;

j = --(-i);
j = -(--i);
```

Opgave 12.

Schrijf een programma dat de omvang van de datatypes *char, int, float* en *double* in bits afdrukt. Gebruik voor het afdrukken de *printf()* functie, waarbij tussen de haakjes een string geplaatst wordt. In deze string kan verklarende tekst staan. Bovendien moet in de string het speciale symbool *%d* staan. Functie *printf()* interpreteert dit symbool als een opdracht om een geheel getal af te drukken; het symbool wordt daarom ook wel een deel van de format string genoemd. Het af te drukken getal wordt na de string, nog binnen het sluithaakje van de *printf()*, gegeven.

Een mogelijk statement is bij voorbeeld

printf("Omvang van een char is %d \n", ...);

Hierbij moet op de plaats van de puntjes een expressie ingevuld worden. Tussen de string en de expressie moet een komma staan. Laat de vertaler zelf de omvang van de verschillende datatypes bepalen!

Opgave 13.

Schrijf een functie *timescalled()* die een boodschap, waarin aangegeven wordt hoevaak *timescalled()* aangeroepen is, naar het scherm schrijft. Bij de eerste aanroep moet zo de boodschap verschijnen: *Functie is 1 keer aangeroepen.* Gebruik de format string *%d* voor het wegschrijven van een getal, zie ook opgave 12.

Test de functie met behulp van het volgende code-frame:

```
#include <stdio.h>

void timescalled ()
{
    // code van timescalled() hier
}

void main ()
{
    timescalled ();                          // eerste aanroep
    timescalled ();                          // tweede aanroep
    timescalled ();                          // derde aanroep
}
```

3.3 Binaire operatoren

Het volgende overzicht belicht de binaire operatoren van **C**. Binaire operatoren staan altijd tussen de twee bijbehorende operanden; er bestaat dus niet iets als een prefix- of postfix-notatie voor binaire operatoren. Volgens onze layout-suggestie worden binaire operatoren door spaties van hun operanden gescheiden.

Vermenigvuldiging wordt in **C** met een asterisk $*$ aangeduid.

Deling wordt door / aangeduid.

De modulobewerking wordt door *%* aangeduid. De modulo bewerking is de rest-uitkomst van een deling: *10 % 3* is gelijk aan 1.

Optelling wordt door + aangeduid.

Aftrekking wordt door – aangeduid. Merk op dat de unaire min operator, die het teken van een operand omwisselt, ook door – aangeduid wordt. De **C** vertaler kan tussen de unaire en binaire min onderscheid maken doordat rond de binaire min twee operanden staan.

Left shift wordt door << aangeduid. De left shift operator schuift de bits van zijn linker operand naar links over het aantal posities dat door de rechter operand wordt aangeduid. Dus, als variabele *a* waarde 12 bevat, dan levert *a << 1* waarde 24 op.

5	0	0	0	0	0	1	0	1	
6	0	0	0	0	0	1	1	0	
									and
4	0	0	0	0	0	1	0	0	

Figuur 3.1: De bitwise and bewerking: 5 & 6 == 4.

De left shift operator kan alleen toegepast worden op operanden die een bitwise representatie hebben. Dit zijn de types *char* of *int*, eventueel *long*, *short*, *signed* of *unsigned*.
De left shift bewerking over één plaats levert een resultaat dat twee maal de waarde van het niet verschoven operand is; een left shift over twee bits levert hetzelfde op als een vermenigvuldiging met vier. Dit is een consequentie van de representatie in binaire vorm.[2]

De right shift operator werkt analoog aan de left shift, en wordt met >> aangeduid. Een right shift bewerking over één bit resulteert in een waarde die identiek is aan een deling door 2.

De kleiner-dan operator wordt aangeduid door <, en wordt in logische expressies gebruikt. Dit wordt in 3.7 verder toegelicht. Analoog aan deze operator werkt de <=, die op 'kleiner dan of gelijk aan' test.

De groter-dan operator wordt aangeduid door >, en wordt in logische expressies gebruikt. Dit wordt in 3.7 verder toegelicht. Analoog aan deze operator werkt de >=, die op 'groter dan of gelijk aan' test.

De (on)gelijk-aan operatoren worden met resp. == en != aangeduid. Ook deze operatoren komen in 3.7 aan de orde.

De bitwise and operator voert een *and*-bewerking uit over alle bits van de twee operanden. Als beide bits 1 zijn, levert de and ook 1 op; anders 0. De bitwise and wordt door & aangeduid.
Zo levert *5 & 6* waarde 4 op, zoals in figuur 3.1 geïllustreerd.

De bitwise exclusive-or operator wordt door ^ aangeduid. Evenals de bitwise and operator bewerkt ^ alle bits van de operanden die eromheen staan. Als één van de bits *maar niet allebei* 1 is, dan levert de exclusive or 1 op; anders 0.
Zo levert *5 ^ 6* waarde 3 op, zoals geïllustreerd in figuur 3.2.

De bitwise or operator wordt door | aangeduid. Deze operator bewerkt, evenals de bitwise exclusive or, alle bits van de operanden die eromheen staan. Als één van de bits of beide bits 1 zijn, dan levert de

2. Overigens is vermenigvuldiging met 2 veel trager dan een left shift over één bit; ongeveer een factor 5 à 6.

5	0	0	0	0	0	1	0	1
6	0	0	0	0	0	1	1	0
3	0	0	0	0	0	0	1	1

xor

Figuur 3.2: De bitwise exclusive or bewerking: 5 ^ 6 == 3.

bitwise or operator 1 op.
Zo levert *5* | *6* waarde 6 op.

De logical and operator , aangeduid door &&, vergelijkt de waarden van de twee operanden, en levert 1 op als *beide* operanden niet-nul zijn. De logical and wordt verder in 3.7 besproken.

De logical or operator werkt analoog aan de logical and, maar levert 1 op als één van de beide operanden of allebei niet-nul zijn. De logical or wordt met || aangeduid. Ook deze operator komt in 3.7 aan de orde.

3.4 Assignment operatoren

Van de toekenningsoperatoren is de = operator al besproken. Deze operator, die ook de simple assignment operator genoemd wordt, kopieert de waarde van de rechter expressie naar de linker lvalue.

Naast de simple assignment operator bestaat een aantal compound assignment operatoren. Al deze operatoren zijn een samenvoegsel van een binaire niet-logische operator en van =. De compound assignment operatoren vervullen een samengestelde functie. Zo kan de toekenning

```
a = a + 5;
```

in een kortere vorm, die ook in kortere code resulteert, geschreven worden als

```
a += 5;
```

Als een tweede voorbeeld van compound assignment illustreren wij hier de ^= operator. Deze operator wordt vaak gebruikt om één of meer bits in een variabele om te zetten. Zo zet de toekenning *a ^= 1;* het minst significante bit (het meest rechtse bit) van variabele *a* om. Dit is te verklaren door het feit dat constante *1* als binair getal *1* geschreven kan worden. De exclusive or-bewerking zal daarom het meest rechtse bit van variabele *a* omzetten. Het resultaat van deze expressie wordt weer aan *a* toegekend doordat de ^-operator in een compound assignment voorkomt. Analoog hieraan zet de toekenning *a ^= 3;* bits 0 en 2 van variabele *a* om, omdat 3 als binair getal *101* geschreven kan worden.

De compound assignment operatoren zijn in figuur 3.3 weergegeven.

Operator	Voorbeeld	Staat voor
$*=$	a $*=$ 3;	a = a $*$ 3;
$/=$	a $/=$ 3;	a = a / 3;
%=	a %= 3;	a = a % 3;
$+=$	a $+=$ 3;	a = a + 3;
$-=$	a $-=$ 3;	a = a – 3;
<<=	a <<= 3;	a = a << 3;
>>=	a >>= 3;	a = a >> 3;
&=	a &= 3;	a = a & 3;
|=	a |= 3;	a = a | 3;
^=	a ^= 3;	a = a ^ 3;

Figuur 3.3: Compound assignment operatoren.

Opgave 14.

Wat zijn de waarden van *i* en *j* na het volgende statement?

```
int
    i = 6,
    j = 6;

j += ++i;
```

Opgave 15.

Wat zijn de waarden van *i* en *j* na het volgende statement?

```
int
    i = 6,
    j = 6;

j *= -++i;
```

Opgave 16.

Wat zijn de waarden van *i* en *j* na het volgende statement?

```
int
    i = 6,
    j = 2;

i += i-- - j;
```

Opgave 17.

Hoe associëren de compound assignment operatoren (zie bijlage A.4)?

Opgave 18.
Wat is de waarde van *a* na het volgende statement, gezien het antwoord op opgave 17?

```
int
    a = 1;

a += a += 1;
```

Opgave 19.
Wat zijn de waarden van *i* en *j* na het volgende statement? Waarom voldoet dit statement niet aan de layout-conventie?

```
int
    i = 6,
    j = 5;

j >>= i-- -j;
```

3.5 Overige operatoren

Naast de unaire, binaire en assignment-operatoren kent **C** nog twee operatoren die in geen van de groepen passen.

De ternaire operator. De ternaire operator wordt aangeduid door een ? en een :, en heeft drie operanden. De eerste operand, die voor het ? staat, is een logische expressie die het resultaat van de gehele expressie bepaalt. Na het ? staat één van de alternatieven, gevolgd door :, gevolgd door het tweede alternatief. Zo is het resultaat van de expressie

```
a > 5 ? 20 : 40
```

20 als *a* groter is dan 5; anders is het resultaat 40. De ternaire operator wordt vaak in toekenningen gebruikt, zoals geïllustreerd in het volgende voorbeeld:

```
int
    a,
    b,
    max;

...
max = a > b ? a : b;
```

Variabele *max* krijgt in dit voorbeeld de waarde van *a* of van *b*, afhankelijk van het feit of *a* groter is dan *b*. De expressie-elementen in een ternaire operator-expressie kunnen ook weer expressies zijn die van de ternaire operator gebruik maken; ternaire operator-expressies kunnen ook *genest* zijn. Dit is geïllustreerd in het volgende voorbeeld:

```
int
    a,
    b,
    c,
    max;

...
max = a > b ?
            (a > c ? a : c)
        :
            (b > c ? b : c);
```

Bij deze toekenning wordt de maximale waarde van variabelen *a*, *b* en *c* bepaald en toegekend aan variabele *max*. Let ook op de layout-suggestie. Complexe expressies kunnen over meerdere regels worden uitgeschreven, waarbij de expressie-elementen geïndenteerd zijn.

De sequentiële evaluatie operator. Meerdere toekenningen kunnen in één statement samengevoegd worden door de sequentiële evaluatie operator, aangeduid door ,. Zo staat het statement:

```
a = 5, b = 6;
```

voor twee toekenningen.

Opgave 20.
Hoe vaak wordt in onderstaand voorbeeld *Hello World!* afgedrukt? N.B.: Het antwoord is niet uit de syntaxis van C af te lezen.

```
int
    i = 10,
    j = 20,
...
while (i--, j--)
    printf ("Hello World! \n");
```

3.6 Type checking en type conversions

Bij toekenningen wordt gewoonlijk een waarde van een bepaald type toegekend aan een variabele van hetzelfde type. Van tevoren is dan bekend dat geen informatieverlies zal optreden.

Soms moet echter bij een toekenning een waarde van het ene type naar een variabele van een ander type overgebracht worden. Bij toekenningen van verschillende typen kan òfwel een waarde naar een nauwkeuriger type overgebracht worden, òfwel naar een minder nauwkeurig type.

Een voorbeeld van een toekenning tussen verschillende typen is hieronder gegeven:

```
int
    int_val;
```

```
float
    float_val;

float_val = int_val;
```

De **C** vertaler moet bij een toekenning waarbij verschillende typen gebruikt worden een type conversie uitvoeren. In bovenstaand voorbeeld moet de waarde van *int_val* naar een andere bit-representatie geconverteerd worden, die vervolgens naar *float_val* overgebracht moet worden.

De **C** vertaler hanteert de vuistregel dat type conversies waarbij *geen* informatieverlies optreedt automatisch uitgevoerd worden. Dit is het geval in bovenstaand voorbeeld; een *int* waarde kan altijd als een *float* gerepresenteerd worden.

Opgave 21.

Gegeven dit programmafragment:

```
char
    ch;
int
    i;
float
    f;
double
    d;

result = (ch / i) + (f * d) - (f + i);
```

Wat moet het type van variabele *result* zijn, om de vertaler de toekenning zonder waarschuwingen te laten vertalen en om het resultaat van de berekening zonder informatieverlies over te brengen?

Wanneer echter bij een toekenning informatieverlies *kan* optreden, geeft de vertaler een waarschuwende melding, zoals bij het volgende voorbeeld:

```
int
    int_val;
char
    char_val;

char_val = int_val;
```

Bij deze toekenning weet de vertaler niet of *int_val* als een *char* waarde gerepresenteerd kan worden zonder informatieverlies. Het kan echter voorkomen dat de programmeur, ongeacht informatieverlies, een toekenning wil plegen naar een minder precies type. Voor deze gevallen kent de taal **C** de 'type cast', waarmee aangeduid wordt dat de vertaler een expressie even als van een ander type moet beschouwen. De type cast wordt aangeduid met een typenaam tussen haakjes. Dit is weergegeven in het volgende voorbeeld:

```
int
    int_val;
char
    char_val;

char_val = (char) int_val;
```

Hier wordt *int_val* door de type cast *(char)* even als type *char* opgevat, zodat de toekenning van deze waarde naar *char_val* geen waarschuwingen oplevert. Wat wordt, na uitvoering van de hierboven beschreven toekenning, de waarde van *char_val* als *int_val* waarde 300 heeft? Waarde 300 kan als *int* geschreven worden als 0x012c. Bij de type cast naar *char* wordt van deze waarde het rechter byte gebruikt, dus 0x2c. Deze waarde wordt naar *char_val* overgebracht; het meest significante byte gaat bij deze toekenning verloren.

Opgave 22.
Gegeven het volgende programmafragment:

```
char
    ch;
long
    l = 707651;

...
ch = (char) l;
printf ("Een letterteken: %c \n", ch);
```

Wat voor letterteken verschijnt op het scherm? Hierbij staat de format string *%c* voor een signaal aan *printf()* om een letterteken af te drukken. De numerieke code (ASCII-code) van dit letterteken verwacht *printf()* in het argument dat op de string volgt; dit is de waarde van variabele *ch*.

3.7 Logische expressies en logische operatoren

In dit onderdeel wordt nader ingegaan op logische expressies en logische operatoren.

Logische expressies zijn expressies waarbij logische operatoren gebruikt worden. Meestal worden logische expressies gebruikt bij de bepaling van een volgende actie. De ternaire operator is een voorbeeld van zo'n situatie: in de toekenning:

```
max = a > b ? a : b;
```

is de expressie $a > b$ beslissend voor de toekenning die gaat plaatsvinden. Variabele *max* krijgt òfwel de waarde van *a* toegekend, òfwel de waarde van *b*. Naast de ternaire operator bestaan in **C** andere constructies waarbij logische bewerkingen gebruikt worden; deze constructies komen in hoofdstuk 4 aan de orde.

Logische expressies kunnen uit sub-expressies bestaan die door logische operatoren aaneen gekoppeld worden. Een voorbeeld van zo'n koppelende logische operator is de logische and operator, aangeduid door &&. Zo is een expressie:

```
subA && subB
```

waar als *beide* subdelen *subA* en *subB* waar zijn.

Alle logische operatoren zijn in figuur 3.4 geïllustreerd. Hierbij hebben de operatoren bovenaan de tabel de hoogste prioriteit. Operatoren met verschillende prioriteit zijn door een lijn van elkaar gescheiden. Alle logische operatoren associëren van links naar rechts.

Operator	Betekenis
!	logische not
<	kleiner dan
>	groter dan
<=	kleiner dan of gelijk aan
>=	groter dan of gelijk aan
==	gelijk aan
! =	ongelijk aan
&&	logische and
||	logische or

Figuur 3.4: Logische operatoren in volgorde van prioriteit.

3.7.1 Nul en niet-nul

Bij het nemen van beslissingen spelen logische expressies een belangrijke rol. In deze context worden logische expressies ook *condities* genoemd. Op grond van condities wordt besloten of code al dan niet zal worden uitgevoerd.

C hanteert hierbij de regel dat de waarde nul als logisch onwaar (false) wordt beschouwd, terwijl *alle* andere waarden als logisch waar (true) worden beschouwd[3].

In de taal **C** bestaan meerdere notaties om waarde nul aan te geven. Vooruit lopend op de hoofdstukken 4 en 8 zijn deze notaties alvast hier, in figuur 3.5 genoemd. Hierbij dient opgemerkt te worden dat *NULL* geen 'ingebakken' symbool is, maar een macro die in de header file *stdio.h* gedefinieerd is.

3.7.2 Volgorde van evaluatie bij logische expressies

Logische expressies waarbij de && of || operatoren gebruikt worden nemen in **C** een aparte plaats in. Bij gebruik van deze operatoren is in **C** namelijk

3. In **C** bestaat dus niet een apart type *boolean*, zoals in **pascal**. Om deze reden geven we er de voorkeur aan dat ook niet te suggereren door het gebruik van termen als true en false. In plaats daarvan spreken we liever over condities die al dan niet nul zijn.

Symbool	Gebruikt bij
0, 0.0, 0E0	aritmetische vergelijking
'\0'	character vergelijking
NULL	pointers

Figuur 3.5: Notaties voor waarde nul.

de *volgorde van evaluatie* van tevoren bepaald. De volgorde van evaluatie is in **C** *alleen* bij deze twee operatoren bepaald; bij de andere operatoren is de volgorde van evaluatie afhankelijk van de gebruikte vertaler. Merk dus op dat prioriteit of associativiteit niets zegt over de volgorde van evaluatie!

Bij de && en || operatoren is de regel dat de code waarmee een dergelijke logische expressie wordt bepaald, gestaakt wordt als de uitkomst van die expressie bekend is.

Bij de onderstaande toekenning:

```
result = a || b ? 20 : 40
```

is de logische expressie *a* || *b* zo'n voorbeeld. Als *a* naar niet-nul evalueert, dan is het resultaat van de expressie al bekend: *result* zal waarde 20 krijgen. Alleen als *a* naar nul evalueert moet sub-expressie *b* nog bekeken worden.

Analoog is in:

```
result = a && b ? 20 : 40
```

het resultaat van expressie *a && b* al bekend als *a* naar nul evalueert. In dat geval zal *result* waarde 40 toegekend krijgen, onafhankelijk van het resultaat van de evaluatie van *b*.

Bij de twee bovenstaande voorbeelden is het wel of niet evalueren van het tweede deel van een logische sub-expressie niet echt relevant. Dit wordt echter anders als wij onderstaand voorbeeld bekijken:

```
int
    result,
    a,
    b;

...
result =
    ++a || ++b ?
            20
        :
            40;
```

In dit voorbeeld wordt bij de toekenning van een waarde aan *result* variabele *a* altijd met één verhoogd. Als het resultaat van deze verhoging niet-nul oplevert, dan wordt *result* 20. Als het resultaat van deze verhoging echter wel nul oplevert, dan wordt *b* met één verhoogd, en met nul vergeleken. Afhankelijk van de waarde van *a* wordt dus niet alleen een bepaalde waarde aan *result* toegekend, maar wordt ook *b* wel of niet verhoogd. Dit verschijnsel wordt een neveneffect (*side effect*) genoemd. Neveneffecten

zijn potentiële bronnen van fouten bij de verwerking van een programma, en moeten zo mogelijk vermeden worden.

De volgorde van evaluatie is tevens relevant als de sub-expressies van een logische expressie aanroepen van functies zijn. Dit wordt in hoofdstuk 5 verder besproken.

3.7.3 De wetten van De Morgan

Het combineren van logische expressies tot één geheel door middel van logische operatoren introduceert de wetten der logica, zoals beschreven door De Morgan [4]. Deze wetten hebben betrekking op het omkeren van een samengestelde logische expressie door middel van de '!'-operator. De wetten worden door de volgende twee regels beschreven:

1. *!(a && b) == !a || !b*
2. *!(a || b) == !a && !b*

De juistheid van de regels kan eenvoudig door de constructie van een *waarheidstabel* geverifieerd worden. Als voorbeeld is de eerste regel uitgewerkt in figuur 3.6. Achtereenvolgens wordt voor elke mogelijke waarde van *a* en *b* een combinatie van de logische expressie afgewerkt. Als de waarheidstabel geconstrueerd is, wordt het resultaat van *!(a && b)* vergeleken met het resultaat van *!a || !b*.

Expressie	**Mogelijkheden gezien *a* en *b***			
a	nul	nul	niet-nul	niet-nul
b	nul	niet-nul	nul	niet-nul
a && b	nul	nul	nul	niet-nul
!(a && b)	niet-nul	niet-nul	niet-nul	nul
!a	niet-nul	niet-nul	nul	nul
!b	niet-nul	nul	niet-nul	nul
!a || !b	niet-nul	niet-nul	niet-nul	nul

Figuur 3.6: Waarheidstabel bij *!(a && b)*.

4. Uit: J.W. Bishir, D.W. Drewes

Hoofdstuk 4

Flow control

Eén van de belangrijkste kenmerken van programma's is dat ze hun code *conditioneel* kunnen uitvoeren. Een programma waarvan de code slechts eenmalig (dus sequentieel, van begin tot eind) uitgevoerd wordt, zal doorgaans niet erg interessant zijn. Zelfs eenvoudige programma's bevatten praktisch altijd condities. Wanneer we een stukje code dat een directory listing laat zien in gedachten nemen (zoals de DOS opdracht *dir*), dan blijkt al snel dat dit programma ook condities kent: de informatie wordt (conditioneel) in kolommen afgedrukt als de '*/w*' vlag is meegegeven. Ook intern moet door een dergelijk programma beslissingen worden genomen: is de laatste filenaam van deze directory afgedrukt? Zo ja, dan kan gestopt worden, zo nee, dan moet de volgende filenaam verwerkt worden.

Dit hoofdstuk gaat over flow control: de syntactische mogelijkheden die door **C** worden geboden om in een programma beslissingen te nemen over de uit te voeren code.

4.1 Selectie statements

4.1.1 De enkelvoudige if-else

Het eenvoudigste flow control statement is het *if-statement*. De algemene vorm van het *if*-statement is[1]:

```
if (expressie)
    statement-1
[[else
    statement-2
]]
```

Hierbij gelden de volgende regels:

- Een *statement* kan een enkelvoudig statement zijn, afgesloten door '**;**' of kan een meervoudig statement zijn. Meervoudige statements (compound statements) worden voorafgegaan door '{' en afgesloten door '}'.
- Het gedeelte tussen de dubbele rechte haken (de *else*-staart) is optioneel.

Wanneer de expressie nul oplevert wordt het statement-1 niet uitgevoerd; wanneer de *else* en statement-2 zijn gegeven, wordt in dat geval statement-2 uitgevoerd. Wanneer de expressie niet-nul oplevert, wordt statement-1 uitgevoerd en statement-2 niet. Een voorbeeld van twee *if*-statements volgt hieronder:

1. In de beschrijving van syntactische constructies gebruiken we de dubbele rechte haken om aan te geven dat het gedeelte tussen de dubbele rechte haken optioneel is, en dus mag vervallen. De dubbele rechte haken horen niet tot de syntactische constructie, en worden bij het programmeren dus nooit getypt.

```
if (x == 3)
    y += 4;                     // conditioneel verhogen van y

if (y == 7)
    x--;                        // reduceer x
else
    y = 0;                      // of zet y op 0
```

4.1.2 De geneste if-else

If-statements kunnen worden *genest*. De nesting ontstaat wanneer het statement onder het *if*-statement zelf weer een *if*-statement bevat, zoals in het volgende voorbeeld, waarbij opzettelijk de layout conventie niet is gevolgd:

```
if (a) if (b) x = 1; else x = 3;
```

Bij de nesting van *if*-statements ontstaat een *ontleedprobleem*. Het bovenstaande voorbeeld kan volgens de syntaxis van **C** worden geïnterpreteerd als een *if*-statement, waaronder een *if-else*-statement is genest, maar kan ook worden geïnterpreteerd als een *if-else*-statement waarbij een enkelvoudig *if*-statement onder de eerste *if* is genest. De beide alternatieven zijn dus:

```
// Nesting van if-else onder if:

if (a)
{
    if (b)
        x = 1;
    else
        x = 3;
}

// Bovenstaand statement wordt ontleed als:

if (expressie)
    statement


// Nesting van if onder if-else:

if (a)
{
    if (b)
        x = 1;
}
else
    x = 3;

// Bovenstaand statement wordt ontleed als:

if (expressie)
    statement
else
    statement
```

Uit de syntaxis van **C** blijkt niet welk alternatief moet worden gekozen. Deze dubbelzinnigheid (ambiguïteit) is inherent aan de grammatica van **C** en staat bekend als het 'dangling else' probleem. Het probleem wordt opgelost doordat (per conventie) een *else* altijd met de dichtsbijzijnde *if* wordt gecombineerd. Wanneer de layout conventie juist wordt toegepast, wordt de conventionele interpretatie van het *if-else*-statement ook gesuggereerd:

```
if (a)
    if (b)
        x = 1;
    else
        x = 3;
```

Om de *else* met de eerste *if* te combineren, moet een compound statement worden gebruikt:

```
if (a)
{
    if (b)
        x = 1;
}
else
    x = 3;
```

4.1.3 if-else ladders

Naast geneste *if-else*-statements komen ook vaak series aaneengeschakelde *if-else*-statements voor. In dat geval begint het *else* deel van een *if-else*-statement met een volgend *if*-statement. Volgens de layout conventies zou dat volgende *if*-statement dus moeten worden geïndenteerd ten opzichte van de eraan voorafgaande *else*. Dat is onprettig, omdat de layout conventie er dan al snel toe leidt dat de code sterk naar rechts schuift. Bovendien is het goed mogelijk het geschakelde *if-else*-statement te beschouwen als de implementatie van een situatie waarbij meervoudige beslissingen noodzakelijk zijn. Bij geschakelde *if-else*-statements wordt het statement bij de eerste conditie die niet-nul is, uitgevoerd, alle overige statements worden overgeslagen.

Een dergelijke situatie noemen we een *if-else-ladder*. De *if-else*-ladder wordt gebruikt wanneer meerdere condities moeten worden getest en het statement bij de eerste conditie die niet nul is moet worden uitgevoerd. Een voorbeeld van zo'n situatie zien we hieronder, waarbij verondersteld wordt dat in *c* een tekenwaarde uit de ASCII tekenverzameling is opgeslagen. Het statement achter de laatste *else* in dit voorbeeld zal worden uitgevoerd indien aan geen van de eerdere voorwaarden is voldaan; het is dus een default actie:

```
if ('a' <= c && c <= 'z')
```

```
    printf ("een kleine letter\n");
else if ('A' <= c && c <= 'Z')
    printf ("een hoofdletter\n");
else if ('0' <= c && c <= '9')
    printf ("een cijfer\n");
else
    printf ("geen letter of cijfer\n");
```

De variabele die de gekozen ingang in de ladder bepaalt heet de *controle variabele* (control variable). In bovenstaande voorbeeld is dat de variabele *c*. Bij gebruik van één controle variabele kan het *switch*-statement (zie 4.1.5) ook vaak goed worden gebruikt. Wanneer echter op meerdere variabelen wordt gelet is een *if-else* ladder doorgaans de aangewezen constructie. Hier volgt nog een *if-else* ladder met twee controle variabelen:

```
if (x == 1)
    printf ("x is 1, y misschien ook wel\n");
else if (y == 1)
    printf ("y is 1, maar x niet\n");
else
    printf ("x is niet 1, en y is ook niet 1\n");
```

4.1.4 De ternaire operator ?:

De *ternaire* operator *?:* kan gebruikt worden om simpele *if-else* constructies te herschrijven naar snellere code. De *?:* operator is besproken in 3.5. Een voorbeeld van het gebruik van de ternaire operator wordt hieronder gegeven:

```
antwoord =
    x < 200 ?
        2
    :
        5;
```

Wanneer iemand hardop leest wat er in het vorige voorbeeld staat, dan wordt het statement geïnterpreteerd als:

"antwoord wordt, als x kleiner dan 200 is, 2, anders 5"

Een dergelijke uitspraak kunnen we ook omzetten naar een *als-dan* uitspraak:

"als x kleiner is dan 200, dan wordt antwoord 2, anders wordt antwoord 5"

Wanneer dat weer wordt vertaald naar **C** ontstaat het volgende *if-else*-statement:

```
if (x < 200)
```

```
        antwoord = 2;
    else
        antwoord = 5;
```

Het is weliswaar altijd mogelijk een expressie, waarin een ternaire operator wordt gebruikt, om te zetten naar een *if*-statement, maar een dergelijke omzetting is niet altijd even fraai. Het gebruik van de ternaire operator levert altijd een expressie op. De ternaire operator kan daarom *zelf* in een expressie worden gebruikt. Het *if*-statement levert echter geen expressie op. Het *if*-statement kan daarom niet in een expressie worden gebruikt. Omgekeerd kan een *if*-statement niet altijd worden omgeschreven naar het gebruik van een ternaire operator. Een *if*-statement kan worden omgeschreven naar een ternaire expressie wanneer aan de volgende voorwaarden is voldaan:

- Er is ook een *else* onderdeel.
- De statements onder het *if-else*-statement zijn waardetoekenningen aan *dezelfde* variabele.

Hieronder geven we nog een paar voorbeelden van omzettingen van *if-else*-statements naar expressies met ternaire operatoren en omgekeerd:

```
// omzetting 1: van: ?:    naar: if-else

printf("Uw antwoord is %s\n",      // ternaire operator
    goed ?
        "goed"
    :
        "fout");

printf("Uw antwoord is" );         // if-else conversie
if (goed)
    printf("goed\n");
else
    printf("fout\n");

// omzetting 2: van: ?:    naar: if-else

x =                                // ternaire operatoren
    ((a > b) ? 3 : 4) * ((c > d) ? 5 : 6);

if (a > b)                         // if-else conversie
{
    if (c > d)
        x = 3 * 5;
    else
        x = 3 * 6;
}
else
{
    if (c > d)
        x = 4 * 5;
    else
        x = 4 * 6;
}

// omzetting 3: van: if-else    naar: ?:

if (a && b || c)                   // if-else statement
    x = 5;
else
```

```
    x = 14;

x =                                  // ternaire operator conversie
    a && b || c ?
        5
    :
        14;
```

4.1.5 De switch

Het *switch*-statement kan in plaats van de *if-else* ladder gebruikt worden als er slechts één controle variabele wordt gebruikt: dit levert beter leesbare en vaak snellere code op. De syntaxis is:

```
switch (expressie)
{
    [[case const_1:]]
        [[statements]]
        [[break;]]

    [[case const_2:]]
        [[statements]]
        [[break;]]

    ...

    [[default:]]
        statements
        [[break;]]
}
```

De statements bestaan uit minstens één statement, maar zijn optioneel met dien verstande dat er *minstens* één statement vóór de sluitaccolade van de *switch* moet staan. Dat statement mag overigens *leeg* (of dummy) zijn: een statement dat niets doet. Het lege statement bestaat uit een puntkomma of uit een openaccolade, onmiddellijk gevolgd door een sluitaccolade.

De *cases* in de *switch* constructie geven het resultaat aan van de evaluatie van de *expressie*. De *cases* en de *default* definiëren zo *ingangen* (entry points) in de *switch*. Er mogen —uiteraard— geen twee *cases* met dezelfde *waarde* voorkomen, terwijl de waarden achter de *case* ingangen *expressies* moeten zijn die louter *int* of *char* constanten als operand bevatten. Wel mogen meerdere *cases* achter elkaar voorkomen.

Het *break*-statement zorgt ervoor dat het *switch*-statement verlaten wordt. De uitvoering van het programma gaat bij een *break* verder achter de sluitaccolade van de *switch*. De *break* kan ook weggelaten worden, in welk geval de statements van een eventuele volgende *case* of *default* ingang in de *switch* ook worden uitgevoerd. Dit heet het *doorvallen* (falling through). Wanneer er geen volgende *case* of *default* ingang is en er is ook geen laatste *break*-statement, dan wordt de *switch* verlaten.

In het hieronder gegeven voorbeeld is de *default* niet afgesloten door een *break*, omdat er toch geen volgende *case* is. Een tweede voorbeeld van doorvallen is de *'a'* en de *'A'* case. Kernighan en Ritchie merken over het doorvallen op *"Falling through cases is a mixed blessing."* Het doorvallen kan namelijk eenvoudig tot problemen leiden: op een dag wordt een programma uitgebreid met een extra *case* aan het einde van een *switch*, terwijl de laatste *case* of *default* ingang niet was afgesloten door een *break,* hetgeen tot merkwaardige problemen kan leiden. Het is dan ook een goede gewoonte het doorvallen (ook uit de *switch* bij de laatste ingang) extra duidelijk in de brontekst van commentaar te voorzien, om zo te voorkomen dat bij latere wijzigingen onbedoeld doorvallen zal optreden. De *default* ingang hoeft overigens niet de *laatste* ingang in de *switch* te zijn. De *default* kan bij voorbeeld ook doorvallen in een andere *case* ingang. Door de layout conventie wordt gesuggereerd dat de *default* ingang de *laatste* ingang in de *switch* is, tenzij de *default* ingang moet doorvallen in een *case:*

```
switch (ch)
{
    case 'a':
    case 'A':
        printf ("Eerste letter van het alfabet, da's de beginletter\n");
        break;

    case 'b':
    case 'B':
        printf ("De tweede letter van het alfabet\n");
        break;

    ...

    default:
        printf ("Geen van alle\n");
    // *** falling through ***
}
```

Een subtiliteit van de vertaler, die tot een fout bij de uitvoering kan leiden, kan optreden bij het gebruik van een 'enumeration type' in combinatie met een *switch*. Vooruitlopend op enumeration types kan worden opgemerkt dat hierbij een opsomming van symbolische namen, die voor *int* constanten staan, wordt gedefinieerd (zie 9.5). Deze namen kunnen worden gebruikt als ingangen in een *switch*. Echter, wanneer de *case* voor de naam uit een enumeration type wordt vergeten, leidt dat niet tot een fout, terwijl de *switch* ingang die werd bedoeld niet wordt gedefinieerd. De symbolische naam die wordt gevolgd door een dubbele punt is namelijk ook de aanduiding van een label waarnaar met behulp van een *goto* (zie 4.3.5) kan worden gesprongen. De vertaler interpreteert de symbolische naam uit een enumeration dus als een label en niet als een *case* ingang in een *switch*.

Ter illustratie en waarschuwing het volgende voorbeeld, waarbij vol-

ledigheidshalve ook een enumeration wordt gedefinieerd, hoewel *enums* pas in 9.5 worden behandeld:

```
typedef enum
{
    enu_1,
    enu_2,
} ENU_;

fun(int a)
{
    switch(a)
    {
        enu_1:                  // dit is GEEN case en GEEN fout
            // *** onbereikte code ***

        case enu_2:             // dit is WEL een case
            ...
    }
}
```

4.2 Repeteer statements

Repeteer statements worden gebruikt om de herhaling van statements te programmeren. Herhaling van statements treedt in vrijwel elk programma op. Wanneer we de DOS *dir* opdracht weer in gedachten nemen, dan zien we dat ook hier een herhaling opdreedt: *zolang* er bestandsnamen in de directory zijn, moet de naam van het eerstvolgende bestand worden getoond. De herhaling (*zolang* ...) is van een eenvoudige soort. In **C** zijn een aantal syntactische constructies beschikbaar voor de implementatie van herhalingen. Wellicht het meest eenvoudige herhalingsstatement is het *while*-statement waarmee een *zolang als* ... herhaling wordt geïmplementeerd. Daarnaast kent **C** echter nog een tweetal constructies: het *for*-statement om een van te voren bekend aantal herhalingen te implementeren, en het *do-while*-statement om minstens één, maar mogelijk meerdere uitvoeringen van een statement te implementeren.

4.2.1 De while

De meest eenvoudige herhaling is wellicht de herhaling van statements zolang dat nodig is. Een dergelijke herhaling kennen we in het Nederlands in zinnen die beginnen met *zolang:* "Zolang het licht is, blijven we wandelen." Een dergelijke herhaling wordt in **C** geïmplementeerd met behulp van het *while*-statement. De syntaxis van het *while*-statement is:

```
while (expression)
    statement;
```

In de meeste gevallen waarbij een *while*-statement wordt gebruikt, is het statement een blok waarbinnen statements zijn gedefinieerd. Een enkelvoudig statement zoals de aanroep van een functie komt echter ook voor.

Het geneste statement van het *while*-statement wordt uitgevoerd zolang de expressie een waarde ongelijk 0 oplevert. Wanneer de expressie direct al 0 oplevert, wordt het geneste statement dus helemaal niet uitgevoerd. In de meeste gevallen wordt een *while*-statement beëindigd doordat de expressie uiteindelijk nul wordt, maar een *while*-statement kan ook geforceerd worden beëindigd door gebruik te maken van de *break*. Een aardige toepassing van de *break* bestaat uit de implementatie van de *eeuwige lus* met ontsnapmogelijkheid. In listing 5 wordt een eeuwige lus begonnen op regel 1. De expressie van het *while*-statement is de constante waarde 1 en de expressie zal dus nooit naar 0 evalueren, waardoor het geneste statement (het blok tussen regels 2 en 9) altijd zal worden uitgevoerd. In het geneste statement worden tekens gelezen en verwerkt totdat end-of-file *(EOF)* wordt bereikt. Het lezen en verwerken is als een drietrapsraket uitgevoerd. Eerst wordt (regel 3) een teken gelezen door de standaard-**C** functie *getchar()*. Deze functie leest gewoonlijk het eerstvolgende teken dat via het toetsenbord (de *standaard invoerstroom (stdin)*) beschikbaar komt. Vervolgens wordt gekeken of de end-of-file conditie van *stdin* bereikt is. Dit gebeurt op regel 5 door de aanroep van de standaard-**C** functie *feof()*. Deze functie levert de waarde 0 op wanneer *EOF* nog niet is bereikt en levert de waarde 1 op wanneer dat wel het geval is. De returnwaarde van de functie *feof()* is dus de waarde van de conditie van het *if*-statement op regel 5. Wanneer *EOF* is bereikt, dan is de conditie niet-nul en wordt het *break*-statement uitgevoerd, waarmee het omhullende *while*-statement wordt beëindigd. Als de *EOF* conditie echter niet wordt bereikt, gaat de verwerking door op regel 8, waar het zojuist gelezen teken wordt verwerkt door de (in dit voorbeeld niet verder gespecificeerde) functie *process_char()*. Na functie *process_char()* begint een volgende cyclus van het *while*-statement. Dit gaat zo door totdat uiteindelijk de *EOF* conditie (in regel 5) is bereikt.

```
1    while (1)                          // eeuwige lus
2    {
3        c = getchar();                 // probeer teken te lezen
4
5        if (feof(stdin))               // klaar bij end of file
6            break;                     // verlaat de while
7
8        process_char(c);               // verwerk 'c'
9    }
10   // vervolg hier de uitvoering bij end-of-file
```

Listing 5: De eeuwige lus.

4.2.2 De do-while

Bij het *do-while*-statement wordt de conditie getest *na* het doorlopen van het geneste statement. Het geneste statement wordt dus minstens éénmaal doorlopen. De syntaxis van het *do-while*-statement is:

```
do
    statement
while (expressie);
```

Het geneste statement kan wederom een enkelvouding statement of een blok zijn. Het *do-while*-statement vindt duidelijk minder toepassingen dan het *while*-statement.

Een voorbeeld waarin het *do-while*-statement kan worden gebruikt, is in het volgende codefragment gegeven. De code is afkomstig uit een programma dat adreslabels kan printen. Voordat het feitelijke printen kan beginnen, krijgt de gebruiker de gelegenheid te controleren of het kettingformulier met de adresplakkers goed in de printer is geplaatst. Daartoe drukt het programma bij herhaling iets af op de plakkers, totdat de gebruiker aangeeft dat de plakkers goed zijn geplaatst:

```
puts ("Druk op 'enter' wanneer de plakkers goed zijn geplaatst");
do
{
    for (i = 1; i <= 20; i++)        // print minstens 1 x
        fprintf(stdprn, "test regel # %d\n", i);
}
while (getch() != '\r');             // stop bij enter
```

Opgave 23.

1. **Wat doen de functies *puts()* en *fprintf()* in het vorige voorbeeld?**
2. **Wat wordt bedoeld met *stdprn*?**

4.2.3 De for

Het *for*-statement heeft de volgende syntaxis:

```
for (init; expr; incr)
    statement;
```

Het *for*-statement heeft, behalve het geneste statement, nog de volgende onderdelen:

- *init* is een initialisatie-expressie. In de expressie kan echter wel gebruik worden gemaakt van de komma-operator, om zo meerdere variabelen bij de initialisatie te initialiseren. Meestal wordt de controlevariabele van het *for*-statement hier geïnitialiseerd. Het initialisatiegedeelte wordt slechts één keer, aan het begin van het *for*-statement, uitgevoerd.

- *expr* is een expressie. Zolang de expressie een waarde ongelijk nul oplevert, wordt het statement dat onder het *for*-statement is genest, uitgevoerd. Wanneer de expressie direct al de waarde nul oplevert, wordt het geneste statement niet uitgevoerd, en wordt het *for*-statement direct na de initialisatie beëindigd.
- *incr* is een expressie, waarin meestal de toename of afname van de controlevariabele van het *for*-statement wordt gedefinieerd. Wederom kan hier goed gebruik worden gemaakt van de komma-operator. De *incr* expressie wordt uitgevoerd telkens na de executie van het geneste statement.

De drie onderdelen mogen ook leeg zijn, waardoor een eeuwige lus ontstaat zoals dat ook bij het *while*-statement mogelijk is. Een voorbeeld van een eeuwige lus met behulp van het *for*-statement is hieronder gegeven:

```
for (;;)                                    // eeuwige lus
{
    c = getchar();                          // probeer teken te lezen

    if (feof(stdin))                        // klaar bij end of file
        break;                              // verlaat de while

    process_char(c);                        // verwerk 'c'
}
// vervolg hier de uitvoering bij end of file
```

Het geneste statement kan opnieuw een enkelvoudig statement of een blok zijn. Een paar voorbeelden van het gebruik van *for*-statements is gegeven in listing 6.

```
1       extern int
2           foo_bar();
3       int
4           delay,
5           row,
6           count,
7           loop,
8           squares [100];
9
10      // voorbeeld 1: dit for-statement kost alleen maar tijd.
11      for (delay = 0; delay < 20000; delay++)
12          ;
13          // let op de ; als leeg statement
14
15      // voorbeeld 2: het array squares bevat de kwadraten van 0 t/m 99
16      for (loop = 0; loop < 100; loop++)
17          squares[loop] = loop * loop;
18
19      // voorbeeld 3: omdat loop nu 100 is, wordt foo_bar() niet uitgevoerd
20      for (row = 0; loop < 79; row += 2)
21          foo_bar();
22
23      // voorbeeld 4: dubbele initialisatie
```

```
24        for (loop = 0, count = 0; loop < 100; loop++)
25            count += ((squares[loop] & 3) == 3);
26
```

Listing 6: *for*-statements.

Merk op:

- Controlevariabelen kunnen binnen het geneste statement een andere waarde krijgen, waardoor het beëindigen van het statement kan worden bespoedigd of vertraagd.
- Na afloop van het *for*-statement houden de controlevariabelen hun laatste waarde[2].
- In nogal wat literatuur over **C** komt de magische functienaam

  foo_bar()

  voor. Foo_bar is afgeleid van fubar, hetgeen een acroniem is voor:

  Fucked Up Beyond All Recognition.

- Het *for*-statement kan vroegtijdig worden verlaten door in het geneste statement een *break*-statement uit te voeren (zie ook 4.3.1).

4.3 Gerelateerde statements

4.3.1 De break

Het *break*-statement beëindigt een lus of *switch*. Het statement kan gebruikt worden om de verwerking van een lus te beëindigen. Wanneer lussen genest zijn, dan wordt de lus waarin de *break* zich bevindt, beëindigd. In onderstaande voorbeelden wordt dit kenmerk geïllustreerd. In het eerste voorbeeld wordt een eeuwige lus beëindigd, in het tweede voorbeeld beëindigt een *break* het *for*-statement wanneer een toets wordt ingedrukt. Het omhullende *while*-statement wordt echter niet beëindigd, en activeert opnieuw het *for*-statement (Het blok achter de *while* is niet nodig, maar is hier voor de leesbaarheid geplaatst):

```
int
    flag,
    i,
    j;

// voorbeeld 1: het verlaten van een eeuwige lus
while (1)
{
    ...
    if (flag == 12)
        break;
    ...
}
```

2. Op deze twee punten wijkt **C** dus af van **pascal**.

```
// voorbeeld 2: het verlaten van een geneste lus
while (1)                          // buitenste lus
{
    for (j = 0; j < 1000; j++)     // binnenste lus
        if (kbhit ())              // statement in binnenste lus
            break;                 // verlaat binnenste lus
}
```

4.3.2 De continue

De werking van het *continue*-statement lijkt enigszins op die van het *break*-statement. Het *continue*-statement forceert de volgende cyclus van een lus, waarbij statements die na de *continue* staan, worden overgeslagen. De *continue* werkt alleen bij repeteerstatements en niet bij de *switch*. Daarmee is meteen een aardige toepassingsmogelijkheid van het *continue*-statement gegeven: in het volgende voorbeeld wordt een implementatie gegeven van de situatie waarbij de invoer bestaat uit cijfers of klinkers. Even en oneven cijfers moeten een unieke en een gemeenschappelijke bewerking ondergaan, terwijl klinkers alleen een unieke bewerking moeten ondergaan. Opgemerkt wordt dat de functie *getchar() EOF* retourneert wanneer end-of-file is bereikt:

```
extern void
    process_even(int),             // externe functies hier niet
    process_odd(int),              // relevant
    process_vowels(int),
    extra_digit_process(int);
int
    c;

while ((c = getchar()) != EOF)     // zolang nog niet EOF
{
    switch(c)                      // doe juiste actie
    {
        case '0':                  // even getallen
        case '2':
        case '4':
        case '6':
        case '8':
            process_even(c);
            break;                 // naar einde switch

        case '1':                  // oneven getallen
        case '3':
        case '5':
        case '7':
        case '9':
            process_odd(c);
            break;                 // naar einde switch

        default:                   // derde mogelijkheid: klinkers
            process_vowels(c);
            continue;              // naar while
    }
    common_digit_process(c);       // getallenverwerking deel 2.
}
```

Wanneer een *continue*-statement in een *for*-statement wordt gebruikt, wordt het geneste statement beëindigd. Echter, de *increment* expressie van het *for*-statement wordt wèl uitgevoerd.

4.3.3 De functie exit()

De functie *exit()* is een functie uit de standaard-C bibliotheek. Deze functie beëindigt het programma. Hoewel *exit()* geen flow control statement is, wordt de functie hier genoemd omdat de flow van een programma afgebroken wordt.

De functie *exit()* heeft één argument: de *exit status* De exit status wordt als byte waarde doorgegeven aan DOS en verschijnt op DOS-niveau terug als de waarde van ERRORLEVEL. Meestal wordt exit status 0 gebruikt indien geen fouten opgetreden zijn in het programma, en een andere statuswaarde (vaak de waarde 1) indien dat wel het geval is. In het volgende voorbeeld wordt met behulp van de standaardfunctie *malloc()* geprobeerd een blok geheugen van 10000 bytes beschikbaar te maken (te *alloceren*). Wanneer dat niet lukt, wordt een passende melding gegeven, waarna het programma via de functie *exit()* stopt. Na het programma heeft ERRORLEVEL de waarde 1:

```
if (!malloc(10000))                          // 10000 bytes gealloceerd ?
{
    printf("Onvoldoende geheugen\n");        // Nee: geef foutmelding
    exit(1);                                 // Stop het programma.
}
```

De functie *exit()* is gedeclareerd in de header file `process.h`, die met een `#include` opdracht wordt opgenomen in de brontekst waarin de functie *exit()* wordt gebruikt (zie ook 6.1).

4.3.4 De return

Met behulp van het *return*-statement kan de functie worden beëindigd waarbinnen dit statement staat. Aangezien in C elke functie is aangeroepen door een aanroepende functie, wordt met behulp van het *return*-statement de *aangeroepen* functie beëindigd, en wordt de uitvoering van de *aanroepende* functie vervolgd op het punt dat onmiddellijk volgt op de aanroep van de aangeroepen functie. Het *return*-statement kent twee syntactische varianten: met en zonder expressie achter het keyword *return*. Een functie die van het type *void* is, levert geen waarde op voor de aanroepende functie. In dat geval wordt de variant zonder expressie gebruikt. Alle andere functies leveren een waarde op, die als *return-waarde* wordt doorgegeven aan de aanroepende functie. Bij deze categorie van functies wordt de variant met expressie gebruikt. De expressie achter het *return*-statement is dan de return-waarde.

Elk **C** programma kent een functie *main()*. De functie *main()* is de functie waar de uitvoering van het programma begint. Deze functie kan eveneens een return-waarde hebben. De return-waarde van de functie *main()* wordt uiteindelijk teruggegeven aan het bedrijfssystem van de computer. In DOS is de return-waarde van *main()* de waarde van de ERRORLEVEL variabele. Een *return*-statement in een *main()* functie kan dus worden vervangen door een aanroep van de *exit()* functie, en omgekeerd.

In het volgende voorbeeld worden de functies *een(), twee()* en *leeg()* aangeroepen vanuit functie *main()* (Het definiëren van functies wordt behandeld in 5.2). De functies *een()* en *twee()* retourneren een waarde, de functie *leeg()* niet. De functie *main()* retourneert zelf de som van de waarden die door *een()* en *twee()* zijn geretourneerd aan het bedrijfssystem. Bestudering van het voorbeeld zal duidelijk maken dat de waarde die aan het bedrijfssystem wordt geretourneerd drie is:

```
void leeg()
{
    printf("De lege functie\n");
    return;

    printf("Dit wordt nooit vertoond\n");
}

int een()
{
    return (1);
}

int twee()
{
    return (2);
}

int main()                          // het programma begint hier
{
    leeg();
    return (een() + twee());
}
```

4.3.5 goto en labels

Het is mogelijk om binnen een functie een onvoorwaardelijke sprong naar een willekeurig statement uit te voeren. Een dergelijke sprong naar een andere plaats in de code van dezelfde functie wordt uitgevoerd met behulp van het *goto*-statement. Die andere plaats wordt aangegeven met een symbolische label naam gevolgd door een dubbele punt.

Er zijn in **C** echter maar weinig toepassingen voor de *goto,* aangezien er een uitgebreide verzameling syntactische constructies is om de flow control van een programma mee vast te leggen. Een programmeur die in **C** gebruik maakt van het *goto*-statement toont daarmee in het algemeen eerder een brevet van onvermogen dan beheersing van de taal. Het probleem

met het gebruik van het *goto*-statement is dat het snel leidt tot onbegrijpelijke (*spaghetti*) code, of tot code waarvan de juistheid niet meer kan worden bewezen. Verdedigbare toepassingen van het *goto*-statement zijn die waarin de verwerking van statements moet stoppen, waarna een volgend statement kan worden verwerkt. Het gebruik van de *goto* is dan een soort *nooduitgang*. Een toepassing zou kunnen worden gevonden in een multipel geneste lus, waar in de diepte van de lus kan worden besloten dat de geneste lus niet verder hoeft te worden verwerkt. Een *goto*-statement kan dan in één klap de buitenste lus beëindigen. Een voorbeeld van zo'n toepassing[3] geven we hieronder in een *code-frame*. Een code-frame bevat *pseudo-code:* in een code-frame staat informatie die op **C** code lijkt, maar geen **C** code is. Met behulp van een code-frame maken we duidelijk wat de essentie van een complex probleem of algoritme is, zonder alle details te hoeven invullen. Code-frames worden door ons vaak gebruikt als tussenstap tussen een verbale probleemstelling en de uiteindelijk implementatie in **C**:

```
...
while (conditie)
    for (init; conditie; increment)
        while (conditie)
        {
            if (conditie)
                goto beyond_loops;
        }

beyond_loops:
...
```

3. De in het voorbeeld gegeven situatie zou overigens ook goed zonder het *goto*-statement kunnen worden geprogrammeerd, namelijk door gebruik te maken van een extra functie waarin de driedubbel geneste lus is opgenomen, welke functie dan gewoon met een *return*-statement kan stoppen.

Hoofdstuk 5

Functies

5.1 Inleiding

In hoofdstuk 1 is opgemerkt dat programma's bestaan uit gegevens (data), die door instructies (code) worden bewerkt. In de vorige hoofdstukken zijn statements besproken waarmee bewerkingen worden gedefinieerd. Informeel zijn daarbij op een aantal plaatsen functies gebruikt, zoals de functie *printf()*.

Bij de oplossing van een programmeerprobleem wordt het probleem opgedeeld in deelproblemen (gecompartimentaliseerd). De deelproblemen worden geïmplementeerd als *functies*. Deelproblemen worden daarmee door functies gedefinieerd. Wanneer er eenmaal een functie beschikbaar is voor de oplossing van een deelprobleem heeft de programmeur een stukje gereedschap (een 'tool') in handen, dat in nieuwe programma's wederom kan worden gebruikt. Aho, Hopcroft, en Ullman (p.10) geven het voorbeeld van een functie waarmee matrixen met elkaar kunnen worden vermenigvuldigd. Met een dergelijke functie wordt de taal in feite uitgebreid met een nieuwe *operator* voor het vermenigvuldigen van matrixen. De programmeur die weet *wat* een functie doet, hoeft zich niet meer bezig te houden met de vraag *hoe* een functie werkt, maar kan de functie gewoon toepassen (aanroepen) waar dat nodig is.

Dijkstra (1972) beschrijft de manier waarop programma's worden ontwikkeld. De beschreven methode staat bekend als het gestructureerd programmeren. Bij het gestructureerd programmeren wordt een probleem stapsgewijs ontleed (stapsgewijze *decompositie),* totdat het probleem in dusdanig kleine deelproblemen is opgedeeld dat elk deelprobleempje met een paar statements kan worden geprogrammeerd. Bij de implementatie van een programma worden deelproblemen functies en worden complexe problemen opgelost door functies aan te roepen waarmee deelproblemen worden opgelost. In deze opzet bestaat het programma zelf uit de aanroep van slechts een paar functies. Er ontstaat een hiërarchie van functies: de werking van functies wordt gedefinieerd met behulp van andere functies, totdat (in de diepte) functies worden aangetroffen waarin slechts een paar statements staan.

Voordat we nader ingaan op de decompositie van problemen, worden in dit hoofdstuk eerst (in 5.2) de syntactische kenmerken van functies behandeld. Verderop (in 5.3) wordt een uitgebreid voorbeeld gegeven van de constructie van een programma.

5.2 Definiëren van functies

De syntactische definitie van een functie is ingewikkeld (zie bijlage A). In dit hoofdstuk en in de praktijk wordt de volgende informele syntactische definitie gebruikt:

```
[[return-type]] identifier ( [[parameter-lijst]] )
{
    [[variabelen definities]]

    [[statements]]
}
```

In deze syntactische definitie worden de volgende componenten onderscheiden:

- Het return-type heeft te maken met de *waarde* die door een functie wordt opgeleverd. Net zoals bij de definitie van variabelen is ook bij functies het default type de *int*. Wanneer het return-type niet wordt gespecificeerd, neemt de vertaler dus aan dat *int* wordt bedoeld. Wanneer géén waarde door een functie moet worden opgeleverd[1], moet het return-type *void* worden gebruikt.
- De identifier is de *naam* van de functie. De naam van de functie wordt gebruikt om de functie te kunnen activeren (aanroepen), zoals in

  puts("Een puts is een soort emmer");
- De open- en sluithaak achter de functienaam zijn verplicht.
- Tussen de open- en sluithaak mag de *parameterlijst* staan. De parameterlijst wordt gebruikt om het aantal en type van de parameters van de functie aan te geven. Een parameter van een functie is een symbolische aanduiding voor een instelwaarde van een functie. Wanneer een functie bijvoorbeeld twee gehele getallen moet optellen, dan krijgt zo'n functie twee *int* parameters, die bij de aanroep van de functie op een bepaalde waarde worden ingesteld (zie ook het voorbeeld van de functie *add()* hieronder). Als een functie geen instelwaarden heeft dan ontbreekt de parameterlijst. Merk op dat de haakjes dan nog steeds verplicht zijn. In ANSI-**C** worden de parameters in een parameterlijst voorzien van hun type. In standaard-**C** worden de parameters gedefinieerd door de namen van de parameters in de parameterlijst te noemen, terwijl de types van de parameters worden gedefinieerd *achter* de sluithaak en *voor* de openaccolade op de wijze waarop variabelen worden gedefinieerd. Een voorbeeld van de definitie van een functie met parameters in ANSI-**C** en standaard-**C** is:

1. vergelijk de *procedure* bij **pascal**

```
// functiedefinitie in ANSI-C:

void main(int argc, char *argv[])
{
    ...
}

// functiedefinitie in standaard-C:

void main(argc, argv)
int
    argc;
char
    *argv[];
{
    ...
}
```

Zie ook bijlage B voor een overzicht van de verschillen tussen ANSI-C en standaard-C.

- De accolades na de haken achter de functienaam zijn verplicht[2].
- Na de openaccolade kunnen lokale variabelen worden gedefinieerd. Zoals in hoofdstuk 2 is aangegeven zijn dit *static* of *auto* variabelen. Lokale *auto* variabelen worden op de stack aangemaakt. Het aanmaken van niet-statische lokale variabelen moet elke keer dat een functie wordt aangeroepen opnieuw worden gedaan. Het kost praktisch geen tijd om variabelen aan te maken[3], en het aanmaken van variabelen kost eenzelfde hoeveelheid tijd ongeacht het aantal variabelen dat wordt aangemaakt. Bij de ontwikkeling van functies moeten lokale variabelen dan ook niet voor meer dan één doel worden gebruikt. Functies worden leesbaarder zonder trager te worden wanneer lokale variabelen voor slechts één doel worden gebruikt.
- De feitelijke acties die door de functie worden uitgevoerd, worden na de definitie van de lokale variabelen door statements gedefinieerd. De Statements zijn optioneel. Een functie zonder statements doet niets.

Als voorbeeld van een functie de functie *add()* die twee parameters heeft van het type *int*. De functie *add()* telt de beide parameterwaarden op en retourneert hun som aan de aanroepende functie.

```
int add(int left, int right)
{
    return (left + right);
}
```

2. De accolades bij een functie zijn afkomstig van een compound-statement. De formele syntactische regel is dan ook dat achter de haken een compound-statement moet volgen.
3. Bij de Microsoft vertaler wordt ruimte voor lokale variabelen aangemaakt met behulp van één aftrekbewerking op een register. Het aantal lokale variabelen beïnvloedt de grootte van de af te trekken waarde, niet de snelheid van de aftrekbewerking.

Opgave 24.

1. Schrijf de (syntactisch) kortst mogelijke functie die zonder fouten of waarschuwingen kan worden vertaald. Kies voor de naam van de functie *a*.
2. Wat doet de functie *a()*?

5.2.1 Parameters van functies

Bij de meeste **C** programma's is er een intensieve communicatie tussen functies. Wanneer functies elkaar aanroepen, geeft de aanroepende functie meestal informatie door aan de aangeroepen functie, terwijl de aangeroepen functie in het algemeen ook informatie teruggeeft aan de aanroepende functie. Uiteraard kunnen functies die elkaar aanroepen beide gebruik maken van globale variabelen, om zo met elkaar te communiceren. Het zou echter lastig worden wanneer communicatie alleen via globale variabelen plaats zou kunnen vinden[4]. We geven een paar redenen waarom dat lastig zou zijn:

- Statische variabelen zouden van de communicatie worden uitgesloten wanneer de functies niet in hetzelfde bestand als de statische variabelen zijn gedefinieerd.
- Lokale variabelen van functies zouden van de communicatie zijn uitgesloten.
- Functies die elkaar (of zichzelf) aanroepen zouden dezelfde globale variabelen gebruiken. De vereiste *boekhouding* om dan te voorkomen dat functies elkaars globale variabelen per ongeluk wijzigen is erg complex.

Om deze redenen vindt de communicatie tussen functies bij voorkeur plaats via de stack: de aanroepende functie geeft de aangeroepen functie argumenten waarmee de parameters van de aangeroepen functie een waarde krijgen. De argumenten voor een functie worden op de stack geplaatst, en de aangeroepen functie verwacht dan ook de waarden voor de parameters op de stack. Deze overdracht van argumenten verloopt automatisch. De programmeur kan een functie met argumenten aanroepen en de feitelijke overdracht van de argumenten wordt geregeld door code die door de vertaler wordt gegenereerd. Wanneer een functie meerdere parameters heeft (zoals de eerder genoemde functie *add()*), dan correspondeert de volgorde van de argumenten met de volgorde van de parameters. Bij de aanroep van de functie *add()* kunnen we de getallen 5 en 8, of 13 en 15 bij elkaar optellen zoals is weergegeven in het volgende voorbeeld.

```
// twee manieren om add() aan te roepen:

result = add(5, 8);                    // bereken 5 + 8
result = add(13, 15);                  // bereken 13 + 15
```

4. Sommige talen kennen alleen globale variabelen, zoals **basic**.

Argumenten voor **C** functies zijn altijd by value. Dat betekent dat een argument voor een functie altijd een *kopie* is van de originele waarde. Dat betekent concreet dat een functie die wordt aangeroepen met een variabele als argument niet de waarde van die variabele kan wijzigen. De functie krijgt namelijk de *waarde* van de variabele, en niet de variabele *zelf* als argument. In listing 7 is een functie *power()* gedefinieerd met twee *int* argumenten, en een functie *main()* waarbinnen *power()* wordt aangeroepen. Het eerste argument voor *power()* is een getal, het tweede argument is de lokale variabele *value* (zie regel 21). Terwijl binnenin de functie *power()* met de parameter *n* wordt gerekend (op regel 7), verandert de lokale variabele *value* niet, omdat bij de aanroep van *power()* een *kopie* van de waarde van *value* wordt doorgegeven. De functie *power()* retourneert (zie 5.2.3) de waarde $base^n$ als *int*.

```
1   // bereken 'base' tot de macht 'n', n >= 0
2   int power (int base, int n)
3   {
4       int
5           p;
6
7       for (p = 1; n > 0; n--)          // 'n' verandert
8           p *= base;
9
10      return (p);
11  }
12
13  void main()
14  {
15      int
16          result,
17          value;
18
19      value = 3;
20
21      result = power(5, value);        // power() retourneert 125
22
23      // 'value' is nog steeds 5.
24  }
```

Listing 7: Het gebruik van variabelen als argument.

Uiteraard is het wel mógelijk om in een functie een variabele, die ergens buiten die functie als globale, statische, of lokale variabele is gedefinieerd, te wijzigen. De methode om dat te doen wordt behandeld in hoofdstuk 8. Voorlopig kunnen parameters het best worden beschouwd als lokale variabelen van een functie, die alvast door de aanroepende functie zijn geïnitialiseerd.

Parameters hoeven niet noodzakelijk enkelvoudige variabelen te zijn, maar kunnen ook arrays zijn. Wanneer een parameter een array is, wordt

door de aanroepende functie de beginlocatie van een array als argument aan de functie doorgegeven. Merk op dat de call by value hier betrekking heeft op de beginlocatie van het array, en niet op de array-elementen zelf. Een functie die een arrayparameter heeft kan dus wèl de waarden van de array-elementen wijzigen van een array dat als argument wordt doorgegeven. De naam van een array kan worden gebruikt om de beginlocatie van dat array door te geven aan een functie die een array als parameter heeft. In listing 8 wordt een voorbeeld gegeven van een van een functie met een arrayparameter (regels 2 tot regel 7). Merk op dat het zesde (en niet het vijfde) element van het in regel 15 doorgegeven array *local* door de functie *delta_5()* wordt gewijzigd (in regel 4).

```
1           // Definitie:
2           int delta_5 (int array[])
3           {
4               array[5]++;                         // wijzig het zesde element
5           }                                       // van het array
6
7           // Aanroep:
8           void fun()
9           {
10              int
11                  local[8];                       // array van 8 ints
12
13              local[5] = 0;
14              delta_5 (local);
15              // local[5] is nu 1
16          }
17
```

Listing 8: Een array als parameter.

Een tweede voorbeeld van een functie waaraan een array wordt doorgegeven, is de standaardfunctie *gets()* die een array van *chars* vult met informatie, die als een string via het toetsenbord wordt ingelezen. In onderstaand voorbeeld wordt zo de beginpositie van het array *buf* aan *gets()* doorgegeven. Doordat de beginpositie van *buf* wordt doorgegeven, kan *gets()* de elementen van het array *buf* van waarden voorzien.

```
char
    buf[80];

printf("Enter your name:" );
gets(buf);
```

5.2.2 Declareren van functies: prototyping

Functies kunnen al worden aangeroepen zonder dat de vertaler de code van zo'n functie gezien heeft. Als voorbeeld kan de listing van het 'Hello World' programma op pagina 34 gelden: een functie *printf()* wordt aangeroepen

zonder dat de vertaler weet wat die functie doet. In dergelijke gevallen moet de vertaler op de hoogte worden gesteld van het bestaan van de functie door middel van een *functieprototype*.

Een functieprototype bestaat uit de definitie van de functie. Het compound statement is bij een prototype echter vervangen door een puntkomma. Het prototype van de functie *add()* in 5.2 is dus

```
int add(int left, int right);
```

De *namen* van de parameters zijn bij functieprototypes onbelangrijk, en mogen ook worden weggelaten. Een equivalent prototype voor de functie *add()* is dus

```
int add(int, int);
```

Een functie zonder parameters heeft een lege parameterlijst. In het functieprototype wordt dat aangegeven door in de parameterlijst het keyword *void* te plaatsen. Als de functie *noparams()* zonder parameters is gedefinieerd, en een *int* oplevert, dan is het prototype van de functie *noparams* dus

```
int noparams(void);
```

Het prototype van de functie is tegelijk ook de declaratie van de functie: de functie is nog niet gedefinieerd, maar wordt al wel bekend gemaakt aan de vertaler. Vóór het functieprototype kan nog het keyword *extern* worden geplaatst, zoals bij de declaratie van variabelen.

Wanneer meer dan één functie van hetzelfde type (zie 5.2.3) wordt gedeclareerd, dan kan ook een lijst van declaraties worden gebruikt. Functieprototypes worden dan van elkaar gescheiden door komma's, en het laatste prototype wordt afgesloten door een puntkomma. Een voorbeeld waarin vier functies worden gedeclareerd, geven we hieronder. Het keyword *extern* zou kunnen worden weggelaten, en het keyword *int* ook, maar we raden aan beide keywords wèl te gebruiken.

```
extern int
    add(int, int),
    divide(int, int),
    multiply(int, int),
    subtract(int, int);
```

Opgave 25.

1. Waarom mag in het bovenstaande voorbeeld het keyword *int* na *extern* worden weggelaten?

2. Waarom mag in het bovenstaande voorbeeld het keyword *extern* worden weggelaten?

Functieprototypes in standaard-C. In standaard-**C** is het specificeren van parametertypes bij functiedeclaraties *niet* toegestaan. In standaard-**C** kan bovengenoemde functie *add()* worden gedeclareerd als

extern int add();

Aangezien elke ANSI-**C** vertaler ook een standaard-**C** vertaler is, is het bij ANSI-**C** vertalers toegestaan de typespecificaties van parameters bij functiedeclaraties weg te laten. We raden echter aan de typespecificaties van parameters bij functieprototypes altijd op te geven, om de vertaler zo in staat te stellen mogelijke fouten bij de overdracht van argumenten aan functies op te sporen en te vermelden.

5.2.3 Het functietype: de return-waarde

Een functie levert normaliter een waarde op die wordt teruggegeven aan de aanroepende functie. Bij het *return*-statement is al behandeld hoe de waarde door een functie aan de aanroepende functie wordt teruggegeven. De soort waarde die wordt teruggegeven bepaalt het *type* van een functie. Wanneer een functie een *int* retourneert is de functie van het type *int,* wanneer een *double* wordt geretourneerd is de functie van het type *double*. Hoewel het niet noodzakelijk is dat functies een return-waarde opleveren, wordt wèl aangeraden om functies een return-waarde op te laten leveren. Via de return-waarde kan de aanroepende functie bijvoorbeeld controleren hoe de aangeroepen functie heeft gefunctioneerd. Een constructie zoals in het volgende voorbeeld, waar de return-waarde van de functie *malloc()* als zo'n controlemogelijkheid wordt gebruikt is gemeengoed voor **C** programmeurs[5]:

```
if (!(mem = malloc(1000)))
{
    puts("No more memory");
    exit(1);
}
```

Desalniettemin, functies kunnen ook geen return-waarde opleveren. Dergelijke functies zijn van het type *void*.

Als voorbeeld geven we drie functies, die respectievelijk van het type *int, double,* en *void* zijn.

```
// een functie van het type int:
int intfun()
{
    return (4);
}

// een functie van het type double:
double doublefun()
{
    return (4.0);
```

5. Aangenomen is dat de variabele *mem* elders is gedefinieerd.

```
}

// een functie van het type void:
void voidfun()
{
    return;
}
```

Opgave 26.

1. Maak het prototype van een functie *proto_1()* die twee *int* parameters heeft en die van het type *char* is.
2. Maak het prototype van een functie *proto_2()* die een *int* array en een *char* array als parameters heeft en die van het type *float* is.
3. Maak het prototype van een functie *proto_3()* die een *char* array als parameter heeft en die geen returnwaarde oplevert.

5.2.4 Functies met variabel aantal argumenten

Functies kunnen in **C** zodanig worden geprogrammeerd dat ze een variabel aantal argumenten kunnen krijgen bij hun aanroep. Een voorbeeld van een dergelijke functie is de functie *printf(),* die bij de aanroep na de format string een variabel aantal argumenten kan krijgen. In hoofdstuk 1 is al opgemerkt dat in **C** functies zoals *printf()* geen deel van de taal uitmaken[6], maar moeten worden beschouwd als een toepassing van de taal. Hier zullen we de methode behandelen om functies, die een variabel aantal argumenten kunnen verwerken, te ontwikkelen.

Ter illustratie zullen we hier een functie *max()* ontwikkelen, waarmee we het maximum kunnen bepalen van een (ongespecificeerd) aantal *int* waarden. De functie moet met de volgende argumenten worden aangeroepen:

- Het eerste argument is de eerste *int* waarde van de serie.
- De overige argumenten zijn de overige waarden van de serie waarvan het maximum moet worden bepaald. Omdat het aantal waarden niet bekend is, spreken we af dat het einde van de serie is bereikt wanneer twee opeenvolgende gelijke waarden worden aangetroffen.

Een paar voorbeelden van de aanroep van de functie *max()* zijn:

```
maxint     = (int)max(-3, 5, 8, -2, 5, 5);
maxint     = (int)max(-3, -3);
maxint     = (int)max(-3, 4, 5, 4, 3, 0, 0);
```

De functie *max()* is in listing 9 gedefinieerd.

```
1           int max(char int1, ...)
2           {
```

6. Merk op dat in **pascal** geen procedures met variabele aantallen argumenten kunnen worden geschreven. Procedures waarbij dat wel lijkt te kunnen (zoals de procedure *read()*) zijn een onderdeel van de taal. Dergelijke procedures worden door de **pascal** vertaler als een soort compound statement behandeld.

```
3               va_list
4                   args;
5               int
6                   maxint,
7                   int2;
8
9               va_start(args, int1);
10              maxint = int1;
11              while (1)
12              {
13                  int2 = va_arg(args, int);
14                  if (int1 == int2)
15                  {
16                      va_end(args);
17                      return (maxint);
18                  }
19                  int1 = int2;
20                  if (maxint < int1)
21                      maxint = int1;
22              }
23          }
```

Listing 9: De functie *max()*.

Over deze functie merken we het volgende op:

- De laatste parameter bestaat uit drie opeenvolgende punten, (...) waarmee wordt aangegeven dat bij de aanroep meerdere argumenten kunnen volgen. Deze aanduiding komt ook terug in het prototype van de functie *max()* die hieronder is gegeven.

  int max(int, ...);

- De eerste lokale variabele (regel 3 en 4) is de variabele *args* van het type *va_list*. De constructie van nieuwe datatypes wordt behandeld in hoofdstuk 9. Voorlopig kan worden aangenomen dat het type *va_list* bestaat en wordt gebruikt bij het werken met variabele aantallen argumenten. De aanduiding *va_list*[7] betekent

  "Variable Argument LIST".

- Op regel 9 wordt de variabele *args* geïnitialiseerd. De initialisatie zorgt ervoor dat *args* de plaats van het eerste argument aanduidt, dat volgt op het genoemde argument *(type)*. Na de initialisatie wijst *args* dus op het eerste argument dat bij de aanroep van *max()* op de plaats van de drie puntjes staat.
- Op regel 13 wordt de functie *va_arg()*[8] gebruikt. De functie retourneert de waarde die door *args* wordt aangewezen, opgevat als een

7. Het type *va_list* is gedefinieerd in de header file `stdarg.h`. Het begrip header file wordt besproken in hoofstuk 6.
8. *va_list(), va_arg()* en *va_end()* zijn geïmplementeerd als macro's.

waarde van het type *int* (het type dat wordt gebruikt is het type dat als tweede argument van *va_arg()* wordt genoemd). Vervolgens wordt *args* door de functie alvast op het volgende argument geplaatst. Een volgende aanroep van *va_arg()* zal dus steeds de volgende argumentwaarde opleveren.

- De werking van de functie is gebaseerd op een initialisatie (regels 8 en 9), waarbij een eerste schatting van de maximumwaarde wordt gemaakt op grond van het eerste argument, en een verwerkingsfase (regels 11 tot 23). Bij de verwerkingsfase wordt steeds een nieuw argument vergeleken met het laatste argument en wordt de maximumwaarde zonodig aangepast.
- De functie wordt verlaten op regel 17. De verkregen maximumwaarde wordt geretourneerd. Voordat de functie wordt verlaten, wordt ter afsluiting van de verwerking van het variabele aantal argumenten *va_end()* aangeroepen.

Als vuistregel voor de constructie van functies met variabele aantallen argumenten kan voorlopig worden aangehouden, dat de variabele argumenten van een standaard datatype (met uitzondering van het type *char)* moeten zijn. Het type van de laatste parameter in een functie met variabele aantallen argumenten moet eveneens van een standaard datatype zijn, of kan een string zijn, zoals wordt gebruikt in de volgende opgave.

Opgave 27.

Construeer een functie *void error()* waarmee op het beeldscherm een foutmelding wordt afgedrukt, waarna het programma stopt met ERRORLEVEL 1. De functie moet ervoor zorgen dat de foutmelding altijd aan het begin van een nieuwe regel begint. De functie *error()* moet kunnen worden aangeroepen zoals *printf()*. (Aanwijzing: gebruik *putchar()* om de eerste nieuwe regel te forceren en *vprintf()* om de variabele aantallen argumenten te verwerken).

5.2.5 De functie main()

Elk **C** programma heeft een functie die *main()* heet. De functie *main()* is de hoofdfunctie van het programma. De functie *main()* wordt uitgevoerd wanneer een programma wordt gestart en na afloop van de functie *main()* is het programma beëindigd. Wanneer de functie *main()* een return-waarde heeft, dan wordt de als *char* opgevatte return-waarde doorgegeven aan het bedrijfssysteem. Voorbeelden van het gebruik van de functie *main()* zijn al op verschillende plaatsen gegeven, zoals in de listings 1 en 2.

De parameters argc, argv en envp

De functie *main()* krijgt bij aanroep door het **C** run-time systeem drie argumenten mee. De functie *main()* kan daarom gedefinieerd worden met drie parameters, die gewoonlijk *argc, argv,* en *envp* worden genoemd. De parameters van de functie *main()* zijn als volgt gedefinieerd:

int argc. De variabele *argc* is de argument counter. Bij het begin van het programma is *argc* gelijk aan het *aantal* argumenten dat werd gegeven bij de aanroep van het programma, waarbij de aanroep van het programma zelf wordt meegetelt. De waarde van *argc* is dus minstens 1.

char *argv[]. De variabele *argv* is een array van strings[9]. Element 0 van *argv* bevat de naam van het programma (bij het DOS bedrijfssysteem is dat de volledige padspecificatie van het programma). Element 1 van *argv* bevat (als string en indien aanwezig) het eerste argument van het programma. Het laatste argument van het programma staat in element *argv[argc - 1]*.

char *envp[]. De variabele *envp* is eveneens een array van strings en bevat de namen en definities van de 'environment' variabelen. De environment-variabelen worden bij de DOS en UNIX bedrijfssystemen gedefinieerd met behulp van de opdracht *set*. Na de definitie van de laatste environment-variabele volgt nog een element met de waarde *NULL*.

Niet alle parameters van de functie *main()* worden altijd gebruikt. De parameter *envp* wordt in de praktijk het minst gebruikt. Bij de definitie van de functie *main()* mogen één of meerdere parameters worden weggelaten. Vier mogelijke definities van de functie *main()* zijn:

```
void main()
{
    // parameters worden niet gebruikt
}

void main(int argc)
{
    // alleen argc wordt gebruikt
}

void main(int argc, char *argv[])
{
    // de meest voorkomende parameters:
    // envp wordt niet gebruikt
}

void main(int argc, char *argv[], char *envp[])
{
    // alle parameters worden gebruikt
}
```

Wanneer onder het DOS bedrijfssysteem de environment-variabelen

```
COMSPEC=C:\DOS\COMMAND.COM
PATH=C:\C\BIN;C:\DOS
```

9. De asterisk voor de naam *argv* dient hier te worden gelezen als pointer. *argv* is een array van zogenaamde character pointers. Pointers worden behandeld in hoofdstuk 8. Voorlopig kan worden aangenomen dat een character pointer een string is. De elementen van *argv* zijn dus strings.

```
INCLUDE=C:\C\INC
LIB=C:\C\LIB
```

zijn gedefinieerd en een programma TEST.EXE (dat is opgeslagen in de directory C:\C\BIN) wordt gestart als

```
test -n -l test.c test.lst
```

dan hebben *argc, argv* en *envp* de volgende waarden:

- De variabele *argc* is 5.
- De variabele *argv* bevat de volgende 5 string-elementen:
  - *argv[0]*: *"C:\C\BIN\TEST.EXE"*
  - *argv[1]*: *"-n"*
  - *argv[2]*: *"-l"*
  - *argv[3]*: *"test.c"*
  - *argv[4]*: *"test.lst"*
- De variabele *envp* bevat de volgende string-elementen:
  - *envp[0]*: *"COMSPEC=C:\DOS\COMMAND.COM"*
  - *envp[1]*: *"PATH=C:\C\BIN;C:\DOS"*
  - *envp[2]*: *"INCLUDE=C:\C\INC"*
  - *envp[3]*: *"LIB=C:\C\LIB"*
  - *envp[4]* is *NULL*

Wild card-expansie

Veel programma's krijgen als argument de naam van een bestand dat door het programma moet worden verwerkt. Een voorbeeld van zo'n programma is de tekst*editor,* een programma om tekstbestanden mee te maken. Een ander voorbeeld is de **C** vertaler, een programma om **C** bronteksten om te zetten naar object modules. Een programma waarmee één bestand kan worden verwerkt is al handig, maar nog handiger is het wanneer zo'n programma met één opdracht meerdere bestanden kan gaan verwerken. Wanneer bij de aanroep van zo'n programma gebruik kan worden gemaakt van wildcard constructies, zoals die ook bij de DOS opdracht dir zijn toegestaan, dan wordt het de gebruiker extra gemakkelijk gemaakt om een programma de opdracht te geven meerdere bestanden te verwerken.

Het is in **C** niet moeilijk om een programma te schrijven dat meerdere bestanden verwerkt. Het prototype van zo'n programma bestaat uit een lus waarin steeds een nieuw bestand wordt geopend, om vervolgens te worden verwerkt. De lus wordt beëindigd wanneer er geen bestanden meer zijn om te openen. Een code-frame van zo'n programma is:

```
void main(int argc, char argv[][])
{
    while (er_is_een_nieuw_bestand_beschikbaar())
        verwerk_het_bestand();
```

```
}
```

Het prototype van het programma bevat alle essentiële kenmerken van het uiteindelijke programma. Met de twee functies *er_is_...()* en *verwerk_...()* wordt gesuggereerd wat wordt bedoeld, zonder dat precies is vastgelegd wat de functies doen.

Met behulp van de parameter **argv[]* kunnen de argumenten worden bereikt die bij de aanroep van het programma werden gegeven. Wanneer een programma wordt geschreven zoals gesuggereerd door het prototype en het programma zou zijn gestart als

```
test -f *.c
```

dan zou *argv[1]* de string *-f"* zijn en *argv[2]* de string *"*.c"*. De wildcard constructie `*.c` verschijnt als letterlijk argument in het programma en kan daar verder worden verwerkt.

In plaats van eigen code de wildcard expansie te laten verzorgen, is het ook mogelijk de expansie door het **C** run-time systeem te laten uitvoeren, zodat de elementen van *argv* al de geëxpandeerde wildcard bestandsnamen bevatten bij het begin van de functie *main()*.

Om de wildcard expansie te laten verzorgen door het **C** run-time systeem moet bij het *linken* van het programma het bestand `setargv.obj` worden meegelinkt. Dit bestand is met de bibliotheken van de Microsoft vertaler meegeleverd. Wanneer gebruik wordt gemaakt van de batchfile `l.bat` dan wordt de opdracht om een programma `test.obj` te linken

```
l test+\c\lib\setargv /noe
```
[10]

De vlag `/noe` is aan de linker meegegeven om te voorkomen dat de linker gaat klagen over het feit dat `setargv.obj` twee keer bekend is: `setargv.obj` bevindt zich namelijk ook in de standaard **C** bibliotheek. Het bestand dat expliciet bij de link opdracht is meegegeven, krijgt de prioriteit en zorgt ervoor dat de wildcard argumenten, waar mogelijk, worden geëxpandeerd naar bestandsnamen. Dat betekent dat in een **C** programma *argv* de geëxpandeerde namen van argumenten bevat. Wanneer in een directory de bestanden `one.c` en `two.c` staan en wanneer het programma met `*.c` als enig argument wordt gestart, dan wordt *argv[1]* de string *öne.c"* terwijl *argv[2]* de string *"two.c"* wordt.

Opgave 28.

Schrijf een programma `wild.c` aan de hand van het volgende code-frame dat alle strings van het array *argv* op aparte regels op het beeldscherm laat zien. Link het programma zodanig dat wildcard argumenten door het `C` run-time systeem worden geëxpandeerd.

```
void main(int argc, char *argv[])
{
```

10. Verondersteld wordt dat het bestand `setargv.obj` in de directory `\c\lib` is geplaatst.

```
        int
            index;

        for (alle argumenten)
            toon de argumentstring
    }
```

5.3 Functies en programma-opbouw

Nu de functies en statements zijn behandeld, wordt het tijd om de ontwikkeling van een programma eens onder de loep te nemen. Als voorbeeld nemen we het volgende probleem:

> *"Schrijf een programma dat van opgegeven bestanden (wildcards zijn toegestaan) per bestand een overzicht afdrukt (met bijbehorende bestandsnaam) van het aantal printable tekens. Tekens die niet in het bestand voorkomen hoeven niet in het overzicht te worden vermeld. Het programma levert als return-waarde het aantal argumenten dat niet als bestand kon worden verwerkt."*

Bij de ontwikkeling van een programma om bovengenoemd probleem mee te kunnen oplossen, zullen we een aantal keren het probleem *herformuleren*. Bij het herformuleren van het probleem proberen we steeds dichter bij het uiteindelijke programma te komen. Elke herformulering wordt genummerd en kan één of meerdere beslissingen met zich meebrengen. We proberen echter, in overeenstemming met een principe dat bekend staat als het principe van de *"latest possible binding"* [11] beslissingen zo laat mogelijk te nemen, zodat de kans zo klein mogelijk is dat een eenmaal genomen beslissing ons bij latere herformuleringen beperkt. Of, zoals Knuth ooit eens opmerkte:

> *"Premature optimization is the root of all evil..."*.

Bij de eerste herschrijving van het probleem wordt het probleem zo veel mogelijk in de gebiedende wijs geformuleerd:

1. Behandel alle argumenten van het programma.
2. Probeer elk argument als een bestand te openen.
3. Als het bestand kan worden geopend,
   4. vermeld dan de naam van het bestand.
   5. verwerk het bestand.
   6. sluit het bestand.

   Als het bestand niet kan worden geopend, verhoog dan de telling van het aantal argumenten dat niet als bestand kon worden geopend.
7. Retourneer tenslotte het aantal keren dat een argument niet als bestand kon worden geopend.

11. zie Dijkstra (1972).

De eerste herschrijving kan in een code-frame worden geplaatst. Vooral onderdeel 5. is op dit moment nog te lastig om te definiërenen en de beste beslissing die we kunnen nemen is het onderdeel als een nader te specificeren functie op te nemen. In hoeverre het code-frame dan nog een 'frame' is en in hoeverre er al sprake is van 'code', is in zekere zin een kwestie van smaak. Listing 10 is de allereerste herschrijving van het oorspronkelijke probleem naar een brontekst van een programma.

Wanneer volgens punt 1 van de herschrijving alle argumenten moeten worden afgelopen, dan kan dat met een *for*-statement, waarbij een index van 1 tot *argc* loopt. Indexwaarde 0 kan worden overgeslagen, omdat *argv[0]* immers de naam van het programma zelf is. Voor de afhandeling van de *for* lus is dus een index variabele nodig. Deze variabele maken we lokaal omdat de *for* lus zelf lokaal in de functie *main()* is. Daarnaast moet worden geteld hoeveel bestanden niet kunnen worden verwerkt. Dit aantal tellen we in een lokale variabele *error*. Op regels 6 en 7 wordt een variabele *file* gedefinieerd, die verderop in het programma nog een paar keer wordt gebruikt. De variabele *file* is een file pointer[12] (zie listing 10).

```
1     int main(int argc, char *argv[])
2     {
3         int
4             index,
5             error = 0;
6         FILE
7             *file;
8
9         for (index = 1; index < argc; index++)// behandel argumenten
10        {
11            if (file = fopen(argv[index], "r"))// open een bestand
12            {                                  // het lukt:
13                printf("%s:\n", argv[index]); // vermeld de naam
14                verwerk(file);                // verwerk een bestand
15                fclose(file);                 // sluit het na afloop
16            }
17            else                               // het mislukt:
18                error++;                       // tel het aantal mislukkingen
19        }
```

12. Het gebruik van de variabele *file* loopt vooruit op de behandeling van bestanden (in hoofdstuk 7).
Een geopend bestand wordt bereikt via een zogenaamde file pointer. De meeste **C** functies die met bestanden (files) werken, verwachten een file pointer als argument, of retourneren een file pointer.
De standaard functie *fopen()* wordt gebruikt om een file te openen. De functie verwacht twee argumenten. Het eerste argument is een string waarin de naam van het programma staat, het tweede argument is (voorlopig) "r".
De standaard functie *fclose()* sluit een geopende file. De functie verwacht een file pointer als argument.
De standaard functie *fgetc()* retourneert het eerstvolgende teken van een geopende file. De functie verwacht een file pointer als argument.

```
20        return (error);                    // retour: aantal mislukkingen
21    }
```

Listing 10: Het hoofdprogramma: herformulering 1.

In listing 10 zien we ook al het gebruik van de standaard functies *fopen()* en *fclose()* om een bestand respectievelijk te openen en te sluiten. Daarmee is ook punt 2 van de eerste herformulering van het probleem gedefinieerd. Punt 3 valt uiteen in een conditie, waarbij in het ene geval een drietal subtaken moet worden uitgevoerd, en in het ander geval een teller moet worden opgehoogd. De conditie is vertaald naar een *if*-statement, terwijl de drie subtaken in een compound statement zijn samengevoegd. We hebben al de beschikking over gereedschap voor het uitvoeren van subtaken 4. en 5.: de functie *printf()* kan worden gebruikt om de naam van het geopende bestand te laten zien, de functie *fclose()* wordt gebruikt om het verwerkte bestand weer te sluiten.

Wat overblijft is de specificatie van punt 5.. Punt 5. is in listing 10 terug te vinden als de aanroep van de functie *verwerk()*. De tweede herformulering richt zich op deze functie.

Het verwerken van een bestand begint bij het eerste teken en stopt bij het laatste teken. Bij elk teken dat wordt gelezen, wordt de variabele opgehoogd waarin het aantal keren dat dat teken is gelezen is opgeslagen. Tenslotte, nadat het bestand volledig is gelezen, wordt een overzicht gegeven van het aantal en soort printable tekens dat in het bestand voorkwam. De tweede herformulering luidt dan:

5.1. Initialiseer de tellers.
5.2. Zolang er tekens zijn gelezen
    5.3. Verhoog de bijbehorende teller.
5.4. Geef een overzicht van de printable tekens.

In listing 11 is het code-frame van de functie *verwerk()* volgens de tweede specificatie gegeven.

```
1         void verwerk(FILE *f)
2         {
3             int
4                 ch;
5
6             // initialiseer de tellers
7
8             while ((ch = fgetc(f)) != EOF)        // zolang er tekens zijn
9                 // verhoog de teller voor teken 'ch'
10                ;
11
12            resultaten();
13        }
```

Listing 11: De functie *verwerk():* herformulering 2.

De functie *verwerk()* heeft één parameter: de *FILE* pointer *f*. Omdat we meerdere bestanden moeten kunnen verwerken, waarbij telkens opnieuw moet worden geteld hoeveel tekens van een bepaalde soort zijn gelezen, moeten de tellers op nul worden gezet. We stellen de beslissing over de tellers echter nog even uit. Punt 5.2 bevat een *"zolang ..."* zin, die vertaald is naar een *while*-statement. Via de standaard functie *fgetc()* worden tekens van het huidige bestand gelezen, totdat de waarde *EOF* aan het einde van het bestand wordt geretourneerd. Telkens wordt het juist gelezen teken bewaard in de lokale variabele *ch*. In punt 5.3 wordt deze variabele gebruikt om de bijbehorende teller op te hogen. De beslissing over hoe dat gebeurt, stellen we nog even uit. Tenslotte wordt in punt 5.4 de functie *resultaten()* aangeroepen om zo de printable tekens af te drukken.

Nu wordt het tijd om de definitie van de tellers nader te beschouwen. Hoewel een bestand groot kan zijn, kunnen er niet meer verschillende tekens in voorkomen dan er ASCII-tekens zijn. Omdat bij elk ASCII-teken een nummer hoort, lopend van 0 tot 256, kunnen we een array van tellers gebruiken, waarbij het ASCII-teken de *index* vormt in het array. Bijvoorbeeld, wanneer het teken `'a'` het eerstvolgende teken van het bestand is, dan wordt de *waarde* `'a'` (dat is de waarde 97 volgens de ASCII-tabel) door *fgetc()* geretourneerd. Op index-plaats 97 van het array tellen we dan het aantal keren dat de letter `'a'` is gelezen. Wanneer het array *count* wordt genoemd, is dat dus element *count[97]*. Omdat `'a'` de *waarde* 97 is, kan ook worden geschreven *count['a']*. Wanneer, tenslotte, de letter `'a'` is opgeslagen in de variabele *ch,* dan kan automatisch de juiste teller worden opgehoogd door de expressie

```
count[ch]++;
```

te gebruiken. Voor het array *count* reserveren we 256 *unsigned int* variabelen. We maken daarbij de veronderstelling dat elk teken ten hoogste 65535 (of: `0xffff`) keer voorkomt. Onderdeel 5.3 van de tweede herformulering is nu voldoende gespecificeerd. Het array wordt gedefinieerd als globaal array[13].

Onderdeel 5.1 van de tweede herformulering betreft het initialiseren van het array *count*. Uiteraard kan daarvoor weer een *for*-statement worden gebruikt. Het ligt echt meer in de stijl van **C** om te kijken, of er al niet gereedschap beschikbaar is om het array *count* te initialiseren. Dat gereedschap is beschikbaar in de vorm van de standaard functie *memset()* die drie argumenten verwacht:

- de naam van het array dat moet worden geïnitialiseerd,
- de byte-waarde waarmee moet worden geïnitialiseerd,
- het aantal bytes dat moet worden geïnitialiseerd.

13. In het algemeen is het verstandig om grotere datastructuren als globale variabele te definiëren, om zo de stack, die normaliter bij de Microsoft implementatie van **C** zo'n 1000 *ints* kan bevatten, niet al te zeer te belasten.

Wanneer we ons realiseren dat de *sizeof* operator de omvang van een array in bytes levert, is de initialisatiestap wel heel eenvoudig geworden. Om *count* te initialiseren op 0 gebruiken we

memset(count, 0, sizeof(count));

```
unsigned
    count[256];

void verwerk(FILE *f)
{
    int
        ch;

    memset(count, 0, sizeof(count));     // initialiseer count

    while ((ch = fgetc(f)) != EOF)       // zolang er tekens zijn
        count[ch]++;                     // verhoog de juiste teller

    resultaten();
}
```

Listing 12: De functie *verwerk():* definitieve versie.

In listing 12 geven we de definitieve versie van de functie *verwerk()*. Wat nog rest is de specificatie van de functie *resultaten()* om de printable tekens af te drukken. Printable tekens zijn ASCII-tekens vanaf ASCII-teken 32, de spatie. Het is echter algemener om de beslissing of een teken printable is over te laten aan gereedschap, waarmee dat kan worden bepaald. Verder besluiten we dat we de resultaten over zes kolommen afdrukken en dan over zullen gaan op een nieuwe regel. Het afdrukken van de resultaten kunnen we nu als volgt herformuleren:

5.4. "Geef een overzicht van de printable tekens." wordt:

5.4.1. Begin met een nieuwe (blanco) regel.
5.4.2. Test alle ASCII-tekens. Doe het volgende bij elk ASCII-teken dat in het bestand voorkwam en dat printable is:
    5.4.3. Druk de string *"c': ddddd "* af, waarbij `c` het printable teken is en *ddddd* het aantal keren dat teken `'c'` voorkwam.
    5.4.4. Wanneer zes tekens zijn afgedrukt, ga dan op een nieuwe regel over.
5.4.5. Eindig met een blanco regel.

In listing 13 is deze formulering naar code herschreven. Onderdeel 5.4.1 van de herformulering is met een *printf()* gerealiseerd. Onderdeel 5.4.2 is gerealiseerd met behulp van een *for* lus, die de elementen van het array

count inspecteert, in combinatie met de standaardfunctie *isprint()*[14] die 0 retourneert wanneer het aan de functie aangeboden teken geen printable teken is en die een andere waarde retourneert wanneer het teken wel een printable teken is.

Onderdeel 5.4.3 is weer met een *printf()* functie geïmplementeerd.

Bij de herschrijving van onderdeel 5.4.4 wordt gebruik gemaakt van een lokale teller *n_displayed*. Elke keer dat een teken wordt geprint wordt *n_displayed* opgehoogd. Wanneer *n_displayed* 6 is geworden, wordt overgegaan op een nieuwe regel en wordt *n_displayed* weer op 0 teruggezet.

Bij onderdeel 5.4.5 moet opnieuw naar *n_displayed* worden gekeken: wanneer *n_displayed* 0 is was net daarvoor een regel afgesloten. Zo niet, dan moet de huidige regel eerst worden afgesloten voordat de extra blanco regel kan worden geschreven. Onderdeel 5.4.5 leent zich goed voor het gebruik van de ternaire operator. Let tevens op het gebruik van de *%c* in de format string van *printf()* bij het afdrukken van de ASCII-tekens. Als argument bij *printf()* wordt een getal meegegeven, dat door de functie *printf()* op grond van de *%c* aanduiding wordt omgezet naar een ASCII-teken. Zo wordt de waarde 97 omgezet in letter `'a'`.

```
1   void resultaten()
2   {
3       int
4           index,
5           n_displayed = 0;
6
7       printf("\n");
8
9       for (index = 0; index < 256; index++)
10      {
11          if (count[index] && isprint(index))
12          {
13              printf(" '%c': %5u" , index, count[index]);
14              if (++n_displayed == 6)
15              {
16                  printf("\n");
17                  n_displayed = 0;
18              }
19          }
20      }
21
22      printf("%c\n", n_displayed ? '\n' : ' ');
23  }
```

Listing 13: De functie *resultaten():* herformulering 3.

14. *isprint()* is geïmplementeerd als een macro.

Opgave 29.

1. Stel het volledige programma samen tot één brontekst `ccount.c`. Definieer alle functies en het globale array *count* vóórdat ze voor het eerst worden gebruikt. Aan het begin van de brontekst moeten nog drie header files worden vermeld (zie ook hoofdstuk 6) voor de declaraties van de gebruikte standaard functies. Dit zijn:

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
```

2. Link het programma samen met het object moduul *setargv.obj* tot het programma `ccount.exe`.

3. Verander het programma zodanig dat individuele tekens een miljoen keer kunnen voorkomen. (Aanwijzing: gebruik de *printf()* aanroep om het aantal keren af te drukken dat een teken voorkwam *"%7ld"*.)

4. Verander het programma zodanig dat ook ASCII-tekens vanaf ASCII-waarde 128 worden afgedrukt.

5. In welke header files zijn de standaard functies *printf(), memset()* en *isprint()* gedeclareerd?

5.3.1 Het probleem van het toiletpapier

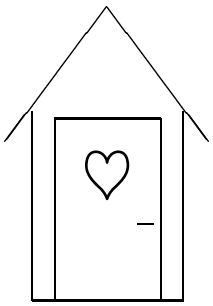
In programma's wordt niet altijd een bekende oplossing voor een probleem gebruikt om een probleem op te lossen. Omdat programma's snel werken, kunnen ze worden gebruikt om een proces te imiteren dat in de werkelijkheid weken, jaren, of zelfs eeuwen tijd zou kunnen kosten. Dergelijke programma's simuleren een situatie die zich in de werkelijkheid kan voordoen. Simulatieprogramma's worden gebruikt om inzicht te krijgen in de aard van problemen die moeilijk anders te krijgen is. Hier zullen we het volgende probleem met behulp van simulatie gaan aanpakken (zie ook figuur 5.1 [15]).

In een restaurant hangen in de wc *twee rollen papier. Er zijn twee soorten bezoekers van het restaurant die naar de* wc *gaan. De ene groep (die we de* kleingebruikers *noemen) kiezen altijd de kleinste rol papier en halen daar een klein aantal velletjes af voor eigen gebruik. De tweede groep (die we de* grootgebruikers *noemen) kiezen altijd de grootste rol papier en halen daar een groot aantal velletjes vanaf voor eigen gebruik. Of iemand die naar de* wc *gaat een groot- of klein-gebruiker is, is een kwestie van* kop of munt. *Evenzo, wanneer beide rollen evenveel velletjes bevatten (hetgeen op miraculeuze wijze direct voor alle bezoekers duidelijk is) wordt op grond van het toeval bepaald welke rol wordt gebruikt. Als er echter een verschil in omvang van de rollen is, neemt de grootgebruiker* altijd *de grootste rol en de kleingebruiker* altijd *de kleinste rol, ook al zou die rol minder dan het gewenste aantal velletjes bevatten.*

Zodra een rol leeg is, worden door de manager van het restaurant beide *rollen vervangen door nieuwe en worden de lege huls van de ene rol en de (bijna) lege andere rol weggegooid.*

Een kennis van de manager komt dit ter ore en vraagt zich af of er zo niet wat veel papier wordt verspild. Omdat het wachten op lege rollen wat lang zou duren, wordt een ***C*** *programma gemaakt om het proces te simuleren.*

15. Met dank aan de heer L. C. Smit Duyzentkunst, beheerder van het kerkje te Oostum, voor het motief van figuur 5.1, gebaseerd op het toilet achter het kerkje.



Figuur 5.1: De toiletpapier-simulatie.

Bij de eerste herformulering van het probleem gaan we ervan uit dat het programma een tijdlang moet simuleren en dat tijdens het simulatieproces tussentijdse resultaten moeten verschijnen en na afloop een samenvatting moet worden gegeven waarop een antwoord op de vraag naar de papierverspilling kan worden gebaseerd. De eerste herformulering luidt:

1. Zolang als de simulatie duurt
   2. plaats nieuwe rollen papier, tel het aantal keren dat dit gebeurt.
   3. gebruik de rollen totdat er één leeg is.
   4. vermeld hoeveel velletjes op elke rol resteren.
   5. onthoud het totaal aantal niet gebruikte velletjes.
6. Vermeld na afloop van de simulatie het aantal keren dat er rollen zijn vernieuwd, het totaal aantal weggegooide velletjes en het gemiddeld aantal weggegooide velletjes.

Bij de tweede herformulering worden een aantal beslissingen genomen. Het vermelden van resultaten kan gebeuren met behulp van de functie *printf()*. Het plaatsen van nieuwe rollen implementeren we door het toekennen van een waarde *NEW_ROLL* aan een teller waarin wordt bijgehouden hoeveel velletjes er nog zijn. Er zijn twee rollen en we gebruiken een *int* array met twee elementen om de aantallen velletjes bij te houden. Daarnaast hebben we nog tellers nodig voor het aantal keren dat de rollen worden ververst en voor het totaal aantal weggegooide velletjes. Het *int* array noemen we *roll,* de tellers respectievelijk *refresh* en *spilled*. Hoeveel velletjes gaan er in een rol wc papier? We gebruiken een `#define NEW_ROLL` die we arbitrair op 1000 zetten.

Een anderssoortige beslissing betreft de vraag hoelang er moet worden gesimuleerd. Een aantal mogelijkheden zijn:

- een vast aantal keren (bijvoorbeeld 100 verversingen).

- een vaste tijd (bijvoorbeeld 5 minuten).
- totdat het gemiddelde convergeert naar één waarde.
- totdat de persoon die het programma start het programma weer stopt.

We kiezen voor de laatste mogelijkheid en maken gebruik van de standaardfunctie *kbhit()* die 0 retourneert wanneer er geen toets is ingedrukt en een andere waarde wanneer dat wel het geval is. De simulatie blijft dus doorgaan totdat een toets wordt ingedrukt. In listing 14 geven we een implementatie van de *main()* functie en definiëren we de nodige variabelen en constanten. Onderdeel 3. laten we onbeslist door voorlopig een functie *consume()* aan te roepen.

```
1       #define NEW_ROLL                1000
2
3       int
4           roll[2];
5
6       void main()
7       {
8           int
9               spilled = 0,
10              refresh = 0;
11
12          while (!kbhit())                        // Tekst: 1.
13          {
14              roll[0] = roll[1] = NEW_ROLL;       // Tekst: 2.
15              refresh++;                          // Tekst: 2.
16
17              consume();                          // Tekst: 3.
18
19              printf ("%6d%6d\n", roll[0], roll[1]);// Tekst: 4.
20
21              spilled += roll[0] + roll[1];       // Tekst: 5.
22          }
23
24          printf ("Totaal verspild: %d"           // Tekst: 6.
25                  "na %d verversingen\n"
26                  "gemiddeld aantal verspild: %g\n",
27                      spilled,
28                      refresh,
29                      (double) spilled / refresh);
30
```

Listing 14: Het wc-papier probleem: de functie *main()*.

Het gebruik van de rollen totdat er één leeg is leidt tot de volgende herformulering:

3.1. Zolang op beide rollen nog papier is:
  3.2. Bepaal of een groot- of klein-gebruiker naar de wc gaat.
  3.3. Verminder het aantal velletjes overeenkomstig de gewoonte van de soort gebruiker.

Onderdeel 3.1 kan worden omgezet in **C** code: een *while* lus waarin wordt getest of de waarde in *roll[0]* en *roll[1]* ongelijk 0 zijn. Onderdeel 3.2. vereist dat op basis van toeval de ene of de andere gebruikerssoort wordt gekozen. De standaardfunctie *rand()* levert een toevalswaarde. We besluiten dat alle oneven toevalswaarden een grootgebruiker zullen aanduiden en alle even toevalswaarden een kleingebruiker. De functie *rand()* is gedeclareerd in de header file `stdlib.h`. Onderdeel 3.3 vinden we nog te moeilijk: we gebruiken de functie *use_paper()* om het aantal velletjes te reduceren. Omdat de reductie afhankelijk is van de gebruiker geven we een argument mee om aan te geven of de reductie voor een groot- of een klein-gebruiker moet worden uitgevoerd. De implementatie van *consume()* is gegeven in listing 15.

```
1           #include <stdlib.h>
2
3           void consume()
4           {
5               while (roll[0] && roll[1])          // Tekst: 3.1. en 3.2.
6                   use_paper(rand() & 1);          // Tekst: 3.3.
7           }
```

Listing 15: Het wc-papier probleem: de functie *consume()*.

De volgende herformulering betreft de reductie van het aantal velletjes volgens punt 3.3. Uit de probleemformulering leiden we de volgende regels af:

3.3.1. Bepaal de te gebruiken rol.

3.3.2. Reduceer de te gebruiken rol met het juiste aantal, gegeven de aard van de gebruiker.

3.3.3. Wanneer het aantal na reductie negatief is, moet het aantal op nul worden gezet.

Het bepalen van de te gebruiken rol laten we voorlopig over aan de functie *select_roll(),* die als argument het gebruikerstype meekrijgt en de index retourneert van de rol waarvan velletjes worden gebruikt. De indexwaarde betreft het array *roll*. We maken gebruik van een lokale variabele *index* voor de opslag van de geretourneerde indexwaarde. Onderdeel 3.3.2. vereist dat we weten hoeveel velletjes door een groot- of kleingebruiker worden gebruikt. Deze waarden leggen we vast in twee `#defines`, respectievelijk *BIGUSER* en *SMALLUSER*. De implementatie van de functie *use_paper()* geven we in listing 16.

```
1           #define BIGUSER    15                   // velletjes per keer
2           #define SMALLUSER   3
3
4           void use_paper(int big_user)
5           {
6               int
```

```
7                   index;
8
9               index = select_roll(big_user);      // Tekst: 3.3.1.
10
11              roll[index] -=                      // Tekst: 3.3.2.
12                  big_user ?
13                      BIGUSER
14                  :
15                      SMALLUSER;
16
17              if (roll[index] < 0)                // Tekst: 3.3.3.
18                  roll[index] = 0;
19          }
```

Listing 16: Het wc-papier probleem: de functie *use_paper()*.

Tenslotte het bepalen van de rol. De functie *select_roll()* wordt aangeroepen met als argument het type gebruiker en retourneert de index in het array *roll* die moet worden gebruikt. De volgende herformulering beschrijft hoe de juiste rol wordt gekozen:

3.3.1.1. Wanneer de rollen even vol zijn, retourneer dan op basis van toeval een rol-indexwaarde.

3.3.1.2. Bepaal de index van de meest gevulde rol.

3.3.1.3. Wanneer de bezoeker een grootgebruiker is, retourneer dan de zojuist berekende index en retourneer de andere index wanneer de gebruiker een kleingebruiker is.

De implementatie van de functie *select_roll* geven we in listing 17.

```
1           int select_roll(int big_user)
2           {
3               int
4                   index;
5
6               if (roll[0] == roll[1])             // Tekst: 3.3.1.1.
7                   return (rand() & 1);            // zie ook: 3.2.
8
9               index = roll[1] > roll[0];          // Tekst: 3.3.3.
10
11              return                              // Tekst: 3.3.1.3.
12              (
13                  big_user ?
14                      index
15                  :
16                      !index
17              );
18          }
```

Listing 17: Het wc-papier probleem: de functie *select_roll()*.

Opgave 30.

1. Stel het programma `toilet.c` samen. Zet aan het begin van het programma de *#include* opdrachten voor de header files *stdio.h, conio.h* en *stdlib.h,* gevolgd door de drie *#defines*. Definieer variabelen en functies voordat ze worden gebruikt. Vertaal en link het programma.
2. Wat geburt er wanneer *toilet.exe* bij herhaling wordt gestart?
3. Initialiseer de functie *rand()* als eerste statement in *main()* met behulp van de functie *srand()* op een toevallige beginwaarde. Gebruik de functie *clock()* (gedeclareerd in *time.h*) om een willekeurig argument aan *srand()* te geven. De return-waarde van *clock()* moet worden gecast naar *unsigned*. Wat gebeurt er nu wanneer *toilet.exe* bij herhaling wordt gestart?
4. Verander de constanten in *int* variabelen, die een waarde krijgen bij de aanroep van het programma. Gebruik de functie *atoi()* om de argumenten die bij de aanroep worden meegegeven te converteren naar *int* waarden. Het eerste argument is het aantal velletjes op een nieuwe rol, het tweede het aantal velletjes dat door een grootgebruiker wordt gebruikt en het derde argument het aantal velletjes dat door een kleingebruiker wordt gebruikt. Experimenteer met verschillende argumenten.

Hoofdstuk 6

De preprocessor

Zoals in 1.5.1 al genoemd, begint de vertaling bij de preprocessor. Het werk van de preprocessor bestaat meestal uit het kopiëren van de brontekst naar een kladbestand. De preprocessor onderneemt speciale acties alleen bij het lezen van preprocessor directives. In dit hoofdstuk worden de belangrijkste directives behandeld.

6.1 De include directive

Bij het lezen van een *#include* directive schakelt de preprocessor tijdelijk over naar een ander bestand, en kopieert de inhoud van dit bestand naar zijn uitvoerbestand. De naam van het bestand dient achter de *#include* te staan. Er zijn twee manieren om de bestandsnaam op te geven:

- Als de bestandsnaam tussen '<' en '>' opgegeven wordt, probeert de preprocessor het bestand in de directory te zoeken die door de systeemvariabele *INCLUDE* wordt aangewezen. Een voorbeeld van zo'n directive is:

```
#include <stdio.h>
```

- Als de bestandsnaam tussen dubbele aanhalingstekens opgegeven wordt, probeert de preprocessor het bestand in de huidige directory te vinden. Als dit faalt, dan kijkt de preprocessor alsnog in de directory die door systeemvariabele *INCLUDE* aangewezen wordt. Een voorbeeld van zo'n directive is:

```
#include "local.h"
```

De bestanden die door middel van de *#include* directive gelezen worden, worden doorgaans header files genoemd, wellicht omdat het aansluiten van deze bestanden meestal bovenaan (in de kop) van **C** bronteksten gebeurt. Een gangbare conventie hierbij is dat header files geen definities, maar alleen declaraties (van functies en data) bevatten. In dit boek wordt deze conventie aangehouden.

In header files kunnen op hun beurt weer *#include* directives staan. Het is dus mogelijk om een header file (bij voorbeeld *all.h*) te maken, waarin *#include*s voor alle bij de **C** vertaler horende header files staan. In bronteksten zou dan de preprocessor directive:

#include <all.h>

staan, waardoor *alle* header files gelezen zouden worden. Het voordeel hiervan zou zijn dat de software-ontwikkelaar niet op zou hoeven te zoeken in welke header file een bepaalde functie gedeclareerd is. Het nadeel is dat de vertaaltijd langer wordt, omdat de preprocessor meer bestanden moet bewerken, en omdat de vertaler daardoor een grotere hoeveelheid informatie moet verwerken.

6.2 De define directive

De *#define* directive wordt gebruikt om symbolische namen voor constanten of bewerkingen te kunnen gebruiken. Deze directive volgt in de praktijk de volgende syntaxis:

#define *symbolic-name expansion*

Hierbij staat *symbolic-name* voor de te definiëren symbolische naam. De letterlijke betekenis van deze naam is in *expansion* gegeven. Volgens onze layout-suggestie, zoals ook in bijlage D gegeven, worden symbolische namen in hoofdletters gegeven.

Als de preprocessor een *#define* directive leest, wordt de betekenis van de symbolische naam opgeslagen. Als de preprocessor later deze symbolische naam tegenkomt, wordt in plaats van deze symbolische naam de betekenis hiervan naar het uitvoerbestand geschreven. Men spreekt daarom ook wel van de *expansie* van *#define*s: een symbolische naam wordt geëxpandeerd tot zijn betekenis.

De *#define*s worden gewoonlijk in twee groepen verdeeld: *#define*s zonder argumenten en *#define*s met argumenten.

6.2.1 Defines zonder argumenten

*#Define*s zonder argumenten worden gewoonlijk gebruikt om constante waarden van een symbolische naam te voorzien. Een voorbeeld hiervan is in onderstaande listing gegeven. Hier wordt met behulp van *#define* een constante *MAX* gedefinieerd, die op meerdere plaatsen in de brontekst gebruikt wordt.

```
#define MAX 30

...
int
    i,
    vector [MAX];
...
for (i = 0; i < MAX; i++)
    ...
```

Het voordeel van het gebruik van de symbolische naam *MAX* ligt in het feit dat een uitbreiding van het array *vector* kan geschieden door alleen de *#define* te wijzigen en de brontekst opnieuw te vertalen. Daarmee wordt zowel de definitie van het array *vector* als de controle-expressie in het *for* statement beïnvloed.

Opgave 31.

Wat is het verschil tussen de hieronder afgedrukte programmafragmenten met betrekking tot:
1. het aantal keren dat string *"data.tmp"* in het datadeel van het uiteindelijke programma voorkomt; en
2. de omvang van de object files (en van de executables)?
Welke code is (dus) te prefereren?

```
/* --------------------- FRAGMENT 1 */
static char
    tmp_file [] = "data.tmp";

void main ()
{
    FILE
        *f;

    if (! (f = fopen (tmp_file, "wb")) )
        printf ("Cannot open %s!\n",
            tmp_file);
}

/* --------------------- FRAGMENT 2 */
#define TMP_FILE "data.tmp"

void main ()
{
    FILE
        *f;

    if (! (f = fopen (TMP_FILE, "wb")) )
        printf ("Cannot open %s!\n",
            TMP_FILE);
}
```

6.2.2 Defines met argumenten

*#Define*s kunnen ook gebruikt worden om een symbolische naam aan een bewerking toe te kennen. De argumenten van de bewerking worden dan gegeven na de symbolische naam, tussen haakjes en gescheiden door komma's.

Een voorbeeld van een *#define* met één argument is in het volgende voorbeeld gegeven. In dit voorbeeld wordt een *#define ABS* gemaakt, die bij expansie tot een expressie leidt waarbij de absolute waarde bepaald wordt met behulp van de ternaire operator.

```
#define ABS(x)      ((x) < 0 ? -(x) : (x))
...
int
    x = -10,
    y;

y = ABS (x);
```

De *#define ABS* wordt in de toekenning op de laatste regel van het voorbeeld geëxpandeerd tot:

```
y = ((x) < 0 ? -(x) : (x));
```

Over *#define*s met argumenten tenslotte dit:

- Als een *#define* syntactisch incorrect is opgesteld, kan dat bij vertaling tot vreemde fouten leiden. De vertaler ziet immers niet de *#define* zelf, maar de expansie ervan.

- Als een *#define* niet correct functioneert, dan wordt het nakijken op fouten extra lastig. Omdat in plaats van de *#define* de expansie vertaald is, kan een *#define* niet door *CodeView* op fouten nagekeken worden.
- Sommige vertalers staan bij *#define*s met argumenten géén spaties toe tussen de symbolische naam en de argumenten (in bovenstaand voorbeeld tussen *ABS* en *(arg)*). De Microsoft vertaler staat op deze plaats wèl spaties toe; het is echter aan te bevelen om geen spaties te gebruiken terwille van de overdraagbaarheid van brontekst en.
- Bij *#defines* met argumenten is het zaak om in de definitie van de *#define* de argumenten met haakjes te beschermen, zoals geïllustreerd in bovenstaand voorbeeld. In het tegenvoorbeeld hieronder zijn de argumenten *niet* beschermd, en levert *ABS(-10 - 1)* waarde *9* op:

```
#define ABS(arg)        (arg < 0 ? -arg : arg)
...
int
    x = -10,
    y;

y = ABS (x - 1);
```

De reden hiervan is het feit dat *ABS(x – 1)* geëxpandeerd wordt tot:

```
x - 1 < 0 ? -x - 1 : x - 1
```

- *#Define*s zijn afgelopen als de regel tekst beëindigd is. Een *#define* kan echter over meerdere regels uitgespreid worden door een '\' aan het einde van een regel te gebruiken. De preprocessor leest dan de volgende regel en interpreteert deze als deel van de *#define*.

Opgave 32.

Schrijf een *#define MIN* die drie argumenten krijgt. De *#define* moet d.m.v. de ternaire operator de kleinste waarde van deze drie bepalen. Test de *#define* met behulp van onderstaand code-frame:

```
#include <stdio.h>                          // voor printf()

#define MIN ....                            // definieer MIN hier

void main ()
{
    printf ("Het laagste getal uit 10, 3 en 51 is: %d\n",
        MIN (10, 3, 51));
}
```

Opgave 33.

Schrijf een *#define ERROREXIT* die één argument krijgt. Dit argument is een af te drukken string. Na het afdrukken moet het programma stoppen. Test de *#define* met behulp van onderstaand code-frame:

```
// definieer ERROREXIT hier
```

```
void main ()
{
    int
        x = 10;

    if (x < 20)
        ERROREXIT ("foutje, ik stop ermee");
    else
        printf ("Of toch maar niet...\n");
}
```

6.3 Overige directives

Andere vaak gebruikte preprocessor directives zijn *#ifdef* en *#ifndef*. Deze directives worden hieronder besproken.

6.3.1 Ifdef

Deze directive test of het argument erachter een gedefinieerde naam is. De *#ifdef* wordt dan gevolgd door instructies die uitgevoerd worden als de erachter staande naam inderdaad gedefinieerd is. Het geheel wordt afgesloten door *#endif*.

Tussen *#ifdef* en *#endif* kan een optioneel *#else* gedeelte voorkomen. Dit is geïllustreerd in onderstaand voorbeeld, waar afhankelijk van de gedefinieerde naam *MSC* de header file *string.h* dan wel *strings.h* ge-*include* wordt[1]:

```
#ifdef MSC
#   include <string.h>
#else
#   include <strings.h>
#endif
```

Dit voorbeeld illustreert tevens de vaak gebruikte opzet bij electronisch gedistribueerde bronteksten. Afhankelijk van gebruikt bedrijfssysteem of vertaler worden verschillende header files aangesloten, worden verschillende functies aangeroepen, enzovoort. Vaak staat dan ergens boven in de brontekst dat *MSC* gedefinieerd moet worden, als de gebruikte vertaler van Microsoft is.

Het definiëren van een symbolische naam als *MSC* kan doorgaans op twee manieren gebeuren. Ten eerste kan de directive:

```
#define MSC
```

in de brontekst gezet worden. Ten tweede kan de brontekst vertaald worden met gebruik van de /D optie (van define). Een voorbeeld is de opdracht:

1. Let op de layout-suggestie waarbij het '#'-teken van *elke* directive op de eerste kolom van de brontekst begint. Overige delen van *#ifdef*s, zoals de *include* directives in het voorbeeld, kunnen wel geïndenteerd zijn.

c /D MSC file.c

waarin *file.c* de bestandsnaam is[2].

6.3.2 Ifndef

De *#ifndef* directive test of de symbolische naam die erachter staat *niet* gedefinieerd is. Het gebruik van *#ifndef* is analoog aan *#ifdef*; er kan een *#else* deel zijn, en het geheel wordt afgesloten door *#endif*.

#Ifndef wordt vaak gebruikt om te voorkomen dat een header file meer dan één maal verwerkt wordt bij meerdere *#include* opdrachten in dezelfde brontekst. Dit is geïllustreerd in het volgende voorbeeld:

```
/* File SAMPLE.H : this header file is processed only once */

#ifndef SAMPLE_DEFINED
#   define SAMPLE_DEFINED
.
. /* declarations of this header file */
.
#endif
```

Wanneer in een brontekst deze header file aangesloten wordt, dan is de constante *SAMPLE_DEFINED* niet gedefinieerd. De romp van de header file wordt dus verwerkt. Bij een tweede *#include* opdracht voor dezelfde header file in dezelfde brontekst is *SAMPLE_DEFINED* wèl gedefinieerd, en wordt de romp van de header file overgeslagen.

6.4 Uitvoer van de preprocessor

Het bestand dat de preprocessor aanmaakt is een kladbestand, dat door de **C** vertaler bewerkt wordt en vervolgens gewist wordt. De Microsoft **C** vertaler kent echter een optie waarmee de uitvoer van de preprocessor bekeken kan worden. Deze optie is */E*. Zo zal opdracht:

c /E file.c

de **C** vertaler aanroepen om de brontekst *file.c* te bewerken. Hierbij wordt uitvoer van de preprocessor naar het scherm gestuurd.

2. Merk op dat hier een expansieloze vorm van *#define* is gebruikt. Dergelijke *#define*s maken slechts een naam aan de preprocessor bekend.

Hoofdstuk 7

Input/output en Files

7.1 Inleiding

Dit hoofdstuk gaat over het werken met bestanden, voor zover dit van belang is bij het 'dagelijkse' gebruik. In dit hoofdstuk wordt de terminologie, die bij het werken met bestanden gebruikt wordt, toegelicht. Daarnaast worden de meeste **C** functies, die met bestandsverwerking te maken hebben, toegelicht. Voor een uitputtende beschrijving wordt verwezen naar de met de vertaler meegeleverde documentatie.

7.1.1 Input en output

Bestanden zijn de belangrijkste informatiestromen in het UNIX bedrijfssysteem. Omdat **C** op een UNIX-systeem ontwikkeld is, is het niet verwonderlijk dat de **C** functiebibliotheken een grote hoeveelheid functies bevatten die met in- en uitvoer van gegevens (input/output, of kortweg I/O) te maken hebben.

In **C** worden in- en uitvoerstromen gewoonlijk als stream aangeduid. Zo is de standaard uitvoerstroom (*stdout*), onder andere gebruikt door de functie *printf()*, uiteindelijk ook een stream. Afhankelijk van de opdracht op DOS niveau gaat deze stream naar het scherm of naar een bestand.

7.2 Buffered I/O en unbuffered I/O

De opzet van de functies die met streams te maken hebben is *gelaagd* van opbouw. De onderste laag bestaat uit functies die gegevens onmiddellijk naar een stream schrijven, dan wel van een stream lezen. Deze functies maken deel uit van de niet-gebufferde I/O, oftewel unbuffered I/O. Wanneer de verwerkte stream met een disk geassocieerd is, leidt dit tot een veelvuldige aansturing van de disk drive, hetgeen een vertraging van de verwerkingstijd betekent.

Om deze reden bevatten de **C** bibliotheken een laag functies die van de onderste laag gebruik maken, en gegevens in buffers opslaan. Bij het lezen wordt zo een gehele buffer in één keer gevuld, en vervolgens stukje bij beetje aan het programma doorgegeven. Bij het schrijven wordt eerst een buffer stukje bij beetje gevuld, en dan pas in één keer naar de uitvoerstream geschreven.

Listing 18 illustreert het gebruik van gebufferde I/O. Van een invoer-stream *infile* worden lettertekens gelezen en naar een uitvoer-stream *outfile* geschreven. De opgestelde functie *fcopy()* is zonder voorbeeld van een aanroep gegeven.

Over listing 18 merken we het volgende op:

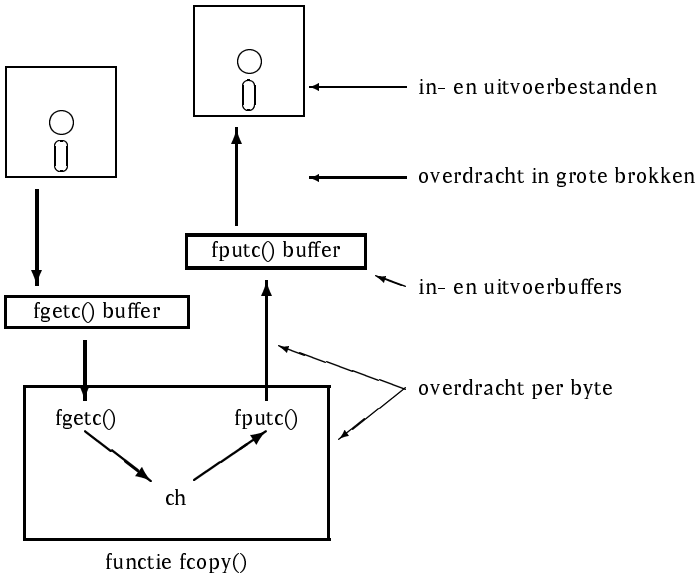
- Functie *fcopy()* verwerkt de in- en uitvoerstromen byte voor byte. Dit betekent niet dat voor elk gelezen of geschreven teken een diskaansturing nodig is; er wordt immers gebruik gemaakt van gebufferde I/O. Dit is in figuur 7.1 geïllustreerd.

```
/*
   fcopy (in, out)
   Kopieert stream infile naar stream outfile, teken voor teken
*/
void fcopy (FILE *infile, FILE *outfile)
{
    int                                 // gelezen/geschreven
        ch;                             // letterteken

    while (1)                           // herhaal:
    {
        ch = fgetc (infile);            // haal een teken van infile
        if (feof (infile))              // als einde v. stream:
            break;                      // stop de lus
        fputc (ch, outfile);            // anders, schrijf teken
    }                                   // naar outfile
}
```

Listing 18: Gebufferde I/O.

- In de parameterlijst van *fcopy()* (regel 5) wordt type *FILE* gebruikt. De beschijving van dit type stellen we uit tot 7.4.
- Op de regels 10 tot en met 16 wordt in een eindeloze lus de invoerstream verwerkt. Functie *feof()* bestuurt deze lus door een niet-nul waarde te retourneren als het einde van *infile* bereikt is. Als *feof()* een niet-nul waarde retourneert, wordt de lus onderbroken.
- Het lezen van *infile* gebeurt op regel 12 door de standaardfunctie *fgetc()*. Deze functie haalt één char op van de stream die als argument wordt meegegeven (in dit geval *infile*). De return-waarde van *fgetc()* is het gelezen ASCII-teken.
- Op regel 15 wordt het gelezen teken naar *outfile* geschreven. Dit gebeurt door de functie *fputc()*. Deze functie heeft twee argumenten: de weg te schijven ASCII-waarde en de stream waar de informatie naar toe moet.

Gebufferde I/O hoeft uiteraard niet byte voor byte te gebeuren, zoals dat in listing 18 het geval is. In figuur 7.2 zijn de gangbare functies die van gebufferde I/O gebruik maken weergegeven. In deze figuur worden, vooruitlopend op 7.4.1, de termen *stdin* en *stdout* gebruikt. Deze termen duiden respectievelijk op de standaard invoer-stream en de standaard uitvoerstream. De functies die met '†' aangeduid zijn, zijn in de header file *stdio.h* geïmplementeerd als macro's (*#defines*).



Figuur 7.1: Het bufferen van I/O.

Functie	Beschrijving
putchar() †	schrijft een byte naar *stdout*
fwrite()	schrijft een willekeurige hoeveelheid naar een willekeurige stream
fputc()	schrijft een byte naar een willekeurige stream
puts()	schrijft een string naar *stdout*
fputs()	schrijft een string naar een willekeurige stream
getchar() †	leest een byte van *stdin*
fread()	leest een willekeurige hoeveelheid van een willekeurige stream
fgetc()	leest een byte van een willekeurige stream
gets()	leest een string van *stdin*
fgets()	leest een string van een willekeurige stream

Figuur 7.2: Functies voor gebufferde I/O.

Opgave 34.
Hoewel in listing 18 van gebufferde I/O gebruik wordt gemaakt, moet in functie *fcopy()* teken-voor-teken de invoer-stream gelezen worden en de uitvoer-stream geschreven worden. Dit is een onnodige vertraging.

Herschrijf functie *fcopy()* met behulp van functies *fread()* en *fwrite()*. Gebruik als opslagruimte voor de gelezen informatie een buffer van bij voorbeeld 200 bytes. Als stop-conditie voor het onderbreken van de eindeloze lus kan de return-waarde van *fread()* gebruikt worden.

7.3 Formated I/O en unformated I/O

In- en uitvoer in **C** wordt gewoonlijk, naast het al dan niet gebufferd zijn, ingedeeld in geformatteerde en ongeformatteerde I/O. Formated[1] I/O verzorgt de lees- of schrijfopdrachten waarbij aan de hand van een format string gegevens verwerkt worden. Functie *printf()* is een voorbeeld van formated I/O, aangezien afhankelijk van de format string verschillende gegevens naar de standaard uitvoer-stream geschreven worden. Unformated I/O daarentegen schrijft of leest gegevens zoals letterlijk opgegeven. Een voorbeeld van een unformated output functie is *fputs()*; deze functie schrijft een string naar een willekeurige stream zonder dat de string een bewerking ondergaat.

7.3.1 Functies voor formated I/O

De **C** bibliotheken beschikken over twaalf functies die de formated I/O verzorgen. Deze functies hebben betrekking op standaardstreams, op willekeurige streams, en op strings. Deze functies zijn in figuur 7.3 genoemd.

Al deze functies hebben de volgende gemeenschappelijke kenmerken:

- Met behulp van deze functies kan een een variabel aantal gegevens worden verwerkt. Bij één aanroep kunnen zo meerdere gegevens gelezen of geschreven worden.[2]
- De functies vereisen een format string als één van de argumenten. Deze string specificeert hoe eventuele volgende argumenten opgevat moeten worden.
- De *(f/s)scanf()* functies retourneren het aantal verwerkte (gelezen) argumenten. De *(f/s)printf()* functies retourneren het aantal geschreven *char*s.

Afhankelijk van hun taak verwachten de verschillende functies een verschillend eerste argument. Zo verwachten *fprintf()* en *fscanf()* als eerste

1. We hebben gekozen voor de spelling van 'formated' met één 't', analoog aan 'combated' en 'automated'. De Engelse spelling is hierover echter niet eenduidig.
2. Opgemerkt wordt dat in **C** functies ontwikkeld kunnen worden die een variabel aantal argumenten bevatten. Dit is één van de sterke kanten van **C**; in **pascal** bij voorbeeld is dit niet mogelijk.

Functie	Beschrijving
printf()	schrijft naar *stdout*
scanf()	leest van *stdin*
vprintf()	*printf()* m.b.v. pointer naar de argumentenlijst
vscanf()	*scanf()* m.b.v. pointer naar de argumentenlijst
fprintf()	schrijft naar een willekeurige stream
fscanf()	leest van een willekeurige stream
vfprintf()	*fprintf()* m.b.v. pointer naar de argumentenlijst
vfscanf()	*fscanf()* m.b.v. pointer naar de argumentenlijst
sprintf()	converteert data tot een string
sscanf()	converteert een string tot data
vsprintf()	*sprintf()* m.b.v. pointer naar de argumentenlijst
vsscanf()	*sscanf()* m.b.v. pointer naar de argumentenlijst

Figuur 7.3: Formated I/O functies.

argument een aanduiding van de te bewerken stream. De functies *sscanf()* en *sprintf()* verwachten als eerste argument de te bewerken string.

Bij de functies *v(f/s)printf()* en *v(f/s)scanf()* worden niet de argumenten zelf achter de format string opgegeven, maar wordt de *plaats* waar de argumenten staan opgegeven. De manier van het verwerken van de argumenten is dan analoog aan de verwerking van de argumenten bij functie *max()* in listing 9 op pagina 101.

7.3.2 De format string

De format string bestaat gewoonlijk uit tekst en uit stuurtekens. In 3.2 werd het stuurteken '*%d*' al genoemd; dit leidt bij de functie *printf()* tot het afdrukken van een *int* waarde. Stuurteken '*%c*' is ook al besproken; onder andere in hoofdstuk 5.

De meest gangbare stuurcodes in format strings zijn weergegeven in figuur 7.4. We wijzen er met nadruk op dat de mogelijkheden van formated I/O veel uitgebreider zijn dan zoals gesuggereerd in figuur 7.4; bijvoorbeeld kan door *printf()* een getal over een bepaalde breedte geschreven worden, zodat *printf()* voor of achter dit getal spaties toevoegt. Deze informatie is te vinden in de met de vertaler meegeleverde documentatie.

Als voorbeeld laten we hier een stukje code zien waarbij gegevens naar een string-representatie geconverteerd worden. De string wordt vervolgens naar het scherm geschreven:

```
char
    ch = 'a',                       // char variable
    buf [200];                      // buffer for data
int
    i = 80;                         // int variable

sprintf (buf, "(%c) Number %d", ch, i);
puts (buf);
```

Stuurcode	Betekenis
%c	verwerking van *char*
%d	verwerking van *int*
%s	verwerking van string
%u	verwerking van *unsigned int*
%x	hexadecimale verwerking van *int*

Figuur 7.4: Vaak gebruikte stuurcodes in format strings.

```
sprintf (buf, "(%c) Same number %x, now as hex", ch + 1, i);
puts (buf);
```

In dit voorbeeld worden een *char* variabele die de ASCII-code voor de letter 'a' bevat en een *int* variabele die de waarde 80 bevat naar string-formaat geconverteerd. Ter illustratie wordt de string vervolgens afgedrukt. Daarna wordt dezelfde *int* variabele in hexadecimale representatie naar string-formaat geconverteerd, en wordt de string wederom naar het scherm gestuurd.

Opgave 35.
Hoe ziet de uitvoer van bovenstaand code-fragment er uit? Beredeneer het antwoord, in plaats van de code uit te proberen.

Bij het gebruik van functies die gegevens lezen (dit zijn de functies *scanf(), sscanf(), fscanf()* en *v(f/s)scanf()*) moet opgemerkt worden dat te lezen argumenten die niet arrays zijn op een speciale manier opgegeven moeten worden. Deze argumenten moeten door de *adres-van* operator (&) worden voorafgegaan. De correcte aanroep van *scanf()* om een *int* variabele *number* in te lezen is daarom:

```
scanf ("%d", &number);
```

De reden hiervoor en het gebruik van de &-operator stellen we uit tot hoofdstuk 8.

7.3.3 Een applicatie: dechex

Ter illustratie van de mogelijkheden van formated I/O wordt in dit onderdeel een programma ontwikkeld, dat een getal leest in òfwel decimale, òfwel hexadecimale notatie, en vervolgens hetzelfde getal afdrukt in decimale èn hexadecimale notatie. Het programma zullen wij 'dechex' noemen. Ter gedachtenbepaling is hieronder een voorbeeld van een sessie met dit programma weergegeven. De uitvoer van het programma is in teletype font weergegeven. De invoer van een gebruiker is *schuin* afgedrukt.

```
Enter a number in either decimal format, or in hexadecimal format
(i.e. prefixed by '0x'), followed by ENTER.
```

```
When finished, press ENTER without having entered a number.

? 1267
The number: 0x4f3 hexadecimal, 1267 decimal.
? 0xffff
The number: 0xffff hexadecimal, -1 decimal.
? 0xec76
The number: 0xec76 hexadecimal, -5002 decimal.
? afdasd
Bad input, try again.
? 0x45
The number: 0x45 hexadecimal, 69 decimal.
?
Bye.
```

Probleemanalyse

De bovenste laag van de besturing van het dechex-programma bestaat uit het afdrukken van een startboodschap, gevolgd door het verwerken van de invoer, waarna een slotboodschap afgedrukt wordt. De eerste herschrijving naar een gebiedende wijs ziet er daarom als volgt uit:

1. Laat startboodschap zien.
2. Verwerk invoer.
3. Laat slotboodschap zien.

De implementatie van punten (1) en (3) kan met behulp van functie *printf()* of *puts()* gebeuren. We concentreren ons daarom op de verdere herschrijving van punt (2).

De verwerking dient door te gaan totdat de gebruiker niets meer invoert. Het ligt dus voor de hand om in een lus invoer te vragen en te verwerken:

2. Zolang als er invoer is:
   2.1. Verwerk de invoer.

De bepaling òf er invoer is, kan echter pas gebeuren als invoer opgehaald is. We herschrijven punt (2) daarom naar:

2. In een eindeloze lus:
   2.1. Probeer invoer op te halen.
   2.2. Onderbreek de lus als er geen invoer is.
   2.3. Verwerk de ingelezen invoer.

Punten (2.1) en (2.2) kunnen als **C** code herschreven worden. Het enige dat nog rest is (2.3), de verwerking van ingelezen invoer.

Uit het sessievoorbeeld op pagina 133 blijkt dat het dechex-programma twee soorten invoer moet kunnen onderscheiden. Invoer die uit de cijfers 0 tot en met 9 bestaat, eventueel voorafgegaan door een –, wordt

als een decimaal getal geïnterpreteerd. Invoer waarbij de eerste twee tekens een 0 en een x zijn, wordt als hexadecimaal geïnterpreteerd. Punt (2.3) kan nu herschreven worden als:

2.3. Verwerking van de ingelezen invoer:
  2.3.1. Als het eerste teken van de invoer 0 is en als het tweede teken van de invoer x is, interpreteer dan de invoer als hexadecimaal getal, en verwerk de invoer.
  2.3.2. Anders, interpreteer de invoer als decimaal getal en verwerk de invoer.

Een vereiste hierbij is dat de invoer als een string opgehaald wordt, zodat de eerste twee tekens bekeken kunnen worden.

Het interpreteren van de invoer als hexadecimaal of decimaal getal doet aan een format string denken, waarbij respectievelijk *%x* of *%d* gebruikt wordt. Het verwerkingsproces zelf zal bestaan uit een conversie van de ingelezen buffer aan de hand van de format string *%x* of de format string *%d*. Als de conversie slaagt, dan wordt de berekende numerieke waarde als decimaal en als hexadecimaal getal afgedrukt. Als de conversie niet slaagt, dan wordt de boodschap '*Bad input, try again*' afgedrukt.

Implementatie

De implementatie van het dechex-programma begint bij de functie *main()*. De brontekst van deze functie is in listing 19 weergegeven. In deze brontekst is de functie *process()*, aangeroepen door *main()*, weggelaten. Uit de brontekst blijkt duidelijk dat de decompositie van het probleem nauwkeurig gevolgd is.

```
1     #include <stdio.h>
2
3     /* voeg de functie process() hier toe */
4
5     void main ()
6     {
7         char
8             buf [80];                                   // invoerbuffer
9
10        puts (                                          // startboodschap
11              "Enter a number in either decimal format,"
12                        "or in hexadecimal format\n"
13              "(i.e. prefixed by '0x'), followed by ENTER.\n"
14              "When finished, press ENTER without having entered a number.\n");
15
16        while (1)                                       // in eindeloze lus:
17        {
18            printf ("?" );                              // laat prompt zien
19            gets (buf);                                 // probeer invoer te lezen
20            if (buf [0] == '\0')                        // als invoerbuffer leeg is:
21                break;                                  // stop lus
22            if (buf [0] == '0' &&                       // als eerste 2 tekens van
23                buf [1] == 'x'                          // de buffer "0x" zijn:
24               )
25                process (buf, "%x");                    // interpreteer as hex
26            else
27                process (buf, "%d");                    // anders, interpreteer as dec
28        }
```

```
29
30          puts ("\nBye.\n");                          // toon eindboodschap
31      }
```

Listing 19: Dechex: de functie *main()*.

Bij listing 19 merken we nog het volgende op:

- De invoerbuffer (regel 8) is als een lokaal array gedefinieerd. Er is geen noodzaak om dit array globaal te maken.
- De aanroep van functie *puts()* op regel 10 toont hoe één string in meerdere losse delen opgegeven kan worden. De vertaler voegt de losse delen van de string samen tot één string, die als argument aan *puts()* doorgegeven wordt. De samenvoeging van delen van één string noemt men 'concatenatie'.
- Op de regels 18 en 19 wordt getracht invoer op te halen. Om de de gebruikersvriendelijkheid van het programma te vergroten wordt, voorafgaand aan de aanroep van functie *gets()*, een prompt (verzoek om invoer) getoond.
- De beslissing of invoer wel of niet gelezen is, wordt genomen op regel 20. Hier wordt het eerste teken van de buffer geinspecteerd. Als dit teken een ASCII-Z byte is, dan bevat de buffer geen informatie. Het *if*-statement kan ook geschreven worden als:

  ```
  if (! buf[0])
  ```

- Let bij de regels 22 en 23 op de layout-suggestie om complexe condities over meerdere regels uit te splitsen.
- Op de regels 25 en 27 wordt de functie *process()* aangeroepen. Deze functie dient de ingelezen buffer naar een getal te converteren. De conversie wordt uitgevoerd aan de hand van het tweede argument, een format string.

Opgave 36.

Stel het prototype van functie *process()* op. Let hierbij op het volgende:
1. Functie *process()* retourneert niets.
2. Functie *process()* heeft twee parameters, beide zijn strings.

Functie *process()* dient de volgende taken te verrichten:

- De gelezen invoer, die aan *process()* als eerste argument wordt doorgegeven, wordt aan de hand van een format string (het tweede argument) geconverteerd tot een *int* waarde. Als *int* variabele kan *process()* een lokale variabele gebruiken.
  De conversie kan gebeuren met behulp van de functie *sscanf()*. Het eerste argument van *sscanf()* is de gelezen buffer, het tweede is de format string en het derde argument is een verwijzing naar een variabele waar het resultaat van de conversie bewaard moet worden. Let er op dat het derde argument voorafgegaan moet worden door de &-operator.

- Als de conversie door *sscanf()* slaagt, dan wordt de verkregen waarde als decimaal en hexadecimaal getal afgedrukt. Het afdrukken kan gebeuren door *printf()*.
- Als de conversie niet slaagt, dan wordt boodschap '*Bad input, try again*' afgedrukt.

Opgave 37.
Wat is de return-waarde van *sscanf()*?

Opgave 38.
1. Stel aan de hand van bovenstaande specificaties functie *process()* op.
2. Vertaal en test het dechex-programma.

7.4 Het openen van een stream

Het werken met een stream begint gewoonlijk met het openen van een bestand, waardoor een stream aan dit bestand gekoppeld wordt. Het C runtime systeem heeft bij de start van de *main()* functie de volgende streams al geopend:

- *stdin*, de 'standard input stream', wordt gebruikt om gegevens te lezen. Functies als *gets()* en *scanf()* maken gebruik van deze stream. De *stdin* stream kan bij de aanroep van een programma gekoppeld worden aan een bestand door gebruik van het DOS redirectie-teken $<$.
- *stdout*, de 'standard output stream', wordt gebruikt om gegevens te schrijven. Functies als *printf()* en *puts()* schrijven naar deze stream.
- *stderr*, de 'standard error stream', wordt gebruikt om foutmeldingen van een programma naar het scherm te sturen. Het verschil tussen *stdout* en *stderr* is het feit dat bij de aanroep van een programma *stdout* wèl aan een bestand gekoppeld kan worden door gebruik van het DOS redirectie-teken $>$. Bij *stderr* is dit niet zo; zelfs als sprake is van redirectie verschijnt uitvoer door middel van *stderr* op het scherm.
- *stdprn*, de 'standard printer stream', stuurt de printer aan. Net als *stderr* kan deze stream niet met een bestand op disk gekoppeld worden.

In C programma's kan uiteraard met meer streams dan hierboven genoemd gewerkt worden. Het werken met een stream die aan een bestand gekoppeld is valt gewoonlijk uiteen in vier acties. Hieronder volgt een opsomming van deze acties. In de volgende onderdelen worden al deze acties toegelicht.

Definitie van een FILE pointer. Om een stream aan te duiden, wordt doorgaans een variabele gebruikt die een verwijzing naar het *FILE* type is.

Het openen van de stream. De gedefinieerde *FILE* pointer wordt gebruikt om met behulp van de functie *fopen()* een bestand te openen.

Het verwerken van de stream. De stream die met behulp van *fopen()* is geopend wordt vervolgens verwerkt. De meeste functies die van een stream lezen of naar een stream schrijven, verwachten de *FILE* pointer variabele als één van de argumenten.

Het sluiten van de stream. Na verwerking wordt de stream gesloten. Gewoonlijk gebeurt dit met een aanroep van de functie *fclose()*.

7.4.1 De FILE pointer

Om streams aan te kunnen duiden is in de header file *stdio.h* een datatype *FILE* gedefinieerd. Dit datatype bestaat uit een aantal velden[3] waarin relevante informatie over een bestand kan worden opgenomen. Het definiëren van niet-standaard types, zoals het geval is met *FILE*, stellen we uit tot hoofdstuk 9. Voorlopig kan met de veronderstelling gewerkt worden dat *FILE* een type is, net als *int*.

De definitie van variabelen die streams aanduiden, is in het onderstaande voorbeeld geïllustreerd. Hier worden twee variabelen *infile* en *outfile* gedefinieerd:

```
#include <stdio.h>

FILE
    *infile,
    *outfile;
```

7.4.2 De functie fopen()

Een bestand wordt gewoonlijk geopend door middel van een aanroep van functie *fopen()*. Deze functie verwacht twee string argumenten: de naam van het te openen bestand en de modus van de bewerking van het bestand. De return-waarde van *fopen()* is een *FILE* pointer, die bij de aanroep toegekend wordt aan een variabele, zoals *infile* uit het bovenstaande voorbeeld. Als *fopen()* er niet in slaagt een bestand te openen, wordt *NULL* geretourneerd.

Met de modus-string wordt specificeerd hoe *fopen()* een bestand moet openen. Er zijn drie basismogelijkheden voor het openen van een bestand:

3. Datatype *FILE* bevat onder andere een verwijzing naar de buffer die met de stream geassocieerd is, de datum van het bewerkte bestand, enzovoort. Gewoonlijk neemt men in **C** programma's het *FILE* type voor wat het is, en laat het bewerken van de velden van dit type over aan de interne administratie van **C**.

- Een bestand kan geopend worden om ervan te lezen. De modus-string voor deze operatie is *"r"*. Bij deze modus-string kan het bestand niet geopend worden als het niet bestaat.
- Een bestand kan geopend worden om te beschrijven. De modus-string voor deze operatie is *"w"*. Het bestand hoeft niet te bestaan; er wordt een nieuw bestand aangemaakt. Als het bestand vóór de aanroep van *fopen()* al bestond, dan wordt het overschreven.
- Een bestand kan geopend worden om informatie aan het einde bij te schrijven (dit wordt ook een 'append' bewerking genoemd). De modus-string is in dit geval *"a"*. Het bestand kan niet geopend worden als het niet bestaat.

De hierboven genoemde basismogelijkheden voor een modus-string kunnen met een *"+"* uitgebreid worden. De uitgebreide modus-string biedt de mogelijkheid een bestand zowel te lezen als te beschrijven.

Zo specificeert modus-string *"r+"* dat een bestand gelezen en beschreven mag worden. Het moet bij openen echter bestaan. Modus-string *"w+"* specificeert dat een bestand beschreven en gelezen mag worden. Bij openen hoeft het bestand niet te bestaan en een bestand met dezelfde naam wordt vernietigd.

Een mogelijke aanroep van *fopen()* is in onderstaand voorbeeld weergegeven. Hier wordt een *FILE* pointer-variabele *infile* gedefineerd en wordt *fopen()* aangeroepen om een bestand *data.doc* te openen. Het bestand wordt in *"r"* modus geopend. De return-waarde van *fopen()* wordt aan *infile* toegekend:

```
FILE
    *infile;

infile = fopen ("data.doc", "r");
```

Als openen faalt

Het openen van een bestand kan falen wanneer een bestand geopend wordt om te lezen en het bestand bestaat niet. Ook wanneer een bestand geopend wordt om te beschrijven kan het openen falen; een bestand met dezelfde naam maar met 'read-only' bescherming[4] kan bestaan, of het maximum aantal toegestane open bestanden kan bereikt zijn. In al deze gevallen retourneert *fopen()* de waarde *NULL*.

De werking van programma's berust vaak op het feit dat een bestand *moet* bestaan, of aangemaakt *moet* kunnen worden. Ter illustratie noemen wij hier de **C** vertaler: de **C** brontekst moet bestaan; anders heeft de **C** vertaler niets om te vertalen. De gegenereerde code moet naar een object-bestand geschreven kunnen worden, anders is de vertaalslag voor

4. Een bestand kan 'read-only' gemaakt worden met behulp van het DOS programma *attrib*.

niets. Daarom komt in **C** bronteksten vaak een constructie voor waarbij getracht wordt een bestand te openen. Als dit faalt, dan wordt het programma beëindigd. In het onderstaande code-fragment is dit geïllustreerd:

```
FILE
    *infile;

if (! (infile = fopen ("data.doc", "r")) )
{
    puts ("Cannot open data.doc for reading.);
    exit (1);
}
```

In dit voorbeeld wordt *fopen()* aangeroepen om bestand *data.doc* te openen. De return-waarde van *fopen* wordt aan de variabele *infile* toegekend. Als dit *NULL* blijkt te zijn, dan wordt het programma beëindigd.

Opgave 39.

Schrijf een functie *xfopen()* (van 'exit or fopen') die dezelfde return-waarde en argumenten heeft als *fopen()*. Het verschil met *fopen()* is dat als een bestand niet geopend kan worden, *xfopen()* een melding geeft en afbreekt.

Een voorbeeld van een aanroep van *xfopen()* zou kunnen zijn:

infile = xfopen ("data.doc", "r");

Na deze actie zou òfwel bestand *data.doc* geopend zijn, òfwel zou het programma beëindigd zijn.

Binaire- en tekstmodus

De programmeertaal **C** komt oorspronkelijk van het bedrijfssysteem UNIX. In dit bedrijfssysteem kent men, in tegenstelling tot DOS, geen 'carriage-return — linefeed' combinaties. In UNIX wordt een regel door één teken, de linefeed (ASCII-10) afgesloten. Daarom kan in **C** een end-of-line teken ook als één char, de '\n', worden weergegeven.

In DOS wordt een regel echter door twee tekens, de carriage-return (ASCII-13) en door de linefeed afgesloten. Dit heeft bij het overzetten van **C** naar DOS tot twee stream-modi geleid: de de tekstmodus die gewoonlijk gebruikt wordt en de binaire modus.

De binaire modus wordt gebruikt wanneer een bestand als een binaire file opgevat moet worden. De binaire modus wordt aan *fopen()* bekend gemaakt door de modus-string van een afsluitende *"b"* te voorzien. Zo specificeert modus-string *"rb"* dat een bestand geopend moet worden om te lezen in binaire modus.

Als een bestand in tekstmodus geopend wordt, onstaat een probleem tussen de representatie van het einde van een regel in het bestand zelf en in **C**. In het bestand zelf moeten carriage-return — linefeed combinaties gebruikt worden; in **C** wordt alleen de linefeed gebruikt. Dit probleem wordt opgelost door de **C** functies die het lezen en schrijven van bestanden

afhandelen: op de juiste tijdstippen moet bij het schrijven een carriage return ertussen gesmokkeld worden; of moet bij het lezen de carriage return overgeslagen worden.

Over deze conversie hoeft de software-ontwikkelaar zich gewoonlijk geen zorgen te maken. Bij het openen van een bestand wordt bepaald of het om een tekst- dan wel binair-bestand gaat, en de conversie wordt vervolgens aan de C functies overgelaten. De regel hierbij is dat een bestand in binaire modus geopend moet worden als het programma *alle* tekens die in het bestand voorkomen moet kunnen verwerken, ook de carriage-return tekens. Een voorbeeld van zo'n programma is *hexdump*.

7.4.3 Tijdelijke streams

Soms komt het in een C programma voor dat informatie tijdelijk opgeslagen moet worden, waarbij van tevoren niet bekend is *hoeveel* informatie dit is. Een elegante oplossing hierbij is de opslag in een tijdelijke stream.

Voor het openen van een tijdelijke stream kan de functie *tmpfile()* gebruikt worden. Deze functie retourneert een *FILE* pointer die geassocieerd is met een tijdelijk bestand. De bestandsnaam wordt door de functie *tmpfile()* gekozen. Functie *tmpfile()* probeert het bestand in *"w+b"* modus te openen; met andere woorden, het bestand mag in binaire modus beschreven en gelezen worden.

Bij de beëindiging van het programma worden alle tijdelijke bestanden gewist, tenzij op het moment dat het programma wordt beëindigd van directory is veranderd.

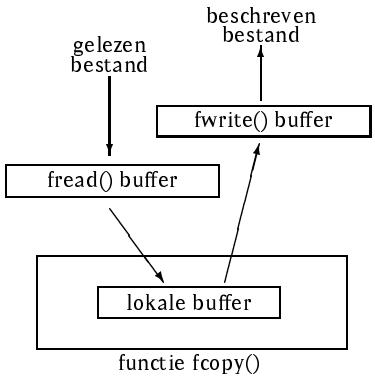
7.5 Overige functies en opmerkingen

7.5.1 Het gebruik van niet-gebufferde I/O

Gewoonlijk wordt in C programma's gebruik gemaakt van gebufferde I/O. Er zijn echter situaties waarin niet-gebufferde I/O de voorkeur verdient.

Zo'n situatie treedt op als een bestand gekopieerd wordt. De in opgave 34 gepresenteerde manier om een bestand te kopiëren heeft het nadeel dat met drie buffers gewerkt wordt; dat is eigenlijk twee buffers te veel. Eerst wordt informatie van een bestand door *fread()* in een buffer geplaatst. Vervolgens plaatst *fread()* gegevens van deze buffer naar de overdracht-buffer van de functie *fcopy()*. Na deze actie plaatst *fwrite()* de gegevens in een uitvoerbuffer. Pas daarna wordt de uitvoerbuffer naar disk geschreven. De gegevens-overdracht en de buffers zijn in figuur 7.5 geïllustreerd.

Het kopiëren van een bestand kan sneller gebeuren als van één buffer gebruik wordt gemaakt. In deze buffer kan zowel gelezen als te schrijven informatie bewaard worden. De opzet hierbij is dat van niet-gebufferde I/O



Figuur 7.5: Drie buffers bij gebufferd kopiëren.

gebruik wordt gemaakt, waarbij een eigen gedefinieerde buffer ter opslag dient. De functies die in dit geval gebruikt worden zijn:

- De functie *open()* opent een bestand. In tegenstelling tot gebufferde I/O, waarbij van streams wordt gesproken, wordt bij niet-gebufferde I/O over handles gesproken. Een handle is een *int* waarde, waarmee een geopend bestand wordt aangeduid.[5]
- De functie *read()* is de niet-gebufferde tegenhanger van *fread()*. Met behulp van *read()* wordt van een handle gelezen. De functie *read()* zet de gelezen gegevens in een buffer die als argument aan *read()* wordt meegegeven.
- De functie *write()* is de niet-gebufferde tegenhanger van *fwrite()*.
- De functie *close()* wordt gebruikt om een geopende handle te sluiten.

Het kopiëren met behulp van handles is in het onderstaande voorbeeld geïllustreerd. In de functie *main()* wordt een input handle geopend, waarbij een bestand *fcopy.c* moet bestaan. Als output handle wordt bestand *fcopy.out* geopend. In een andere situatie zouden bestanden geopend worden die bij voorbeeld door parameters van *main()* aangewezen werden.

Let tevens op het opgeven van argumenten aan functie *open()*. Verschillende constanten worden, gecombineerd door de binaire 'or' operator, als één waarde aan *open()* doorgegeven. De input handle wordt zo geopend in binaire modus, om te lezen. De output handle wordt in binaire modus geopend, waarbij het bijbehorende bestand eerst aangemaakt wordt en vervolgens beschreven mag worden.

5. Overigens heeft elke stream een handle-nummer. Gewoonlijk wordt bij gebufferde I/O echter de *FILE* pointer in plaats van de handle gebruikt. De standaardstreams (*stdin*, enzovoort) hebben elk ook een eigen handle.

```
1     #include <fcntl.h>                  // voor read()/write(),
2     #include <io.h>                     //  open(), etc.
3     #include <stdio.h>                  // voor puts()
4
5     void fcopy (int inh, int outh)      // kopieert input handle
6     {                                   // naar output handle
7         char
8             buffer [512];               // overdrachts-buffer
9         unsigned                        // aantal gelezen of
10            result;                     // geschreven bytes
11
12        while (result = read (inh,      // lees/schrijf totdat geen
13                              buffer, 512))// bytes meer gelezen
14            write (outh, buffer, result);// kunnen worden
15    }
16
17    void main ()
18    {
19        int
20            in,                         // input handle
21            out;                        // output handle
22
23        if ( (in = open ("fcopy.c",     // open input handle
24                O_BINARY | O_RDONLY)
25              ) == -1
26            )
27            puts ("Cannot open fcopy.c for reading.");
28        else if ( (out = open ("fcopy.out",// open output handle
29                O_BINARY | O_CREAT | O_WRONLY)
30                  ) == -1
31                )
32            puts ("Cannot open fcopy.out for writing.");
33        else
34        {
35            fcopy (in, out);            // kopieer in naar out
36            close (in);                 // en sluit de handles
37            close (out);
38        }
39    }
```

Listing 20: Niet-gebufferde I/O bij het kopiëren van bestanden.

Samengevat kan gezegd worden dat niet-gebufferde I/O de voorkeur verdient als:

- snelheid van belang is;
- geen operaties die uitgebreide bewerking van de stream (zoals regelgewijs inlezen) uitgevoerd hoeven te worden;
- de gelezen informatie geen bewerking hoeft te ondergaan voordat deze geschreven wordt.

7.5.2 Het positioneren van een stream

Een stream-operatie die ook regelmatig toegepast wordt is het positioneren van een stream. Hiermee wordt bedoeld dat de positie van een stream op een bepaalde waarde gezet wordt, zodat een volgende lees- of schrijfoperatie vanaf die positie in het bijbehorende bestand plaats zal vinden. Zo zal de functie *fgetc()*, indien aangeroepen vlak na *fopen()*, het eerste teken van een stream lezen. Als tussen de *fopen()* aanroep en de *fgetc()* aanroep de stream-positie op 10 wordt gezet, dan leest *fgetc()* niet het eerste maar het het elfde teken.

De functies die bij het positioneren van streams gebruikt worden zijn:

- *fseek()* zet de stream-positie op een bepaalde waarde. Als eerste twee argumenten worden de *FILE* pointer en de positie opgegeven. Het derde argument specificeert of de stream-verschuiving vanaf de beginpositie, vanaf de huidige positie, of vanaf de eindpositie moet plaatsvinden.
- *ftell()* retourneert de huidige stream-positie.

Omdat het bereik van *int* variabelen niet groot genoeg is, verwacht *fseek()* en retourneert *ftell()* een *long int* als positiespecificatie.

Een applicatie: simdiff

Als voorbeeld van het gebruik van de functies *ftell()* en *fseek()* wordt hier een programma ontwikkeld dat twee tekstbestanden vergelijkt. Als een regel gevonden wordt die in het ene bestand verschilt van het andere bestand, dan wordt deze discrepantie gemeld: het programma drukt de regel van het ene bestand en de regel van het andere bestand af. Bij de regels wordt tevens het regelnummer getoond. Het programma voert dus een eenvoudige vergelijking van twee bestanden uit. Daarom noemen we het programma 'simdiff'.

We stellen het programma zodanig op dat twee namen van te vergelijken bestanden bij de DOS opdracht opgegeven moeten worden. Een opdracht als:

simdiff file1 file2

zal dus de bestanden file1 en file2 op verschillen nakijken.

Een eerste probleemontleding leidt tot de volgende drie punten:

1. Open de beide bestanden.
2. Vergelijk de bestanden.
3. Sluit de bestanden.

Na deze eerste ontleding wordt het tijd om over te gaan op een aantal design-overwegingen. Hierbij rijzen drie vragen:

a. Wordt gebruik gemaakt van gebufferde of van ongebufferde I/O?
b. Worden de bestanden in tekst- of in binaire modus geopend?
c. Hoe worden de twee invoer-streams gelezen?

In dit voorbeeld kiezen we voor gebufferde I/O. De reden hiervoor ligt in het feit dat de stream-posities veelvuldig verschoven zullen moeten worden. Bij gebufferde I/O kan dit gedeeltelijk in het geheugen gebeuren; de administratie daarvan laten we over aan het **C** run-time systeem. Daarmee kan de eerste ontleding in **C** code omgezet worden. Bovendien besluiten we om de bestanden in tekstmodus te openen. Dit heeft tot gevolg dat in

het **C** programma newlines als één teken gerepresenteerd zullen worden. Listing 21 illustreert dit.

```
1     #include <stdio.h>
2
3     void main (int argc, char *argv[])
4     {
5         FILE
6             *in1,
7             *in2;
8
9         if (argc < 3)
10            puts ("Usage: SIMDIFF file1 file2");
11        else if (! (in1 = fopen (argv [1], "r")) )
12            printf ("Cannot open %s for reading.\n", argv [1]);
13        else if (! (in2 = fopen (argv [2], "r")) )
14            printf ("Cannot open %s for reading.\n", argv [2]);
15        else
16        {
17            compare (in1, in2);
18            fclose (in1);
19            fclose (in2);
20        }
21    }
```

Listing 21: De functie *main()* van simdiff.

De tweede ontleding van het probleem betreft het verwerken van de te vergelijken bestanden. Uit de probleemspecificatie blijkt dat de bestanden regelgewijs vergeleken moeten worden; als twee regels niet gelijk zijn, dan is een verschil vastgesteld. Het regelgewijs bestanden verwerken kan gerealiseerd worden met behulp van de functie *fgets()*, waarbij een buffer met gelezen tekst gevuld wordt. De buffer-omvang stelt ons echter voor een probleem: *fgets()* moet weten hoeveel tekst maximaal gelezen mag worden[6]. Het kan dus voorkomen dat *fgets()* een aantal tekens inleest, maar dat daarmee niet de regel in het gelezen bestand is afgelopen. We kiezen daarom voor een andere benadering, waarbij regelgewijze invoer nagebootst wordt.

Het nabootsen van regelgewijze invoer wordt gerealiseerd doordat de twee bestanden teken voor teken gelezen worden. De tekens worden vergeleken; zolang als deze tekens gelijk zijn, treedt geen verschil tussen de bestanden op. Als een '\n'-teken gelezen wordt, dan weten we dat een regel afgelopen is. Voor het gelezen regelnummer houden we een teller bij, omdat dit regelnummer bij het rapporteren van een verschil getoond gaat worden.

Als twee gelezen tekens ongelijk zijn, dan moeten de bijbehorende regels uit het eerste en uit het tweede bestand getoond worden. Dit houdt concreet in dat de stream-posities naar het begin van de regel gezet moe-

6. In de informatie over *fgets()* kan gevonden worden dat *fgets()* tot een maximum aantal of tot een '\n'-teken leest.

ten worden, en dat tekens op het scherm getoond worden totdat de regel is afgelopen. Het verschuiven van de stream-positie houdt in dat bij het tegenkomen van een '\n'-teken de positie onthouden moet worden.

Het regelgewijs bewerken van de streams kan nu als volgt worden herschreven:

2. Zolang als er tekens zijn:
   2.1. Lees een teken van bestand 1 en van bestand 2.
   2.2. Als de tekens ongelijk zijn, laat dan de huidige regel van bestand 1 en van bestand 2 zien.
   2.3. Als de tekens wel gelijk zijn, en ze betreffen een '\n'-teken, dan is net een regel verwerkt:
      2.3.1. Onthoud de posities in de streams.
      2.3.2. Onthoud het regelnummer.

```
void showline (int number, unsigned line, FILE *f, long pos)
{
    int
        ch;

    printf ("File %d, line %u:" , number, line);
    fseek (f, pos, SEEK_SET);
    while ( (ch = fgetc (f)) != '\n' &&
            ! feof (f)
          )
        putchar (ch);
    putchar ('\n');
}
```

Listing 22: De functie *showline()* van simdiff.

Het verwerken van de streams zolang als er tekens zijn, kan met behulp van een eindeloze lus gerealiseerd worden. Binnen de lus vinden alle acties plaats; de lus wordt onderbroken als de end-of-file-conditie (bepaald door middel van de functie *feof()*) waar wordt. Het lezen van tekens wordt gerealiseerd door de functie *fgetc()*.

Het tonen van de huidige regel wordt door de functie *showline()* gerealiseerd, zie listing 22. Deze functie verwacht drie argumenten. Het eerste argument, een *int*, duidt aan of een regel uit het eerste of uit het tweede bestand gelezen wordt. Het tweede argument, een *unsigned*, specificeert het regelnummer. Aan de hand van deze twee argumenten kan *showline()* een boodschap afdrukken, die specificeert om welk bestand en om welke regel het gaat. Het derde argument is een *FILE* *, die de stream aanduidt. Dit argument wordt gebruikt om tekens uit de stream naar het scherm te schrijven. Het vierde argument tenslotte is een *long*. Deze waarde specificeert waar de te tonen regel begint.

Opgave 40.

Stel de functie *compare()* samen aan de hand van de bovengegeven beschrijving. Merk op dat de laatste herschrijving het probleem (nog) niet exact weergeeft. Zorg ervoor dat op de juiste plaats functie *showline()* wordt aangeroepen (en met de juiste argumenten). Knoop vervolgens het hele programma aaneen, en test het programma.

7.5.3 Het gebruik van de functie open()

In tegenstelling tot de functie *fopen()* wordt bij het openen van handles door middel van de functie *open()* de modus operandi niet met behulp van een modus-string maar met behulp van een *int* waarde opgegeven. Dit is geïllustreerd in listing 20. De *int* waarde, die gewoonlijk samengesteld wordt door verschillende constanten met elkaar te 'or'-en, bestaat uit een aantal bits die de modus specificeren. De constanten die gebruikt worden om deze modus-vlag samen te stellen zijn weergegeven in figuur 7.6.

Constante	*Betekenis*
O_APPEND	bijschrijven mag aan het einde van het bestand
O_BINARY	het bestand wordt in binaire modus geopend
O_CREAT	het bestand wordt aangemaakt als het niet bestaat
O_RDONLY	het bestand mag alleen gelezen worden
O_RDWR	het bestand mag gelezen en beschreven worden
O_TEXT	het bestand wordt in tekst modus geopend
O_TRUNC	het bestand wordt (opnieuw) aangemaakt
O_WRONLY	het bestand mag alleen beschreven worden

Figuur 7.6: Constanten gebruikt bij de functie *open()*.

Opgave 41.

Zoek de exacte beschrijving van de constanten die in figuur 7.6 genoemd zijn op. Construeer vervolgens een functie *str2flag()* die één argument, een modus-string, heeft. De functie retourneert een *int* waarde die als vlag bij een aanroep van *open()* gebruikt kan worden. Een voorbeeld van een aanroep is:

handle = open ("data.txt", str2flag ("r+b"));

De functie retourneert een vlagwaarde, samengesteld uit de constanten die in figuur 7.6 geïllustreerd zijn.

Hoofdstuk 8

Pointers

8.1 Pointeroperatoren

In hoofdstuk 3 zijn twee operatoren genoemd die tot nu toe niet zijn behandeld. Het zijn de volgende twee pointeroperatoren:

- $*$, de *inhoud-van* operator en
- &, de *adres-van* operator.

Deze operatoren zijn qua notatie identiek aan de vermenigvuldigings- en *bitwise and*-operatoren, maar de pointeroperatoren zijn unaire operatoren en hebben een hogere prioriteit dan alle binaire operatoren (zie ook bijlage A.4).

Pointeroperatoren worden gebruikt bij pointervariabelen om de locatie van variabelen (of functies) in het geheugen te achterhalen, dan wel om de inhoud van variabelen op indirecte wijze te bereiken of om functies indirect aan te roepen. Men spreekt ook wel van *indirectie*. Pointers zijn een bijzonder krachtig gereedschap, waardoor het geheugen van de computer uitermate flexibel kan worden bewerkt en bereikt. Pointers worden bij voorbeeld gebruikt om een functie in staat te stellen een willekeurige variabele ergens in het geheugen direct te bereiken, bij voorbeeld om de waarde ervan te wijzigen. De pointer is dan een *verwijzing* naar een variabele ergens in het geheugen.

8.1.1 Definitie van pointervariabelen

Pointers worden gedefinieerd (en gedeclareerd) met behulp van de $*$ operator. Bij de definitie of declaratie wordt de asterisk uitgesproken als *pointer,* zoals in de definitie:

*int *int_pointer;*

waar de variabele *int_pointer* is gedefinieerd als een *int pointer*. Een dergelijke definitie kan ook anders worden uitgesproken door met de naam van de gedefinieerde variabele te beginnen. Bovenstaande definitie wordt dan uitgesproken als:

"int_pointer is een pointer naar een int"

en de asterisk wordt dan dus uitgesproken als *"is een pointer naar een"*.

Pointervariabelen hebben, net als andere variabelen, waarden. Echter, de waarde van een pointervariabele is de locatie in het werkgeheugen (het *adres*) van een andere variabele. Het *type* van die andere variabele is het type dat gebruikt is bij de definitie van de pointervariabele. Pointer variabelen bevatten dus verwijzingen naar waarden die ergens in het geheugen zijn opgeslagen.

8.1.2 Waardetoekenning

Pointervariabelen hebben als waarde een locatie in het geheugen. Zo wijst een *int* pointervariabele naar een *int* waarde ergens in het geheugen en

een *char* pointervariabele naar een *char* waarde ergens in het geheugen. In praktisch alle gevallen dat pointers worden gebruikt, wijst de pointervariabele naar een variabele. De plaats in het geheugen van *elke* variabele wordt verkregen met behulp van de *adres-van* operator &. Om een pointervariabele te voorzien van het adres van een variabele kan gebruik worden gemaakt van de *adres-van* operator. In het volgende voorbeeld is dat toegelicht. De variabele *value* is een *int* variabele, terwijl *value_ptr* een pointervariabele is die het adres zal krijgen van een *int* waarde ergens in het geheugen. In het volgende voorbeeld is dat het adres van de variabele *value:*

```
int
    value,
    *value_ptr;

value = 10;
value_ptr = &value;
```

Merk op dat het adres van *value* wordt verkregen door middel van de & operator. Dit adres wordt toegekend aan de variabele *value_ptr,* die daarna *wijst* op de plaats van variabele *value*. We zeggen:

"value_ptr *wijst op* value"

Merk op dat *value_ptr* nu inderdaad wijst op een plaats in het geheugen waar zich een *int* waarde bevindt. In *value* is immers een *int* opgeslagen. In figuur 8.1 is een illustratie gegeven van een waardetoekenning aan een pointervariabele. Het is een klassiek voor-na plaatje. Voor de waardetoekenning is niet bekend waar *value_ptr* op wijst, na de waardetoekenning wijst *value_ptr* op *value*.

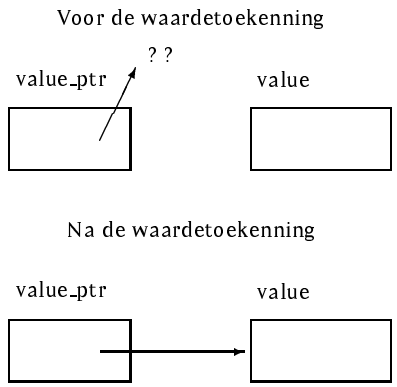
De waarde waarop een pointervariabele wijst kan bereikt worden met behulp van de *inhoud-van* operator *. In het volgende voorbeeld wordt de waarde waarop *value_ptr* wijst afgedrukt.

```
int
    value,
    *value_ptr;

value = 10;
value_ptr = &value;

printf("value_ptr wijst op de waarde %d\n", *value_ptr);
```

Omdat *value_ptr* het adres van *value* bevat en *value* de waarde 10 bevat wordt de waarde 10 door de *printf()* functie afgedrukt. De expressie **value_ptr* wordt uitgesproken als:

"de inhoud van value_ptr"

of ook wel als:



Figuur 8.1: Waardetoekenning aan *value_ptr*.

"de inhoud van datgene waar value_ptr op wijst"

Net zoals variabelen een waarde toegekend kunnen krijgen, kunnen waarden ook toegekend worden via pointervariabelen. Wanneer het om de waarde van een variabele gaat kan ook de *inhoud-van* operator in combinatie met een pointervariabele worden gebruikt. Zo kan ook de waarde van een variabele via een pointervariabele worden gewijzigd, zoals geïllustreerd in het volgende voorbeeld.

```
int
    value,
    *value_ptr;

value = 10;
value_ptr = &value;

// value is nu 10, *value_ptr dus ook

*value_ptr = 18;

// *value_ptr is nu 18, value dus ook
```

Waardetoekenning aan variabelen buiten functies

In 5.2 is opgemerkt dat functies hun argumenten via een call by value proces aangeboden krijgen. Dat heeft tot gevolg dat, wanneer een variabele als argument aan een functie wordt doorgegeven, de functie slechts een kopie krijgt van de waarde van de variabele op het moment dat de functie wordt aangeroepen. De aangeroepen functie kan dan wel de waarde van een parameter wijzigen, maar dat heeft dezelfde reikwijdte als het wijzigen van een lokale variabele. In **C** zijn parameters slechts handig geïnitialiseerde lokale variabelen.

Wanneer een functie een globale variabele moet wijzigen dan leidt dat niet tot problemen, omdat de globale variabele immers voor elke functie bereikbaar is. Wanneer een functie een lokale variabele van een andere functie moet kunnen wijzigen, dan kan dat vaak erg goed gebeuren via de return-waarde van de aangeroepen functie, zoals dat vaak gebeurt bij het gebruik van de functie *realloc()*. De aanroepende functie geeft de waarde van een pointervariabele door aan *realloc(),* en gebruikt de return-waarde van *realloc()* om de waarde van de pointervariabele te herzien. Soms kan dat niet, omdat de return-waarde een andere interpretatie heeft. Bij de standaard functie *fread()* wordt het aantal gelezen data-elementen geretourneerd, en wordt bovendien een buffer gevuld. De manier om een array door een functie te laten wijzigen is al genoemd: door de naam van het array aan een functie door te geven wordt de beginlocatie doorgegeven en de functie kan de array-elementen bereiken via de desbetreffende parameter, die als pointer naar een variabele van het type van het doorgegeven array is gedefinieerd. Een *int* array dat aan een functie wordt doorgegeven leidt tot een *int* ∗ als parameter.

Hetzelfde proces wordt gevolgd wanneer functies variabelen buiten de functie moeten kunnen wijzigen. De desbetreffende parameter wordt gedefinieerd als een *pointer* parameter. Bij de aanroep van zo'n functie moet het *adres* van de te wijzigen variabele, in plaats van de *waarde* van de te wijzigen variabele, worden doorgegeven. In het volgende voorbeeld wordt een functie *change()* gedefinieerd met twee parameters: een *int* en een *int* pointer. Bij de aanroep worden een *int* variabele en het adres van een *int* variabele doorgegeven. Na de aanroep is de eerste variabele ongewijzigd, terwijl de tweede variabele wèl is gewijzigd. Doordat de functie het adres van de tweede variabele heeft ontvangen, kan de inhoud van de desbetreffende variabele via de *inhoud van*-operator *(∗)* worden gewijzigd.

```
void change(int parameter_1, int *parameter_2)
{
    parameter_1++;
    (*parameter_2)++;
}

void caller()
{
    int
        argument_1 = 5,
        argument_2 = 5;

    change(argument_1, &argument_2);

    // argument_1: still 5
    // argument_2: now 6.
}
```

Bij het gebruik van de standaardfunctie *scanf()* moeten variabelen die een waarde moeten krijgen om die reden dan ook niet zonder meer worden genoemd (zoals bij de functie *printf(),* maar moeten de adressen van de

variabelen worden doorgegeven. In het volgende statement wordt van de adres-operatoren gebruik gemaakt om de variabelen *argument_1* en *argument_2* een waarde te geven via de functie *scanf()*.

```
scanf("%d%d", &argument_1, &argument_2);
```

Merk op dat de naam van een array hetzelfde is als het adres van het eerste element van het array. Om een *char* buffer *buf[80]* te vullen, kan bijvoorbeeld het volgende statement worden gebruikt:

```
scanf("%s", buf);
```

Opgave 42.

Beredeneer wat er gebeurt wanneer de functie *change()* in het vorige voorbeeld zou zijn aangeroepen als

```
change(argument_1, argument_2);
```

Functies die pointers retourneren

Functies kunnen waarden retourneren, maar ook pointers naar variabelen. Een paar voorbeelden daarvan zijn al genoemd. Zo levert de standaardfunctie *malloc()* een pointer naar een gealloceerd blok geheugen en levert de functie *fopen()* een pointer naar een *FILE*. In plaats van pointers naar variabelen te retourneren, zou een functie ook een parameter kunnen hebben waar een pointervariabele als argument wordt verwacht. Functies die pointers retourneren worden echter in het algemeen geprefereerd om een aantal redenen:

- Het niveau van indirectie is bij een functie die een pointer retourneert lager dan bij een functie die een pointervariabele verwacht. Wanneer een functie een pointer retourneert, kan die waarde direct worden toegekend aan de pointervariabele. Om de pointervariabele via een parameter van de functie een waarde te geven, moet het adres van de pointervariabele, dus een pointer naar een pointer, worden gebruikt.
- De functie kan geheugen alloceren en indelen op een manier die voor de aanroepende functie onmogelijk of volstrekt irrelevant is. Voor de aanroepende functie geldt dan dat alleen het resultaat van belang is en niet de manier waarop het resultaat tot stand is gekomen. Dit principe staat bekend als data hiding: het verbergen van irrelevante data-aspecten waar dat maar mogelijk is. Data-hiding kan worden beschouwd als het equivalent op data-niveau van het opsplitsen van een probleem in deelproblemen. In beide gevallen zijn de details van de implementatie in de diepte van de implementatie verborgen en worden hogere niveaus van de zorg voor de implementatie gevrijwaard.

Voorbeelden van een dergelijk gebruik van pointers die door functies worden geretourneerd zijn al gegeven: de functie *malloc()* retourneert een pointer naar een blok geheugen dat niet eerst door de aanroepende functie beschikbaar kan worden gemaakt, want dat is de taak van de functie *malloc()*. De functie *fopen()* retourneert een pointer naar een complexe datastructuur, waarvan de organisatie irrelevant is voor de aanroepende functies. Functies die met bestanden werken verwachten eenvoudig een pointer naar een *FILE* structuur. Alleen voor deze functies is de organisatie van de *FILE* structuur van belang. De aanroepende functies hoeven zich er niet mee bezig te houden.

- Het gebruik van functies die pointers naar variabelen retourneren leidt tot een soepel gebruik van de functie in expressies. De pointerwaarde *NULL* kan worden gebruikt om een fout te signaleren en constructies als:

```
if (!(block = malloc(size)))
{
    ...
}
```

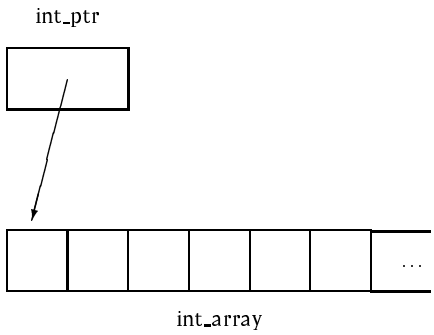
zijn dan ook standaard **C** idioom, in tegenstelling tot de variant met twee statements, die nodig zou zijn wanneer een functie *pmalloc()* zou bestaan die een pointer naar een pointervariabele als argument zou verwachten, zoals in het volgende voorbeeld.

```
pmalloc(&block);
if (!block)
{
    ...
}
```

8.1.3 Rekenen met pointers

Pointers worden vaak gebruikt om elementen van arrays te bereiken. In het volgende fragment wordt een array van integers gedefinieerd. Tevens wordt een pointer gedefinieerd die op het eerste element van het array wijst. Bij arrays kan dat op twee manieren. De eerste manier maakt gebruik van de *adres-van* operator: *int_ptr* krijgt als waarde het adres van *int_array[0]*. De tweede manier maakt gebruik van de wetenschap dat de *naam* van een array per definitie de beginlocatie van dat array is en dus gelijk is aan het adres van *int_array[0]*. De pointer variabele *int_ptr* kan dus als waarde de arraynaam krijgen en daarmee dus de beginlocatie van het array. Beide manieren leiden tot dezelfde waarde in *int_ptr*.

```
int
    int_array [100],
    *int_ptr;
```



Figuur 8.2: Een pointer naar een array-element.

```
// Manier 1: adresberekening via &-operator
int_ptr = &int_array [0];

// Manier 2: adresberekening via de arraynaam
int_ptr = int_array;
```

Ter illustratie geven we de situatie na de waardetoekenning aan *int_ptr* nog eens weer in figuur 8.2.

Met een pointervariabele kan enigszins worden gerekend. Er kunnen waarden bij worden opgeteld en vanaf getrokken. Alle optel- en aftrekoperatoren kunnen worden toegepast, dus ook de increment en decrement operatoren, en de compound assignments met de optel- en aftrekoperatoren. De **C** vertaler weet de omvang van het datatype van de pointervariabele (bij voorbeeld 2 bytes bij *int* variabelen en 4 bytes bij *longs,* en corrigeert de optelling of aftrekking met een factor die gelijk is aan de omvang van het gebruikte datatype in bytes. In het vorige voorbeeld werd aan *int_ptr* het adres van *int_array[0]* toegekend (we zeggen: *"int_ptr wijst op int_array[0]"*). Na de opdracht:

int_ptr += 5;

wijst *int_ptr* 5 elementen (*ints*) verder, dus op element *int_array[5]*.

Een ander prettig kenmerk van pointers is dat ze van elkaar kunnen worden afgetrokken. Dat pointers van elkaar kunnen worden afgetrokken, is het gevolg van algebra. Wanneer een constante bij een pointervariabele wordt opgeteld, ontstaat er opnieuw een pointerwaarde. Wanneer *beyond_ptr* en *before_ptr* beide pointers zijn, dan leidt het statement:

beyond_ptr = before_ptr + 5;

ertoe dat *beyond_ptr* vijf data-elementen verder wijst dan *before_ptr*. Wanneer we nu algebra toepassen en links en rechts van de waardetoekennings-

operator *before_ptr* aftrekken, ontstaat de volgende gelijkheid:

beyond_ptr - before_ptr == 5;

Dit kenmerk is bijzonder handig in situaties waarbij functies adressen van array-elementen retourneren. Ongeacht de omvang van de array-elementen kan dan de index van het array-element worden gevonden door de geretourneerde pointerwaarde af te trekken van de beginlocatie van het array.

Andere rekenkundige bewerkingen zijn niet toegestaan. Hoewel het logisch *lijkt* dat een pointer *ptr* die op het vijfde element van een *int* array wijst na:

*ptr * = 2;*

op het tiende element wijst, kan dit niet worden gegarandeerd, omdat de adresberekening niet werkt met een vast nulpunt. De pointer kan bij voorbeeld de waarde *0x1234* bevatten. Na vermenigvuldiging met 2 zou de pointer dan de waarde *0x2468* krijgen. Dat is een locatie in het geheugen die *0x1234* bytes, oftewel *0x91a ints* verder is[1].

8.2 Allocatie van geheugen

Het geheugen voor alle globale variabelen en voor de stack wordt door de vertaler gereserveerd en is beschikbaar wanneer een programma start. Het komt echter regelmatig voor dat bij het begin van een programma niet bekend is hoeveel geheugen precies door dat programma moet worden gebruikt. Een voorbeeld van zo'n programma is een tekstverwerker. Een tekstverwerker kan bestanden van verschillende grootte verwerken en kan ook nieuwe bestanden creëren. Wanneer zo'n programma wordt gestart is niet bekend hoeveel geheugen het programma zal moeten reserveren voor het tekstbestand. Dit probleem wordt in **C** programma's meestal opgelost door geheugen pas te alloceren wanneer het nodig is en weer vrij te geven wanneer het geheugen niet meer hoeft te worden gebruikt. Vrijwel altijd worden pointervariabelen gebruikt bij het alloceren van geheugen. Na de allocatie van de gewenste hoeveelheid geheugen wijst de pointervariabele op het gealloceerde blok geheugen en de pointervariabele kan in functies worden gebruikt alsof het een array was.

8.2.1 De functies malloc() en free()

De standaardfunctie om geheugen mee te alloceren is de functie *malloc()*. Het alloceren van geheugen tijdens de verwerking van een programma heet

1. Het meetniveau bij pointervariabelen is het zogeheten *interval* niveau, waarbij er geen sprake is van een vast nulpunt. Om vermenigvuldiging zinvol te laten zijn is een vast nulpunt noodzakelijk, zoals bij het zogeheten *ratio* niveau.

ook wel het *dynamisch* alloceren van geheugen. De functie *malloc()* is gedeclareerd in de **C** header file *malloc.h*. Er zijn twee manieren om het gealloceerde geheugen weer vrij te geven (te *dealloceren)*. De eerste manier bestaat uit het beëindigen van het programma. Dat betekent dat in een programma niet altijd hoeft te worden bijgehouden hoeveel en welk geheugen via *malloc()* is gealloceerd. Bij het beëindigen van het programma is het geheugen immers weer vrijgegeven. De tweede manier maakt gebruik van de standaard **C** functie *free()* om geheugen dat via *malloc()* is gealloceerd weer vrij te geven. Het gebruik van *malloc()* en *free()* wordt in het volgende voorbeeld geïllustreerd. Eerst wordt een blok geheugen gealloceerd, dan wordt het geheugen gevuld met byte-waarden *'m',* om vervolgens weer te worden vrijgegeven via de functie *free()*.

```
char *ptr;

ptr = malloc (100);
memset(ptr, 'm', 100);
free (ptr);
```

De functie *malloc()* kan ook *falen*. Als er niet meer genoeg geheugen beschikbaar is retourneert *malloc()* de waarde *NULL* in plaats van het adres van het gealloceerde blok. Het is dan ook zaak *altijd* de return-waarde van *malloc()* op de waarde *NULL* te testen. Wanneer dat niet gebeurt en *malloc()* retourneert *NULL* dan wordt in feite niet-gealloceerd geheugen gebruikt. In het gunstigste geval geeft het programma na afloop een foutmelding als:

"null pointer assignment"

maar het kan ook gebeuren, dat tijdens de verdere verwerking essentiële code wordt overschreven, waardoor het programma niet meer werkt of de computer opnieuw moet worden gestart (de computer *hangt* dan). Vaak is het rampzalig voor de verdere verwerking van een programma wanneer onvoldoende geheugen beschikbaar is. Veelal zal een programma dan ook voortijdig moeten worden beëindigd wanneer dat het geval is. De functie *xmalloc()* in listing 23 gedraagt zich als *malloc()* maar beëindigt het programma met een foutmelding wanneer onvoldoende geheugen beschikbaar is. Een (weinig aantrekkelijk) alternatief voor *xmalloc()* is om elke keer een test, zoals in *xmalloc()* wordt uitgevoerd, zelf uit te voeren. Wanneer in het bovenstaande voorbeeld *malloc()* wordt vervangen door *xmalloc()* wordt voorkomen dat het programma evenueel werkt met een *NULL* pointer. Merk op dat de return-waarde van *xmalloc()* een *void* pointer is. Een *void* pointer is een pointer die op allerlei datatypes kan wijzen. Een dergelijke pointerwaarde kan zonder meer worden toegekend aan pointervariabelen van een willekeurig datatype. De return-waarde van *malloc()* of *xmalloc()* kan dus worden toegekend aan een *char* pointer, een *int* pointer, of aan een willekeurig andere pointervariabele.

```
1           void *xmalloc(unsigned size)
2           {
3               char
4                   *tmp;
5
6               if (!(tmp = malloc (size)))
7               {
8                   puts ("\nNot enough memory");
9                   exit(1);
10              }
11
12              return (tmp);
13          }
14
```

Listing 23: De functie *xmalloc()*.

8.2.2 De grootte van het te alloceren geheugen

Bij de allocatie van geheugen moet rekening worden gehouden met de omvang van het datatype waarvoor geheugen wordt gealloceerd. Voor een array van n elementen van datatype t zijn:

$$n * sizeof(t)$$

bytes nodig. Bij een *char* array zijn dit dus n bytes, maar voor een *int* array zijn meer bytes nodig, omdat:

$$sizeof(int) > sizeof(char)$$

De omvang van een datatype in bytes kan het best via de *sizeof()* operator achterhaald worden, omdat dan geen gebruik wordt gemaakt van implementatie afhankelijke constanten (zoals 2 voor *sizeof(int)*). Programma's die op een dergelijke manier worden geschreven, kunnen relatief eenvoudig worden overgedragen (gesproken wordt over het porten[2] van een programma) van de ene computer naar een andere waar wellicht een andere hoeveelheid bytes voor een bepaald datatype geldt.

8.2.3 De functie realloc()

De functie *realloc()* kan worden gebruikt om een blok geheugen te realloceren naar een grotere of kleinere omvang. De functie *realloc()* heeft twee argumenten. Het eerste argument is een pointer naar het blok geheugen dat moet worden gerealloceerd. Het tweede argument is de nieuwe omvang van het blok geheugen in bytes. Het eerste argument mag ook *NULL* zijn. In dat geval werkt *realloc()* identiek aan *malloc()*.

2. De term *porten* in de uitdrukking *"Het porten van een programma"* is afgeleid van het Engelse werkwoord to port. Een programma dat makkelijk kan worden geport heeft een grote *portabiliteit*.

Het blok geheugen dat door het eerste argument wordt aangegeven wordt vergroot of verkleind, al naar gelang de nieuwe omvang. Wanneer het blok wordt vergroot van omvang *oldsize* naar *newsize,* blijft de inhoud van de eerste *oldsize* bytes ongewijzigd, terwijl de inhoud van de extra gealloceerde bytes ongedefinieerd is. Wanneer de omvang wordt verkleind tot *newsize,* is de inhoud van het nieuwe blok gelijk aan de eerste *newsize* bytes van het oude blok.

realloc() retourneert een pointer naar het nieuwe blok geheugen. Deze pointer kan *NULL* zijn wanneer de reallocatie mislukt, maar is ook *NULL* wanneer de geneste nieuwe omvang van het blok geheugen 0 bytes is. In dat geval is het oorspronkelijk gealloceerde blok geheugen vrijgegeven en gedraagt *realloc* zich dus als de functie *free()*.

Opgave 43.

Schrijf een functie *xrealloc()* naar analogie van de functie *xmalloc()*. De functie *xrealloc()* controleert of de reallocatie is gelukt en beëindigt het programma wanneer dat niet het geval is. Echter, wanneer de gevraagde nieuwe omvang van het blok geheugen 0 bytes is, moet de waarde *NULL* worden geretourneerd.

Ter illustratie van het gebruik van de functie beschouwen we het volgende probleem. In een programma wordt een van tevoren onbekend aantal *int* waarden verwerkt. Deze *int* waarden worden ingevoerd via het toetsenbord. Het einde van de invoer wordt bereikt zodra een regel niet met een *int* waarde begint (zoals bij een lege regel). De waarden moeten in een array van integers opgeslagen worden. Echter, de grootte van het array is niet van tevoren bekend. Het probleem kan als volgt worden geherformuleerd:

1. Doe het volgende zolang er een *int* waarde in een tijdelijke variabele kan worden ingelezen:
   2 Verhoog een teller van het aantal ingelezen elementen.
   3 Realloceer een blok geheugen naar het aantal *ints* dat wordt aangegeven door deze teller. Beëindig het programma wanneer de reallocatie mislukt.
   4 Ken de waarde van de tijdelijke variabele toe aan de laatste *int* positie van het gerealloceerde blok geheugen.

De implementatie van het probleem is gegeven in listing 24 en bestaat uit de functie *readint()*.

```
1     int *readint (int *countptr)
2     {
3         int
4             value,
5             *int_array = NULL,                    // Eerste realloc(): als malloc()
6
7         *countptr = 0;                            // Er zijn nog geen ints gelezen
8
```

```
9         while (scanf("%d", &value))              // Lees ints
10        {
11            ++*countptr;                         // Verhoog de teller
12
13            if                                   // Realloceer geheugen
14            (
15                !(int_array = realloc(int_array, *countptr * sizeof (int)))
16            )
17            {                                    // Het mislukt:
18                puts("\nNot enough memory");     // geef foutmelding
19                exit(1);                         // en stop het programma
20            }
21
22            int_array[*countptr - 1] = value;    // Het lukt: waardetoekenning
23        }
24        return (num_array);                      // Retourneer het array
25    }
26
```

Listing 24: De functie *readint():* een toepassing van *realloc()*.

Deze functie verwacht als argument een pointer naar een *int* waarin het uiteindelijke aantal ingelezen getallen zal worden opgeslagen en retourneert een *int* pointer naar de ingelezen getallen.

Opgave 44.

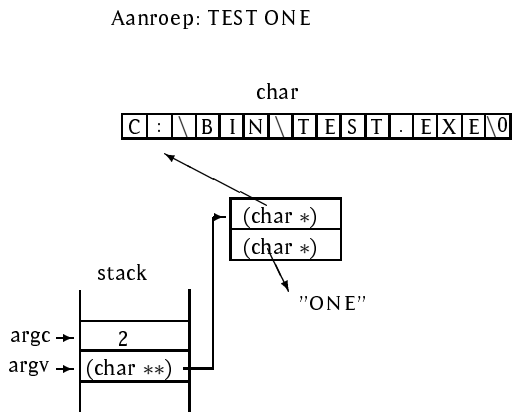
Stel dat op regel 11 van de functie *readint()* had gestaan

1. ***++countptr;**
2. ***countptr++;**
3. ***(*countptr)++;**
4. ***(countptr++);**

In welke gevallen zou de functie dan ook hebben gewerkt. In welke gevallen niet (beredeneer het antwoord door de werking van de statements te beschrijven).

8.3 Multiple indirection

Pointers worden niet alleen gebruikt om naar variabelen te wijzen. Er zijn ook pointers naar pointers, pointers naar pointers naar pointers enzovoorts. Bij variabelen die zijn gedefinieerd als meervoudige pointervariabelen wordt het uiteindelijke datatype niet via een enkele indirectie bereikt, maar via multipele indirectie. Bij een dubbele pointervariabele (een pointer naar een pointer) wordt eerst de pointer naar het uiteindelijke datatype bepaald en dan pas wordt het uiteindelijke datatype bereikt. Een voorbeeld van een dergelijke situatie hebben we in feite al gezien bij de *argv* parameter van de functie *main()*. De parameter *argv* is altijd gedefinieerd als een array van pointers naar strings. Echter, bij de aanroep van



Figuur 8.3: *argv* bij *main():* multipele indirectie.

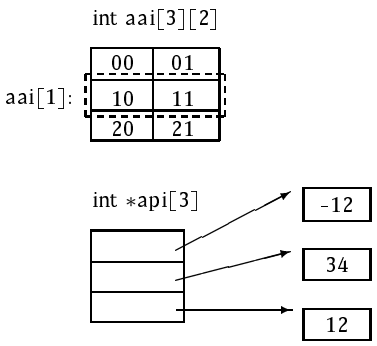
de functie *main()* door het **C** run-time systeem wordt het beginadres van het array van pointers naar strings doorgegeven aan de functie *main()*. Zodra een *adres* wordt doorgegeven is er sprake van een pointer. Wanneer dat adres het beginadres van een array is, dan is dat adres dus een pointer naar het eerste element van dat array. Wanneer het datatype waar de pointer naar wijst even tussen haakjes wordt gezet, dan is zo'n pointer bij een *int* array dus een "pointer naar een *(int)*". Wanneer zo'n array echter pointers naar *chars* bevat, dan is de pointer naar het eerste element van het array dus een "pointer naar een (pointer naar een *char*)". Dit laatste is een dubbele pointer. Een definitie van de parameter *argv* die preciezer aangeeft wat *argv* voor soort variabele is, is dus *char **argv*. In figuur 8.3 is de situatie bij de aanroep van de functie *main()* in beeld gebracht. Op de stack staan onder andere *argc* en *argv,* en *argv* wijst naar het eerste element van een array van pointers naar strings. De parameter *argv* is een pointer naar een pointer naar een *char*.

8.3.1 Arrays van pointers

Wanneer de elementen van een array pointers zijn (zoals de pointers waar *argv* naar wijst) is er sprake van een array van pointers naar een of ander datatype. Een array *api* van 3 pointers naar *ints* wordt gedefinieerd als:

```
int *api[3];
```

Het object is hier een geheugenblok van 3 pointers, waarbij elke pointer naar een *int* waarde wijst. Het object heet *api*. Wanneer de vertaler deze definitie tegenkomt, worden de operatoren om de naam overeenkomstig hun associativiteit en prioriteit geanalyseerd. De operatoren zijn



Figuur 8.4: Een array van pointers en een tweedimensionaal array.

de pointer-operator '*' en de array-operator '[]'. De array-operator heeft een hogere prioriteit en dus is *api* in eerste instantie een array. Wat voor een array? Een array van pointers naar *int* waarden. Het array *api* wordt geïllustreerd in figuur 8.4. De elementen van het array *api* zijn zelf pointers, die naar *int* waarden wijzen. Het array *api* is wezenlijk anders opgebouwd dan een array van een array van *ints,* dat bijvoorbeeld zou kunnen zijn gedefinieerd als:

int aai[3][2];

Het array *aai* is een tweedimensionaal array en elk array element is zelf een *int,* die via een dubbele index wordt bereikt. Met de notatie *aai[1]* wordt geen variabele aangeduid, maar een locatie in het geheugen waar de twee *ints aai[1][0]* en *aai[1][1]* worden aangetroffen (zie figuur 8.4).

Arrays met pointers naar *chars* komen regelmatig voor. Elk element van zo'n array is een *char* pointer en elke individuele *char* pointer wijst op een *char* variabele. De *char* pointer wijst doorgaans, maar niet noodzakelijk, op het eerste element van de string. Een klassiek voorbeeld in dit verband is de functie *strcpy()* waarmee een string *source* naar een string *destination* wordt gekopieerd. In listing 25 wordt de functie *strcpy()* gegeven.

De functie *strcpy()* retourneert *destination*. Het opmerkelijke aan de functie is de *while*-lus. In de expressie van het *while* statement krijgt het element waar *destination* op wijst de waarde van het element waar *source* op wijst. Echter, ten gevolge van de postfix increment-operator zal zowel *source* als *destination* worden opgehoogd nadat de waarden zijn gekopieerd. Bovendien is de laatste waarde die wordt gekopieerd de *ascii-z* van de *source* string. In de *while* conditie wordt telkens op de laatst gekopieerde waarde getest. Omdat de *ascii-z* de waarde 0 is, is het *while*-

```
1           char *strcpy(char *destination, char *source)
2           {
3               char
4                   *ret;
5
6               ret = destination;
7               while (*destination++ = *source++)
8                   ;
9               return (ret);
10          }
```

Listing 25: De stringcopy functie *strcpy()*.

statement voltooid nadat de *ascii-z* is gekopieerd. Op dat moment is ook de string *source* gekopieerd naar *destination*. Merk op dat de *char* pointers *source* en *destination* door de increment-operatie steeds van waarde wisselen en dus niet steeds op hetzelfde teken zullen wijzen.

De *argv* parameter leent zich ook goed voor de illustratie van een aantal kenmerken van arrays van pointers. We geven de volgende voorbeelden:

- *argv* is een variabele die een pointer naar een *char* pointer bevat. Dat achter de eerste *char* pointer nog andere volgen, doet niets af aan het feit dat *argv* simpelweg het adres van een *char* pointer bevat. Na *argv++* is *argv* toegenomen en wijst dus op de tweede *char* pointer. Dus:

  $$argv[i] == *(argv + i)$$

- Evenzo kunnen individuele tekens van strings worden bereikt. Wanneer *argv[i]* op het eerste teken van string *i* wijst, dan is *argv[i][j]* teken *j* van string *i*. Hetzelfde teken wordt bereikt met de volgende twee statements:

  $$(*(argv + i))[j]$$

  en:

  $$*(*(argv + i) + j)$$

- Door middel van waardetoekenningen kunnen pointers op andere strings wijzen, of tekens van waarde veranderen. Na het statement:

  $$argv += 3;$$

  wijst *argv* niet meer op de eerste string, maar op de vierde.
  Na het statement:

  $$*(argv + 2) += 3;$$

  Wijst de pointer op de derde string niet meer op het eerste teken van die string, maar op het vierde. *argv + 2* duidt de plaats van de

derde *char* pointer aan, *(argv + 2)* is de inhoud van het aangeduide adres en is dus de derde *char* pointerwaarde, waar de waarde 3 bij wordt opgeteld.
Na het statement:

((argv + 2) + 3) = 'a';*

wordt dus eerst het adres van het vierde teken van de derde string bepaald *(*(argv + 2) + 3)*. Aan de inhoud van dat adres wordt vervolgens de waarde *'a'* toegekend.

- Strings kunnen snel worden omgewisseld door de pointers naar de strings om te wisselen. In de functie *strsort()* in listing 26 wordt daarvan gebruik gemaakt. Het is de bedoeling strings in alfabetische volgorde te sorteren. Het sorteren van pointers naar strings[3] is vaak eleganter (en sneller) dan het sorteren van de data zelf. Immers, bij het sorteren van strings zouden de tekens in de strings zelf verwisseld moeten worden. Als we pointers naar strings sorteren, hoeven we alleen pointerwaarden te verwisselen.
  Het sorteeralgoritme dat in de functie *strsort()* wordt gebruikt, is een eenvoudige, lineaire omwisseling (swapping): het is niet het snelste sorteeralgoritme, maar wel eenvoudig te doorzien.
  De functie *strsort()* maakt gebruik van de functie *strcmp()* die een negatieve waarde retourneert als de eerste string in het alfabet voor de tweede string komt, een positieve waarde retourneert als de eerste string in het alfabet na de tweede string komt en 0 retourneert als beide strings aan elkaar gelijk zijn.

```
1    void strsort (int nstrings; char **s)
2    {
3        int
4            swapped = 1,
5            i;
6        char
7            *tmp;
8
9        while (swapped)                          // niet gesorteerd ?
10       {
11           swapped = 0;                         // neem aan: gesorteerd
12           for                                  // inspecteer alle strings
13           (
14               i = 0;
15                   i < nstrings - 1;            // tot aan de laatste
16                       i++
17           )
18               if (strcmp (s[i], s[i+1]) > 0)   // vergelijk met volgende
19               {                                // omwisselen indien nodig
```

3. In het voorbeeld worden strings gesorteerd. In het algemeen kan de benadering ook heel goed worden gevolgd bij andere datatypes. In de praktijk worden pointers naar data gebruikt wanneer de data-elementen zelf omvangrijk zijn.

```
20                      tmp   = s[i];                    // via 'tmp'
21                      s[i]  = s[i + 1];
22                      s [i+1] = tmp;
23                      swapped++;                       // strings verwisseld
24                  }
25          }
26      }
```

Listing 26: De stringsorteer functie *strsort()*.

Opgave 45.

Schrijf een programma *sortarg.c* dat gebruik maakt van de genoemde functie *strsort()* en de argumenten, die aan een programma mee worden gegeven, in gesorteerde volgorde vermeldt. Aanwijzing: geef *argc* en *argv* vanuit *main()* door aan *strsort()* en druk dan alle strings van *argv* af.

Opgave 46.

Schrijf een functie met als prototype:

*char *progname(char *);*

die een stringargument meekrijgt van de volledige naam van een programma. De functie haalt de extensie van het programma af en retourneert het eerste teken van de naam van het programma.
Een aanroep als

puts(progname("C:\\BIN\\TEST.EXE"));

laat op het beeldscherm dus zien

TEST

Aangenomen mag worden dat de volledige naam van het programma een extensie en een directory specificatie heeft.

8.3.2 Pointers naar functies

Pointers kunnen naar variabelen, maar ook naar functies wijzen. Wanneer een pointer naar een variabele wijst, bevat de pointer het adres van de variabele in het geheugen. Een dergelijke pointer zal doorgaans op een locatie in het data segment wijzen. Wanneer een pointer naar een functie wijst, bevat de pointer het adres van de functie, doorgaans een locatie in het code segment. In 8.1.3 is opgemerkt dat de naam van een array gebruikt kan worden als de aanduiding voor het beginadres van dat array. Analoog kan de naam van een functie gebruikt worden als de aanduiding voor het beginadres van de functie. Let op de volgende overeenkomst:

	adres:	gebruik of aanroep
array	array_naam	array_naam[index]
functie	functie_naam	functie_naam(argumenten)

Een pointer naar een functie wordt gedefinieerd als het prototype van het soort functies waarop de pointer moet kunnen wijzen, waarbij de functienaam uit het prototype wordt vervangen door een definitie van een pointervariabele.

Wanneer een pointervariabele moet worden gedefinieerd voor een pointer naar een functie die een *int* retourneert en geen parameters heeft, dan is een prototype van een dergelijke functie:

int intfun(void);

Wanneer *funptr* een pointer naar een dergelijke functie is, dan wordt *funptr* gedefinieerd als:

*int (*funptr)(void);*

De haakjes zijn belangrijk, omdat de pointer-operator een lagere prioriteit heeft dan de haakjes die gebruikt worden voor de parameterlijst van de functie. Volgens de definitie van *funptr* is er in eerste instantie sprake van een pointer, omdat de haakjes om **funptr* ervoor zorgen dat dat deel van de definitie door de vertaler het eerst wordt ontleed. Pas dan wordt de rest ontleed en blijkt *funptr* een pointer naar een functie te zijn die een *int* retourneert. Wanneer de haakjes om **funptr* worden weggelaten, staat er:

*int *funptr(void);*

en is *funptr* het prototype van een *functie funptr* die een pointer naar een *int* oplevert.

Wanneer een pointer naar een functie eenmaal het adres bevat van een functie, kan de naam van de pointervariabele worden gebruikt op plaatsen waar anders de naam van een functie zou hebben gestaan. Bij het aanroepen van functies via pointervariabelen vervangt de naam van de pointervariabele dus de naam van de functie. Let opnieuw op de analogie tussen het gebruik van pointers naar een array en pointers naar functies in het volgende voorbeeld:

```
int
    fun(void),                  // functieprototype
    array[5],                   // array
    *int_ptr,                   // pointer naar een int
    (*fun_ptr)(void);           // pointer naar een functie

// statements:

int_ptr = array;                // int_ptr wijst op het array
fun_ptr = fun;                  // fun_ptr wijst op de functie

array[5]++;                     // het gebruik van het array
fun();                          // het aanroepen van de functie

int_ptr[5]++;                   // arraygebruik via int_ptr
fun_ptr();                      // functieaanroep via fun_ptr
```

Pointers naar functies worden regelmatig gebruikt als parameters van functies. In listing 27 wordt het gebruik van dergelijke parameters

gedemonstreerd. Vanuit een functie *main()* wordt een functie *compare()* aangeroepen met drie argumenten:

- Het eerste argument is *argv*
- Het tweede argument is het adres van een functie *numbercompare*.
- Het derde argument is het adres van een functie *stringcompare*.

```
1           void main (int argc, char **argv)
2           {
3               compare(argv, numbercompare, stringcompare);
4           }
5
```

Listing 27: Pointers naar functies als argument.

De functie *compare()* bepaalt of *argv[1]* en *argv[2]* getallen representeren. Zo ja, dan wordt de functie *numbercompare()* aangeroepen via de doorgegeven functiepointer. Zo nee, dan wordt de functie *stringcompare()* aangeroepen via de doorgegeven functiepointer. In listing 28 is de functie *compare()* gegeven.

```
1           void compare(char **argv, void (*numbers)(int, int),
2                                     void (*strings)(char *, char *))
3           {
4               int
5                   lvalue,
6                   rvalue;
7
8               printf("Argument '%s'" , argv[1]);
9
10              if                                  // conversie naar
11              (                                   // getallen lukt ?
12                  sscanf(argv[1], "%d", &lvalue)
13                  &&
14                  sscanf(argv[2], "%d", &rvalue)
15              )                                   // zo ja:
16                  numbers(lvalue, rvalue);        // vergelijk getallen
17              else                                // zo nee:
18                  strings(argv[1], argv[2]);      // vergelijk strings
19
20              printf(" argument '%s'\n", argv[2]);
21          }
22
23
```

Listing 28: Pointers naar functies als parameter.

Via de functie *sscanf()* wordt bepaald of de argumenten in *argv[1]* en *argv[2]* kunnen worden opgevat als getal. Zo ja, dan worden de getallen doorgegeven aan de functie waar *numbers* op wijst. Zo nee, dan wordt de

functie aangeroepen waar *strings* op wijst. Laatstgenoemde twee functies zijn op zich uitermate eenvoudig en staan vermeld in listing 29. De functie *stringcompare()* maakt gebruik van de standaardfunctie *stricmp()* die de inhoud van twee strings vergelijkt zonder het verschil tussen hoofd- en kleine-letters serieus te nemen.

```
1           void numbercompare(int lvalue, int rvalue)
2           {
3               if (lvalue < rvalue)
4                   printf("is kleiner dan");
5               else if (lvalue == rvalue)
6                   printf("is gelijk aan");
7               else
8                   printf("is groter dan");
9           }
10
11          void stringcompare(char *lstring, char *rstring)
12          {
13              int
14                  result;
15
16              if ((result = stricmp(lstring, rstring)) < 0)
17                  printf("komt in het alfabet voor");
18              else if (result == 0)
19                  printf("is gelijk aan");
20              else
21                  printf("komt in het alfabet na");
22          }
```

Listing 29: Via pointers aangeroepen functies.

Opgave 47.

Stel het programma *cmpargs.c* samen uit listings 27, 28 en 29. Zorg ervoor dat functies gedefinieerd zijn voordat ze worden gebruikt. De functie *strcmp()* is gedeclareerd in de header file *string.h,* en moet aan het begin van het programma via een *#include* opdracht worden ingelezen. Wat levert het programma wanneer het als volgt wordt gestart:

1. ***cmpargs 1 2***
2. ***cmpargs een 2***
3. ***cmpargs 1 twee***
4. ***cmpargs een twee***
5. ***cmpargs Een twee***
6. ***cmpargs string STRING***

Sorteren en zoeken

Het nut van pointers naar functies wordt duidelijk wanneer functies als *qsort()* of *lsearch()* gebruikt worden. Functie *qsort()* is een functie die een array zeer snel kan sorteren. Functie *lfind()* zoekt een element op in een ongesorteerd array. Om dergelijke functies algemeen toepasbaar te laten zijn, moet het gebruikte datatype van de elementen van het te bewerken

array niet vastliggen in de code van dergelijke functies, maar moet deze informatie via argumenten aan de functies worden doorgegeven. Onder andere moet de manier worden doorgegeven volgens welke de sorteerfunctie kan bepalen of twee array-elementen al dan niet zijn gesorteerd en moet de manier worden doorgegeven volgens welke de zoekfunctie kan bepalen of een te zoeken element zich al dan niet in het array bevindt. Het doorgeven van verschillende manieren aan functies kan prachtig gerealiseerd worden met behulp van pointers naar functies. De gewenste manier wordt geïmplementeerd in een functie en het adres van de functie wordt doorgegeven aan een andere functie waarin het algoritme, om bijvoorbeeld te sorteren of te zoeken, is vastgelegd.

In de header file *search.h* zijn de prototypes van de functies[4] *qsort()* en *lfind()* gedeclareerd.

```
void qsort(void *base,
           unsigned num, unsigned width,
           int (*compare)(void *elem1, void *elem2));
void *lfind(void *key, void *base,
           unsigned *num, unsigned width,
           int (*compare)(void *key, void *elem));
```

De functie *qsort()* verwacht respectievelijk een pointer naar het array dat moet worden gesorteerd, het aantal elementen van het array, de omvang in bytes van de elementen van het array en het adres van een functie waarmee twee elementen van het array kunnen worden vergeleken. Deze functie krijgt zelf twee argumenten, namelijk de adressen van de twee array-elementen die moeten worden vergeleken. Vergeleken met *qsort()* heeft de functie *lfind()* nog een extra eerste argument: het adres van de variabele van hetzelfde type als de array-elementen waarvan de waarde moet worden gezocht in het array. Daarnaast is de omvang van het array niet direct als waarde doorgegeven, maar wordt een pointer verwacht naar een variabele waarin het aantal elementen van het array is opgeslagen. De functie *lfind()* retourneert *NULL* wanneer de te zoeken variabele niet in het array voorkomt en retourneert het adres van het gevonden array-element wanneer dat wel het geval is. De index van het array-element wordt in dat laatste geval gevonden als het verschil tussen de return-waarde en het beginadres van het array.

Merk op dat de *compare* pointer op een functie moet wijzen die als argumenten de adressen van de te vergelijken array-elementen krijgt. Wannneer het array zelf uit pointers bestaat (zoals bij *argv),* dan zijn de parameters van de hulpfuncties dus adressen van pointers en dus pointers naar pointers. We geven ter illustratie een toepassing van de functies *qsort()* en *lfind()* in listing 30. In de eerste toepassing worden de strings

4. Ter wille van de behandeling van de functies in de tekst zijn de parameters in de prototypes van de functies van een naam voorzien, hoewel dat bij de declaratie niet strikt noodzakelijk (maar ook niet fout) is.

van het array *argv* gesorteerd door de pointerwaarden in *argv* te sorteren met behulp van *qsort()*. In de tweede toepassing wordt de string *"opa"* gezocht in het array *argv* met behulp van de functie *lfind()*.

```
1           #include <stdio.h>
2           #include <string.h>
3           #include <search.h>
4
5           char
6               *opa = "opa";
7
8           int indirect_strcmp(char **el_1, char **el_2)
9           {
10              return (strcmp(*el_1, *el_2));
11          }
12
13          void main(int argc, char **argv)
14          {
15              char
16                  **cp;
17
18              qsort(argv, argc, sizeof(char *),    // sorteer de strings
19                      indirect_strcmp);
20
21              if
22              (
23                  cp = lfind(&opa, argv,           // zoek "opa"
24                        &argc, sizeof(char *),
25                        indirect_strcmp)
26              )                                    // gevonden:
27                  printf("'opa' na sorteren op index"
28                         "%d\n", cp - argv);
29              else                                 // niet gevonden:
30                  puts("'opa' is geen argument");
31          }
```

Listing 30: Sorteren en zoeken met *qsort()* en *lfind()*.

Opgave 48.

In listing 30 worden de strings in *argv* van voor naar achter in het alfabet gesorteerd. Schrijf een programma dat de strings van achter naar voor sorteert en dan de gesorteerde strings afdrukt. Aanwijzing: gebruik een hulpfunctie waarin de return-waarde van *strcmp()* wordt geretourneerd, terwijl *strcmp()* met verwisselde argumenten wordt aangeroepen.

8.4 Frequente fouten

De programmeertaal **C** staat een zeer flexibel gebruik van pointers toe. De flexibiliteit heeft niet alleen betrekking op de syntaxis, waardoor pointers naar allerlei soorten data en functies kunnen worden gedefinieerd, maar

heeft ook betrekking op de *semantiek:* bij onnauwkeurig gebruik van pointers kunnen onverwachte (en soms zeer onprettige) situaties optreden. Een aantal van dergelijke fouten komt redelijk vaak voor. In dit deel van het boek wordt er nader op ingegaan.

8.4.1 Wilde pointers

Veel problemen ontstaan doordat met niet-geïnitialiseerde pointers wordt gewerkt. In het volgende voorbeeld wordt de waarde 100 toegekend aan de geheugenlocatie waar *pi* naar wijst. Maar, waar wijst *pi* naar?

```
int
    i,
    *pi;

i = 100;
*pi = i;
```

Het antwoord kan worden gegeven voor globale variabelen, die immers door de vertaler op 0 worden geïnitialiseerd. Het is echter niet de bedoeling om informatie naar een locatie waar een *NULL*-pointer op wijst te schrijven. Wanneer dat toch gebeurt is de run-time foutmelding:

"null pointer assignment"

nog het minst dramatisch. Deze foutmelding wordt door het **C** run-time systeem getoond nadat het programma is afgelopen. In dat geval werd tijdens de verwerking van een programma een waarde toegekend aan de locatie waar een *NULL* pointer op wees. Echter, er kunnen zich ook situaties voordoen die minder onschuldig zijn. Het geheugenmodel (zie hoofdstuk 12) dat wordt gehanteerd kan met zich meebrengen dat een *NULL*-pointer naar de allereerste geheugenlocatie van de computer wijst. Bij de familie van IBM personal computers heeft het schrijven naar een locatie waar een *NULL*-pointer op wijst tot gevolg dat essentiële informatie van het Dos operating system wordt overschreven. In die situatie komt de computer 'met gillende remmen' tot stilstand en opnieuw opstarten (rebooting) is het enige dat nog werkt.

8.4.2 Beschrijven van niet gealloceerd geheugen

Naast het gebruik van wild pointers is een andere vaak voorkomende fout dat pointers voorbij gealloceerd geheugen wijzen. Wanneer dan informatie naar de desbetreffende geheugenlocatie wordt geschreven, kunnen waarden van andere variabelen worden overschreven, of kan (opnieuw) informatie die eigendom is van het operating system worden overschreven.

Al in hoofdstuk 1 werd opgemerkt dat **C** ontwikkeld is als een taal voor programmeurs die primair in een produktie-omgeving werken. De verantwoordelijkheid voor het correct gebruik van de taal ligt bij de programmeertaal **C** bij de programmeurs en veel minder bij de vertaler of

bij het run-time systeem[5]. De afwezigheid van extensieve controles brengt een grote mate van vrijheid met zich mee, waardoor het mogelijk werd een operating system in **C** te schrijven, in plaats van in een assembleertaal[6]. Door de grote vrijheid is er ook een toename van de mogelijkheden om fouten te maken. Een vaak voorkomende fout bij het gebruik van gealloceerd geheugen is dat toch niet-gealloceerd geheugen wordt overschreven. We geven drie voorbeelden in listing 31.

```
1         char
2             string1[] = "opa",
3             string2[] = "en oma zitten op de bank";
4
5         void main()
6         {
7             int
8                  i,
9                  *ptr;
10            char
11                *cp;
12
13            puts(string1);
14            cp = string1;
15            cp += 3;
16            *cp = ' ';
17            puts(string1);
18
19            ptr = malloc(100 * sizeof(int));
20            for (i = 0; i <= 100; i++)
21                ptr[i] = i;
22
23            ptr = malloc(100);
24            for (i = 0; i < 100; i++)
25                ptr[i] = i;
26        }
```

Listing 31: Overschrijven van gealloceerd geheugen.

De eerste fout treedt op in regel 16. In regel 15 werd *cp* op het teken voorbij de *'a'* van *"opa"* gezet. De variabele *cp* wijst dan dus op de *ascii-z*. In regel 16 wordt op die plaats een spatie geschreven, waardoor in regel 17 de *puts()* functie de tekst:

"opa en oma zitten op de bank"

zal laten zien. In feite werd door het schrijven van de spatie informatie overschreven die niet zou moeten worden overschreven (namelijk het afsluitteken van een string), met als effect dat twee strings aan elkaar worden

5. Bij een taal als **pascal** worden normaliter controles uitgevoerd op het correct gebruik van de taal die veel omvangrijker zijn dan de controles die bij **C** worden uitgevoerd. Maar **pascal** is dan ook primair een *onderwijs*taal en veel minder een *produktie* taal.
6. Een assembleertaal is een symbolische machinetaal. In een assembleertaal worden individuele instructies van de instructieverzameling van de computer geprogrammeerd via een symbolische in plaats van puur numerieke notatie.

gekoppeld. Wanneer dat de bedoeling is dan is dat prima, maar het kan ook onbedoeld zijn, waardoor ongewenste effecten optreden.

Een tweede fout treedt op in het *for*-statement op regel 20. In regel 19 werden 100 *ints* gealloceerd. Daarmee zijn *ptr[0]* tot en met *ptr[99]* gedefinieerd. Een veel voorkomende fout is dat deze grens nèt wordt overschreden: een boundary error. Element *ptr[100]* is niet gealloceerd en wordt in het *for*-statement toch van een waarde voorzien, waardoor onbedoelde informatie wordt gewijzigd.

De derde fout is een variant op de tweede: bij de allocatie is verzuimd rekening te houden met de omvang van de data-elementen. De functie *malloc()* verwacht het aantal bytes dat moet worden gealloceerd. In plaats van ruimte voor 100 *ints* te alloceren, wordt door het statement op regel 23 ruimte voor 100 *chars* gealloceerd. In het *for*-statement op regel 24 wordt echter met elementen ter grootte van *ints* gerekend, omdat *ptr* nou eenmaal een pointer naar een *int* is. De geheugenoverschrijding is hier veel groter, en zal er bij het Dos operating system zeer waarschijnlijk toe leiden dat informatie over geheugen dat door Dos ten behoeve van het programma beschikbaar was gemaakt wordt overschreven. Zolang het programma actief is, is een dergelijke fout niet rampzalig. Echter, na afloop van het programma controleert Dos het geheugen en komt tot de ontdekking dat er iets niet klopt. De melding:

> *"Memory control blocks destroyed, cannot load COMMAND.COM, system halted."*

is het gevolg en de computer moet opnieuw worden gestart. Een onprettige zaak.

8.5 De Variable Definer: VD

Om het probleem van het definiëren van complexe variabelen te vereenvoudigen, wordt hier het programma *VD.EXE* (Variable Definer) ontwikkeld. Het programma *VD.EXE* helpt de programmeur bij het maken van een complexe variabele definitie, zoals:

> *"Wat is de definitie van een variabele* ptr *die een array van pointers naar functies is, waarbij de functies waarnaar gewezen wordt zelf pointers naar strings (dus: pointers naar pointers naar* chars*) retourneren.*

Het programma *VD.EXE* verwacht bij de aanroep drie argumenten:

- De *naam* van de te definiëren variabele.
- Een aanduiding van de aard van de complexiteit van de variabele. Hierbij wordt gebruik gemaakt van de drie letters 'a' (array van), 'f' (een functie, die iets retourneert) en een 'p' (pointer naar). De letters 'a', 'f' en 'p' kunnen bij herhaling worden opgegeven in elke combinatie die door de syntaxis van **C** is toegestaan.

	a	f	p
a	arrays van arrays	–	arrays van pointers
f	—	–	functies die pointers retourneren
p	—	pointers naar functies	pointers naar pointers

Figuur 8.5: Toegestane variabele definities.

- Het uiteindelijke datatype dat wordt bereikt (zoals *int* of *char*).

Om de definitie te vinden van de genoemde variabele *ptr* kan *VD.EXE* worden gestart als:

vd ptr apfpp char

Er wordt dan gevraagd naar de definitie van *ptr,* waarbij *ptr* een array (a) is van pointers (p) naar functies (f) die pointers (p) naar pointers (p) naar *char* opleveren. De definitie van zo'n variabele is:

*char **(*ptr[])();*

De in **C** toegestane definities van variabelen waarbij pointers, functies, of arrays zijn betrokken, zijn weergegeven in figuur 8.5[7].

8.5.1 De ontwikkeling van VD

Bij de ontwikkeling van het programma *VD.C* wordt als eerste de beschrijving van de werking van *VD* in de gebiedende wijs geformuleerd. Hoewel er in de beschrijving van het programma niet expliciet om wordt gevraagd, is het een goede gewoonte programma's zodanig te ontwikkelen, dat ze de gebruiker kort informeren over het doel van het programma wanneer dat zonder verdere argumenten wordt gestart. Die gewoonte zullen we hier ook volgen. De eerste herformulering van het probleem wordt dan:

1. Kontroleer of het aantal argumenten van het programma klopt.
   2. Zo niet, geef hulpinformatie.
   3. Zo ja, analyseer de gewenste definitie.

De controle of het aantal argumenten klopt kan met behulp van *argc,* terwijl de hulpinformatie eenvoudig met een *puts()* statement kan worden gerealiseerd. De analyse van het probleem is 't lastigst en wordt voorlopig overgelaten aan de functie *analyze(),* die als argument *argv* krijgt. De *main()* functie kan nu worden geconstrueerd:

7. Merk op dat pointers naar arrays volgens de syntaxis wel kunnen worden gedefinieerd. We raden het echter af, omdat zo'n pointer het adres van een array zou moeten krijgen. Echter, een array is al een adres, en dus een *waarde*. De adres-operator werkt niet op arrays, maar alleen op variabelen.

```
1           #include <stdio.h>
2
3           void main(int argc, char **argv)
4           {
5               if (argc != 4)                  // foutief aantal argumenten ?
6                   puts(
7                       "\nVD Version 1.00\n"
8                       "Usage:\n"
9                       "\tVD variable [a|f|p]+ datatype\n"
10                      "Where:\n"
11                      "\t\tvariable: the name of the variable to define.\n"
12                      "\t\t[a|f|p]+: at least one occurrence of\n"
13                      "\t\t\ta: 'an array of',\n"
14                      "\t\t\tf: 'a function returning,\n"
15                      "\t\t\tp: 'a pointer to'.\n"
16                      "\t\tdatatype: the name of the C datatype.\n");
17              else
18                  analyze(argv);
19          }
20  %
21
```

Listing 32: De functie *main()* van *VD.C*.

Vervolgens wordt de functie *analyze()* ontwikkeld. De herformulering van onderdeel 3 leidt tot de functie *analyze()*:

3.1. Plaats de variabelenaam in een realloceerbare string.
3.2. Breid de string uit met de variabeledefinities zoals aangegeven in *argv[2]*.
3.3 Vermeld het type *(argv[3])* gevolgd door de string die bij 3.2 is geconstrueerd.

Onderdeel 3.1 kan worden uitgevoerd met behulp van de functie *strdup()* die standaard bij de Microsoft vertaler wordt meegeleverd. De functie *strdup()* dupliceert een string naar een realloceerbare string. Wanneer de functie *strdup()* echter *NULL* oplevert, is er niet voldoende geheugen beschikbaar. In plaats van *strdup()* zonder meer aan te roepen, moet er dus worden getest op de return-waarde van *strdup()*. Het programma stopt met een foutmelding wanneer *strdup()* *NULL* retourneert.

Het uitbreiden van de definitiestring is het lastige onderdeel van de herformulering. De functie *expand()* wordt daarvoor in het leven geroepen. De argumenten van *expand()* zijn de definitiestring en *argv[2]:* de aanduiding van de gewenste definitie.

```
1           #include <malloc.h>
2
3           char
4               out_of_memory[] = "Out of memory";
5
```

```
6           void analyze(char **argv)
7           {
8               char
9                   *defstring;
10              // tekst: onderdeel 3.1
11              if (!(defstring = strdup(argv[1])))
12              {
13                  puts(out_of_memory);
14                  return;
15              }
16
17              // tekst: onderdeel 3.2
18              defstring = expand(argv[2], defstring);
19
20              // tekst: onderdeel 3.3
21              printf("%s %s\n", argv[3], defstring);
22          }
```

Listing 33: De functie *analyze()* van *VD.C*.

Merk op dat de variabele *out_of_memory* als een globaal array van *chars* is gedefinieerd. Op deze wijze kunnen verschillende functies van dezefde string gebruik maken.

De analyse van punt 3.2 leidt tot de derde herformulering en daarmee tot de definitie van de functie *expand()*. Bij de constructie van een variabeledefinitie moet echter rekening worden gehouden met mogelijke en onmogelijke definities en moet worden gelet op de lagere prioriteit van de pointeroperator. Wanneer het laatste verzoek een pointerspecificatie was en het huidige verzoek betreft een functie- of arraydefinitie, dan moet de pointerdefinitie worden beschermd door omhullende haakjes. Evenzo moet bij het verzoek om bijvoorbeeld een functiedefinitie worden gecontroleerd of het vorige verzoek wellicht een arraydefinitie of ook een functiedefinitie betrof. In dat geval wordt een onmogelijke definitie gevraagd en is de specificatie dus fout. Het is dus zaak bij te houden wat het laatste verzoek was. Het laatste verzoek wordt bijgehouden in de globale *int* variabele *last_request* die eenvoudig de waarde van het laatst verwerkte teken in *argv[1]* krijgt.

3.2.1. Zolang er nog tekens zijn in het definitieverzoek doe het volgende.
- 3.2.2. Wanneer het teken een *'a'* is, breidt de definitie zo mogelijk uit met een arrayspecificatie.
- 3.2.3. Wanneer het teken een *'f'* is, breidt de definitie zo mogelijk uit met een functiespecificatie.
- 3.2.4. Wanneer het teken een *'p'* is, breidt de definitie zo mogelijk uit met een pointerspecificatie.
- 3.2.5. Beëindig het programma met een foutmelding in alle andere gevallen.

3.2.6. Geef *last_request* de waarde van het juist verwerkte verzoek. Reduceer het definitieverzoek met één teken.
3.2.7. Retourneer de definitiestring.

Onderdeel 3.2.1. leidt tot een *while*-statement, waarin alle tekens van het verzoek worden verwerkt. Op grond van het waargenomen teken wordt één van de keuzes 3.2.2. tot en met 3.2.5. uitgevoerd. De natuurlijke oplossing hier is de *switch*. Onderdeel 3.2.5. is de default ingang van de *switch,* de overige onderdelen worden afgewerkt door functies die als argument de definitiestring krijgen en een nieuwe definitiestring retourneren. De functie *expand()* wordt dan:

```
1           #include <process.h>
2
3           int
4               last_request;
5
6           char *expand(char *request, char *defstring)
7           {
8               while (*request)                    // tekst: 3.2.1.
9               {
10                  switch(*request)
11                  {
12                      case 'a':                   // tekst: 3.2.2.
13                          defstring = array(defstring);
14                      break;
15
16                      case 'f':                   // tekst: 3.2.3.
17                          defstring = function(defstring);
18                      break;
19
20                      case 'p':                   // tekst: 3.2.4
21                          defstring = pointer(defstring);
22                      break;
23
24                      default:                    // tekst: 3.2.5
25                          puts("Error in definition request");
26                          exit(1);
27                  }
28
29                  last_request = *request++;      // tekst: 3.2.6
30              }
31              return (defstring);
32          }
```

Listing 34: De functie *expand()* van *VD.C*.

De functie *pointer()* is wellicht het eenvoudigst, omdat pointers naar pointers kunnen optreden en er bij het verzoek om een pointervariabele geen haakjes hoeven te worden geplaatst. De volgende herformulering leidt tot de definitie van de functie *pointer()* (zie listing 35):

3.2.4.1. Plaats een asterisk voor de definitiestring
3.2.4.2. Retourneer de nieuwe definitiestring.

Bij de implementatie van de functie *pointer()* worden de onderdelen 3.2.4.1. en 3.2.4.2. samengevoegd tot één statement. Onderdeel 3.2.4.1. wordt gerealiseerd met behulp van een functie *catenate()* die twee realloceerbare strings verwacht en een pointer naar de twee achter elkaar geplaatste strings retourneert. De definitie van *catenate()* stellen we nog even uit.

```
1           char *pointer(char *defstring)
2           {
3               return (catenate(strdup("*"), defstring));
4           }
```

Listing 35: De functie *pointer()* van *VD.C*.

De functie *array()* is iets complexer dan *pointer()* omdat extra haakjes moeten worden geplaatst wanneer het vorige verzoek een pointer betrof. Bovendien kunnen functies geen arrays retourneren. Wanneer het vorige verzoek een functie betrof, werd er dus een foutief verzoek gedaan. De volgende herformulering leidt tot de definitie van de functie *array()* (zie listing 36):

3.2.2.1. Wanneer het vorige verzoek een functie betrof, geef dan een foutmelding en beëindig het programma.
3.2.2.2. Wanneer het vorige verzoek een pointer betrof, plaats dan haakjes om de definitiestring.
3.2.2.3. Plaats rechte haken achter de definitiestring en retourneer de gewijzigde definitiestring.

```
1           char *array(char *defstring)
2           {
3               if (last_request == 'f')            // tekst: 3.2.2.1.
4               {
5                   puts("Functions don't return arrays");
6                   exit(1);
7               }
8
9               if (last_request == 'p')            // tekst: 3.2.2.2.
10                  defstring = priority(defstring);
11
12              return (catenate(defstring, "[]"));  // tekst: 3.2.2.3.
13          }
```

Listing 36: De functie *array()* van *VD.C*.

Het plaatsen van de haakjes om de definitiestring wordt overgelaten aan de functie *priority()* die ook bij de functie *function()* weer zal worden gebruikt. De functie *function()* is weer iets complexer dan *array()* omdat een

extra fout kan optreden: arrays die functies als elementen hebben of functies die functies retourneren bestaan niet in **C**. Wanneer het vorige verzoek een functie of array betrof werd er dus een foutief verzoek gedaan. De volgende herformulering leidt tot de definitie van de functie *function()* (zie listing 37):

3.2.3.1. Wanneer het vorige verzoek een array betrof, geef dan een foutmelding en beëindig het programma.
3.2.3.2. Wanneer het vorige verzoek een functie betrof, geef dan een foutmelding en beëindig het programma.
3.2.3.3. Wanneer het vorige verzoek een pointer betrof plaats dan haakjes om de definitiestring.
3.2.3.4. Plaats ronde haken achter de definitiestring en retourneer de gewijzigde definitiestring.

```
1           char *function(char *defstring)
2           {
3               switch (last_request)
4               {
5                   case 'a':                       // tekst: 3.2.3.1.
6                       puts("array elements are never functions");
7                       exit(1);
8
9                   case 'f':                       // tekst: 3.2.3.2.
10                      puts("functions don't return functions");
11                      exit(1);
12
13                  case 'p':                       // tekst: 3.2.3.3.
14                      defstring = priority(defstring);
15              }
16
17              return (catenate(defstring, "()")); // tekst: 3.2.3.4.
18          }
```

Listing 37: De functie *function()* van *VD.C*.

Tenslotte de twee hulpfuncties. De functie *priority()* plaatst vóór en achter de huidige definitiestring een haakje en retourneert de gewijzigde definitiestring. Met behulp van de functie *catenate()* wordt dit in één statement gerealiseerd. De functie *catenate()* bepaalt de lengte van de beide string-argumenten en maakt de eerste string zo lang dat beide strings achter elkaar kunnen worden geplaatst. De bepaling van de lengte van de strings gebeurt met behulp van de standaardfunctie *strlen()* die de lengte van de string zonder de afsluitende *ascii-z* retourneert. Bij de reallocatie moet dus rekening worden gehouden met de extra *ascii-z* waarde. De functie *catenate()* stopt het programma wanneer *NULL* argument wordt gekregen of wanneer de reallocatie mislukt. De beide functies zijn gegeven in listing 38.

```
1           #include <string.h>
2
3           char *catenate(char *left, char *right)
4           {
5               if
6               (
7                   !left                           // geen 'left' string
8                   ||
9                   !right                          // geen 'right' string
10                  ||
11                  !(left = realloc(left,          // geen reallocatie
12                              strlen(left) + strlen(right) + 1))
13              )
14              {                                   // stop het programma
15                  puts(out_of_memory);
16                  exit(1);
17              }
18
19              strcat(left, right);                // plaats 'right' na 'left'
20              free(right);                        // geef geheugen vrij
21
22              return (left);                      // retourneer 'left right'
23          }
24
25          char *priority(char *defstring)
26          {
27              return
28                  (
29                      catenate
30                      (
31                          strdup("("),
32                          catenate(defstring, strdup(")"))
33                      )
34                  );
35          }
```

Listing 38: De functies *priority()* en *catenate()*.

Hoofdstuk 9

Nieuwe types

Dit hoofdstuk gaat over het definiëren van en omgaan met andere datatypes dan de standaard types *char, int, float* en *double*. **C** kent twee keywords waarmee uit gepredefinieerde types nieuwe (samengestelde) types kunnen worden geconstrueerd. Dat zijn de keywords *struct* en *union*. Met behulp van het keyword *enum* kunnen symbolische namen worden toegekend aan *int* waarden. Tenslotte is het mogelijk om een naam aan een definitie van datatype te koppelen met behulp van het keyword *typedef*.

De genoemde keywords worden in het eerste deel van dit hoofdstuk besproken. In het tweede deel van het hoofdstuk worden twee programma's ontwikkeld waarin van de hier besproken keywords uitgebreid gebruik zal worden gemaakt.

9.1 De typedef

Met behulp van het keyword *typedef* kunnen nieuwe types van een naam worden voorzien. De syntactische regel voor het gebruik van het keyword *typedef* is het keyword *typedef* gevolgd door een definitie van een nieuw type, afgesloten door een puntkomma. De definitie van het nieuwe type gebeurt op dezelfde wijze als de definitie van een variabele. Het effect van het keyword *typedef* is echter dat de naam van de variabele wordt opgevat als de naam van het nieuwe type. De syntactische regel voor een *typedef* luidt dan ook:

typedef variabeledefinitie ;

```
1           typedef unsigned int WORD_;
2           typedef int (*PFI_)();
3           typedef FILE *PFILE_;
4
5           WORD_
6               u_int;
7           PFI_
8               fun_ptr;
9           PFILE_
10              stream;
11          int
12              add(int, int);
13
14          u_int = 50000;
15          fun_ptr = add;
16          stream = fopen("myfile", "r");
```

Listing 39: Het gebruik van *typedef*.

We geven een paar voorbeelden in listing 39:

- In de regels 1 tot 4 worden drie nieuwe typenamen gedefinieerd. Het type *WORD_*[1] kan worden gebruikt als aanduiding voor een *unsigned int,* het type *PFI_* kan worden gebruikt als aanduiding voor een pointer naar een functie die een *int* retourneert, het type *PFILE_* kan worden gebruikt als aanduiding voor een *FILE* pointer.
- In de regels 5 en 6 wordt de variabele *u_int* als een *WORD_,* en dus als een *unsigned int* gedefinieerd. De variabele krijgt een waarde in regel 14.
- In de regels 7 en 8 wordt de variabele *fun_ptr* als een *PFI_,* (dus als een pointer naar een functie die een *int* retourneert) gedefinieerd. De variabele *fun_ptr* krijgt in regel 15 als waarde het adres van de functie *add()* waarvan het prototype gegeven is in de regels 11 en 12.
- In de regels 9 en 10 wordt de variabele *stream* als een *PFILE_,* dus als een pointer naar een *FILE* gedefinieerd. De variabele *stream* krijgt in regel 16 als waarde het adres van de *FILE* structuur dat geretourneerd wordt door de functie *fopen()*.

Het keyword *typedef* wordt voornamelijk gebruikt in combinatie met de keywords *struct, union* en *enum,* die hierna zullen worden behandeld.

9.2 De struct

In **C** kunnen variabelen van mogelijk verschillende datatypes worden samengevoegd in een variabele door middel van een structuur *(struct)*. De variabelen die in de structuur worden samengevoegd heten de *velden* van de structuur. De variabele die met behulp van het keyword *struct* is gedefinieerd, heet een structuur, of kortweg *struct*. Een structuur zou bijvoorbeeld een veld *char name[80]* kunnen hebben, om een naam in op te slaan en een veld *int age* om een leeftijd in op te slaan. Een *struct* variabele wordt als volgt gedefinieerd:

```
struct [[tag]]
{
    variable definitions
}
    variables ;
```

In bovenstaande syntactische regel zijn de variable definitions definities van één of meerdere variabelen van één of meerdere datatypes. Met de variable definitions worden de velden van de structuur gedefinieerd. De variables achter het blok van de *struct* definitie is een lijst van één of meer

1. De namen van zelfgedefinieerde types worden geschreven in hoofdletters, waaraan een underscore is toegevoegd. Dat is een conventie, geen noodzaak. De conventie leidt ertoe dat direct duidelijk is dat er van een zelf gedefinieerd type sprake is en draagt daardoor bij aan de leesbaarheid van **C** bronteksten (zie ook bijlage D).

door komma's gescheiden variabelen die de gedefinieerde *structs* vormen. Deze variabelen worden dus net zo gedefinieerd als bijvoorbeeld *int* variabelen: eerst het datatype (*int* of de *struct* definitie), dan de variabelenlijst. Merk op dat variabelen ook als pointervariabelen kunnen worden gedefinieerd. Wanneer een asterisk voor een *struct*-variabele wordt geplaatst, dan is die variabele een pointer naar een *struct*. Een voorbeeld van een dergelijke *struct* zijn de variabelen *person,* die een *struct* is van de twee velden *name* en *age,* en *refperson,* die een pointer is naar eenzelfde structuur. In het voorbeeld krijgt *refperson* nog als waarde het adres van de structuur *person*:

```
1         struct
2         {
3             char
4                 name[80];
5             int
6                 age;
7         }
8             person,
9             *refperson;
10
11        refperson = &person;                    // pointer toekenning
```

Er zijn twee alternatieven om *struct*-variabelen te definiëren. Het eerste alternatief wordt door ons geprefereerd en maakt gebruik van het keyword *typedef* om een nieuw datatype te definiëren. De tweede manier kan worden gebruikt wanneer de *struct* niet als datatype is gedefinieerd en maakt gebruik van de tag in de *struct* definitie.

De *struct* definitie kan vooraf worden gegaan door een keyword *typedef*. De namen achter de *struct* definitie worden dan de namen van nieuwe *struct* datatypes. De namen van deze datatypes worden bij voorkeur in hoofdletters geschreven en krijgen als laatste teken een underscore. In het bovengenoemde voorbeeld zou het type *PERSON_* kunnen zijn gedefinieerd. De variabelen *person* en *refperson* worden dan gedefinieerd als variabelen van het type *PERSON_:*

```
typedef struct
{
    char
        name[80];
    int
        age;
} PERSON_;

PERSON_
    person,
    *refperson;
```

De layout conventie bij de definitie van *struct* datatypes laat toe dat de naam van een *struct* type direct achter de sluitaccolade van de definitie wordt vermeld (zie ook bijlage D).

In situaties waarin een veld van een *struct* definitie een pointer is naar de *struct,* is er sprake van een *recursieve data definitie* (self referential data definition). In deel 71 worden in hoofdstuk 17 toepassingen van dergelijke datatypes behandeld. Bij een recursieve datadefinitie wordt een datatype zèlf bij de definitie ervan gebruikt. Een veld van de *struct* definitie heeft dan betrekking op de *struct* die op dat moment wordt gedefinieerd. Dergelijke velden zijn altijd pointers naar de *struct* die gedefinieerd wordt. Maar omdat de *struct* nog niet gedefinieerd ìs wanneer de *struct* gedefinieerd wòrdt, kan de naam van het *struct* type nog niet worden gebruikt, want de vertaler heeft naam van dat type nog niet gezien op het moment dat de pointer naar de *struct* als veld moet worden gedefinieerd. Voor dergelijke situaties wordt de tag van de *struct* gebruikt, zoals in het volgende voorbeeld.

```
typedef struct person_
{
    struct person_
        *next;
    char
        name[80];
    int
        age;
} PERSON_;

PERSON_
    *first;
```

Het datatype *PERSON_* wordt opnieuw gedefinieerd, maar achter het keyword *struct* volgt de tag *person_*[2] die in de *struct* definitie wordt gebruikt om het veld *next* te definiëren als pointer naar een *struct person_*. De type aanduiding *PERSON_* en de aanduiding *struct person_* zijn equivalent. In plaats van de variabeledefinitie:

*PERSON_ *first;*

kan ook de variabeledefinitie:

*struct person_ *first;*

worden gebruikt. Wanneer een *struct* definitie een tag veld heeft, is een *struct* typedefinitie niet beslist nodig.

De *veld selector* (field selector) operator '.' wordt gebruikt om de velden van een *struct*-variabele te bereiken. Om de velden van een *struct* te bereiken bij een pointer naar een *struct* wordt de veld pointer (field pointer) '→' gebruikt. De '→' wordt in bronteksten aangeduid door '->'.

Als voorbeeld worden de eerder genoemde variabelen *person* en *refperson* gebruikt. Om het veld *age* van de variabele *person* een waarde te geven wordt de *veld selector*-operator gebruikt, zoals in:

person.age = 10;

2. Volgens onze layout conventie is de naam die als tag wordt gegeven gelijk aan de naam van het *struct* type, geschreven in kleine letters.

Om het veld *age* te bereiken vanuit de pointervariabele *refperson* moet eerst met de *inhoud van*-operator de *struct* zelf worden bereikt. Daarbij moet dan rekening worden gehouden met de lagere prioriteit van de *inhoud van*-operator ten opzichte van de *veld selector*-operator, zoals in:

(∗refperson).age = 10;

Wanneer er echter sprake is van een pointer naar een *struct* is het gebruik van de veld pointer-operator '→' veel eleganter. Het bovenstaande statement is equivalent met het volgende statement waarin de veld pointer-operator wordt gebruikt:

refperson→age = 10;

De veld pointer-operator combineert de *inhoud van*-operator en de *veld selector*-operator, waardoor vanuit een pointer naar een *struct* de velden van de *struct* eenvoudig kunnen worden bereikt.

9.2.1 Bitvelden bij structs

Een *struct* kan ook worden gedefinieerd als een structuur van bits, in plaats van een structuur van variabelen van een bepaald datatype. De syntactische regel voor het definiëren van *struct* variabelen met bitvelden is:

```
struct [[tag]]
{
    bit-definitions
}
    variables ;
```

De velden van de *struct* zijn gedefinieerd als *bit-definitions*. De syntactische regel voor een enkele *bit-definition* is:

```
unsigned int bitfield : width ;
```

Het onderdeel *width* in de syntactische regel is een constante die het aantal bits voor het desbetreffende veld aangeeft. Waardetoekenning aan bitvelden is direct mogelijk, zonder dat daarbij de bitwise operatoren hoeven te worden gebruikt. In het volgende voorbeeld wordt een variabele *flag* gedefinieerd als een *struct* met bitvelden, waaraan waarden worden toegekend:

```
typedef struct                                  // type definitie
{
    unsigned int is_open:     1;
    unsigned int is_binary:   1;
    unsigned int is_writable: 1;
} FLAGS_;

FLAGS_                                          // variabele definitie
    flag;

...                                             // gebruik:
flag.is_binary = 1;                             // zet de vlag
flag.is_binary = 0;                             // verwijder de vlag
```

9.3 De union

De *union* wordt gebruikt om een stuk geheugen op verschillende manieren te kunnen opvatten. Afhankelijk van de situatie kan hetzelfde stuk geheugen dan worden opgevat als een *long,* een *int,* een *pointer,* of als een waarde van een ander type. De velden van een *union* liggen dus òver elkaar, in plaats van naast elkaar zoals bij de *struct*. Men spreekt ook wel van een *masker* of een *mal* dat over een stuk geheugen wordt gelegd. Afhankelijk van het gekozen veld wordt een bepaald masker op het geheugen gelegd, waardoor dat stuk geheugen als een variabele van het geselecteerde type wordt geïnterpreteerd.

Een *union* wordt gedefinieerd en gebruikt als een *struct* (behandeld in 9.2). De *veld selector*- en *veld pointer*-operatoren kunnen worden gebruikt, *union* datatypes kunnen worden gedefinieerd en variabelen kunnen gedefinieerd worden als *unions* of als pointers naar *unions*. De definitie van een *union* en een voorbeeld van het gebruik ervan is:

```
typedef union
{
    long longval;
    int  intval[2];
} INTLONG_;

INTLONG_
    intlong;

// statements:

intlong.longval = 0x12345678;                    // 'long' toekenning
printf("low int: %x, high int: %x.\n",           // vermeld 'int'-waardes
    intlong.intval[0], intlong.intval[1]);
```

Het *printf()* statement laat de volgende uitvoer zien:

low int: 5678, high int: 1234.

De grootte van een *union* variabele is de omvang van het grootste veld in de *union*. Wanneer het bovenstaande voorbeeld met de Microsoft vertaler zou worden vertaald, dan zou *intlong* acht bytes groot zijn. De grootte van een *union* variabele of type kan met de *sizeof()*-operator worden bepaald:

printf("%d, %d", sizeof(INT_LONG_), sizeof(intlong));

9.4 Het gebruik van structs en unions

Er zijn een paar aspecten aan het gebruik van *structs* en *unions* die extra aandacht verdienen:

- De definitie en het gebruik van geneste *structs* en *unions*.
- De initialisatie van *structs* en *unions*.
- De allocatie van *structs* en *unions*.

9.4.1 Geneste structs en unions

In **C** kunnen *structs* en *unions* in elkaar worden genest. *Structs* van *structs, structs* van *unions, unions* van *structs* en *unions* van *unions* zijn alle mogelijk. Zodra een *struct* of *union* is gedefinieerd met behulp van een *typedef,* kan een variabele van het desbetreffende type in een andere *struct* of *union* als veld worden opgenomen. In listing 40 geven we de definitie en het gebruik van een *struct* met een *struct* en een *union* als veld, en de definitie en het gebruik van een *union* met een *struct* en een *union* als veld.

```
1           typedef struct
2           {
3               int
4                   i1,
5                   i2;
6           } I2_;
7
8           typedef union
9           {
10              long
11                  l;
12              char
13                  c[4];
14          } LC4_;
15
16          typedef struct
17          {
18              I2_
19                  i;
20              LC4_
21                  lc4;
22          } NEST_STRUCT_;
23
24          typedef union
25          {
26              I2_
27                  i;
28              LC4_
29                  lc4;
30          } NEST_UNION_;
31
32          NEST_STRUCT_
33              ns;
34
35          NEST_UNION_
36              nu;
37
38          ns.lc4.l = 0x12345678;
39          nu.lc4.l = 0x12345678;
```

Listing 40: Geneste *structs* en *unions* (I).

In de regels 1 tot en met 6 wordt de *struct I2_* gedefinieerd. In deze struct

zijn de twee *int* velden *i1* en *i2* beschikbaar als afzonderlijke *ints*. In de regels 8 tot en met 14 is de *union LC4_* gedefinieerd. Het geheugen dat door de *long l* wordt ingenomen, kan via de *c[]* elementen ook als afzonderlijke *chars* worden geïnterpreteerd. In de regels 16 tot en met 22 wordt de *struct NEST_STRUCT_* gedefinieerd. Het eerste veld van deze *struct* is de *struct (i),* het tweede veld is een *union (lc4)*. De *union NEST_UNION_* is gedefinieerd in de regels 24 tot en met 30. De beide velden van deze *union* heten eveneens *i* en *lc4*. De variabele *ns* is in regel 33 gedefinieerd als een *NEST_STRUCT_,* de variabele *nu* is in regel 36 gedefinieerd als een *NEST_UNION_*.

Wanneer in regel 38 aan het *l* veld van de *union lc4* van de *struct ns* de waarde *0x12345678* wordt toegekend, dan geldt (bij de Microsoft implementatie van **C**) het volgende:

- *ns.lc4.c[0]* is *0x78,*
- *ns.lc4.c[1]* is *0x56,*
- *ns.lc4.c[2]* is *0x34,*
- *ns.lc4.c[3]* is *0x12,*

aangezien het array *c[]* en de *long* waarde l velden zijn van eenzelfde *union lc4*. Het *struct* veld *i* van de *struct ns* blijft ongedefinieerd.

Wanneer in regel 39 aan het *l* veld van de *union lc4* van de *union nu* de waarde *0x12345678* wordt toegekend, dan geldt het volgende:

- *nu.lc4.c[0]* is *0x78,*
- *nu.lc4.c[1]* is *0x56,*
- *nu.lc4.c[2]* is *0x34,*
- *nu.lc4.c[3]* is *0x12,*

aangezien het array *c[]* en de *long* waarde *l* velden zijn van eenzelfde *union lc4*. Echter, omdat het *struct* veld *i* een veld is van de *union nu* hebben de velden van *struct i* nu ook een waarde gekregen:

- *nu.i.i1* is *0x5678,*
- *nu.i.i2* is *0x1234*.

De nesting van structs kan ook in de definitie plaatsvinden. Een alternatieve definitie van de *NESTED_STRUCT_* en *NESTED_UNION_* datatypes is gegeven in listing 41.

```
1           typedef struct
2           {
3               struct
4               {
5                   int
6                       i1,
7                       i2;
8               }
9                   i;
10
11              union
12              {
```

```
13                  long
14                      l;
15                  char
16                      c[4];
17              }
18                  lc4;
19          } NEST_STRUCT_;
20
21          typedef union
22          {
23              struct
24              {
25                  int
26                      i1,
27                      i2;
28              }
29                  i;
30
31              union
32              {
33                  long
34                      l;
35                  char
36                      c[4];
37              }
38                  lc4;
39          } NEST_UNION_;
```

Listing 41: Geneste *structs* en *unions* (II).

9.4.2 Initialisatie van structs en unions

struct-variabelen worden geïnitialiseerd als arrays. Bij default zijn alle velden van een *struct* geïnitialiseerd op 0. De syntactische regel voor de initialisatie van een *struct*-variabele is:

```
struct_var =
{
    initializers
}
```

De initialisatie hoort bij één variabele. De initializers zijn initialisatie-expressies, die door komma's van elkaar zijn gescheiden en waarvan de volgorde correspondeert met de volgorde van de velden van de *struct*-variabele. Wanneer een *struct* wordt geïnitialiseerd is minstens één initialisatie-expressie vereist. Wanneer minder velden worden geïnitialiseerd dan er in de *struct* zijn gedefinieerd, dan worden de resterende velden op de default waarde 0 geïnitialiseerd.

Een voorbeeld van de definitie en initialisatie van een *struct* is gegeven in listing 42. Let op de onvolledige initialisatie van het array veld *intar* in regels 16 tot en met 19 en op de (optionele) komma na de initializers op regels 18 en 20.

```
1           typedef struct
2           {
3               char
4                   c,
5                   *string;
6               int
7                   intar[5],
8                   value;
9           } CI_STRUCT_;
10
11          CI_STRUCT_
12              ci =
13              {
14                  'a',
15                  "Good morning sir, here's your breakfast",
16                  {
17                      1,
18                      350,
19                  },
20                  12,
21              };
```

Listing 42: De initialisatie van een *struct*.

Een *union* wordt geïnitialiseerd door een door accolades omhulde initialisatie van het eerste veld van de *union* te geven. Wederom is de default initialisatie voor de velden van een *union* 0. Wanneer al het geheugen van een *union*-variabele moet worden geïnitialiseerd moet het veld dat het meeste geheugen vergt dus het eerste veld van de *union* zijn. In het volgende voorbeeld wordt een *union*-variabele gedefinieerd en geïnitialiseerd.

```
typedef union
{
    int
        intar[5];
    char
        c,
        *string;
} IC_UNION_;

IC_UNION_
    ic =
    {
        1, 2, 3, 4, 5
    };
```

9.4.3 Allocatie van structs en unions

Geheugen voor *struct* of *union* variabelen kan worden gealloceerd via de functie *malloc()* of verwante functies. Via de *sizeof()* operator kan de omvang van een *struct* of *union* worden bepaald. In listing 43 wordt het geheugen voor 20 *structs* van het type *COORD_* gealloceerd[3], en het ge-

3. In listing 43 is afgezien van het testen van de return-waarde van *malloc()*.

heugen voor 10 *unions* van het type *XLAT_*. Na de allocatie kunnen de individuele *structs* of *unions* worden bereikt door middel van een index-waarde, zoals in regel 28 van listing 43, of via de veld pointer-operator zoals in regel 29 van de listing. Let ook op de manier waarop het geheugen voor de *unions* is gealloceerd. Als argument van de *sizeof()* operator wordt de omvang van datgene waar door *xlat_ptr* naar wordt gewezen gebruikt. De vertaler interpreteert *sizeof(*xlat_ptr)* als de omvang van het datatype waarop door *xlat_ptr* wordt gewezen, ook al is *xlat_ptr* initieel *NULL*.

```
1           typedef struct
2           {
3               int
4                   x,
5                   y,
6                   z;
7           } COORD_;
8
9           typedef union
10          {
11              long
12                  longval;
13              int
14                  intval[sizeof(long) / sizeof(int)];
15              char
16                  charval[sizeof(long) / sizeof(char)];
17          } XLAT_;
18
19          COORD_
20              *coord_ptr;
21          XLAT_
22              *xlat_ptr;
23
24          ...
25          coord_ptr = malloc(20 * sizeof(COORD_));
26          xlat_ptr = malloc(10 * sizeof(*xlat_ptr));
27
28          coord_ptr[3].y = 12;
29          (xlat_ptr + 4)->intval[1] = 12;
```

Listing 43: De allocatie van *structs* en *unions*.

9.5 De enum

Enumerated types bevatten een opsomming van symbolische namen voor *int* waarden. De *enum* wordt gebruikt wanneer de waarden van de symbolen niet van primair belang zijn, maar wanneer er wel onderscheid tussen moet kunnen worden gemaakt. In plaats van een opsomming in een *enum* te geven, kan altijd ook gebruik worden gemaakt van een aantal *#define*

directives, waarmee symbolische namen van een waarde worden voorzien. De syntactische regel voor het definiëren van *enum* variabelen is:

```
enum [[tag]]
{
    enumerators
}
    variables ;
```

De enumerators zijn één of meer door komma's van elkaar gescheiden symbolische namen (identifiers), die de waarden van de *enum* vormen. Elke symbolische naam mag worden gevolgd door een initialisatie-expressie bestaande uit een '='-teken, gevolgd door een constante-expressie die de waarde van de desbetreffende naam definieert. Wanneer geen gebruik wordt gemaakt van intitialisatie-expressies is de waarde van de eerste naam in een *enum* definitie 0 en krijgt elke volgende naam een waarde die 1 hoger is dan de waarde van de voorgaande naam. Wanneer initialisatie-expressies worden gebruikt, krijgt de desbetreffende enumerator de waarde van de initialisatie-expressie, van waaruit wordt doorgeteld, tenzij er daarna enumerators worden gebruikt die uitdrukkelijk zijn geïnitialiseerd. Twee enumerators in één *enum* mogen echter niet dezelfde waarde hebben. Met behulp van het keyword *typedef* kunnen *enum* datatypes worden gedefinieerd.

Als voorbeeld wordt hier het *enum* type *MONTH_* gedefinieerd voor de 12 maanden van het jaar:

```
typedef enum
{
    january,
    february,
    march,
    april,
    may,
    june,
    july,
    august,
    september,
    october,
    november,
    december
} MONTH_;
```

Merk op dat de elementen van een *enum* type niet door puntkomma's maar door komma's van elkaar gescheiden worden. Overigens mag achter de laatste enumerator ook een komma worden gezet.

Variabelen van *enum* types zijn dus uiteindelijk integers en overal waar in expressies *int* waarden zouden kunnen worden gebruikt, kunnen ook enumerators worden gebruikt. De enumerators moeten dus niet letterlijk worden geïnterpreteerd. De *enum* waarde *march* is niet de tekst *"march",* maar de waarde 2. Daarom is het eerste deel van het volgende voorbeeld correct en het tweede deel incorrect, omdat in het tweede deel

een fout wordt gemaakt door aan te nemen dat de waarde *march* een string is.

```
MONTH_
    this_month;

...

// deel 1: correct gebruik van een enumerator
if (this_month == november)
    printf ("'t Is bijna Sinterklaas!\n");

// deel 2: incorrect gebruik van een enumerator
printf ("Het is al %s!\n", this_month);
```

Om in het vorige voorbeeld de maandnaam te kunnen afdrukken, zou een array met strings kunnen worden geïnitialiseerd, waarbij de juiste string op grond van de waarde van een *enum* variabele wordt gekozen en dan wordt afgedrukt. Merk op dat een *switch* met 12 *cases* ook mogelijk zou zijn geweest, echter ten koste van omvangrijkere en tragere code. De initialisatie van een array verdient dan ook de voorkeur.

```
MONTH_
    this_month;

char *month_name [] =
{
    "januari",
    "februari",
    "maart",
    "april",
    "mei",
    "juni",
    "juli",
    "augustus",
    "september",
    "oktober",
    "november",
    "december"
};

...
printf ("Het is al %s!\n",
    month_name [this_month]);
```

In het volgende voorbeeld wordt een *enum* gedefinieerd waarin de enumerators specifieke waarden krijgen via initialisatie-expressies. Ook wordt een illustratie gegeven van het gebruik van een *enum* variabele van een dergelijk type.

```
typedef enum
{
    cent = 1,
    dubbeltje = 10,
    kwartje = 25,
    gulden = 100,
    rijksdaalder = 250
} MUNT_;
```

```
MUNT_
    munt;

....
munt = gulden;
printf ("Er gaan %d kwartjes in deze munt.\n", munt / kwartje);
```

Opgave 49.

Definieer een *struct* datatype, waarin twee velden zijn opgenomen: een veld *naam* van het type *char* ∗; en een veld *leeftijd* van het type *int*. Gebruik een *typedef* om dit datatype te maken.
Definieer vervolgens twee variabelen van dit datatype. Vul de velden van deze variabelen met bijbehorende data. Druk als laatste de variabelen af.
Aanwijzing: Aangezien veld *naam* een pointer naar een *char* is, zal het geheugen waar deze pointer naar wijst op de een of andere wijze moeten worden aangemaakt.

Opgave 50.

Stel een *union* type op met de volgende velden: veld *str* van het type *char [11]*,
veld *long_nr* van het type *long*,
veld *int_nr* van het type *int*,
veld *byte_nr* van het type *unsigned char*.
Definieer een variabele van dit type en initialiseer de union op de waarde *"I love it!"*.
Wat is nu:
veld *long_nr*, afgedrukt als een *long;*
veld *int_nr*, afgedrukt als een *integer;*
veld *byte_nr*, afgedrukt als een *integer*.
Waarom hebben de velden die waarden? Waarom is *str* niet gedefinieerd als een *char[10]* array?

9.6 Een database applicatie

In dit deel van het boek wordt een eenvoudig programma voor het bewerken van een adressenbestand ontwikkeld. Bij de ontwikkeling van dit programma zullen de volgende onderwerpen nog eens de revue passeren:

- Gebruik van *structs, enums* en *typedefs*
- File I/O
- Allocatie en deallocatie
- Searching and Sorting

Het programma dat zal worden ontwikkeld is een zeer vereenvoudigde vorm van de uitgebreide database programma's zoals *dbase*. De structuur van de gegevens in het hier ontwikkelde programma ligt vast, waardoor de verwerking van de gegevens sterk wordt vereenvoudigd. Elk element *(record)* van het databestand bevat een naam, straat, stad en telefoonnummer. Het is handig om de gevens in een tekstbestand beschikbaar te hebben. Het voordeel daarvan is dat het bestand niet alleen door het te

ontwikkelen programma kan worden bewerkt, maar dat elke tekstverwerker of editor het adressenbestand ook kan bewerken. Wanneer elk record eenvoudig uit een aparte regel tekst per onderdeel bestaat, dan ligt de structuur in zoverre vast, dat na elke regel een volgend onderdeel van het record begint, en na het laatste onderdeel van een record een volgend record kan beginnen. De lengte van de onderdelen zelf is echter nog flexibel. Zowel lange als korte namen, adressen, enzovoort, kunnen in het adressenbestand voorkomen.

Een ander kenmerk van het programma is dat het een *interactief* programma is. Aan de gebruiker van het programma worden vragen gesteld en het programma reageert afhankelijk van de reactie van de gebruiker op de gestelde vraag. Soms zal de vraag een keuze toelaten (zoals een nieuw element aan het bestand toevoegen, of het programma verlaten), soms zal het programma informatie vragen (zoals een nieuw adres).

De ontwikkeling van een programma is tot nu toe steeds gebaseerd geweest op herformuleringen van een beschrijving van de werking van het programma. Een alternatief uitgangspunt is te beginnen met een beschrijving van de gegevensstructuur. Het programma wordt dan verder ontwikkeld vanuit die structuur. Vooral bij grote programma's is een heldere specificatie van de gegevensstructuur een noodzakelijke voorwaarde voor verdere programma ontwikkeling. De gegevensstructuur voor het huidige addressenbestand programma is eigenlijk al besproken: het bestand bestaat uit records van vier tekstregels, waarbij elk record de naam, straat, plaats en telefoonnummer van een persoon zal bevatten. Het ligt dan ook voor de hand een record te definiëren als een *struct* met vier velden. Elk veld bevat een onderdeel van het record. Omdat de lengte van die onderdelen niet bekend is, wordt gebruik gemaakt van een *struct* met pointers naar *chars* die uiteindelijk naar de desbetreffende informatie zullen wijzen. De structuur van het *struct* datatype *ADDRESS_* wordt:

```
typedef struct
{
    char
       *name,
       *street,
       *city,
       *phone;
} ADDRESS_;
```

De records zullen worden opgeslagen in een array van *ADDRESS_ structs*. Uiteraard is van tevoren niet bekend hoeveel records er zijn. Daarom zal het array run-time worden gealloceerd. Het begin van het array zal worden aangewezen door een pointervariabele *(ADDRESS_ *address),* terwijl het aantal records dat is ingelezen zal worden onthouden in de *int* variabele *maxadr*.

De mogelijke opdrachten die door de gebruiker aan het programma kunnen worden gegeven, worden in de brontekst van het programma aan-

geduid met behulp van waarden uit een *enum* type *CHOICE_*. De *enum* definitie is:

```
typedef enum
{
    choice_quit,
    choice_list,
    choice_add,
} CHOICE_;
```

Voorlopig zijn er nog maar drie keuzes: stoppen, een adres toevoegen, of alle adressen tonen. Uitbreidingen op deze basisverzameling opdrachten kunnen later eenvoudig worden gerealiseerd.

9.6.1 De functie main()

De eerste herformulering van de beschrijving van het programma leidt tot de functie *main()* van het programma. Herformuleringen van de beschrijving van het programma kunnen nu worden gebaseerd op de wetenschap dat de records van het bestand zullen worden opgeslagen in een array, waarnaar door de variabele *address* zal worden gewezen en dat *maxadr* elementen heeft. De eerste herformulering wordt:

1. Test het aantal argumenten van het programma.
   2. Wanneer het aantal onjuist is, geef dan een foutmelding, en beëindig het programma.
3. Sla de records van het adressenbestand op in het geheugen.
4. Zolang de gebruiker niet aangeeft te willen stoppen
   5. Geef een overzicht van de inhoud van het adressenbestand wanneer daarom wordt gevraagd.
   6. Voeg een nieuw element toe aan het adressenbestand wanneer daarom wordt gevraagd.

Onderdeel 1 wordt getest met behulp van *argc*. In de beschrijving wordt niet uitdrukkelijk vermeld wat het adressenbestand zal zijn en we besluiten dat de gebruiker de naam van het adressenbestand als argument aan het programma mee moet geven. De aanroep van het programma kan bij voorbeeld zijn:

adr leden.adr

Onderdeel 2 is ook eenvoudig met behulp van een *printf()* statement te realiseren.

De overige onderdelen zijn voorlopig nog te lastig om direct te implementeren, hoewel de onderdelen 4, 5 en 6 in een *switch* kunnen worden samengevoegd. In listing 44 is de implementatie van de functie *main()* gegeven. Merk op dat voor de onderdelen 3, 4, 5 en 6 een beroep wordt

gedaan op de functies *readbase()* om het adressenbestand te lezen, *mainmenu()* om de gebruiker keuzes te bieden, *listbase()* om het adressenbestand af te drukken en *addentry()* om een nieuw element toe te voegen. Deze functies worden in volgende herformuleringen gedefinieerd.

```
1        #include <stdio.h>
2        #include <string.h>
3
4        typedef struct                              // struct definitie
5        {
6            char
7               *name,
8               *street,
9               *city,
10              *phone;
11       } ADDRESS_;
12
13      typedef enum                                 // menu ingang
14      {                                            // enumeratie
15          choice_add,
16          choice_list,
17          choice_quit,
18      } CHOICE_;
19
20       ADDRESS_
21           *address;                               // array van adressen
22       int
23           maxadr;                                 // aantal adressen
24
25       int main (int argc, char **argv)
26       {
27           CHOICE_
28               choice;
29
30           if (argc < 2)                           // tekst: 1.
31           {
32               printf ("Usage : %s dbase\n"        // tekst: 2.
33                       "Where : dbase - name of address data base\n\n",
34                   argv[0]);
35               return (1);
36           }
37           readbase(argv[1]);                      // tekst: 3.
38           while
39           (
40               (choice = mainmenu())               // tekst: 4.
41               !=
42               choice_quit
43           )
44               switch (choice)
45               {
46                   case choice_list:               // tekst: 5.
47                       listbase ();
48                   break;
49
```

```
50                  case choice_add:                    // tekst: 6.
51                      addentry ();
52                  break;
53              }
54          return (0);
55      }
```

Listing 44: De functie *main()* van *ADR.C*.

9.6.2 De functie readbase()

Onderdeel 3 betreft het inlezen van het adressenbestand. Deze taak zal worden uitgevoerd door de functie *readbase()* die als argument de naam van het adressenbestand krijgt. De functie *readbase()* moet het volgende doen:

3.1. Open het adressenbestand om te worden gelezen.
  3.2. Wanneer het openen niet lukt, geef dan de melding dat een nieuw databestand wordt gecreëerd en beëindig de functie.
3.3. Lees het addressenbestand tot het einde van het bestand. Doe per record het volgende:
  3.4. Maak ruimte voor een extra element in het array *address*.
  3.5. Ken aan de vier velden van het extra *ADDRESS_* element van het array *address* de inhoud van de eerstvolgende vier regels van het adressenbestand toe.
3.6. Sluit het adressenbestand.

Onderdeel 3.1 kan worden uitgevoerd met de standaardfunctie *fopen()*. Onderdeel 3.2 wordt gerealiseerd met behulp van een *printf()* functie en onderdeel 3.6 wordt uitgevoerd door de standaard functie *fclose()*.

Bij de implementatie van onderdeel 3.3 is gebruik gemaakt van een *do-while*- statement omdat het adressenbestand immers bestaat, aangezien het kon worden geopend in 3.1. Mocht de organisatie van het adressenbestand foutief zijn, dan zal het programma dat later, bij het inlezen van de records, ontdekken waarna het programma zal afbreken. De controle op de juiste organisatie van het databestand bestaat dan ook eenvoudigweg uit een controle op het juiste aantal regels in het databestand. Meer complexe controles zijn wel mogelijk, maar zijn nauwelijks de moeite waard. In het ongunstigste geval geeft de gebruiker een bestand op dat helemaal geen adressenbestand is. Een dergelijk bestand zal in de hier gevolgde opzet leiden tot ofwel een vroegtijdige beëindiging van het programma, ofwel onzinnige informatie op het beeldscherm, waardoor de gebruiker het programma al snel zal verlaten.

Onderdeel 3.4 wordt gerealiseerd met behulp van de eerder genoemde functie *xrealloc()* (zie pagina 160), die het programma beëindigt met een foutmelding wanneer de uitbreiding van het array mislukt.

Onderdeel 3.5 wordt gerealiseerd met behulp van een functie *getline()* die een pointer naar een *char* retourneert. De functie wordt vier keer aangeroepen. Elke aanroep leidt tot een waardetoekenning aan een veld van een *ADDRESS_* structuur.

De implementatie van de functie *readbase()* is gegeven in listing 45.

```
1       #include <io.h>
2
3       void readbase (char *name)
4       {
5           long
6               length;
7           register FILE
8               *f;
9
10          if (! (f = fopen (name, "r")) )         // tekst: 3.1
11          {                                       // tekst: 3.2
12              printf ("Creating new base %s\n", name);
13              return;
14          }
15
16          length = filelength(fileno(f));         // tekst: 3.3
17          while (ftell(f) < length)               // tekst: 3.3
18          {
19              address = xrealloc(address,         // tekst: 3.4
20                                 ++maxadr * sizeof(ADDRESS_));
21
22              address[maxadr - 1].name   = getline(f);// tekst: 3.5
23              address[maxadr - 1].street = getline(f);
24              address[maxadr - 1].city   = getline(f);
25              address[maxadr - 1].phone  = getline(f);
26          }
27          fclose(f);                              // tekst: 3.6
28      }
```

Listing 45: De functie *readbase()* van *ADR.C*.

9.6.3 De functie mainmenu()

Onderdeel 4 is gerealiseerd in een *while*-statement dat wordt gestuurd door de functie *mainmenu()* (zie listing 44. Deze functie laat het hoofdmenu van het programma zien, waarin voorlopig slechts de drie opties stoppen, afdrukken en bijvoegen voorkomen. De return-waarde van de functie is een waarde van het *enum* datatype *CHOICE_*. De functie *mainmenu()* moet het volgende doen:

4.1. Presenteer het menu, waarbij aan elke ingang in het menu een toets is gekoppeld om een directe keuze mogelijk te maken.
4.2. Geef de gebruiker aan dat een keuze wordt verwacht.

4.3. Bepaal de keuze van de gebruiker. Geef een signaal wanneer de gebruiker een andere toets indrukt dan een gedefinieerde keuzetoets.
4.4. Maak de uiteindelijke keuze op het beeldscherm zichtbaar.
4.5. Retourneer de waarde uit de *CHOICE_ enum* die overeenkomt met de gemaakte keuze.

De implementatie van de functie *mainmenu()* is niet moeilijk, en wordt gegeven in listing 46. Merk op dat de volgorde van de keuzeletters in de statische string *keys* (regels 3 en 4) correspondeert met de volgorde van de waarden van de *CHOICE_ enum*. Daardoor is het verschil tussen het adres in *keys* waar de geselecteerde letter staat en de beginlocatie van *keys* de juiste *enum* waarde. Zo correspondeert de letter *'q'* met de keuze *choice_quit,* (waarde 0) en de letter *'a'* met de keuze *choice_add* (waarde 2).

```
1       #include <conio.h>
2
3       char
4           keys [] = "alq";
5
6       CHOICE_ mainmenu ()
7       {
8           char
9               *key_ptr;
10
11          puts ("\n"                              // Tekst: 4.1
12                "(a) Add entry\n"
13                "(l) List base\n"
14                "(q) Quit\n"
15               );
16
17          printf ("Enter your choice:" );         // Tekst: 4.2
18
19          while (!(key_ptr = strchr(keys, getch())))// Tekst: 4.3
20              putchar('\a');
21
22          printf ("%c\n", *key_ptr);              // Tekst 4.4
23
24          return ((CHOICE_)(key_ptr - keys));     // Tekst 4.5
25      }
```

Listing 46: De functie *mainmenu()* van *ADR.C*.

9.6.4 De functie listbase()

Onderdeel 5 geeft een overzicht van de inhoud van het adressenbestand. Dit overzicht wordt gegeven door de functie *listbase()*. De functie *listbase()* is wederom een eenvoudige functie waarvan de implementatie direct kan worden gegeven (zie listing 47).

```
1       void listbase ()
2       {
3           unsigned
4               i;
5
6           if (!maxadr)                        // direct klaar ?
7           {
8               puts("\nEMPTY DATABASE !");     // vermeld het
9               return;
10          }
11          putchar('\n');                      // begin met '\n'
12          for (i = 0; i < maxadr; i++)        // verwerk alle elementen
13              printf ("\n"                    // geef per element:
14                      "Name:   %s\n"          //    naam
15                      "Street: %s\n"          //    straat
16                      "City:   %s\n"          //    plaats
17                      "Phone:  %s\n",         //    telefoonnr.
18                  address[i].name,
19                  address[i].street,
20                  address[i].city,
21                  address[i].phone);
22      }
```

Listing 47: De functie *listbase()* van *ADR.C*.

In onderdeel 6 wordt een nieuw element aan het array *address* toegevoegd door de functie *addentry()*. De functie *addentry()* is een eenvoudige functie doordat er alleen een nieuw element aan *address* wordt toegevoegd. Na het toevoegen van een nieuw element is er dus aan het adressenbestand nog niets gewijzigd. Daarnaast voegt de functie *addentry()* slechts één nieuw element aan *address* toe en wordt er geen controle uitgevoerd op mogelijke verdubbelingen van de velden van de *ADDRESS_* structuur. De functie *addentry()* lijkt op de functie *readbase()* waarmee het adressenbestand werd ingelezen. De functie *getline()* in listing 45 is in de functie *addentry()* vervangen door de functie *scanline()* (zie listing 48).

```
1       void addentry ()
2       {
3           address = xrealloc (address, ++maxadr * sizeof (ADDRESS_));
4
5           putchar('\n');
6
7           address[maxadr - 1].name   = scanline("Name:"   );
8           address[maxadr - 1].street = scanline("Street:" );
9           address[maxadr - 1].city   = scanline("City:"   );
10          address[maxadr - 1].phone  = scanline("Phone:"  );
11      }
```

Listing 48: De functie *addentry()* van *ADR.C*.

9.6.5 De hulpfuncties

Tenslotte (zie listing 49) de twee hulpfuncties *getline()* en *scanline()*. Beide functies retourneren het adres van een duplicaat van een ingelezen string. De string mag ten hoogste 250 tekens lang zijn. Wanneer er geen duplicaat kan worden gemaakt, of wanneer de gelezen string te lang is, wordt het programma afgebroken. De functie *getline()* krijgt als argument de *FILE* pointer die bij het adressenbestand hoort. De functie *scanline()* krijgt als argument de string waarmee aan de gebruiker wordt aangegeven welk onderdeel van de *ADDRESS_* structuur wordt verwacht. Beide functies maken gebruik van een hulpfunctie *duplicate()* die het duplicaat van de ingelezen string maakt. De functie *duplicate()* maakt gebruik van de hulpfunctie *stripstr()* die spaties aan het begin en eind van de string verwijdert. De twee laatste hulpfuncties zijn, voorzien van toelichtend commentaar, eveneens gegeven in listing 49.

```
1       #define WHITESPACE      " \t\n"
2
3       char
4           whitespace[] = WHITESPACE;
5
6       char *stripstr(char *cp)
7       {
8           char
9               *start;
10
11          while
12          (
13              memchr(whitespace,                  // negeer spaties
14                     *cp, strlen(WHITESPACE))
15          )
16              cp++;                               // aan het begin
17
18          start = cp;                             // bepaal return-waarde
19
20          cp += strlen(cp) - 1;                   // cp wijst laatste teken
21          while
22          (
23              cp >= start                         // cp wijst op stringchar
24              &&                                  // en
25              memchr(whitespace, *cp,             // laatste teken is spatie
26                     strlen(WHITESPACE))
27          )
28              *cp-- = 0;                          // verwijder laatste teken
29
30          return (start);                         // retourneer stringbegin
31      }
32
33      char *duplicate(char *cp)
34      {
35          char
36              *tail;
```

```
37
38          if (!cp)                                // string niet gelezen
39          {
40              printf("\nCannot read textline"
41                     "for record %d\n",
42                         maxadr);
43              exit(1);
44          }
45
46          cp = stripstr(cp);                      // zie de tekst
47
48          if (!(cp = strdup(cp)))                 // dupliceer de string
49          {
50              puts(out_of_memory);                // zie xmalloc()
51              exit(1);
52          }
53          return (cp);                            // retourneer duplicaat
54      }
55
56      char *getline(FILE *f)
57      {
58          char
59              buf[251];
60
61          return                                  // retourneer duplicaat
62          (
63              duplicate
64              (                                   // lees string
65                  fscanf(f, "%250[^\n]\n", buf) ?
66                      buf                         // gelukt.
67                  :
68                      NULL                        // mislukt
69             )
70          );
71      }
72
73      char *scanline(char *prompt)
74      {
75          char
76              buf[251];
77
78          printf(prompt);
79          return (duplicate(gets(buf)));
80      }
```

Listing 49: De functies *getline()* en *scanline()*.

Het adresprogramma wordt in de volgende opgaven met een aantal extra functies uitgebreid. Zorg bij elke aanpassing voor de volgende indeling van het programma:

- Alle *#include* opdrachten aan het begin van de brontekst.
- Alle *#define* opdrachten volgen op de *#includes*.
- Dan alle *typedefs*.

- Vervolgens alle globale variabelen.
- Tenslotte alle functies in dusdanige volgorde dat functies gedefinieerd zijn voordat ze worden gebruikt.
- De functie *main()* is (dus) de laatste functie in de brontekst van het programma.

Opgave 51.

Bij een adressenbestand is een zoekfunctie handig. Schrijf een functie *findname()*, die om een naam vraagt (aangenomen wordt dat de gevraagde naam uit hoogstens 80 tekens bestaat), en die naam probeert te vinden in het adressenbestand.

Gebruik de standaardfunctie functie *strstr()* voor het matchen van de gezochte naam met de namen in de data base, zodat ook op delen van een naam kan worden gezocht. De naam *Kernighan* kan dan ook worden gezocht als *nighan*. Bovendien moet het zoekproces ongevoelig zijn voor hoofd- en kleine-letters. De functie *strlwr()* kan worden gebruikt om de hoofdletters van een string naar kleine letters te converteren. Uiteraard moet de informatie in het adressenbestand niet door de conversie worden aangetast.

Alle gevonden records moeten op het scherm afgebeeld worden. Maak gebruik van het volgende code-frame om de functie te implementeren.

```
void findname()
{
    char
        target[80];
        source[80];
    int
        found = 0;
        index;

    // 1. vraag om een naam
    // 2. lees de naam in in het target[] array
    // 3. converteer naar lower-case
    // 4. inspecteer alle records van het bestand
        // 5. kopieer het name-veld in het source[] array
        // 6. converteer de source-string naar lower-case
        // 7. kijk of de target-string in de source string
        //    kan worden aangetroffen
            // 8. Zo ja, verhoog found, en laat de inhoud van
            //    het record zien.
    // 9. Geef een passende melding wanneer found 0 is gebleven.
}
```

Opgave 52.

Neem de functie *findname()* op in het programma. Breid het hoofdmenu uit met een ingang *"(s) Search for a name"*, en de *CHOICE_ enum* met een waarde *choice_search*. De functie *main()* moet *findname()* aanroepen als dit gewenst is.

Opgave 53.

Schrijf een functie *delete()*, die het geselecteerde record wist. Als geen record geselecteerd is, moet uiteraard niets gebeuren. Vóór het feitelijke wissen dient *delete()* te vragen of het record ècht weg moet. De functie *delete()* kan worden opgenomen in de functie *findname()* die een parameter *CHOICE_ function* krijgt. Wanneer *function* de (nieuwe) waarde *choice_delete* is, moet de functie *delete()* worden aangeroepen. Het menu moet worden uigebreid met een ingang *"(d) Delete a record"*, en de functie *findname()* wordt aangeroepen met argument *choice_search* wanneer de zoekfunctie is gekozen en met argument *choice_delete* wanneer de wis-functie is gekozen.

De delete-functie zelf krijgt als argument de huidige waarde van de variabele *index* van de functie *findname()* en retourneert de nieuwe waarde van *index*. Het wissen zelf bestaat uit
1. het verlagen van de adressenteller *maxadr,*
2. het vrijgeven van het geheugen waarop door de velden van het te wissen record wordt gewezen,
3. het 'terugschuiven' van de resterende *maxadr - index* records van het array waarop door *address* wordt gewezen, te beginnen bij record *address[index + 1]*. Hiervoor kan gebruik worden gemaakt van de standaardfunctie *memcpy()*
4. Het reduceren van de omvang van het *address* array tot de omvang *maxadr* record-elementen.
5. Het reduceren van *index*.

De functie retourneert de waarde van *index* (eventueel na reductie). Neem de functie *delete()* op in het programma.

Opgave 54.

Definieer een globale *int changed*, die initieel waarde 0 heeft en op 1 gezet wordt als iets in het adressenbestand wordt toegevoegd of wordt gewist.

Opgave 55.

Schrijf een functie *writebase()*, die het adressenbestand naar file schrijft. Het nieuwe adressenbestand moet naar het bestand geschreven worden waarvan de naam als argument aan de functie *writebase()* wordt meegegeven. De te gebruiken naam is de naam van het bestand dat als argument aan het programma is meegegeven. Het wegschrijven hoeft niet te gebeuren als het adressenbestand ongewijzigd is, maar moet automatisch gebeuren wanneer er wel een wijziging is opgetreden. Wanneer het adressenbestand leeg is, moet een eventueel oud bestand worden verwijderd (Aanwijzing: gebruik de standaardfunctie *unlink()*). Neem de functie vervolgens op in het programma.

Hoofdstuk 10

Werken met bits

10.1 Het werken met bits

In hoofdstuk 3 zijn de operatoren besproken die betrekking hebben op het werken met bits en in hoofdstuk 9 zijn *structs* behandeld waarvan de velden uit bits bestaan. Toepassingen van het werken met bits zijn echter nog niet behandeld. Het werken met bits zal nader worden geïllustreerd aan de hand van het probleem om *priemgetallen* te genereren. Priemgetallen zijn gehele getallen die alleen door zichzelf en door 1 zonder rest deelbaar zijn. Zo is het getal 7 een priemgetal, want 7 is alleen deelbaar door 7 en 1. Het getal 4 daarentegen is geen priemgetal, want 4 is zonder rest deelbaar door 1, 2 en 4. Het getal 2 is het enige even priemgetal.

Priemgetallen worden voor allerlei toepassingen gebruikt. Een handige eigenschap van priemgetallen is dat herhaalde toepassing van de bewerking:

index = (index + constante) % priem;

wanneer *constante* niet 0 of een veelvoud van *priem* is, *index* in een repeterend patroon *alle* waarden tot *priem* aanneemt. Bijvoorbeeld, als *getal* 4 is, *constante* 3 is en *priem* 7, dan wordt *getal* achtereenvolgens 4, 0, 3, 6, 2, 5 en 1, voordat weer de waarde 4 wordt bereikt. Deze eigenschap is erg handig bij het snel doorzoeken van tabellen.

Voordat priemgetallen kunnen worden gebruikt moeten ze er eerst zijn. Tabellen met priemgetallen zijn er in overvloed (bijvoorbeeld: Selby, 1972). In dergelijke tabellen wordt echter geen aandacht geschonken aan het werken met bits, welk aspect dan ook centraal staat in dit hoofdstuk.

10.2 Het genereren van priemgetallen

Priemgetallen zijn positieve getallen die slechts door zichzelf en het getal 1 zonder rest deelbaar zijn. We noemen hier twee manieren om priemgetallen te berekenen. De eerste manier om priemgetallen te bereken is een eenvoudige, intuïtief aansprekende manier, die echter trager wordt naarmate de priemgetallen groter worden. De tweede manier, die bekend staat als "de Zeef van Aristothenes", is in het begin juist wat trager, maar wordt sneller naarmate er meer priemgetallen zijn gevonden. De 'Zeef' is bovendien erg geschikt om het werken met bits mee te illustreren.

10.2.1 Een eenvoudig algoritme

Om te bepalen of een getal een priemgetal is, kan het getal worden gedeeld door alle eerdere getallen. Wanneer een getal zonder rest deelbaar is door een eerder getal is het getal geen priemgetal en anders wel. Dus, om te weten of 5 een priemgetal is, wordt gekeken of 5 zonder rest deelbaar is door 2, 3 en 4. Wanneer 5 echter door een van deze drie getallen wordt gedeeld, is er sprake van een rest en dus is 5 een priemgetal.

De hier beschreven methode (algoritme) is wel bijzonder eenvoudig. Het is geen bijzonder snel algoritme, maar de traagheid wordt pas bij grote priemgetallen een factor van betekenis.

Opgave 56.

Schrijf een programma *priem1.c* dat alle priemgetallen tot een *long* waarde (die als argument aan het programma is meegegeven) volgens de eerste methode berekent. Een code-frame voor het programma kan zijn:

```
#include <stdio.h>

void main(int argc, char **argv)
{
    long
        last,
        current,
        index;

    // bepaal de waarde 'last' uit argv[1]
    // test alle waarden van 'current' = 2 tot'last'
    {
        // probeer het getal te delen door alle getallen
        // van 2 tot ('current' - 1)
        {
            // als de deling 0 oplevert, stop dan met testen
        }
        // als alle getallen tot 'current' zijn getest
            // vermeld dan 'current' als een priem
    }
}
```

Uiteraard geldt dat een getal dat door 4 zonder rest deelbaar is, ook door 2 zonder rest deelbaar is. Een verfijning van het eerste algoritme bestaat er dan ook uit dat de priemgetallen die zijn gevonden worden onthouden en dat alleen wordt gekeken of een getal deelbaar is door gevonden priemgetallen. Wanneer dat niet het geval is, is het nieuwe getal zelf een priemgetal. Het aardige van het tweede algoritme is dat het algoritme sneller is dan het eerste, hoewel er extra code nodig is om bij te houden welke getallen al als priemgetallen waren gevonden.

Opgave 57.

Schrijf een programma *priem2.c* dat alle priemgetallen tot een *long* waarde (die als argument aan het programma is meegegeven) volgens de tweede methode berekent. Een code frame voor het programma is hieronder gegeven. Gebruik voor het uitbreiden van het array de functie *realloc()*

```
#include <stdio.h>
#include <malloc.h>

int
    npriemen;
long
    *priem;

void main(int argc, char **argv)
{
    long
```

```
        last,
        current;
    int
        index;

    // bepaal de waarde last uit argv[1]
    // test alle waarden van current = 2 tot last
    {
        // probeer het getal te delen door alle gevonden
        // priemgetallen
        {
            // als de deling 0 oplevert, stop dan met testen
        }
        // als alle priemgetallen zijn getest
        {
            // vermeld dan current als een priem, en onthoud
            // het gevonden priemgetal na uitbreiding van het
            // array priem
        }
    }
}
```

10.2.2 De Zeef van Aristothenes

Een geheel andere manier om priemgetallen te berekenen wordt gevolgd door de "Zeef van Aristothenes". Volgens de Zeef van Aristothenes kunnen priemgetallen als volgt gegenereerd worden:

- Er wordt een verzameling opgesteld, waarin (in het begin) alle getallen aanwezig zijn. Deze verzameling heet ook wel de 'getallenlijn'. Het getallenbereik waarbinnen priemgetallen worden gezocht is dus ook bij deze methode een gegeven. Het eerste getal op deze getallenlijn is 2. Het getal 1 is dus niet in de verzameling opgenomen.
- De getallenlijn wordt vanaf het eerste getal tot het laatste getal afgelopen. Wanneer een getal wordt aangetroffen (het eerste getal is dus 2) dan wordt het volgende gedaan:
  1. Het getal wordt vermeld, want het is een priem.
  2. Alle veelvouden van het gevonden getal worden uit de verzameling verwijderd. Deze getallen vallen als het ware door de Zeef.

Het zal duidelijk zijn, dat het algoritme vooral in het begin relatief traag is, omdat dan nog een groot deel van de getallenlijn moet worden geïnspecteerd. Naarmate het proces langer duurt zal de verzameling steeds leger worden en zullen er steeds minder getallen resteren om te worden getest.

Een eerste implementatie van de Zeef van Aristothenes

De getallenlijn kan als een array van integers gerepresenteerd worden. Afgesproken wordt, dat een getal n nog niet door de Zeef is gevallen wanneer het n^e element van dat array de waarde 0 heeft. Als het n^e element

de waarde 1 heeft, dan is het getal niet meer op de getallenlijn aanwezig. De getallenlijn wordt (voorlopig) op een vaste grootte gedefinieerd:

```
#define MAX 10000

int line [MAX];
```

De maximale waarde van een priemgetal is hier ten hoogste 9999, omdat de indices op *line* immers van 0 t/m 9999 lopen.

Vervolgens kan de getallenlijn worden afgelopen, te beginnen bij 2. Als het getal nog niet door de Zeef is gevallen (dus als *!line[i]),* dan kan het getal op het scherm worden getoond als een priem en vallen de veelvouden van het gevonden getal door de Zeef. De desbetreffende array-elementen krijgen dus de waarde 1:

```
for (i = 2; i < MAX; i++)
    if (!line[i])
    {
        printf ("prime: %8d\n", i);
        for (multiple = i + i; multiple < MAX; multiple += i)
            line[multiple] = 1;
    }
```

De hierboven gegeven **C** code dekt alle essentiële informatie van het programma. Er kan nog een verfijning in de uitvoer worden aangebracht: per regel worden zes priemgetallen vermeld, waarna de regel wordt afgesloten. Om het aantal getallen op een regel te tellen, wordt een teller *nprimes* gebruikt, die van 0 t/m 5 loopt. Als de teller de waarde 6 bereikt, wordt een \n teken geschreven en wordt de teller weer op de beginwaarde gezet. In listing 50 is het hele programma gegeven.

```
1           #include <stdio.h>
2
3           #define MAX 10000
4
5           int line [MAX];
6
7           void main ()
8           {
9               int
10                  prime,
11                  multiple,
12                  nprimes = 0;
13
14              for (prime = 2; prime < MAX; prime++)
15              {
16                  if (!line[prime])
17                  {
18                      printf ("priem: %4d,", prime);
19                      if (++nprimes == 6)
20                      {
```

```
21                          putchar ('\n');
22                          nprimes = 0;
23                      }
24                      for
25                      (
26                          multiple = prime + prime;
27                              multiple < MAX;
28                                  multiple += prime
29                       )
30                          line[multiple]++;
31                  }
32              }
33          }
```

Listing 50: Een implementatie van de Zeef van Aristothenes.

Een getallenlijn van bits

Om aan te geven of een getal in de getallenverzameling voorkomt worden niet zozeer de getallen zelf gebruikt, maar fungeren de array-elementen als *vlaggen*. De waarde van een array-element (een vlag) kan 0 of 1 zijn. Hoewel een waarde van een vlag in één bit kan worden gerepresenteerd, worden voor het array *int* elementen gebruikt. Dat betekent dat 15 van de 16 bits van een *int* werkeloos blijven, hetgeen een aanzienlijke verspilling van geheugen met zich mee brengt. Door de verzameling niet met behulp van *int* waarden te representeren, maar met behulp van bits kan aan de verspilling van het geheugen een einde worden gemaakt. Er ontstaat dan een representatie van een verzameling met behulp van bits: een getallenlijn van bits.

Om een representatie met bits te realiseren wordt de volgende aanpak gevolgd:

- Ten eerste wordt een zodanige omvang van het array bepaald dat alle bits erin kunnen worden opgeslagen.
- Ten tweede moet kunnen worden getest of een individueel bit al dan niet *op* staat (dus: de waarde 1 heeft).
- Ten derde moet een manier worden gevonden om een individueel bit de waarde 1 te geven.

De grootte van het array wordt als volgt bepaald: Eén *int* bevat maximaal 16 bits (In het algemeen: n bits). Dat betekent, dat als slechts de getallen 0 tot en met 15 worden getest, er maar één *int* nodig is. Wanneer echter de getallen 0 tot en met 16 moeten worden getest, dan zijn er al twee *ints* nodig om de getallenlijn in bits te kunnen representeren. Met andere woorden, het aantal *ints* dat beschikbaar moet zijn om *MAX* bits te kunnen bewerken, is:

elementen = (MAX + 15) / 16;

In het algemeen, wanneer *MAX* data-elementen nodig zijn en er passen n elementen in een geheugen-element (bij voorbeeld *char* of *int*), dan zijn er:

elementen = (MAX + n - 1) / n;

geheugen-elementen nodig. De hier gegeven regels maken gebruik van de *int*-deling: bij de deling verwijnt een eventuele rest.

Vervolgens bepalen we de manier om te testen of een bit *op* staat. Wanneer er slechts één *int* nodig zou zijn, dan was er uiteraard geen probleem, omdat de bitwise-operatoren kunnen worden gebruikt om individuele bits in een *int* te testen. Om echter te testen of het bit (de vlag) voor het getal 4001 nog in de verzameling voorkomt, moet eerst worden bepaald in welke *int* de vlag voor de waarde 4001 wordt gevonden. Vervolgens kan in de desbetreffende *int* de passende bit worden getest.

De *int*-index die bij een bepaalde waarde hoort wordt eenvoudig gevonden door de waarde te delen door het aantal bits per *int:*

index = value / 16;

Immers, de vlaggen van 16 getallen kunnen in een *int* worden gerepresenteerd. In het algemeen, wanneer het aantal data-elementen per geheugen-element n is, dan is de geheugen-index voor het gewenste data-element dus:

index = value / n;

Nu nog de bepaling van het juiste bit, gegeven de gevonden *int*. Wanneer het geteste getal bij voorbeeld 5 is, dan is er uiteraard sprake van bit 5 bij *int* index 0. Wanneer het getal echter 17 is, dan is er sprake van bit 1 bij *int* index 1. Deze situatie is weergegeven in figuur 10.1. Uit figuur 10.1 kan worden afgeleid dat het bitnummer dat hoort bij een gegeven getal eenvoudig de rest is van de deling van het getal door 16. Het bitnummer wordt dus gevonden als:

bitnr = value % 16;

In het algemeen, wanneer er n data-elementen in een geheugen-element passen, is de element-index binnen een geheugen-element dus:

element_index = value % n;

Met behulp van de deling en modulo-berekening van het te testen getal door het aantal bits per *int* kan nu eenvoudig het bijpassende array-element en bit worden gevonden.

Om te testen of een bit *op* staat kan gebruik worden gemaakt van de bitwise and-operator. Er moet dan worden gecontroleerd of het bit in het array-element dat correspondeert met het te testen getal *op* staat. Er zijn een paar methoden om een dergelijke test uit te voeren. De de bitwise and-operator moet in ieder geval worden uitgevoerd op het array-element dat het bit van het te testen getal bevat. De andere operand is een *int*

int index: 1

getal:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
bit nr:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

int index: 0

getal:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	
bit nr:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Figuur 10.1: Getallen ontleed in *ints* en bits.

waarde waarin alleen het bit *op* staat dat correspondeert met het bit dat hoort bij het te testen getal. Wanneer het te testen getal bij voorbeeld 5 is, dan moet worden getest of bit 5 van element 0 van het array *op* staat. De vereiste bewerking om het bit van het getal 5 te testen is dus:

element[0] & (1 << 5)

Merk op dat de bewerking $1 << 5$ ervoor zorgt dat er een waarde ontstaat waarin alleen bit 5 is gezet. Een dergelijke waarde wordt een bit-masker (of kortweg: masker) genoemd. Deze waarde kan ook worden aangeduid als (decimaal) 32 of (hexadecimaal) $0x20$.

In het algemene geval wordt niet de waarde 5, maar een getal *value* getest, waarbij element *index* en bit *bitnr* horen. De vereiste bewerking om te testen of het bit dat bij een getal *value* hoort is gezet is dan:

element[index] & (1 << bitnr)

Wanneer het resultaat van deze expressie 0 is, dan is het getal *value* nog niet van de getallenlijn verwijderd. Wanneer het resultaat ongelijk 0 is, dan is het desbetreffende bit gezet. Het getal is dan verwijderd en dus geen priem.

De gebruikte left shift operator is relatief snel. Echter, de variatie in vereiste bitmaskers is gering (er zijn er 16) en is een functie van de bit nummers. In plaats van het masker telkens te berekenen bij een gegeven bit nummer, kan een dergelijke functie ook worden weergegeven met een tabel. Door gebruik te maken van een tabel wordt code vervangen door data. In het algemeen leidt het vervangen van code door data tot snellere programma's, die echter wèl een grotere geheugenbelasting met zich meebrengen. Het vervangen van code door data is een algemeen gebruikte techniek, die ook wel bekend staat als de time-space tradeoff:

tijdwinst wordt geboekt door ruimte op te offeren. Wanneer er een tabel *masker* wordt gebruikt, waarvan de elementen de bitmaskers zijn die bij de 16 bitnummers horen, dan kan de test of een bepaald bit *op* staat worden geïmplementeerd als:

element[index] & masker[bitnr]

Het array *masker* kan door de vertaler worden geïnitialiseerd. De hieronder gegeven definitie van het array is eenvoudig, en maakt gebruik van de left-shift operator op constanten.

```
int
    masker[] =
    {
        1 << 0,
        1 << 1,
        1 << 2,
        1 << 3,
        1 << 4,
        1 << 5,
        1 << 6,
        1 << 7,
        1 << 8,
        1 << 9,
        1 << 10,
        1 << 11,
        1 << 12,
        1 << 13,
        1 << 14,
        1 << 15,
    };
```

Om het bit dat bij een getal *value* hoort te zetten (dat wil zeggen: de waarde 1 te geven), zijn eveneens de index in het array en het bitnummer nodig. In de hier gegeven implementatie van de Zeef van Aristothenes wordt een bit gezet wanneer een getal *value* géén priem blijkt te zijn. Bits kunnen worden gezet met behulp van de bitwise or operator ('|'). Door de bitwise or bewerking uit te voeren op enerzijds het array element dat hoort bij de waarde *value* en anderzijds het masker dat correspondeert met het bitnummer behorend bij de waarde *value,* wordt in ieder geval bereikt dat het gewenste bit wordt gezet, terwijl de overige bits onaangetast blijven. Om bit *bitnr* in element *index* te zetten kan de bitwise or compound assignment operator worden gebruikt[1]:

element[index] |= masker[bitnr];

1. In de implementatie van de Zeef van Aristothenes is het niet nodig een bit te *wissen*. In het algemeen geldt echter dat wanneer de bits in *mask* worden gezet in de variabele *result* met behulp van de bewerking:

result |= mask;

deze bits worden *gewist* door bij *mask* eerst de bitwise not bewerking uit te voeren, gevolgd door de bitwise and compound assignment:

result &= mask;

Implementatie

De implementatie van Zeef van Aristothenes bestaat uit twee statements. Het eerste statement is een initialisatie-statement waarin wordt gecontroleerd of het laatste getal dat moet worden getest in het toegestane waardenbereik ligt en waarin geheugen wordt gealloceerd voor de getallenlijn. Het toegestane waardenbereik is (voorlopig) 1 tot 490.000. De waarde 490.000 is het gevolg van het feit dat op PC-type computers het maximum aantal elementen van een *int* array iets meer dan 30000 is. In hoofdstuk 12 wordt op deze grens nader ingegaan en wordt een manier gepresenteerd om deze grens te passeren. In het initialisatie-statement wordt het geheugen gealloceerd met behulp van de standaardfunctie *calloc()*. De functie *calloc()* verwacht twee argumenten: het aantal elementen dat moet worden gealloceerd en de omvang van de elementen in bytes. De functie alloceert een blok geheugen van de vereiste omvang, initialiseert het geheugen met 0-bytes en retourneert een pointer naar het blok geheugen (of *NULL* wanneer de vereiste hoeveelheid geheugen niet beschikbaar is).

Het tweede statement is een *for*-statement waarin alle getallen op de getallenlijn worden getest. Telkens wanneer een getal wordt getest en het bijbehorend bit is niet gezet, dan is er een priem waargenomen en wordt de priemwaarde op het scherm getoond. Bovendien worden in een genest *for*-statement de bits gezet van alle veelvouden van het gevonden priemgetal.

Een verfijning (en versnelling) van het proces wordt bereikt door de deling en modulo berekening te vermijden. Delingen en vermenigvuldigingen zijn in het algemeen de *duurste* bewerkingen. De modulo waarde wordt gevonden als het *bijproduct* van een deling en is daarmee ook een dure bewerking. De verfijning kan worden gerealiseerd doordat het aantal bits per *int* een macht van 2 is. Daarvan kan gebruik gemaakt worden, omdat de left shift 1 en de right shift 1 bewerkingen respectievelijk een vermenigvuldiging en deling door 2 bewerkstelligen. Om een getal door 16 te delen kan een right shift 4 bewerking worden uitgevoerd[2]. De *rest* van die deling wordt gevonden als de waarde die in de laagste vier bits van het getal staan. De uiteindelijke implementatie van de Zeef van Aristothenes is, tenslotte, gegeven in listing 51.

```
1     #include <stdio.h>
2     #include <malloc.h>
3
4     #define BITS_PER_INT    16                      // bits per int
5     #define SHIFT           4
6     #define MODULO_MASK     0xf
7
8     #define MAX_NUMBER      490000
```

2. Bedenk dat 2^4 gelijk is aan 16. Door de bits 4 posities naar rechts te schuiven wordt 4 keer achtereen gedeeld door 2 en dus door 16.

```
9
10    int
11        mask[] =
12        {
13            1 << 0,
14            1 << 1,
15            1 << 2,
16            1 << 3,
17            1 << 4,
18            1 << 5,
19            1 << 6,
20            1 << 7,
21            1 << 8,
22            1 << 9,
23            1 << 10,
24            1 << 11,
25            1 << 12,
26            1 << 13,
27            1 << 14,
28            1 << 15,
29        };
30
31    void main (int argc, char **argv)
32    {
33        long
34            last,
35            current,
36            next;
37        int
38            *line,
39            nwritten = 0;
40
41        if
42        (
43            !sscanf(argv[1], "%ld", &last)
44            ||
45            last > MAX_NUMBER
46            ||
47            !
48            (
49                line =
50                    calloc((unsigned)((last + BITS_PER_INT - 1) / BITS_PER_INT),
51                           sizeof(int))
52            )
53        )
54        {
55            printf("Usage: %s number\n"
56                 "where:\n"
57                 "       0 < number <= %ld\n", argv[0], MAX_NUMBER);
58            return;
59        }
60
61        for (current = 2; current < last; current++)
62        {
63            if
```

```
64          (
65              !
66              (
67                  line[(unsigned)(current >> SHIFT)]
68                  &
69                  mask[(unsigned)(current & MODULO_MASK)]
70              )
71          )
72          {
73              printf ("%6ld," , current);
74              if (++nwritten == 8)
75              {
76                  putchar ('\n');
77                  nwritten = 0;
78              }
79
80              for
81              (
82                  next = current + current;
83                      next < last;
84                          next += current
85              )
86                  line[(unsigned)(next >> SHIFT)] |=
87                      mask[(unsigned)(next & MODULO_MASK)];
88          }
89      }
90  }
91
```

Listing 51: SIEVE.C: De Zeef van Aristothenes.

Opgave 58.

Vertaal en test het programma *SIEVE.C* Laat het programma ook eens beginnen de priemgetallen tot 480.000 te bepalen. Merk op dat de berekening van de eerste paar priemgetallen relatief traag verloopt, maar dat de volgende priemgetallen steeds sneller worden gevonden.

Opgave 59.

In programma's worden vaak *vlaggen* gebruikt, om aan te geven wat er (niet) moet gebeuren. Voor elke vlag kan een *int* of *char* worden gereserveerd, maar omdat vlaggen ofwel de waarde 1 ofwel de waarde 0 hebben, kunnen ze ook in individuele bits van bijvoorbeeld een *int flags* worden opgeslagen.
Schrijf een aantal macro's, die de bits van een variabele *flags* bewerken. De volgende macro's moeten worden gedefinieerd:

1. clearflag (x) – wist vlag x
2. setflag (x) – zet vlag x
3. testflag (x) – kijkt of vlag x opstaat
4. toggleflag (x) – verwisselt de waarde van vlag x, dus van 1 naar 0 en omgekeerd.

De vlaggen die gezet, gewist, getest en ge-toggle-t worden zijn opgegeven als integer waarden, die één bit in *flags* bereiken. Het argument van de macro's is telkens het gewenste bit nummer.

Test de werking van de macro's met behulp van het volgende programma fragment.

```
#include <stdio.h>

// plaats hier de definities van de macro's

typedef enum
{
    flag_print,
    flag_error,
    flag_goodbye,
} FLAGS_;

FLAGS_
    flags;

void main ()
{
    setflag(flag_print);
    setflag(flag_goodbye);
    clearflag(flag_error);

    if (testflag(flag_print))
        puts("Hello World !");

    if (testflag(flag_error))
        puts("error");

    if (testflag(flag_goodbye))
        puts("Bye bye.");

    toggleflag(flag_error);
    if (testflag(flag_error))
        printf ("retry: error\n");
}
```

Hoofdstuk 11

Code splitsen

11.1 Inleiding

Programma-ontwikkeling waarbij gebruik wordt gemaakt van functies is één van de eigenschappen van **C** die deze taal juist zo geschikt maakt voor het ontwikkelen van grote programma's. Een programma-ontwikkelaar kan op deze wijze gereedschap ontwikkelen dat in latere programma's weer gebruikt wordt; zo wordt vermeden dat het wiel meer dan eens uitgevonden moet worden. Het werken met *modules* vormt een naadloze aansluiting op dit principe. Een functie die eenmaal opgesteld is, wordt vertaald naar een objectmodule; bij volgende programma's hoeft zowel het opstellen van de functie als het vertalen van de code van die functie niet meer te gebeuren. Ook de library manager behoort tot deze opzet; dankzij dit programma kunnen objectmodules in een functiebibliotheek gegroepeerd worden.

Het omgaan met andere functiebibliotheken dan de 'gewone' **C** bibliotheken komt ook ter sprake bij commerciële software voor **C**. Vaak worden pakketten van functies die een bepaald doel hebben, samengevoegd tot één bibliotheek; een software-ontwikkelaar kan dan naar believen functies uit deze bibliotheek gebruiken. De *graphics.lib* bibliotheek, die met de Microsoft vertaler wordt meegeleverd, is zo'n bibliotheek. De hierin opgenomen functies kunnen gebruikt worden voor schermuitvoer bij grafische toepassingen.

In dit hoofdstuk wordt ingegaan op het splitsen van code in modules en op het werken met de linker en met de library manager. Eén en ander wordt aan de hand van twee applicaties geïllustreerd.

11.2 Hoe te splitsen

Bij het splitsen van een brontekst in afzonderlijke modules staat een opzet centraal waarbij elke functie in een aparte brontekst wordt gedefinieerd. Via declaraties (met behulp van het keyword *extern*) wordt dan een functie op de hoogte gesteld van functies of globale variabelen die in andere bronteksten zijn aangemaakt.

Een programma waarbij een *main()* functie gebruik maakt van een hulpfunctie *f()* kan zo gesplitst worden in twee bestanden *main.c* en *f.c*. Dit is geïllustreerd in listing 52.

De programma-ontwikkelaar maakt het zich nog eenvoudiger als bij de constructie van grote programma's een aantal vuistregels wordt aangehouden. We geven hieronder een enige regels, die ook verderop in het boek worden aangehouden:

- Alle bestanden die voor één project bedoeld zijn, krijgen plaats in één directory. Hiermee wordt voorkomen dat, als het aantal bestanden dat bij één project hoort te groot wordt, het overzicht verloren gaat. Bovendien kunnen op deze manier alle bronteksten van één project in één keer verwijderd of gekopieerd worden.

```
/* Bestand: MAIN.C */
extern void                    // declaratie van
    f ();                      // externe functie f()

void main ()
{
    f ();
}
-----------------------------------------------------------------
/* Bestand: F.C */
void f ()
{
    // code van functie komt hier
}
```

Listing 52: Voorbeeld van code splitsen.

- De declaraties van functies en globale variabelen worden in één lokale header file verzameld. Deze header file wordt in alle bronteksten opgenomen met behulp van een *#include* opdracht. Merk op dat lokale header files aangesloten worden met de *#include* directive waarbij dubbele aanhalingstekens gebruikt worden in plaats van de symbolen < en > (zie ook hoofdstuk 6).
- In deze lokale header file staan tevens alle *#define*s en *typedef*s. Op die manier staan de declaraties van constanten, van nieuwe types en van globale variabelen bijeen.
- Het definiëren van globale variabelen gebeurt *niet* in de lokale header file; dan zou immers de definitie in elke module plaatsvinden, doordat elke module de lokale header file aansluit. Een goede plaats om globale variabelen te definiëren is in de module waar de functie *main()* is gedefinieerd, of in een aparte module (bij voorbeeld met de naam *data.c*).
- Elke functie krijgt een plaats in een eigen brontekst. Bij het definiëren van de functie wordt meteen de lokale header file met de declaratie van die functie uitgebreid, waarbij het prototype van de functie gespecificeerd wordt.
  Een uitzondering op deze regel vormen functies die alleen ten dienste van één functie staan en daarom binnen dezelfde module als *static* gedefinieerd zijn. Het prototype van een statische functie moet dan ook niet in de lokale header file komen. Of een functie slechts een *static* is, of dat de functie een globale status verdient, is vaak een kwestie van smaak.
  In het algemeen wordt een functie *static* gedefinieerd als deze functie door slechts één andere functie gebruikt wordt en als deze functie diep in de nesting van de aanroep-volgorde ligt. Het *static* definiëren van een functie levert in de praktijk slechts een kleine winst op; de linker hoeft een kleinere lijst van symbolen bij te houden.

Het voordeel van *static* functies ligt eerder op het conceptuele vlak.

11.3 Bibliotheken en de library manager

De constructie van een programma begint uiteraard met het opstellen van de code en de data in de bijbehorende bronteksten. Vervolgens worden alle bronteksten gecompileerd. Het resultaat hiervan is dat in de directory waarin het project plaats heeft een (groot) aantal object-bestanden staat.

Het aaneenknopen van losse objecten tot één uitvoerbaar programma gebeurt door middel van de linker. Als het project slechts uit een klein aantal modules bestaat, dan kunnen de objecten samen worden gevoegd met behulp van opdracht:

l obj_1 + obj_2 + obj_3 ... + obj_N[1]

Hierbij staan de namen *obj_1 ... obj_N* voor de namen van de afzonderlijke modules. De extensie *.obj* hoeft niet gespecificeerd te worden. De linker maakt een uitvoerbaar programma aan, met de extensie *.exe* en met de naam van de eerst gespecificeerde objectmodule.

Het aaneenknopen van losse objectmodules wordt lastiger als bij een project veel objecten gebruikt worden. Ten eerste wordt de DOS opdracht waarmee objecten gelinkt worden erg lang. Ten tweede staan alle objectmodules als losse bestanden op schijf, hetgeen de schijfruimte beperkt en niet bevorderlijk is voor het overzicht. De library manager biedt dan uitkomst: alle objecten behalve het object dat de functie *main()* bevat worden in één functiebibliotheek verzameld. Bij het link-proces wordt de naam van de bibliotheek opgegeven in plaats van alle namen van objecten. De library manager wordt hierbij aangeroepen met de volgende opdracht[2]:

lib library-naam bewerking object [bewerking object ..] ;

In deze opdracht staat 'library-naam' voor de naam van de functiebibliotheek. De extensie *.lib* hoeft niet gespecificeerd te worden. Het is een goede gewoonte om de bibliotheeknaam gelijk te laten zijn aan de naam van het uiteindelijke programma. Na de bibliotheeknaam staan de opdrachten die de library manager met de functiebibliotheek moet uitvoeren. Elke opdracht bestaat uit een specificatie van een bewerking en uit een specificatie van een objectmodule. De opdracht wordt afgesloten door een ';'.

De library manager kent de volgende bewerkingen:

- : Het – teken specificeert dat de objectmodule waarvan de naam achter de – volgt uit de bibliotheek verwijderd moet worden. Zo houdt opdracht:

1. Opdracht *l* is middels een batch-bestand geïmplementeerd, zie hoofdstuk 1.
2. Merk op dat de library manager meer opties kent dan hier opgesomd. Voor een volledig overzicht verwijzen wij naar de met de vertaler meegeleverde documentatie.

lib mylib -a -b ;

in dat objectmodules *a.obj* en *b.obj* uit de bibliotheek *mylib.lib* verwijderd moeten worden.

+ : Het + teken specificeert dat de objectmodule waarvan de naam achter de + staat in de bibliotheek opgenomen moet worden.

∗ : De ∗ wordt gebruikt als van een module die al eerder in een bibliotheek opgenomen werd een kopie gemaakt moet worden. De library manager maakt bij deze opdracht aan de hand van informatie in de bibliotheek een objectmodule buiten de bibliotheek aan.

Meerdere bewerking-specificaties kunnen achter elkaar, vóór de naam van het betreffende object, opgegeven worden. Vaak gebruikte combinaties zijn de volgende:

−+ : Bij deze bewerking wordt een module eerst uit een bibliotheek gewist en wordt vervolgens een losse module in de bibliotheek opgenomen. Deze bewerking is dus een vervanging. Bijvoorbeeld vervangt de opdracht:

lib mylib −+mod

module *mod.obj* in bibliotheek *mylib.lib*.

∗− : Deze opdracht maakt een kopie van een module in een bibliotheek, en verwijdert vervolgens de module uit die bibliotheek. Na deze bewerking bestaat de module dus alleen als los object.

11.4 Bibliotheken en de linker

In 1.5.1 is besproken dat de taak van de linker het aaneenknopen van objectmodules is. Hiertoe heeft de linker minstens één objectmodule nodig, en zoekt externe referenties[3] die in deze objectmodule genoemd worden op in andere objectmodules of in functiebibliotheken. In dit deel wordt nader ingegaan op het link-proces.

Het aaneenknopen van objectmodules begint bij de eerste, eventueel enige, objectmodule die bij de aanroep van de linker als argument is opgegeven. In het voorbeeld van het 'Hello World' programma (zie listing 1 in 1.6) leidt de opdracht:

l hello

ertoe dat de linker objectmodule *hello.obj* gaat analyseren. In deze module wordt één externe referentie genoemd die de functie *printf()* betreft. De C vertaler neemt in elke vertaalde module de naam op van de bijbehorende standaard functiebibliotheek[4]; door deze informatie kan de linker de

3. De term 'externe referentie' slaat zowel op externe functies als op externe data.
4. In het geval van *hello.c* is dit *slibce.lib* in directory *\C\LIB*.

externe referentie oplossen. Het zoeken naar functies in standaard functiebibliotheken wordt de 'default library search' genoemd.

Als bij de ontwikkeling van programmatuur eigen bibliotheken gebruikt worden, dan moeten de namen van deze bibliotheken aan de linker meegedeeld worden. Dit kan op twee manieren gebeuren:

- De naam van een eigen bibliotheken kan bij de opdracht die de linker activeert opgegeven worden. Wanneer bijvoorbeeld door de objectmodule *prog.obj* een bibliotheek *mylib.lib* gebruikt wordt, dan zorgt de DOS opdracht:

  l prog,,,mylib

  ervoor dat de linker geactiveerd wordt, waarbij *mylib.lib* als extra functiebibliotheek verwerkt wordt. De bibliotheeknaam wordt hierbij met drie komma's van de objectmodulenaam gescheiden[5].
- De naam van de eigen bibliotheken kan in de brontekst van één van de modulen waaruit een programma is opgebouwd worden opgenomen. Voor dit doel is in de Microsoft vertaler de *#pragma* opdracht beschikbaar. Een regel als:

  #pragma comment (lib, "mylib")

  zorgt ervoor dat de vertaler in de gegenereerde objectmodule informatie opneemt dat *mylib.lib* één van de bibliotheken is die in aanmerking komt voor het opzoeken van externe referenties. Bij de aanroep van de linker hoeft deze naam niet meer opgegeven te worden; de bibliotheek is daarmee opgenomen in de default library search van de linker.

11.5 Een applicatie: movelib

Een groot nadeel van de library manager is dat wildcards niet toegestaan zijn bij het opgeven van de opdracht die de library manager oproept. Dat is jammer, want juist bij grote projecten, waar het uiteindelijke programma uit vele objecten wordt opgebouwd, moet in een bibliotheek vaak iets gewijzigd worden. De meest gebruikte opdracht betreft de vervangingsoperatie van één of meer objecten; maar een opdracht als:

*lib mylib –+ *.obj ;*

wordt door de library manager niet verwerkt als verwacht zou worden.

In dit onderdeel wordt daarom een applicatie ontwikkeld die we *movelib* zullen noemen. Ter gedachtenbepaling volgt hier een aantal mogelijke acties van *movelib*:

5. Voor meer informatie over de aanroep van de linker verwijzen we naar de met de vertaler meegeleverde documentatie.

movelib Als het programma zonder argumenten wordt opgestart, dan geeft het programma informatie over het gebruik van dit programma.

movelib lib objects Bij een aanroep met minstens twee argumenten probeert *movelib* objectmodules in een bibliotheek te vervangen. In de specificaties mogen wildcards gebruikt worden.
Movelib zal uit de argumenten een opdracht bouwen, die door het bedrijfssysteem wordt uitgevoerd. Bij een aanroep als:

movelib mylib ∗.obj

zal *movelib* de volgende opdracht uitvoeren (er vanuit gaande dat er twee objectmodules, *a.obj* en *b.obj* zijn):

lib mylib –+a.obj–+b.obj ;

Hierdoor wordt de library manager gactiveerd om *mylib.lib* te bewerken. Zelfs als de objectmodules *a.obj* en *b.obj* niet in de bibliotheek *mylib.lib* aanwezig zijn, gaat de operatie goed: het programma *lib.exe* negeert het verzoek om een objectmodule in de bibliotheek te wissen (de – operatie) en neemt de objectmodule vervolgens in de bibliotheek op (de + operatie).
Nadat de objectmodules in een bibliotheek zijn opgenomen, kunnen de afzonderlijke object-bestanden gewist worden. Ook dit wordt door *movelib* verzorgd.

movelib -t lib objects Bij deze aanroep wordt *lib.exe* niet feitelijk gestart, maar laat *movelib* de actie zien die uitgevoerd zou worden als de –t vlag niet opgegeven was. De –t vlag is een 'tryout-mode'.

11.5.1 De hoofdtaak van movelib

De decompositie van het *movelib* programma begint bij de hoofdtaak: als er bij de aanroep niet genoeg argumenten zijn meegegeven, dan laat het programma informatie over gebruik zien en breekt af. Als wel genoeg argumenten zijn meegegeven, dan kan het programma zijn werk doen.

Het 'echte' werk bestaat òfwel uit het feitelijk aanroepen van programma *lib.exe* òfwel uit het tonen van wat er zou gebeuren (de tryout-mode). De tryout is afhankelijk van het al dan niet aanwezig zijn van –t als eerste argument. De verwerking laten we voorlopig over aan een nader te specificeren functie.

De eerste herschrijving van het probleem ziet er als volgt uit:

1. Als het eerste argument string –t is, dan betreft het de tryout-mode.
2. Als niet voldoende argumenten meegegeven zijn, laat informatie over het gebruik zien en beëindig het programma.
3. Verwerk alle argumenten, ervan uitgaand dat het eerste argument (buiten een eventuele –t) de bibliotheekspecificatie is, en dat alle overige argumenten objectmodules zijn.

```
1     #include <stdio.h>
2     #include <string.h>
3     #include "movelib.h"
4
5     #pragma comment (lib, "movelib")
6
7     int main (int argc, char **argv)
8     {
9         progname = strrchr (argv [0], '\\') + 1;
10        *strrchr (progname, '.') = '\0';
11
12        if (! strcmp (argv [1], "-t"))
13        {
14            tryout++;
15            argv++;
16            argc--;
17        }
18
19        if (argc < 3)
20        {
21            printf ("Usage: %s [-t] library objects\n"
22                    "where: -t      - tryout mode (optional)\n"
23                    "       library - library to move objects to\n"
24                    "       objects - objects to put into library\n"
25                    "%s will call %s to move all objects into the specified\n"
26                    "library. Wild cards are supported.\n\n"
27                , progname, progname, lib_exe);
28            return (1);
29        }
30
31        return (process (argc, argv));
32    }
```

Listing 53: MOVELIB: de functie *main()* in *movelib.c*.

Deze herschrijving is in listing 53 weergegeven. Bij de code maken we de volgende opmerkingen:

Regel 5: De *#pragma* directive is in 11.4 behandeld. Deze opdracht zorgt ervoor dat in de gegenereerde module (dus, in dit voorbeeld in *movelib.obj*) informatie wordt opgenomen dat functiebibliotheek *movelib.lib* bij de default library search van de linker hoort.

Regels 9 en 10: Hier wordt met behulp van *argv[0]* de naam van het programma bepaald. Een *char *progname* wijst na regel 9 voorbij de laatste '\' van *argv[0]*. In regel 10 wordt de extensie *.exe* van de string *argv[0]* afgekapt: op de plaats van de punt wordt een ASCII-Z byte geplaatst.
Het bepalen van een programmanaam aan de hand van *argv[0]* is een service van de programma-ontwikkelaar aan de gebruiker. Zelfs als een gebruiker een programma van een andere naam voorziet, blijft het programma de juiste juiste naam tonen als informatie over het gebruik getoond wordt.

Regels 12 t/m 17: Het eerste argument van de aanroep wordt geïnspecteerd. Als dit argument gelijk is aan string –t (dus, als de standaardfunctie *strcmp()* de waarde nul retourneert) dan wordt een vlag, *int tryout*, verhoogd. De functie van deze vlag komt later aan de orde.

Let bij deze code tevens op het verschuiven van *argv* en *argc*. Hierdoor wordt als het ware een argument gewist; *argv[1]* wordt het argument na de –t. Hierdoor wijst *argv[0]* niet meer op de programmanaam, maar dat is in dit geval niet relevant omdat pointer *progname* dat doet.

Regels 19 t/m 29: Als (op de –t na) minder dan twee argumenten bij de aanroep gegeven zijn, dan laat het programma informatie over het gebruik zien.

Regel 31: Het verwerken wordt overgelaten aan een externe functie *process()*. Deze functie krijgt alle argumenten als *main()*; dus *argv[1]* is de bibliotheeknaam, en *argv[2..argc-1]* zijn de namen van objectmodules.

```
1     // ———————————————————————————— global variables
2     extern char
3         lib_exe [],
4         *progname;
5     extern int
6         tryout;
7
8     // ———————————————————————————— functions
9     extern int
10        process (int, char **);
11
12    extern void
13        command (char *);
14
```

Listing 54: MOVELIB: de lokale header file *movelib.h*.

De de lokale header file *movelib.h* is in listing 54 weergegeven. Let er op dat in de lokale header geen enkele definitie staat; globale variabelen en functies worden enkel gedeclareerd. De definities van globale data is in deze applicatie verzameld in bestand *data.c*, afgebeeld als listing 55.

```
1     #include "movelib.h"
2
3     char
4         lib_exe [] = "LIB.EXE",
5         *progname;
6
7     int
8         tryout = 0;
9
```

Listing 55: MOVELIB: globale data in *data.c*.

11.5.2 Het verwerken van objectmodules

De feitelijke verwerking van het *movelib* programma wordt, zoals blijkt uit listing 53, door de functie *process()* uitgevoerd. De hoofdtaak van *process()*

bestaat uit het bouwen van een DOS-opdracht voor *lib.exe*. Het uitvoeren van deze opdracht laten we voorlopig ongespecificeerd.

Een eigenschap van DOS is dat een opdracht niet langer dan 127 tekens kan zijn. Met deze beperking moet *process()* rekening houden. Het kan dus voorkomen dat ten gevolge van een opdracht:

movelib mylib ∗.obj

het *movelib* programma meer dan eens een *lib* opdracht moet uitvoeren, waarbij per keer de opdracht niet langer dan 127 tekens mag zijn.

De herschrijving van de verwerking ziet er als volgt uit:

3.1. Initialiseer de toekomstige DOS-opdracht met de naam van het op te starten programma (*lib.exe*), gevolgd door een spatie en door de naam van de te bewerken bibliotheek, afgesloten door een spatie.

3.2. Voor alle te bewerken objectmodules:

3.3. Plak achter de DOS-opdracht string −+. Plak hier achter de naam van de objectmodule.

3.4. Als de lengte van de opdracht te groot dreigt te worden (bijvoorbeeld langer dan 100 tekens):

3.5. Voer de opdracht uit.

3.6. Her-initialiseer de DOS-opdracht met *lib.exe* en de bibliotheeknaam, gescheiden door een spatie en afgesloten door een spatie.

3.7. Als in de opgebouwde DOS-opdracht nog te verwerken objectmodules voorkomen, dan moet deze opdracht nog uitgevoerd worden.

3.8. Wis alle verwerkte object-bestanden.

```
1     #include <io.h>
2     #include <stdio.h>
3     #include <string.h>
4     #include "movelib.h"
5
6     int process (int argc, char **argv)
7     {
8         char
9             sysbuf [150];
10        register int
11            flushed,
12            i;
13
14        strcpy (sysbuf, lib_exe);
15        strcat (sysbuf, " " );
16        strcat (sysbuf, argv [1]);
17        strcat (sysbuf, " " );
18
19        for (i = 2; i < argc; i++)
20        {
21            flushed = 0;
22            strcat (sysbuf, "-+");
23            strcat (sysbuf, argv [i]);
24            if (strlen (sysbuf) > 100)
25            {
26                flushed++;
27                command (sysbuf);
28                strcpy (sysbuf, lib_exe);
```

```
29                strcat (sysbuf, " " );
30                strcat (sysbuf, argv [1]);
31                strcat (sysbuf, " " );
32            }
33        }
34
35        if (! flushed)
36            command (sysbuf);
37
38        if (! tryout)
39            for (i = 2; i < argc; i++)
40                unlink (argv [i]);
41
42        printf ("\n%s done. %d objects processed.\n"
43            , progname, argc - 2);
44
45        return (0);
46    }
```

Listing 56: MOVELIB: de functie *process()* in bestand *process.c*.

De code van de functie *process()* is als listing 56 weergegeven. Bij de code maken we de volgende opmerkingen:

- De uit te voeren DOS-opdracht wordt in buffer *sysbuf[]* opgebouwd. Deze string wordt aan de functie *command()* als argument doorgegeven (zie 11.5.3).
- De functie *process()* gebruikt een *int flushed* om aan te geven of na de verwerking van alle objectmodules nog een opdracht uitgevoerd dient te worden. Deze vlagwaarde wordt op 1 gezet als een objectmodulenaam aan de DOS-opdracht wordt toegevoegd, en wordt op 0 gezet als de opdracht uitgevoerd wordt. Na het doorlopen van de lus heeft de vlag dus waarde 1 als in de opdrachtbuffer iets is blijven staan.
- Voor het wissen van object-bestanden wordt gebruik gemaakt van de standaardfunctie *unlink()*. Deze functie geeft een return-waarde terug, die aangeeft of het wissen succesvol is verlopen. Deze return-waarde wordt in dit programma niet geïnspecteerd, omdat het falen van een wis-opdracht niet fataal is voor *movelib*.
- Het opbouwen van de DOS-opdracht illustreert het gebruik van de string-functies *strcpy()*, *strcat()* en *strlen()*. De functie *strcpy()* kopieert een string naar een andere; de functie *strcat()* concateneert een string aan een andere en de functie *strlen()* retourneert de lengte van een string.

Opgave 60.

Wijzig de functie *process()* zodanig dat als het wissen van een object file faalt een waarschuwing gegenereerd wordt.

11.5.3 Het uitvoeren van een DOS-opdracht

Het uitvoeren van een DOS-opdracht is in het *movelib* programma overgelaten aan functie *command()*. Deze functie krijgt één argument: de string die als opdracht uitgevoerd moet worden. De functie *command()* is als listing 57 weergegeven.

```
1     #include <errno.h>
2     #include <stdio.h>
3     #include <stdlib.h>
4     #include <string.h>
5     #include <process.h>
6     #include "movelib.h"
7
8     void command (char *cmd)
9     {
10        strcat (cmd, ";");
11
12        if (tryout)
13            printf ("%s tryout: %s\n", progname, cmd);
14        else
15            if (system (cmd))
16            {
17                printf ("%s: Execution error -" , progname);
18                switch (errno)
19                {
20                    case E2BIG:
21                        puts ("argument list too long");
22                        break;
23                    case ENOENT:
24                        puts ("program not found");
25                        break;
26                    case ENOEXEC:
27                        puts ("error in executable");
28                        break;
29                    case ENOMEM:
30                        puts ("out of memory");
31                        break;
32                }
33                exit (1);
34            }
35    }
```

Listing 57: MOVELIB: de functie *command()* (bestand *command.c*).

Het aanroepen van het bedrijfssysteem gebeurt vanuit **C** programma's gewoonlijk door middel van de standaardfunctie *system()*. Deze functie retourneert een waarde ongelijk aan 0 als bij de executie van de opdracht een fout is opgetreden. In dat geval bevat de externe variabele *errno* een aanduiding van de soort fout. Bij het optreden van een fout drukt de functie *command()* een bijbehorende foutmelding af en beëindigt het programma.

Opgave 61.

Voer alle bronteksten in die bij het programma *movelib* horen. Vertaal de bronteksten.

Opgave 62.

Zet alle verkregen objectmodules, op *movelib.obj* na, in bibliotheek *movelib.lib*. Wis de in de bibliotheek opgenomen objectmodules.

Opgave 63.
Maak met behulp van de linker het programma *movelib.exe*.

Opgave 64.
Zet programma *movelib.exe* in de directory waar de programmatuur staat die bij de C vertaler hoort.

11.6 Een help-systeem applicatie

11.6.1 Inleiding

Het probleem dat we ons stellen is de constructie van een help faciliteit zoals die op het VAX/VMS bedrijfssysteem aanwezig is. Hierbij kan informatie over een bestaande opdracht (bijvoorbeeld *dir*) worden opgevraagd met behulp van opdracht *help dir*. De help faciliteit van VAX/VMS zoekt dan in een databestand de informatie over opdracht *dir*, en laat die zien. Er kan niet alleen informatie over een bepaalde opdracht worden gegeven, maar eveneens kan een overzicht van alle opdrachten opgevraagd worden.

Bij de ontwikkeling van deze applicatie komen de volgende onderwerpen ter sprake:

- *typedef*s, *enum*s en *struct*s,
- verwerking van bestanden,
- het werken met een lokale header file en het werken met de library manager.

Ter gedachtenbepaling laten we alvast zien hoe de help faciliteit er 'van de buitenkant' uit moet zien; met andere woorden wat de gebruiker kan verwachten:

- De opdracht *help*, zonder argumenten, laat zien waarvoor het programma bedoeld is en wat het programma doet.
- Om een overzicht van mogelijke onderwerpen te krijgen, kan opdracht *help -o* (o van overview) gegeven worden.
- Alle andere opdrachten, zoals *help dir*, zorgen ervoor dat de help faciliteit probeert om informatie over het onderwerp af te drukken. Bij de aanroep van de help faciliteit mogen meerdere onderwerpen opgegeven worden.
- De schermuitvoer van de help faciliteit wordt naar de standaard uitvoerstroom *stdout* geschreven, en kan daarmee door de gebruiker naar een bestand gestuurd worden als bij de aanroep het DOS redirectieteken > gebruikt wordt.

Bij de constructie van de help faciliteit zal, evenals bij het VAX/VMS bedrijfssysteem, van een databestand gebruik gemaakt worden. Hierdoor kunnen onderwerpen aan de database van de help faciliteit toegevoegd worden en kan informatie gewijzigd worden.

Voor het snel opzoeken van informatie over een onderwerp zal de help faciliteit gebruik maken van een indexbestand. Het voordeel hiervan is dat de plaats van een bepaald onderwerp snel opgezocht kan worden in het indexbestand, waarna de feitelijke informatie in het databestand wordt opgezocht. De snelheidswinst wordt duidelijker naarmate de omvang van het databestand toeneemt. Om het zoeken naar informatie over een bepaald onderwerp verder te versnellen, wordt het indexbestand gesorteerd op alfabetische volgorde van de onderwerpen.

Het bijhouden van het indexbestand zal 'achter de schermen' gebeuren, dat wil zeggen zonder dat de gebruiker er iets van merkt. Concreet betekent dit dat de help faciliteit soms een indexbestand zal moeten maken. De eerste situatie waarbij een indexbestand aangemaakt moet worden, is als er nog geen indexbestand bestaat; bijvoorbeeld bij de eerste installatie van het programma. Een tweede situatie waarbij een indexbestand aangemaakt moet worden, treedt op als informatie in het databestand gewijzigd wordt. In dat geval moet een recentere versie van de index gemaakt worden.

Wat de uiteindelijke vorm van de help faciliteit betreft spreken we de volgende bestandsnamen af:

- *help.exe* voor het programma van de help faciliteit;
- *help.idx* voor het indexbestand;
- *help.dat* voor het databestand.

Verder spreken we af dat het indexbestand en het databestand in dezelfde directory staan als *help.exe*.

11.6.2 Organisatie van het databestand

In de hier voorgestelde opzet is het de bedoeling dat gebruikers de mogelijkheid krijgen hun databestand bij te werken. De help faciliteit moet aangebrachte wijzigingen kunnen verwerken; hieruit vloeit voort dat het databestand op een bepaalde manier georganiseerd moet zijn. Het databestand moet dus een bepaald *formaat* volgen.

We spreken het volgende formaat voor het databestand af:

- Onderwerpen zijn gegeven als tekst, die tussen [en] staat. De eerste vierkante haak moet daarbij op de eerste positie van de regel staan.
- Het databestand mag commentaar bevatten. Commentaarregels beginnen met een # op de eerste positie van de regel. Deze opzet is ontleend aan commentaar in batch-bestanden (script files) in het UNIX bedrijfssysteem.

- Alle andere regels bevatten feitelijke informatie over een onderwerp.

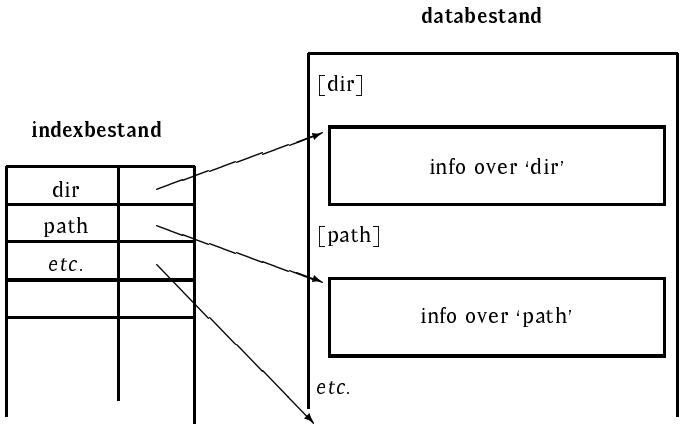
Hieronder is een voorbeeld van een databestand gegeven:

```
# dit is de data file voor HELP

# informatie over opdracht DIR
[dir]
    Laat overzicht van bestanden zien. Optioneel mag een pad
    meegegeven worden (zie "path"). Daarachter mogen optioneel
    vlaggen staan: "/w" voor 'wide' printen (in kolommen) of "/p"
    voor 'pauze' na een scherm vol.

    Samenvatting:
        DIR [path] [flags]
            path  - pad van file(s) die getoond moeten worden;
                    default is "*.*"
            flags - /w voor 'wide' printing
                    /p voor 'pauze' na een scherm vol informatie

# informatie over padnaam, en over opdracht PATH
[path]
    (1)
    Geeft aan waar een file (of files) gezocht moeten worden. Bestaat uit
    drie optionele delen: de drive (bijvoorbeeld "a:"), de directory of
    directories (bijvoorbeeld "\bin\tmp") en de filenaam zelf
    (bijvoorbeeld "help.dat"). Sommige opdrachten staan wildcards (zie
    "wildcard") toe in de filenaam specificaties, bijvoorbeeld "dir" (zie
    "dir").
    (2)
    Geeft aan waar DOS programma's moet zoeken. Wordt gezet door
        PATH directories                    of
        PATH=directories                    of
        SET PATH=directories
    Hierbij mogen meerdere directories opgegeven worden; die dan
    gescheiden moeten zijn door ';' .
    (3)
    Geeft een overzicht van de PATH instelling (zie punt 2), als
    achter PATH geen argumenten zijn gegeven.

# informatie over wildcards
[wildcard]
    ...... bla bla bla ......
```

11.6.3 Organisatie van het indexbestand

Het indexbestand zal bij de interne verwerking van het databestand gebruikt worden. Het indexbestand moet daarvoor informatie bevatten over onderwerpen en over hun plaats in het databestand. Een voor de hand liggende opzet is daarom het indexbestand te laten bestaan uit een aantal informatieblokken, die per onderwerp de naam en de plaats in het databestand weergeven. De opzet van verwijzingen naar het databestand door middel van het indexbestand is in figuur 11.1 geïllustreerd.

Om het verwerken van het indexbestand te versnellen kiezen we voor een aanpak waarbij een naam van een onderwerp *exact* 16 bytepposities in het indexbestand zal innemen. De plaats van dit onderwerp in het databestand wordt als vier bytes, dus als één *long*, bewaard. De informatie over één onderwerp wordt dus opgeslagen in 20 bytes. Het voordeel



Figuur 11.1: Verwijzingen in het indexbestand naar het databestand.

hiervan is dat bij het lezen en schrijven van het indexbestand vaste blokken geschreven worden. Als voorbeeld van een indexbestand geven we hieronder de uitvoer van het *hexdump* programma[6], waarmee een indexbestand 'gedumpt' werd. De uitvoer van *hexdump* is in drie blokken van 20 bytes gesplitst. Het eerste blok geeft weer dat informatie over onderwerp *dir* te vinden is op offset 0x4a in het databestand. Het tweede blok geeft weer dat informatie over *path* op offset 0x272 staat. Het derde blok geeft weer dat het databestand wordt afgesloten met informatie over onderwerp *wildcard*; dit onderwerp begint op offset 0x5a0 in het databestand.

```
64 69 72 00 00 00 00 00 00 00 00 00 00 00 00 00 4A 00 00 00
                                        dir.................
70 61 74 68 00 00 00 00 00 00 00 00 00 00 00 00 72 02 00 00
                                        path................
77 69 6C 64 63 61 72 64 00 00 00 00 00 00 00 00 A0 05 00 00
                                        wildcard............
```

Omdat in het indexbestand letterlijke byte-waarden voorkomen, moet het bestand in binaire modus gelezen en geschreven worden. Het moet immers vermeden worden dat het **C** runtime systeem bij lezen en schrijven een \n-byte vervangt door \r\n.

6. Het *hexdump* programma toont de inhoud van een bestand in hexadecimale- en character-notatie. Voor meer informatie verwijzen we naar DOS handboeken; bij voorbeeld de DOS Encyclopedia.

11.6.4 Probleemanalyse en implementatie

Bij de implementatie van de help faciliteit zal gebruik worden gemaakt van één lokale header file, waarin declaraties van functies en globale variabelen staan. Elke functie wordt in een eigen brontekst gedefinieerd.

De hoofdtaak van de help faciliteit

De eerste herschrijving van het probleem betreft de hoofdtaak van de help faciliteit. Deze taak kunnen we als volgt herschrijven:

1. Stel de programmanaam vast en de directory waarin het programma zich bevindt. De programmanaam wordt gebruikt bij het tonen van informatie over het programma, deze opzet is analoog aan die van het programma *movelib*, zie listing 53.
   De directory waarin het programma staat moet bekend zijn als verderop in het programma de naam van het databestand en de naam van het indexbestand gebruikt moeten worden. Deze namen zijn identiek aan de volledige programmanaam maar met de extensie *.dat* respectievelijk *.idx*.
2. Als bij de aanroep van *help.exe* de '–o' vlag is meegegeven, dan zullen de onderwerpen die in het databestand voorkomen afgedrukt moeten worden. Zet in dit geval een vlag op.
3. Beëindig het programma met passende informatie als bij de aanroep van *help.exe* minder dan twee argumenten zijn meegegeven.
4. Open het databestand en het indexbestand, en lees de indexinformatie in.
5. Toon een een lijst van onderwerpen, indien gewenst.
6. Toon informatie voor alle onderwerpen die bij de aanroep zijn opgegeven.

In listing 58 van de functie *main()* zijn deze herformuleringen terug te vinden. Punten 4, 5 en 6 worden in nader te specificeren functies gedefinieerd.

```
1     #include <stdio.h>
2     #include <string.h>
3     #include "help.h"
4
5     #pragma comment (lib, "help")
6
7     int main (int argc, char **argv)
8     {
9         register unsigned
10            show_subjects = 0,
11            i;
12
13        progname =                    // tekst 1
14            strrchr (argv [0], '\\') + 1;
15        *strrchr (progname, '.') = '\0';
16        argv0 = argv [0];
17
18        if (! strcmp (argv [1]. "-o")) // tekst 2
19        {
20            show_subjects++;
```

```
21            argv++;
22            argc--;
23        }
24
25        if (argc < 2)                  // tekst 3
26            return (usage ());
27
28        init ();                       // tekst 4
29
30        if (show_subjects)             // tekst 5
31            showsub ();
32
33        for (i = 1; i < argc; i++)     // tekst 6
34            showhelp (argv [i]);
35
36        return (0);
37    }
```

Listing 58: De functie *main()* (bestand *help.c*).

Opgave 65.
Wat zal, gezien listing 58, het type zijn van:
a. variabele *progname*
b. variabele *argv0*

De functie *usage()*, die in *main()* aageroepen wordt, toont informatie over het gebruik van het programma *help.exe*. Deze functie is in listing 59 weergegeven.

In de functie *usage()* wordt een aantal globale variabelen gebruikt. Vooruitlopend op de definitie van deze variabelen volgt hier een beschrijving:

- *char version[]* is een string waarin het versienummer van de help faciliteit staat. Het is een goede gewoonte *elk* programma van een nummer te voorzien; zo kan bij het optreden van fouten aan een bepaalde versie gerefereerd worden. Nieuwere versies zijn op deze manier gemakkelijk te onderscheiden van oudere versies.
  Gewoonlijk bestaat een versienummer uit een hoofd-versienummer en een sub-versienummer[7]. Het hoofd-versienummer wordt pas veranderd als een programma substantiële wijzigingen ondergaat.
- *char data_ext[]* is een string die de extensie van het databestand specificeert. Deze extensie is .dat (zie ook bestand *data.c* in listing 63).
- *char index_ext[]* is een string die de extensie van het indexbestand specificeert. Deze extensie is .idx.

```
1     #include <stdio.h>
2     #include "help.h"
3
4     unsigned usage ()
5     {
```

7. We bedoelen hier *niet* subversie in de betekenis van omverwerping.

```
6         printf ("Help Utility V%s\n"
7                 "Usage: %s [flags] subject\n"
8                 "\n"
9                 "%s will show help on <subject>, if found in the help data file.\n"
10                "An index file is built if necessary.\n"
11                "Default data file  : %s%s\n"
12                "Default index file : %s%s\n"
13                "Recognized flags are:\n"
14                "   -o      -       shows overview of subjects\n"
15                /* enter other flag definitions here */
16            , version, progname, progname, argv0, data_ext, argv0, index_ext);
17        return (1);
18    }
```

Listing 59: De functie *usage()* (bestand *usage.c*).

Verwerking van het indexbestand

De tweede herschijving van het probleem richt zich op de verwerking van de index- en databestanden. De help faciliteit zal, afhankelijk van het bestaan van de data- en indexbestanden, de volgende drie taken moeten kunnen volbrengen:

- De eerste taak is het lezen van het indexbestand. Deze actie wordt uitgevoerd als het indexbestand bestaat, en als de datum en tijd van het databestand en van het indexbestand aangeven dat het indexbestand recent is. We kunnen ervan uitgaan dat het indexbestand recent is als de datum van dit bestand een later tijdstip aangeeft dan het datum van het databestand; met andere woorden als het indexbestand *jonger* is.
- De tweede taak is het aanmaken van een index aan de hand van het databestand. Deze actie wordt uitgevoerd als het indexbestand niet bestaat, of als het indexbestand *ouder* is dan het databestand. Bij de implementatie van de help faciliteit zal de index in het geheugen, dus als data van het programma, aangemaakt worden.
- De derde taak is het schrijven van de index naar het indexbestand.

Punt 4 kan nu als volgt worden herschreven:

4.1. Maak in een string-buffer de namen van het databestand en van het indexbestand. Deze namen kunnen aan de hand van de directory waar het aangeroepen programma in staat en aan de hand van de vereiste extensies gemaakt worden.

4.2. Open het databestand.

4.3. Als het indexbestand niet bestaat, dan moet een nieuw indexbestand gemaakt worden. Open een nieuw indexbestand, maak een index aan de hand van het databestand, en schrijf de index naar het indexbestand.

4.4. Als het indexbestand wel bestaat en als dit bestand ouder is dan het databestand dan moet, evenals bij 4.3, een nieuw indexbestand gemaakt worden.

4.5. Als het indexbestand bestaat en als dit bestand jonger is dan het databestand, dan moet de index van dit bestand ingelezen worden.

Het inlezen van het indexbestand laten we voorlopig ongespecificeerd. Deze taak zal door de functie *read_index()* uitgevoerd worden. Ook de constructie van een index aan de hand van het indexbestand laten we over aan een nader te specificeren de functie *build_index()*. Tevens stellen we het schrijven van het indexbestand uit; deze taak zal door functie *write_index()* uitgevoerd worden.

De implementatie van punten 4.1 tot en met 4.5 is in listing 60 weergegeven. In deze listing wordt gebruik gemaakt van de standaard-**C** functies *strcpy()* en *strcat()*. Deze functies zijn al eerder genoemd.

Een andere standaardfunctie die gebruikt wordt is *fstat()*. Deze functie verwacht twee argumenten: het handle-nummer van een stream die geopend moet zijn en het adres van een variabele van type *struct stat*. De functie *fstat()* vult de velden van deze variabele met informatie over het bestand dat bij de stream hoort. In deze applicatie wordt alleen veld *st_atime* van deze *struct* geïnspecteerd. Dit veld geeft de datum weer van de laatste wijziging van het betreffende bestand.

```
1     #include <stdio.h>
2     #include <types.h>
3     #include <stat.h>
4     #include <string.h>
5     #include "help.h"
6
7     void init ()
8     {
9         struct stat
10            index_buf,
11            data_buf;
12
13        strcpy (index_fname, argv0);   // tekst 4.1
14        strcat (index_fname, index_ext);
15
16        strcpy (data_fname, argv0);
17        strcat (data_fname, data_ext);
18
19        if (! (fdata = fopen (data_fname,// tekst 4.2
20                              "r")))
21            error (cannot_open, data_fname);
22
23        if (! (findex = fopen (index_fname,// tekst 4.3
24                               "r+b")) )
25        {
26            if (! (findex = fopen (index_fname, "wb")) )
27                error (cannot_open, index_fname);
28            build_index ();
29            write_index ();
30        }
31        else                            // tekst 4.4
32        {
33            if ( fstat (fileno (fdata), &data_buf) ||
34                 fstat (fileno (findex), &index_buf)
35               )
36                error ("Cannot get file statistics (index/data file)");
37            if (index_buf.st_atime <= data_buf.st_atime)
38            {
```

```
39                  build_index ();
40                  write_index ();
41              }
42              else
43                  read_index ();          // tekst 4.5
44          }
45      }
```

Listing 60: De functie *init()* (bestand *init.c*).

Opgave 66.

In de functie *init()* wordt een functie *error()* genoemd. Deze functie wordt aangeroepen als een handeling faalt die voor het programma zo belangrijk is dat ten gevolge van het falen het programma beëindigd moet worden. Stel de functie *error()* op aan de hand van de volgende specificaties:
a. De functie *error()* wordt aangeroepen met een variabel aantal argumenten, waarbij het eerste argument altijd een string is. De aanroep van *error()* is analoog aan de aanroep van *printf()*.
b. De functie *error()* schrijft als eerste de melding 'Fatal error:' naar de standaardstream *stderr*.
c. Vervolgens schrijft *error()* de informatie die door de parameters is gegeven naar *stderr*.
d. Daarna wordt melding 'Aborting.' naar *stderr* geschreven.
e. Tenslotte beëindigt *error()* het programma met een exit status 2.

Het wordt nu tijd om de datastructuur die naar informatie in het databestand verwijst nader te specificeren. Gezien de organisatie van het indexbestand (zie figuur 11.1) moet een verwijzing de naam van het onderwerp en de plaats van de informatie over dit onderwerp in het databestand bevatten. De *typedef* voor een verwijzing is hieronder gegeven:

```
typedef struct
{
    char
        *name;
    long
        filepos;
} SYMTAB_;
```

In de gedefinieerde structuren zijn twee velden opgenomen: een *char*∗*name* en een *long filepos*. De pointer *name* wordt gebruikt om naar de naam van een onderwerp, opgeslagen als ASCII-Z string, te wijzen. Waarde *filepos* is de offset in het databestand waar informatie over het onderwerp te vinden is.

In listing 61 is de functie *read_index()* weergegeven. Deze functie is verantwoordelijk voor het inlezen van het indexbestand. De bijbehorende stream is in de functie *init()* al geopend, en wordt aangeduid met de variabele *findex*. De opslag van gelezen indexinformatie wordt voorlopig uitgesteld tot de functie *assert()*.

```
1     #include <io.h>
2     #include <stdio.h>
3     #include "help.h"
```

```
4
5     void read_index ()
6     {
7         long
8             pos;
9         char
10            buf [17];
11
12        while (1)
13        {
14            if (! fread (buf, 16, 1, findex) )
15                return;
16            if (! fread (&pos, sizeof (long), 1, findex) )
17                error ("Read failure while scanning index file %s", index_fname);
18            assert (buf, pos);
19        }
20    }
```

Listing 61: De functie *read_index()* (bestand *readin.c*).

De gelezen informatie wordt opgeslagen als een serie *SYMTAB_*[8] structuren. Voor deze opslag moet dus een aantal van deze structuren beschikbaar zijn. Eén van de manieren om een aantal structuren te maken, is het definiëren van een array van *SYMTAB_* structuren. Volgens deze methode is een maximaal aantal structuren beschikbaar, bijvoorbeeld 100. Als in het indexbestand meer dan 100 verwijzingen staan, kan het programma alleen de eerste 100 verwerken; dit legt aan het programma een onnodige beperking op. We zullen daarom voor een andere aanpak kiezen, waarbij het aantal *SYMTAB_* structuren niet van een van tevoren vastgestelde waarde afhangt, maar waarbij dit aantal runtime bepaald wordt. Deze manier houdt in dat de reeks *SYMTAB_* structuren moet worden gealloceerd.

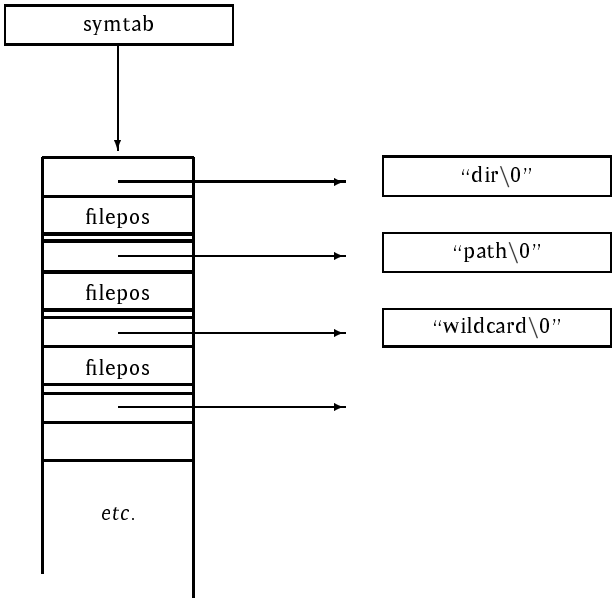
Concreet betekent deze aanpak dat de reeks *SYMTAB_* structuren door een pointer wordt aangewezen. De reeks wordt naar de juiste omvang gerealloceerd als een element aan de reeks toegevoegd moet worden. Deze aanpak is in figuur 11.2 geïllustreerd. Hier wordt de tabel van *SYMTAB_* structuren aangewezen door de pointer *symtab*. Het aantal elementen in de tabel wordt bijgehouden door de variabele *nsym*.

De complete lokale header file *help.h* is in listing 62 weergegeven. In deze header file wordt onder andere het type *SYMTAB_* gedefinieerd, en wordt een pointer *symtab* van dit type gedeclareerd. Daarnaast wordt het type *LEXER_TOKEN_* gedefinieerd. Dit type komt op pagina 248 aan de orde.

In listing 63 vinden de definities plaats van alle globale variabelen die in *help.h* gedeclareerd zijn.

```
1     // ———————————————————————————— typedefs
2     typedef enum                        // return-waarden van lexer:
3     {
4         subject,                        // een onderwerp, of
```

8. De naam *SYMTAB_* is niet toevallig gekozen. Het is een samentrekking van symbol table', en is afkomstig van de vertalerbouw. Meestal verwijzen variabelen met deze naam naar een tabel van symbolische namen, die voor een programma belangrijk zijn.



Figuur 11.2: De tabel *SYMTAB_*.

```
5         textline,                       // een tekstregel
6         // EOF is hier niet opgenomen
7     } LEXER_TOKEN_;
8
9     typedef struct                      // symbolentabel:
10    {
11        char
12            *name;                      // onderwerpnaam
13        long
14            filepos;                    // offset in het databestand
15    } SYMTAB_;
16
17    // ———————————————————————————— globale variabelen
18    extern SYMTAB_
19        *symtab;                        // ptr naar symbolentabel
20    extern char
21        *argv0,                         // programmma-directory
22        cannot_open [],                 // foutmelding
23        data_ext [],                    // extensie van databestand
24        data_fname [],                  // naam van databestand
25        index_ext [],                   // extensie van indexbestand
26        index_fname [],                 // naam van indexbestand
27        lexbuf [],                      // buffer van lex. analyzer
28        version [],                     // programmaversie
29        *progname;                      // programmanaam
30
31    extern FILE
32        *fdata,                         // data stream
33        *findex;                        // index stream
34
35    extern unsigned
36        nsym;                           // aantal symbolen in tabel
37
38    // ————————————————————————————————— functies
39    extern unsigned
40        lookup (char *),                // zoekt onderwerp op
41        usage (void);                   // toont gebruiks-info
42
43    extern int
44        symcomp (SYMTAB_ *, SYMTAB_ *);// vergelijkt 2 symbolen
45
46    extern LEXER_TOKEN_
47        lexer (void);                   // lexical analyzer
48
49    extern void
50        assert (char *, long),          // voegt elem. toe aan symtab
51        build_index (void),             // bouwt index
52        error (char *, ...),            // foutmelding + afbreken
53        init (void),                    // initialiseert bestanden
54        read_index (void),              // leest index
55        showhelp (char *),              // toont info over een onderwerp
56        showsub (void),                 // toont onderwerpen
57        write_index (void);             // schrijft indexbestand
```

Listing 62: De lokale header file *help.h*.

```
1     #include <stdio.h>
2     #include "help.h"
3
4     SYMTAB_
5         *symtab;
6
7     char
8         *argv0,
9         cannot_open []      = "Cannot open file %s",
10        data_ext []         = ".DAT",
11        data_fname [128],
```

```
12          index_ext []          = ".IDX",
13          index_fname [128],
14          lexbuf [201],
15          version []            = "1.00",
16          *progname;
17
18      FILE
19          *fdata,
20          *findex;
21
22      unsigned
23          nsym;
24
```

Listing 63: De globale data in bestand *data.c*.

Het toevoegen van informatie aan de tabel van *SYMTAB_* structuren gebeurt door de functie *assert()*. Deze functie wordt aangeroepen met twee argumenten: de naam van het toe te voegen onderwerp en de positie in het databestand van de informatie over dit onderwerp. We kunnen de taken van *assert()* als volgt specificeren:

1. Kijk na of het onderwerp al in de tabel voorkomt. Zo ja, breek af met een foutmelding.
2. Vergroot de tabel met één element.
3. Neem de informatie die als parameters is meegegeven op in de tabel.

Punt 1 kan als een *for* statement geïmplementeerd worden. Hierbij wordt veld *name* van elk element van *symtab* geïnspecteerd, en vergeleken met de naam van het nieuwe onderwerp. Voor het vergelijken van beide strings kan gebruik worden gemaakt van de functie *strcmp()*. Als beide strings identiek zijn, wordt het programma met een foutmelding beëindigd.

Punt 2, het vergroten van de tabel, wordt met *realloc()* gerealiseerd. De functie *realloc()* verwacht twee argumenten: een pointer naar het te realloceren geheugen, en de omvang in bytes die voor dit stuk geheugen gereserveerd moet worden. Het eerste argument is *symtab*. Het tweede argument kan bepaald worden door bij het aantal al aanwezige elementen (*nsym*) één op te tellen en deze waarde met de omvang van een *SYMTAB_* struct te vermenigvuldigen.

De functie *realloc()* retourneert een pointer naar het gerealloceerde gebied. Deze waarde dient in *assert()* weer aan *symtab* toegekend te worden; anders gaat de verwijzing naar de symbolische tabel verloren. Als het realloceren faalt, dan retourneert *realloc()* waarde *NULL*. In dat geval moet het programma met een passende foutmelding beëindigd worden.

Bij punt 3 moeten de parameters van *assert()* in het aangemaakte element van tabel *symtab* worden geplaatst. Het opnemen van de *long* waarde die de offset van informatie in het databestand weergeeft, kan door middel van een toekenning (assignment) gebeuren. Het opnemen van de naam van het onderwerp is iets gecompliceerder. Eerst moet de string die

het onderwerp weergeeft in het geheugen gedupliceerd worden (dit kan met de functie *strdup()*[9] gebeuren). Het betreffende *symtab* element heeft een pointer-veld, dat de return-waarde van *strdup()* dient te krijgen. Als de string-duplicatie faalt, dan moet het programma beëindigd worden.

Opgave 67.

Implementeer de functie *assert()* aan de hand van bovenstaande specificaties. Let er op dat de variabele *nsym* verhoogd moet worden als een nieuw element in de *symtab* tabel is aangemaakt.

Het wegschrijven van de indexinformatie naar het indexbestand gebeurt door de functie *write_index()* en is in listing 64 weergegeven. In deze functie wordt ervan uitgegaan dat stream *findex* al geopend is in de functie *init()*. Eerst wordt het indexbestand naar een omvang van 0 bytes teruggebracht. Vervolgens worden in een *for* statement alle *symtab* blokken daar de stream geschreven.

```
1     #include <io.h>
2     #include <stdio.h>
3     #include "help.h"
4
5     void write_index ()
6     {
7         register unsigned
8             i;
9
10        if (chsize (fileno (findex), 0L))
11            error ("Cannot reset index file %s", index_fname);
12        for (i = 0; i < nsym; i++)
13            if (! fwrite (symtab [i].name, 16, 1, findex) ||
14                ! fwrite (& (symtab [i].filepos), sizeof (long), 1, findex)
15               )
16                error ("Write failure while saving index file %s", index_fname);
17    }
```

Listing 64: De functie *write_index()* (bestand *writein.c*).

Het aanmaken van een nieuw indexbestand

Als in de functie *init()* (zie listing 60) blijkt dat nog geen indexbestand bestaat, of als blijkt dat het indexbestand niet meer recent is, dan wordt de functie *build_index()* aangeroepen om een nieuw indexbestand te maken.

Het maken van nieuwe indexinformatie behelst het analyseren van het databestand. In overeenstemming met de filosofie van modulair programmeren zullen we deze analyse overlaten aan een aparte functie. Deze functie zal als een *voelspriet* van het programma fungeren; het herkennen

9. De functie *strdup()* roept *malloc()* aan om een stukje geheugen te alloceren. Vervolgens wordt de string die als argument is meegegeven naar dit stuk geheugen gekopieerd. De functie *strdup()* retourneert een pointer naar het geheugen dat een duplicaat van de string bevat. Als alloceren faalt, dan retourneert *strdup()* waarde *NULL*. Voor meer informatie verwijzen we naar de documentatie.

van gegevens die in het databestand staan wordt aan deze functie overgelaten, en de rest van het programma hoeft zich niet bezig te houden met vragen als *hoe* deze gegevens herkend zijn of aan de hand van *welke kenmerken* dit gebeurd is.

We introduceren hier de term 'lexical analyzer'. Deze term, die afkomstig is uit de vertalerbouw, duidt gewoonlijk een functie aan die op lexicografische wijze invoer verwerkt. De lexical analyzer analyseert invoer aan de hand van *syntactische regels*, en informeert andere delen van het programma over de gelezen informatie. De aard van de gelezen informatie wordt de *syntactische waarde* van de invoer genoemd. De inhoudelijke informatie wordt de *semantische waarde* genoemd. Zo is in de zin 'De bank zit op opa en oma' geen syntactische fout te vinden; er is sprake van een onderwerp, van een gezegde, enzovoort. Semantisch klopt de zin echter niet[10].

De lexical analyzer wordt gewoonlijk geconstrueerd zodat de opgeleverde syntactische en semantische informatie zo eenvoudig mogelijk door overige programmadelen verwerkt kan worden. Een voorbeeld hiervan is de lexical analyzer van de **C** vertaler, die spaties, tabulatietekens en newline-tekens overslaat. De rest van de **C** vertaler wordt dus niet belast met het wel of niet voorkomen van deze tekens in de invoer. Een tweede voorbeeld van de vereenvoudiging door de lexical analyzer van de **C** vertaler wordt gedemonstreerd door het feit dat de lexical analyzer een aantal ASCII-tekens in de invoer als één syntactisch teken aan andere delen van de **C** vertaler kan doorgeven. Zo wordt het keyword *else*, dat als vier ASCII-tekens in een brontekst voorkomt, als één symbolisch getal doorgegeven aan de rest van de vertaler. In het geval van een *else* keyword is er alleen een syntactische waarde. Als de lexical analyzer een getal leest, dan wordt een syntactische waarde die een nummer representeert teruggegeven; maar hierbij is tevens sprake van semantische informatie die de numerieke waarde van de gelezen invoer representeert.

In de praktijk wordt een lexical analyzer geïmplementeerd als één functie, die met behulp van de return-waarde informatie verschaft over de verwerkte syntactische informatie. Als de verwerkte invoer tevens semantische informatie heeft, dan wordt deze waarde in een globale variabele opgeslagen.

Gezien de opbouw van een databestand (zie pagina 237) kan de lexical analyzer van de help faciliteit drie soorten invoer tegenkomen:

- Een regel kan commentaar zijn. Commentaarregels worden in het databestand aangegeven door een #-teken op de eerste kolom van de regel.
- Een regel kan een onderwerp specificeren. Onderwerpen worden in het databestand aangegeven als tekst tussen [en], waarbij de

10. Tenzij bedoeld wordt dat een financiele instelling opa en oma op de nek zit.

eerste vierkante haak op de eerste kolom van de regel staat.

- Een regel kan feitelijke informatie over een onderwerp bevatten. Deze regels beginnen niet met een # of [.

Naast deze invoer kan het voorkomen dat de lexical analyzer geen regel meer kan lezen; dit gebeurt op de end-of-file positie van het databestand.

In overeenstemming met de gangbare implementatie zullen we hier een lexical analyzer als een functie *lexer()* implementeren. Deze functie zal de semantische informatie die in het databestand wordt gelezen als return-waarde teruggeven. Daarnaast zal de semantische informatie in een globale buffer *lexbuf* opgeslagen worden. De functie *lexer()* zal commentaarregels overslaan; dat wil zeggen dat andere programmadelen met de aanwezigheid van commentaar in het databestand niet worden belast. De functie *lexer()* zal de gelezen informatie zo goed mogelijk voorverwerken, om overige programmadelen van deze taak te ontlasten.

Voor de return-waarde van *lexer()* is in bestand *help.h* (zie listing 62) het type *LEXER_TOKEN_* gedefinieerd. Dit type geeft twee mogelijke return-waarden van functie *lexer()* weer: *subject* als de gelezen regel een onderwerp weergeeft, en *textline* als de gelezen regel feitelijke informatie over een onderwerp bevat. Als *lexer()* aan moet duiden dat geen regel meer gelezen kon worden, dan wordt *EOF* geretourneerd. Deze waarde is in de header file *stdio.h* gedefinieerd.

We laten voorlopig de functie *make_index()* rusten, en concentreren ons op de constructie van de functie *lexer()*. Een herformulering van de taken van de lexical analyzer is de volgende:

1. Lees een regel tekst van het databestand.
2. Als dit faalt, dan kan van het databestand niet meer gelezen worden. Retourneer *EOF*.
3. Als het eerste teken van de regel een [is, verwerk dan de onderwerpnaam en retourneer *subject*.
4. Als het eerste teken *geen* # is, dan betreft de regel gewone tekst. Retourneer *textline*.
5. Als het eerste teken *wel* # is, ga door met punt 1.

Punt 1 kan met behulp van de functie *fgets()* geïmplementeerd worden. Deze functie verwacht drie argumenten: de buffer die gevuld moet worden, het aantal tekens dat maximaal gelezen mag worden, en de stream waar tekens uit gelezen moeten worden. Het aantal te lezen tekens is terug te vinden in de omvang van *lexbuf*, zie *data.c* in listing 63. De functie *fgets()* retourneert waarde *NULL* als uit de stream niet gelezen kan worden. Door inspectie van de return-waarde kan dus punt 2 geïmplementeerd worden.

Punt 3 begint met de inspectie van het eerste teken van de gelezen regel (dit teken is *lexbuf[0]*). De verwerking van de gelezen regel zou er uit kunnen bestaan dat de '[' en ']' tekens uit de gelezen regel verwijderd

worden. Een regel als *[dir]* zou geconverteerd worden tot *dir*, hetgeen de verwerking door andere programmadelen vereenvoudigt. Een functie die deze conversie verricht is in listing 65 gegeven.

De punten 4 en 5 pleiten voor een lus-constructie voor het lezen van invoer. Een mogelijke implementatie is een eindeloze lus (*while (1)*), waarbij afhankelijk van de gelezen informatie wel of niet geretourneerd wordt.

```
1     #include <stdio.h>
2     #include <string.h>
3     #include "help.h"
4
5     static void process_subject ()
6     {
7         register char
8             *cp;
9
10        strcpy (lexbuf, lexbuf + 1);    // haal 1e '[' weg
11
12        if (! (cp = strchr (lexbuf, ']')) )     // zoek laatste ']'
13            error ("Line %s: subject field not closed", lexbuf);
14
15        *cp = '\0';                             // haal laatste ']' weg
16    }
17
18    LEXER_TOKEN_ lexer ()
19    {
20        // code van lexer() komt hier
21    }
```

Listing 65: De functie *lexer()* (bestand *lexer.c*), voorlopige versie.

Opgave 68.

Implementeer de functie *lexer()* aan de hand van bovenstaande specificaties. Maak gebruik van de *static* functie *process_subject()*, die in listing 65 is weergegeven.

Met behulp van de functie *lexer()* kan de functie *build_index()* geïmplementeerd worden. De eerste taak van *build_index()* bestaat uit herhaaldelijke aanroepen van *lexer()*. Hierbij roept *build_index()* de functie *assert()* aan als *lexer()* signaleert dat een onderwerpspecificatie (return-waarde *subject*) gelezen is. De activatie van *lexer()* gaat door totdat *lexer()* signaleert dat het databestand verwerkt is. Dit gebeurt door return-waarde *EOF*. Daarna zorgt *build_index()* ervoor dat de indexinformatie, opgeslagen in de tabel *symtab*, gesorteerd wordt. De functie *build_index()* is in listing 66 weergegeven.

```
1     #include <stdio.h>
2     #include <search.h>
3     #include "help.h"
4
5     void build_index ()
6     {
7         register LEXER_TOKEN_
8             token;
9
```

```
10        puts ("Building index");
11        while (1)
12        {
13            if ( (token = lexer ()) == EOF )
14                break;
15            if (token == subject)
16                assert (lexbuf, ftell (fdata));
17        }
18        qsort (symtab, nsym, sizeof (SYMTAB_), symcomp);
19    }
```

Listing 66: De functie *build_index()* (bestand *build.c*).

Voor het sorteren van tabel *symtab* maakt *build_index()* gebruik van de standaardfunctie *qsort()*. Deze functie verwacht vier argumenten:

- Het eerste argument is een pointer naar de te sorteren tabel. In het help-programma is dit de waarde die pointer *symtab* bevat, omdat *symtab* naar een gealloceerd gebied wijst waarin indexinformatie is opgeslagen.
- Het tweede argument is het aantal elementen in de te sorteren tabel. In dit geval is dit de waarde van variabele *nsym*.
- Het derde argument is de omvang van elk element in de tabel. Deze omvang is in dit programma *sizeof(SYMTAB_)*.
- Het vierde argument is een pointer naar een functie. *qsort()* heeft deze functie nodig om elementen in de tabel met elkaar te kunnen vergelijken; *qsort()* kan immers niet weten wat het type van de elementen is, en op grond waarvan de tabel gesorteerd moet worden. In dit programma is dit argument het adres van de functie *symcomp()*.

De functie *qsort()* roept tijdens het sorteren *symcomp()* herhaaldelijk aan, waarbij tijdens elke aanroep *symcomp()* van twee argumenten wordt voorzien. Deze argumenten zijn pointers naar elementen in de tabel.

Aangezien tabel *symtab* uit *SYMTAB_* structuren bestaat, is het type van de argumenten een pointer naar *SYMTAB_*. *qsort()* verwacht dat *symcomp()* een *int* waarde retourneert, die specificeert of het element dat met het eerste argument wordt aangeduid groter, kleiner, dan wel even groot is als het element dat met het tweede argument wordt aangeduid. Het prototype van *symcomp()* is daarom (zie ook *help.h* in listing 62):

*int symcomp (SYMTAB_ *, SYMTAB_ *) ;*

Opgave 69.

Implementeer de functie *symcomp()*. De return-waarde kan bepaald worden door de functie *strcmp()*, zodat twee elementen aan de hand van de alfabetische volgorde van het *name*-veld vergeleken worden.

Het overzicht van onderwerpen

Als bij de aanroep van *help.exe* de –o vlag meegegeven is, dan dient een overzicht van onderwerpen afgedrukt te worden. Dit wordt verzorgd door de functie *showsub()* (zie listing 58). In deze functie wordt tabel *symtab* in een *for* statement bewerkt, waarbij veld *name* van ieder element afgedrukt wordt. Als verfijning van de uitvoer wordt na elke vijf onderwerpen een newline-teken geschreven. De functie *showsub()* is in listing 67 weergegeven.

```
1     #include <stdio.h>
2     #include <string.h>
3     #include "help.h"
4
5     void showsub ()
6     {
7         register int
8             i;
9
10        puts ("Subjects in HELP database:");
11        for (i = 0; i < nsym; i++)
12        {
13            printf ("%15s", symtab [i].name);
14            if (! ((i + 1) % 5) )
15                putchar ('\n');
16        }
17        puts ("\n\n");
18    }
```

Listing 67: De functie *showsub()* (bestand *showsub.c*).

11.6.5 Het tonen van help-informatie

Het tonen van feitelijke informatie wordt door de functie *showhelp()* verzorgd. Deze functie wordt aangeroepen met één argument, een *char* ∗ die de onderwerpnaam specificeert. De taken van *showhelp()* zijn de volgende:

1. Zoek het onderwerp op in tabel *symtab*.
2. Zet de stream-positie van *fdata*, de stream die met het databestand geassocieerd is, op de plaats waar de informatie begint. De stream-positie is opgeslagen in veld *filepos* van het *symtab*-element dat bij punt 1 bepaald is.
3. Toon de informatie op het scherm.

Punt 1 laten we over aan een aparte functie, die we *lookup()* noemen. Punt 2 kan met behulp van de standaardfunctie *fseek()* geïmplementeerd worden. Bij punt 3 kan de functie *lexer()* gebruikt worden om informatie uit het databestand te lezen, analoog aan het gebruik van deze functie bij *build_index()* (listing 66). De brontekst van de functie *showhelp()* is in listing 68 afgebeeld.

```
1     #include <stdio.h>
2     #include "help.h"
3
```

```
4     void showhelp (char *name)
5     {
6         register unsigned
7             index;
8
9         if ( (index = lookup (name)) == -1 )
10        {
11            printf ("Subject '%s' not found.\n", name);
12            return;
13        }
14
15        if (fseek (fdata, symtab [index].filepos, SEEK_SET))
16            error ("Data file corrupt");
17
18        while (lexer () == textline)
19            printf (lexbuf);
20        puts ("\n");
21    }
```

Listing 68: De functie *showhelp()* (bestand *showhelp.c*).

De functie *lookup()*, die ten dienste staat van *showhelp()*, moet aan de hand van een naam een element in *symtab* opzoeken. Als het onderwerp in de tabel bekend is, wordt het indexnummer van dit element geretourneerd. Als het onderwerp onbekend is, wordt –1 geretourneerd.

Een eenvoudige implementatie van *lookup()* zou met een *for* statement gerealiseerd kunnen worden. Hierbij zou elk *name*-veld van de elementen van *symtab* met de op te zoeken naam vergeleken worden. Deze implementatie is in zoverre onelegant dat het gesorteerd zijn van *symtab* niet gebruikt wordt; terwijl in een gesorteerde tabel veel sneller gezocht kan worden dan in een ongesorteerde tabel. We kiezen daarom voor een andere aanpak, waarbij *symtab* aan een binair zoekproces onderworpen wordt. Binair zoeken houdt in dat het te doorzoeken gebied in twee stukken wordt verdeeld, waarbij gekeken wordt in welk deel het op te zoeken element moet zijn. Dit deel wordt weer in tweeën verdeeld. Het proces wordt herhaald totdat het op te zoeken element gevonden is, of totdat bekend is dat het element niet in de tabel voorkomt.

Een voorbeeld van binair zoeken is het *hoger-lager* spelletje dat wel eens met kinderen wordt gespeeld. Je vraagt een kind om een getal tussen 1 en 100 te raden. Een kind dat de **C** documentatie bestudeerd heeft en dat het binaire zoekproces kent, zal '50 soms?' antwoorden. Je zegt dan bij voorbeeld: 'nee, lager'. Het kind verdeelt de groep weer in tweeën en antwoordt: '25 soms?' Binair zoeken is de snelste manier om in een gesorteerde tabel een element op te zoeken.

De standaardfunctie *bsearch()* verwacht vijf argumenten. Deze argumenten zijn hieronder beschreven:

- Het eerste argument is een pointer naar een voorbeeld-element waarnaar gezocht wordt. Vóór de aanroep van *bsearch()* moet dus een datastructuur gemaakt worden, die *bsearch()* zal gebruiken om elementen in het te doorzoeken array te vergelijken met hetgene dat gevonden moet worden.

- Het tweede argument is een pointer naar het begin van de tabel. In dit geval is dat pointer *symtab*.
- Het derde argument specificeert hoeveel elementen in de tabel aanwezig zijn. In dit programma is dit de waarde van variabele *nsym*.
- Het vierde argument is de omvang van elk element in de tabel, oftewel *sizeof(SYMTAB_)*.
- Het vijfde argument is een pointer naar een functie die door de functie *bsearch()* aangeroepen wordt om het voorbeeld-element met een element in de tabel te vergelijken. Deze functie wordt van twee argumenten voorzien, die beide een pointer naar een element zijn. De functie retourneert een *int* waarde, die aangeeft welk argument groter is.
  Als vergelijkende functie kan uiteraard *symcomp()* gebruikt worden.

De return-waarde van *bsearch()* is, als het voorbeeld-element in de tabel gevonden is, een pointer naar een element in *symtab*. Als het voorbeeld-element *niet* in *symtab* voorkomt, retourneert *bsearch()* de waarde *NULL*.

Terwille van *showhelp()* moet de return-waarde van *bsearch()* naar een indexwaarde vertaald worden. Bij return-waarde *NULL* moet *lookup()* –1 retourneren. Bij een andere return-waarde moet de door *bsearch()* opgeleverde pointer naar een index in *symtab* vertaald worden. Dankzij het feit dat in **C** met pointers gerekend kan worden, kan dit gebeuren door *symtab* van de de return-waarde van *bsearch()* af te trekken.

Opgave 70.
Implementeer de functie *lookup()* aan de hand van bovenstaande specificaties.

Opgave 71.
Voer alle bronteksten in die bij de help faciliteit horen. Maak voor dit doel een aparte directory aan, bijvoorbeeld \c*sourceshelp*. Vertaal alle bronteksten. Zet vervolgens alle objectmodules behalve *help.obj* in één bibliotheek *help.lib*. Gebruik hiervoor het *movelib* programma. Maak vervolgens *help.exe* met behulp van de linker.**

DEEL II

GEVORDERD GEBRUIK VAN C

Hoofdstuk 12

Geheugenmodellen

12.1 Inleiding

In dit hoofdstuk komen de geheugenmodellen *(memory models)* die door Microsoft zijn gedefinieerd voor PC-achtige computers ter sprake. De geheugenmodellen zijn ontwikkeld om de 64 Kbyte grens op een manier te doorbreken die het best bij de aard van het te ontwikkelen programma past. De 64 Kbyte grens heeft betrekking op de manier waarop de Intel 80x86 chips geheugen adresseren. Afhankelijk van de aard van het te ontwikkelen programma kan een geheugenmodel worden gekozen waarbij relatief veel of relatief weinig geheugen beschikbaar is voor de code en/of de data.

Het werkgeheugen van computers die op Intel 80x86 chips zijn gebaseerd is opgedeeld in *segmenten,* die ten hoogste 64 Kbytes groot[1] zijn. Om deze reden moet de vertaler worden geïnformeerd over de maximale omvang van de data en de code. Dat lijkt ingewikkeld, maar omdat alleen hoeft te worden aangegeven of de hoeveelheid data of code meer of minder zal zijn dan 64 Kbytes (64Kb, één segment) is het doorgaans niet lastig om deze informatie aan de vertaler door te geven.

Een intuïtief aantrekkelijk alternatief is om altijd zonder de 64Kb beperkingen te werken. Toch is dat alternatief niet zo praktisch. De reden hiervoor is dat de adresberekeningen die nodig zijn wanneer meer dan 64Kb geheugen wordt gebruikt meer ruimte en meer tijd vergen dan wanneer het bekend is dat de 64Kb grens niet hoeft te worden gepasseerd[2]. Wanneer bijvoorbeeld de data in één segment passen, dan kunnen de data met één *unsigned int* worden geadresseerd. Wanneer de data echter over meer dan één segment zijn verspreid, dan moet behalve de plaats van het data-element *in* het segment ook nog het segment *zelf* in een *unsigned int* beschikbaar worden gemaakt voordat het volledige adres bekend is.

De nomenclatuur die wordt gebruikt om de plaats van code en data aan te geven is bijzonder suggestief. Microsoft gebruikt extra keywords[3] *_near:* om aan te geven dat binnen één segment kan worden geadresseerd, terwijl *_far* wordt gebruikt wanneer bekend is dat de 64Kb grens moet kunnen worden gepasseerd. Objecten die met het keyword *_near* zijn gedefinieerd zijn dus echt *dichtbij,* terwijl objecten die met het keyword *_far*

1. Segmenten zijn in de praktijk kleiner dan de maximale omvang, omdat zelden of nooit *precies* 64 Kbyte wordt gebruikt.
2. Merk op dat bij adresberekeningen de time-space tradeoff niet opgaat: wanneer meer dan 64Kb moet worden geadresseerd kosten de adresberekeningen *zowel* extra tijd *als* extra ruimte.
3. Dat Microsoft extra keywords zoals *_near* en *_far* heeft gedefinieerd, heeft consequenties voor de portabiliteit van programma's. Bronteksten waarin dergelijke keywords worden gebruikt, kunnen niet zonder meer door andere C vertalers worden vertaald. Echter, door (bijvoorbeeld in *stdio.h)* 'lege' *#define* opdrachten op te nemen, zoals:

#define _near

worden dergelijke keywords door de vertaler niet meer geïnterpreteerd en vervalt de noodzaak de bronteksten op deze aspecten te wijzigen.

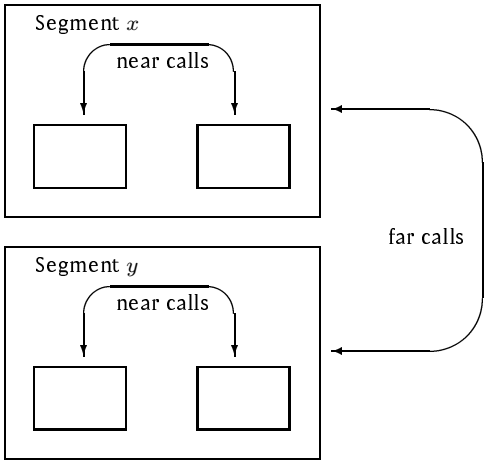
zijn gedefinieerd zich ergens *ver weg* in het geheugen kunnen bevinden.

12.2 Segment Registers

In **C** kunnen *auto* variabelen bij hun definitie de storage class *register* krijgen. De vertaler probeert dergelijke variabelen niet op de stack te plaatsen, maar toe te wijzen aan een intern register van de 80x86 processor. Naast deze registers heeft de processor onder andere ook nog minstens vier registers die speciaal bedoeld zijn voor het aanduiden van segmenten. Drie van deze registers hebben nog weer een extra specialiteit, omdat ze alleen worden gebruikt voor de aanduiding van het segment waarin zich de code van functies, de data en de stack bevinden. Deze drie registers kunnen in de assembleertaal van de 80x86 processors impliciet worden gebruikt om het segment waarin functies, data en de stack zich bevinden te adresseren. Dergelijke adresseringen kunnen dan met korte en snelle code worden uitgevoerd, hetgeen de basis vormt voor het gebruik in Microsoft **C** van default code- en datasegmenten. Als vuistregel geldt dat de registers die worden gebruikt voor het aangeven van het datasegment en het stacksegment in **C** programma's altijd dezelfde waarde hebben. Het datasegment en het stacksegment zijn in **C** programma's dan ook altijd aan elkaar gelijk, tenzij bij de vertaling uitdrukkelijk wordt aangegeven dat daarvan moet worden afgeweken. Wij zullen dat in **C** programma's overigens nergens doen.

12.2.1 Code

De code die uitgevoerd wordt, bevindt zich altijd in een *code segment*, afgekort cs. In programma's met minder dan 64Kb code past alle code binnen één segment. We spreken dan van near code. In zo'n programma verandert het codesegment dus niet van waarde en functies roepen elkaar aan via een procedure die weinig geheugen vergt en snel is (zogenaamde near calls). De code van programma's kan echter ook over meerdere segmenten zijn verdeeld. Het codesegment verandert dan van waarde wanneer een functie wordt aangeroepen die in een ander segment ligt dan de aanroepende functie. De aangeroepen functie bevindt zich dan als het ware *ver weg* ten opzichte van de aanroepende functie en we spreken van far code. Bij far code wordt het codesegment bij de aanroep van een functie expliciet bepaald en functies roepen elkaar aan via een procedure die, vergeleken met near calls meer geheugen kost en trager is (zogenaamde far calls). Bij near code is het niet nodig bij de aanroep van een functie het codesegment expliciet te bepalen, omdat het codesegment bij near code immers niet verandert. Per definitie is het huidige codesegment het default codesegment. Wanneer een functie met een far call wordt aangeroepen, verandert het default codesegment van het segment van de aanroepende functie naar het segment van de aangeroepen functie. Functies die



Figuur 12.1: Near en far calls bij functies.

in hetzelfde segment liggen als de aangeroepen functie, kunnen elkaar dus weer aanroepen via near calls. In figuur 12.1 is deze situatie geïllustreerd. Sommige geheugenmodellen zijn bedoeld voor programma's waarin alleen near code voorkomt. Dit zijn de zogeheten small en compact geheugenmodellen. Bij de overige geheugenmodellen kan ook far code voorkomen. Dit zijn de zogeheten medium, large en huge geheugenmodellen.

12.2.2 Data

Net zoals de code van programma's in segmenten zijn geplaatst, zijn ook de data in segmenten geplaatst. Bij elk programma is er echter maar één default datasegment (afgekort tot DS). Het is echter ook mogelijk data buiten het default datasegment te definiëren. Dergelijke data bevinden zich buiten het default datasegment, en heten far data. Merk op dat in tegenstelling tot het default codesegment het default datasegment dus niet van waarde verandert. Data die in het default datasegment zijn gedefinieerd heten near data. Om data expliciet in het default datasegment te definiëren, wordt gebruik gemaakt van het keyword *_near*. Variabelen die *_near* worden gedefinieerd worden ondergebracht in het default datasegment en kunnen met minder code en sneller worden geadresseerd dan variabelen die buiten het default datasegment zijn geplaatst. Bij de ontwikkeling van programma's kan de ontwikkelaar door gebruik te maken van het keyword *_near* ervoor zorgen dat globale of statische variabelen die intensief worden gebruikt in het default datasegment worden gelocaliseerd.

Dergelijke variabelen worden dan sneller bereikt dan wanneer de variabelen niet in het default datasegment zouden zijn geplaatst.

Sommige geheugenmodellen zijn bedoeld voor programma's met alleen maar near data. Dit zijn de small en medium geheugenmodellen. Bij de overige geheugenmodellen kunnen ook far data voorkomen. Dit zijn de zogeheten compact, large en huge geheugenmodellen. Wanneer deze geheugenmodellen worden gebruikt zijn *geïnitialiseerde* statische variabelen en globale variabelen die minder dan 32.767 bytes geheugen in beslag nemen in het default datasegment geplaatst[4]. De waarde 32.767 is de data drempel (data threshold). Variabelen waarvan de omvang in bytes de data drempel bereikt of passeert, worden in een nieuw datasegment geplaatst. Dergelijke data worden dus automatisch far. De vertaler heeft sterk de neiging om variabelen near te maken. De drempel kan echter worden gewijzigd door middel van de /**Gt** vertaleroptie. Wanneer alleen de optie /**Gt** wordt gegeven, zakt de drempel drastisch tot de waarde 256. Globale variabelen die 256 of meer bytes geheugen in beslag nemen, worden dan al in een eigen datasegment geplaatst. Wanneer onmiddellijk achter de /**Gt** optie een getal volgt dan wordt dat de nieuwe data drempel. Zo zet de compileroptie /**Gt1000** de data drempel op 1000 bytes.

12.2.3 Stack

De stack bevindt zich in het *stacksegment*, afgekort ss. Dit datagebied wordt gebruikt om tijdelijke informatie op te slaan zoals lokale *(auto)* variabelen van functies.

In **C** is de stack een onderdeel van het default datasegment tenzij dat uitdrukkelijk niet de bedoeling is. In alle programma's die in dit boek worden beschreven, vallen het stack en default datasegment echter samen en sommige standaardfuncties kunnen alleen worden gebruikt, wanneer de veronderstelling kan worden gemaakt dat deze twee segmenten samen vallen.

De omvang van de stack is bij programma's die door de Microsoft **C** vertaler zijn vertaald standaard 2 Kbytes. De grootte van de stack kan eenvoudig met behulp van de /**st**: linker optie worden gewijzigd. Achter de /**st**: optie volgt de gewenste omvang van de stack in bytes. De maximale stack omvang is 64 Kbytes. Om in een programma *test* een stack van 4000 bytes te kunnen gebruiken kan, wanneer gebruik wordt gemaakt van de batchfile *l.bat* uit hoofdstuk 1, worden gelinkt met de opdracht:

l test /st:4000

Het vergroten van de stack is echter zelden nodig. Bij sterk recursieve programma's (zie ook hoofdstuk 15) of bij programma's waarin functies wor-

4. Merk op dat de totale omvang van het default datasegment maximaal 64 Kbytes mag zijn. Drie arrays van 25.000 bytes kunnen dus niet alle in het default datasegment worden ondergebracht.

den gebruikt die extreem veel lokale variabelen gebruiken, kan de noodzaak van een vergrote stack zich voordoen. Het verkleinen van de stack is ook zelden nodig. Bij programma's waarin een grote hoeveelheid globale data nodig is, terwijl de stack-belasting gering blijft kan het verkleinen van de stack ertoe leiden dat het small of medium geheugenmodel nog kan worden gehanteerd.

12.2.4 Nomenclatuur bij adresseringen

De plaats van een variabele of een functie in een segment wordt aangeduid als de offset van het object in het desbetreffende segment. Het eerste byte in een segment heeft offsetwaarde 0. De waarde van een offset is bij PC-achtige computers een *unsigned int*. Zo kan een *int i* op offset 0x12a7 ten opzichte van het default datasegment staan. In dat geval betreft het dus een near variabele. Wanneer objecten over verschillende segmenten verspreid kunnen zijn, moet behalve de offset ook het segment zelf bekend zijn. De waarde van het segment is eveneens een *unsigned int*. Een volledig (far) adres bestaat dus uit 32 bits, verdeeld over 2 *unsigned ints*. Het adres is echter geen *long* waarde. De Intel 80x86 chips adresseren het geheugen als een combinatie van segment en offsetwaarde. De segmentwaarde maal 16 plus de offsetwaarde geeft dan het absolute adres, waarbij het geheugen wordt opgevat als een lineaire aaneenschakeling van bytes. Dus:

$$lineair\ adres = segment * 16 + offset.$$

Merk op dat in het algemeen lineaire adressen op meer dan één manier uit segment- en offsetwaarden kunnen worden samengesteld. Zolang als *nieuw_segment* en *nieuw_offset* niet negatief of groter dan *0xffff* worden geldt, bij elke gehele waarde *x,* dat:

$$nieuw_segment = segment + x;$$

$$nieuw_offset = offset - 16x;$$

Het adres van een variabele of functie kan worden afgedrukt met behulp van de *%p* formaat specificatie bij *printf()*. Bij de *%p* formaat specificatie wordt een adres verwacht[5]. Als het object, waarvan het adres moet worden afgedrukt, gedefinieerd is als een near object, dan moet een type casting naar een far adres worden meegegeven wanneer het volledige adres[6] moet worden afgedrukt, zoals geïllustreerd in het volgende voorbeeld:

```
#include <stdio.h>
```

5. Door Microsoft zijn de specificaties *%Fp* en *%Np* gedefinieerd waarmee expliciet een far, respectievelijk near adres (alleen de offset) wordt verwacht en afgedrukt. In small en medium geheugenmodellen is *%p* equivalent aan *%Np,* in de overige modellen is *%p* equivalent aan *%Fp*.
6. In het volledige adres wordt zowel het segment als de offset afgedrukt. Beide onderdelen van het adres worden gescheiden door een dubbele punt (in de vorm: *segment:offset)*.

```
int
    near near_int;

void main()
{
    printf("Het adres van near_int: %Fp\n", (int _far *)&near_int);
    printf("De offset van near_int: %Np\n", &near_int);
}
```

Merk op dat de type casting ook nodig is bij geheugenmodellen waarbij per definitie sprake is van near objecten, zoals het small geheugenmodel.

Het adres van een object wordt door de functie *printf()* weergegeven als *ssss:oooo,* waarbij *ssss* de hexadecimale waarde van het segment is waarin het object zich bevindt en *oooo* de hexadecimale waarde van de offset is.

12.3 Modellen

De Microsoft **C** vertaler voor de Intel 80x86 processor familie kan op een aantal verschillende manieren code genereren.

Elke manier wordt aangeduid als een bepaald geheugenmodel (memory model). De modellen verschillen in het aantal toegestane code- en datasegmenten. Het aantal code- en datasegmenten beïnvloedt de manier waarop addressen worden bepaald en functies worden aangeroepen. Het gewenste geheugenmodel wordt aan de vertaler bekend gemaakt door middel van de vertaleroptie /**A...** De optie wordt gebruikt om het gewenste standaard geheugenmodel te kiezen. De vertaler genereert vervolgens code overeenkomstig de eisen van het opgegeven geheugenmodel. De standaardfuncties zijn al door Microsoft vertaald en opgenomen in functiebibliotheken die aangepast zijn aan het gewenste model. Wanneer een programma vertaald wordt voor een bepaald geheugenmodel wordt de naam van de vereiste functiebibliotheek opgenomen in de objectmodule. Bij het linken worden dan automatisch de vereiste functies uit de passende bibliotheek aan het programma gekoppeld. De eerste letter van de naam van deze bibliotheken (s, M, C, L, H) geeft aan bij welk model de bibliotheek hoort. De modellen worden small, medium, compact, large en huge genoemd. Het laatste model is eigenlijk geen apart model maar een extensie op het large geheugenmodel.

Wanneer het erom gaat een programma te ontwikkelen, moet de ontwikkelaar een beslissing nemen over het te gebruiken geheugenmodel. De software-ontwikkeling kan echter ook betrekking hebben op functies die aan andere programma's moeten kunnen worden aangesloten. In dat laatste geval wordt in feite een uitbreiding van de standaardfuncties beoogd. Het kan dus voorkomen dat dergelijke functies later moeten worden aangesloten aan programma's die ontwikkeld worden in één van de vijf

geheugenmodellen. De ontwikkeling van dergelijke functies moet dan ook leiden tot in ieder geval vier soorten functiebibliotheken: voor het small, medium, compact en large geheugenmodel[7].

Wanneer gebruik wordt gemaakt van de batch files die in hoofdstuk 1 zijn geïntroduceerd om **C** bronteksten mee te vertalen, dan blijkt de **C** vertaler te worden aangeroepen met onder andere de optie:

/A%MM%

Hierbij wordt het te gebruiken geheugenmodel afgeleid van de waarde van de systeemvariabele *MM,* die de waarde *S, M, C, L* of *H* kan hebben, afhankelijk van het gewenste geheugenmodel. In hoofdstuk 1 is aangeraden om in *autoexec.bat* de DOS opdracht:

set mm=S

te geven. Nu zal duidelijk zijn dat de vertaler de bronteksten tot nu toe voor het small geheugenmodel heeft vertaald.

12.3.1 Small geheugenmodel: optie: /As

Het small geheugenmodel is bedoeld voor programma's met slechts één codesegment en één datasegment. Dergelijke programma's bevatten niet meer dan 64 Kbytes code en niet meer dan 64 Kbytes data. Het totale programma kan dus hoogstens 128 Kbytes groot zijn.

Omdat tijdens de verwerking van het small geheugenmodel programma de code- en datasegmenten niet veranderen, zijn slechts 16-bits adresberekeningen en near calls van functies nodig. Door deze near adressering is de code kort en zijn de adresberekeningen en functie-activaties snel. Zeer veel programma's kunnen heel goed in het small geheugenmodel worden ontwikkeld. Alle programma's in deel I van dit boek zijn in eerste instantie ontwikkeld in het small geheugenmodel. Wanneer de **C** vertaler wordt gestart met behulp van de batch bestanden die in hoofdstuk 1 zijn genoemd, dan moet de DOS opdracht:

set mm=S

worden gegeven wanneer de vertaling voor het small geheugenmodel moet worden uitgevoerd.

12.3.2 Medium geheugenmodel: optie: /Am

In het medium geheugenmodel kunnen programma's worden ontwikkeld die uit meerdere codesegmenten, maar slechts uit één datasegment bestaan. De code bij iedere brontekst die wordt vertaald, komt in een eigen codesegment terecht. Omdat de adressen van functies in dit model uit een expliciet segment- en offset gedeelte zijn opgebouwd, wordt de aanroep van een functie iets omvangrijker en trager dan bijvoorbeeld bij het small

7. Een bibliotheek die specifiek voor het huge model is ontwikkeld, is in de praktijk zelden nodig, omdat het huge model een verbijzondering is van het large model.

geheugenmodel. Functies worden in dit model dus met een far call aangeroepen, en de adressen van de functies zijn far. De data bevinden zich echter alle, net zoals bij het small geheugenmodel, in het default datasegment. De data kunnen dus met relatief korte en snelle opdrachten worden gemanipuleerd.

Wanneer de **C** vertaler wordt gestart met behulp van de batch bestanden die in hoofdstuk 1 zijn genoemd, dan moet de DOS opdracht:

set mm=M

worden gegeven om de vertaling voor het medium geheugenmodel uit te laten voeren.

12.3.3 Compact geheugenmodel: optie: /Ac

In het compact geheugenmodel worden programma's ontwikkeld die uit meerdere datasegmenten, maar slechts uit één codesegment bestaan. Dit model is bedoeld voor programma's met veel data en relatief weinig code. Alle code adressen zijn near. De data worden echter via far adressen bereikt. In dit model kan er meer dan 64 Kbyte aan data beschikbaar zijn, maar individuele variabelen mogen niet groter zijn dan 64 Kbytes. Een *int* array van 50000 elementen kan dus niet in het compact geheugenmodel worden gedefinieerd of gealloceerd. De bij het *icce* ontwikkelde editor *oe!* (Brokken, 1990) is een voorbeeld van een programma waarvoor het compact model zeer geschikt is: relatief weinig code, maar het is mogelijk om grote hoeveelheden tekst in het geheugen te bewerken.

Wanneer de **C** vertaler wordt gestart met behulp van de batch bestanden die in hoofdstuk 1 zijn genoemd, dan moet de DOS opdracht:

set mm=C

worden gegeven wanneer de vertaling voor het compact geheugenmodel moet worden uitgevoerd.

12.3.4 Large geheugenmodel: optie: /Al

Wanneer het large geheugenmodel wordt gebruikt, kunnen meerdere code- en datasegmenten in een programma worden gebruikt. Zowel de code- als de data adressen zijn dan far. Variabelen kunnen in dit model, zoals bij het compact model, ten hoogste 64 Kbyte ruimte innemen.

Wanneer de **C** vertaler wordt gestart met behulp van de batch bestanden die in hoofdstuk 1 zijn genoemd, dan moet de DOS opdracht:

set mm=L

worden gegeven wanneer de vertaling voor het large geheugenmodel moet worden uitgevoerd.

12.3.5 Huge geheugenmodel: optie: /Ah

Het huge geheugenmodel is in feite een uitbreiding op het large geheugenmodel. Bij het huge geheugenmodel kunnen arrays groter zijn dan 64 Kbytes. Aan deze grote arrays worden de volgende speciale eisen gesteld:

1. De array-elementen zelf mogen niet groter zijn dan 64 Kbytes.
2. Indien het array groter is dan 128 Kbytes, moeten de elementen een omvang hebben die gelijk is aan een macht van 2.

Dit model kan worden gebruikt voor programma's met zeer grote dataitems. De verwerkingstijd van programma's met dit model is door de complexe adresberekening die nodig is bij huge arrays, uiteraard langer dan die bij de andere modellen optreedt.

Wanneer de C vertaler wordt gestart met behulp van de batch bestanden die in hoofdstuk 1 zijn genoemd, dan moet de DOS opdracht:

set mm=H

worden gegeven wanneer de vertaling voor het huge geheugenmodel moet worden uitgevoerd.

12.4 Gemengde modellen

De standaard geheugenmodellen kunnen worden gebruikt om adresseringsschema's te kiezen die in het algemeen goed zijn aangepast aan de aard van programma's. Echter, er kunnen zich situaties voordoen waarin de standaard geheugenmodellen niet zo geschikt lijken te zijn. Zo'n situatie doet zich bijvoorbeeld voor wanneer een klein programma twee arrays van elk 40 Kbytes moet kunnen manipuleren. In dat geval zou het compact geheugenmodel noodzakelijk worden, maar de overige variabelen zouden dan op dezelfde manier als de arrays worden geaddresseerd. Terwijl de addressering volgens de methode die bij het compact geheugenmodel nodig is de aangewezen methode voor de twee arrays is, is de methode onnodig complex en traag voor de overige variabelen. Het omgekeerde kan zich ook voordoen: programma's die veel data gebruiken, maken ook vaak gebruik van tellers en pointers die beter in het default datasegment dan in een eigen segment kunnen worden geplaatst. Vergelijkbare situaties als deze kunnen zich ook voordoen bij de code van functies. Het zou prettig zijn wanneer in een groot programma functioneel onafhankelijke delen zouden kunnen worden ontwikkeld alsof het een klein programma betrof, waardoor binnen een functionele eenheid de aanroep van functies relatief snel en met weinig code kan worden uitgevoerd.

Programma's die zodanig worden ontwikkeld dat ze op onderdelen afwijken van het gekozen geheugenmodel en zich bij die onderdelen gedragen alsof ze met een ander geheugenmodel zijn ontwikkeld heten 'gemengd model' programma's. Als vuistregel geldt dat gemengd model pro-

gramma's worden ontwikkeld in een uitgebreid geheugenmodel. Onderdelen van dergelijke programma's maken dan gebruik van de kenmerken van minder uitgebreide geheugenmodellen.

De door Microsoft gedefinieerde keywords *_near, _far* en *_huge* kunnen worden gebruikt om binnen een gekozen geheugenmodel aan te geven dat een object (functie of variabele) moet worden behandeld op een manier die past bij een ander geheugenmodel. De keywords bepalen de manier waarop data-elementen worden geadresseerd en de manier waarop functies worden aangeroepen. Deze keywords worden altijd gekoppeld aan het onderdeel van de definitie dat *onmiddellijk* volgt op het keyword. Zo kan een variabele worden gedefinieerd als een *_far* pointer naar een *char* door het keyword *_far* in de definitie vóór de pointer operator te plaatsen, of als een variabele die een pointer naar een *char* is, maar zelf niet in het default datasegment wordt gelokaliseerd door het keyword *_far* voor de naam van de variabele te plaatsen, zoals geïllustreerd in het volgende voorbeeld:

```
char _far *ptr_1;                    // pointer naar far chars
char *_far ptr_2;                    // far pointer naar chars
```

De variabele *ptr_1* is een variabele die op een willekeurige *char* in het geheugen van de computer kan wijzen (het is een far pointer), terwijl de variabele *ptr_2* een variabele is die buiten het default datasegment wordt geplaatst en op *char* waarden kan wijzen, die onder het gekozen geheugenmodel bereikbaar zijn (*ptr_2* is zelf een far variabele). De keywords *_near, _far* en *_huge* kunnen bij de definitie van variabelen worden gebruikt, maar ook bij type casts. Omdat ze worden toegepast wanneer andere adresseermethodes moeten worden gebruikt dan die welke automatisch door de vertaler worden gekozen, spreken we van 'segment overriding'.

12.4.1 Data

In geheugenmodellen waar *far* data bestaan, kan het keyword *_near* worden gebruikt om adresseringen ten opzichte van het default datasegment te forceren. Andersom kunnen de keywords *_far* en *_huge* worden gebruikt om de 64 Kbyte grens voor data te passeren in modellen waarbij de data near zijn[8].

Door verstandig gebruik te maken van de keywords *_near* en *_far* kan de ontwikkelaar veel data adresseren bij de small en medium modellen, of snelle adressering realiseren in de compact en large modellen. In het volgende voorbeeld is de variabele *val_1* een variabele die altijd in het default datasegment zal worden geplaatst, dus ook in de compact en large

8. In de praktijk kunnen *_far* en *_huge* elkaar bij data vervangen. Het keyword *_huge* zal daarom alleen nog expliciet worden vermeld, wanneer een afwijking ten opzichte van het keyword *_far* optreedt.

geheugenmodellen. De variabele *val_1* kan daardoor snel en met korte instructies worden gemanipuleerd. De variabele *val_2* daarentegen is een variabele die ook in programma's die met het small en medium geheugenmodel zijn vertaald, buiten het default datasegment kan worden geplaatst. De adressering van *val_2* wordt daardoor trager en omvangrijker, maar *val_2* hoeft dus niet meer noodzakelijk in het default datasegment te zijn geplaatst:

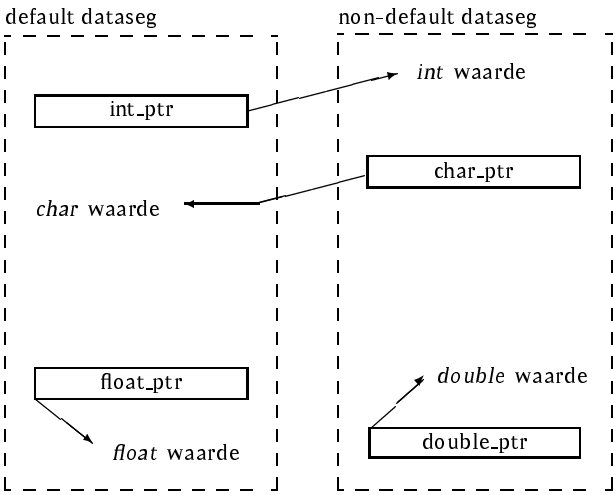
```
// LARGE, COMPACT geheugenmodel
int                                     // val_1 is altijd in het
    _near val_1;                        // default datasegment

// SMALL, MEDIUM geheugenmodel
int                                     // val_2 is niet noodzakelijk
    _far val_2;                         // in het default datasegment
```

Variabelen die expliciet als near variabelen worden gedefinieerd, komen regelmatig voor bij programma's die in het compact of large geheugenmodel worden vertaald. Het gebeurt minder vaak dat variabelen expliciet als far variabelen worden gedefinieerd in het small of medium model. Pointers vormen daarop enigszins een uitzondering, omdat in het small en medium geheugenmodel een far pointer kan worden gebruikt om het gehele geheugen van de computer te bereiken.

Bij pointers is het onderscheid tussen de pointervariabele zelf en datgene waarnaar wordt gewezen belangrijk. De pointervariabele is zelf een variabele die ergens (binnen of buiten het default datasegment) is gelokaliseerd. De pointervariabele zelf kan dus near of far zijn. Daarnaast wijst de pointervariabele naar een waarde van een bepaald type. Op zijn beurt kan die waarde ook binnen of buiten het default datasegment staan. De waarde waarnaar wordt gewezen kan dus ook een near of far waarde zijn. In figuur 12.2 is *int_ptr* een pointer in het default datasegment, die echter naar een waarde wijst in een ander segment. Variabele *char_ptr* is een pointer in een niet-default datasegment, die echter naar een *near* waarde moet wijzen. Variabele *float_ptr* staat in het default datasegment (is near) en moet op een *float* wijzen die zich ook in het default datasegment bevindt. Variabele *double_ptr* tenslotte is *far* en wijst op een *far* double waarde.

De declaraties van deze variabelen zijn hieronder gegeven. Alle mogelijke segment specificaties zijn gegeven: bij small en medium geheugenmodellen (dus bij near data) zouden de *_near* keywords weggelaten kunnen worden. Bij de andere geheugenmodellen zouden de *_far* keywords weggelaten kunnen worden:

```
int
    _far *_near int_ptr;
char
    _near *_far char_ptr;
float
```



Figuur 12.2: Near en far pointers naar near en far data.

```
    _near *_near float_ptr;
double
    _far  *_far double_ptr;
```

De opzet van *near* en *far* pointers wordt in figuur 12.3 nogmaals geïllustreerd. Merk het volgende op:

- Alle variabelen die in het default datasegment zijn gelokaliseerd, hebben het keyword *_near* vlak voor de naam van de variabele staan (zie de middelste kolom van figuur 12.3).
- Alle variabelen die buiten het default datasegment zijn gelokaliseerd, hebben het keyword *_far* vlak voor de naam van de variabele staan (zie de rechter kolom van figuur 12.3).
- Alle variabelen die op een waarde in het default datasegment wijzen, hebben het keyword *_near* voor de pointer operator staan (zie de middelste rij van figuur 12.3).
- Alle variabelen die op een waarde buiten het default datasegment wijzen, hebben het keyword *_far* voor de pointer operator staan (zie de onderste rij van figuur 12.3).

Bij alle gegeven definities is het keyword *_near* overbodig in small en medium model programma's. Het keyword *_far* is overbodig in compact en large model programma's.

Wanneer far data pointers worden gebruikt in small en medium model programma's, moet er rekening mee worden gehouden dat stan-

Pointer verwijst naar:	Plaats van de pointer: in default datasegment	Plaats van de pointer: buiten default datasegment
waarde in default datasegment	*char _near *_near var;*	*char _near *_far var;*
waarde buiten default datasegment	*char _far *_near var;*	*char _far *_far var;*

Figuur 12.3: De definitie van near en far pointers.

daardfuncties die pointers als argumenten accepteren (zoals bijvoorbeeld de functie *printf()*) er vanuit gaan dat een near pointer als argument wordt doorgegeven. Wanneer in dergelijke gevallen toch een far pointer wordt gebruikt, kan de vertaler (onder protest) een conversie naar een near pointer uitvoeren. De doorgegeven waarde wijkt dan af van de waarde van de pointervariabele zelf. Een andere mogelijkheid is dat het volledige adres (offset en segment) weliswaar wordt doorgegeven, maar dat de code van de functie een near adres (alleen offset) verwacht. In dat geval treedt een verschuiving ten opzichte van latere argumenten op, waardoor voor die argumenten een verkeerde waarde wordt gebruikt. Wanneer far pointers moeten worden doorgegeven aan functies in small en medium model programma's, moeten de functies expliciet zijn gedefinieerd met far pointer parameters. Een voorbeeld van hoe het niet en wel kan is gegeven in listing 69. In de functie *right()* wordt in regel 21 eerst *string* gekopieerd in het lokale array *buffer,* omdat de functie *printf()* wèl veronderstelt dat near pointers worden gebruikt.

```
1     #include <stdio.h>
2
3     // SMALL MEMORY MODEL
4
5     char
6         far medium[] = "The medium is the message !";
7
8     void wrong(char *string)
9     {
10        printf("function wrong():\n"
11               "    string: %s\n", string);
12    }
13
14    void right(char _far *string)
15    {
16        char
17            *cp,
18            buffer[80];
19
20        cp = buffer;
```

```
21        while (*cp++ = *string++)
22            ;
23
24        printf("function right():\n"
25                "    string: %s\n", buffer);
26    }
27
28    void main()
29    {
30        right(medium);
31        wrong(medium);
32    }
```

Listing 69: Far pointer parameters in small en compact programma's.

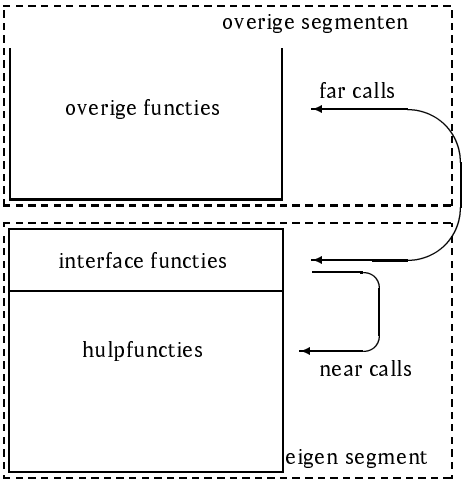
12.4.2 Code

Net zoals variabelen near en far kunnen zijn, kunnen ook functies near en far zijn. Bij het small en compact model worden functies doorgaans niet als far functies gedefinieerd, maar blijven alle functies in één default codesegment. Bij het medium en large model wordt door de vertaler echter voor elke brontekst een apart codesegment gedefinieerd. Dat is ook het geval wanneer een brontekst meer dan één functiedefinitie bevat. Alle functies in één brontekst komen dan automatisch in één codesegment. Wanneer zo'n functie wordt aangeroepen, wordt het segment waarin de functie zich bevindt het default codesegment. De functies binnen één brontekst zouden elkaar dan dus als near functies kunnen aanroepen: ze liggen immers in hetzelfde segment. In het medium of large geheugenmodel is het dan ook handig om *static* functies (die dus alleen binnen één brontekst bekend zijn) als near functies te definiëren: de aanroep van dergelijke functies gaat dan snel en er is weinig code voor nodig. Ter illustratie geven we in listing 70 de functie *lookup()* die als een *static _near* functie is gedefinieerd en wordt aangeroepen als hulpfunctie van de functie *fun()* die zelf niet als een *_near* functie is gedefinieerd:

```
static void _near lookup (char key)
{
    // code van de functie lookup()
}

void fun ()
{
    ....
    lookup ('a');
    ....
}
```

Listing 70: Aanroep van een *static _near* functie.

Near funties kunnen ook goed worden gebruikt als hulpfuncties binnen een bepaalde context. De context is dan een functie zoals die zich



Figuur 12.4: Zichtbaarheid van hulp- en interface-functies.

aan de software-ontwikkelaar voordoet. Als voorbeeld kan worden gedacht aan functies waarmee vensters op het beeldscherm kunnen worden gedefinieerd en gemanipuleerd [9]. Dergelijke functies vormen de buitenkant van het systeem: de 'interface' naar de overige programmatuur. In medium en large model programma's zijn deze interface-functies far. Zodra zo'n far functie is geactiveerd, is het default codesegment veranderd. De interface-functie kan vervolgens een beroep doen op hulpfuncties die binnen hetzelfde codesegment zijn gelokaliseerd als de interface functie zelf. De hulpfuncties kunnen dan als near functies worden gedefinieerd, met de bijbehorende voordelen van korte en snelle aanroepcode. De interface-functies vormen als het ware het topje van de ijsberg van het systeem: alleen de interface-functies zijn 'zichtbaar' voor andere functies in het programma, de hulpfuncties van het systeem blijven voor andere functies onzichtbaar. Deze situatie is geïllusteerd in figuur 12.4.

Wanneer de (hulp)functies van zo'n systeem vertaald worden voor het medium of large geheugenmodel, moeten alle functies van het systeem uiteindelijk door de linker in één codesegment worden geplaatst. Dan kunnen alle near èn far functies van het systeem elkaar aanroepen. Alleen de interface functies zijn vervolgens vanuit functies die in andere codesegmenten zijn geplaatst aanroepbaar.

9. Het windowsysteem cow (**C** Output Windows) (Kubat, 1990) is een concreet voorbeeld van een systeem waarbij near hulpfuncties zijn gebruikt.

De vertaler leidt in het medium en large model codesegment namen af uit de naam van de brontekst, waarachter nog *_text* wordt geplaatst. Wanneer alle functies die bij een bepaalde context horen uiteindelijk in eenzelfde segment moeten worden geplaatst, dan moet de vertaler met behulp van de /**Nt** optie op de hoogte worden gesteld van de gewenste codesegmentnaam. Zo zijn alle functies van het cow systeem vertaald met de optie:

/Nt cow_text

waardoor het codesegment van de cow functies *cow_text* heet[10].

Uiteraard gelden deze overwegingen niet in de small en compact geheugenmodellen, waarbinnen immers maar één default codesegment gehanteerd wordt. Wanneer het small of compact geheugenmodel wordt gebruikt, moet de /**Nt** vertaler optie niet worden gebruikt.

12.4.3 Return-waarden van functies

Als functies niet *void* zijn, dan retourneren zij een waarde. Voor de geretourneerde waarde geldt niet dat deze far of near kan zijn: een waarde is gewoon een waarde. De keywords *_near* en *_far* voor de naam van een functie hebben betrekking op de wijze waarop de functie wordt aangeroepen (respectievelijk met een near call en far call).

Echter, functies kunnen ook pointerwaarden retourneren. Bij pointerwaarden is het wèl van belang om te weten of datgene waar naar wordt gewezen al dan niet in het default-segment ligt. In het ene geval is er sprake van een near pointer, in het andere geval is er sprake van een far pointer. Er kunnen zich vier verschillende situaties voordoen: een functie kan zelf near of far zijn, en onafhankelijk daarvan een near of far pointer retourneren. In figuur 12.5 zijn prototypes van functies voor deze vier situaties gegeven. In het voorbeeld retourneren de functies *void* pointers, maar het uiteindelijke datatype kan evengoed *int, char,* zelf weer een pointertype of een ander, zelf gedefinieerd type zijn. Belangrijk is de vuistregel dat de keywords *_near* en *_far* betrekking hebben op datgene dat onmiddellijk volgt op het keyword.

Wie de vuistregel opvolgt, komt een heel eind met complexe definities. Wat te denken van:

Definieer een pointer naar een functie die een void pointer retourneert. Echter, de pointer moet in het default datasegment worden gelokaliseerd, de functie waarnaar wordt verwezen moet buiten het default codesegment kunnen zijn gelokaliseerd en de pointer die wordt geretourneerd wijst op een object in het default datasegment.

10. Aangeraden wordt om zelf gedefinieerde segmentnamen het achtervoegsel *_text* te geven. Deze gewoonte wordt ook al gevolgd door de vertaler en zorgt ervoor dat alle segmenten waarin code staat duidelijk herkenbaar zijn in het overzicht van segmenten dat door de linker kan worden geproduceerd.

Returnwaarde:	Functie staat in default codesegment	Functie staat niet in default codesegment
In default-segment	*void _near ∗ _near fun();*	*void ∗ _far fun();*
Buiten default-segment	*void _far ∗ _near fun();*	*void _far ∗ _far fun();*

Figuur 12.5: Prototypes van near/far functies die near/far pointers retourneren.

Toch is de constructie van een dergelijke variabeledefinitie niet moeilijk. De aanduidingen van de aard van de segmenten (near of far) is een uitbreiding van Microsoft. Deze aanduidingen laten we eerst even weg, om ze later op de juiste plaats weer in te voegen. We krijgen dan:

- Er moet een pointer naar een functie die een pointer naar een void retourneert, worden gedefinieerd. Een dergelijke variabele is bijvoorbeeld (zie ook 8.5):

  void ∗ (∗funptr)();

- De pointer is in het default datasegment gelokaliseerd. Voor de naam van de variabele komt dus het keyword *_near:*

  void ∗ (∗_near funptr)();

- De functie waarnaar wordt gewezen kan buiten het default codesegment zijn gelokaliseerd. De pointer is dus far en voor de pointer operator die betrekking heeft op de functie komt dus het keyword *_far:*

  void ∗ (_far ∗_near funptr)();

- Tenslotte wordt een pointer naar het default datasegment geretourneerd. Voor de pointer operator die betrekking heeft op de return-waarde komt dus het keyword *_near:*

  void _near ∗(_far ∗_near funptr)();

De laatste definitie wordt vervolgens uitgesproken als:

funptr *is een near pointer naar een far functie, die een near* void *pointer oplevert.*

Opgave 72.

Hoe kan een small model programma toch groter zijn dan 128 Kbytes?

Opgave 73.

Geef in volledige termen van near, far, pointer naar, array van, functie die ... retourneert en basistype een beschrijving van de hieronder genoemde variabelen. Ook waar géén modifiers zijn gegeven moet, afhankelijk van het model en waar van toepassing, de omvang van onderdelen in termen van near en far worden beschreven.

1. Small model: double _far *(* var)();
2. Large model: double _near *(* var)();
3. Compact model: char *(*var[])();
4. Large model: static char _far *_near fun();

Opgave 74.

Geef de variabele definitie van een far pointer naar een functie die een pointer retourneert naar een blok van 70.000 integers, dit in een small model programma.

Opgave 75.

Een voorbeeld van een applicatie die goed in compact model kan worden vertaald is het help-programma, dat is beschreven in 11.6. De reden daarvan zal duidelijk zijn: het programma kan in voorkomende gevallen veel geheugen voor data nodig hebben, terwijl er maar weinig code is. Vertaal de functies van het help-programma opnieuw, maar nu voor het compact model en maak een nieuw programma. Beredeneer het antwoord op de vraag of oude indexbestanden nog kunnen worden gebruikt wanneer het programma zelf opnieuw is vertaald en gelinkt.

Hoofdstuk 13

Iteratieve processen

13.1 Inleiding

In dit hoofdstuk worden *iteratieve processen* besproken. Bij een iteratief proces geldt dat een bewerking herhaald wordt, totdat het gewenste resultaat bereikt is. Bovendien wordt het resultaat van de bewerking gebruikt als invoer voor de volgende stap. De iteratie (herhaling van een actie) bereft hier hier *niet* de herhaling zoals deze optreedt bij iteratieve statements (zoals het *for* of *while* statement). Veeleer gaat het om iteratie op een hoger conceptueel niveau.

Een voorbeeld van een iteratief proces is de berekening van de verspreiding van een besmettelijke ziekte in een populatie. Hier bepaalt het relatieve aantal zieken hoeveel procent van de populatie besmet wordt, met andere woorden hoeveel zieken er in de volgende tijdseenheid zullen zijn. Als er meer zieken zijn, worden meer gezonden besmet. In die volgende tijdseenheid is dit aantal weer bepalend voor verdere verspreiding. Bij modellen van de verspreiding van een ziekte spelen tevens factoren als incubatietijd, sterfte door ziekte, aanvoer van gezonden door geboorten, verspreiding door migratie, dichtheid van de bevolking, enzovoort, ook een rol.

Een tweede voorbeeld van een iteratief proces is de bepaling van een nulput van een functie aan de hand van de functie zelf en van zijn afgeleide. Dit proces heet de Newton-Raphson [1] methode. Hierbij wordt op een willekeurig punt op de functie een schatting gemaakt van het nulpunt. Deze geschatte waarde wordt weer gebruikt om een volgende, nauwkeuriger, schatting te maken. Dit proces wordt herhaald totdat de verkregen schatting nauwkeurig genoeg is.

Samengevat kan een iteratief algoritme gebruikt worden

- als de oplossing van een probleem inhoudt dat een bepaalde bewerking herhaald wordt,
- als bij een iteratie de resultaten van de vorige iteratie gebruikt worden,
- als de oplossing van het probleem een berekening over discrete tijdstappen inhoudt.

Het laatste punt in deze opsomming is geen voorwaarde per se, maar eerder een sterke aanwijzing.

13.1.1 Annuïteitsberekening

Als een concreet voorbeeld van een iteratief proces is in listing 71 een functie gegeven, die berekent hoe het kapitaal op een spaarrekening groeit. Als bijvoorbeeld het rentepercentage 5% is, dan groeit het kapitaal na één jaar met vijf procent. Na nog eens een jaar wordt echter ook rente over de rente

1. Zie Press (1988) e.a., *Numerical Recipes in C*.

van het eerste jaar uitgekeerd. In listing 71 is tevens een *main()* functie, die een en ander test, gegeven.

Overigens dient opgemerkt te worden dat dit voorbeeld triviaal is; de berekening van het kapitaal kan sneller plaatsvinden als een logaritmische berekening (dus zonder iteratie) gebruikt wordt.

```
1     #include <stdio.h>
2     #include <stdlib.h>
3
4     double bank (double capital, int years, double percentage)
5     {
6         register int
7             i;                          // jaren teller
8
9         percentage /= 100.0;            // gebruik fracties
10        for (i = 0; i < years; i++)     // elk jaar:
11            capital +=                  // tel de rente erbij
12                capital * percentage;
13
14        return (capital);               // retourneer nieuw kapitaal
15    }
16
17    void main ()
18    {
19        char
20            buffer [80];
21        double
22            cap,
23            per;
24        int
25            years;
26
27        printf ("Hoeveel geld heb je ?" );
28        cap = atof (gets (buffer));
29        printf ("Hoeveel percent rente krijg je op de bank ?" );
30        per = atof (gets (buffer));
31        printf ("Hoeveel jaar laat je je geld op de bank staan ?" );
32        years = atoi (gets (buffer));
33
34        printf ("\nDan heb je na %d jaren %0.2lf gulden.\n",
35            years, bank (cap, years, per));
36    }
```

Listing 71: Annuïteitsberekening door middel van een iteratieve functie.

In de functie *bank()* is de iteratie duidelijk te zien: het kapitaal wordt met een bepaald percentage verhoogd; en dit getal wordt in een volgend jaar (bij het doorlopen van het *for* statement) weer als argument voor de berekening gebruikt.

13.1.2 Een ecologisch evenwicht

Als tweede voorbeeld van een iteratief proces laten we hier zien hoe een ecologisch systeem met een prooidier en een jager gesimuleerd kan worden. We stellen ons het volgende probleem:

Op de Hoge Veluwe zijn de laatste jaren erg veel konijnen gesignaleerd. Dit zou op zich niet erg zijn, ware het niet dat door de overbevolking van de konijnen de plantengroei achteruit gaat en ziekten onder de konijnen steeds vaker voorkomen.

Staatsbosbeheer overweegt daarom 10 vossen uit te zetten. Hoe groot is de verwachte uiteindelijke afname van de konijnenpopulatie en hoe lang zal deze afname op zich laten wachten?

In dit onderdeel zullen we een model van een ecologisch systeem met vossen en konijnen ontwikkelen, waarbij we de volgende aannamen maken[2]:

- We gaan ervan uit dat het betreffende gebied 400 km^2 groot is. In dit gebied kunnen de dieren zich vrij bewegen.
- We beginnen met een bepaalde startpopulatie konijnen (bijvoorbeeld 2000) en een bepaalde startpopulatie vossen (bijvoorbeeld 10).
- De konijnenpopulatie is onderhevig aan een bepaalde jaarlijkse groei en sterfte. De jaarlijkse groei is in principe 10%, maar is ook afhankelijk van het aantal konijnen per vierkante kilometer. We nemen in het model aan dat bij vijf konijnen per km^2 de groei inderdaad 10% is. Bij tien konijnen per km^2 loopt de groei echter terug tot 0%. Deze afname van de groei is het resultaat van een tekort aan voedsel en van het uitbreken van ziekten. Bij één konijn per km^2 is de groei 20%.
- De sterfte in de konijnenpopulatie is in principe 25% (de gemiddelde leeftijd van een konijn is vier jaar), maar is uiteraard ook afhankelijk van het aantal vossen. Als in het gebied één vos per km^2 rondloopt, dan neemt de sterfte toe tot 75%.
- Ook de vossenpopulatie is onderhevig aan een jaarlijkse groei en sterfte. Aangezien de vossen geen natuurlijke vijanden hebben, kunnen we de sterfte constant houden op 15% (een vos leeft gemiddeld 6 à 7 jaar). De groei van de vossenpopulatie is echter niet constant maar afhankelijk van het aantal konijnen. We gaan ervan uit dat de groei van de vossenpopulatie in principe 2% bedraagt. Maar aangezien een vos jaarlijks 50 konijnen verschalkt, kan deze groei alleen gehandhaafd worden als er genoeg konijnen in de buurt zijn. Als er te weinig konijnen zijn, is de groei van de vossenpopulatie evenredig kleiner.

Het iteratieve karakter van deze simulatie blijkt als we voor een benadering kiezen waarbij de aantallen konijnen en vossen in een jaar bepalend zijn voor de aantallen in het volgende jaar. Hier komt duidelijk naar voren dat de resultaten van een berekening weer bij een volgende berekening gebruikt worden.

```
1     #define STARTRABBITS          2000.0
2     #define RABBITGROWTHRATE      1.1
3     #define RABBITDEATHRATE       0.25
```

2. De hier genoemde getallen zijn willekeurig. Elke overeenkomst met levende personen, dieren of ware gebeurtenissen berust op toeval.

```
4
5     #define STARTFOXES              10.0
6     #define FOXGROWTHRATE           1.02
7     #define FOXDEATHRATE            0.15
8
9     #define LANDSIZE                (20.0 * 20.0)
10
11    double
12        nrabbits = STARTRABBITS,
13        nfoxes   = STARTFOXES;
14
15    double next_rabbits ()
16    {
17        double
18            growthrate = RABBITGROWTHRATE,
19            deathrate  = RABBITDEATHRATE;
20
21        growthrate *= (-0.2 * (nrabbits / LANDSIZE) + 2);
22        deathrate  *= (0.03 * nfoxes + 1.0);
23
24        return ( nrabbits + (nrabbits * growthrate) -
25                            (nrabbits * deathrate)
26               );
27    }
28
29    double next_foxes ()
30    {
31        double
32            growthrate = FOXGROWTHRATE;
33
34        growthrate *= (0.02 * (nrabbits / nfoxes));
35        return (nfoxes + (nfoxes * growthrate) -
36                         (nfoxes * FOXDEATHRATE)
37               );
38    }
39
```

Listing 72: Een ecologisch model: functies *next_rabbits()* en *next_foxes()*.

Aan de hand van de hierboven gegeven regels is de groei en sterfte van de konijnen- en vossenpopulatie in formulevorm samen te vatten. Dit is in de functies *next_rabbits()* en *next_foxes()* te zien, zie listing 72. De groei in de konijnenpopulatie is omgekeerd recht evenredig met het aantal konijnen. De sterfte in de konijnenpopulatie is recht evenredig met het aantal vossen. Voor de groei van de vossenpopulatie geldt dat deze groei recht evenredig is met het aantal konijnen dat per vos gejaagd kan worden, dus de konijnen / vossen ratio.

De functie *main()* die het iteratieve proces drijft, is in listing 73 gegeven.

```
1     #include <conio.h>
2     #include <stdio.h>
3
4     void main ()
5     {
6         register unsigned
7             nlines = 0,
8             year = 0;
9         double
10            newr,
11            newf;
12
```

```
13        while (1)
14        {
15            printf ("Year: %3d\t Rabbits: %7ld, foxes: %7ld\n",
16                ++year, (long) nrabbits, (long) nfoxes);
17            if ( (newr = next_rabbits ()) < 0.0)
18            {
19                puts ("Rabbits died out.");
20                return;
21            }
22            if ( (newf = next_foxes()) < 0.0)
23            {
24                puts ("Foxes died out.");
25                return;
26            }
27            nrabbits = newr;
28            nfoxes   = newf;
29            if (! (++nlines % 24) )
30            {
31                printf ("Any key.");
32                getch ();
33                printf ("\r        \r");
34            }
35        }
36    }
```

Listing 73: Een ecologisch model: De *main()* functie.

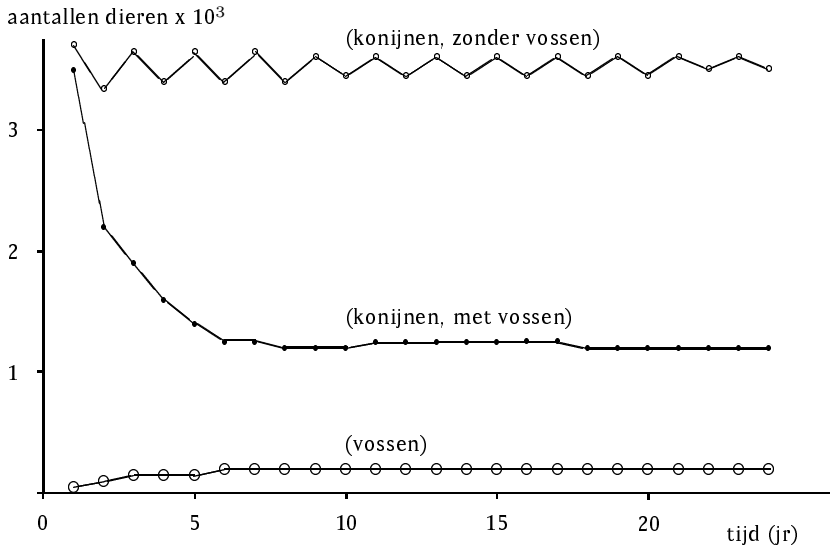
Evaluatie van het ecologische model

De evaluatie van het hier ontworpen ecologische model vindt in twee stappen plaats. Ten eerste moet geverifieerd worden of het model de hoge groei van de konijnenpopulatie goed weergeeft. Hiertoe wordt in de brontekst de constante *STARTFOXES* op waarde 0.0 gezet. Tevens wordt in functie *next_foxes()* ingegrepen: de functie retourneert altijd 0.0. Door deze twee ingrepen is de vossenpopulatie uit het model gehaald. De resultaten van deze ingreep zijn in figuur 13.1 weergegeven als de serie kleine cirkels. We zien dat de konijnenpopulatie omhoog schiet, waardoor een toestand van overbevolking optreedt. In het volgende jaar is de konijnenpopulatie daardoor lager. Hierdoor kan in het volgende jaar de populatie weer groeien, enzovoort. Deze slingering dempt op den duur uit naar een stabiel evenwicht; we zeggen dat de slingering 'naar een ruststand convergeert'.

Bij de tweede stap van de evaluatie van het model wordt de vossenpopulatie in het model gebracht. De effecten hiervan op de konijnenpopulatie zijn in figuur 13.1 weergegeven als de serie bolletjes. De vossenpopulatie is weergegeven als de serie grotere cirkels. Duidelijk blijkt dat door de aanwezigheid van vossen de konijnenpopulatie in toom gehouden wordt. Het model bereikt een stabiel evenwicht.

Overigens dient opgemerkt te worden dat het hier gepresenteerde model voor de 'prey-predator' situatie niet het enige toepasbare model is. In 1926 werd al een vergelijkbaar model met succes toegepast op populaties van bepaalde vissoorten in de Adriatische zee [3]. Hierbij werden de

3. Dit model staat bekend als het Lotka-Volterra model. Het is o.a. beschreven in *Mathematical Biology* van J.D. Murray.



Figuur 13.1: Resultaten van het ecologische model.

populaties beschreven door differentiaalvergelijkingen.

13.2 De zin die zichzelf beschrijft

In dit hoofdstuk wordt een iteratief proces ontworpen, waarbij een zin ontstaat die over zichzelf vertelt hoeveel a's, hoeveel b's, hoeveel c's, enzovoort, erin staan [4]. De zin gaat er als volgt uitzien:

> "Deze zin bevat 1 keer een A, 1 keer een B, ... 1 keer een a, 1 keer een b, ... 1 keer een 1, 1 keer een 2, ... 1 keer een 9."

Hierin komt op de plaats van de puntjes de rest van de informatie (dus C t/m Z, c t/m z, en 3 t/m 8). Van elke letter wordt vermeld hoevaak deze voorkomt. De getallen die in bovenstaand voorbeeld genoemd zijn, zijn dus al fout: cijfer 1 komt al 8 maal voor. Ook de b komt niet één maar twee keer voor; één keer in *bevat* en één keer in *b*.

4. Dit idee is uitgebreid beschreven in D.R. Hofstadter's boek *"Metamagical Themas"*.

13.2.1 Waarom iteratie

Het iteratieve karakter van de bepaling van deze zin blijkt bij het kiezen van de methode waarmee de zin gemaakt wordt. We beginnen met een zin, waar de aantallen van de letters en cijfers willekeurig zijn (bijvoorbeeld nul). Deze zin noemen we de 'nulde generatie zin', aangezien hier alles begint. De nulde generatie zin ziet er daarom als volgt uit:

"Deze zin bevat 0 keer een A, 0 keer een B, ... 0 keer een 9.

Deze zin wordt zodanig bewerkt dat de aantallen bijgesteld worden. De 'eerste generatie zin' zal dus vermelden dat er heel veel nullen in de zin voorkomen. Die zin wordt weer bewerkt, enzovoort, totdat de informatie in de zin over de zin klopt. Dan is het programma klaar: de informatie wordt op het scherm afgedrukt.

We kunnen twee taken in dit proces onderscheiden: het *maken* van de zin uit bekende aantallen van de cijfers en letters, en het *berekenen* van aantallen van de cijfers en letters aan de hand van de zojuist gemaakte zin. Bij de volgende slag (de nieuwere generatie) worden die aantallen weer gebruikt om een nieuwe zin te maken.

De iteratie is in deze opzet duidelijk aanwezig. De berekening van een volgende generatie zin houdt in dat de zin die bij de vorige generatie is onstaan als uitgangspunt gebruikt wordt. De berekening resulteert in een nieuwe zin. De iteratie kan stoppen als de nieuw berekende zin niet afwijkt van de zin van de vorige generatie.

13.2.2 Organisatie van de data en eerste herschrijving

In dit iteratieve proces zijn twee taken te onderscheiden. De eerste taak zorgt voor het berekenen van de aantallen letters en cijfers aan de hand van een *x*-de generatie zin. We kiezen voor een aanpak waarbij de aantallen cijfers en letters in een array van *unsigned int*s bewaard worden. We zullen dit array *freq[]* noemen.

Over de grootte van het array kunnen we al iets zeggen. De minimale omvang van dit array is 26 + 26 + 10; namelijk 26 posities voor de hoofdletters, 26 posities voor de kleine letters en 10 posities voor de cijfers. De index in dit array voor de letter A zou hierbij 0 zijn; de index voor de letter a zou hierbij 26 zijn. Deze vreemde indexering kunnen we vermijden door een array-grootte van 256 te kiezen en door de ASCII-waarde van een letter als index te gebruiken. Het aantal A's in de zin wordt daarmee bewaard op index 65 (dit is de ASCII-waarde van A), oftewel in *freq['A']*. De procedure om letters en cijfers te tellen is in het *ccount* programma (zie 5.3) al besproken.

De letters en cijfers die geteld moeten worden zullen we opslaan als een string. Deze string zullen we *relevant[]* noemen. In deze opzet is het dus mogelijk om de te tellen cijfers en letters uit te breiden of te beperken.

De tweede hoofdtaak bij het bouwen van een kloppende, zichzelf beschrijvende zin bestaat uit het aanmaken van de zin aan de hand van de getallen die in array *freq[]* zijn opgeslagen. Een zin wordt geconstrueerd door de volgende stappen te volgen:

- Het begin van de zin wordt string *"Deze zin bevat "*.
- Voor alle cijfers en letters die geteld moeten worden, wordt het aantal van de gevonden cijfers of letters bij de zin toegevoegd. Hierna wordt *" keer een "* aan de zin toegevoegd, gevolgd door de letter of cijfer waar het om gaat. Vervolgens wordt een komma aan de zin toegevoegd.
- De zin wordt afgesloten door een punt.

```
char
    sentence [4000],                                    // de zin
    relevant [] =                                       // relevante tekens
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        "abcdefghijklmnopqrstuvwxyz"
        "0123456789";

void make_sentence ()
{
    char
        thischar,                                       // relevant teken
        conversion_buf [80];                            // itoa() conversie
    register int
        nchars,                                         // aantal te verwerken
        i;                                              // lus teller

    strcpy (sentence, "Deze zin bevat" );
    nchars = strlen (relevant);                         // bepaal te verwerken
                                                        // aantal
    for (i = 0; i < nchars; i++)                        // doe voor elk teken:
    {
        thischar = relevant [i];                        // bepaal relevant teken
        if (i == nchars - 1)                            // gebruik 'en' bij
            strcat (sentence, "en" );                   // voorlaatst teken
        strcat (sentence,                               // voeg 'keer' toe
            itoa (freq [thischar], conversion_buf, 10));
        strcat (sentence, " keer een '" );
        sentence [strlen (sentence) - 1] =
            thischar;
        strcat (sentence, "'");                         // afsluitende '
        if (i == nchars - 1)                            // voeg tenslotte . toe
            strcat (sentence, ".");
        else                                            // of voeg , toe
            strcat (sentence, "," );
    }
}
```

Listing 74: De functie *make_sentence()*.

Het aanmaken van een zin aan de hand van de aantallen van letters en cijfers gebeurt in de functie *make_sentence()*. De code van deze functie is in listing 74 gegeven. Bij deze listing maken we nog de volgende opmerkingen:

Regel 2: Variabele *sentence* wordt gebruikt om de zin op te bouwen. Hiervoor zijn 4000 bytes beschikbaar. Merk op dat *sentence* een globale variabele is. De reden hiervan is, dat andere functies de zin moeten kunnen bewerken.

Regel 3: Variabele *relevant* is een *char* array waarin de te tellen cijfers en letters staan.

Regel 17: De nieuwe zin begint met de tekst *"Deze zin bevat"*. De initialisatie van de nieuwe zin is met behulp van functie *strcpy()* gerealiseerd.

Regel 21: Een letter waarvan de frequentie gezocht wordt, wordt in de variabele *thischar* geplaatst. *thischar* wordt verderop in de code gebruikt om *freq[]* te indiceren.

Regel 22: Als de laatste letter of het laatste cijfer bewerkt wordt, wordt de zin uitgebreid met *"en"*.

Regel 25: Voor de conversie van *int* naar string wordt de standaardfunctie *itoa()* (integer to ASCII) gebruikt. Deze functie verwacht drie argumenten: de te converteren *int* waarde, een pointer naar een buffer waar de conversie kan plaatsvinden en het grondtal van de conversie.

Het aantal dat naar string-formaat geconverteerd wordt is in dit geval *freq[thischar]*. Aangezien *thischar* de waarde van *relevant[i]* heeft, zou het te converteren getal evengoed door *freq [relevant [i]]* weergegeven kunnen worden. In de hier voorgestelde code is van de dubbele indicering afgezien, daar de leesbaarheid van bronteksten hierdoor afneemt.

Regels 30-33: Als de laatste letter of het laatste cijfer bewerkt is, dan wordt de zin met een punt afgesloten. Als dit niet het geval is, dan wordt een komma aan de zin toegevoegd.

13.2.3 De functie main()

Het iteratieve karakter van dit probleem is geïmplementeerd in de functie *main()*. Deze functie roept *make_sentence()* aan om een nieuwe zin te maken en roept vervolgens *count_sentence()* aan om de aantallen van letters en cijfers in de zin na te tellen. De functie *count_sentence()* signaleert via de return-waarde of de aantallen van letters en cijfers gewijzigd zijn ten opzichte van de vorige zin. Als er een wijziging optreedt, moet *main()* wederom *make_sentence()* en *count_sentence()* aanroepen. De functie *main()* is in listing 75 weergegeven.

13.2.4 De functie count_sentence()

De functie *count_sentence()* is de 'pièce de résistance' van dit probleem. De taken van deze functie zijn als volgt samen te vatten:

1. Tel alle tekens in *sentence*, en onthoud hun respectievelijke aantallen.
2. Als deze aantallen overeenkomen met de aantallen van de zin in de vorige generatie (bewaard in *freq[]*), dan moet de aanroeper hiervan op de hoogte gebracht worden. De iteratie kan dan stoppen.

```
void main ()
{
    register int
        i;

    do
        make_sentence ();
    while (! count_sentence ());
    puts (sentence);
}
```

Listing 75: De functie *main()*.

3. Als deze aantallen *niet* overeenkomen met de aantallen van de zin in de vorige generatie, dan moeten de berekende aantallen in *freq[]* gezet worden.

Opgave 76.

Implementeer de functie *count_sentence()* aan de hand van de hierboven gegeven samenvatting. Maak gebruik van het code-frame dat in listing 76 is gegeven.

```
int count_sentence ()
{
    register int
        ret,                                        // return-waarde
        i;                                          // lus teller
    unsigned
        local_freq [256];                           // lokale teken-frequenties

    // initialiseer local_freq[] op 0
    memset (local_freq, 0, sizeof (local_freq));

    // tel de zin
    for (i = 0; i < strlen (sentence); i++)
        local_freq[sentence[i]]++;

    // veronderstel dat de telling klopt
    ret = 1;
    // vergelijk lokale en globale telling
    for (i = 0; i < strlen (relevant); i++)
    {
        if ( de lokale frekwentie niet gelijk is
             aan de globale frekewntie
           )
        {
            zet de return-waarde op 'niet klaar'
            de globale frekwentie wordt de lokale frekw.
        }
    }
    indien klaar, retourneer 'ret'
}
```

Listing 76: Code-frame van de functie *count_sentence()*.

Opgave 77.

Knoop de programmadelen aaneen tot een programma *sentence.exe*. Hoe lang doet het programma erover om een kloppende zin te maken?

13.2.5 Een uitbreiding

Het is ook aardig om te zien op grond waarvan het programma besluit dat de zin nog niet klaar is. Deze informatie is beschikbaar in de functie *count_sentence()*, wanneer bekeken wordt of de zojuist bepaalde frequenties in array *local_freq[]* overeen komen met de frequenties van de vorige slag, bewaard in *freq[]*. Als bij deze controle blijkt dat de zojuist bepaalde frequenties afwijkend zijn van de vorige frequenties, zou het programma af kunnen drukken dat voor die en die letter de frequenties nog niet kloppen. Stel bijvoorbeeld dat gevonden wordt dat de frequentie voor de letter a nog niet in orde is, dan zou het programma een melding als *"Frequentie voor a bijgesteld"* kunnen afdrukken.

Opgave 78.
Implementeer de hierboven beschreven aanpassing. Maak de hieronder gestelde opgaven met behulp van het aangepaste programma.

Een tweede aardigheid is het tellen van écht alle tekens die in de zin voorkomen; dus ook aanhalingstekens, spaties, komma's en punten. Dat is gemakkelijk implementeerbaar; immers, array *relevant[]* kan eenvoudig uitgebreid worden.

Opgave 79.
Laat het programma ook spaties, komma's, punten, en enkele aanhalingstekens (open en sluiten) tellen. Voer het programma uit. Wat zie je?

Hoofdstuk 14

Transitiematrixen en Eindige Toestandsautomaten

In dit hoofdstuk wordt een manier behandeld om functies te ontwikkelen die gebaseerd is op de theorie van de eindige toestandsautomaten. Hierbij wordt gebruik gemaakt van zogenaamde transitiematrixen (overgangsschema's) om functies mee te definiëren. In deze benadering wordt opnieuw data gebruikt waar eerder code werd gebruikt (zie ook pagina 214). De consequentie is hier niet alleen een gunstige 'time-space tradeoff', maar de werking van een functie wordt bovendien op zeer overzichtelijke wijze in tabelvorm vastgelegd. De methode wordt in de praktijk toegepast bij relatief eenvoudige programma's. Bij grotere programma's wordt doorgaans gebruik gemaakt van krachtiger hulpmiddelen, zoals programma's die tekstherkenners (lexical analyzers) genereren en programma's die grammaticale ontleders (parsers) genereren. Deze hulpmiddelen komen in dit boek ter sprake in hoofdstuk 18.

Een toepassing van de hier geïntroduceerde benadering wordt aan het einde van dit hoofdstuk gegeven in de vorm van een string grabber. De string grabber kan worden gebruikt om strings in een **C** brontekst te herkennen en naar het beeldscherm te schrijven. Een dergelijk programma vormt de basis voor een systeem dat **C** bronteksten zodanig kan modificeren dat de erin voorkomende strings in een apart bestand met strings worden geplaatst en worden verwijderd uit de oorspronkelijke bronteksten [1].

14.1 De eindige toestandsautomaat

Minsky (1972) definieert een eindige toestandsautomaat (finite state automaton, FSA) als een machine die uit een eindig aantal componenten bestaat, waarbij elke component een eindig aantal toestanden (een state) kent. Het aantal toestanden waarin een dergelijk apparaat zich kan bevinden, is het produkt over alle componenten van het aantal toestanden waarin ze zich kunnen bevinden. Dit aantal kan weliswaar zeer groot zijn, maar blijft eindig. We geven een paar voorbeelden:

- Een lichtschakelaar bestaat uit één component met twee toestanden (aan/uit). Het aantal toestanden van de lichtschakelaar is dus $1 * 2$.
- Een hotelschakelaar bestaat uit twee lichtschakelaars, die elk twee toestanden kennen. De lichtschakelaars zijn hier de componenten, en het aantal toestanden van de hotelschakelaar is dus $2 * 2 = 4$. Het effect van de vier toestanden is overigens dat een lamp òfwel

1. De functies van de 'OCOSGRAB environment' (Kubat, 1989) kunnen worden gebruikt om strings uit **C** bronteksten te extraheren en te vervangen door de aanroep van functies uit het OCOSGRAB systeem. Het systeem leidt ertoe dat strings uit het uiteindelijke programma worden verwijderd, waardoor het uiteindelijke programma kleiner wordt zonder noemenswaardig verschil in de snelheid van uitvoering.

brandt òfwel uit is. Sommige van de vier toestanden van de hotelschakelaar moeten dus hetzelfde bewerkstelligen.

- Een byte bestaat uit 8 bits, die elk twee toestanden kennen. Het aantal toestanden van de eindige toestandsautomaat die wordt gevormd door de byte is dus $2^8 = 256$.
- Het werkgeheugen van een PC bestaat uit maximaal 1 Megabyte $(= 1024^2)$[2]. Elk onderdeel (een byte) is een component met 256 toestanden. Het aantal toestanden waarin het geheugen van een PC zich kan bevinden is dus $(1024 * 1024)^{256}$ (We stellen voor dat de lezer zelf uitrekent hoeveel dat is).

We zullen om het begrip eindige toestandsautomaat nog wat meer 'handen en voeten' te geven een koffiezetter nader beschouwen. De koffiezetter is aan, òf uit. Om tussen deze toestanden te schakelen moet de aan-uit knop worden ingedrukt. Als de koffiezetter aan is geschakeld, dan kan er water in worden gegoten. Er begint dan koffie te druipen (terwille van de illustratie gaan we ervan uit dat er al koffie in het filter is gedaan). Als het water in het waterreservoir op is, dan is de koffie klaar en fungeert de koffiezetter als elektrisch theelichtje.

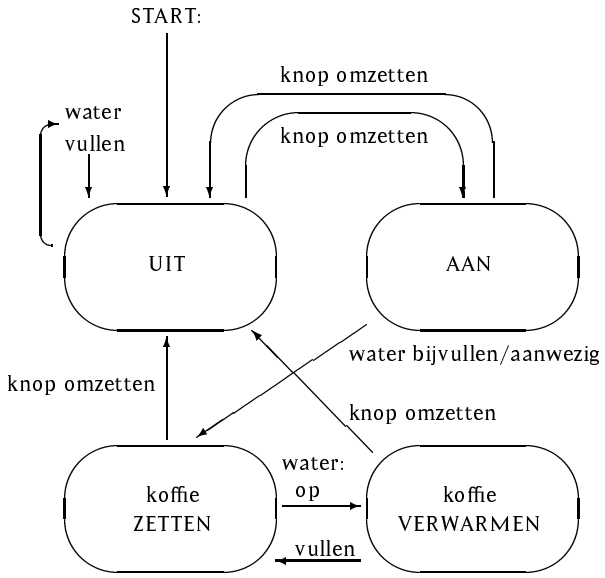
Voor ons doel zijn de volgende toestanden van de koffiezetter van belang:

1. De koffiezetter is uit.
2. De koffiezetter is aan.
3. Er wordt koffie gemaakt.
4. De koffiezetter verwarmt de gemaakte koffie.

Het verband tussen de toestanden van de koffiezetter en de handelingen die door ons worden uitgevoerd, wordt geïllustreerd door figuur 14.1. In deze figuur zijn de toestanden weergegeven als ovaaltjes, waarin de naam van de toestand staat. In elke toestand gebeurt er iets (in de toestand 'uit' wacht de koffiezetter op een opdracht, in toestant 'koffie zetten' wordt er koffie gezet). Tussen de toestands-ovaaltjes zijn pijlen getekend waarmee de mogelijke overgang van de ene toestand naar de andere wordt aangegeven. Naast de pijltjes staan de handelingen die ervoor zorgen dat de overgang wordt gerealiseerd. Deze handelingen noemen we de 'invoer'. Figuur 14.1 wordt een toestandsdiagram genoemd. Toestandsdiagrammen worden gebruikt om de toestanden van eenvoudige, eindige toetstandsautomaten te beschrijven. We noemen de volgende kenmerken aan de hand van figuur 14.1:

- Het toestandsdiagram heeft een speciale toestand die de begintoestand wordt genoemd. De begintoestand wordt onmiddellijk bereikt vanuit de pijl die vanuit de invoer 'START' is getrokken. Het is in dit voorbeeld de UIT toestand. Elke eindige toestandsautomaat heeft één begintoestand.

2. Het 'extended memory' is werkgeheugen boven het 1 Megabyte adres. Dit geheugen is onder DOS echter niet direct adresseerbaar



Figuur 14.1: De toestanden van de koffiezetter.

- Vanuit elke toestand is er bij een normaal verlopend proces[3] minstens één overgang naar een andere toestand. Zo zorgt het omschakelen van de schakelaar ervoor dat de toestand UIT afwisselend wordt verlaten en weer wordt bereikt. De invoer die de overgang verzorgt is in dat geval het omzetten van de schakelaar.
- Wanneer er niets gebeurt blijft de toestand ongewijzigd.
- Soms leidt invoer ertoe dat de toestand niet wordt gewijzigd. Zo blijft de toestand 'UIT' gehandhaafd wanneer water in het waterreservoir wordt gegoten. In deze situatie leidt alle irrelevante invoer als het ware tot dezelfde toestand. In dergelijke situaties spreken we van de 'default transitie'.

Verderop in dit hoofdstuk zullen eindige toestandsautomaten de revue passeren die toestanden kennen waarbij een default transitie wordt uitgevoerd bij alle niet verder gespecificeerde invoer. Alle invoer die bij dergelijke toestanden mogelijk is, maar niet expliciet is vermeld, leidt dan tot de default transitie.

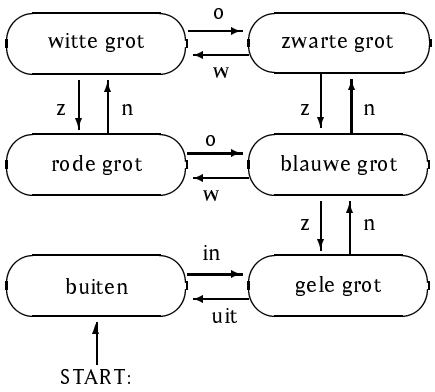
14.2 De transitiematrix

In Kidder (1983, p. 82 ff.) komen we de volgende passage tegen:

> *...before me ... lay dozens of roughly drawn maps. They consisted of circles, inside of which were scrawled names such as Dirty Passages .... Webs of lines connected the circles, and each line was labeled, some with points of the compass, some with words* up *and* down.

Lijkt deze beschrijving niet sterk op de beschrijving van een toestandsdiagram? Het is een beschrijving van wat de schrijver (Tracy Kidder) aantreft op het bureau van iemand die het computerspel adventure speelt. Wie nog nooit adventure heeft gespeeld, wordt aangeraden pogingen in het werk te stellen dit alsnog te doen. Het spel adventure brengt de speler als hoofdpersoon in een (verbaal gepresenteerde) wereld die lijkt op die welke door Tolkien (1976) in 'Lord of the Rings' is beschreven. De speler waant zich in een wereld van grotten, waarin schatten moeten worden verzameld, tegenstanders moeten worden overwonnen en waarin de weg moet worden gevonden. De hierboven aangehaalde passage heeft betrekking op dat laatste aspect: de grottenwereld wordt gerepresenteerd als een transitiediagram, waarbij de overgangen tussen de toestanden worden bereikt door de vereiste opdrachten zoals *n* (north), *s* (south) en dergelijke in te tikken. Deze informatie kan ook in de vorm van een tabel in plaats van een diagram worden weergegeven. Dan ontstaat de transitiematrix. Een

3. Een toestandsdiagram kent soms toestanden zonder een overgang naar andere toestanden, zoals toestanden die bij fouten worden bereikt, of zoals bij een eindtoestand die aan het einde van een proces kan worden bereikt.



Figuur 14.2: Een transitiediagram voor een grottenstelsel.

zeer eenvoudig grottenstelsel zoals dat bij een programma als adventure zou kunnen voorkomen is gegeven in het transitiediagram in figuur 14.2. Dit transitiediagram zal nu worden omgezet naar een transitiematrix.

De constructie van een transitiematrix begint met het opzetten van een matrix met voor de rijen de verschillende toestanden en boven de kolommen alle mogelijke invoer. De invoer kan worden aangegeven als de feitelijke invoer, als verzamelingen van mogelijke invoer, of als een restcategorie (de default invoer). Een veel gebruikte ingang is 'alle overige invoer', waarbij de huidige toestand niet wordt gewijzigd. Het uitgangspunt bij de constructie van de transitiematrix van het grottenstelsel is dan ook een matrix waarbij de horizontale ingang (dus: de rij-ingang) de plaats is waar een speler zich op dit moment bevindt en de verticale ingang (dus: de kolom-ingang) de ingetikte opdracht is. Op het aldus aangeduide matrixelement moet de volgende positie van de speler komen te staan. Bijvoorbeeld, als de speler in de gele grot is en de opdracht *n* geeft, dan komt de speler in de blauwe grot. Op het kruispunt van de rij *geel* en de kolom *n* komt dan dus *blauw* te staan. Op deze wijze kan de volledige transitiematrix worden gevuld, uitgaande van het transitiediagram. De volledig gevulde transitiematrix is gegeven in figuur 14.3. Desgewenst kan de matrix worden uitgebreid met een extra kolom voor de overige invoer, waarbij de huidige toestand gehandhaafd blijft.

Een belangrijk kenmerk van transitiematrixen zoals die uit figuur 14.3 is dat bij elke willekeurige invoer slechts één overgang (transitie) naar een nieuwe toestand is (eventueel kan de nieuwe toestand dus dezelfde toestand zijn). Bijvoorbeeld: bij de invoer *n* in de blauwe grot volgt altijd de overgang naar de zwarte grot, nooit naar een andere grot. Wanneer,

Huidige	Invoer:					
plaats	in	uit	n	o	z	w
buiten	geel	buiten	buiten	buiten	buiten	buiten
geel	geel	buiten	blauw	geel	geel	geel
rood	rood	rood	wit	blauw	rood	rood
blauw	blauw	blauw	zwart	blauw	geel	rood
wit	wit	wit	wit	zwart	rood	wit
zwart	zwart	zwart	zwart	zwart	blauw	wit

Figuur 14.3: De gevulde transitiematrix.

in het algemeen, de huidige toestand wordt aangegeven door de variabele *state,* de invoer wordt aangegeven door de variabele *command,* en *transition* een tweedimensionale matrix is van toestanden bij invoermogelijkheden, dan wordt de nieuwe toestand gevonden als:

newstate = transition[state][command];

Een transitiematrix die dit kenmerk heeft, beschrijft een deterministische eindige automaat (deterministic finite state automaton). Er zijn ook niet-deterministische eindige automaten (zie Aho, 1986, p. 114 ff.), waarbij bij een gegeven invoer meer dan één nieuwe toestand mogelijk is. Echter, voor elke niet-deterministische eindige automaat kan een deterministische eindige automaat worden ontwikkeld die hetzelfde gedrag vertoont. De transitiematrix van een dergelijke deterministische eindige automaat kan al snel te groot worden voor 'hand-constructie' en daardoor het doel van dit hoofdstuk voorbijschieten. Bij de constructie van complexe en grote, eindige toestandsautomaten worden daarom speciale programmageneratoren gebruikt, zoals gepresenteerd in hoofdstuk 18.

14.3 Een toepassing: de string grabber

Toepassingen voor eindige toestandsautomaten worden vooral gevonden in situaties waar zogenaamde reguliere expressies (zie ook hoofdstuk 18) moeten worden herkend. In de praktijk zijn dat situaties waarbij door een programma invoer moet worden herkend, die volgens bepaalde regels moet zijn georganiseerd. De invoer zelf kan afkomstig zijn van informatie die via het toetsenbord wordt aangeboden, maar de invoer kan ook een bestand zijn dat door het programma wordt verwerkt.

Wanneer een programma strings uit een **C** brontekst moet herkennen, dan lijkt dat op het eerste gezicht een eenvoudig probleem, met als intuïtieve oplossing dat alles wat tussen dubbele aanhalingstekens staat een string is. Zo eenvoudig is het echter niet. We noemen een aantal situa-

ties waarin wèl dubbele aanhalingstekens voorkomen, terwijl er toch geen sprake is van een string:

- In commentaar (tot het einde van de regel of verdeeld over meerdere regels) komen geen strings voor. Dubbele aanhalingstekens in dergelijke situaties worden geïnterpreteerd als onderdeel van het commentaar.
- In een string kan een escape-sequence worden gebruikt om een dubbel aanhalingsteken als zodanig te definiëren, in plaats van als het einde van de string.
- Een dubbel aanhalingsteken kan ook als een character constante voorkomen. De character constante 'dubbel aanhalingsteken' is echter nooit het begin van een string.

Behalve de hier genoemde situaties, waarin een dubbel aanhalingsteken toch niet het begin of einde van een string aangeeft, kan het natuurlijk ook voorkomen dat de aangeboden brontekst niet volgens de syntaxis van **C** is opgesteld en dus syntactische fouten oplevert bij vertaling door een **C** vertaler. Dat probleem wordt opgelost door het uit de weg te gaan. Het uitgangspunt is dat het te ontwikkelen programma alleen strings in syntactisch correcte **C** bronteksten hoeft te kunnen verwerken.

14.3.1 Het transitiediagram van de string grabber

De invoer van de string grabber bestaat uit individuele tekens die vanuit een brontekst worden ingelezen. In eerste instantie bevindt de string grabber zich in een starttoestand, waarbij er niets bijzonders aan de hand is. Dit is het uitgangspunt van de constructie van het transitiediagram. De invoer die van belang is, wordt vervolgens 'op een rijtje' gezet. Van belang zijn:

- Het dubbele aanhalingsteken omdat daarbij een string kan beginnen.
- Het enkele aanhalingsteken, omdat daarbij een character constante kan beginnen.
- De schuine streep *(/)*, omdat daarbij een commentaar kan beginnen.
- De asterisk, omdat daarna (in combinatie met de schuine streep) commentaar begint of eindigt.
- De omgekeerde schuine streep *(\)*, omdat daarachter een escape-sequence kan staan.
- Het end-of-line teken (`\n`), omdat daarachter een commentaar tot end-of-line ophoudt.
- Alle overige tekens, die geen speciale status hebben.

Niet alle invoer is in alle gevallen belangrijk. Zo is er in de *start* toestand geen sprake van commentaar en wordt een teken als het sterretje dan ook

opgenomen in de restcategorie van tekens zonder speciale status. In het algemeen kunnen invoercategorieën die in een toestand eenzelfde betekenis hebben, worden samengevoegd tot één (rest)categorie.

Uitgaande van de *start* toestand wordt vervolgens bij alle mogelijke invoer de vraag gesteld of er bij die invoer iets bijzonders aan de hand is. Als dat het geval is, dan leidt een dergelijke invoer tot een nieuwe toestand, waarbij de desbetreffende invoer dus tot een toestandswijziging zal leiden.

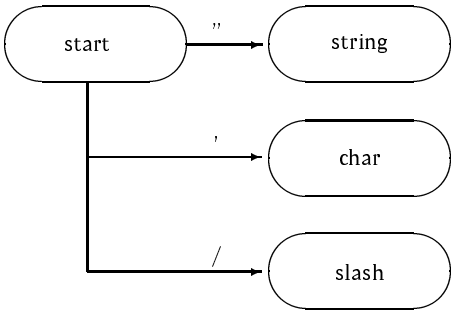
Dit proces wordt net zolang herhaald, totdat er geen nieuwe toestanden meer zijn ontstaan. We kunnen het algoritme om een toestandsmatrix te construeren heel aardig herformuleren in de gebiedende wijs. Daardoor wordt een basis gelegd voor een te ontwikkelen programma dat behulpzaam kan zijn bij het opstellen van transitiematrixen of transitiediagrammen. Deze herformulering is:

1. Bepaal de invoer-verzameling.
2. Initialiseer de transitiematrix op de starttoestand.
3. Zet een toestands-index op de start-toestand.
4. Zolang nog niet alle toestanden zijn verwerkt:
    5. Stel bij elk element van de invoerverzameling de vraag of bijbehorende invoer in de huidige toestand tot iets bijzonders moet leiden.
    6. Zo ja,
        7. Vraag de naam van de toestand en voeg een nieuwe toestand toe aan de toestandsmatrix wanneer de naam nog niet bekend was.
        8. Geef in de transitiematrix aan dat in de combinatie van huidige toestand en het huidige invoerelement een overgang naar de nieuwe toestand moet worden uitgevoerd.
    9. Zo nee, geef dan in de transitiematrix aan dat bij het gegeven invoerelement de huidige toestand gehandhaafd moet blijven.

De constructie van het programma zal hier niet worden uitgevoerd. We zullen ons beperken tot de constructie van het transitiediagram zelf, waarbij we het hier gegeven algoritme zullen volgen. De invoerverzameling is hierboven al genoemd en we gaan ervan uit dat het transitiediagram geïnitialiseerd is op de *start* toestand.

In de *start* toestand leiden de volgende invoerelementen tot een andere toestand:

- Een dubbel aanhalingsteken zal een overgang naar een toestand *string* realiseren. Op dat moment is immers in een neutrale situatie het beginteken van een string waargenomen.
- Evenzo leidt een enkel aanhalingteken tot de toestand *char,* omdat het enkele aanhalingsteken het begin van een *char*-constante aangeeft. Deze situatie moet kunnen worden herkend, omdat een dubbel aanhalingsteken in een *char*-constante kan voorkomen.



Figuur 14.4: De string grabber: transitiediagram voor de toestand *start*.

- Een schuine streep kan het begin zijn van commentaar, maar het kan natuurlijk ook zo zijn dat eenvoudig een deel-operator is aangetroffen. Welke situatie is aangetroffen is op dit moment niet bekend, maar het zal duidelijk zijn dat de schuine streep iets bijzonders betekent. Het teken leidt dan ook tot de toestand *slash*.
- Alle overige tekens van de invoerverzameling leiden niet tot een toestandswijziging. Merk op dat de omgekeerde schuine streep in **C** bronteksten buiten strings, *char*-constanten en commentaar niet voorkomt en dus ook niet tot een toestandswijziging leidt.

De analyse van de toestand *start* heeft tot drie nieuwe toestanden geleid. Het huidige (incomplete) transitiediagram is gegeven in figuur 14.4.

Vervolgens wordt de invoerverzameling met de nieuwe toestanden vergeleken. In de toestand *string* leiden de volgende invoerelementen tot een andere toestand:

- Wanneer in een string opnieuw een dubbel aanhalingsteken wordt aangetroffen dan is de string beëindigd. De string grabber keert dan weer terug in de toestand *start*.
- Wanneer een omgekeerde schuine streep wordt aangetroffen in een string, dan is er sprake van een escape-sequence. Dat leidt tot iets bijzonders, omdat de sequentie \" niet als het einde van de string moet worden opgevat. De omgekeerde schuine streep leidt daarom tot de toestand *escape_in_string*.
- Alle overige tekens van de invoerverzameling leiden niet tot een toestandswijziging.

In de toestand *char* wordt op analoge wijze als in de toestand *string* op invoer gereageerd. Hier leidt de omgekeerde schuine streep tot de nieuwe toestand *escape_in_char*.

In de toestand *slash* leiden de volgende invoerelementen tot een andere toestand:

- Als een sterretje wordt gelezen, dan is de combinatie /* aangetroffen. Op dat moment is het standaard **C** commentaar begonnen. Een sterretje leidt in deze toestand dan ook tot de nieuwe toestand *stdcomment*.
- Als nog een schuine streep wordt gelezen, dan is de combinatie // aangetroffen. Dat is het begin van het commentaar tot het einde van de regel. In deze situatie wordt overgegaan naar de toestand *eolncomment*.
- Bij alle overige invoer is er sprake van een enkele schuine streep en wordt teruggekeerd naar de toestand *start*.

De toestanden die ten gevolge van de analyse van de toestand *start* zijn gecreëerd zijn nu weliswaar behandeld, maar tengevolge van die behandeling zijn er nieuwe toestanden toegevoegd. We zijn dus nog niet klaar, maar moeten de toestanden *escape_in_string, escape_in_char, stdcommment* en *eolncomment* nog analyseren.

In de beide toestanden *escape_in_string* en *escape_in_char* kan het volgende invoerteken worden gelezen, om vervolgens terug te keren tot respectievelijk de toestand *string* en de toestand *char*. In deze toestanden zijn *alle* mogelijke tekens dus verenigd in één verzameling.

In de toestand *stdcomment* worden alle tekens verwerkt, totdat een sterretje wordt aangetroffen. Zo'n sterretje kan het einde van het standaard commentaar aanduiden, maar dat is op dit moment nog niet bekend. Wel is duidelijk dat het sterretje iets bijzonders is en dus wordt in dat geval over gegaan naar de toestand *starcomment*.

In de toestand *eolncomment* moeten alle invoerelementen worden gelezen totdat het end-of-line teken \n is aangetroffen. In dat geval is het commentaar tot het einde van de regel verwerkt en keert het programma terug naar de toestand *start*.

Op dit moment is er nog slechts één nieuwe toestand, namelijk de toestand *starcomment*. De string grabber bevindt zich in deze toestand nadat een sterretje is waargenomen in de toestand *stdcomment*. In deze toestand leiden de volgende tekens tot een toestandswijziging:

- Wanneer een schuine streep wordt aangetroffen, dan is de combinatie */ opgetreden en is het standaard commentaar verwerkt. De string grabber keert dan terug naar de toestand *start*.
- Wanneer opnieuw een sterretje wordt aangetroffen is er nog steeds sprake van een sterretje in een standaard commentaar. De toestand *starcomment* blijft dan ongewijzigd.
- Bij alle overige invoer had het sterretje in het standaard commentaar geen speciale betekenis en is er nog steeds sprake van standaard commentaar. In deze situatie wordt weer teruggekeerd naar de toestand *stdcomment*.

Alle toestanden zijn nu verwerkt en het transitiediagram kan worden opgesteld.

Opgave 80.

Construeer het transitiediagram van de string grabber. Het diagram wordt analoog aan het diagram van de koffiezetter opgesteld. Een onderdeel van het diagram is al gegeven in figuur 14.4. Overgangen van de ene toestand naar de andere toestand op grond van 'overige' invoer kan worden aangegeven als een overgang op basis van de invoer ?. Wanneer bij de 'overige' invoer geen overgang naar een andere toestand optreedt, hoeft dat niet expliciet te worden aangegeven.

14.3.2 Toestanden en herkenningstekens

Aangezien in het transitiediagram van de string grabber negen toestanden zijn gedefinieerd, is nu bekend dat de transitiematrix negen rijen zal hebben, één rij voor elke toestand. Elke toestand heeft een naam gekregen die verderop zal worden gebruikt bij de initialisatie van de transitiematrix. Daartoe wordt een *enum* type gedefinieerd waarin de namen van de toestanden worden opgenomen. De bedoelde *enum* definitie is dan:

```
typedef enum
{
    start,
    string,
    character,
    slash,
    stringescape,
    charescape,
    stdcomment,
    eolncomment,
    starcomment,
} STATE_;
```

Terwijl de toestanden van de string grabber heel elegant met behulp van de waarden van de *STATE_ enum* kunnen worden aangeduid, ligt dat bij de elementen van de invoerverzameling iets lastiger. Het programma zal in principe alle tekens moeten kunnen herkennen om de gepaste overgang van de ene naar de andere toestand te kunnen maken. Anders dan bij de waarden van een *enum* zijn de ASCII-waarden van de tekens binnen de elementen van de invoerverzameling niet mooi oplopend. Om de kolom-index van de transitiematrix te kunnen bepalen op grond van het eerstvolgende teken van de brontekst, wordt daarom gebruik gemaakt van een hulpfunctie die ASCII-waarden afbeeldt op waarden van een tweede *enum* waarmee de elementen van de invoerverzameling worden aangeduid. De *enum* definitie waarmee de elementen van de invoerverzameling worden aangeduid is:

```
typedef enum
{
    double_quote,
    single_quote,
```

```
        slashchar,
        backslash,
        endline,
        star,
        other_char
    } INPUT_SET_;
```

Op grond van de waarden van de *STATE_ enum* enerzijds en de waarden van de *INPUT_SET_ enum* anderzijds, kan verderop een nieuwe toestand uit een oude toestand worden bepaald met behulp van een statement als:

newstate = transition[state][input_element];

waarbij *newstate* en *state* variabelen zijn van het type *STATE_*, en *input_element* een variabele is van het type *INPUT_SET_*.

14.3.3 Acties via een transitiematrix

Tot nu toe hebben we ons beperkt tot het beschrijven van de transitiematrix en de manier waarop de string grabber de invoer zal lezen. Op dit moment moet het al mogelijk zijn een programma te construeren waarmee de strings uit een **C** brontekst kunnen worden herkend. Een code-frame voor zo'n programma is gegeven in listing 77. De daarin genoemde functie *mapchar()* zorgt voor het bepalen van het juiste invoerelement gegeven het zojuist gelezen teken van de **C** brontekst.

```
// initialiseer de transitiematrix,
// definieer de nodige variabelen.

void main(int argc, char **argv)
{
    // open het invoerbestand

    // verwerk alle tekens
    while ((c = fgetc(stream)) != EOF)
    {
        // bepaal nieuwe toestand
        state = transition[state][mapchar(c)];
    }
}
```

Listing 77: Een code-frame voor het prototype van de string grabber.

De string grabber moet echter meer doen dan alleen van de ene toestand overgaan naar de andere toestand. De string grabber zou bijvoorbeeld bij het begin van een string kunnen vermelden dat het begin van een string is aangetroffen. Andere voor de hand liggende acties zijn het vermelden van de tekens van een string, het aangeven van het einde van de string en het bijhouden van het nummer van de regel die op dat moment wordt verwerkt.

Dergelijke acties moeten in speciale gevallen worden uitgevoerd. Het enige moment dat het begin van een string moet worden gesignaleerd is wanneer een string begint. Inspectie van het transitiediagram maakt duidelijk dat deze actie moet worden uitgevoerd op het moment dat de string

grabber van de toestant *start* overgaat naar de toestand *string*. Evenzo moet de string grabber in de toestand *string* alle waargenomen tekens vermelden zolang de string grabber in de toestand *string* of *escapestring* is. Meer algemeen geformuleerd hoort bij een bepaalde toestand *state* en een bepaald invoerelement *element* een zekere actie die moet worden uitvoerd, waarna de string grabber naar de toestand over kan gaan die (ook) wordt afgeleid uit de combinatie van *state* en *element*.

Kennelijk is de combinatie van huidige toestand en huidig invoerelement zowel van belang voor het bepalen van de actie die moet worden uitgevoerd, als voor het bepalen van de nieuwe toestand. De uit te voeren actie en de nieuwe toestand kunnen dan ook samen worden gevoegd in een *struct*. De elementen van de transitiematrix zijn dan geen waarden meer van de *STATE_ enum,* maar zijn *structs,* waarin een *STATE_* veld en een pointer naar een functie zijn gedefinieerd. De functie waarnaar gewezen wordt, definieert de actie die moet worden uitgevoerd bij toestand *state* en invoerelement *element*.

De vereiste *struct* definitie is:

```
typedef struct
{
    STATE_
        newstate;
    void
        (*action)();
} TRANS_;
```

14.3.4 Implementatie van de string grabber

De implementie van de string grabber begint wederom bij de functie *main()*. De functie probeert het bestand, waarvan de naam als argument aan het programma is meegegeven, te openen. Als dit slaagt, dan wordt functie *process()* aangeroepen om het bestand te verwerken (zie listing 78).

```
1           #include <stdio.h>
2
3           void main (int argc, char **argv)
4           {
5               FILE
6                   *infile;
7
8               if (argc < 2)
9                   puts ("Usage: STRGRAB infile");
10              else if (! (infile = fopen (argv[1], "r")) )
11                  puts ("Cannot open input file.");
12              else
13                  process (infile);
14          }
15
```

Listing 78: De functie *main()* van de string grabber.

De functie *process()* (listing 79) vormt het hart van het programma. Deze functie leest tekens uit het aangesloten bestand en zorgt ervoor dat de juiste actie bij de gegeven combinatie van huidige toestand en huidig invoerelement wordt uitgevoerd, waarna over wordt gegaan naar een nieuwe toestand. Voor de overgangen gebruikt *process()* de transitiematrix *transition,* die verderop (pagina 309) wordt gedefinieerd. De functie gebruikt een globale variabele *c* om het net gelezen teken in op te slaan. Op pagina 307 zal een hulpfunctie worden gedefinieerd die het net gelezen teken zal afdrukken op het beeldscherm.

```
1           int
2               c;
3
4           void process (FILE *stream)
5           {
6               int
7                   element;
8               STATE_
9                   state = start;
10
11              while ((c = fgetc(stream)) != EOF)
12              {
13                  element = mapchar(c);           // bepaal input element
14
15                  // voer de vereiste actie uit
16                  transition[state][element].action();
17
18                  // bepaal nieuwe toestand
19                  state = transition[state][element].newstate;
20              }
21          }
```

Listing 79: De functie *process()* van de string grabber.

De enige functie die hier direct wordt aangeroepen, is de functie *mapchar()* waarmee de invoertekens worden afgebeeld op de waarden van de *INPUT_SET_ enum*. Bij de implementatie wordt gebruik gemaakt van het gegeven dat de waarden van een *enum* beginnen bij 0 en oplopen tot het aantal elementen van de *enum* minus 1. In plaats van een *switch* te gebruiken om bij elk invoerteken het juiste invoerelement te plaatsen, wordt het aangetroffen teken gezocht in een string waarin zich de *ascii* tekens al op een plaats bevinden waarvan de index correspondeert met de waarde van het desbetreffende invoerelement in de *INPUT_SET_ enum*. Zo staat het teken " op index positie 0 (corresponderend met de *enum* waarde *double_quote)* en het teken `\n` op index positie 4 (corresponderend met de *enum* waarde *endline)*.

De standaardfunctie *strchr()* wordt gebruikt om de conversie naar een waarde van de *INPUT_SET_ enum* uit te voeren voor alle expliciet genoemde tekens. De functie verwacht als eerste argument een pointer naar

een string en als tweede argument een *char* waarde die moet worden opgezocht in de string. Wanneer het te zoeken teken zich in de string bevindt, wordt een pointer naar de plaats waar het teken voor het eerst in de string voorkomt geretourneerd. Wanneer de functie *NULL* retourneert, werd het huidige teken niet aangetroffen in de string en wordt categorie *other_char* geretourneerd. In alle andere gevallen wordt het verschil tussen de gevonden pointerwaarde en het adres van het begin van de string als *INPUT_STATE_* waarde geretourneerd. De functie *mapchar()* is gegeven in listing 80. In de standaardfunctie *strchr()* wordt een lineair zoekproces gebruikt: de elementen worden stuk voor stuk afgelopen. In situaties waarbij het aantal elementen dat moet worden afgezocht klein is, kan goed gebruik worden gemaakt van lineaire zoekprocedures. 'Slimmere' procedures (zoals het binair zoeken) zijn dan altijd trager en omvangrijker vanwege de extra code die nodig is om het proces 'slim' te maken.

```
1           #include <string.h>
2
3           char
4               specials[] =
5               {
6                   '"',                            // "-teken
7                   '\'',                       // '-teken
8                   '/',                            // /-teken
9                   '\\',                           // \-teken
10                  '\n',                           // regeleinde
11                  '*',                            // asterisk
12                  0                               // stringeinde
13              };
14
15          INPUT_SET_ mapchar(int c)
16          {
17              char
18                  *cp;
19
20              cp = strchr(specials, c);           // zoek: c
21              return
22              (
23                  cp ?                            // retourneer juiste
24                      cp - specials               // INPUT_SET_ waarde
25                  :
26                      other_char
27              );
28          }
```

Listing 80: De functie *mapchar()* van de string grabber.

De transitiematrix is tot nu toe nog niet gedefinieerd. Voordat dat zal gebeuren, worden de functies bepaald waarmee acties worden gedefinieerd. Welke akties zijn er zoal ?

- In de string grabber is een veel voorkomende actie dat er 'niets' moet gebeuren. Zo hoeft er niets te gebeuren wanneer de string

grabber van de toestand *start* naar de toestand *char* gaat. Voor deze 'actie' zal de functie *do_nothing()* zorgen.

- Een andere actie die moet worden uitgevoerd, is het tellen van de regels. De functie *update_linenr()* zorgt ervoor dat de waarde van de globale variabele *linenr* wordt opgehoogd.
- Wanneer het begin van een string wordt ontdekt, moet dat worden gemeld. Dat gebeurt door de functie *open_string()*.
- Zolang er sprake is van een string moeten de tekens van de string worden afgedrukt. Dat gebeurt door de functie *write_char()*.
- Aan het einde van een string moet de huidige regel worden afgesloten. Dat gebeurt door de functie *close_string()*.

De hulpfuncties zijn gedefinieerd in listing 81. Merk op dat elke hulpfunctie een uitermate eenvoudige handeling verricht. De hulpfuncties worden vanuit *process()* indirect via het *action* veld van een *TRANS_* struct aangeroepen.

```
1           int
2               linenr = 1;
3
4           void do_nothing()
5           {
6           }
7
8           void update_linenr()
9           {
10              linenr++;
11          }
12
13          void open_string()
14          {
15              printf("Er staat een string op regel %d:\n"
16                     "\t\"", linenr);
17          }
18
19          void write_char()
20          {
21              putchar(c);
22          }
23
24          void close_string()
25          {
26              puts("\"");
27          }
```

Listing 81: Hulpfuncties van de string grabber.

De definitie van de transitiematrix vormt de laatste stap in de ontwikkeling van de string grabber. Bij de definitie van de transitiematrix worden de *TRANS_* structs behorend bij elke combinatie van *STATE_* waarde en *INPUT_SET_* waarde op een waarde geïnitialiseerd. Elk *STATE_* veld

krijgt de waarde van de toestand waarin de string grabber komt gegeven de huidige toestand en het huidige invoerelement. Elk *action* veld krijgt als waarde het adres van de hulpfunctie die moet worden aangeroepen voor de overgang naar de volgende toestand. De definitie van de transitiematrix is gegeven in listing 82.

```
1         STATE_
2             transition[][] =
3             {
4                 {                                // start toestand
5                     {string,    open_string},  // input: "-teken
6                     {character, do_nothing},   // input: '-teken
7                     {slash,     do_nothing},   // input: /-teken
8                     {start,     do_nothing},   // input: \-teken
9                     {start,     update_linenr},// input: regeleinde
10                    {start,     do_nothing},   // input: asterisk
11                    {start,     do_nothing}    // input: other_char
12                },
13                {                                // string toestand
14                    {start,        close_string},// input: "-teken
15                    {string,       write_char},// input: '-teken
16                    {string,       write_char},// input: /-teken
17                    {stringescape, write_char},// input: \-teken
18                    {string,       update_linenr},// input: regeleinde
19                    {string,       write_char},// input: asterisk
20                    {string,       write_char}// input: other_char
21                },
22                {                                // character toestand
23                    {character,  do_nothing}, // input: "-teken
24                    {start,      do_nothing}, // input: '-teken
25                    {character,  do_nothing}, // input: /-teken
26                    {charescape, do_nothing}, // input: \-teken
27                    {character,  update_linenr},// input: regeleinde
28                    {character,  do_nothing}, // input: asterisk
29                    {character,  do_nothing}  // input: other_char
30                },
31                {                                // slash toestand
32                    {start,       do_nothing},// input: "-teken
33                    {start,       do_nothing},// input: '-teken
34                    {eolncomment, do_nothing},// input: /-teken
35                    {start,       do_nothing},// input: \-teken
36                    {start,       update_linenr},// input: regeleinde
37                    {stdcomment,  do_nothing},// input: asterisk
38                    {start,       do_nothing} // input: other_char
39                },
40                {                                // stringescape toestand
41                    {string,  write_char},    // input: "-teken
42                    {string,  write_char},    // input: '-teken
43                    {string,  write_char},    // input: /-teken
44                    {string,  write_char},    // input: \-teken
45                    {string,  update_linenr}, // input: regeleinde
46                    {string,  write_char},    // input: asterisk
47                    {string,  write_char}     // input: other_char
48                },
```

```
49              {                                   // charescape toestand
50                  {character,  do_nothing}, // input: ''-teken
51                  {character,  do_nothing}, // input: '-teken
52                  {character,  do_nothing}, // input: /-teken
53                  {character,  do_nothing}, // input: \-teken
54                  {character,  update_linenr},// input: regeleinde
55                  {character,  do_nothing}, // input: asterisk
56                  {character,  do_nothing}  // input: other_char
57              },
58              {                                   // stdcomment toestand
59                  {stdcomment,  do_nothing},// input: ''-teken
60                  {stdcomment,  do_nothing},// input: '-teken
61                  {stdcomment,  do_nothing},// input: /-teken
62                  {stdcomment,  do_nothing},// input: \-teken
63                  {stdcomment,  update_linenr},// input: regeleinde
64                  {asteriskcomment, do_nothing},// input: asterisk
65                  {stdcomment,  do_nothing} // input: other_char
66              },
67              {                                   // eolncomment toestand
68                  {eolncomment,  do_nothing},// input: ''-teken
69                  {eolncomment,  do_nothing},// input: '-teken
70                  {eolncomment,  do_nothing},// input: /-teken
71                  {eolncomment,  do_nothing},// input: \-teken
72                  {start,        update_linenr},// input: regeleinde
73                  {eolncomment,  do_nothing},// input: asterisk
74                  {eolncomment,  do_nothing}// input: other_char
75              },
76              {                                   // asteriskcomment toestand
77                  {stdcomment,  do_nothing},// input: ''-teken
78                  {stdcomment,  do_nothing},// input: '-teken
79                  {start,       do_nothing},// input: /-teken
80                  {stdcomment,  do_nothing},// input: \-teken
81                  {stdcomment,  update_linenr},// input: regeleinde
82                  {asteriskcomment, do_nothing},// input: asterisk
83                  {stdcomment,  do_nothing} // input: other_char
84              }
85          };
```

Listing 82: De transitiematrix van de string grabber.

Het wijzigen van de transitiematrix leidt tot de wijziging van het programma. Hiervan wordt gebruik gemaakt bij de volgende opgave, waarbij het programma *strgrab* wordt uitgebreid met een (zeer) eenvoudige manier om fouten te herkennen. De foutenherkenning is zeer eenvoudig in de zin dat bij een geconstateerde fout de plaats waar de fout wordt herkend, wordt afgedrukt, waarna het programma beëindigt. Normaal is dat niet de gewenste gang van zaken bij vertalers: De meeste vertalers herkennen fouten, vermelden ze en controleren vervolgens de rest van een brontekst op verdere fouten. Deze procedure is hier niet gevolgd omdat van de veronderstelling wordt uitgegaan dat de aangeboden bronteksten door een **C** vertaler foutloos kunnen worden vertaald. Wanneer de string grabber fouten constateert, dient de gebruiker de brontekst alsnog eerst

syntactisch door de **C** vertaler te laten controleren. Op zich is een controle of een brontekst syntactisch juist is eenvoudig uit te voeren. De vertaler optie /**Zs** zorgt ervoor dat de brontekst alleen op syntactische kenmerken wordt gecontroleerd. De vertaler genereert geen code bij gebruik van deze optie.

De hier beschreven string grabber is een programma dat voornamelijk een illustratief doel heeft. Er kunnen zich situaties voordoen waarin de string grabber niet 'mooi' werkt. Hierbij kan aan het volgende worden gedacht:

- Wanneer een *#include* opdracht in een brontekst is opgenomen waarbij een lokaal header bestand wordt aangeduid, dan staat de naam van het bestand tussen dubbele aanhalingstekens. Toch is dit geen 'echte' string (dat wil zeggen, een string die in het uiteindelijke programma voorkomt). De string grabber zal dergelijke oneigenlijke strings echter vrolijk als string herkennen.
- Een tweede probleemsituatie treedt op wanneer een macro gedefinieerd wordt waarin een string wordt gebruikt. Bijvoorbeeld:

  #define a_string "a string"

  De preprocessor zal overal *a_string* in de brontekst vervangen door de string *"a string"*. De string grabber zal deze strings echter niet herkennen, maar wel de string in de *#define* opdracht.

Deze problemen zijn gerelateerd aan het gebruik van de preprocessor. Voor wie deze problemen een doorn in het oog zijn, is er de /**E** vertaleroptie, waarmee een brontekst wordt gegenereerd nadat de preprocessor zijn werk heeft gedaan. De *#define* en *#include* opdrachten zijn dan door de vertaler verwerkt en komen dan niet meer voor in de gewijzigde brontekst die door de vertaler is gegenereerd.

Een ander weinig elegant kenmerk van de string grabber is dat bij stringconcatenatie meer dan één string wordt herkend. Het is weliswaar te verdedigen dat er in de brontekst meer dan één string staat wanneer er stringconcatenatie optreedt, maar er is slechts sprake van één *begin*punt van deze strings. Om bij stringconcatenatie één string te herkennen, moet het transitiediagram worden uitgebreid. Na een dubbel aanhalingsteken in een string moet de string grabber dan niet direct terugkeren naar de toestand *start,* maar moet worden overgegaan naar een toestand, waarin alle spaties, tabulatietekens, nieuwe regels en commentaar kunnen worden herkend. Wanneer in deze extra toestand een dubbel aanhalingsteken voorkomt, moet worden teruggekeerd naar de toestand *string*. In alle andere gevallen kan alsnog naar de toestand *start* worden teruggekeerd. In de hier ontwikkelde versie van de string grabber is deze verfijning niet uitgevoerd, omdat het transitiediagram dan, gegeven het onderwerp van dit hoofdstuk, nodeloos uitgebreid zou worden.

Opgave 81.

Construeer de string grabber uit de informatie gegeven in deherkend listings 78 tot en met 82. Noem het bestand waarin de brontekst van de functie *main()* staat *strgrab.c* zodat het programma *strgrab.exe* gaat heten. Zorg ervoor dat alle *#include* opdrachten aan het begin van de brontekst komen te staan en definieer functies voordat ze gebruikt worden. Dat betekent onder andere dat de transitiematrix na de hulpfuncties wordt gedefinieerd.

Opgave 82.

Controleer de werking van het programma door de DOS opdracht
strgrab strgrab.c
te geven.

Opgave 83.

Construeer een functie *error()* die de melding *"Syntactische fout in de brontekst op regel ..."* afdrukt, waarbij het regelnummer waarbij de fout werd geconstateerd, wordt afgedrukt op de plaats van de puntjes. Nadat de melding is afgedrukt moet de functie *error()* het programma beëindigen. De functie moet in de volgende situaties worden aangeroepen:

- Wanneer een nieuwe regel wordt aangetroffen in de toestand *string*.
- Wanneer een nieuwe regel wordt aangetroffen in de toestand *char*.

Wijzig de transitiematrix zodanig dat de functie *error()* door de string grabber op de aangegeven momenten zal worden aangeroepen.

Hoofdstuk 15

Recursieve processen

15.1 Inleiding

De firma Droste brengt cacao op de markt met op de verpakking een verpleegster die een presenteerblaadje vasthoudt. Op het presenteerblaadje staat een kopje chocolademelk en een pakje Droste cacao. Op de verpakking van het pakje cacao op het presenteerblaadje staat opnieuw een verpleegster die een presenteerblaadje vasthoudt. Daarop staat weer een pakje Droste cacao waarop wederom een verpleegster staat die ... (enzovoort). De verpakking van de Droste cacao staat bekend als het 'Droste effect' en illustreert op prachtige wijze een recursief proces. Een verdere illustratie van een recursief effect is te vinden op pagina 314.

Recursie treedt op wanneer een proces zichzelf activeert. In **C** worden recursieve processen geïmplementeerd als recursieve functies: functies die zichzelf aanroepen. Een zeer eenvoudig voorbeeld van een functie die zichzelf aanroept (een recursieve functie) is hieronder gegeven:

```
void f()
{
    f();
}
```

Bij de aanroep van functie *f()* gebeurt het volgende:

- Het statement in functie *f()* is een aanroep van functie *f()* zelf. Functie *f()* wordt aangeroepen voordat de oorspronkelijk aangeroepen functie *f()* is beëindigd. Deze extra activering van functie *f()* is de eerste recursieve activering.
- In de eerste recursieve activering van functie *f()* moet als eerste actie functie *f()* worden geactiveerd. Dat leidt tot de tweede recursieve activering.
- In de tweede recursieve activering wordt opnieuw de functie *f()* geactiveerd.
- Enzovoort enzovoort.

Merk op dat bij elke recursieve activering de aanroepende functie nog niet is beëindigd. Om de eerste aanroep van functie *f()* te beëindigen zou immers de recursieve aanroep moeten zijn voltooid. Maar voordat de eerste recursieve aanroep is voltooid, moet de tweede recursieve aanroep zijn voltooid enzovoort. In theorie zal de oorspronkelijke aanroep van functie *f()* nooit worden voltooid, omdat telkens een nieuwe recursie begint, waardoor geen enkele (recursieve) aanroep van functie *f()* wordt voltooid. In de praktijk zal het zover niet komen. Wanneer een functie wordt aangeroepen, wordt op de stack de plaats in de code van het statement na de functie-aanroep onthouden (het zogenaamde return-adres). Wanneer een functie is voltooid, wordt het return-adres weer van de stack verwijderd en wordt de code verwerkt vanaf de plaats die door het return-adres wordt aangegeven. Bij elke recursieve aanroep van een functie wordt ook

een return-adres op de stack gezet, maar omdat bij functie *f()* geen enkele aanroep wordt voltooid, worden de return-adressen nooit van de stack verwijderd. Het gevolg is dat de stack bij een oneindige recursie vol loopt en het programma wordt afgebroken met een 'stack overflow' melding. Om te voorkomen dat de stack vol loopt zal een recursieve aanroep van een functie dus altijd in een conditie moeten worden geplaatst, die op een gegeven ogenblik de recursie doorbreekt.

Het enig belangrijke onderscheid tussen een recursieve functie en een niet-recursieve functie is het feit dat een recursieve functie zichzelf aanroept, terwijl een niet-recursieve functie zichzelf niet aanroept. Alle overige eigenschappen van functies gelden ook voor recursieve functies. Dat betekent dat recursieve functies lokale variabelen en parameters kunnen hebben en globale variabelen kunnen manipuleren. De lokale variabelen en parameters van een recursieve functie zijn lokaal beschikbaar *binnen* een activering van een recursieve functie en niet *tussen* recursieve activeringen. Wanneer een recursieve functie *f()* een lokale variabele *x* heeft, dan is de variabele *x* bij elke aanroep van *f()* een unieke variabele. Als voorbeeld geven we in listing 83 een functie *f()* met een parameter *y*. Bij de aanroep van de functie *f()* krijgt *y* de waarde 5. Dan wordt *f()* recursief aangeroepen en in de recursie krijgt *y* de waarde 10. Deze variabele *y* is fysiek een andere variabele dan de variabele *y* die ontstond bij de eerste aanroep van *f()*. Na afloop van de recursieve aanroep van de functie *f()* is ook de levensduur van de recursief gecreëerde variabele *y* (met waarde 10) beëindigd. De oorspronkelijke functie *f()* wordt dan verder afgewerkt, waarbij de oorspronkelijke variabele *y* (met waarde 5) weer wordt gebruikt.

```
1       #include <stdio.h>
2
3       void f(int y)
4       {
5           printf("In functie f(): 'y' heeft de waarde %d\n", y);
6
7           if (y == 5)
8           {
9               puts("Recursieve aanroep van f() ...");
10              f(y + y);
11              puts("Recursieve aanroep van f() voltooid");
12          }
13          printf("In functie f(): 'y' heeft de waarde %d\n", y);
14      }
15
16      void main()
17      {
18          f(5);
19      }
20
21
```

Listing 83: Een recursieve functie met parameters.

Functies kunnen ook indirect recursief zijn. Deze situatie doet zich voor wanneer twee functies elkaar aanroepen. Dat kan direct gebeuren, zoals in listing 84 is geïllustreerd, maar er kunnen ook een aantal extra functies zijn aangeroepen voordat de recursie zichtbaar is. Zo zou de functie *g()* in listing 84 een functie *h()* kunnen aanroepen, die pas de functie *f()* aanroept. Er is dan nog steeds sprake van indirecte recursie omdat *uiteindelijk* de functie *f()* ook weer door *g()* wordt aangeroepen. Het essentiële kenmerk van indirecte recursie is dat er bij deze vorm van recursie altijd een functie bestaat die moet worden aangeroepen vóórdat de functie is gedefinieerd. In listing 84 is dat dus functie *f()*. Dat betekent dat er bij indirect recursieve functies een prototype van die functie moet zijn gegeven voordat de functie het eerst wordt aangeroepen.

```
1       void f();                       // prototype van f()
2
3       void g()                        // definitie van g()
4       {
5           ...
6           f();
7           ...
8       }
9
10      void f()                        // definitie van f()
11      {
12          ...
13          g()
14          ...
15      }
```

Listing 84: Indirect recursieve functies.

Recursieve algoritmes worden met name gebruikt wanneer het onderwerp van het algoritme in recursieve termen kan worden beschreven. Dat kan bijvoorbeeld het geval zijn bij een recursieve definitie van een wiskundig begrip, zoals de faculteit, maar er zijn ook andere toepassingen:

- Recursieve datastructuren lenen zich goed voor verwerking door recursieve functies. Een datastructuur is recursief gedefinieerd als in de definitie ervan de datastructuur zelf voorkomt. Een voorbeeld is de definitie van het statement in **C**. Een statement kan onder andere een compound statement zijn. De recursiviteit treedt op doordat in de definitie van het compound statement het statement zelf weer opdoemt. Een voorbeeld van een programma dat recursieve datastructuren kan verwerken is het programma *scalc* uit hoofdstuk 16.
- Eenvoudige problemen kunnen vaak goed met behulp van recursieve algoritmes worden opgelost. Het probleem kan weliswaar moeilijk oplosbaar zijn, maar de manier om het probleem op te lossen is hier eenvoudig. Het prototype van dergelijke problemen

bestaat uit het vinden van een route in een labyrint. Op een splitsing aangekomen wordt een keuze gemaakt die uiteindelijk fout kan blijken te zijn. Op dat moment keren we op onze schreden terug (backtracking) om een alternatieve keuze te maken. Wanneer er geen alternatieve keuze meer is, moet een vorige keuze worden herzien enzovoort. Een voorbeeld van een dergelijk zoekproces is in dit hoofdstuk gegeven bij de behandeling van de paardesprongpuzzel (in 15.4)[1].

- Soms kunnen problemen worden opgeplitst in een aantal gelijksoortige deelproblemen. De oplossing van het probleem bestaat dan uit het recursief oplossen van de gelijksoortige deelproblemen. Aan het einde van dit hoofdstuk wordt de functie *quicksort()* geïntrodu ceerd waarmee een array wordt gesorteerd door steeds kleinere subarrays te sorteren. Het sorteren van de subarrays gebeurt recursief, doordat de functie *quicksort()* daarvoor zichzelf activeert.

Als vuistregel kan gelden dat recursieve functies kunnen worden ontwikkeld voor problemen waarvan de oplossing bestaat uit het herhaaldelijk oplossen van het probleem op kleinere schaal[2].

15.2 Recursieve faculteitberekening

Een voorbeeld van een recursief proces waarbij een functie zichzelf activeert is de berekening van de faculteit van een positief getal. In de definitie van het begrip faculteit wordt zowel de recursie als de conditie aangetroffen. De faculteit van een positief getal is als volgt gedefinieerd:

1. De faculteit van 1 is 1.
2. De faculteit van een ander positief getal n is n keer de faculteit van het getal $n - 1$.

Deel 1 van de definitie bevat de conditie waarbij de recursie stopt, deel 2 van de definitie bevat de recursie: om het begrip 'faculteit' te definiëren wordt het begrip faculteit zelf gebruikt. De faculteitsberekening wordt aangeduid met het symbool $!$ De waarde $3!$ (drie faculteit) is, overeenkomstig de hier gehanteerde definitie, dus:

$$3 * 2!$$

Vervolgens is $2!$ gelijk aan:

1. Sommige programmeertalen zijn gespecialiseerd in het gebruik van het backtracking algoritme. Dergelijke talen zijn voornamelijk speciale stukken gereedschap met een speciale functie. Voor het programmeren in algemene zin zijn ze minder geschikt. Voorbeelden van dergelijke talen zijn **lisp** en **prolog.**
2. Veel recursieve functies kennen ook een iteratieve variant. Het is vaak een kwestie van smaak of de iteratieve of recursieve versie van een algoritme wordt geprefereerd. Recursieve functies zijn vaak eleganter en compacter dan iteratieve functies, maar iteratieve functies zijn in het algemeen iets sneller.

$$2 * 1!$$

terwijl *1!* per definitie gelijk is aan 1, waardoor de recursie wordt doorbroken. De implementatie van de faculteitsfunctie in **C** (zie listing 85) volgt precies de definitie van het begrip faculteit, bevat op regel 3 een conditie om te testen of de waarde van de parameter 1 is en gebruikt op regel 6 een recursieve aanroep van de functie voor het geval de waarde van de parameter ongelijk 1 is.

```
1     int recursive_fac (int nr)
2     {
3         if (nr == 1)
4             return (nr);
5
6         return (nr * recursive_fac(nr - 1));
7     }
```

Listing 85: De recursieve faculteitsfunctie in **C**.

Hoewel het faculteitsbegrip zich goed leent voor een illustratie van een recursieve functie, moet opgemerkt worden dat de recursieve implementatie van de faculteitsfunctie geen efficiënte manier is om de faculteit te berekenen. Omdat voor elke stap opnieuw een functie moet worden aangeroepen, is de recursieve implementatie traag vergeleken met de iteratieve implementatie van de faculteitsfunctie zoals gegeven in listing 86.

```
1         int interative_fac (register int nr)
2         {
3             register int
4                 result = nr;
5
6             while (--nr > 1)
7                 result *= nr;
8
9             return (result);
10        }
```

Listing 86: De iteratieve faculteitsfunctie in **C**.

Wanneer recursie eenvoudig vermeden kan worden, is het verstandig dat ook te doen, omdat iteratieve versies van recursieve functies sneller zijn en een minder zware belasting leggen op de stack. Sommige processen zijn echter 'van nature' recursief, doordat de recursie middenin een proces optreedt of omdat er meer dan één recursief moment in het proces kan worden aangewezen. Dergelijke situaties doen zich onder andere voor bij hiërarchische structuren die recursief zijn gedefinieerd. Als voorbeeld kan worden gedacht aan compound statements in **C** bronteksten. Een compound statement is zelf een statement en bevat zelf statements. De statements in een compound statement kunnen dus ook weer compound

statements zijn, waardoor het compound statement een voorbeeld is van een recursieve structuur. Een recursieve implementatie van een proces dat met behulp van zichzelf is gedefinieerd, is niet altijd noodzakelijk, maar wel vaak voor de hand liggend en elegant. In het resterend deel van dit hoofdstuk zullen een paar toepassingen van recursieve functies tot programma's worden uitgewerkt.

15.3 Verzamelingen

In dit deel van het hoofdstuk zal een programma *(subset)* worden geconstrueerd, waarmee alle deelverzamelingen van een gegeven beginverzameling worden gegenereerd. In het programma *subset* wordt de beginverzameling gedefinieerd als de verzameling van alle argumenten die aan het programma zijn meegegeven. Dat zijn dus de strings in *argv[1]* tot en met *argv[argc - 1]*. Wanneer het programma wordt aangeroepen als:

subset een twee drie

dan moet de volgende uitvoer worden gegenereerd:

```
Set: een twee drie
Set: een twee
Set: een drie
Set: een
Set: twee drie
Set: twee
Set: drie
Set:
```

Bij deze uitvoer bevat de eerste verzameling alle elementen en is de laatste verzameling de lege verzameling.

De constructie van deelverzamelingen leent zich goed voor de illustratie van een recursief proces. Wanneer de elementen van een verzameling worden genummerd van element 1 tot element n dan geldt voor elk element i $(1 \leq i \leq n)$ dat het element wèl of niet in een deelverzameling voorkomt. Gegeven dat de aan- of afwezigheid van een element i is bepaald, dan is een deelverzameling bepaald zodra de aan- of afwezigheid van alle volgende elementen is bepaald. Wanneer de volledige verzameling slechts één element bevat, dan zijn er twee deelverzamelingen: de eerste deelverzameling bevat het eerste element, de tweede deelverzameling niet en is dus de *lege* verzameling. Wanneer de volledige verzameling uit twee elementen bestaat verloopt het proces als volgt:

- Het eerste element wordt in de deelverzameling opgenomen. Er zijn echter nog meer elementen waarover een beslissing moet worden genomen. De deelverzamelingen die bij die elementen horen moeten alle worden bepaald, gegeven de aanwezigheid van het eerste element. Dus:

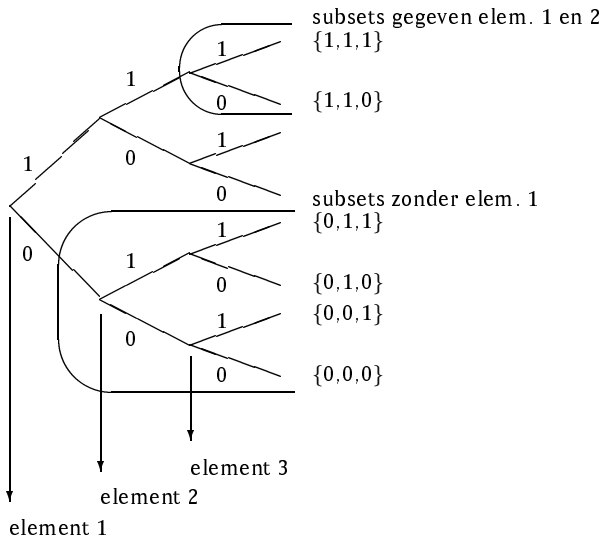
- Het tweede element wordt in de deelverzameling opgenomen. In dit voorbeeld zijn er slechts twee elementen, zodat we op dit moment de situatie hebben bereikt die lijkt op de situatie waarbij er slechts één element in de verzameling was. De geconstrueerde verzameling kan nu worden vermeld, bestaande uit de elementen 1 en 2.
- Het tweede element wordt ook een keer *niet* in de deelverzameling opgenomen. Dat leidt tot de deelverzameling met element 1.

• Vervolgens wordt het eerste element uit alle volgende deelverzamelingen verwijderd. Er moet nog een beslissing worden genomen over het volgende element. De deelverzamelingen die bij dat element horen moeten alle worden bepaald, gegeven de afwezigheid van het eerste element. Dus:
  - Het tweede element wordt in de deelverzameling opgenomen. De geconstrueerde verzameling kan nu worden vermeld, slechts bestaande uit element 2.
  - Het tweede element wordt ook een keer *niet* in de deelverzameling opgenomen. Dat leidt tot de lege verzameling.

Omdat elk element één keer wèl en één keer niet wordt opgenomen en omdat vervolgens eenzelfde beslissing moet worden genomen over alle volgende elementen, kan een overzicht van alle mogelijke deelverzamelingen ook heel goed als een boomstructuur (zie ook hoofdstuk 17) worden weergegeven. In figuur 15.1 is zo'n boom voor alle deelverzamelingen van een verzameling van drie elementen weergegeven. Op elk knooppunt van deze boom wordt een beslissing genomen over één element. De bovenste helft van de boom leidt daarom tot alle deelverzamelingen waarin element 1 is opgenomen, de onderste helft tot alle deelverzamelingen waarin element 1 niet is opgenomen. Zodra een belissing is genomen, worden alle deelverzamelingen voor de (eventuele) volgende elementen bepaald. In figuur 15.1 is dat weergegeven door middel van de halve ovaaltjes. Bij het bovenste ovaaltje moeten nog beslissingen worden genomen over element 3. Bij het onderste ovaaltje is slechts over element 1 een beslissing genomen. De elementen 2 en 3 kunnen nog al dan niet worden opgenomen.

Het proces om deelverzamelingen te construeren kan als volgt in recursieve termen worden gegoten:

1. Wanneer er geen elementen meer zijn waarover een beslissing moet worden genomen
   2. Vermeld dan de huidige deelverzameling.
   3. Beëindig het huidige proces.
4. Neem het eerstvolgende element *i* op in de huidige deelverzameling.
5. Voer het hier beschreven proces uit bij het volgende element.
6. Verwijder element *i* uit de huidige deelverzameling.



Figuur 15.1: Deelverzamelingen van drie elementen.

7. Herhaal punt 5.

De recursiviteit treedt op in de punten 5 en 7 van het algoritme, waarin het hele proces nogmaals wordt uitgevoerd wanneer nog niet over alle elementen een beslissing is genomen.

15.3.1 Implementatie

De implementatie van het programma *subset* is niet moeilijk en volgt letterlijk de recursieve formulering van het proces. De enige beslissing die nog moet worden genomen is hoe onthouden wordt of een element al dan niet in de verzameling is opgenomen. Het eenvoudigst kan daarvoor een array van *chars* worden gebruikt, waarbij de index in het array correspondeert met de index van een element. Wanneer het element in de verzameling is opgenomen, is het array-element 1, anders 0. Omdat van tevoren niet bekend is hoeveel elementen in de verzameling kunnen worden opgenomen, zou het array aan het begin van het programma op de juiste grootte kunnen worden gealloceerd. In het programma *subset* is dat niet gedaan, maar heeft het array een vaste omvang van 100 elementen gekregen. Dat lijkt voldoende voor alle praktische situaties[3].

3. Een verzameling van 100 elementen kent 2^{100} deelverzamelingen. Wanneer het volledig afwerken (inclusief afdrukken op het beeldscherm) van een deelverzameling 1 microseconde

De implementatie is gegeven in listing 87. Het eigenlijke werk wordt verricht door de functie *subset()* die vanuit *main()* wordt aangeroepen. De argumenten *argv* en *argc* worden gekopieerd in globale variabelen om *subset()* in de gelegenheid te stellen van hun waarde gebruik te maken.

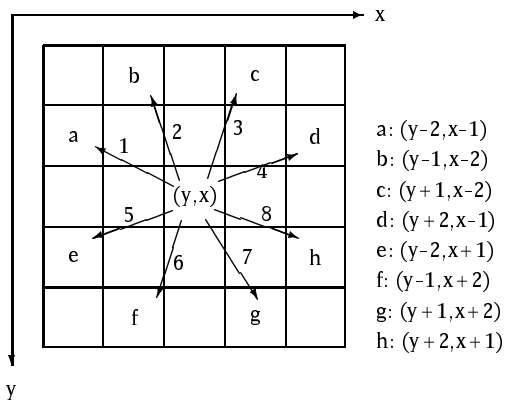
```
1           #include <stdio.h>
2
3           int
4               gargc;
5           char
6               set[100],
7               **gargv;
8
9           void subset(int index)
10          {
11              if (index == gargc)                 // Tekst: 1.
12              {
13                  printf("Subset:" );             // Tekst: 2.
14                  for (index = 1; index < gargc; index++)
15                      if (set[index])
16                          printf("%s" , gargv[index]);
17                  putchar('\n');
18                  return;                         // Tekst: 3.
19              }
20
21              set[index] = 1;                     // Tekst: 4.
22              subset(index + 1);                  // Tekst: 5.
23
24              set[index] = 0;                     // Tekst: 6.
25              subset(index + 1);                  // Tekst: 7.
26          }
27
28          void main(int argc, char **argv)
29          {
30              gargc = argc;
31              gargv = argv;
32
33              subset(1);
34          }
35
```

Listing 87: Deelverzamelingen: *subset.c*.

15.4 De paardesprongpuzzel

Het tweede recursieve programma dat we zullen construeren heeft te maken met de sprongen die paarden in het schaakspel kunnen uitvoeren. Het

(10^{-6} sec.) zou duren, dan zijn er ruim 40.000.000 miljard jaren nodig voordat het programma is beëindigd . . . .



Figuur 15.2: De paardesprong op een 5 x 5 bord.

probleem dat door het programma wordt opgelost, is eenvoudig te formuleren: Gegeven een bord met velden (bijvoorbeeld een schaakbord (8 x 8 velden), een dambord (10 x 10 velden), of het hieronder gebruikte bord van 5 x 5 velden); vind een zodanige serie paardesprongen dat alle velden van het bord precies één keer worden bezocht door het paard.

Voordat het programma wordt geconstrueerd geven we eerst de kenmerken van de paardesprong uit het schaakspel. Bij een paardesprong wordt het paard over één veld verticaal en twee velden horizontaal verplaatst, of wordt het paard over twee velden verticaal en één horizontaal verplaatst. Wanneer het bord wordt gerepresenteerd als een matrix van *n* rijen en *n* kolommen en een positie van het paard op het bord als het coördinatenpaar *[y][x]* [4], waarbij *[y]* de rij-coördinaat is en *[x]* de kolomcoördinaat, dan kan een paard dat zich op positie *[2][2]* van een 5 x 5 bord bevindt acht sprongen uitvoeren. Deze sprongen zijn weergegeven in figuur 15.2.

Het te construeren programma zal het probleem moeten oplossen hoe het paard zodanig over het bord moet springen dat alle velden precies één keer worden bereikt. Wanneer de puzzel door het programma is opgelost, moet het programma de route afdrukken, die het paard over het bord aflegt. Deze routebeschrijving kan bijvoorbeeld de volgende vorm aannemen:

```
Startveld:      [0][0]
```

4. Merk op, dat bij de indicering van elementen van (tweedimensionale) arrays in **C** de *rij*-index vóór de *kolom*-index staat. Deze manier van indicering is net omgekeerd ten opzichte van de 'normale' wiskundige notatie, waarbij *(x, y)*-,coördinaten worden gebruikt.

```
Sprong naar:    [2][1]
Sprong naar:    [1][3]
Sprong naar:    [0][1]
...
Elk veld is bereikt.
```

Wanneer geen oplossing wordt gevonden moet een overeenkomstige melding worden gegeven voordat het programma wordt beëindigd.

15.4.1 Analyse van de puzzel

Uitgangspunt bij de paardesprongpuzzel is een bord waarop een paard is gezet. Elk veld op dit bord is een mogelijke beginplaats voor het paard. Vanuit de beginplaats kan het paard gaan springen. Hoewel er in het algemeen acht mogelijke velden zijn waar het paard naar toe kan springen, zijn er een aantal situaties waarbij dat niet het geval is. Het paard hoeft niet te springen wanneer alle velden zijn bereikt. Bij een 'bord' dat uit één veld bestaat is dat dus al direct het geval. Bij een 5 x 5 bord is dat het geval na 24 sprongen (het paard springt niet naar het eerste veld, maar wordt erop geplaatst). Het paard *mag* niet springen in de volgende situaties:

1. wanneer het paard door de sprong buiten het bord zou belanden,
2. wanneer het veld waar het paard naar toe wil springen al eerder was bereikt.

Wat moet er gebeuren in het geval dat een paard niet mag springen? In dat geval moet een volgend sprongalternatief worden geprobeerd. Zo kan een paard op veld [0][0] niet naar veld [-2][-1] springen, want dat zou een sprong buiten het bord zijn. Daarentegen is een sprong naar veld [2][1] wellicht wèl mogelijk. Wat moet er gebeuren wanneer geen enkele sprong mogelijk is? Op dat moment kan vanuit een gegeven stelling dus geen nieuwe paardesprong worden uitgevoerd en moet de vorige stelling worden herzien. De laatste paardesprong moet dan opnieuw, naar een volgend mogelijk veld, worden uitgevoerd voordat het proces verder kan worden uitgevoerd. Het kan echter voorkomen dat voor de laatste paardesprong geen velden meer beschikbaar zijn. In dat geval moet de voorlaatste sprong opnieuw, naar een volgend mogelijk veld, worden uitgevoerd. Vanuit die positie moet het proces dan verder worden vervolgd.

De recursie in de paardesprongpuzzel wordt gevormd doordat vanuit een gegeven paardesprong alle volgende paardesprongen worden getest. Wanneer die niet tot een oplossing leiden, moet een andere mogelijkheid worden gekozen, van waaruit opnieuw alle volgende paardesprongen worden getest. Bij elke volgende paardesprong moet dit proces opnieuw worden uitgevoerd.

De paardesprongpuzzel kan nu als volgt worden geherformuleerd, waarbij vooralsnog geen aandacht wordt geschonken aan de plaatsing van het eerste paard, maar we ons concentreren op het recursieve algoritme om een paard op een veld te plaatsen.

1. Verlaag de telling van het aantal nog te bereiken velden.
2. Wanneer alle velden zijn bereikt, beëindig het algoritme dan succesvol.
3. Zolang nog niet alle potentiële alternatieven voor een volgende paardesprong zijn getest,
   4. Probeer een paard naar een veld te laten springen dat correspondeert met het te testen alternatief. Als dat lukt, beëindig het algoritme dan succesvol.
5. Op dit moment zijn alle alternatieven zonder succes getest. Het huidige veld is zelf dus geen goed alternatief voor een paardesprong. Verhoog het aantal alsnog te bereiken velden en beëindig het algoritme met een mislukking.

De recursieve aard van het algoritme treedt op in punt 4. Wanneer er nog meer paardesprongen moeten worden uitgevoerd, wordt op dat moment het algoritme opnieuw uitgevoerd om de volgende paardesprong te bepalen.

15.4.2 Implementatie

Bij de implementatie van het programma *horses* moeten nog een aantal beslissingen worden genomen die in 15.4.1 niet aan de orde zijn gekomen. Eén van de beslissingen betreft de datastructuur waarin een overzicht van de sprongen wordt opgeslagen. Het ligt voor de hand om hiervoor een array te gebruiken, waarvan de velden de rij- en kolomindexen zullen zijn van het veld waarnaartoe wordt gesprongen. Omdat de elementen van het array twee waarden bevatten (een coördinatenpaar) definiëren we een *struct COORD_* waarin de coördinaten kunnen worden opgeslagen. Deze struct is gegeven in listing 88.

```
1     typedef struct
2     {
3         int
4             y,
5             x;
6     } COORD_;
```

Listing 88: De *struct COORD_* van *horse.c*.

Vervolgens kan, wanneer de matrix met *COORD_* elementen *board* heet, met behulp van de statements:

```
board[2][1].y = 4;
board[2][1].x = 2;
```

worden aangegeven dat vanuit veld [2][1] een sprong naar veld [4][2] moet worden uitgevoerd. Om aan te geven dat een veld nog vrij is zullen we de waarde -1 toekennen aan het *x* veld van een element van het array *board*.

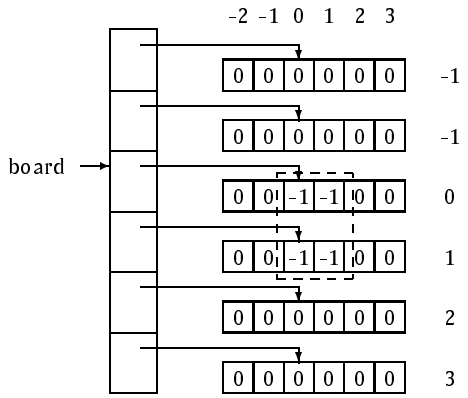
Omdat de grootte van het bord niet door de puzzel wordt vastgelegd, besluiten we het programma een argument mee te geven waarmee de omvang van het bord kan worden bepaald. Het argument is het aantal rijen (en kolommen) van het bord. Het array *board* wordt vervolgens gealloceerd, waarbij *board* zelf een array van pointers naar *COORD_* elementen wordt, analoog aan *argv* dat een array van pointers naar *char* elementen is. Hoewel *board* een array van pointers is, is het conceptueel eenvoudiger om *board* als een tweedimensionaal array op te vatten wanneer het erom gaat het bord met paardesprongen te vullen. Tijdens de allocatie zal *board* daarom eerst als pointervariabele worden behandeld, om vervolgens in het recursieve oplossingsproces als tweedimensionaal array te worden gebruikt.

De functie waarmee wordt getest of een paardesprong mogelijk is, moet onder andere controleren of een sprong buiten de grenzen van het bord dreigt te worden uitgevoerd. In dat geval is de sprong uiteraard niet mogelijk. In plaats van een directe test of de coördinaten van het voorgestelde veld wel binnen het bord vallen, wordt hier een iets andere benadering gevolgd. Om het eigenlijke bord wordt namelijk een band van twee velden gelegd, waarin altijd de informatie staat dat deze velden al in gebruik zijn. Omdat de positie van een paard bij een paardesprong hoogstens over twee velden in de horizontale of verticale richting kan veranderen, is een band van twee velden voldoende. De *x* velden van de elementen van een leeg bord van 2 x 2 velden zullen dus in feite worden geïnitialiseerd zoals is weergegeven in figuur 15.3. De variabele *board* is een pointer variabele die op het tweede element van een vector met pointers wijst, waardoor de rij coördinaten -2 en -1 ontstaan. Evenzo wijst elke pointer *board[i]* op het tweede element van een vector van zes elementen, waardoor de negatieve rij-coördinaten -2 en -1 ontstaan. Het element *board[-2][-2]* is dus het meest linkse element van de bovenste vector van zes elementen.

De organisatie van het programma *horse* kan nu als volgt worden geformuleerd:

1. Bepaal de dimensie van het probleem en het aantal te bereiken velden.
2. Alloceer en intialiseer de variabele *board*
3. Zolang niet alle zinvolle beginvelden zijn getest:
    4. Probeer dan alle velden met een paardesprong te bereiken.
    5. Zodra dat lukt, vermeld dan de gevonden oplossing en beëindig het programma.
6. Geef een melding dat geen oplossing werd gevonden.

De implementatie van de functie *main()* van het programma is gegeven in listing 89. De onderdelen 2, 4 en 5 zijn overgelaten aan functies die tot taak hebben de desbetreffende onderdelen te realiseren. Onderdeel 3



Figuur 15.3: Een leeg 2 x 2 bord bij *horse.c*.

is geïmplementeerd met een dubbel *for*-statement, waardoor slechts een achtste deel van het array *board* de beginvelden zal leveren. Omdat een vierkante matrix vier assen van symmetrie heeft[5], kan bij een oplossing in een gegeven beginpositie een overeenkomstige oplossing worden gevonden, wanneer de overeenkomstige beginpositie in de overige delen van het vierkant zou worden gekozen. Zo wordt, wanneer in een 6 x 6 matrix een oplossing wordt gevonden op veld [1][0], ook een oplossing gevonden wanneer wordt begonnen op veld [0][1], op [0][4], op [1][5], op [4][5], op [5][4], op [5][1], of op [0][4]. Merk op dat in het *y for*-statement tot de middelste regel van de matrix wordt doorgegaan met testen wanneer het aantal regels oneven is en tot de helft van het aantal regels wanneer het aantal regels even is. De reden om *y* tot de waarde *(max + 1) / 2* te laten oplopen is gebaseerd op de overwegingen die op pagina 214 ter sprake zijn geweest.

```
1     #include <stdio.h>
2
3     int
4         max,
5         to_do;
6
7     void main (int argc, char **argv)
8     {
```

5. De assen verdelen een vierkant in een bovenste- en onderste helft, in een linker en rechter helft, in een links-boven en rechts-onder driehoek en in een rechts-boven en links-onder driehoek.

```
9         register int
10            x,
11            y;
12
13        sscanf(argv[1], "%d", &max);                  // Tekst: 1.
14        to_do = max * max;
15
16        initialize_board();                           // Tekst: 2.
17
18        for (y = 0; y < (max + 1) / 2; y++)           // Tekst: 3.
19            for (x = 0; x <= y; x++)
20            {
21                if (puthorse (y, x))                  // Tekst: 4.
22                {
23                    show_trace(y, x);                 // Tekst: 5.
24                    return;
25                }
26            }
27        puts ("Geen oplossing gevonden.");            // Tekst: 6.
28    }
29
```

Listing 89: De functie *main()* van *horse.c*.

De overwegingen die bij de initialisatie van de variabele *board* een rol spelen zijn hierboven al genoemd. De functie *initialize_board()* kan als opgave worden geconstrueerd.

Opgave 84.

Construeer de functie *initialize_board()* aan de hand van de volgende overwegingen:

- **Het aantal rijen en kolommen moet 4 groter worden dan de dimensie van het probleem.**
- **De rijen worden geïnitialiseerd op de waarde 0. De allocatie en initialisatie kan met behulp van de standaardfunctie *calloc()* worden uitgevoerd. Hiermee wordt bereikt dat gedurende het oplossingsproces wordt aangenomen dat de omhullende rand van het array al in gebruik is. Na allocatie van een rij kan de bijbehorende rij-pointer met 2 worden opgehoogd.**
- **De pointer *board* wordt met 2 opgehoogd, waardoor *board* en de bijbehorende pointers op het feitelijk te vullen array wijzen.**
- **De elementen van het array zelf worden geïnitialiseerd op de waarde -1, waarvoor de standaardfunctie *memset()* kan worden gebruikt.**

Bedenk dat de uiteindelijke elementen waarop door de elementen van *board* wordt gewezen *COORD_ structs* zijn.

Onderdeel 5 kan ook als opgave worden uitgevoerd. De te construeren functie *show_trace()* krijgt twee argumenten (de begin-y en begin-x coördinaat) en vermeldt vanuit die uitgangspositie de route van het paard over het bord. Een extra bewerking kan eruit bestaan dat aan het einde van het traject het volledige bord als een matrix wordt afgedrukt, waarvan de elementen de volgordenummers zijn van de sprongen van het paard op het bord. Bij de constructie van de functie *show_trace()* zijn de volgende overwegingen van belang:

- Het einde van de routebeschrijving is bereikt wanneer een *x*-coördinaat -1 wordt aangetroffen.
- Op het moment dat een sprong is vermeld, kan de *x*-coördinaat van het bordelement het volgordenummer van de desbetreffende sprong krijgen. Nadat de routebeschrijving is afgedrukt, kan dan de matrix met sprongnummers eenvoudig aan de hand van de gewijzigde *x*-coördinaten worden vermeld.

Opgave 85.
Construeer de functie *show_trace()* aan de hand van bovenstaande beschrijving.

De functie puthorse()

De functie *puthorse()* wordt aangeroepen om een paard op een bepaald veld te plaatsen. Dat lukt in twee gevallen:

1. Wanneer de laatste sprong is uitgevoerd (en de variabele *to_do* 0 is geworden).
2. Wanneer vanuit de huidige positie een volgende sprong kan worden uitgevoerd.

Wanneer de sprong niet kan worden uitgevoerd, moet de *x*-coördinaat van het huidige veld weer op -1 worden gezet, omdat het veld dan alsnog ongebruikt is. Bovendien is een paardesprong dan *niet* gelukt en moet de *to_do* variabele dus weer worden opgehoogd.

De taken van de functie *puthorse()* kunnen als volgt worden geherformuleerd:

4.1. Verlaag de *to_do* variabele. Beëindig de functie succesvol wanneer *to_do* 0 is geworden.
4.2. Doe voor elk van de acht sprongalternatieven het volgende:
  4.3. Als een sprong naar een nieuw veld mogelijk is, beëindig de functie dan succesvol.
4.4. Geef aan dat het huidige veld nog ongebruikt is, verhoog de *to_do* variabele en beëindig de functie met een mislukking.

De implementatie van de functie *puthorse()* is gegeven in listing 90. Onderdeel 4.2. is geïmplementeerd met een *for*-statement waarbij een indexvariabele wordt gebruikt over de elementen van een array *testmat*. Dit array bevat *COORD_* elementen, die de verschuiving van de *x*- en *y*-coördinaten ten opzichte van de huidige positie bevatten. Zo heeft element 0 van het array de waarden *-1, -2*. In dit geval bevindt het veld voor de voorgestelde sprong zich één rij hoger en twee kolommen eerder op het bord dan het huidige veld. Het array *testmat* kan door de vertaler worden geïnitialiseerd. De definitie van het array is eveneens in listing 90 gegeven.

```
1         COORD_
2             testmat[] =
3             {
```

```
4                -1, -2,
5                -2, -1,
6                -2,  1,
7                -1,  2,
8                 1, -2,
9                 2, -1,
10                2,  1,
11                1,  2,
12            };
13
14        int puthorse (int cury, int curx)
15        {
16            register int
17                index;
18
19            if (!--to_do)
20                return (1);
21
22            for (index = 0; index < sizeof(testmat) / sizeof(COORD_); index++)
23            {
24                if (jump(cury, curx, index))
25                    return (1);
26            }
27
28            board[cury][curx].x = -1;
29            to_do++;
30            return (0);
31        }
```

Listing 90: De functie *puthorse()* van *horse.c*.

Onderdeel 4.3. is geïmplementeerd als een aanroep van de functie *jump()* waarin geprobeerd wordt een sprong uit te voeren naar een volgend veld. De functie *jump()* krijgt drie argumenten vanuit *puthorse():* de *y*- en *x*-coördinaten van het huidige veld en de huidige indexwaarde voor het array *testmat*. De coördinaten van het huidige veld en de verschuiving die in het element van *testmat* is opgeslagen, bepalen het veld dat met de volgende sprong moet worden bereikt. De coördinaten van het volgende veld worden opgeslagen in het huidige bordelement ten behoeve van de functie *show_trace()*. Wanneer het veld waar naartoe wordt gesprongen nog vrij is en wanneer de paarden ten opzichte van het nieuwe veld kunnen worden geplaatst, kan de functie *jump()* succesvol worden beëindigd. Zo niet, dan moet een mislukte poging worden geretourneerd.

De herformulering van de taken van de functie *jump()* is:

4.3.1. Plaats de coördinaten van het volgende veld in het huidige element van het array *board*.

4.3.2. Wanneer het nieuwe veld nog vrij is en wanneer de resterende velden ten opzichte van het nieuwe veld op het bord kunnen worden

bereikt, beëindig de functie dan succesvol. Wanneer niet aan beide condities is voldaan, beëindig de functie dan met een mislukking.

De implementatie van de functie *jump()* is gegeven in listing 91. Het recursieve moment in het programma *horse.c* is indirect: *puthorse()* wordt vanuit *jump()* aangeroepen en *jump()* wordt in *puthorse()* aangeroepen. Wanneer in de brontekst van het programma *horse()* de functie *jump()* gedefinieerd wordt vóór de functie *puthorse()* moet het prototype van de functie *puthorse()* voorafgaand aan de definitie van de functie *jump()* zijn gegeven.

```
1         int puthorse (int, int);
2
3         int jump(register int cury, register int curx, int testnr)
4         {
5             register int
6                 newx,
7                 newy;
8
9             board[cury][curx].y = newy = cury + testmat[testnr].y;
10            board[cury][curx].x = newx = curx + testmat[testnr].x;
11
12            return
13            (
14                board[newy][newx].x == -1
15                &&
16                puthorse (newy, newx)
17            );
18        }
```

Listing 91: De functie *jump()* van *horse.c*.

Opgave 86.

Stel het programma *horse.c* samen uit de listings 88 tot en met 91 en de functies *initialize_board()* en *show_trace()* uit de vorige opgaven. Zorg ervoor dat functies en variabelen waar mogelijk zijn gedefinieerd voordat ze worden gebruikt. Vertaal en link het programma en test het programma voor een 5 x 5 bord.

15.5 Toepassingen: Quicksort en Binair zoeken

Hoewel het programma *horse.c* een aardige illustratie is van een recursief proces, is het praktisch nut ervan beperkt. In de volgende twee opgaven worden een paar nuttiger toepassingen van recursieve functies geïllustreerd. De functie *quicksort()* (Hoare, 1962) is gebaseerd op een recursief algoritme om een array te sorteren. De functie *binsearch()* is gebaseerd

op een recursief algoritme om een element in een gesorteerd array op te zoeken met behulp van een binair zoekproces[6].

Een eenvoudige vorm van de functie *quicksort()* heeft twee argumenten: een benedengrens en bovengrens voor de indexen in een array waar tussen de elementen moeten worden gesorteerd. Wanneer de waarde van de ondergrens niet kleiner is dan de waarde van de bovengrens hoeft er geen sortering plaats te vinden. In alle andere gevallen wordt een element geselecteerd ten opzichte waarvan de overige elementen worden gesorteerd. Het array is na deze stap gesorteerd in drie subarrays:

1. Het subarray waarin alle (overige) elementen met waarden kleiner dan of gelijk aan het geselecteerde element staan.
2. Het subarray waarin het geselecteerde element staat.
3. Het subarray waarin alle elementen met waarden groter dan het geselecteerde element staan.

In grensgevallen kan het voorkomen dat de subarrays 1 en 3 niet optreden. Omdat het array nu ten opzichte van één element is gesorteerd, kan het proces worden herhaald op de beide subarrays, die dus ook weer ten opzichte van een element worden gesorteerd. Dit proces wordt herhaald totdat de subarrays niet meer optreden, waarna het array is gesorteerd. Het sorteren van de subarrays wordt uitgevoerd door een recursieve aanroep van de functie *quicksort()*. In listing 92 is een programma gegeven waarmee woorden die als argument met een programma worden meegegeven alfabetisch worden gesorteerd. Voor het vergelijken van de individuele woorden wordt de standaardfunctie *strcmp()* gebruikt. De functie *main()* roept een functie *quicksort()* aan om de gegeven argumenten te sorteren. De functie *quicksort()* is als code-frame gegeven en kan als opgave worden geïmplementeerd.

```
1         #include <stdio.h>
2         #include <string.h>
3
4         char
5             **gargv;
6
7         void quicksort(int lower, int upper)
8         {
9             char
10                *change,
11                *item;
12            int
13                left,
14                right;
15
16            if (lower >= upper)                    // array van hoogstens
```

6. De quicksort functie en de binaire zoekfunctie zijn beide opgenomen in de standaardfunctie-bibliotheek van Microsoft als, respectievelijk, de functies *qsort()* en *bsearch()*.

```
17                return;                                   // 1 element: klaar
18
19            item = gargv[upper];                          // selectie-element
20
21            left = lower;                                 // sorteergrenzen
22            right = upper;
23
24            do
25            {
26                // verhoog left zolang gargv[left] voor item
27                // in het alfabet optreedt.
28
29                // verlaag right zolang  item voor gargv[right]
30                // in het alfabet optreedt.
31
32                if (left <= right)
33                {
34                    // verwissel gargv[left] en gargv[right]
35                    // verhoog left, verlaag right
36                }
37            }
38            while (left <= right);
39
40            // quicksort tot en met element right;
41            // quicksort vanaf element left;
42        }
43
44        void main(int argc, char **argv)
45        {
46            int
47                index;
48
49            gargv = argv;
50            quicksort(0, argc - 1);
51            for (index = argc; index; index--, argv++)
52                puts(*argv);
53        }
```

Listing 92: Een programma dat gebruik maakt van *quicksort()*.

Opgave 87.

Construeer de functie *quicksort()*. Vertaal en link het programma en test het met een aantal argumenten

De functie *binsearch()* heeft in een eenvoudige variant eveneens twee argumenten, waarmee de grenzen van het array waarin een te zoeken element zich kan bevinden, zijn vastgelegd. Het te zoeken element zelf kan een globale variabele zijn, evenals het (gesorteerde) array. Het binair zoekalgoritme verloopt als volgt:

- Wanneer de ondergrens groter is dan de bovengrens, dan is het te zoeken element niet in het array aanwezig en beëindigt de functie met een mislukking.

- Bepaal de index van het element halverwege de ondergrens en de bovengrens.
- Test of het daar aangetroffen element gelijk is aan het te zoeken element. Als dat het geval is retourneer dan de index van het gevonden element in het array.
- Als het element zich vóór het middelste element moet bevinden, retourneer de return-waarde van *binsearch()* op het subarray links van het middelste element.
- Retourneer de return-waarde van *binsearch()* op het subarray rechts van het middelste element.

Opgave 88.

Breid het *quicksort()* programma uit de vorige opgave uit met een binaire zoekfunctie. Het programma kan, nadat de argumenten zijn gesorteerd, vragen een string in te typen die vervolgens wordt opgezocht in het gesorteerde array. Indien de string is gevonden, wordt de index in het array vermeld. Indien de string niet wordt gevonden, wordt een overeenkomstige melding gegeven. Laat de functie *binsearch()* de waarde -1 opleveren wanneer de te zoeken string niet in het array is aangetroffen.

Hoofdstuk 16

Grammatica's en grammaticaherkenners

16.1 Inleiding

In dit hoofdstuk wordt nader ingegaan op het begrip *grammatica*. Dit begrip is in hoofdstuk 11 al summier besproken, toen de structuur van een databestand bij de help-faciliteit aan de orde was. In dit hoofdstuk wordt aandacht besteed aan het *opstellen* en *formaliseren* van een grammatica; met andere woorden het omzetten van een structuur in formele regels. Tevens komt het begrip *lexical analyzer* nogmaals aan de orde.

16.1.1 Het nut van grammatica's

In de praktijk worden grammatica's gebruikt bij programma's die gecompliceerde invoer moeten verwerken. De invoer van een dergelijk programma is op een vaststaande manier georganiseerd. Voor de software-ontwikkelaar is een grammatica nuttig doordat een probleem aangepakt wordt *voordat* de feitelijke implementatie aan de orde komt. Het opstellen van een grammatica staat daarom op een hoger abstractieniveau dan de implementatie van een programma. In dit hoofdstuk zullen we dan ook voorbeelden van grammatica's laten zien zonder dat feitelijke **C**-code eraan te pas komt. Vervolgens zullen we laten zien hoe aan de hand van de opgestelde grammatica tot implementatie kan worden overgegaan.

16.2 Een grammaticavoorbeeld: taal

Als eerste voorbeeld van een grammatica zullen we trachten een deel van de Nederlandse taal te formaliseren. We gebruiken dit voorbeeld omdat die taal 'dicht bij huis' ligt en omdat wellicht iedereen vertrouwd is met begrippen als onderwerp en gezegde. Toch moet opgemerkt worden dat er een fundamenteel verschil is tussen de grammatica van de taal en een grammatica zoals die van **C**. De Nederlandse taal is empirisch ontstaan en is daardoor niet voor honderd procent te formaliseren. Dat is in het dagelijks taalgebruik geen probleem, omdat de bedoeling van een zin vaak uit de context afgeleid kan worden. Bij taal hoeven geen strikte regels te worden gehanteerd; het dagelijks taalgebruik is niet exact. Daarentegen mag bij een programmeertaal als **C** geen onduidelijkheid over de betekenis van een brontekst bestaan. De grammatica van **C** is dan ook niet geformaliseerd toen **C** vertalers al bestonden, maar andersom: **C** vertalers zijn opgesteld aan de hand van een formele grammatica.

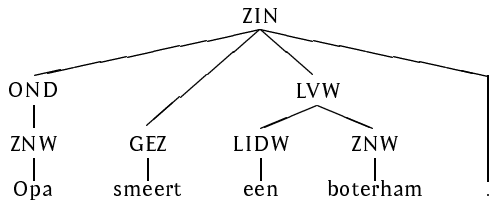
We beschouwen hier een eenvoudige taalgrammatica, waarin een zin bestaat uit een onderwerp gevolgd door een gezegde en een lijdend voorwerp. De zin wordt door een punt afgesloten. Een voorbeeld van zo'n zin is: "Opa eet een boterham.". De hier opgestelde grammatica is in figuur 16.1 weergegeven. Over de in deze figuur gebruikte notatie maken we de volgende opmerkingen:

```
zin:
        onderwerp gezegde lijdend-voorwerp .
onderwerp:
        lidwoord zelfstandig-naamwoord ||
        zelfstandig-naamwoord
gezegde:
        eet || smeert || zit || fietst
lijdend-voorwerp:
        lidwoord zelfstandig-naamwoord ||
        zelfstandig-naamwoord
lidwoord:
        de || het || een
zelfstandig-naamwoord:
        opa || oma || boterham
```

Figuur 16.1: Een eenvoudige grammatica van taal.

- De grammatica is zodanig opgeschreven dat bovenaan het lijstje hetgene staat dat de grammatica beschrijft. In dit geval is dat een *zin*.
- Voor elk begrip van de grammatica wordt een regel gegeven die de vorm beschrijft die dat begrip heeft. Een *zin* kan volgens deze grammatica slechts één vorm aannemen. Er zijn echter ook begrippen die meerdere vormen kunnen aannemen.
- Alternatieven worden aangeduid door '||'. Zo is een *lidwoord* òf 'de', òf 'het', òf 'een'.
- De beschrijving van een begrip kan gebruik maken van symbolen die zelf uit verschillende symbolen kunnen bestaan. Zo wordt een *zin* beschreven door onder andere een *lijdend-voorwerp;* het *lijdend-voorwerp* zelf wordt weer beschreven door *zelfstandig-naamwoord.*
- Aan de basis van de regels staan symbolen die niet verder beschreven zijn. Doorgaans zijn dit symbolen die letterlijk in de tekst voorkomen; zo staat "zelfstandig-naamwoord" niet letterlijk in de tekst maar "opa" wel. De letterlijke symbolen zijn in figuur 16.1 vet afgedrukt en worden eindsymbolen (terminal symbols) genoemd.

Als voorbeeld van het gebruik van deze grammatica zullen we hier laten zien hoe de zin "Opa smeert een boterham." ontleed wordt. De grammaticale regel voor een zin verwacht als eerste symbool een onderwerp; met andere woorden een lidwoord en zelfstandig naamwoord of een zelfstandig naamwoord. Als we de zin van links naar rechts lezen, dan komen we als eerste het woord "Opa" tegen. Het onderwerp-deel van de zin is daarmee afgesloten: aangezien "Opa" een zelfstandig naamwoord is, is een onderwerp gevonden. Als tweede deel van de zin verwachten we een gezegde te zien. Dat klopt, want "smeert" komt als één van de alternatieven van



Figuur 16.2: Ontleding van een zin.

een gezegde voor in de grammaticale regels. Hierna verwachten we een lijdend voorwerp; dus een lidwoord en een zelfstandig naamwoord of alleen een zelfstandig naamwoord. In de zin vinden we een lidwoord ("een") en zelfstandig naamwoord ("boterham"). Daarmee is het lijdend voorwerp afgesloten. Tot slot verwachten we een punt; ook deze wordt in de zin gevonden. We kunnen dus concluderen dat de zin "Opa smeert een boterham." grammaticaal correct is.

De zin wordt van links naar rechts ontleed. De grammatica kan op grond van één symbool in de invoer beslissen wat er herkend wordt. Deze kenmerken maken de hier ontwikkelde grammatica tot een 'left-to-right, look-ahead 1 symbol' (LALR-1) grammatica. Naast de LALR-1 grammatica's bestaan andere typen [1]; we zullen ons echter in dit boek tot LALR-1 grammatica's beperken.

De ontleding van bovenstaande zin is in figuur 16.2 weergegeven. Het ontleedproces is hier als een boom die ondersteboven staat afgebeeld. Aan de onderkant van het plaatje staan de invoersymbolen zoals die door de grammatica gezien worden. Naar boven toe worden deze delen tot grammaticale symbolen verenigd, totdat een zin gevormd is.

Grammatica's geven in het algemeen alleen uitsluitsel over de vraag of iets grammaticaal, dus *syntactisch*, correct is. De vraag of bepaalde invoer ook semantisch correct is, is van geheel andere aard. Volgens de hier opgestelde grammatica is de zin "De fiets eet opa." dan ook syntactisch correct. Syntactisch juiste invoer die semantisch niet correct is, is ook bij **C** brontekstenterug te vinden. Een brontekst die vertaald kan worden is syntactisch correct, maar hoeft niet te doen wat van het programma verlangd wordt. In een dergelijk geval treedt dus een semantische fout op.

1. Zie Tremblay (1985)

16.3 Bouwstenen van grammatica's

In dit onderdeel zullen we een aantal bouwstenen behandelen waarmee grammatica's opgesteld kunnen worden. Evenals bij de beschrijving van grammatica's zullen we de implementatie van deze bouwstenen buiten beschouwing laten en zullen we alleen de betekenis en het gebruik van deze bouwstenen schetsen.

De regels die hier geschetst worden, voldoen aan de LALR-1 kenmerken.

16.3.1 Alternatieven

Met de regel voor alternatieven hebben we al kennis gemaakt bij onder andere de definitie van een lidwoord. Zo stelt de regel:

```
lidwoord:
    de || het || een
```

dat er sprake is van een een lidwoord als in de invoer 'de', 'het' of 'een' staat.

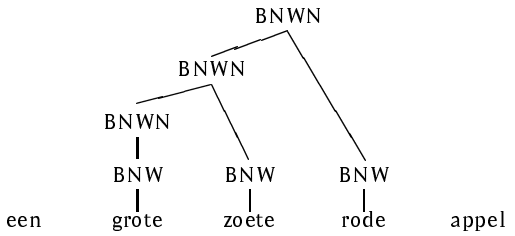
Een regel met alternatieven hoeft niet alleen symbolen te betreffen die letterlijk in de invoer staan. De alternatieven kunnen ook weer regels zijn. De alternatieve regels *moeten* echter uiteindelijk op een eindsymbool uitkomen; anders geeft de grammatica geen uitsluitsel over welke keuze gemaakt moet worden. Dit is in de onderstaande grammatica geïllustreerd, waarin een deel van de **C** syntaxis is geformaliseerd. Volgens deze grammatica kan een statement onder andere een if-statement en een while-statement zijn. Een if-statement begint met het keyword *if*; een while-statement begint met het keyword *while*. In de statement-regel kan dus op grond van het feit of een *if* of een *while* in de invoer staat, beslist worden welk alternatief gekozen moet worden:

```
statement:
    if-statement || while-statement || etc.
if-statement:
    if etc.
while-statement:
    while etc.
```

16.3.2 Herhalingen van één of meer elementen

Een serie van één of meer elementen in de invoer wordt met behulp van een recursieve grammaticale regel gedefinieerd. De regel:

```
brontekst:
    brontekst functie ||
    functie
functie:
    etc.
```



Figuur 16.3: De ontleedboom bij een links-recursieve regel.

stelt dat een brontekst kan bestaan uit één of meer functies. Een dergelijke regel is bijvoorbeeld van toepassing op een **C** brontekst. Merk op dat de recursie in de brontekstregel in het linker element van het eerste alternatief optreedt. Dit heeft effect op de manier waarop de ontleedboom wordt opgebouwd. In het voorbeeld hierboven spreken we daarom van een links-recursieve regel.

Een ander voorbeeld van een links-recursieve regel die op de Nederlandse taal van toepassing is, is deze:

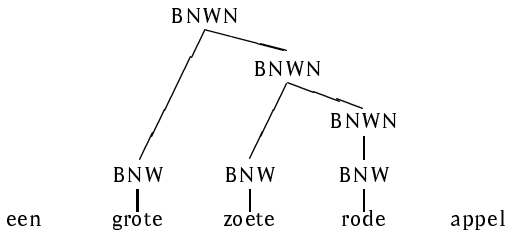
```
bijv-naamwoorden:
    bijv-naamwoorden bijv-naamwoord ||
    bijv-naamwoord
bijv-naamwoord:
    grote || zoete || rode
```

Volgens deze regel is sprake van bijvoeglijke naamwoorden als één of meer bijvoeglijke naamwoorden in de invoer voorkomen. Figuur 16.3 laat zien hoe de zin "een grote zoete rode appel" geanalyseerd wordt. In deze figuur wordt alleen aandacht besteed aan de bijvoeglijke naamwoorden; de woorden "een" en "appel" zijn buiten beschouwing gelaten. Te zien is dat "grote" en "zoete" samengevoegd worden vóórdat "rode" erbij betrokken wordt.

Dezelfde regel voor bijvoeglijke naamwoorden kan als een rechts-recursieve regel opgesteld worden. In dat geval is de definitie de volgende:

```
bijv-naamwoorden:
    bijv-naamwoord bijv-naamwoorden ||
    bijv-naamwoord
bijv-naamwoord:
    grote || zoete || rode
```

Bij deze regel wordt de zin "een grote zoete rode appel" geanalyseerd zoals weergegeven in figuur 16.4. Het samenvoegen van meerdere bijvoeglijke



Figuur 16.4: De ontleedboom bij een rechts-recursieve regel.

naamwoorden treedt hier later op; bijvoeglijk naamwoord "zoete" wordt nu bij "rode" gevoegd en pas daarna wordt "grote" erbij gevoegd.

De beslissing of een recursieve grammaticale regel rechts- of links-recursief moet zijn, is gewoonlijk aan degene die de grammatica opstelt. De rechts- of links-recursiviteit heeft echter consequenties voor de manier waarop de regels toegepast worden. We zullen in dit hoofdstuk daar nog meer voorbeelden van zien (zie 16.5.1).

16.3.3 Herhalingen van nul of meer elementen

Evenals series van één of meer elementen zijn series van nul of meer elementen mogelijk. Zo stelt:

```
bijvoeglijke-naamwoorden:
    bijvoeglijke-naamwoorden bijvoeglijk-naamwoord ||
    ϵ
```

dat de regel "bijvoeglijke-naamwoorden" van toepassing is als geen, één of meer bijvoeglijke naamwoorden in de invoer staan. Symbool ϵwordt in deze notatie gebruikt om 'niets' aan te geven[2].

Series van nul of meer elementen kunnen, evenals series van één of meer elementen, links- of rechts-recursief zijn. De recursiviteit heeft dan wederom betrekking op de samenvoeging van de symbolen.

16.3.4 Herhalingen met scheidingstekens

Bij series van nul of meer elementen wordt vaak vereist dat de losse elementen door eindsymbolen gescheiden zijn. Een voorbeeld hiervan is de parameterlijst van een functie. Als in de parameterlijst meer dan één parameter staat, dan moeten de parameters door middel van komma's gescheiden zijn.

2. In dit boek wordt op oneindig veel plaatsen niets aangegeven. In die situaties is het ϵ-symbool echter niet gebruikt.

Een serie van één of meer parameters wordt gedefinieerd door de volgende regel:

```
parameter-lijst:
    parameter-lijst , parameter ||
    parameter
```

In deze definitie is "parameter" niet verder omschreven. Merk op dat het scheidingsteken in het recursieve alternatief van de regel voorkomt.

Een serie van nul of meer parameters kan met behulp van de bovenstaande regel voor een parameterlijst opgesteld worden. Onderstaande regel definieert een optionele parameterlijst:

```
opt-parameter-lijst:
    parameter-lijst ||
    ϵ
```

16.3.5 Nestingen van niet-herhaalde elementen

In veel grammatica's worden nestingen gebruikt om aan te geven dat symbolen bij elkaar gevoegd moeten worden. Een voorbeeld van een nesting is het gebruik van haakjes bij rekenkundige expressies zoals in *(3 – 5) / 12*; hier zorgen de haakjes ervoor dat de deelexpressie *3 – 5* een hogere prioriteit krijgt dan de deling. Met de nesting wordt bedoeld dat de haakjes zelf ook door haakjes omgeven kunnen zijn.

Als concreet voorbeeld laten wij hieronder een grammatica zien waarbij in de invoer *a+b* moet staan. Deze serie symbolen mag door haakjes omgeven zijn. Correcte invoer is dus *a+b* of *(a+b)* of *((a+b))* enzovoort. We spreken hierbij van een nesting van *niet-herhaalde* elementen, omdat hetgene dat tussen de haakjes staat *geen* herhaling mag zijn. In dit voorbeeld moet tussen de haakjes exact éénmaal *a+b* staan:

```
expressie:
    ( expressie ) || optelling
optelling:
    a + b
```

16.3.6 Nestingen van herhaalde elementen

Bij nestingen komt het ook vaak voor dat binnen de nestingssymbolen een herhaling (van nul of meer elementen) mag staan. Eventueel zijn de elementen van de herhaling gescheiden door scheidingstekens.

Een nesting van een herhaling kan met behulp van een hulpregel weergegeven worden. Als voorbeeld is hieronder een grammatica weergegeven waarbij *a*, *aa*, *(a)*, *(aa)*, *((aaa))*, *(((a))* enzovoort, geldige invoer is:

```
invoer:
    serie
serie:
    serie a ||
    a ||
    ( invoer )
```

In deze grammatica is te zien dat een *serie* uit één of meer *a*'s moet bestaan (zie ook 16.3.2). Daarnaast kan de serie *a*'s tussen haakjes zijn gegeven.

Om in een dergelijke grammatica minstens één paar haakjes te verlangen kan de *invoer*-regel gewijzigd worden in:

```
invoer:
    ( serie )
```

Het aanbrengen van scheidingstekens wordt in de grammatica opgenomen in de regel die de herhaling weergeeft (zie ook 16.3.4).

Opgemerkt moet worden dat nestingen met herhalingen een algemeen geval zijn van nestingen zonder herhalingen. Een nesting zonder herhaling kan ook met behulp van een hulpregel weergegeven worden.

Opgave 89.

Schrijf de grammaticale regels voor een compound statement. Geef hierbij aan dat een compound statement uit nul of meer statements bestaat, waarbij de statements door { en } zijn omgeven. Elk statement kan op zich weer een compound statement zijn. Neem aan dat een statement een *if*-statement of een *while*-statement kan zijn; deze laatste twee grammaticale regels hoeven niet uitgewerkt te worden.

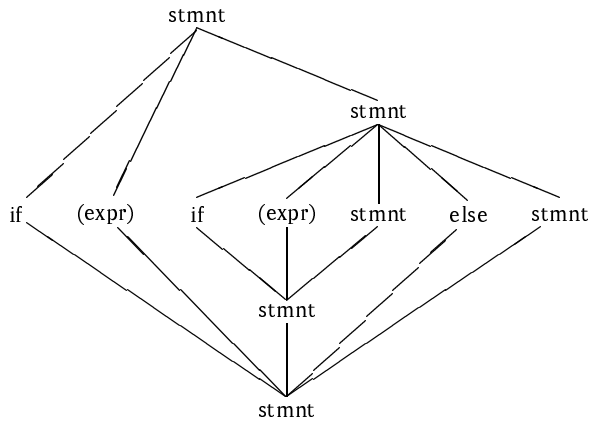
16.4 Conflicten in een grammatica

Het opstellen van een grammatica kan problemen met zich meebrengen. Vaak onstaan deze problemen doordat de grammaticale regels niet eenduidig zijn: één regel of een set van regels kan dan op meerdere manieren uitgelegd worden. We zullen in dit deel twee vaak voorkomende conflicten behandelen.

16.4.1 Onvoltooide regelconflicten

Het eerste conflict treedt op bij een situatie waarin onduidelijk is of een regel al afgelopen is of niet. Conflicten van dit type kunnen in een grammatica niet altijd vermeden worden. In hoofdstuk 4 is een dergelijk conflict in de grammatica van **C** al genoemd: het 'dangling-else'-probleem. We zullen dit probleem aan de hand van grammaticaregels toelichten.

De regel voor een statement stelt onder andere dat een statement een *if*-statement kan zijn. Een *if*-statement bestaat uit het keyword *if*, gevolgd door een expressie tussen haakjes, waarachter een statement staat. Achter dit geheel volgt het *else*-deel.



Figuur 16.5: Ontleedbomen van het 'dangling-else' conflict.

Het *else*-deel kan bestaan uit het keyword *else* gevolgd door een statement. Maar het *else*-deel mag ook leeg zijn. De grammatica van het *if*-statement is hieronder gegeven; de regels voor een statement en voor een expressie zijn niet verder uitgewerkt:

```
if-statement:
    if ( expression ) statement else-tail
else-tail:
    else statement ||
    ϵ
```

Het conflict in deze grammatica onstaat als het statement dat na de *if* komt zelf weer een *if*-statement met *else*-deel is. Dit is in figuur 16.5 weergegeven. In deze figuur zijn twee ontleedbomen getekend; bij de bovenste boom is de keuze gemaakt om de *else* bij de dichtst bijzijnde *if* te betrekken. Dit is ook de keuze die de **C** vertaler maakt. Bij de onderste ontleedboom wordt ervan uitgegaan dat de *else* bij de buitenste *if* hoort.

Conflicten waarbij onduidelijk is wanneer een regel afgelopen is zijn, zoals al vermeld, niet altijd te voorkomen. Wel moet vaststaan op welke manier een ontleedboom moet worden opgesteld als zo'n conflict optreedt.

16.4.2 Keuzeconflicten

Een tweede vaak voorkomend conflict treedt op als bij een regel twee keuzes gemaakt kunnen worden, maar als onduidelijk is *welke* keuze de juiste is. We illustreren dit conflict aan de hand van een taalgrammatica, waarin

het symbool 'vliegen' een zelfstandig naamwoord èn een werkwoord kan zijn. Het doel van deze grammatica is het onderscheiden van zinnen als:

Mensen fietsen.
Fietsen mensen?
Mensen vliegen.
Vliegen vliegen.

De grammaticale regels zijn hieronder gegeven:

zin:
    zelfstandig-naamwoord werkwoord || **.**
    werkwoord zelfstandig-naamwoord **?**
werkwoord:
    fietsen || **vliegen**
zelfstandig-naamwoord:
    mensen || **vliegen**

Het conflict in deze grammatica treedt op wanneer als eerste twee symbolen in de invoer "vliegen" worden gevonden. De grammatica geeft geen uitsluitsel over de vraag of het eerste "vliegen" een zelfstandig naamwoord of een werkwoord is. Deze vraag kan pas beantwoord worden als het derde symbool, een . of een ?, gelezen wordt. Maar omdat met een LALR-1 grammatica gewerkt wordt is er maar één look-ahead symbool beschikbaar en dat is nog niet de . of het ?.

16.5 Een eenvoudige rekenmachine

De theorie over grammatica's die tot nu toe in dit hoofdstuk is behandeld zullen we nu in praktijk brengen aan de hand van een concreet programma. Hierbij zal de implementatie van grammaticale regels aan de orde komen.

De opgave die we ons stellen is de constructie van een eenvoudige rekenmachine, die om invoer van de gebruiker vraagt en deze invoer als een expressie opvat en tracht te analyseren. We besteden niet veel aandacht aan de 'buitenkant' (het gebruikersgemak) van het programma; het gaat hier om de grammaticale regels bij de analyse van expressies. Het doel van de rekenmachine is namelijk een ontleding van ingelezen invoer aan de hand van *voorrangsregels* (prioriteitsregels). Ter gedachtenbepaling laten we hieronder een sessie zien met het te ontwikkelen programma, dat we *scalc* (simple calculator) zullen noemen. De invoer van de gebruiker bestaat in dit voorbeeld uit expressies zoals *12 / 4 - 2.5*:

```
SCALC : Simple four-function calculator
Enter expressions using * / + - .
Sub-expressions may be enclosed in ( and ).
Just press the enter key when done.
```

```
12 / 4 - 2.5
    = 0.500000
13 * 5 - -3
    = 68.000000
13 * (5 - 3)
    = 26.000000
```

Zoals uit dit voorbeeld blijkt, evalueert het programma eerst de vermenigvuldiging of deling en daarna pas de aftrekking. Dit is in overeenstemming met de gangbare voorrangsregels; we spreken dan ook af:

- dat haakjes de hoogste prioriteit hebben, samen met de unaire min of plus;
- dat deling en vermenigvuldiging de tweede hoogste prioriteit hebben;
- dat de optelling en aftrekking de laagste prioriteit hebben.

De associativiteit van de unaire min en van de unaire plus laten we van rechts naar links zijn. De associativiteit van alle andere operatoren laten we van links naar rechts zijn. Dit is in overeenstemming met de grammatica die door de **C** vertaler wordt gehanteerd.

16.5.1 Herkenning van de invoer

We kunnen intuïtief zeggen dat in de expressie $43 - 5 * 4$ de symbolen 5, $*$ en 4 als een eenheid moeten worden beschouwd. Dit onderdeel van de expressie wordt immers het eerst geëvalueerd. Bij de symbolen 43, $-$ en 5 is dat niet zo. De grammaticale regels zullen dus uitsluitsel moeten geven *welke* symbolen bij elkaar geveegd moeten worden om een expressie op juiste wijze te evalueren.

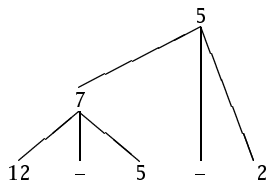
De regel voor een expressie

De stukjes van een expressie krijgen ten behoeve van de grammaticale regels elk een eigen naam. We houden hier de namen aan die historisch gezien voor de ontleding van expressies gebruikt worden [3]. Volgens deze regels bestaat een expressie uit een *term*, eventueel voorafgegaan door een expressie met een plus of min erachter. De regel voor een expressie kan als volgt geschreven worden:

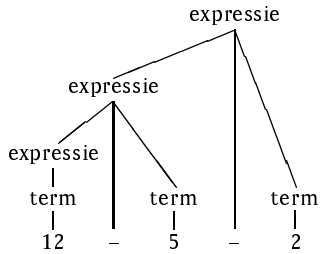
```
expressie:
        term ||
        expressie + term ||
        expressie – term
```

Wat een term is laten we voorlopig ongespecificeerd; echter, de eenvoudigste vorm van een term is een getal. De grammaticale regel voor een term

3. Zie Aho, Sethi en Ullman (1986): *Compilers*



Figuur 16.6: De evaluatieboom van expressie 12 – 5 – 2.



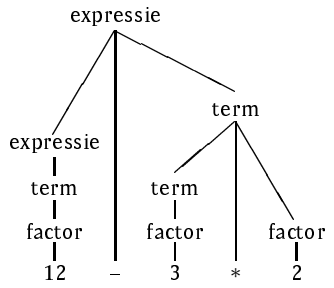
Figuur 16.7: De ontleedboom van expressie 12 – 5 – 2.

komt later aan de orde. Aan de hand van deze regel kunnen we een expressie als *12 – 5 – 2* als een boom weergeven, waarbij elke samenvoeging van symbolen van een naam wordt voorzien. Dit proces is in figuren 16.6 en 16.7 geïllustreerd. In figuur 16.6 is weergegeven hoe *12 – 5 – 2* geëvalueerd wordt. Analoog aan deze evaluatieboom is in figuur 16.7 weergegeven hoe de grammaticale onderdelen van een expressie (de elementen expressie en term) gegroepeerd worden. Te zien is dat *12 – 5* berekend wordt vóórdat *– 2* erbij gehaald wordt. Dit is te danken aan het feit dat in de expressieregel staat dat een expressie kan bestaan uit een expressie, gevolgd door een –, gevolgd door een term. In de expressie-regel is dus sprake van links-recursie. Omkering van deze regel (dus eerst een term, dan de –, dan een expressie) zou leiden tot rechts-recursie en tot rechts-associativiteit van de – operator, hetgeen niet de bedoeling is.

Een concreet uitgebreid voorbeeld van de werking van de expressieregel stellen we uit tot pagina 349.

De regel voor een term

Analoog aan de grammaticale regel voor een expressie kunnen we de regel voor een term opstellen. Met behulp van de term zullen we de ∗ en / ope-



Figuur 16.8: De ontleedboom van expressie 12 – 3 ∗ 2.

ratoren in de grammatica verwerken. Hiervoor introduceren we een *factor* en laten deze grammaticale eenheid nog ongespecificeerd. We volstaan met de aanname dat een factor een getal kan zijn.

Een term bestaat uit een factor, eventueel voorafgegaan door een term en een ∗ of /. De regel voor een term luidt:

```
term:
        factor ||
        term ∗ factor ||
        term / factor
```

Het gebruik van een factor is in figuur 16.8 weergegeven. Hier wordt expressie *12 – 3 ∗ 2* ontleed. Dankzij het feit dat de term de vermenigvuldiging voor zijn rekening neemt, is de volgorde van ontleding juist.

Een uitgebreide uitleg van deze regel stellen we uit tot pagina 349.

De regel voor factor

De grammaticale regels voor het herkennen van een expressie zijn bijna compleet. De laatste regel betreft de factor.

Ten eerste kan een factor bestaan uit een getal. Onze grammatica zal geen verdere specificatie van een getal nodig hebben; er zijn wat een getal betreft geen alternatieven zoals bij een term. Een getal is daarom in deze grammatica een eindsymbool (*terminaal* symbool).

Ten tweede kan een factor bestaan uit een unaire min of unaire plus, gevolgd door een factor. Deze regel zorgt ervoor dat *–13* correct ontleed wordt.

Ten derde kan een factor bestaan uit een (, gevolgd door een expressie, gevolgd door). Deze mogelijkheid zorgt ervoor dat haakjes de hoogste prioriteit hebben en dat binnen de haakjes van alles mag staan (zolang het een expressie is).

De factorregel kan als volgt geschreven worden:

```
factor:
        getal ||
        – factor ||
        + factor ||
        ( expressie )
```

Merk op dat deze regel bij de alternatieven voor de unaire min en de unaire plus rechts-recursief is. Dit bewerkstelligt dat bij de toepassing van deze regel de unaire min en unaire plus rechts-associatief zijn.

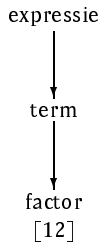
Hoe de regels werken

Regel	Herschijving	
expressie	term	||
	expressie – term	||
	expressie + term	
term	factor	||
	term ∗ factor	||
	term / factor	
factor	getal	||
	– factor	||
	+ factor	||
	(expressie)	

Figuur 16.9: Grammaticale regels voor een expressie.

De grammaticale regels voor een expressie zijn nogmaals in figuur 16.9 weergegeven. In dit onderdeel behandelen wij de feitelijke ontleding van een expressie; met andere woorden *welke* regel *wanneer* van toepassing is. Als voorbeeld zullen we expressie *12 – 3 ∗ –2* 'met de hand' ontleden:

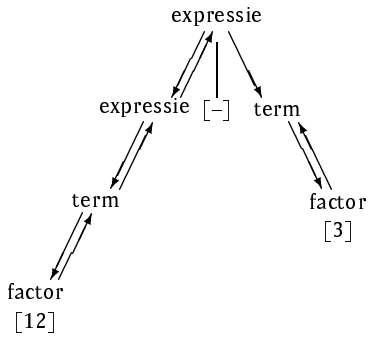
1. Het doel van de ontleding van een expressie is het evalueren van de expressieregel. Uit figuur 16.9 blijkt dat een expressie gedefinieerd is als een serie termen, waarbij de termen door een + of – gescheiden zijn. Het eerste onderdeel van een expressie moet dus een term zijn.
2. Een term bestaat uit één of meer factoren, gescheiden door ∗ of /. Om aan de termregel te voldoen moet het eerste deel van de term dus een factor zijn.
3. In de factorregel moet voor het eerst naar de feitelijke invoer gekeken worden, omdat in de factorregel voor het eerst een keuze gemaakt wordt aan de hand van terminale symbolen. In de factorregel kan hierbij gekozen worden tussen de eindsymbolen getal, –, + en (. Aan de hand van de invoer *12* wordt voor een getal gekozen. De ontleding tot nu toe is in figuur 16.10 weergegeven. In deze figuur is te zien hoe de expressieregel de termregel activeert en hoe



Figuur 16.10: Ontleding van 12 – 3 ∗ –2 (eerste deel).

de termregel op zijn beurt de factorregel activeert. De activering is als een pijl naar beneden afgebeeld. De verwerkte invoer is tussen vierkante haakjes weergegeven.

4. De factorregel is nu klaar. We keren daarom terug in de termregel. In deze regel is intussen één factor gevonden en kan nog een ∗ of / verwacht worden. Geen van deze terminale symbolen komt in de invoer voor — daarom is de termregel klaar. Het gekozen alternatief in de termregel is dus één factor.
   Het terugkeren uit een regel naar een regel van een hogere orde leidt ertoe dat een waarde geretourneerd wordt. In figuur 16.10 betekent dit dat de geanalyseerde deelexpressie (voorlopig) de waarde 12 krijgt.
5. Daarmee keren we terug in de expressieregel. Hier is intussen één term gevonden en kan nog een – of + met een volgende term erachter verwacht worden. Voor de tweede maal wordt een invoersymbool verwerkt: de –. In de expressieregel wordt dus gekozen voor het alternatief expressie – term. Het gevolg van deze keuze is dat het eerste invoersymbool (12) als expressie wordt opgevat en dat de termregel voor de tweede keer gekozen wordt.
6. De termregel stelt dat een term uit minstens één factor kan bestaan. Daardoor wordt de factorregel weer gekozen.
7. In de factorregel wordt voor de derde maal invoer verwerkt. Invoerteken 3 bepaalt dat het getal-alternatief gekozen wordt.
   De ontleding tot nu toe is in figuur 16.11 weergegeven. Evenals in figuur 16.10 is de activering van regels weergegeven als pijlen die naar beneden wijzen. Het terugkeren in een regel is weergegeven door middel van pijlen die naar boven wijzen. Merk op dat de pijlen naar beneden en naar boven een *route* langs de ontleedboom weergeven.

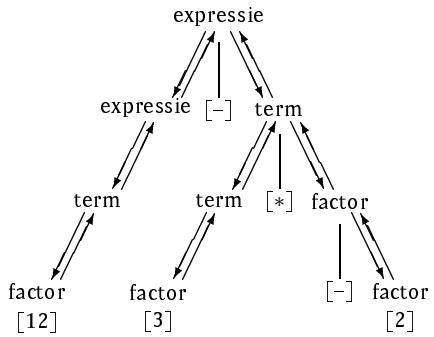


Figuur 16.11: Ontleding van 12 – 3 ∗ –2 (tweede deel).

8. De factorregel is daarmee klaar, en keert terug naar de termregel. In de termregel is inmiddels één factor gevonden en mag een ∗ of / volgen met nog een factor. Hier wordt voor de vierde keer naar de invoer gekeken: het symbool ∗ wordt verwerkt. Op grond van dit teken wordt voor het alternatief 'term ∗ factor' gekozen: de 3 wordt als term opgevat en de factorregel wordt nogmaals gekozen.
9. De factorregel kent vier alternatieven, afhankelijk van het volgende teken in de invoer. Aan de hand van het teken – wordt gekozen voor het alternatief waarbij een factor kan bestaan uit een unaire min, gevolgd door een factor. De unaire min wordt verwerkt en de factorregel wordt gekozen.
10. De (recursief gekozen) factorregel verwerkt het volgende symbool, de 2. Hiermee is de gehele expressie verwerkt. De ontleding is in figuur 16.12 weergegeven.

Aan de hand van dit voorbeeld kunnen wij een aantal eigenschappen van ontledingen met behulp van grammaticaregels beschrijven:

- Elke grammatica kent een *startregel*. Het doel van de ontleding is een zodanige analyse van de invoer dat de startregel voltooid kan worden. Deze regel kan op zijn beurt ondergeschikte regels activeren. In dit voorbeeld is de startregel de expressieregel.
- Naast activering van ondergeschikte regels kan een regel zichzelf activeren. Dit gebeurt in dit voorbeeld onder andere bij de expressieregel. Daarnaast kan het voorkomen dat een ondergeschikte regel een bovengeschikte regel aanroept. Dit gebeurt in dit voorbeeld bij de factorregel, die de expressieregel aanroept als er een (in de invoer staat.



Figuur 16.12: Ontleding van 12 − 3 ∗ −2 (derde deel).

- Elke regel verwerkt zoveel mogelijk van de op dat moment aanwezige invoer. In dit voorbeeld verwerkt de termregel indien mogelijk de eindsymbolen ∗ en /.
- Aan de hand van terminale symbolen in de grammatica besluit een regel welke actie ondernomen moet worden. De expressieregel bijvoorbeeld 'weet' dat een serie termen verwerkt moet worden zolang als de termen door − of + gescheiden zijn.
- Sommige regels vereisen bepaalde invoerelementen. Een voorbeeld hiervan is de factorregel. Als deze regel eenmaal geactiveerd is, dan *moet* in de invoer een −, een +, een (of een getal staan. Bovendien moet, als een (en een expressie verwerkt is, een) volgen.
- De ontleding houdt zich niet bezig met wat er feitelijk als invoer beschikbaar is. Zo wordt een getal *12* niet als de tekens 1 en 2 gelezen, maar wordt verwacht dat de *waarde* 12 op een of andere manier beschikbaar komt. Deze scheiding tussen het ontledings- en het herkenningsdeel van een programma is eerder al in hoofdstuk 11 genoemd. In het te ontwikkelen *scalc* programma wordt deze scheiding opnieuw aangebracht: de invoer wordt door een lexical analyzer voorbewerkt.
- Het uiteindelijke resultaat van een ontleding (in dit voorbeeld de waarde van een expressie) wordt weergegeven door een *route* langs de boom, van links (naar beneden) naar rechts (omhoog). Naar beneden gaand wordt een begrip in een regel uiteengelegd, naar boven gaand retourneert een regel de waarde van een begrip.

16.5.2 Implementatie

Bij de implementatie van het *scalc* programma zullen we elke grammaticale regel door één functie representeren. Dit ligt erg voor de hand; regels kunnen immers andere regels of zichzelf aanroepen net zoals functies dat kunnen. Voor de return-waarden van de regels kiezen we type *double*, omdat de rekenmachine met niet-gehele getallen zal werken (vergelijk het sessievoorbeeld op pagina 345).

De feitelijke constructie van het *scalc* programma zal de constructie betreffen van een lexical analyzer en van de functies die de grammaticale regels voor hun rekening nemen.

De lexical analyzer

De taak van de lexical analyzer is het voorbewerken van invoer tot kant-en-klare brokken[4]. De opdeling in stukken wordt bepaald door de soort invoerelementen die de grammatica van een expressie moet kunnen herkennen. Deze elementen zijn de operatoren $-$, $+$, $*$, $/$ en getal. Van deze elementen heeft alleen het getalsymbool een semantische waarde; de elementen die operatoren representeren hebben alleen een syntactische waarde.

We kiezen hier voor een benadering waarbij de operatoren door hun ASCII-waarde gerepresenteerd worden. Het voordeel hiervan is dat een gelezen operator niet naar een andere representatie geconverteerd hoeft te worden; we kunnen volstaan met de ASCII-waarde van het teken zelf. We spreken dus af dat de syntactische waarde van een operator-teken de ASCII-waarde van het teken zelf is. De syntactische waarde van gelezen invoer slaan we op in een globale variabele *token*.

Een ander geval treedt op als een getal gelezen wordt; immers, getallen hoeven niet uit één ASCII-teken te bestaan. We moeten daarom een syntactische waarde voor het getal afspreken. Omdat de ASCII-tabel van 0 tot en met 255 (*0x0* tot en met *0xff*) loopt, kiezen we 256 (0x100) als syntactische waarde van het getal. De semantische waarde van een getal slaan we op in een globale variabele *value*.

De lexical analyzer van het *scalc* programma is in listing 93 gegeven. In deze functie worden eerst spaties en tabulatietekens in de invoer overgeslagen. Vervolgens wordt gekeken of in de invoer een punt of cijfer voorkomt. Als dit niet het geval is, dan staat de ASCII-waarde van het gelezen teken in *token* en hoeft functie *lexer()* geen verdere actie meer te ondernemen. Als wèl een punt of cijfer gelezen is, dan verwacht *lexer()* een getal. Het gelezen ASCII-teken wordt eerst in de invoer 'teruggeduwd'. Het terugduwen van een ASCII-waarde gebeurt door middel van de standaardfunctie *ungetc()*. Vervolgens wordt de standaardfunctie *scanf()* aangeroepen om een getal in te lezen. Doordat een teken in de invoer teruggeduwd kan worden is het mogelijk om als het ware in de invoer te 'spieken'.

4. Kant-en-klare kubats worden door de lexical analyzer niet verwerkt.

```
1     #define NUM     0x100
2
3     double
4         value;
5     int
6         token;
7
8     void lexer (void)
9     {
10        while ( (token = getchar ())  == ' ' ||       // negeer spaties
11                token == '\t'                         // en tabulaties
12              )
13            ;
14
15        if ( token == '.' || isdigit (token) )        // als . of getal:
16        {
17            ungetc (token, stdin);                    // zet teken terug
18            scanf ("%lf", &value);                    // bepaal 'value'
19            token = NUM;                              // geef aan dat 't
20        }                                             // een getal is
21    }
```

Listing 93: De functie *lexer()*.

De functie main()

De functie *main()* van het *scalc* programma zorgt voor herhaaldelijke activering van de expressieregel. Vooruitlopend op de implementatie van de grammaticale regels is de expressieregel ondergebracht in een functie *expression()*, die een *double* retourneert. De functie *main()* roept *lexer()* aan om een teken uit de invoer te lezen. Zolang als dit teken geen \n is, roept *main()* de functie *expression()* aan om de invoer te analyseren[5]. De code van deze functie is in listing 94 gegeven.

```
1     #include <ctype.h>
2     #include <process.h>
3     #include <stdio.h>
4     #include <stdlib.h>
5
6     void main (void)
7     {
8         puts ("SCALC : Simple four-function calculator\n"
9               "Enter expressions using * / + - .\n"
10              "Sub-expressions may be enclosed in ( and ).\n"
11              "Just press the enter key when done.\n");
12
13        while (1)
14        {
15            lexer ();
16            if (token == '\n')
17                break;
18            printf ("\t = %f\n\n", expression ());
```

5. Een andere aanpak is om de herhaling van een expressie in een grammaticale regel op te nemen. Hierbij zou een 'super-regel', bijvoorbeeld *input*, gebruikt worden. Volgens deze regel zou invoer bestaan uit nul of meer expressies, gescheiden door \n-tekens.

```
19        }
20    }
```

Listing 94: De functie *main()*.

De functie expression()

De functie *expression()* is de implementatie van de expressieregel van figuur 16.9. Volgens deze regel bestaat een expressie uit minstens één term. Een serie termen wordt gescheiden door – of +.

De functie *expression()* is in listing 95 gegeven. In deze code wordt eerst één termwaarde bepaald. Dit gebeurt door de functie *term()* aan te roepen. Deze functie retourneert, analoog aan *expression()*, een *double* waarde. Vervolgens wordt gekeken of het volgende invoerelement een – of + is. Zo ja, dan wordt *term()* nogmaals aangeroepen. Merk op dat als *expression()* een invoersymbool verwerkt, de functie *lexer()* aangeroepen wordt om een nieuw invoerelement op te halen. De functie *expression()* verwerkt alleen de symbolen – en +.

```
1     double expression (void)
2     {
3         double
4             ret;                                    // return-waarde
5
6         ret = term ();                              // bepaal een term
7
8         while (token == '+' ||                      // nog een term?
9                token == '-'
10             )
11            if (token == '+')                       // ja: optelling
12            {
13                lexer ();
14                ret += term ();
15            }
16            else                                    // ja: aftrekking
17            {
18                lexer ();
19                ret -= term ();
20            }
21
22        return (ret);                               // nee: retourneer
23    }
```

Listing 95: De functie *expression()*.

De functie term()

De functie *term()* is de herschrijving van de termregel in figuur 16.9. Analoog aan *expression()* geldt:

- dat *term()* minstens één factor op moet halen;

- dat *term()* een serie factoren, gescheiden door $*$ of / moet kunnen verwerken;
- dat *term()* de factorwaarden verenigt tot één return-waarde.

Opgave 90.
Implementeer de functie *term()* aan de hand van bovenstaande specificaties. Gebruik de brontekst van de functie *expression()* als voorbeeld.

De functie factor()

De functie *factor()* is als enige van de grammaticale regels direct recursief geïmplementeerd. Indirecte recursie treedt ook op, omdat de functie *expression()* aangeroepen kan worden door *factor()*. De directe recursie wordt in de functie *factor()* gebruikt om een factorwaarde te bepalen als een unaire min of plus gelezen wordt. Merk tevens op dat zodra *factor()* een invoersymbool verwerkt, *lexer()* aangeroepen wordt om een nieuw invoersymbool te bepalen.

De functie *factor()* verwacht dat een –, een +, een (of een getal verwerkt moet worden. Als aan geen van deze voorwaarden voldaan wordt, roept *factor()* de functie *error()* aan om een foutmelding af te drukken en het programma te beëindigen. De functie *error()* is in listing 97 weergegeven.

```
1     double factor (void)
2     {
3         double
4             ret;
5
6         switch (token)
7         {
8             case '-':                                   // unaire min
9                 lexer ();
10                return (-factor ());
11
12            case '+':                                   // unaire plus
13                lexer ();
14                return (factor ());
15
16            case '(':                                   // ( expression )
17                lexer();
18                ret = expr ();
19                if (token != ')')
20                    error ("') expected");
21                lexer ();
22                return (ret);
23
24            case NUM:                                   // getal
25                lexer ();
26                return (value);
27
```

```
28            default:
29                error ("+ or - or NUM or ( expected");
30        }
31    }
```

Listing 96: De functie *factor()*.

```
1     void error (char *msg)
2     {
3         puts (msg);                                   // geef melding
4         exit (1);                                     // en sterf
5     }
```

Listing 97: De functie *error()*.

Opgave 91.
Voer de bronteksten van het *scalc* programma in. Merk op dat het prototype van *expression()* vóór *factor()* moet worden gegeven. Vertaal en link de bronteksten.

Opgave 92.
Als in een expressie een deling door nul voorkomt (bijvoorbeeld in *12 / (3 – 2 – 1)*), is dan sprake van een syntactische fout? In welke functie kan deze fout het beste ondervangen worden?
Implementeer de uitbreiding waarbij deling door nul een fout oplevert. De expressie *12 / (3 – 2 – 1)* zou dan tot een foutmelding *"Division by zero."* leiden.

16.5.3 Een uitbreiding

In dit onderdeel zullen we met behulp van een aantal opgaven het *scalc* programma uitbreiden met machtsverheffing. Aan de hand van deze opgaven wordt de grammatica voor een expressie uitgebreid.

We zullen de machtsverheffing met het ^-teken aangeven. Bij deze bewerking geldt het volgende:

- Machtsverheffing is rechts-associatief. Zo is de expressie *2^3^4* gelijk aan *2^(3^4)*.
- Machtsverheffing heeft in een expressie een gelijke prioriteit als de unaire operatoren. Alleen de haakjes hebben een hogere prioriteit.

Omdat bij een expressie de haakjes de hoogste prioriteit hebben, introduceren we een nieuwe grammaticale regel: de unit. Een unit kan een getal of een expressie tussen haakjes zijn:

```
unit:
    getal ||
    ( expressie )
```

Opgave 93.
Stel de functie *unit()* op zodanig dat aan de bovenstaande grammaticale regel wordt voldaan. Deze functie moet bij de al ontwikkelde functies van het SCALC programma passen; dat wil zeggen dat het *return*-type van deze functie *double* moet zijn, dat de functie *lexer()* moet worden aangeroepen, enzovoort.
Zorg voor een passende foutmelding als de functie zijn werk niet kan doen.

Het opnemen van de unitregel bij de andere grammaticale regels gebeurt in de factorregel. In deze regel wordt tevens de ^ operator gedefinieerd. Als factor kan het volgende voorkomen:

- Ten eerste kan een factor bestaan uit een unaire min, gevolgd door een factor. Dit alternatief is al eerder, op pagina 348, beschreven.
- Ten tweede kan, analoog aan de unaire min, een factor voorafgegaan worden door een unaire plus.
- Ten derde kan een factor bestaan uit één of meer units, gescheiden door ^-tekens.

Opgave 94.
Herschrijf de regel voor een factor aan de hand van de hierboven genoemde alternatieven. Let erop dat het derde alternatief (een serie van één of meer units, gescheiden door ^-tekens) een rechts-recursieve regel moet zijn. De machtsverheffing is tenslotte rechts-associatief.

Opgave 95.
Teken de ontleedboom voor de expressie *–2^3^–4*. Zet bij de boom de berekende tussenresultaten die door de grammaticale regels opgeleverd worden. Zet tevens op de juiste plaatsen *waar* en *welke* invoer verwerkt wordt.

Opgave 96.
Implementeer de grammaticale regel uit opgave 94 als de functie *factor()*.

Hoofdstuk 17

Gevorderde datatypen

17.1 Inleiding

In dit hoofdstuk worden een aantal bijzondere datastructuren behandeld. De behandelde structuren zijn bijzonder in de zin dat ze bij veel programmeerproblemen een rol spelen, maar niet standaard als datatype beschikbaar zijn. Met andere woorden: de hier behandelde structuren moeten expliciet door de software-ontwikkelaar worden geprogrammeerd. De datastructuren die in dit hoofdstuk de revue passeren kunnen niettemin tot het standaard gereedschap van de software-ontwikkelaar worden gerekend. Knuth (1979) behandeld de hier besproken datastructuren samen met het array, maar met uitzondering van het zogenaamde hash-algoritme, als fundamentele datastructuren. Het hash-algoritme wordt door Knuth behandeld als een specifiek zoekalgoritme (Knuth, 1973). Het onderscheid ligt voor de hand, omdat hashen een speciale manier is om informatie in een array op te zoeken, en dus niet zozeer een datastructuur is. Toch is het in onze optiek wel te verdedigen het hash-algoritme bij de bijzondere datastructuren te behandelen: hoewel het hash-algoritme gewoonlijk gebruik maakt van een array waarin de beschikbare informatie is opgeslagen, is de manier waarop de informatie in het array wordt opgeslagen sterk afwijkend van alle tot nu toe behandelde manieren om dat te doen. Het hash-algoritme werkt daarmee op een speciale datastructuur, zodat het hash-algoritme bij het huidige onderwerp niet is misplaatst.

Bij het ontwerp van complexe programmatuur is het opstellen van de datastructuren één van de belangrijkste stappen die moet worden genomen: het is in de praktijk veel eenvoudiger eerst een goed overzicht te hebben van de organisatie van de data om vervolgens de code te ontwikkelen waarmee de data kunnen worden gemanipuleerd, dan het is om eerst code te ontwikkelen voor datastructuren die al doende worden geconstrueerd. In dat verband is de 'formule':

Data Structures + Algorithms = Programs

wellicht een betere dan het omgekeerde:

Algorithms + Data Structures = Programs

hetgeen de titel is van een (overigens zeer de moeite van het lezen waard) boek van Wirth (Wirth, 1976), één van de auteurs van de programmeertaal **pascal** (Jensen, 1985).

17.2 Stacks

De stack is op verschillende plaatsen (bijvoorbeeld in hoofdstuk 5 over functies) al ter sprake geweest. Met de stack werd steeds de datastructuur bedoeld die door programma's wordt gebruikt om tijdelijke informatie in op te slaan, zoals lokale variabelen en return-adressen van functies. De manipulatie van de systeem-stack is niet iets waar de software-

ontwikkelaar zich mee bezig hoeft te houden: de vertaler genereert bij de vertaling van een **C** brontekst de nodige instructies voor de correcte bewerking van de stack. De stack kan echter als datastructuur ook heel goed worden gebruikt in programma's waarin tijdelijke informatie moet worden opgeslagen. De systeem-stack blijft ongemoeid, maar er wordt een aparte stack gedefinieerd voor de opslag van tijdelijke informatie die van andere aard is dan die welke in de systeem-stack wordt opgeslagen.

Het essentiële kenmerk van de stack is dat het een zogenaamde 'lifo' (Last In, First Out) of 'filo' (First In, Last Out) structuur is. De informatie die het eerst op de stack wordt gezet, wordt er het laatst uit verwijderd, terwijl de informatie die het laatst op de stack wordt gezet er het eerst uit wordt verwijderd. Dit kenmerk bepaalt sterk de aard van de tijdelijke informatie waarbij een stack gebruikt kan worden. Stacks kunnen vooral worden toegepast in situaties waarbij deelprocessen worden uitgevoerd die tijdelijk informatie moeten kunnen opslaan. Zodra het deelproces is afgelopen, is ook de noodzaak voor de opslag van de tijdelijke informatie vervallen. Dergelijke situaties komen zeer veel voor. Uiteraard kan de verwerking van een programma in termen van hoofd- en deelprocessen worden beschreven. In het algemeen kan de bewerking van recursieve datastructuren ook in termen van hoofd- en deelprocessen worden beschreven. In het hoofdstuk over grammatica's (hoofdstuk 16) zijn de regels van een rekenkundige expressie genoemd. Rekenkundige expressies kunnen worden beschouwd als een recursieve datastructuur, omdat de 'expressie' uiteindelijk in de definitie van een expressie wordt gebruikt (bijvoorbeeld bij het begrip 'factor'). In dit deel van het hoofdstuk zullen we een toepassing ontwikkelen waarbij stacks een belangrijke functie vervullen.

Op stacks kunnen twee bewerkingen worden uitgevoerd:

- Een element kan op de stack worden geplaatst. Hierbij wordt een element bovenop de stack geplaatst en wordt het laatst geplaatste element aan het zicht onttrokken.
- Het bovenste element kan weer van de stack worden gehaald, wat ertoe leidt dat het voorlaatst geplaatste element weer zichtbaar wordt.

Wanneer wordt geprobeerd een element van de stack te verwijderen treedt er een fout op. Gesproken wordt van een 'leeggestroomde stack' (of: stack underflow). Het omgekeerde kan ook voorkomen doordat de stack in de praktijk niet onbegrensd groot is. Wanneer de laatste plaats in de stack is gevuld en een poging wordt ondernomen om nog een element op de stack te plaatsen, treedt er opnieuw een fout op. In dit geval is er sprake van een 'overstroomde stack' (of: stack overflow).

Bij het gebruik van zelfgedefinieerde stacks zijn er maar twee manieren om de informatie in de stack te manipuleren. Elementen kunnen op de stack worden geplaatst (to push) en het bovenste element kan van

de stack worden verwijderd (to pop). Een functie *pop()* levert het eerstvolgende element van de stack, en genereert een foutmelding bij een lege stack. Een functie *push()* plaatst een argument op de stack, en genereert een foutmelding bij een volle stack. Bij de implementatie van de functies *push()* en *pop()* kan de stack in hetzelfde bestand worden gedefinieerd als de functies *push()* en *pop()*. De stack wordt bij voorkeur gedefinieerd als een statische variabele, omdat op die manier direct duidelijk is dat de stack alleen via de functies *push()* en *pop()* wordt gemanipuleerd. Er is hier sprake van 'data hiding': de stack zelf blijft verborgen voor de overige functies van het programma. Overige functies gebruiken de stack alleen via de interface-functies *push()* en *pop()*.

Naast de stack zelf is er nog een variabele die de plaats van het laatst gevulde element in het stack-array aangeeft. Deze variabele is de stack pointer, die eenvoudig als een *int* kan zijn gedefinieerd en dan als indexvariabele fungeert.

Elementen kunnen ofwel vanaf de hoogste index naar de laagste index op de stack worden geplaatst, ofwel vanaf de laagste index naar de hoogste index. In het eerstgenoemde geval 'groeit' de stack van het hoogste adres naar het laagste adres. Deze methode wordt bij de systeemstack gebruikt. Er is geen speciale noodzaak om die methode over te nemen. Het voordeel van de tweede methode (de stack groeit van laag naar hoog) is dat bij een dreigende stack-overflow nog kan worden geprobeerd de stack te realloceren.

```
1         static int
2             stack[MAX_STACK_SIZE],
3             sp = MAX_STACK_SIZE;
4
5         void push(int value)
6         {
7             if (!sp)
8                 error("Stack overflow");
9
10            stack[--sp] = value;
11        }
12
13        int pop()
14        {
15            if (sp == MAX_STACK_SIZE)
16                error("Stack underflow");
17
18            return (stack[sp++]);
19        }
```

Listing 98: Eenvoudige *push()* en *pop()* functies.

De implementatie van de functies *push()* en *pop()* is zeer eenvoudig. In listing 98 zijn twee functies *push()* en *pop()* gedefinieerd die

een stack van *ints* bewerken. Elementen worden vanaf het hoogste stack-adres in de richting van het laagste stack-adres op de stack gezet. De stack 'groeit' van het hoogste adres naar het laagste adres. Het verwijderen gaat omgekeerd: van het laagste adres naar het hoogste adres. De aangeroepen functie *error()* en de constante *MAX_STACK_SIZE* worden verondersteld elders te zijn gedefinieerd.

17.2.1 Het rangeerprobleem

Het probleem dat met behulp van stacks zal worden opgelost heeft betrekking op het rangeren met goederenwagons. Het probleem kan als volgt worden geformuleerd:

> *Bij het dorpje Oostum ligt over het Reitdiep een oude spoorbrug die naar een steenfabriek leidt. Bij de steenfabriek is een eenvoudig rangeeremplacement waarover de rangeermeester Been de scepter zwaait. Een schets van de situatie is gegeven in figuur 17.1. De trein die uit Groningen komt bestaat uit een locomotief en een zeker aantal wagons, die genummerd zijn vanaf 1. De volgorde van de wagons is echter zelden zodanig dat het laden van de wagons efficiënt kan gebeuren, omdat de wagons bij aankomst niet op steensoort zijn gesorteerd. Wel krijgt rangeermeester Been de vrachtbrieven van de afzonderlijke wagons. Het is zijn taak een zodanige rangeeropdracht te maken dat de wagons gesorteerd naar gewenste steensoort de fabriek binnenkomen. Dat stelde meester Been bij herhaling voor grote problemen. Op een dag bracht een kennis van meester Been hem in contact met het software-bedrijf Plomp B.V., en al snel werd men het eens over de ontwikkeling van een computerprogramma waarmee de rangeeropdrachten konden worden voorbereid. In het vervolg hoeft meester Been nog slechts de gewenste volgorde in te typen, waarna de rangeeropdrachten op het beeldscherm van zijn computer verschijnen.*

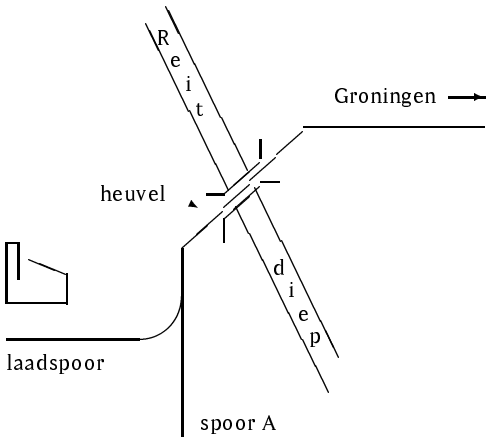
Bij de brug over het Reitdiep moet de trein een hoogteverschil overwinnen. Eenmaal over de brug loopt het spoor langzaam naar beneden totdat uiteindelijk het laadspoor of spoor A als het laagste punt is bereikt.

17.2.2 Analyse van het probleem

Bij de analyse van het probleem constateert de heer Plomp van de firma Plomp B.V. dat rangeerspoor A het karakter heeft van een stack. Immers, de wagon die het eerst op spoor A binnenkomt, moet als laatste spoor A weer verlaten. Verder is bekend dat de wagons genummerd zijn vanaf nummer 1. Het lijkt het eenvoudigst voor meester Been om het programma *rangeer* als argument een aantal getallen te geven waarmee de gewenste volgorde van aankomst bij de steenfabriek wordt weergegeven. Het programma kan dan bijvoorbeeld worden gestart als:

rangeer 3 4 2 1

in welk geval de trein op de spoorbrug verschijnt als de locomotief, gevolgd door wagon 1, 2, 3 en 4. De volgorde die aan het programma wordt meegegeven, geeft aan dat de wagons in de volgorde 3, 4, 2 en 1 bij de steenfabriek moeten binnenkomen. Uiteindelijk vertrekt de trein weer in



Figuur 17.1: Het rangeeremplacement bij Oostum.

de richting Groningen als de locomotief, gevolgd door de wagons 1, 2, 4 en 3. Om dat te bereiken moeten de locomotief en de wagons 1 en 2 eerst op spoor A worden geplaatst. Dan kunnen de wagons 3 en 4 naar de steenfabriek rijden. Vervolgens worden de wagons 1 en 2 door de locomotief weer op de rangeerheuvel geduwd. De locomotief en wagon 1 rijden dan weer naar spoor A, zodat wagon 2 naar de steenfabriek kan rijden. Daarna wordt wagon 1 weer op de heuvel geduwd. Nadat de locomotief weer op spoor A is gezet kan ook wagon 1 naar de steenfabriek. Tenslotte haalt de locomotief via de rangeerheuvel de geladen trein op van de steenfabriek om te vertrekken in de richting Groningen.

De heuvel bij de brug kan dienst doen als rangeerheuvel, van waaruit de wagons vanzelf naar het laadspoor of naar het rangeerspoor A kunnen rollen. De heer Plomp komt met de volgende herformulering van het probleem, waarbij spoor A als een stack is geïmplementeerd. Een tweede stack wordt gebruikt om de wagons die in eerste instantie op de heuvel verschijnen mee te representeren:

1. Plaats alle wagonnummers in omgekeerde volgorde op de stack die de rangeerheuvel representeert, waarbij de locomotief als wagon 0 op de stack wordt geplaatst.
2. Doe het volgende zolang er nog wagons op de heuvel staan:
    3. Plaats de nummers van alle wagons tot het volgende gewenste wagonnummer op de stack voor spoor A.
    4. Plaats het nummer van de wagon die vooraan op de heuvel verschijnt op het laadspoor.

5. Als het nummer van de volgende gewenste wagon kleiner is dan het nummer van de wagon die net op het laadspoor is geplaatst,
    6. laat de locomotief dan alle wagons op de heuvel terugduwen en
    7. plaats de locomotief weer op de stack voor spoor A.
8. Rijd de locomotief via de heuvel naar het laadspoor.

De implementatie van het programma werd vervolgens door de heer Plomp in een half uurtje gerealiseerd.

17.2.3 Implementatie

```
1         #define MAX_TRAIN_LENGTH    50
2
3         typedef enum
4         {
5             hilltrack,
6             track_A,
7         } TRACKS_;
8
9         static int
10            stack[2][MAX_TRAIN_LENGTH],
11            sp[2] =
12            {
13                MAX_TRAIN_LENGTH,
14                MAX_TRAIN_LENGTH
15            };
16
17        void push(TRACKS_ trackname, int carnr)
18        {
19            if (!sp[trackname])
20                error("Stack overflow");
21
22            stack[trackname][--sp[trackname]] = carnr;
23        }
24
25        int pop(TRACKS_ trackname)
26        {
27            if (sp[trackname] == MAX_TRAIN_LENGTH)
28                error("Stack underflow");
29
30            return (stack[trackname][sp[trackname]++]);
31        }
```

Listing 99: De stacks en de functies *push()* en *pop()* van *rangeer.c*.

Bij de implementatie van het programma wordt begonnen met de functies waarmee de stacks kunnen worden gemanipuleerd. Daarna wordt de rest van het programma geconstrueerd. Het programma *rangeer* gebruikt twee stacks. Eén stack wordt gebruikt om de rangeerheuvel te representeren, een andere stack wordt gebruikt om spoor A mee te representeren.

Beide stacks worden gedefinieerd in één module, samen met de functies *push()* en *pop(),* die als eerste argument de *enum* naam van de gewenste stack krijgen. Voor de stackgrootte wordt een vaste omvang gekozen, die is vastgelegd in de constante *MAX_TRAIN_LENGTH*. [1]. De *enum,* stacks en de functies *push()* en *pop()* zijn gegeven in listing 99 en behoeven geen extra commentaar. De functie *error()* is een oude bekende: de foutmelding wordt afgedrukt en het programma wordt beëindigd. De functie *error()* is hier niet verder gedefinieerd.

Het programma kan nu worden ontwikkeld aan de hand van de analyse van het rangeerprobleem (zie 17.2.2). Onderdelen 1, 3, 6 en 7 zijn geïmplementeerd als aparte functies, onderdelen 2 en 5 als respectievelijk een *for*- en *if*-statement. Onderdeel 4 is gerealiseerd met behulp van een *printf()* aanroep waarmee de opdracht wordt afgedrukt om een wagon van de heuvel naar het laadspoor te laten rollen en onderdeel 8 tenslotte is gerealiseerd door een aanroep van de functie *puts()*.

```
1       void main(int argc, char **argv)
2       {
3           int
4               requested,
5               next_car = 1;
6
7           initialize_hill(argc);                          // Tekst: 1.
8           for (next_car = 1; next_car < argc; next_car++)// Tekst: 2.
9           {
10              requested = atoi(argv[next_car]);
11
12              to_track_A(requested);                      // Tekst: 3.
13              printf("Wagon %d naar"                      // Tekst: 4.
14                     "het laadspoor\n", requested);
15
16              if                                          // Tekst: 5.
17              (
18                  next_car < argc - 1
19                  &&
20                  atoi(argv[next_car + 1]) < requested
21              )
22              {
23                  from_A_to_hill();                       // Tekst: 6.
24                  loc_to_track_A();                       // Tekst: 7.
25              }
26          }
27          puts("Locomotief via de heuvel"                 // Tekst: 8.
28              "naar de wagons op het laadspoor");
29      }
```

1. *MAX_TRAIN_LENGTH* is gedefinieerd als de waarde 50. Hierbij hebben we ons gebaseerd op de heer Sulkers, oud-medewerker van de NS, die de maximale lengte van een trein in Nederland relateerde aan het inhaalspoor bij Putten, dat 600 meter lang is. Rekening houdend met de gemiddelde wagonlengte zullen treinen in Nederland hoogstens uit 50 wagons bestaan.

Listing 100: De functie *main()* van *rangeer.c*.

De listing van de functie *main()* is gegeven in listing 100. De functie maakt gebruik van twee hulpvariabelen, beide *ints*. De eerste hulpvariabele is *requested* waarin de waarde wordt opgeslagen van het eerstvolgende wagonnummer. Het eerstvolgende wagonnummer is als string beschikbaar als element *next_car* van *argv*. De variabele *next_car* loopt daarom in de implementatie van onderdeel 2 van 1 (*argv[1]* is het nummer van de eerst gewenste wagon) tot en met *argc - 1*. (Het nummer van de laatst gewenste wagon is *argv[argc - 1])*. In de implementatie van onderdeel 5 wordt gekeken of er nog meer dan één wagons zijn *(next_car < argc - 1)* en zo ja, of het nummer van de volgende gewenste wagon groter is dan het nummer van de huidige wagon. Als dat het geval is, bevindt de volgende wagon zich nog op de heuvel. Als dat niet het geval is, bevindt de volgende wagon zich ergens op spoor A en moet de locomotief de wagons terugduwen op de heuvel.

De functie *initialize_hill()* plaatst alle nummers teruglopend vanaf *argc* tot en met *0* op de heuvel-stack. De functie is als één *while*-statement geïmplementeerd (zie listing 101).

```
1       void initialize_hill(int argc)
2       {
3           while (argc >= 0)
4               push(hilltrack, argc--);
5       }
```

Listing 101: De functie *initialize_hill()* van *rangeer.c*.

De functie *to_track_A()* krijgt als argument het nummer van de eerste wagon die op het laadspoor moet komen. Totdat dit nummer bovenaan de heuvel-stack verschijnt, worden nummers van deze stack verwijderd en op de stack van spoor A geplaatst. Het gezochte nummer kan bij deze bewerking ook van de stack worden verwijderd, omdat bekend is dat de desbetreffende wagon vervolgens naar het laadspoor wordt gestuurd. De implementatie van de functie is gegeven in listing 102.

```
1       void to_track_A(int request)
2       {
3           int
4               popped;
5
6           while ((popped = pop(hilltrack)) != request)
7           {
8               if (!popped)
9                   puts("Locomotief naar spoor A");
10              else
11                  printf("Wagon %d naar spoor A\n", popped);
12
```

```
13                push(track_A, popped);
14            }
15        }
```

Listing 102: De functie *to_track_A()* van *rangeer.c*.

De functie *from_A_to_hill()* geeft een melding dat alle wagons door de locomotief de heuvel op worden geduwd, en verwijdert dan alle elementen van de stack van spoor A. De verwijderde elementen worden direct na verwijdering op de stack van de heuvel geplaatst. Het is bekend wanneer de stack van spoor A leeg is doordat de locomotief altijd als eerste op spoor A wordt geplaatst. Zodra het nummer van de locomotief van de stack van spoor A is verwijderd is die stack dus, op grond van het 'first in last out' principe, leeg. De listing van de functie *from_A_to_hill()* is gegeven in listing 103.

```
1         void from_A_to_hill()
2         {
3             int
4                 popped;
5
6             puts("De locomotief duwt alle wagons van spoor A de heuvel op");
7             do
8                 push(hilltrack, popped = pop(track_A));
9             while (popped);
10        }
```

Listing 103: De functie *from_A_to_hill()* van *rangeer.c*.

De functie *loc_to_track_A()* tenslotte geeft de melding dat de locomotief van de heuvel naar spoor A moet rijden, en realiseert dat door een element van de heuvel-stack te verwijderen en het verwijderde element op de stack voor spoor A te plaatsen. Opnieuw wordt gebruik gemaakt van een essentieel kenmerk van stacks: omdat de locomotief als laatste op de stack van de heuvel is geplaatst (bij de aanroep van de vorige functie *from_A_to_hill()*) kan nu op grond van het 'last in first out' principe worden afgeleid dat het nummer van de locomotief dus als eerste weer van de stack van de heuvel wordt verwijderd.

```
1         void loc_to_track_A()
2         {
3             puts("Locomotief naar spoor A");
4             push(track_A, pop(hilltrack));
5         }
```

Listing 104: De functie *loc_to_track_A()* van *rangeer.c*.

Opgave 97.
Stel het programma *rangeer* samen aan de hand van de listings 99 tot en met 104. De functie *error()* moet nog worden gedefinieerd.

Opgave 98.
Het programma *rangeer* werkt niet goed wanneer wagonnummers bij de argumenten van het programma ontbreken. Breid het programma uit met een functie *checkargs()* waarmee wordt gecontroleerd of alle nummers van 1 tot en met *argc* bij de invoer zijn meegegeven. De functie krijgt als argumenten *argc* en *argv*. Wanneer een wagonnummer ontbreekt moet het programma worden beëindigd met een melding waarin het ontbrekende nummer wordt vermeld. Gebruik hiervoor de functie *error()* (eventueel in combinatie met een *printf()* aanroep, wanneer *error()* niet voor een variabel aantal argumenten geschikt is gemaakt). Waar moet de functie *checkargs()* worden aangeroepen in de functie *main()?*

17.3 Queues

Het volgende bijzondere datatype dat in dit hoofdstuk wordt behandeld is de queue. Een queue is een wachtrij (of in het Nederlands: file). Bij de queue wordt het element dat het eerst in de queue wordt geschreven er ook weer het eerst uit verwijderd. De queue is dus een 'First In, First Out' (fifo) datastructuur, en geen 'First In, Last Out' structuur zoals de stack.

De queue wordt met name gebruikt bij zogenaamde producent-consument (producer-consumer, Tannenbaum, 1987) processen, waarbij datgene dat door de producent wordt geproduceerd door de consument wordt verwerkt. De termen 'producent' en 'consument' moeten hier niet al te letterlijk worden opgevat: een producent kan bijvoorbeeld een patiënt zijn die met klachten in de spreekkamer van een arts komt. De consument is hier de arts die de patiënt 'verwerkt'.

Wanneer in computerprogramma's producent-consument processen worden geïmplementteerd, dan delen deze processen een gemeenschappelijke buffer. Omdat de buffer niet eindeloos groot is, kunnen er problemen ontstaan wanneer de producent iets in de buffer wil zetten, terwijl de buffer vol is. In dat geval moet, in Tannenbaum's termen, de producent maar even gaan slapen, om later nog maar eens weer te proberen iets in de buffer te zetten. Omgekeerd kan het voorkomen dat de consument iets uit de buffer wil verwijderen terwijl er niets is geproduceerd. In dat geval moet de consument maar even gaan slapen. De term 'slapen' is voor gebruikers van UNIX bedrijfssystemen niet vreemd. Het UNIX bedrijfssysteem is een 'multi tasking' systeem, waarbij de computer naast elkaar meerdere programma's (tasks) kan verwerken. Wanneer consument- en producent-processen in dit systeem op elkaar moeten wachten, betekent dat niet dat de computer dan niets anders kan doen: andere programma's kunnen gewoon verder worden verwerkt. Bij PC-achtige compu-

ters is er geen sprake van multitasking, maar kan een proces een ander proces onderbreken (interrumperen). Met behulp van dergelijke interrupties kan een producent-consument systeem worden gerealiseerd voor bijvoorbeeld de toetsenbordbuffer: telkens wanneer een toets wordt ingedrukt wordt een 'hardware interrupt' gegenereerd, waarbij de waarde van de ingedrukte toets in de toetsenbordbuffer wordt geplaatst. Het toetsenbord is dus de producent. Aan de andere kant is er het programma, de consument, dat de ingetypte toetsen weer uit de buffer verwijdert. Op PC-achtige computers kan het programma niet gaan slapen wanneer er niets meer in de buffer staat, maar is het programma 'busy waiting': het programma voert een lus uit waarin steeds wordt gekeken of er iets in de buffer staat. De lus wordt doorbroken zodra er iets in de buffer verschijnt. Een blauwdruk van een programma waarin een toets wordt verwerkt zodra een toets in de toetsenbordbuffer verschijnt, is gegeven in listing 105. Hierbij wordt gebruik gemaakt van de standaardfunctie *kbhit()* die de waarde nul oplevert wanneer er geen toets in de toetsenbordbuffer wacht, en die een niet-nul waarde oplevert wanneer er wèl een toets in de toetsenbordbuffer wacht.

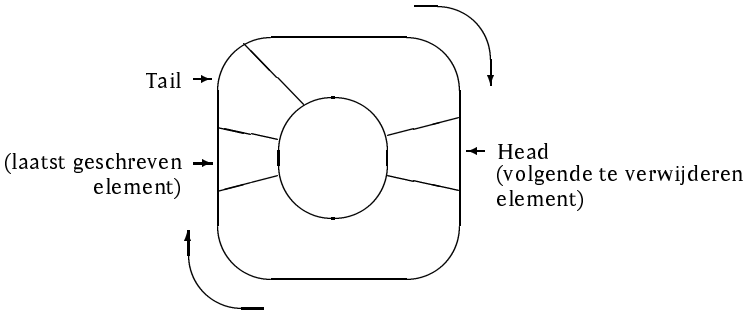
```
1       void main()
2       {
3           while (1)
4           {
5               while (!kbhit())
6                   ;                                   // busy waiting
7               process_one_key();
8           }
9       }
```

Listing 105: De blauwdruk van het busy waiting programma.

De buffer die wordt gebruikt bij producent-consument processen bestaat in de praktijk uit een array, dat wordt behandeld als een circulaire buffer. In overeenstemming met de terminologie die bij de toetsenbordbuffer wordt gebruikt door IBM[2] zullen we spreken van de kop (head) en de staart (tail) van de queue.

Informatie wordt aan de staart van de queue bijgeschreven, en aan de kop van de queue gelezen. In figuur 17.2 is een circulaire buffer weergegeven waarin is aangegeven waar het volgende te lezen element zich bevindt en waar het laatst geschreven element zich bevindt. Op grond van deze figuur kan een blauwdruk voor functies waarmee een queue wordt gemanipuleerd, worden geconstrueerd. De functie *write_queue()* krijgt als argument een waarde die in de queue moet worden gezet, en levert de waarde nul op wanneer dat lukt, en een niet-nul waarde wanneer de queue

2. De code die wordt gebruikt om uit de toetsenbordbuffer te lezen is bijvoorbeeld gedocumenteerd in IBM, 1983, pagina A-26, regel 1854 tot en met 1864.



Figuur 17.2: De circulaire buffer (queue).

vol is. De functie *read_queue()* krijgt als argument het adres van de variabele die de waarde van het volgende queue-element moet krijgen. Deze functie levert de waarde 0 op wanneer er een volgende waarde is gelezen, en levert een niet-nul waarde op wanneer de queue leeg is. Het lezen van de queue verloopt als volgt:

- Wanneer de staartpositie en de koppositie aan elkaar gelijk zijn, is de queue leeg en wordt een niet-nul waarde geretourneerd.
- De waarde aan de koppositie wordt toegekend aan de variabele waarvan het adres aan de functie *read_queue()* is meegegeven.
- De kopindex wordt verhoogd. Omdat de circulaire buffer met een lineair array wordt geïmplementeerd, moet de kopindex bij het overschrijden van de laatste arrayindex de waarde 0 krijgen[3].
- De waarde nul wordt geretourneerd.

Het schrijven in de queue verloopt als volgt:

- De volgende positie van de staartindex wordt berekend als de verhoogde huidige staartindex modulo het maximaal aantal elementen dat in de queue kan worden opgenomen.
- Wanneer de volgende positie van de staartindex gelijk is aan de koppositie is de queue vol, en wordt een niet-nul waarde geretourneerd. Merk op dat een gelijke waarde van de kop- en staartindex

3. Het verhogen van een variabele binnen een beperkt waardebereik kan met behulp van de modulo bewerking in één statement worden uitgevoerd, zoals:

index = ++index % N_ELEMENTS;

Echter, de modulo bewerking is relatief traag. Een alternatief is een *if*-statement waarin de waarde van *index* na verhoging wordt vergeleken met *N_ELEMENTS*. Het *if*-statement kost weinig meer ruimte aan code en is sneller.

al betekent dat de queue leeg is. Bij het schrijven moet dus worden voorkomen dat de kopindex gelijk wordt aan de staartindex.
- De te schrijven waarde wordt op de staartpositie geschreven en de staartindex krijgt de waarde van de al berekende volgende positie.
- De waarde 0 wordt geretourneerd.

Omdat de situatie waarbij de kopindex en de staartindex aan elkaar gelijk zijn de lege queue representeert, kunnen de beide indexvariabelen op twee gelijke waarden worden geïnitialiseerd bij het begin van het programma. Overigens kunnen de queue en de bijbehorende indexvariabelen, net zoals dat bij de stack het geval was, aan het zicht van de overige functies van een programma worden onttrokken door ze *static* te maken. De functies *read_queue()* en *write_queue()* zijn wederom interface-functies die als enige toegang hebben tot de feitelijke queue. Er is hier dus opnieuw sprake van 'data hiding'. Een blauwdruk voor de functies *read_queue()* en *write_queue()* is gegeven in listing 106.

```
1       static int
2           queue[MAX_QUEUE_SIZE],
3           head,
4           tail;
5
6       int read_queue(int *target)
7       {
8           if (head == tail)
9               return (1);
10          *target = queue[head];
11          if (++head == MAX_QUEUE_SIZE)
12              head = 0;
13          return (0);
14      }
15
16      int write_queue(int value)
17      {
18          int
19              next_tail;
20
21          next_tail = (tail + 1) % MAX_QUEUE_SIZE;
22          if (next_tail == head)
23              return (1);
24          queue[tail = next_tail] = value;
25          return (0);
26      }
```

Listing 106: De functies *read_queue()* en *write_queue()*.

17.3.1 Een wachtkamer als voorbeeld

De situaties die zich voordoen bij de wachtkamer van een arts of bij de bakker waarbij de klanten een volgordenummer uit een automaat trekken, lenen zich goed voor de illustratie van het gebruik van een queue.

De situatie die zich voordoet bij de wachtkamer van een arts zal worden gesimuleerd door een programma *queue.c,* waarin van een queue datastructuur gebruik wordt gemaakt.

De producenten in dit programma zijn de patiënten die de wachtkamer binnenkomen. Een patiënt die de wachtkamer binnenkomt wordt opgenomen in de queue. Zodra de arts de patiënten begint te behandelen wordt de queue gereduceerd: de arts is de consument die met behulp van het queueproces de volgende patiënt in de spreekkamer vraagt te komen. Een aardige bijkomstigheid is nog dat het programma een verwacht behandeltijdstip zal vermelden. In het programma wordt dat in eerste instantie gedaan door een vaste waarde voor de gemiddelde behandeltijd te kiezen van 10 minuten. In een uitbreiding van het programma wordt het gemiddelde bepaald op grond van feitelijke behandelingstijden.

De herformulering van de werking van het *queue* programma wordt:

1. Laat in een eeuwige lus een keuzemenu zien waarbij tussen de opties W: wachten, B: behandelen, of S: stoppen kan worden gekozen.
2. Als de 'wacht' optie is gekozen, voeg dan een patiënt toe aan de queue.
3. Als de 'behandelen' optie is gekozen, verwijder dan een patiënt uit de queue.
4. Beëindig het programma als de 'stop' optie is gekozen.

Het datatype

De uiteindelijke data in het wachtkamerprobleem zijn geen *ints,* maar het zijn strings. De functies *read_queue()* en *write_queue()* moeten dus op een array van strings werken.

Opgave 99.

Construeer de functies *read_queue()* en *write_queue()* die een queue van strings bewerken in plaats van een queue van *ints* In de queue moeten kopieën van de string die als argument aan *write_queue()* is meegegeven worden geplaatst. Wanneer een element wordt gelezen wordt aan een *char* pointer, waarvan het adres als argument aan *read_queue()* is meegegeven, het adres toegekend van de string, die in de queue is opgeslagen. Uiteindelijk zal de functie waardoor de functie *read_queue()* wordt aangeroepen het geheugen van de string dus weer vrijgeven. Laat het aantal element in de queue *MAX_QUEUE_SIZE* zijn, welke waarde met een *#define* als 10 kan worden gedefinieerd.

Implementatie

De functie *main()* kan worden geconstrueerd uit de herformulering van het wachtkamerprobleem. De onderdelen worden gereliseerd door functies aan te roepen die de in het onderdeel genoemde taak uitvoeren. De implementatie van de functie *main()* is gegeven in listing 107. Bij de functie is een *enum* gedefinieerd waarmee symbolische namen voor de mogelijke acties die door het programma kunnen worden uitgevoerd zijn vastgelegd.

Onderdeel 1 betreft de presentatie van het keuzemenu en wordt als volgt geherformuleerd:

1.1. Doe het volgende totdat een gedefinieerde actie is gevraagd:
    1.2. Laat een menu zien.
    1.3. Bepaal de gewenste actie.
    1.4. Als een niet gedefinieerde actie wordt gevraagd:
        1.5. Geef dan een foutmelding.
1.6. Retourneer de aanduiding van de gewenste actie.

```
1     typedef enum
2     {
3         action_wait,
4         action_treat,
5         action_stop
6     } ACTION_;
7
8     void main()
9     {
10        while (1)
11        {
12            switch(menu())
13            {
14                case action_wait:
15                    add_patient();
16                break;
17
18                case action_treat:
19                    treat_patient();
20                break;
21
22                case action_stop:
23                    return;
24                break;
25            }
26        }
27    }
```

Listing 107: De functie *main()* van *queue.c*.

De implementatie van de functie *menu()* maakt gebruik van het gegeven dat de waarden van een *enum* in oplopende numerieke volgorde zijn gedefinieerd, en dat het verschil tussen adressen een indexwaarde oplevert. Bij het bepalen van de gewenste actie worden eventuele hoofdletters omgezet naar kleine letters. Voor het inlezen van de gewenste actie wordt de standaardfunctie *getche()* gebruikt waarmee de volgende toetsaanslag wordt geëchood en gelezen. Voor het overige volgt de implementatie van *menu()* de hier gegeven herformulering. De listing van de functie *menu()* is gegeven in listing 108.

In onderdeel 2 wordt gebruik gemaakt van een lokale buffer van 250 tekens om de naam van de patiënt, die in de wachtkamer komt, op te

slaan. Om de verwachte wachttijd te bepalen wordt een globale teller bijgehouden, die wordt opgehoogd zodra een patiënt in de wachtkamer komt en wordt verlaagd, zodra een patient naar de spreekkamer gaat. Voordat de teller wordt verhoogd, kan aan de patiënt worden verteld hoeveel wachtenden er nog zijn, en wanneer de patiënt vermoedelijk aan de beurt is. De herformulering van onderdeel 2 luidt:

2.1. Bepaal de naam van de patiënt.
2.2. Vermeld het aantal wachtenden, en de verwachte wachttijd.
2.3. Als de naam van de patiënt in de queue kan worden opgenomen, verhoog dan het aantal wachtenden.
2.4. Als de naam niet in de queue kan worden opgenomen, meld dan dat er geen patiënten meer kunnen worden behandeld.

```
1       #include <ctypes.h>
2       #include <conio.h>
3
4       static char
5           wbs[] = "wbs";
6
7       ACTION_ menu()
8       {
9           char
10              c,
11              *cp;
12
13          do                                      // Tekst: 1.1.
14          {
15              printf("Kies:\n"                    // Tekst: 1.2.
16                  "\tW: Wachten in de wachtkamer\n"
17                  "\tB: Behandelen in de spreekkamer\n"
18                  "\tS: Stop het programma\n"
19                  "?" );
20              c = getche();                       // Tekst: 1.3.
21              putch('\n');
22              c = tolower(c);
23              if (!(cp = strchr(wbs, c)))         // Tekst: 1.4.
24                  printf("\n'%c' is geen gedefinieerde"// Tekst: 1.5.
25                      "actie van het programma\n", c);
26          }
27          while (!cp);
28          return (cp - wbs);                      // Tekst: 1.6.
29      }
```

Listing 108: De functie *menu()* van *queue.c*.

De functie *add_patient()* is weergegeven in listing 109.

Onderdeel 3 verloopt analoog aan onderdeel 2. De herformulering luidt:

3.1. Als de naam van een patiënt uit de queue kan worden verwijderd,
3.2 verlaag dan de teller van het aantal wachtenden,

3.3 vermeld de naam van de te behandelen patiënt en het aantal patiënten dat nog in de wachtkamer aanwezig is.
3.4. geef het geheugen vrij waarin de naam van de patiënt is opgeslagen.
3.5. Als de naam niet uit de queue kan worden verwijderd, meld dan dat er geen patiënten meer wachten.

```
1         #define TREATMENT  10
2
3         int
4             waiting;
5
6         void add_patient()
7         {
8             char
9                 buffer[250];
10
11            printf("Naam van de pati
verb+"+ent ?" );                                    // Tekst: 2.1.
12            gets(buffer);
13            if (!waiting)                         // Tekst: 2.2.
14                puts("U bent direct aan de beurt");
15            else
16                printf("Er %s nog %d pati
verb+"+ent%s voor U,\n"
17                       "U bent vermoedelijk over %d minuten aan de beurt.\n",
18                            waiting == 1 ?
19                                "is"
20                            :
21                                "zijn",
22                            waiting,
23                            waiting == 1 ?
24                                ""
25                            :
26                                "en",
27                            waiting * TREATMENT);
28            if (!write_queue(buffer))             // Tekst: 2.3.
29                waiting++;
30            else                                  // Tekst: 2.4.
31                puts("Sorry, er kunnen geen pati
verb+"+enten meer worden behandeld");
32        }
```

Listing 109: De functie *add_patient()* van *queue.c*.

Functie *treat_patient()* is weergegeven in listing 110.

```
1         #include <malloc.h>
2
3         void treat_patient()
4         {
5             char
6                 *name;
```

```
7
8             if (!read_queue(&name))                    // Tekst: 3.1.
9             {
10                waiting--;                             // Tekst: 3.2.
11                printf("Volgende pati
verb+"+ent is %s\n"                                      // Tekst: 3.3.
12                        "en" ,
13                        name);
14                switch(waiting)
15                {
16                    case 0:
17                        puts("de wachtkamer is leeg.");
18                    break;
19
20                    case 1:
21                        puts("er wacht nog één pati
verb+"+ent.");
22                    break;
23
24                    default:
25                        printf("er wachten nog %d pati
verb+"+enten.\n", waiting);
26                    break;
27                }
28                free(name);                            // Tekst: 3.4.
29            }
30            else                                       // Tekst: 3.5.
31                puts("De wachtkamer is momenteel leeg.");
32        }
```

Listing 110: De functie *treat_patient()* van *queue.c*.

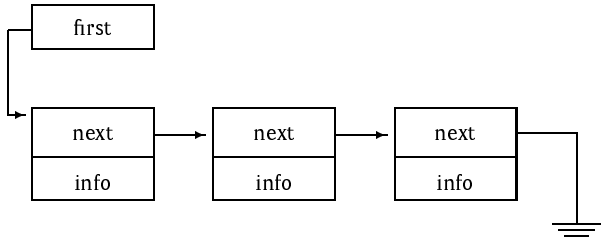
Opgave 100.

Stel het programma *queue* samen uit de functies *read_queue()* en *write_queue()* en uit de listings 107 tot en met 110. Zorg ervoor dat alle opdrachten voor de preprocessor aan het begin van het bestand staan en dat functies zijn gedefinieerd voordat ze worden gebruikt. Vertaal, link en test het programma.

17.4 Lijsten

Een lijst (list of linked list) is een datatype, dat bestaat uit structuren die onder andere velden (pointers) bevatten, die naar hetzelfde type structuren wijzen (self referential of recursieve structuren en pointers). De kracht van een lijst bestaat eruit, dat elementen in zo'n lijst op eenvoudige wijze overal aan de lijst toegevoegd kunnen worden en overal uit de lijst kunnen worden verwijderd, zonder dat er 'gaten' vallen in de datastructuur.

De elementen van lijsten zijn *structs* met minstens twee velden: een veld dat de verwijzing bevat naar het volgende element in de lijst en een veld dat de overige informatie bevat (zie ook figuur 17.3). In het geheugen



Figuur 17.3: De lijststructuur.

bestaat de lijst uit een keten van elementen, waarbij elk element verwijst naar de plaats van het volgende element. Doorgaans is er een variabele die alleen de verwijzing bevat naar het eerste element van de lijst. Deze variabele is de enige variabele van waaruit de elementen van de lijst kunnen worden bereikt. Het laatste element wijst niet meer naar een volgend element; het *next*-veld van het laatste element van de lijst wordt daarom op de waarde *NULL* gezet. Per definitie betekent een pointerwaarde *NULL* dat de pointer op géén lijstelement wijst.

Bij de definitie van het datatype voor lijsten is de *struct*-tag een noodzakelijk onderdeel van de *struct*-definitie. Het tag-veld is noodzakelijk omdat een veld van de struct (per conventie: het eerste veld) een verwijzing moet bevatten naar volgende structuren van hetzelfde type. De naam van het datatype is echter op het moment dat het pointerveld moet worden gedefinieerd nog niet bekend aan de vertaler, terwijl de *struct*-tag op dat moment al wel bekend is. De blauwdruk van een *struct*-definitie voor een lijststructuur is gegeven in listing 111. Op regel 1 wordt de tag gedefinieerd. Een goede conventie is de naam van de tag gelijk te laten zijn aan de naam van het uiteindelijke datatype (*LIST_,* regel 8). Op regel 3 begint de definitie van het self-referential veld *next:* omdat de tag al wel bekend is, en de definitie van het datatype zelf nog niet is voltooid, wordt hier expliciet het type *struct LIST_* gebruikt. De overige velden van de struct zijn in listing 111 samengevoegd in een extra *LIST_INFO_ struct* om het onderscheid tussen het pointerveld *next* en de inhoudelijke informatie te benadrukken. De inhoudelijke velden van de *struct* zouden ook heel goed in de definitie zelf kunnen worden gedefinieerd.

```
1           typedef struct LIST_
2           {
3               struct LIST_
4                   *next;
```

```
5
6                LIST_INFO_
7                    info;
8            } LIST_;
```

Listing 111: De blauwdruk van een definitie van een lijst-*struct*.

De standaardbewerkingen die op lijsten kunnen worden uitgevoerd zijn:

- Initialisatie van de lijst.
- Opzoeken van een bepaald element uit de lijst.
- Toevoegen van een nieuw element achter een bestaand element.
- Weglaten van een element achter een bestaand element.

Net zoals dat bij de stacks en de queues het geval was, kunnen ook voor de bewerkingen op lijsten blauwdrukken van functies worden ontwikkeld, waarmee de genoemde standaardbewerkingen kunnen worden uitgevoerd. Bij de initialisatie van de lijst zijn er nog geen elementen. Omdat de pointervariabele die op het begin van de lijst zal gaan wijzen dan dus nog niet op een lijstelement wijst, zal deze variabele de waarde *NULL* moeten hebben. Dat gebeurt automatisch bij de definitie van een globale pointervariabele (zie listing 112).

```
1        extern int compare(LIST_ *, LIST_INFO_ *);
2
3        LIST_
4            *first;
5
6        LIST_ *search_list(LIST_ *head, LIST_INFO_ *info)
7        {
8            while (head)
9            {
10               if (!compare(head, info))
11                   break;
12               head = head->next;
13           }
14           return (head);
15       }
16
```

Listing 112: De blauwdruk van de functie *search_list()*.

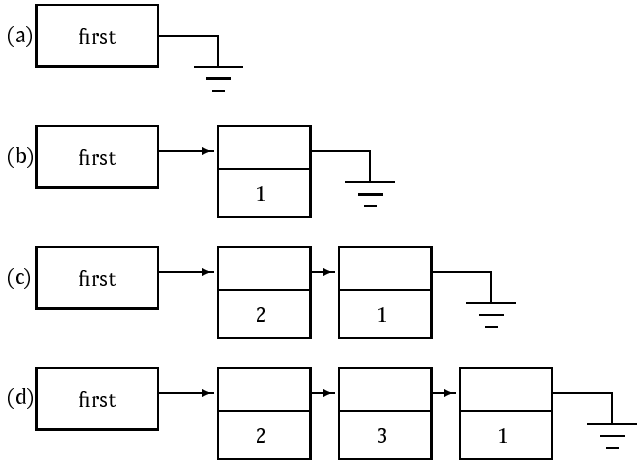
```
1        LIST_ *add_list(LIST_ *current)
2        {
3            LIST_
4                *new;
5
6            if (new = malloc(sizeof(LIST_)))
7                new->next = current;
8
9            return (new);
10       }
```

Listing 113: De blauwdruk van de functie *add_list()*.

De blauwdruk van de functie waarmee een element uit de lijst moet worden opgezocht, is als de functie *search_list()* eveneens gegeven in listing 112. De functie retourneert een pointer naar een gevonden element, of *NULL* wanneer het gezochte element zich niet in de lijst bevindt. De functie krijgt als argument het adres van het eerste element van de lijst en een pointer naar een *LIST_INFO_struct* waarin de gezochte informatie staat. Met behulp van de functie *compare(),* waarvan alleen het prototype is gegeven, wordt gekeken of het huidige element van de lijst de gezochte informatie bevat. In de blauwdruk van de functie *search_list()* wordt verondersteld dat de functie *compare()* 0 retourneert wanneer de gezochte informatie is gevonden. Wanneer de informatie is gevonden, dan wordt het zoekproces afgebroken, en wordt het adres van het gevonden lijstelement geretourneerd. Wanneer de informatie niet is gevonden en het einde van de lijst nog niet is bereikt, wordt het proces herhaald op het volgend element. Wanneer het einde van de lijst is bereikt wordt *NULL* geretourneerd.

De blauwdruk van de functie *add_list()* waarmee een nieuw element voor een bestaand element moet worden toegevoegd is gegeven in listing 113. De functie *add_list()* krijgt als argument de waarde van een lijstpointer die op een nieuw element moet gaan wijzen. De functie creëert geheugen voor het nieuwe element via de aanroep van de functie *malloc()*. Wanneer onvoldoende geheugen beschikbaar is, wordt *NULL* geretourneerd. Wanneer er wel voldoende geheugen beschikbaar is, wordt het adres van het nieuwe element geretourneerd. Buiten de functie krijgt de pointervariabele, die als argument aan de functie is doorgegeven, de return-waarde van de functie. Hier moet dan een test worden uitgevoerd op de return-waarde *NULL*. Merk op dat een nieuw element op deze wijze altijd net vóór een bestaand element wordt toegevoegd. Met de code uit het volgende voorbeeld wordt een lijst van drie elementen geconstrueerd. Ter wille van het voorbeeld wordt aangenomen dat de functie *add_list()* geen waarde *NULL* retourneert. In figuur 17.4 wordt geïllustreerd hoe de uiteindelijke lijst wordt gevormd.

```
LIST_
    *first;

void list_of_three()
{
    // (a)
    first = add_list(first);
    first->info = 1;
    // (b)
    first = add_list(first);
    first->info = 2;
    // (c)
    first->next = add_list(first->next);
    first->next->info = 3;
```



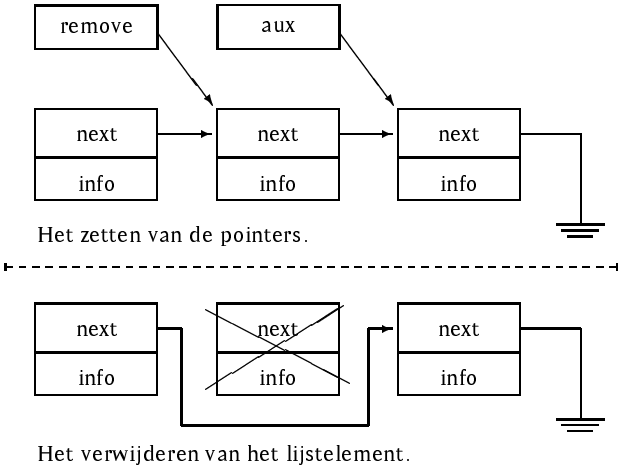
Figuur 17.4: De constructie van een lijst.

```
    // (d)
}
```

In figuur 17.4 (a) is de oorspronkelijke situatie weergegeven: *first* wijst nog nergens op. In figuur 17.4 (b) is de lijst van één element gevormd. De variabele *first* wijst op dit element. In figuur 17.4 (c) wordt een extra element toegevoegd. Het eerste element is nu bereikbaar via het nieuwe element waarop door *first* wordt gewezen. In figuur 17.4 (d) wordt een extra element tussen element 2 en 1 toegevoegd.

De blauwdruk van de functie *delete_list()* waarmee een element van een lijst wordt verwijderd, is gegeven in listing 114. De functie heeft als argument het adres van het lijstelement dat moet worden verwijderd. Het verwijderen bestaat uit de volgende stappen:

- Wanneer het adres van het te verwijderen element *NULL* is, wordt *NULL* geretourneerd, omdat er dan geen te verwijderen element is.
- Vervolgens krijgt een hulppointer het adres van het volgende element van de lijst.
- Dan wordt het geheugen dat door het te verwijderen element werd ingenomen vrijgegeven. De aanroepende functie is verantwoorde-



Figuur 17.5: Het verwijderen uit een lijst.

lijk voor het vrijgeven van eventueel extra geheugen geassocieerd met het *info* veld vóórdat de functie *delete_list()* wordt aangeroepen.

- Tenslotte wordt de waarde van de hulppointer geretourneerd. De geretourneerde waarde wordt buiten de functie toegekend aan de variabele die als argument aan de functie was meegegeven.

Het verwijderen van een element uit een lijst is weergegeven in figuur 17.5.

```
1       LIST_ *delete_list(LIST *current)
2       {
3           LIST_
4               *aux;
5
6           if (!current)
7               return (NULL);
8
9           aux = current->next;
10          free(current);
11          return (aux);
12      }
```

Listing 114: De blauwdruk van de functie *delete_list()*.

Wanneer elementen aan lijsten worden toegevoegd of eruit worden verwijderd, is de belangrijkste vuistregel dat alleen pointers die een on-

derdeel van de lijststructuur zelf zijn, mogen worden gewijzigd. Vaak worden hulppointers gebruikt, die zelf géén onderdeel van de lijststructuur zijn. Wanneer zo'n hulppointer als argument aan de functie *delete_list()* of *add_list()* wordt gegeven, kan de lijststructuur worden vernield. Zo'n situatie wordt geïllustreerd in listing 115. In regel 2 krijgt een hulppointer *ptr* het adres van het derde element van de lijst. In regel 3 wordt het derde element uit de lijst verwijderd, maar in de lijst zelf is die wijziging niet waarneembaar. De variabele *first->next->next* is niet veranderd en wijst dus na de aanroep van de functie *delete_list()* in regel 3 op geheugen dat niet meer gealloceerd is. De variabele *first->next->next* is daardoor een 'wilde pointer' geworden. In regel 5 daarentegen wordt de pointervariabele *ptr->next,* die onderdeel is van de lijststructuur, gewijzigd en treedt de wilde pointer niet meer op.

```
1        ...
2        ptr = first->next->next;
3        ptr = delete_list(ptr);
4        ...
5        ptr = first->next;
6        ptr->next = delete_list(ptr->next);
```

Listing 115: Het vernielen van een lijst.

17.4.1 Het vorige lijstelement

Wanneer bij lijststructuren een combinatie van zoeken en toevoegen of verwijderen noodzakelijk is, kan er een probleem ontstaan waaraan hier wat aandacht wordt besteed. Het probleem ontstaat doordat bij het zoeken door een lijst het adres van een gevonden element wordt geretourneerd, en niet het adres van het vorige element. Anders dan bij het array, waar het vorige element eenvoudig op de vorige indexpositie wordt aangetroffen, bevindt het lijstelement waarop door een variabele *current* wordt gewezen zich niet noodzakelijk op adres *current - 1*. Echter, om een gevonden element te verwijderen, of om een element vóór het gevonden element toe te voegen, moet het adres van het vorige element bekend zijn.

Er zijn in principe twee manieren om het vorige element te kunnen bereiken. Ten eerste kan de lijst worden uitgevoerd als een dubbele lijst ('double linked list') waarbij niet alleen een pointer naar het volgend element beschikbaar is, maar ook een pointer naar het vorige element. Ten tweede kan een functie worden geconstrueerd die het adres van het huidige element van de lijst èn van het vorige element van de lijst retourneert. We zullen een blauwdruk van beide benaderingen geven.

De dubbele lijst

```
1        typedef struct DLIST_
2        {
```

```
3            struct DLIST_
4                *next,
5                *previous;
6
7            LIST_INFO_
8                info;
9        } DLIST_;
```

Listing 116: De *struct*-definitie bij een dubbele lijst.

De dubbele lijst heeft twee self referential pointers: één pointer *(next)* wijst naar het volgende element van de lijst en de andere pointer *(previous)* wijst terug, naar het vorige element van de lijst. De blauwdruk van het datatype van de dubbele lijst is gegeven in listing 116. De dubbele lijst zal iets meer geheugen vergen dan de enkelvoudige lijst, omdat bij elk element een extra pointerveld wordt opgeslagen. Daar staat tegenover dat de dubbele lijst een grote flexibiliteit heeft, omdat de lijst in twee richtingen kan worden doorlopen. Wanneer de functie *search_dlist()* een lijstelement vindt, kan dat element eenvoudig worden gewijzigd, zoals geïllustreerd in het volgende programmafragment, waarin wordt verondersteld dat de waarden van *first* en *info* al zijn bepaald.

```
DLIST_
    *aux;

...
if (aux = search_dlist(first, &info))
    delete_dlist(aux->previous);
...
```

De functie *search_dlist()* is dezelfde functie als de eerder beschreven functie *search_list()* wanneer ervoor wordt gezorgd dat het *next* veld van de *DLIST_ struct* het eerste veld van de *struct* is. De functies *add_dlist()* en *delete_dlist()* zijn wèl iets uitgebreider dan de vergelijkbare functies *add_list()* en *delete_list(),* omdat er twee pointers moeten worden bijgehouden. De extra pointer is de pointer *previous,* die het adres van het vorige element van de lijst bevat. Door de variabele *first* niet langer een pointer naar een lijstelement te laten zijn, maar zelf tot een lijstelement met een ongebruikt *info* veld te maken, wordt de manipulatie van de dubbele lijst sterk vereenvoudigd.

De initialisatie van de dubbele lijst bestaat eruit dat de *next* en *previous* velden van de variabele *first* het adres van de variabele *first* krijgen.

Bij het toevoegen van een nieuw element aan de dubbele lijst is er sprake van een huidig element, een volgende element en een nieuw element. In eerste instantie bevindt het volgende element zich achter het huidige element. Om het nieuwe element tussen het huidige en het volgende element te plaatsen, moeten (na allocatie van het nieuwe element) de volgende vier stappen worden uitgevoerd:

- Het *next* veld van het nieuwe element krijgt de waarde van het *next* veld van het huidige element.
- Het *previous* veld van het nieuwe element krijgt de waarde van het adres van het huidige element.
- Het *next* veld van het huidige element krijgt het adres van het nieuwe element.
- Het *previous* veld van het volgende element krijgt het adres van het nieuwe element.

Het verwijderen van het huidige element tussen een vorig en volgend element verloopt als volgt:

- Het *next* veld van het vorige element krijgt de waarde van het *next* veld van het huidige element.
- Het *previous* veld van het volgende element krijgt de waarde van het *previous* veld van het huidige element.
- Het geheugen dat voor het huidige element was gealloceerd wordt vrijgegeven.

Bij het verwijderen van een element uit de dubbele lijst is de aanroepende functie ervoor verantwoordelijk dat geheugen, dat voor de inhoudelijke informatie van het te verwijderen elementen is gealloceerd, wordt vrijgegeven vóórdat het element zelf wordt verwijderd.

Opgave 101.

Construeer de functies *add_dlist()* en *delete_dlist()* die als argumenten het adres van het huidige lijstelement meekrijgen. De functie *add_dlist()* moet het adres van het nieuwe element retourneren, of *NULL* wanneer geen geheugen kan worden gealloceerd. De functie *delete_dlist()* is een *void* functie. Definieer en initialiseer ook een variabele *first* van het type *DLIST_*.

17.4.2 De functie lagged_listsearch()

Een tweede manier om een vorig element van een lijst te kunnen bereiken, bestaat eruit dat een pointer naar een vorig lijstelement wordt bijgehouden. Deze pointer ijlt één lijstelement na ten opzichte van een pointer naar het huidige lijstelement. Met deze benadering kan de enkelvoudige lijst nog steeds worden gebruikt, maar de methode is eigenlijk alleen maar goed bruikbaar in situaties waarin slechts één stap terug moet kunnen worden gegaan. Wanneer meerdere stappen terug moeten kunnen worden gezet, verdient de dubbele lijst de voorkeur. De methode is geïmplementeerd in de functie *lagged_listsearch(),* die als argumenten drie pointervariabelen krijgt:

- Het eerste argument is het adres van een pointer naar een lijstelement (dus een dubbele pointer).
- Het tweede argument is het adres van het element van de lijst vanaf waar moet worden gezocht.

- Het derde argument is een pointer naar de informatie die in de lijst moet worden gezocht.

```
1        extern int compare(LIST_ *, LIST_INFO_ *);
2
3        LIST_ *lagged_listsearch(LIST_ **previous, LIST_ *head, LIST_INFO_ *info)
4        {
5            *previous = NULL;
6            while (head)
7            {
8                if (!compare(head, info))
9                    break;
10               *previous = head;
11               head = head->next;
12           }
13           return (head);
14       }
15
```

Listing 117: De functie *lagged_listsearch()*.

De functie retourneert hetzelfde als de functie *search_list()* (zie listing 112). Bovendien krijgt de pointervariabele waarvan het adres als eerste argument is meegegeven het adres van het vorige lijstelement, of *NULL* wanneer er geen vorig lijstelement is. De functie *lagged_listsearch()* maakt evenals de functie *search_list()* gebruik van een functie *compare()*. De implementatie van de functie *lagged_listsearch()* is gegeven in listing 117. In regel 5 wordt **previous* de waarde *NULL* gegeven waardoor er geen verwijzing naar een vorig element is wanneer de functie *compare()* direct al de gezochte informatie vindt. Voordat naar het volgende lijstelement wordt gegaan, wordt in regel 10 eerst aan **previous* het adres van het huidige lijstelement toegkend, waardoor het na-ijl effect wordt bereikt. De volgende vier situaties kunnen zich voordoen:

1. **previous* is *NULL* en de functie retourneert *NULL* wanneer er geen lijst is.
2. **previous* is *NULL* en de functie retourneert een waarde ongelijk *NULL* wanneer de gezochte informatie direct al bij de eerste aanroep van *compare()* werd gevonden.
3. **previous* is niet-*NULL* en de functie retourneert *NULL* wanneer de informatie niet is gevonden. **previous* wijst dan op het laatste element van de lijst.
4. **previous* is niet-*NULL* en de functie retourneert een waarde ongelijk *NULL* wanneer de informatie ergens in de lijst is gevonden. **previous* wijst dan op het element vóór het element dat de gezochte informatie bevat.

17.4.3 Een lijst van namen als voorbeeld

```
1     void add_name()
2     {
3         LIST_
4             *prev;
5         char
6             buffer[256];
7
8         printf("\nToe te voegen naam ?" );          // Tekst: 3.1.
9         gets(buffer);
10
11        if (!lagged_listsearch(&prev, first, buffer))// Tekst: 3.2.
12            printf("'%s' is al in de lijst aanwezig\n",// Tekst: 3.3.
13                    buffer);
14        else
15            new_name(strdup(buffer),                // Tekst: 3.4.
16                     prev ? &prev->next : &first);
17    }
18
```

Listing 118: De functie *add_name()* van *list.c*.

Als illustratie van een programma waarin lijsten worden gebruikt, wordt hier een programma ontwikkeld waarin een alfabetisch gesorteerde lijst van namen wordt bijgehouden. De gebruiker van het programma kan een naam aan de lijst toevoegen, verwijderen of de lijst kan op het beeldscherm worden afgedrukt. De herformulering van het probleem is:

1. Doe het volgende in een eeuwige lus:
   2. Bepaal aan de hand van een menu de gewenste actie.
   3. Wanneer de actie 'toevoegen' is, voeg dan een naam toe aan de lijst.
   4. Wanneer de actie 'verwijderen' is, verwijder dan een naam uit de lijst.
   5. Wanneer de actie 'afdrukken' is, laat de lijst dan zien op het beeldscherm.
   6. Wanneer de actie 'stoppen' is, beëindig dan het programma.

Het programma wordt analoog aan het *queue.c* programma geconstrueerd. Het menu is iets anders, maar de menufunctie wordt overeenkomstig geconstrueerd. De functie *main()* wordt ook overeenkomstig het *queue.c* programma opgezet. De implementaite van de functie *main()* en van onderdeel 2 wordt dan ook aan de lezer overgelaten. Onderdeel 3 wordt gerealiseerd door een functie *add_name()*. Om een naam toe te voegen moet het volgende worden ondernomen:

3.1. Bepaal de naam die moet worden toegevoegd.
3.2. Zoek de naam op in de lijst met namen.
3.3. Wanneer de naam al in de lijst aanwezig is, moet dat worden vermeld.

3.4. Wanneer de naam nog niet in de lijst aanwezig is, moet een kopie van de naam in de lijst worden opgenomen.

De implementatie van de functie *add_name()* is gegeven in listing 118. Omdat het mogelijk moet zijn een naam vóór een andere naam in de lijst op te nemen, wordt de functie *lagged_listsearch()* gebruikt. Deze functie wordt zodanig aangepast dat de return-waarde aangeeft of de nieuwe naam al dan niet in de lijst voorkomt. De aanpassing houdt onder andere in dat de standaardfunctie *strcmp()* kan worden gebruikt in plaats van een speciale functie *compare()*. Het eerste argument van de functie *lagged_listsearch()* wijst na de aanroep onveranderd op het lijstelement waarachter de nieuwe naam moet worden toegevoegd. Wanneer de nieuwe naam direct na *first* moet worden ingevoegd, krijgt het eerste argument na afloop van de functie *lagged_listsearch()* de waarde *NULL*. Hiervan wordt gebruik gemaakt in onderdeel 3.4. Dit onderdeel wordt gerealiseerd door de functie *new_name()* die als argument een kopie van de ingelezen naam krijgt en het adres van de pointervariabele die op het nieuwe lijstelement moet gaan wijzen. De functie *new_name()* heeft geen bijzondere kenmerken. De listing ervan is gegeven in listing 119. De aanpassing van de functie *lagged_listsearch()* is als opgave overgelaten aan de lezer.

```
1     void new_name(char *dupname, LIST_ **link)
2     {
3         if
4         (
5             !dupname
6             ||
7             !(*link = add_list(*link))
8         )
9             puts("Out of memory");
10        else
11        {
12            (*link)->info = dupname;
13            printf("'%s' is opgenomen in de lijst\n", dupname);
14        }
15    }
16
```

Listing 119: De functie *new_name()* van *list.c*.

Opgave 102.

Pas de functie *lagged_listsearch()* zodanig aan dat de waarde 0 wordt geretourneerd wanneer de nieuw toe te voegen naam al in de lijst voorkomt en de waarde 1 wanneer dat nog niet het geval is. De functie kan gebruik maken van de standaardfunctie *strcmp()* om de nieuwe naam te vergelijken met een naam in de lijst. Wanneer de naam in de lijst bij alfabetische ordening achter de nieuwe naam wordt aangetroffen, moet het volgende lijstelement worden geïnspecteerd.

Onderdeel 4, het verwijderen van een element uit de lijst, wordt uitgevoerd door de functie *remove_name()*. In deze functie worden de volgende acties

uitgevoerd:

4.1. Bepaal de naam die moet worden toegevoegd.
4.2. Zoek de naam op in de lijst met namen.
4.3. Wanneer de naam in de lijst wordt gevonden, dan moet de naam worden verwijderd.
4.4. Wanneer de naam niet in de lijst aanwezig is, moet dat worden vermeld.

```
1    void remove_name()
2    {
3        LIST_
4            *prev;
5        char
6            buffer[256];
7
8        printf("\nTe verwijderen naam ?" );          // Tekst: 4.1.
9        gets(buffer);
10
11       if (!lagged_listsearch(&prev, first, buffer))// Tekst: 4.2.
12           delete_name(prev ? &prev->next : &first);// Tekst: 4.3.
13       else
14           printf("'%s' niet in de lijst aanwezig\n",// Tekst: 4.4.
15                   buffer);
16   }
```

Listing 120: De functie *remove_name()* van *list.c*.

De implementatie van de functie *remove_name()* is gegeven in listing 120. De functie lijkt sterk op de functie *add_name()* en maakt onder andere gebruik van de functie *lagged_listsearch()*. Onderdeel 4.3. wordt gerealiseerd door de functie *delete_name()* die analoog aan de functie *new_name()* wordt geconstrueerd. De functie moet het geheugen dat is gealloceerd voor de naam vrijgeven en dan het destbetreffende lijstelement verwijderen. De implementatie van de functie *delete_name()* is als opgave aan de lezer overgelaten. De implementatie van onderdeel 5 wordt overgelaten aan de functie *display_names()*. Deze functie kan ook als opgave worden geïmplementeerd.

Opgave 103.

Construeer de functie *delete_name()*. De functie krijgt als argument het adres van de pointer die naar het te verwijderen lijstelement wijst. Het geheugen voor de naam moet worden vrijgegeven voordat het lijstelement zelf wordt verwijderd.

Opgave 104.

Construeer de functie *display_names()*. De functie krijgt als argument de waarde van de variabele *first* en loopt alle lijstelementen af, waarbij elke naam in de lijst op het beeldscherm wordt afgedrukt met behulp van de standaardfunctie *puts()*.

Opgave 105.

Construeer het volledige programma aan de hand van de listings 118 tot en met 120 en de functies die hier als opgave zijn geconstrueerd. Pas de blauwdruk van de functies *add_list()* en *delete_list()* zodanig aan dat ze in het programma *list.c* kunnen worden gebruikt.

17.5 Bomen

Boomstructuren (of: trees) zijn evenals lists recursieve structuren. Bomen zijn echter niet lineair opgebouwd, maar vertakt (als een boom). Hierdoor verschillen bomen van de vorige datastructuren.

- In tegenstelling tot de stack en de queue is de plaats van een element in de boom niet afhankelijk van de volgorde van binnenkomst. De plaats van een element in de boomstructuur is afhankelijk van het doel van de boom. Meestal worden bomen zodanig geconstrueerd dat het terugvinden van de elementen relatief snel kan gebeuren.
- Bewerkingen op boomstructuren zijn, in tegenstelling tot bewerkingen op lijststructuren, van nature recursief. Daarnaast hebben bomen altijd twee of meer verwijzingen daar data van hetzelfde type, terwijl lijsten kunnen volstaan met één enkele verwijzing[4].

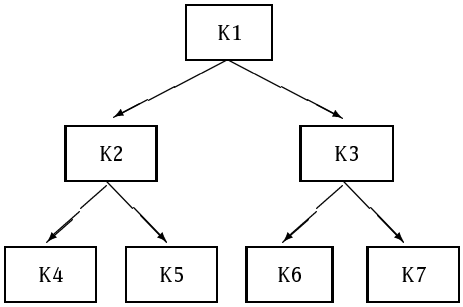
Hoewel boomstructuren met meer dan twee verwijzingen naar boomstructuren kunnen worden geconstrueerd, zijn de meeste toepassingen gericht op structuren met twee verwijzingen naar onderliggende boomstructuren. Dergelijke datastructuren heten binaire bomen. Bij een binaire boom bestaat elk knooppunt uit twee verwijzingen naar onderliggende (geneste) boomstructuren en uit een informatiedragend deel zoals bij de lijststructuren werd aangetroffen. De geneste boomstructuren heten de 'kinderen' van de knoop. Vrijwel onveranderlijk wordt er gesproken van het 'linker kind' en het 'rechter kind'. De beide verwijzingen naar de kinderen van een knoop worden meestal aangeduid als *left* en *right*.

Het datatype van de boom is een *struct*, die de volgende velden bevat:

- een (self referential) pointer naar een linker kind;
- een (self referential) pointer naar een rechter kind;
- een informatiedeel dat uit meerdere velden kan bestaan, of dat zelf een *struct* kan zijn.

```
1        typedef struct NODE_
2        {
3            struct NODE_
4                *left,
```

4. De dubbele lijst is een lijst waarin wèl twee verwijzingen staan, maar van een bijzondere aard: bij dubbele lijsten is er als het ware sprake van twee lineaire lijsten, die met dezelfde lijstelementen werken.



Figuur 17.6: Een boom met zeven knopen.

```
5                *right;
6            NODE_INFO_
7                info;
8        } NODE_;
```

Listing 121: De blauwdruk van een *struct* bij een boomstructuur.

De blauwdruk van de typedefinitie van een boomstructuur is gegeven in listing 121. De knooppunten van waaruit de vertakkingen uitwaaieren, worden doorgaans aangeduid als 'nodes'. Het begin van de boom wordt de basis of wortel (root) genoemd. Wanneer een knoop geen linker en rechter kinderen meer heeft, is een eindpunt van de boomstructuur bereikt. Zo'n eindpunt wordt een blad (leaf) genoemd. In figuur 17.6 is een boom met zeven knopen afgebeeld. De wortel van de boom is knoop *K1,* de bladeren zijn de knopen *K4* tot en met *K7,* de overige knopen zijn gewone vertakkingsknopen.

Bij boomstructuren kunnen de volgende standaardbewerkingen uitgevoerd worden:

1. Er kan een knoop worden aangemaakt.
2. De boomstructuur kan worden doorlopen (traversal).
3. Een subboom kan aan een knooppunt worden bevestigd
4. Een subboom kan van een knooppunt worden verwijderd (snoeien, to prune).

De blauwdruk van een functie waarmee een knoop wordt aangemaakt is gegeven in listing 122. De functie retourneert een pointer naar een blok geheugen, waarvan de kinderen de waarde *NULL* hebben. De aanroepende functie kent eventueel een waarde toe aan het *info* veld. Wanneer er onvoldoende geheugen beschikbaar is, wordt *NULL* geretourneerd. De aanroepende functie zal hierop moeten testen.

```
1       NODE_ *makenode()
2       {
3           NODE_
4               *new;
5
6           if (new = malloc(sizeof(NODE_)))
7           {
8               new->left  = NULL;
9               new->right = NULL;
10          }
11          return (new);
12      }
```

Listing 122: De functie *makenode()*.

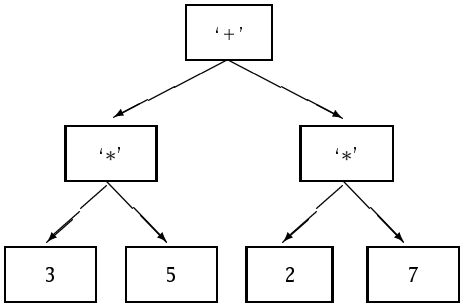
Het doorlopen van de boomstructuur bestaat uit het doorlopen van de linker subboom gevolgd door het doorlopen van de rechter subboom. Afhankelijk van het moment waarop het knooppunt zelf wordt verwerkt, wordt de boom in *preorde, inorde,* of *postorde* afgelopen. Welke methode om de boom te doorlopen wordt gebruikt, is afhankelijk van het type probleem dat met behulp van boomstructuren wordt opgelost. Wanneer we kijken naar het moment waarop de knooppunten worden verwerkt, dan zien we het volgende bij een boom zoals in figuur 17.6:

- Het doorlopen in *preorde* bestaat uit de volgende stappen:
  - Bewerk de informatie van de knoop,
  - Loop de linker boom af,
  - Look de rechter boom af.

  Bij deze methode is de volgorde waarin de knopen worden verwerkt dus K1, K2, K4, K5, K3, K6 en K7. Eerst wordt de informatie van een knooppunt verwerkt, vervolgens wordt de hele linkerboom afgewerkt en tenslotte de hele rechterboom.
- Het doorlopen in *inorde* bestaat uit de volgende stappen:
  - Loop de linker boom af,
  - Bewerk de informatie van de knoop,
  - Loop de rechter boom af.

  Bij deze methode is de volgorde K4, K2, K5, K1, K6, K3 en K7. Hier wordt eerst de hele linkerboom afgewerkt. Hierna wordt de informatie van een knooppunt verwerkt en tenslotte wordt de hele rechterboom afgewerkt.
- Het doorlopen in *postorde* bestaat uit de volgende stappen:
  - Loop de linker boom af,
  - Loop de rechter boom af,
  - Bewerk de informatie van de knoop.

  Bij deze methode is de volgorde K4, K5, K2, K6, K7, K3 en K1. Hier wordt eerst de hele linkerboom afgewerkt, vervolgens wordt de hele rechterboom afgewerkt en tenslotte wordt de informatie in een knooppunt verwerkt.



Figuur 17.7: Een boom voor de expressie $3 * 5 + 2 * 7$.

Het doorlopen van een lege boom is onmiddellijk volbracht. Een lege boom heeft geen kinderen en de knoop bevat geen informatie.

Verschillende manieren om een boom te doorlopen worden bijvoorbeeld gebruikt bij programma's die informatie omzetten van de ene vorm naar een andere vorm. Als voorbeeld noemen we een programma dat expressies leest en omzet naar een andere schrijfwijze van zo'n expressie. Het deel van het programma dat de expressie leest construeert de boom, waarna de boom op een andere wijze wordt doorlopen, om de informatie om te zetten naar een andere vorm. Bij de expressie:

$$3 * 5 + 2 * 7$$

staan de operatoren tussen de operanden. Bij het lezen van deze expressie volgens de manier zoals die bijvoorbeeld in hoofdstuk 16 is behandeld, kan een boom worden geconstrueerd, die is afgebeeld in figuur 17.7. *Hewlett Packard* rekenmachines verwachten uit te rekenen expressies in de zogenaamde Omgekeerde Poolse Notatie (Reversed Polish Notation), waarbij eerst de operanden en dan de operator moet worden ingevoerd. De grammatica voor deze expressies is een postfix-grammatica. Zo'n omzetting wordt gerealiseerd wanneer de boom in postorde wordt doorlopen en datgene dat zich op een blad of knooppunt bevindt wordt afgedrukt. In dat geval ontstaat de expressie:

$$3\ 5 * 2\ 7 * +.$$

De blauwdruk van een functie waarmee een boom in inorde wordt afgelopen, is gegeven in listing 123. Door de functie *process_info()* vóór de eerste recursieve aanroep te plaatsen, wordt verwerking in preorde gerealiseerd. Door de functie *process_info()* na de tweede recursieve aanroep te plaatsen, wordt verwerking in postorde gerealiseerd.

```
void traverse(NODE_ *node)
{
    if (!node)
        return;

    traverse(node->left);
    process_info(node->info);
    traverse(node->right);
}
```

Listing 123: De functie *traverse()*.

De blauwdruk van een functie waarmee subbomen aan een knooppunt worden bevestigd, is eigenlijk hetzelfde als de blauwdruk van een functie waarmee bomen worden doorlopen. Zo'n functie bepaalt of een blad is bereikt en wanneer dat niet het geval is, worden de linker en rechter subbomen toegekend aan de pointers naar het linker en rechter kind en wordt een pointer naar de hele boom geretourneerd. Wanneer wel een blad is bereikt, wordt een pointer naar het blad geretourneerd. Het moment waarop wordt bepaald of een blad is bereikt, is afhankelijk van de grammatica volgens welke de invoer georganiseerd is. Bij de normale algebraïsche notatie van expressies is er sprake van een infix-grammatica, omdat de operatoren tussen de operanden staan. Hierbij zal de boom dan ook volgens een inorde methode worden geconstrueerd. Om volgens de grammaticale regels van een expressie, zoals die in hoofdstuk 16 in 16.5.1 is genoemd, een boomstructuur voor een expressie te construeren moet de functie *expression()* zodanig te worden aangepast dat een boomstructuur in plaats van een getal wordt geretourneerd. De constructie van de boom is dan gebaseerd op een infix-grammatica en de constructie van de boom voor een expressie is gebaseerd op het doorlopen van de boom via de inorde methode. Aangenomen dat de functies *term()* en *factor()* ook een pointer naar een knooppunt retourneren, moet de functie *expression()* de volgende taken verrichten:

1. De wortel van de expressie wordt het adres van de boom die door een aanroep van *term()* wordt opgeleverd.
2. Doe het volgende zolang het volgende teken een + of - is:
    3. Onthoud het adres van de wortel.
    4. Maak een nieuwe knoop aan. Dit wordt de nieuwe wortel van de expressie.
    5. Het linker kind van de (nieuwe) wortel krijgt het adres van de oude wortel (onthouden bij 2).
    6. De operator wordt in de wortel opgeslagen.
    7. Het rechter kind van de wortel krijgt het adres van de boom die door de aanroep van *term()* wordt opgeleverd.
8. Het adres van de wortel wordt geretourneerd.

De functie *expression()* die bij deze herformulering hoort is gegeven in listing 124. Merk op dat de functie sterke overeenkomsten vertoont met de functie *expression()* die op pagina 355 is ontwikkeld. De *typedef* voor de *struct NODE_* is eveneens gegeven in listing 124. Aangezien een knooppunt òfwel een operator heeft, òfwel een getal bevat, kunnen de *struct*-velden voor een operator en een waarde in één *union* worden opgenomen. Vanaf versie 6 van de Microsoft **C** vertaler kunnen *unions* en *structs* worden gedefinieerd, waarin andere *structs* en *unions* naamloos zijn genest. In listing 124 is daarvan gebruik gemaakt, zodat het *operator*-veld met een eenvoudige notatie kan worden bereikt. De vraag of het *operator*- dan wel het *value*-veld moet worden gebruikt, wordt afgeleid uit de waarden van de pointers *left* en *right:* als beide *NULL* zijn, is een eindpunt bereikt en moet het *union* veld *value* worden gebruikt. In alle andere gevallen moet het *operator*-veld worden gebruikt.

```
1     typedef struct NODE_
2     {
3         struct NODE_
4             *left,
5             *right;
6         union
7         {
8             int
9                 operator;
10            double
11                value;
12        }
13    } NODE_;
14
15    NODE_ *expression (void)
16    {
17        NODE_
18            *root,
19            *left;
20
21        root = term();                              // Tekst: 1.
22
23        while ((token == '+' || token == '-')       // Tekst: 2.
24        {
25            left = root;                            // Tekst: 3.
26            root = makenode();                      // Tekst: 4.
27            root->left = left;                      // Tekst: 5.
28            root->operator = token;                 // Tekst: 6.
29            lexer();                                // Nieuw 'token'
30            root->right = term();                   // Tekst: 7.
31        }
32        return (root);                              // Tekst: 8.
33    }
```

Listing 124: De functie *expression()* om een boom te construeren.

Een boom kan ook weer worden verwijderd. Bij dit 'snoeiproces' kan het geheugen van een knooppunt worden vrijgegeven, zodra het geheugen is vrijgegeven dat voor beide kinderen was gealloceerd. Omdat eerst de kinderen moeten zijn verwerkt voordat het knooppunt zelf kan worden vrijgegeven, moet de boom in postorde worden doorlopen. In listing 125 is een voorbeeld gegeven van een functie *prune(),* waarmee het geheugen van een boom weer kan worden vrijgegeven. Wanneer geheugen is gealloceerd voor het inhoudelijk deel van een knooppunt, moet dat geheugen worden vrijgegeven voordat het geheugen van de knoop zelf wordt vrijgegeven.

```
1       #include <malloc.h>
2
3       void prune(NODE_ *node)
4       {
5           if (node)
6           {
7               prune(node->left);
8               prune(node->right);
9               // geef inhoudelijk geheugen vrij
10              free(node);
11          }
12      }
```

Listing 125: De vrijgeven van het geheugen van een boom: de functie *prune()*.

17.5.1 Huffman compressie

De methode om bestanden te comprimeren volgens Huffman (Huffman, 1952) zal worden gebruikt om het gebruik van binaire bomen te illustreren. De centrale gedachte achter de Huffman codering is dat ruimtewinst wordt geboekt, wanneer bitpatronen van variabele lengte worden gebruikt om ASCII-tekens op te slaan. Normaal gesproken is dat niet het geval en wordt elk ASCII-teken opgeslagen in een 8-bits byte. Er kan echter ruimtewinst worden geboekt wanneer ASCII-tekens die erg vaak voorkomen met een kort bitpatroon worden gecodeerd, terwijl ASCII-tekens die zelden voorkomen met een langer bitpatroon worden gecodeerd.

Wanneer een bestand bijvoorbeeld uit 20 'e'-tekens bestaat, is het overbodig om elk teken als een byte te representeren. In plaats van 20 bytes zouden ook 20 enkele 1-bits kunnen worden gebruikt en een afsluitend 0-bit om het einde van de serie aan te geven. In plaats van 20 bytes zijn dan 21 bits nodig. Omdat 21 bits in drie bytes passen wordt het oorspronkelijke bestand door de Huffman codering tot 15 procent van de oorspronkelijke lengte gecomprimeerd[5]. Bij de Huffman codering, die onder andere in

5. In de praktijk zal de compressie iets minder sterk zijn, omdat ook nog informatie over welk teken bedoeld is wordt opgeslagen in het gecomprimeerde bestand.

De Geus, Jongejan en Koelman (1985) en Aho, Hopcroft en Ullman (1983) wordt beschreven, wordt eerst bepaald hoe vaak de afzonderlijke ASCII-tekens in een bestand voorkomen. Vervolgens wordt per ASCII-teken een bitpatroon bepaald volgens welke het desbetreffende teken zal worden gehercodeerd, waarna een gecomprimeerd bestand volgens het bijbehorende bitpatroon wordt weggeschreven. Omgekeerd kan het oorspronkelijke bestand ook weer worden gereconstrueerd uit het gecomprimeerde bestand.

Een methode om een Huffman codering te bepalen zal hier worden behandeld en worden uitgewerkt tot een programma *huffman* waarmee bestanden kunnen worden gecomprimeerd en gedecomprimeerd. Uitgangspunt bij de Huffman codering is een naar frequentie gesorteerd array van de ASCII-tekens in een bestand. Aan de hand van de informatie in dit array wordt een binaire boom geconstrueerd. De bladeren van de boom verwijzen naar elementen van het gesorteerde array en dus naar de ASCII-tekens die in het bestand voorkomen. Een route door de boom, vanaf de wortel naar een blad, definieert het bitpatroon voor het desbetreffende teken volgens de afspraak dat een afslag naar links als een 0-bit zal worden gecodeerd, en een afslag naar rechts als een 1-bit[6]. Wanneer de boom eenmaal is geconstrueerd, kan het bestand worden gecomprimeerd op grond van de geconstrueerde bitpatronen bij de afzonderlijke tekens in het bestand. Voor de decompressie is de boom opnieuw nodig. In de praktijk bestaat het gecomprimeerde bestand dan ook uit de binaire boom, gevolgd door de gecomprimeerde informatie.

De binaire boom wordt als volgt geconstrueerd uit het gesorteerde array:

1. Zolang nog niet alle elementen van het array zijn verwerkt, wordt het volgende gedaan:
    2. De eerste twee elementen worden samengevoegd tot een binaire boom waarvan de wortel als frequentie de som van de frequenties van de beide elementen krijgt.
    3. De geconstrueerde boom wordt zodanig in het array geplaatst dat het array naar frequentie gesorteerd blijft.

Het constructieproces is geïllustreerd in figuur 17.8. In figuur 17.8 (a) is een gesorteerd array gegeven. Het teken *eof* is een afsluitingsteken (sentinel) waarmee in het decoderingsproces wordt aangegeven dat het einde van de te decoderen informatie is bereikt. De sentinel wordt als extra teken aan het bestand toegevoegd om bij de reconstructie van het oorspronkelijke bestand uit het gecomprimeerde bestand het einde van de informatie te kunnen detecteren. De sentinel is nodig omdat, anders dan bij bestanden die uit bytes bestaan, het einde van de informatie bij een bestand dat uit bitpatronen bestaat midden in een byte kan optreden. De eventuele bits

6. De 1/0 codering is een keuze en mag ook andersom worden uitgevoerd: een 1-bit voor de linker afslag en een 0-bit voor de rechter afslag.

achter de sentinel in het laatste byte van het gecomprimeerde bestand worden dan verder niet geïnterpreteerd. De sentinel komt één keer voor, en is dus een pseudo-teken. Na de sentinel staan de feitelijke ASCII-tekens vermeld. De letter *e* komt bijvoorbeeld negen keer voor. In figuur 17.8 (b) zijn de eerste twee elementen (*eof* en *a)* samengevoegd tot een binaire boom met frequentie 9. De boom is geplaatst vóór het element dat een frequentie heeft die kleiner of gelijk is aan de frequentie van de boom. In figuur 17.8 (c) zijn de zojuist gemaakte boom en het element met de letter *e* samengevoegd tot een boom. De frequentie van deze boom is groter dan elke andere frequentie, zodat de nieuwe boom als laatste element in het array wordt geplaatst. Merk op dat het array naar frequentie gesorteerd blijft. In figuur 17.8 (d) zijn de letters *i* en *o* samengevoegd tot een boom die op zijn beurt weer de laatste plaats inneemt in het array. In de laatste stap van het algoritme (in figuur 17.8 (e)) worden de laatste twee elementen samengevoegd en is de boom geconstrueerd.

De codering van de tekens in het bestand wordt gedefinieerd door een route vanaf de wortel van de uiteindelijke boom naar het desbetreffende teken. Om vanaf de wortel bij het teken *a* te komen wordt de route links-links-rechts gevolgd. De bit-codering van de *a* wordt daarom 110. In figuur 17.9 zijn de routes en bit-coderingen voor elk teken samengevat. De compressie bestaat eruit dat elk teken in het te comprimeren bestand wordt weggeschreven als het bijbehorende bitpatroon. De decompressie bestaat eruit dat bij elk bit vanaf de wortel een stap in de boom wordt gezet totdat een blad wordt bereikt, waarna het teken dat bij dat blad hoort kan worden weggeschreven.

Het programma *huffman* dat hier wordt ontwikkeld, wordt aangeroepen met een vlag gevolgd door twee bestandsnamen. De vlag wordt gebruikt om aan te geven of compressie dan wel decompressie wordt gewenst. Het eerste bestand moet bestaan, en is de naam van het te comprimeren of te decomprimeren bestand. Het tweede bestand wordt door het programma *huffman* geschreven, en is dus het uitvoerbestand. Zo zal de aanroep:

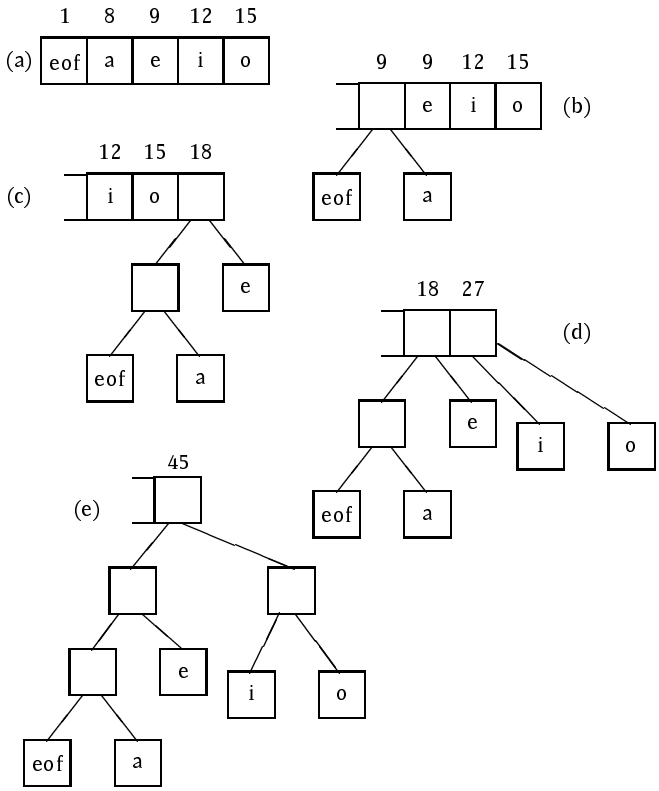
huffman -c huffman.c huffman.huf

ertoe leiden dat het bestand *huffman.c* wordt gecomprimeerd tot het bestand *huffman.huf* De vlag -c geeft aan dat de compressie wordt gevraagd. Omgekeerd zal de aanroep:

huffman -d huffman.huf huffman.c

ertoe leiden dat het gecomprimeerde bestand weer wordt gedecomprimeerd tot het oorspronkelijke bestand. De vlag -d geeft aan dat de decompressie wordt gevraagd.

Op dit moment kan het probleem al zodanig worden geherformuleerd dat de functie *main()* van het programma kan worden geconstrueerd. De herformulering die tot de functie *main()* moet leiden luidt:



Figuur 17.8: De constructie van de binaire boom.

Teken	Route	Bitpatroon
eof	links-links-links	000
a	links-links-rechts	001
e	links-rechts	01
i	rechts-links	10
o	rechts-rechts	11

Figuur 17.9: De uiteindelijke bitpatronen.

1. Beëindig het programma wanneer het aantal argumenten ongelijk is aan drie.
2. Open het te lezen bestand in binaire modus.
3. Open het te schrijven bestand in binaire modus.
4. Als de vlag -c is, comprimeer dan het eerste bestand.
5. Als de vlag -d is, decomprimeer dan het eerste bestand.
6. Geef een foutmelding bij elke andere vlag.

Bij onderdeel 1 wordt een functie *info()* aangeroepen, die informatie geeft over het gebruik van het programma. Deze functie is hier niet verder uitgewerkt. Onderdelen 2 en 3 worden uitgevoerd door een functie *open_file()* die een *FILE* ∗ retourneert, of het programma met een foutmelding laat afbreken, wanneer het bestand niet in de aangegeven modus kan worden geopend. De functie *open_file()* is hier evenmin uitgewerkt. De listing van de functie *main()* is gegeven in listing 126.

```
1       #include <stdio.h>
2
3       FILE
4           *source,
5           *dest;
6
7       void main(int argc, char **argv)
8       {
9           if (argc < 4)                          // Tekst: 1.
10              info(argv[0]);
11          else
12          {
13              source = open_file(argv[2], "rb");  // Tekst: 2.
14              dest   = open_file(argv[3], "wb");  // Tekst: 3.
15              if (!strcmp(argv[1], "-c"))         // Tekst: 4.
16                  compress();
17              else if (!strcmp(argv[1], "-d"))    // Tekst: 5.
18                  decompress();
19              else                                // Tekst: 6
20                  printf("'%s' is not a valid flag\n", argv[1]);
21          }
22      }
```

Listing 126: De functie *main()* van *huffman.c*.

De functie compress()

De stappen die moeten worden ondernomen om een bestand te comprimeren zijn de volgende:

4.1. Tel de frequenties van alle tekens van het bestand in een array *count*, waaraan een extra element voor de EOF-sentinel is toegevoegd.
4.2. Definieer het knooppunten-array *node*.
4.3. Sorteer het knooppunten-array naar frequentie.

4.4. Construeer de binaire boom voor de Huffman codering.
4.5. Schrijf de informatie om de binaire boom te kunnen reconstrueren naar het gecomprimeerde bestand.
4.6. Schrijf de Huffman gecodeerde bitpatronen voor de tekens in het te comprimeren bestand naar het gecomprimeerde bestand.

```
typedef struct
{
    long
        freq;
    char
        *bits;
} CHAR_INFO_;

typedef struct NODE_
{
    struct NODE_
        *left,
        *right;
    long
        freq;
    int
        index;
} NODE_;
```

Listing 127: De *struct* definities van *huffman.c*.

Het array waarin de frequenties van de tekens wordt opgeslagen, wordt gedefinieerd als een array van 257 *structs* waarin een *long* en een *char* pointerveld zijn gedefinieerd. De *char* pointer wordt gebruikt om het uiteindelijke bitpatroon op te slaan; de *long* waarde wordt gebruikt om de frequentie van het teken op te slaan dat als ASCII-waarde de index van de desbetreffende *struct* in het array heeft. Wanneer bijvoorbeeld *node[32].freq* de waarde 100 heeft en *node[32].bits* op de string *"011"* wijst, dan betekent dat dat de spatie (ASCII-teken 32) 100 keer voorkomt en zal worden gecodeerd als het 3 bits grote bitpatroon 0, 1, 1. Element 257 wordt gebruikt voor de *EOF*-sentinel en krijgt per definitie een frequentie 1 toegekend. Bij het sorteren wordt een hulparray gebruikt van *structs,* die ook kunnen worden gebruikt bij de constructie van de uiteindelijke binaire boom. In deze *structs* geeft een *int*-veld de plaats aan in het array met letterfrequenties, een *long* veld de frequentie van de desbetreffende knoop en wijzen twee pointers naar een linker en rechter subboom. Wanneer beide pointers *NULL* zijn, is (per definitie) een blad bereikt; in alle andere gevallen is er sprake van een vertakkingsknoop. De beide *structs* zijn gedefinieerd in listing 127. De functie *compress()* kan nu worden geconstrueerd. Elke stap wordt uitgevoerd door een aparte functie. Het sorteren van het array *node* wordt gedaan door de standaardfunctie *qsort()*. De functie *compress()* is gedefinieerd in listing 128. Bij de ontwikkeling van het programma zullen

bij herhaling controles moeten worden uitgevoerd. Hierbij wordt gebruik gemaakt van een functie *error()*. Deze functie beëindigt het programma na afdruk van een foutmelding, wanneer bij een controle blijkt dat iets niet is gelukt. De functie *error()* wordt bekend verondersteld.

```
1       #include <search.h>
2
3       NODE_
4           *node;
5       int
6           n_nodes;
7
8       void compress()
9       {
10          count_frequencies();                    // Tekst: 4.1
11          define_node();                          // Tekst: 4.2
12          qsort(node, n_nodes, sizeof(NODE_),     // Tekst: 4.3
13                compare_node);
14          build_tree();                           // Tekst: 4.4
15          write_tree("", &node[n_nodes - 1]);     // Tekst: 4.5
16          compress_source();                      // Tekst: 4.6
17      }
```

Listing 128: De functie *compress()*.

Opgave 106.

Construeer de functie *count_frequencies()*. Definieer bij de functie het array *count[]* van 257 elementen. Het laatste element is voor de sentinel en krijgt een *freq* waarde 1.

Opgave 107.

Construeer de functie *define_node()*. Het array *node* moet zo groot worden dat voor elk element in *count* met een frequentie ongelijk 0 een element in *node* beschikbaar is. De functie moet de variabele *n_nodes* een waarde geven die gelijk is aan het aantal elementen in het array *node*. Gebruik de standaardfunctie *calloc()* om het array *node()* te alloceren. De *freq*-velden in het array *node* moeten gelijk worden aan de frequenties van de tekens in het array *count* en de *index*-velden van het array *node* moeten de index van het desbetreffende element in *count* bevatten.

Opgave 108.

Construeer de functie *compare_node()* waarmee het array *node* kan worden gesorteerd. De sortering moet naar toenemende frequentie zijn.

```
1         void build_tree()
2         {
3             register int
4                 last,
5                 index;
6             NODE_
7                 *new;
8
9             for (index = 1; index < n_nodes; index++)// Tekst: 4.3.1
10            {
11                if (!(new = makenode()))            // Tekst: 4.3.2.
12                    error(out_of_memory);
13                if (!(new->right = makenode()))
14                    error(out_of_memory);
15                *(new->right) = node[index];
16                new->left  = &node[index - 1];
17                new->freq  = node[index - 1].freq + node[index].freq;
18
19                for                                 // Tekst: 4.3.3
20                (
21                    last = index + 1;
22                        last < n_nodes;
23                            last++
24                )
25                    if (node[last].freq >= new->freq)
26                        break;
27                last--;
28
29                if (index < last)                   // Tekst: 4.3.4.
30                    memcpy(&node[index],
31                           &node[index + 1],
32                           (last - index) * sizeof(NODE_));
33
34                node[last] = *new;                  // Tekst: 4.3.5.
35            }
36        }
```

Listing 129: De functie *build_tree()*.

```
bytes:      11101001 00001100 01010011 01001001 01100011 110110--
bitnummers: ^1       ^9       ^17      ^25      ^33      ^41
```

Figuur 17.10: De bit-representatie van een binaire boom.

Onderdeel 4.3. wordt gerealiseerd door de functie *build_tree()* De taken van deze functie kunnen als volgt worden geformuleerd:

4.3.1. Zolang er meer dan één element in het array *node* is doe het volgende:

4.3.2. Voeg de eerste twee elementen samen tot een binaire boom, waarbij het *freq*-veld van de wortel van de boom als waarde de som van de *freq*-velden van de beide kinderen van de boom krijgt.

4.3.3. Zoek de laatste indexpositie in het array *node* waarvoor geldt dat de waarde in het *freq*-veld kleiner is dan de waarde in het *freq*-veld van de zojuist geconstrueerde boom.

4.3.4. Verschuif de elementen in het array *node* vanaf index positie 2 tot en met de bij 4.3.3. berekende index één positie naar links.

4.3.5. Plaats de wortel van de in 4.3.2. geconstrueerde boom op de bij 4.3.3. berekende indexpositie.

De implementatie van de functie *build_tree()* is gegeven in listing 129. Onderdeel 4.3.2. maakt gebruik van de functie *makenode()* die eerder in listing 122 is gegeven. Onderdeel 4.3.4. maakt gebruik van de standaardfunctie *memcpy()* om snel een aantal array-elementen te kunnen verschuiven. Aan het eind van het proces staat de wortel van de uiteindelijke binaire boom in element *node[n_nodes - 1]*.

Onderdeel 4.4. wordt gerealiseerd door de functie *write_tree()* De functie krijgt als argumenten het geconstrueerde bitpatroon en het adres van de boom die moet worden doorlopen. Het bitpatroon is geconstrueerd als een string van ASCII-tekens *1* en *0*. De boom wordt in preorde doorlopen, waarbij op elk knooppunt een 1-bit[7] naar het gecomprimeerde bestand wordt weggeschreven en bij elk blad een 0-bit, gevolgd door de waarde van het ASCII-teken. Elk ASCII-teken wordt gecodeerd als een 0-bit, gevolgd door de 8-bits ASCII-waarde, of als een enkel 1-bit wanneer de *EOF*-sentinel moet worden weggeschreven. In figuur 17.10 zijn de bits vermeld, die de representatie van de binaire boom vormen die bij figuur 17.8 is geconstrueerd. In de eerste regel van figuur 17.10 zijn de eerste zes by-

7. Merk op dat hier *bits* worden weggeschreven, en geen bytes. Omdat alleen bytes naar een bestand kunnen worden geschreven, wordt een buffer gebruikt waarin de te schrijven bits tijdelijk worden opgeslagen. Zodra er 8 bits zijn gegenereerd is de buffer vol en kan de buffer als byte worden weggeschreven naar het gecomprimeerde bestand.

tes van het gecomprimeerde bestand gegeven: de boom is opgeslagen in zes bytes. Het eerste byte staat links, het zesde byte rechts. De bits zelf zijn genummerd van 1 tot en met 48. Deze nummering heeft betrekking op de volgorde waarin de bits zijn gegenereerd. In de hier gehanteerde implementatie zijn de bitnummers 1, 9, 17, 25, 33 en 41 in feite de meest significante bits van, respectievelijk, de bytes 1 tot en met 6, terwijl de bitnummers 8, 16, 24, 32, 40 en 48 de minst significante bits van, respectievelijk, de bytes 1 tot en met 6 zijn. De bitnummers staan vermeld in de tweede regel van figuur 17.8: Bit 1 is een 1-bit, bit 9 is een 0-bit, bit 17 is een 0-bit, enzovoorts. De laatste twee bits van het zesde byte horen niet meer tot de representatie van de boom. In figuur 17.8 zijn deze bits niet nader gedefinieerd, maar zijn ze aangeduid met een -.

De interpretatie van de 46 bits waarin de boom is gerepresenteerd, is hieronder gegeven. De genoemde bitnummers hebben betrekking op figuur 17.8:

Bitnummer 1: Een knoop. Dit is de knoop bij de wortel van de boom.

Bitnummer 2: Een knoop. Dit is het linker kind van de wortel.

Bitnummer 3: Een knoop. Dit is het linker kind van het linker kind van de wortel.

Bitnummer 4: Een blad. Dit is het linker kind van het linker kind van het linker kind van de wortel. Hierna volgt de informatie over dit blad.

Bitnummer 5: De sentinel. De codering van de sentinel zal, gerekend vanaf de wortel tot het huidige blad, dus *000* zijn.

Bitnummer 6: Een blad. Dit is het rechter kind van het linker kind van het linker kind van de wortel. Hierna volgt de informatie over dit blad.

Bitnummer 7: Een ASCII-teken. De waarde volgt.

Bitnummers 8-15: De binaire ASCII-waarden zijn omgekeerd gecodeerd. Omdat de bits 8 tot en met 15 respectievelijk 1, 0, 0, 0, 0, 1, 1 en 0 zijn, wordt de (binaire) ASCII-waarde gevonden door deze serie om te keren. De binaire ASCII-waarde is dus 01100001. Dit is ASCII-teken *a*. De codering van de *a* zal, gerekend vanaf de wortel tot het blad, dus *001* zijn. Het linker kind van het linker kind van de wortel is nu volledig bepaald.

Bitnummer 16: Een blad. Dit is het rechter kind van het linker kind van de wortel.

Bitnummer 17: Een ASCII-teken. De waarde volgt.

Bitnummers 18-25: De ASCII-waarde is (binair) 01100101. Dit is ASCII-teken *e*. De codering van de *e* zal, gerekend vanaf de wortel tot het blad, dus *01* zijn. Het linker kind van van de wortel is nu volledig bepaald.

Bitnummer 26: Een knoop. Dit is het rechter kind van de wortel.

Bitnummer 27: Een blad. Dit is het linker kind van het rechter kind van de wortel.

Bitnummer 28: Een ASCII-teken. De waarde volgt.

Bitnummer 29-36: De ASCII-waarde is (binair) 01101001. Dit is ASCII-teken *i*. De codering van de *i* zal, gerekend vanaf de wortel tot het blad, dus *10* zijn.
Bitnummer 37: Een blad. Dit is het rechter kind van het rechter kind van de wortel.
Bitnummer 38: Een ASCII-teken. De waarde volgt.
Bitnummers 39-46: De ASCII-waarde is (binair) 01101111. Dit is ASCII-teken *o*. De codering van de *o* zal, gerekend vanaf de wortel tot het blad, dus *11* zijn. Het rechter kind van van de wortel is nu volledig bepaald en dus is de hele boom nu bepaald. De volgende bits horen daarom niet meer bij de boom, maar zijn direct al de eerste bits van de gecomprimeerde bytes van het oorspronkelijke bestand.
Bitnummer 47-48: De twee laatste bits van het zesde byte horen niet meer bij de representatie van de binaire boom.

Zodra bij het wegschrijven van de boom naar het gecomprimeerde bestand een blad is bereikt, wordt het geconstrueerde bitpatroon toegekend aan het *bits*-veld van het desbetreffende teken in het array *count,* ten behoeve van de uiteindelijke compressie. In eerste instantie is het geconstrueerde bitpatroon de lege string en is het adres van de te doorlopen boom *&node[n_nodes - 1]*.

De werking van de functie *write_tree()* kan nu worden geherformuleerd:
4.4.1. Als het huidige onderdeel van de boom een blad is,
  4.4.2. schrijf dan een 0-bit weg, gevolgd door de codering van het huidige teken.
  4.4.3. Ken het ontvangen bitpatroon toe aan het *bits*-veld van het element van het array *count* waarvan de index in het *index*-veld van het blad is opgeslagen.
  4.4.4. Verlaat de functie.
4.4.5. Schrijf een 1-bit weg.
4.4.6. Bepaal de lengte van het meegegeven bitpatroon.
4.4.7. Alloceer geheugen voor een string die één teken langer is dan het meegegeven bitpatroon.
4.4.8. Kopieer het ontvangen bitpatroon in de zojuist gealloceerde string en breid de string uit met ASCII-teken *1*.
4.4.9. Activeer *write_tree()* met als argumenten het uitgebreide bitpatroon en het adres van het linker kind.
4.4.10. Zet ASCII-teken *0* op de laatste plaats in het uitgebreide bitpatroon.
4.4.11. Activeer *write_tree()* met als argumenten het uitgebreide bitpatroon en het adres van het rechter kind.
Of het huidige onderdeel van de boom een blad is, kan eenvoudig worden nagegaan door te controleren of er een linker kind is. Immers, alle knooppunten hebben ofwel twee kinderen, ofwel geen kind. Het wegschrijven

van een bit wordt overgelaten aan de functie *write_bit(),* die als argument ASCII-teken *1* of *0* krijgt. De overige onderdelen van de functie behoeven geen extra commentaar. De implementatie van de functie *write_tree()* is gegeven in listing 130.

```
1       void write_tree(char *bits, NODE_ *node)
2       {
3           register char
4               *expanded;
5           register int
6               len;
7
8           if (!node->left)                        // Tekst: 4.4.1.
9           {
10              write_bit('0');
11              if (node->index == 256)             // Tekst: 4.4.2.
12                  write_bit('1');
13              else
14              {
15                  write_bit('0');
16                  write_byte(node->index);
17              }
18              if
19              (                                   // Tekst: 4.4.3.
20                  !(count[node->index].bits
21                          = strdup(bits))
22              )
23                  error(out_of_memory);
24              return;                             // Tekst: 4.4.4.
25          }
26
27          write_bit('1');                         // Tekst: 4.4.5.
28          len = strlen(bits);                     // Tekst: 4.4.6.
29          if (!(expanded = malloc(len + 2)))      // Tekst: 4.4.7.
30              error(out_of_memory);
31          strcpy(expanded, bits);                 // Tekst: 4.4.8.
32          strcat(expanded, "1");
33          write_tree(expanded, node->left);       // Tekst: 4.4.9.
34          expanded[len] = '0';                    // Tekst: 4.4.10.
35          write_tree(expanded, node->right);      // Tekst: 4.4.11.
36          free(expanded);                         // Tekst: 4.4.12.
37      }
```

Listing 130: De functie *write_tree()*.

De functie *write_bit()* plaatst een bit in een buffer. Echter, wanneer de buffer vol is, wordt de buffer eerst weggeschreven naar het gecomprimeerde bestand. Voordat een bit in de buffer wordt geplaatst, worden eerst alle bits één positie naar rechts opgeschoven. De functie *write_bit()* is weergegeven in listing 131.

```
1       char
2           bitbuffer;
```

```
3         int
4             bitindex;
5
6         void write_bit(int bit)
7         {
8             bitbuffer <<= 1;
9             bitbuffer |= (bit == '1');
10            if (++bitindex == 8)
11            {
12                fputc(bitbuffer, dest);
13                bitindex = 0;
14            }
15        }
```

Listing 131: De functie *write_bit()*.

De functie *write_byte()* bepaalt in een *for*-statement welke bits van de acht bits in het byte, dat als argument is meegegeven, zijn gezet en activeert de functie *write_bit()* om een al dan niet gezet bit weg te schrijven. De implementatie is gegeven in listing 132.

```
1         void write_byte(register int byte)
2         {
3             register int
4                 index;
5
6             for (index = 0; index < 8; index++)
7             {
8                 write_bit((byte & 1) + '0');
9                 byte >>= 1;
10            }
11        }
```

Listing 132: De functie *write_byte()*.

Opgave 109.

Construeer het programma *huffman.c* zodat de compressie van bestanden kan worden uitgevoerd. Vermeld de benodigde header files voor de typedefinities, de typedefinities voor de definities van de variabelen en definieer alle functies voordat ze worden gebruikt.

De decompressie

Nu de functies voor het comprimeren van bestanden zijn ontwikkeld, zijn de functies om de bestanden weer te decomprimeren eenvoudig te construeren. De constructie van de functies om het oorspronkelijke bestand te decomprimeren wordt daarom overgelaten aan de lezer. Als aanwijzing bij de constructie worden hier een aantal kenmerken van de benodigde functies genoemd. De functie *decompress()* wordt aangeroepen wanneer een bestand moet worden gereconstrueerd. De functie moet twee taken uitvoeren:

5.1. De in het gecomprimeerde bestand opgeslagen boom moet worden ingelezen,
5.2. De resterende informatie in het gecomprimeerde bestand moet worden omgezet naar de oorspronkelijke ASCII-tekens.

Het inlezen van de boom gebeurt met behulp van de recursieve functie *read_tree()* die een pointer naar de wortel van de gelezen boom retourneert. De binaire boom is in het gecomprimeerde bestand volgens een prefix-grammatica weggeschreven, omdat eerst de informatie over de soort knoop is weggeschreven, en daarna pas de informatie die bij die knoop hoort. De functie *read_tree()* waarmee de binaire boom wordt gereconstrueerd, zal daarom in preorde een boom moeten doorlopen. De taken van de functie *read_tree()* kunnen als volgt worden geformuleerd:

5.1.1. Maak een nieuwe knoop aan.
5.1.2. Als het eerstvolgende gelezen bit een 0-bit is, dan is een blad bereikt, in dat geval wordt het volgende gedaan:
    5.1.3. Als het volgende bit een 1-bit is, dan is het sentinelblad bereikt, en krijgt het *index*-veld van het blad de waarde 257, welke waarde verder niet als ASCII-waarde voor kan komen.
    5.1.4. Als het bit een 0-bit was dan volgt de ASCII-waarde van een teken. Deze waarde wordt gelezen en toegekend aan het *index*-veld van het blad.
    5.1.5. Het adres van het blad wordt geretourneerd.
5.1.6. De verwijzing naar het linker kind van de huidige knoop krijgt het adres van de boom die door *read_tree()* wordt geretourneerd.
5.1.6. De verwijzing naar het rechter kind van de huidige knoop krijgt het adres van de boom die door *read_tree()* wordt geretourneerd.
5.1.8. Het adres van de knoop wordt geretourneerd.

De decompressiefunctie *decompress_source()* zelf is geen recursieve functie, omdat voor het bepalen van het ASCII-teken bij het eerstvolgende bitpatroon niet de hele boom moet worden doorlopen, maar slechts een route hoeft te worden bepaald vanaf de wortel tot aan een blad. Deze routebepaling moet worden herhaald voor elk bitpatroon, totdat het bitpatroon van de sentinel is aangetroffen. De functie *decompress_source()* is daarom een iteratieve functie die de volgende taken moet uitvoeren:

5.2.1. In een eeuwige lus krijgt een hulppointer het adres van de wortel van de binaire boom.
5.2.2. Zolang het knooppunt waarnaar wordt gewezen een kind heeft, krijgt de hulppointer een nieuwe waarde:
    5.2.3. Als een 1-bit wordt gelezen krijgt de hulppointer de waarde van het adres van het linker kind van de huidige knoop.
    5.2.4. Als een 0-bit was gelezen wordt het adres van het rechter kind aan de hulppointer toegekend.

5.2.5. Als de waarde van het *index*-veld van het uiteindelijke blad -1 is, is de *EOF*-sentinel bereikt en is de decompressie voltooid. De functie wordt dan beëindigd.

5.2.6. De waarde van het *index*-veld van het blad wordt als ASCII-waarde als een byte naar het gedecomprimeerde bestand geschreven.

Behalve de hier beschreven functies moeten de functies *read_bit()* en *read_byte()* nog worden geconstrueerd. Deze functies werken analoog aan de functies *write_bit()* en *write_byte()*.

Opgave 110.
Breid het programma *huffman.c* uit met de functie *decompress()* en de daarbij benodigde hulpfuncties.

17.6 Hashing

Het laatste onderdeel van dit hoofdstuk heeft betrekking op het terugzoeken van informatie. Tot nu toe is het terugzoeken van informatie gebaseerd geweest op een lineair of binair zoekalgoritme. Ook bij de vorige datastructuren werden dergelijke zoekalgoritmen toegepast: lineair zoeken bij lijsten, binair bij bomen. 'Hashing' is gebaseerd op een andere zoekmethode en leidt zowel tot een zeer snelle opslag- als terugzoekmethode. Ter illustratie: bij lineair zoeken in een lijst van n elementen is, volgens een nader te bepalen tijdseenheid, de gemiddelde zoektijd ongeveer $0.5 * n$ tijdseenheden, bij binair zoeken in een boom van n elementen is de gemiddelde zoektijd ongeveer ${}^{2}log\ n$ tijdseenheden en bij een zoekstrategie die op hashing is gebaseerd is de gemiddelde zoektijd ongeveer 1 tijdseenheid.

Wanneer informatie wordt opgeslagen dan wordt dat deel van de informatie waarop gezocht wordt de sleutel (key) genoemd. Zo zijn de namen van de abonnees de sleutels bij een telefoongids: de naam van een abonnee wordt gezocht en bij een gevonden naam is ook een telefoonnummer beschikbaar. Hashing werkt volgens het principe dat bij elke sleutel een (praktisch) unieke numerieke waarde berekend kan worden. Deze waarde wordt als index in een array gebruikt, waarin de informatie is opgeslagen[8].

Hashing betekent letterlijk 'hakken' of 'fijn maken' en heeft in dit verband niets met cannabis-produkten te maken. De unieke index wordt dan ook verkregen door op de sleutel een bewerking uit te voeren waardoor het verband tussen de index en de oorspronkelijke sleutel volstrekt zoek raakt. Wanneer als voorbeeld de telefoongids wordt gebruikt, dan

8. In principe kan het array ook een bestand zijn. De berekende index is dan een index in het bestand.

worden de namen van de abonnees door het hash-algoritme 'fijngehakt' tot indexen in een array.

De manier om de hash-waarde te bepalen is arbitrair. Zo zouden, om de hash-waarde te bepalen, alle ASCII-waarden van de tekens van een string kunnen worden opgeteld en worden vermenigvuldigd met het aantal tekens van een string. De string *"Corianne"* zou zo de hash-waarde 6520 opleveren. Om dit getal als index in een array te kunnen gebruiken, moet er nog voor worden gezorgd dat de berekende waarde niet groter is dan de omvang van het array. Dat kan eenvoudig door als index de berekende waarde modulo het aantal elementen in het array te gebruiken. Wanneer het array uit 457 elementen bestaat, zou de berekende index dus 6520 % 457 = 122 zijn. Door de modulo bewerking wordt ervoor gezorgd dat de hash-waarde altijd een mogelijke index in het array is.

```
1         unsigned hashpjw (char *key)
2         {
3             register unsigned
4                 h = 0,
5                 g;
6
7             while (*key)
8             {
9                 h <<= 4;
10                h += *key;
11                if (g = h & 0xf000)
12                {
13                    h ^= g >> 8;
14                    h ^= g;
15                }
16                key++;
17            }
18
19            return (h % TABLESIZE);
20        }
```

Listing 133: De functie *hashpjw()*.

Niet alle hash-algoritmes zijn even praktisch. Bovengenoemd algoritme heeft als nadeel, dat de strings *"noot"* en *"toon"* dezelfde hash-waarde opleveren. Dit wordt een botsing (collisie) genoemd. Botsingen kunnen bij alle hash-algoritmes optreden, omdat een in principe groot waardenbereik[9] wordt afgebeeld op een kleiner waardenbereik (het aantal elementen van het array). Hoewel collisies in theorie niet kunnen worden vermeden, is het mogelijk om de waarschijnlijkheid dat ze optreden te verkleinen door een verfijnd hash-algoritme te gebruiken. Aho, Sethi

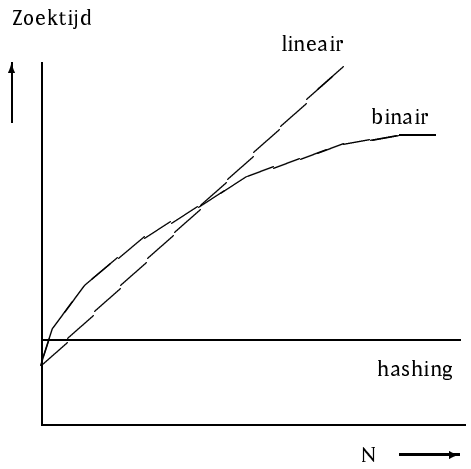
9. Merk op dat een naam ook kan worden opgevat als een nummer door de letters als getalsymbolen van het 26-tallig stelsel (alle letters van het alfabet) op te vatten. Een nog verdere generalisatie bestaat eruit een naam op te vatten als een nummer in het 256-tallig stelsel: de ASCII-tekens zijn dan de getalsymbolen, met als waarde hun volgordenummer.

en Ullman (1986) geven een door Weinberger ontwikkelde hash-functie waarbij de waarschijnlijkheid op collisies relatief klein is. Het algoritme is gebruikt in de functie *hashpjw()*[10], die is gegeven in listing 133. Er zijn een aantal redenen waarom bij Weinberger's hash-algoritme minder collisies optreden dan bij het eerste algoritme, waarbij de ASCII-waarden werden opgeteld:

- Bij Weinberger's algoritme worden tekens tot een waarde samengevoegd, waarbij bij de berekening een gedeeltelijke 'overlap' van de tekens optreedt. Zo wordt, wanneer uit de string *"ab"* een hashwaarde moet worden berekend, eerst de *a* verwerkt. De variabele *h* is dan 0x61. Bij de verwerking van de *b* wordt de waarde in *h* eerst vier bits naar links geschoven zodat *h* dan *0x610* wordt. Daarna wordt de *b* erbij geteld. Bij de optelling speelt het laagste nibble van de *a* (de waarde 1) dus nog een rol. Dat leidt er onder andere toe dat bij de strings *"ab"* en *"ba"* verschillende hash-waarden worden berekend.
- Bij het eenvoudige optellingsalgoritme wordt alleen door de modulo-bewerking voorkomen dat een te grote index wordt berekend. Daaruit kan worden afgeleid dat de hash-waarden die bij verschillende strings van dezelfde lengte worden berekend in een beperkt interval terecht zullen komen. Bij Weinberger's algoritme is echter vóór de modulo bewerking al een overloopbegrenzing aangebracht: als het hoogste nibble van de hash-waarde gelijk is aan *0xf*, dan wordt via een exclusieve *or* bewerking een nieuwe voorlopige waarde bepaald.

Een vuistregel voor het gebruik van hashing is dat hashing daar wordt toegepast waar het bijzonder snel terugzoeken van informatie uit een redelijk omvangrijk bestand noodzakelijk is. Het verband tussen de zoektijden bij lineair zoeken, binair zoeken en hashing is weergegeven in figuur 17.11. De methode die op hashing is gebaseerd is al vrijwel direct sneller dan welke andere methode dan ook, maar de absolute tijdwinst blijft bij kleine aantallen altijd gering. Bij een databestand van zo'n 50 elementen is lineair zoeken vaak zeer acceptabel. Een argument dat bij de overweging om een hash-algoritme te gaan toepassen ook moet meespelen is dat het array waarin de data worden opgeslagen, bij hashing altijd minstens zo'n 25 procent groter moet zijn dan het aantal data-elementen om het risico op collisies klein te houden. Zo'n array kan niet eenvoudig worden uitgebreid met extra elementen, omdat het aantal elementen in het array in het hash-algoritme wordt gebruikt. Bij vergroting van het array zullen de hash-waarden daarom veranderen en zullen de elementen op andere plaatsen in het array terecht komen.

10. De naam *hashpjw* is overgenomen van Aho, Sethi en Ullman en bevat de aanduiding *hash* en de initialen *p, j* en *w* van P. J. Weinberger.



Figuur 17.11: Het verband tussen zoektijden en zoekalgoritmes.

17.6.1 Collisies

Wanneer een nieuw element aan een tabel moet worden toegevoegd, dan kan het voorkomen dat de hash-waarde een index is die wijst op een element in het array dat al bezet is. Op dat moment treedt er een collisie op, omdat meerdere elementen op dezelfde plaats in het array zouden moeten worden opgeslagen. We noemen een paar manieren om het probleem van collisies op te lossen:

1. Er wordt een nieuwe hash-waarde berekend, net zolang tot wèl een vrij element is gevonden.
2. Het array wordt uitgevoerd als een array van pointers, die wijzen op een lijst van bij de desbetreffende index opgeslagen elementen.

Herhaaldelijke indexbepaling

De eerste methode om het probleem van collisies op te lossen bestaat eruit dat bij een collisie een nieuwe index wordt bepaald, waar dan opnieuw kan worden gekeken of het element er kan worden geplaatst. Wanneer ook dàt element bezet is, wordt het proces herhaald totdat een vrije plaats is gevonden, of totdat blijkt dat het array vol is en het nieuwe element dus niet meer kan worden opgenomen. De eenvoudigste manier om een nieuwe index te bepalen is wellicht de volgende indexpositie te nemen: als plaats 15 bezet is, wordt gekeken of plaats 16 nog vrij is enzovoort. Bij de bepaling van de nieuwe index moet rekening worden gehouden met de omvang van het array: de verhoging van de laatste indexwaarde levert de

indexwaarde 0 op. Het array is volledig gevuld wanneer de oorspronkelijke indexwaarde weer is bereikt.

De methode is weliswaar eenvoudig, maar heeft de onprettige eigenschap dat bij collisies klontering (clustering) gaat optreden. Zo zullen de elementen die eigenlijk op de indexposities 7 en 10 zouden moeten worden opgeslagen, in een klont (cluster) worden opgeslagen, zodra er drie elementen voor indexwaarde 7 zijn opgetreden. Een vierde element zou op positie 10 terecht moeten komen, maar omdat 10 ook al bezet is, wordt het de eerste vrije plaats na positie 10: er ontstaat een cluster van elementen vanaf indexpositie 7. Een betere methode is de *add the hash rehash* methode. Hierbij wordt een nieuwe indexwaarde berekend door bij de huidige indexwaarde de oorspronkelijke hash-waarde op te tellen. In deze situatie treden klonten veel minder snel op: de net genoemde vier elementen voor indexpositie 7 komen dan terecht op de indexposities 7, 14, 21 en 28, terwijl een stel elementen voor indexpositie 10 terechtkomen op de posities 10, 20, 30, enzovoort. Er is hier duidelijk minder snel sprake van clustering.

Bij de *add the hash rehash* methode moet er voor worden gezorgd dat de tabelomvang N een priemgetal is, omdat dan bij elke indexwaarde alle mogelijke indexwaarden een keer worden geïnspecteerd (zie ook hoofdstuk 10). Ook hierbij kan dan worden geconstateerd dat de tabel vol is wanneer de oorspronkelijke hash-waarde weer is bereikt. Ter illustratie: wanneer een hash-tabel uit 5 elementen bestaat en bij een sleutel wordt hash-waarde 3 berekend, dan worden de volgende waarden gevonden bij de *add the hash rehash* methode:

3, 1, 4, 2, 0, 3.

Wanneer de oorspronkelijke hash-waarde voor de tweede keer optreedt, zijn alle mogelijke indexwaarden gegenereerd en kan worden geconstateerd dat het hash-array vol is.

Arrays van pointers

Bij een tweede manier om collisies te behandelen wordt het array anders gedefinieerd. De informatie wordt dan niet direct in het array opgeslagen, maar indirect. Elk array-element is een pointer naar een datastructuur waarin de feitelijke elementen zijn opgeslagen. Die datastructuur kan elke uitbreidbare datastructuur zijn: een realloceerbaar array, een lijst, een binaire boom, enzovoort. Elk element dat op een bepaalde indexpositie moet worden opgeslagen, wordt niet direct in het array opgeslagen, maar in de datastructuur waarop door het element wordt gewezen. Met deze methode wordt een combinatie (hybride) van zoekstrategieën toegepast: het hash-algoritme voor een eerste bepaling van de plaats van een element. Vervolgens wordt een tweede (eenvoudig) algoritme toegepast om de uiteindelijke plaats van het element in een eenvoudige datastructuur te bepalen. Omdat collisies bij een goede hash-functie zelden voorkomen,

wordt het tweede algoritme bij voorkeur zeer eenvoudig gehouden: lineair zoeken verdient hierbij de voorkeur, omdat die methode erg efficiënt is bij kleine aantallen.

Bij een array van pointers hoeft het aantal elementen geen priemgetal te zijn, omdat er immers nooit een rehash-bewerking nodig is.

17.6.2 Vullen, zoeken en verwijderen

Wanneer informatie met behulp van het hash-algoritme in een array moet worden opgeslagen, moet eigenlijk worden voorkomen dat het array vol raakt, omdat dan het hash-algoritme degenereert tot een dure methode om lineair te zoeken. Hetzelfde geldt voor het probleem van de collisies: wanneer het aantal collisies te groot wordt, is de tabelomvang te klein gekozen, of is het hash-algoritme aan verbetering toe. Een vuistregel is dat de omvang van het array zo'n 25 procent groter moet zijn dan het maximale aantal elementen dat in het array moet worden opgeslagen. Omdat collisies eigenlijk rampzalig zijn voor de efficiëntie van het algoritme en omdat collisies waarschijnlijker worden naarmate het array voller wordt, is een bescherming tegen een hoge collisiefrequentie wellicht te prefereren boven het domweg hanteren van een groot array.

Wanneer een array een priemgetal als aantal elementen heeft, dan kan het volgende algoritme worden toegepast om een element in het array te plaatsen:

1.1. Bepaal de hash-waarde aan de hand van de sleutel.
1.2. Voer het volgende *MAX_TRY* keren uit:
    1.3. Wanneer de berekende indexpositie vrij is, plaats het element dan op de berekende positie en verlaat de functie succesvol.
    1.4. Bepaal een nieuwe index volgens de *add the hash rehash* methode.
1.5. Verlaat de functie met een mislukking.

Bij onderdeel 1.3 kan de indexwaarde van het opgeslagen element worden geretourneerd ten teken dat de opslag is gelukt. Bij onderdeel 1.5 kan de tabelomvang worden geretourneerd ten teken dat de opslag is mislukt.

De manier om een element op te zoeken, verloopt analoog aan de manier om een element te plaatsen. Bij het opzoeken wordt gecontroleerd of de meegegeven sleutel gelijk is aan de aangetroffen sleutel. Als dat het geval is, wordt de index van het element geretourneerd. Wanneer het element niet wordt gevonden, wordt de tabelomvang geretourneerd. De herformulering voor het zoekalgoritme luidt:

2.1. Bepaal de hash-waarde aan de hand van de sleutel.
2.2. Voer het volgende *MAX_TRY* keren uit:
    2.3. Als de gezochte sleutel op de berekende indexpositie wordt aangetroffen, verlaat de functie dan succesvol.
    2.4. Als op de berekende indexpositie niets staat, beëindig het zoekproces dan.

2.5. Bepaal een nieuwe index volgens de *add the hash rehash* methode.

2.6. Verlaat de functie met een mislukking.

```
1     #include <string.h>
2
3         #define MAX_TRY         5
4         #define HASH_PRIME      491
5
6         typedef struct
7         {
8             char
9                 *key;
10            unsigned
11                freq;
12        } HASH_;
13
14        extern unsigned hashpjw(char *);
15
16        HASH_
17            hash[HASH_PRIME];
18
19        unsigned hash_lookup(char *key)
20        {
21            register unsigned
22                hashvalue,
23                index;
24            unsigned
25                repeat;
26
27            hashvalue = hashpjw(key);               // Tekst: 2.1.
28            index = hashvalue;
29            for (repeat = MAX_TRY; repeat; repeat--)// Tekst: 2.2.
30            {
31                if (!strcmp(hash[index].key, key))  // Tekst: 2.3.
32                    return (index);
33                if (!hash[index].key)               // Tekst: 2.4
34                    break;
35                index = (hashvalue + index) % HASH_PRIME;// Tekst: 2.5.
36            }
37            return (HASH_PRIME);                    // Tekst: 2.6.
38        }
39
40        unsigned hash_store(HASH_ *element)
41        {
42            register unsigned
43                hashvalue,
44                index;
45            unsigned
46                repeat;
47
48            hashvalue = hashpjw(element->key);      // Tekst: 1.1.
49            index = hashvalue;
50            for (repeat = MAX_TRY; repeat; repeat--)// Tekst: 1.2.
```

```
51          {
52              if (hash[index].key <= (char *)1)   // Tekst: 1.3
53              {
54                  hash[index] = *element;
55                  return (index);
56              }
57              index = (hashvalue + index) % HASH_PRIME; // Tekst: 1.4.
58          }
59          return (HASH_PRIME);                    // Tekst: 1.5.
60      }
61
62      unsigned hash_remove(char *key)
63      {
64          register int
65              index;
66
67          if ((index = hash_lookup(key)) != HASH_PRIME)
68          {
69              // free memory associated with hash[index]
70              hash[index].key = (char *)1;
71          }
72          return (index);
73      }
```

Listing 134: Blauwdrukken van hash-functies.

Om een element te verwijderen, wordt het eerst opgezocht. Wanneer het element is gevonden, wordt het verwijderd. Het verwijderen kan betekenen dat geheugen dat voor het element is aangemaakt weer wordt vrijgegeven. In ieder geval moet een indicatie worden opgeslagen dat het element weliswaar vrij is, maar wel in gebruik is geweest. Op die manier kan het zoekalgoritme een ooit gebruikte plaats passeren. Wanneer de sleutel een *char* pointer is, kan de pointerwaarde *NULL* worden gebruikt om een ongebruikte plaats aan te geven en een pointerwaarde 1 om een vrijgegeven plaats aan te geven. Blauwdrukken van de functies *hash_store(), hash_remove()* en *hash_lookup()* zijn gegeven in listing 134. De functies maken gebruik van de hash-functie *hashpjw()* om de eerste hash-waarde te bepalen. Er wordt verondersteld dat het aantal elementen van het array *hash[]* een priemgetal is en dat de sleutel een *char* pointer is: veld *key* van de *struct HASH_*.

17.6.3 Het tellen van woorden in een bestand

Het werken met een hash-tabel wordt geïllustreerd aan de hand van een eenvoudig programma om de frequentie van woorden in een bestand mee te tellen. Het programma noemen we *wc* van 'word count'[11]. Nadat de

11. Er bestaat een programma *wc* op UNIX systemen waarmee het aantal woorden, letters en regels in een bestand kan worden geteld. Het hier beschreven programma doet dus iets anders: het telt de frequenties van de afzonderlijke woorden in een bestand.

frequenties zijn geteld, moeten de woorden en hun frequenties worden afgedrukt.

De hash-table wordt een array van *structs,* waarin het adres van een string en de frequentie van die string opgeslagen wordt. Het aantal elementen van de tabel wordt een priemgetal, zodat de blauwdrukfuncties uit 17.6.2 kunnen worden gebruikt.

De taken van de functie *main()* kunnen als volgt worden geherformuleerd:

1. Open het bestand dat als argument aan het programma is meegegeven.
2. Doe het volgende zolang er woorden in het bestand worden aangetroffen:
   3. Verhoog de frequentie van het woord als het in de hash-tabel wordt aangetroffen.
   4. Wanneer het woord niet in de tabel wordt aangetroffen, neem het dan op in de tabel met frequentie 1. Als dat niet lukt, beëindig het programma dan met de melding dat de hash-tabel vol is.
5. Druk de tabel van woorden en hun frequenties af.

Opgave 111.

Construeer de functie *main()*. Definieer een globale *FILE* pointer *source* bij het te verwerken bestand. Gebruik een functie *getword()* die *NULL* of een pointer naar het gelezen woord retourneert, om de afzonderlijke woorden te isoleren.

De functie *getword()* moet een kopie van het ingelezen woord retourneren. Wanneer dat niet lukt, wordt *NULL* geretourneerd. Een woord wordt eenvoudig gedefinieerd als een aaneenschakeling van één of meerdere lettertekens. De taken van de functie *getword()* kunnen als volgt worden geformuleerd:

2.1. Lees tekens in een eeuwige lus in. Doe bij elk teken het volgende:
   2.2. Als het teken *EOF* is, retourneer dan NULL.
   2.3. Als het teken een letter is, beëindig dan de eeuwige lus.
2.4. Doe het volgende in een eeuwige lus:
   2.5. Plaats het laatst gelezen teken in een buffer. Als dat niet lukt, retourneer dan *NULL.*
   2.6. Lees een teken.
   2.7. Als het teken *EOF,* of geen letterteken is, doorbreek dan de eeuwige lus.
2.8. Retourneer een kopie van de inhoud van de gelezen buffer.

Opgave 112.

Construeer de functie *getword()*. Maak gebruik van de standaardfunctie *strdup()* om een kopie van de inhoud van de buffer te retourneren. Maak gebruik van een lokale buffer van 100 tekens om een woord in op te slaan. Gebruik de macro *isalpha()* om te controleren of een letterteken werd gelezen.

De functie *printtable()* tenslotte heeft tot taak alle woorden in de tabel en hun frequentie te vermelden.

Opgave 113.
Construeer de functie *printtable()*

17.6.4 Het sorteren van de hash-tabel

In het programma *wc* wordt uiteindelijk een lijst afgedrukt van alle woorden en woordfrequenties. Bij het afdrukken worden de woorden vermeld in de volgorde waarin ze in de hash-tabel worden aangetroffen. De woorden worden daardoor in ongesorteerde volgorde afgedrukt. Wanneer de hash-tabel eenmaal is geconstrueerd en verder niet meer hoeft te worden gebruikt, kunnen de STRUCTS van de tabel met een sorteerfunctie worden gesorteerd. De sortering kan dan op basis van de sleutel of frequentie worden uitgevoerd. Nadat de elementen zijn gesorteerd, is het array niet meer als hash-tabel te gebruiken omdat de elementen op een andere plaats terechtkomen dan aangegeven wordt door hun hash-waarde.

Wanneer de hash-tabel zelf onaangetast moet blijven (bijvoorbeeld omdat elementen in de tabel verderop in het programma alsnog moeten kunnen worden opgezocht), dan kan een extra array met pointers naar de elementen in de hash-tabel zodanig worden gesorteerd dat de volgorde van de pointers in het extra array de gesorteerde volgorde aangeeft. De procedure om de woorden in de hash-tabel via een extra array alfabetisch gesorteerd af te drukken, verloopt als volgt:

1. Alloceer een extra array van pointers naar *HASH_structs*. Het aantal elementen in dit array moet gelijk zijn aan het aantal gebruikte elementen in de hash-tabel.
2. Elk element in het extra array wijst op een gebruikt element in de hash-tabel.
3. Het extra array wordt gesorteerd.
4. De woorden en woordfrequenties worden via het extra array gesorteerd afgedrukt.
5. Het geheugen voor het extra array wordt weer vrijgegeven.

Opgave 114.
Wijzig de functie *printtable()* van het programma *wc* zodanig dat de woorden via een extra array in gesorteerde volgorde worden afgedrukt. De functie *hash_store()* kan ten behoeve van de allocatie van het extra array worden uitgebreid met een statement waarin een globale variabele *n_entries* wordt opgehoogd wanneer een nieuw woord in de hash-tabel wordt opgenomen.
Aanwijzing: gebruik de functie *qsort()* voor het sorteren van de extra vector. De functie *qsort()* gebruikt een functie *compare()* die twee pointers naar pointers naar *HASH_structs* krijgt en de return-waarde van de functie *strcmp()* retourneert.

Hoofdstuk 18

Codegeneratoren bison en flex

18.1 Inleiding

Bij programma-ontwikkeling is de constructie van functies die de invoer herkennen in het algemeen een moeizaam proces. Wanneer is de invoer zodanig dat een programma er iets mee kan? Wanneer zijn er foutcondities? Wat moet gebeuren als fouten optreden? De beantwoording van deze en dergelijke vragen kunnen de programma-ontwikkeling nogal vertragen en bemoeilijken. Tot nu toe zijn programma's 'van de grond af aan' ontwikkeld, zoals bij de *help* en *scalc* programma's (zie de hoofdstukken 11 en 16) waar 'met de hand' een lexical analyzer en een grammaticale ontleder geconstrueerd werden.

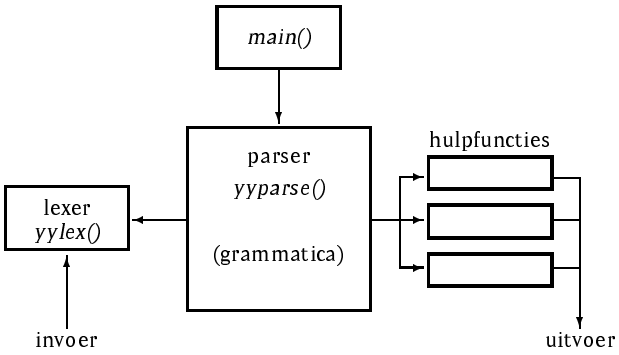
Een volstrekt andere benadering van een ontwikkelprobleem zullen we in dit hoofdstuk introduceren. Hierbij wordt gebruik gemaakt van programma's waarmee we bepaalde onderdelen van bronteksten kunnen *genereren* op basis van een specificatie die vele malen eenvoudiger is dan de software die zich overeenkomstig die specificaties gedraagt. We spreken daarom van *codegeneratoren* die op basis van een eenvoudige specificatie kant-en-klare **C** brontekst genereren.

De codegeneratoren die we voor het voetlicht brengen, helpen ons met de ontwikkeling van twee belangrijke onderdelen van veel computerprogramma's.

De invoertaal specificeert *wat* we bij de invoer kunnen aantreffen en hoe we datgene wat we aantreffen moeten interpreteren. We kunnen hierbij denken aan keywords, aan speciale tekens (bijvoorbeeld operatoren) en aan overige invoer. De invoertaal zullen we gaan specificeren met behulp van *reguliere expressies*. Een programma dat de invoer volgens reguliere expressies herkent, kan worden gemaakt met het programma *flex,* dat een variant is van het programma *lex* dat op veel UNIX systemen standaard beschikbaar is.

De invoerorganisatie specificeert *hoe* we de verschillende invoer-elementen willen aantreffen. Deze organisatie wordt beschreven met behulp van een grammatica, waarin met behulp van regels de structuur van de invoer wordt vastgelegd. Een functie om de regels in ons programma te herkennen kunnen we genereren met het programma *bison*, dat een variant is van het programma *yacc* (een acroniem voor *Yet Another Compiler Compiler*, zie Johnson, 1975), dat op veel UNIX systemen standaard beschikbaar is.

De functie die door een programma als *bison* wordt gegenereerd, heet een 'ontleder' (parser). De theorie van het zelf ontwikkelen van lexical analyzers en parsers is voor een deel al in hoofdstuk 16 aan de orde geweest. We zullen niet verder op deze theorie ingaan: de behandeling ervan zou het doel van dit boek voorbij schieten. Wie echter geïnteresseerd raakt: Aho, Sethi en Ullman's *Compilers: Principles, Techniques and Tools* geeft een



Figuur 18.1: Een programma met functies *yyparse()* en *yylex()*.

goede beschrijving van de achterliggende theorie.

Wanneer gebruik wordt gemaakt van *bison* en *flex* is de grammatica in vrijwel alle situaties het uitgangspunt van de programma-ontwikkeling. Op grond van de grammatica kunnen de elementen beschreven worden die bij de invoer moeten worden herkend en kunnen de bijbehorende regels (reguliere expressies) ontwikkeld worden waaruit FLEX een lexical analyzer construeert. De organisatie van het programma is dan zo dat de parser de lexer om invoer vraagt. De lexer retourneert een invoersymbool op grond waarvan de parser kan beslissen wat er verder moet gebeuren. De lexer is in deze zin een subroutine, oftewel een slaafje van de parser.

De functies yylex() en yyparse()

Vooruitlopend op de feitelijke beschrijving van *bison* en *flex* beschrijven we hier hoe een programma georganiseerd is, dat met behulp van deze twee codegeneratoren gemaakt is. Als voorbeeld van zo'n programma kan de **C** vertaler dienen: ook hier is sprake van een grammatica, die bepaalt hoe invoer geïnterpreteerd wordt. Daarnaast wordt de invoer in stukjes verdeeld door de lexical analyzer.

Deze opzet is in figuur 18.1 weergegeven. Over deze figuur merken we het volgende op:

- De grammatica waaraan de invoer moet voldoen, is gedefinieerd in de parser. Dit onderdeel is als één functie *yyparse()* geïmplementeerd. De naam *yyparse()* is door *bison* bepaald[1].

1. In een programma kunnen meerdere parsers opgenomen worden. Om naamconflicten te vermijden zou de software-ontwikkelaar bij de ene parser *yy* kunnen wijzigen in *y1*, bij de andere parser in *y2*, enzovoort. In dit hoofdstuk zullen we ons echter tot één parser per programma beperken en zullen dan ook geen namen wijzigen.

De functie *yyparse()* wordt door *bison* gemaakt aan de hand van een specificatie die door de ontwikkelaar wordt opgesteld.

- De functie *yyparse()* wordt door de functie *main()* gestart. Andere taken van *main()* zijn bijvoorbeeld het openen van de invoer- en uitvoerbestanden, het initialiseren van variabelen, enzovoort.
- De parser vraagt de lexical analyzer om invoer. De lexer is geïmplementeerd als functie *yylex()*, deze naam is door *flex* bepaald.
  De functie *yylex()* wordt door *flex* gemaakt aan de hand van een specificatie die door de ontwikkelaar wordt opgesteld.
- Het verwerken van invoer brengt (gewoonlijk) *acties* met zich mee. In het voorbeeld van de **C** vertaler bestaan de acties uit het genereren van object-code. Een ander voorbeeld van een programma dat aan de hand van een grammatica acties uitvoert, is de rekenmachine *scalc*: hier bestaan de acties uit het uitrekenen en afdrukken van uitkomsten.
  De acties worden gewoonlijk door hulpfuncties uitgevoerd. De parser roept deze functies aan als dit nodig is. De hulpfuncties moeten uiteraard door de ontwikkelaar geschreven worden.

18.2 Flex en lexical analyzers

Zoals al vermeld, wordt het programma *flex* gebruikt om lexical analyzers op te stellen. In dit deel zullen we de specificatie, die een ontwikkelaar moet opstellen om *flex* een lexical analyzer te laten genereren, nader bekijken. Daarnaast zullen we een voorbeeld geven van een programma dat communiceert met een lexical analyzer die door *flex* is gemaakt.

18.2.1 De specificatie van een lexical analyzer

Zoals al vermeld, bouwt *flex* een lexical analyzer aan de hand van een specificatie die door de programma-ontwikkelaar wordt opgesteld. De specificatie wordt als één bestand aan *flex* aangeboden. In deze specificatie zijn globaal twee delen te onderscheiden:

- Het eerste deel, dat mag ontbreken, bevat letterlijke **C**-code die in de uiteindelijk gegenereerde brontekst komt. In dit deel staan bij voorbeeld *#include* opdrachten. Bij de verwerking van de specificatie kopieert *flex* dit deel letterlijk naar het uitvoerbestand.
- Het tweede deel, dat niet mag ontbreken, bevat *reguliere expressies* die aangeven wat voor soort invoer de lexer kan verwachten. Deze reguliere expressies *moeten* op de eerste kolom, van het specificatiebestand, dus aan het begin van de regel, beginnen. Reguliere expressies komen in 18.2.2 aan de orde. Achter de reguliere expressies staan de *acties* die de lexer moet ondernemen als de invoer wordt gevonden. De acties zijn, zoals we zullen zien, vaak

return-statements. Bij de verwerking van de specificatie bouwt *flex* de functie *yylex()* aan de hand van deze reguliere expressies en de bijbehorende acties.

De beide delen zijn door twee procenttekens (%%) van elkaar gescheiden. Een eenvoudig voorbeeld van een *flex*-specificatie is in listing 135 gegeven. In dit voorbeeld staat in het eerste deel een *#include*-opdracht die voor de uiteindelijke lexer bedoeld is. In het tweede deel introduceren we de meest eenvoudige reguliere expressie: letterlijke tekst. In het voorbeeld zijn drie van deze reguliere expressies opgenomen: expressie *de* die wordt gekozen als in de invoer "de" wordt gevonden, expressie *het* die wordt gekozen als in de invoer "het" wordt gevonden en tenslotte expressie *een* die wordt gekozen als in de invoer "een" wordt gevonden. De acties die *yylex()* moet ondernemen als de reguliere expressies in de invoer worden gevonden, staan achter de reguliere expressies: *yylex()* wordt verzocht om, afhankelijk van de gevonden invoer, de waarde 1, 2 of 3 te retourneren.

Merk op dat end-of-line commentaar in een *flex*-specificatie niet is toegestaan. De *flex*-specificatie is geen **C**, maar lijkt daar een beetje op — de constructeurs van *flex* hebben toegestaan dat standaard-**C** commentaar in de specificatie voorkomt. In de specificatie staan echter ook acties: dit is wèl **C**-code. Als een actie een compound statement is, mag hierin end-of-line commentaar voorkomen.

```
1     /* Deel 1: letterlijke C code */
2     #include <stdio.h>
3
4     %%
5     /* Deel 2: reguliere expressies en acties */
6     de          return (1);
7     het         return (2);
8     een         return (3);
9
```

Listing 135: Een eenvoudige *flex*-specificatie.

Een bijbehorende functie *main()*, die *yylex()* activeert, is in listing 136 gegeven. Deze functie opent eerst een bestand waarvan de naam als argument is meegegeven en roept vervolgens *yylex()* aan totdat *yylex()* de waarde 0 retourneert. Deze return-waarde is een signaal waarmee *yylex()* aangeeft dat de invoer verwerkt is.

Afhankelijk van de return-waarde wordt in de functie *main()* een teller verhoogd, die respectievelijk het aantal "de", "het" of "een" in de invoer bijhoudt. Aan het einde geeft *main()* een overzicht van de aantallen getelde woorden.

```
1     #include <stdio.h>
2
3     extern int
4         yylex (void);
```

```
5
6     FILE
7         *yyin;
8
9     void main (int argc, char **argv)
10    {
11        int
12            ret,
13            de_counter  = 0,
14            het_counter = 0,
15            een_counter = 0;
16
17        if (argc < 2)
18            puts ("Je moet een argument meegeven.");
19        else if (! (yyin = fopen (argv [1], "r")) )
20            printf ("Ik kan bestand %s niet vinden.\n", argv [1]);
21        else
22        {
23            while (ret = yylex ())
24                switch (ret)
25                {
26                    case 1:
27                        de_counter++;
28                        break;
29                    case 2:
30                        het_counter++;
31                        break;
32                    case 3:
33                        een_counter++;
34                        break;
35                }
36            printf (" %4d keer 'de'\n"
37                    " %4d keer 'het'\n"
38                    " %4d keer 'een'\n",
39                        de_counter, het_counter, een_counter);
40        }
41    }
```

Listing 136: Een functie *main()* die met *yylex()* communiceert.

Over de communicatie tussen een (*main()*) functie en *yylex()* maken we de volgende opmerkingen:

- *yylex()* leest invoer uit een stream die met *yyin* wordt aangeduid. Deze variabele, die van het type *FILE∗* is, wijst bij verstek naar *stdin*. Als *yylex()* van een andere stream moet lezen, dan moet de aanroeper ervoor zorgen dat *yyin* met een bijbehorend bestand geopend wordt.
  In het voorbeeld hierboven opent *main()* een bestand dat door de string *argv[1]* wordt aangeduid.
- *yylex()* geeft aan dat het einde van de invoer bereikt is door de waarde 0 te retourneren. Dit is een impliciete actie; de *flex*-specificatie hoeft geen regel in betrekking tot deze actie te bevatten.
  In het voorbeeld hierboven roept *main()* de functie *yylex()* aan zolang als de return-waarde niet nul is.
- De communicatie tussen de functie *yylex()* en andere functies vindt doorgaans plaats met behulp van return-waarden. Daarnaast slaat

de functie *yylex()* de gelezen invoer op in een buffer die *yytext* genoemd wordt en van het type *char[]* is. Andere programmadelen kunnen altijd aan de hand van deze buffer de laatst gelezen invoer bekijken, maar mogen deze buffer niet wijzigen.

Opgave 115.

Voer de teksten uit de listings 135 en 136 in. Noem de bestanden respectievelijk *lexer* en *test.c*. Laat vervolgens bestand *lexer* door *flex* vertalen door de opdracht:

flex lexer

Welk bestand is hierdoor onstaan?

Opgave 116.

Vertaal *test.c* en het bestand dat in opgave 115 ontstaan is. De C vertaler kan één of twee waarschuwingen geven; dit is niet erg.
Link vervolgens de twee object-bestanden aaneen tot een programma *test.exe*. Laat het programma een tekstbestand aflopen. Beantwoord de volgende vragen:
1. Wat doet *yylex()* als invoer wordt gelezen die aan geen van de reguliere expressies voldoet?
2. Telt het programma alleen lidwoorden?

18.2.2 Reguliere expressies in de flex-specificatie

We zullen nu de reguliere expressies nader bekijken die bij de *flex*-specificatie de te verwachten invoer definiëren.

Letterlijke strings staan in de *flex*-specificatie zoals ze in de invoer voorkomen. We hebben hiermee al in listing 135 kennis gemaakt. Zo wordt met de reguliere expressie *else* de sequentie `else` in de invoer bedoeld.

Reguliere expressies kunnen ook gebruik maken van de speciale symbolen ", ., +, ∗, [,], ˆ, (,), {, }, <, > en | (zie later). Als in een reguliere expressie de letterlijke tekens ., [, enzovoort bedoeld worden, moeten de tekens door een \zijn voorafgegaan. Zo wordt met de reguliere expressie *(opa**)* de sequentie `(opa)` bedoeld. In de reguliere expressie is aangegeven dat de haakjes letterlijk bedoeld worden. Als alternatief kan de reguliere expressie tussen dubbele aanhalingstekens geplaatst worden: in dat geval worden *alle* tekens in de reguliere expressie letterlijk opgevat. Zo kan de reguliere expressie *(opa**)* ook als "*(opa)*" geschreven worden.

De \wordt ook gebruikt om de escape-sequences *n*, *b* en *t* aan te geven, die respectievelijk de newline, de backspace en het tabulatieteken betekenen.

De punt (.) wordt gebruikt om een willekeurige letter behalve de newline aan te geven. Zo wordt met de reguliere expressie *o.a* zowel de sequentie `opa` als de sequentie `oma` bedoeld.

Het dollar-teken ($) wordt gebruikt om het einde van een regel aan te geven. Bijvoorbeeld wordt met de reguliere expressie *opa$* de sequentie opa bedoeld, maar alleen als deze sequentie aan het einde van een regel staat.

Het dakje (^) wordt gebruikt om het begin van een regel aan te geven.

Het plus-teken (+) wordt gebruikt om één of meer herhalingen van de vorige letter of sub-expressie aan te duiden. Bijvoorbeeld komt de reguliere expressie *a+rgh* overeen met de sequenties argh, aargh, aaargh, enzovoort. Een tweede voorbeeld is de reguliere expressie .+\., die elke sequentie weergeeft waarin minstens één teken staat en die afgesloten wordt door een punt.

De asterisk (*) wordt gebruikt om nul of meer herhalingen van de vorige letter of sub-expressie aan te duiden. Bijvoorbeeld komt de reguliere expressie *x.** overeen met alle sequenties die met een x beginnen.

Het vraagteken (?) wordt gebruikt om aan te geven dat de vorige letter of sub-expressie nul of één keer moet voorkomen. Bijvoorbeeld komt de reguliere expressie *aa?rgh* overeen met de invoer argh en aargh: de tweede a is optioneel.

Een verzameling van tekens wordt aangegeven met behulp van de symbolen [en]. Tussen de vierkante haken staan gewoonlijk de tekens die bij de verzameling horen. Zo wordt met de reguliere expressie *[Hh]et* de sequentie Het of de sequentie het bedoeld. In een verzameling mogen ook meer dan twee tekens staan: de reguliere expressie *[HhPp]et* komt overeen met het of pet, al dan niet beginnend met een hoofdletter.

De verzameling kan ook als een bereik worden aangegeven. In dat geval staat het teken dat het eerst in de ASCII-tabel voorkomt links, gevolgd door een –, gevolgd door het teken dat het laatst in de ASCII-tabel voorkomt. Bijvoorbeeld geeft de verzameling *[a–zA–Z]* alle letters weer.

Met behulp van de asterisk en de verzameling is ook een reguliere expressie op te stellen die de namen van variabelen of functies in een **C**-brontekst weergeeft. Dit is de reguliere expressie:

*[A-Za-z_][A-Za-z0-9_]**

Complementaire verzamelingen worden aangegeven door een ^ teken als eerste teken in de verzameling te plaatsen. Zo geeft de verzameling *[^eE]* een willekeurig teken weer zolang het geen E of e is.

Sub-expressies worden aangegeven door (en). Zo worden met de reguliere expressie *(ab)+* de sequenties ab, abab, enzovoort, aangeduid. Zonder de haakjes wordt alleen een herhaling van de b aangeduid.

Alternatieven worden aangeduid door |. Zo wordt met de reguliere expressie *opa|oma* de sequentie opa of de sequentie oma aangeduid.

De schuine streep (/) wordt gebruikt om aan te geven dat een expressie wordt aangeduid *alleen* als deze gevolgd wordt door een andere expressie. Zo duidt de reguliere expressie *opa/*" " op de sequentie opa, maar alleen als deze gevolgd wordt door een spatie.

Herhalingen van een vaststaand aantal kunnen met behulp van { en } worden aangegeven. Met de reguliere expressie *a{3,6}* wordt bijvoorbeeld een herhaling van drie tot en met zes a's weergegeven.

Symbool	Betekenis
"	afbakening van een string
\	escape sequence
[ ]	verzameling
^	complementaire verzameling of begin van regel
-	bereik in verzameling
?	optioneel element
.	willekeurig teken behalve *n*
*	herhaling van nul of meer elementen
+	herhaling van één of meer elementen
|	alternatief
( )	sub-expressie
/	expressie gevolgd door expressie
$	einde van regel
< >	miniscanner

Figuur 18.2: Speciale symbolen in reguliere expressies.

De speciale symbolen die in reguliere expressies kunnen worden gebruikt zijn nogmaals in figuur 18.2 weergegeven. De symbolen < en > die in deze figuur genoemd zijn komen in 18.2.4 aan de orde.

18.2.3 Hoe yylex() reguliere expressies vindt

```
opa             { puts ("Regel: opa");        return (1); }
o[pm]a          { puts ("Regel: o[pm]a");     return (2); }
[a-z]+          { puts ("Regel: [a-z]+");     return (3); }
```

Listing 137: Een *flex*-specificatie.

We zullen nu de manier waarop de functie *yylex()* de gelezen invoer met reguliere expressies paart, nader bekijken. De functie *yylex()* hanteert hierbij vier principes:

- Bij het vinden van een reguliere expressie die de gelezen invoer weergeeft, probeert de functie *yylex()* een zo lang mogelijk deel van de invoer te verwerken.

- Als twee of meer reguliere expressies de invoer weergeven en als de lengte van de invoer bij alle expressies even lang is, dan zal de functie *yylex()* de eerste reguliere expressie kiezen.
- De invoer wordt regelgeoriënteerd ingelezen. Dat betekent dat *n*-tekens 'van nature' een scheiding weergeven. Als in een reguliere expressie invoer over meer dan één regel bedoeld wordt, dan moet dit in de expressie expliciet worden weergegeven.
- Als een deel van de invoer met geen enkele reguliere expressie overeenkomt, dan wordt deze invoer naar *stdout* geschreven. Dit is de default actie van *yylex()*. De software-ontwikkelaar zal daarom in het algemeen een vangnet-expressie (catch-all) in de *flex*-specificatie willen opnemen, die de niet herkende invoer negeert. Een reguliere expressie die deze actie uitvoert is de volgende:

  .|*n* ;

  Volgens deze expressie wordt bij elk willekeurig teken of bij een newline-teken geen actie ondernomen. Uiteraard moet een dergelijke vangnet-expressie als *laatste* expressie in de *flex*-specificatie staan.

Aan de hand van een voorbeeld zullen we deze drie principes weergeven. Een voorbeeld van een *flex*-specificatie is in listing 137 weergegeven. In deze specificatie zijn drie reguliere expressies weergegeven: de eerste expressie komt overeen met de invoer `opa`, de tweede expressie komt overeen met de invoer `opa` of met de invoer `oma` en de derde expressie komt overeen met een serie van één of meer kleine letters.

Als de functie *yylex()* de invoer `opa` tegenkomt, dan wordt voor de eerste expressie gekozen. De reden hiervoor is dat een andere regel niet meer invoer kan verwerken en dat deze expressie als eerste in de *flex*-specificatie voorkomt.

Als de invoer `opaal` gelezen wordt, dan kiest de functie *yylex()* voor de derde expressie: deze expressie verwerkt meer invoer dan de eerste of de tweede expressie.

Opgave 117.

Pas de *flex*-specificatie uit listing 135 zodanig aan dat alleen als de lidwoorden "de", "het" of "een" in de invoer voorkomen de functie *yylex()* een waarde retourneert. Bij alle andere invoer hoeft de functie *yylex()* niet te retourneren. Dit kan in de *flex*-specificatie aangegeven worden door een leeg statement (;) als actie op te nemen. De reguliere expressies die tot een lege actie moeten leiden zijn series van één of meer letters of cijfers, het newline-teken en elk ander willekeurig teken.

18.2.4 Miniscanners

Bij het verwerken van invoer kan het voorkomen dat er meer dan één 'soort' invoer bestaat waarbij elk type invoer zijn eigen verwerking heeft.

Een voorbeeld van twee verschillende typen invoer is een **C** brontekst. Hierin heeft een invoerelement, zoals bijvoorbeeld "int", een bepaalde betekenis. Maar als de lexer in de invoer "/*" tegenkomt, dan moet vanaf dat moment de invoer op een andere manier verwerkt worden: een string zoals "int" heeft dan een heel andere betekenis, totdat de lexer in de invoer "*/" tegenkomt.

```
1     /* declaraties voor functie yylex () */
2     #include <stdio.h>
3
4     /* miniscanner : quote */
5     %Start quote
6
7     %%
8     /* reguliere expressies */
9     <quote>\"            { putchar ('\n'); BEGIN 0; }
10    <quote>.             { putchar (yytext [0]); }
11    \"                   { printf ("String:" ); BEGIN quote; }
12    .|\n                 ;
```

Listing 138: Een *flex*-specificatie: de miniscanner.

In listing 138 is de *flex*-specificatie van een lexical analyzer gegeven die twee types invoer kan onderscheiden. Het eerste type, dat het 'nul-type' genoemd wordt, wordt verwerkt door de laatste twee expressies van de specificatie. De expressie en de actie op regel 11:

```
\"      { printf ("String: "); BEGIN quote; }
```

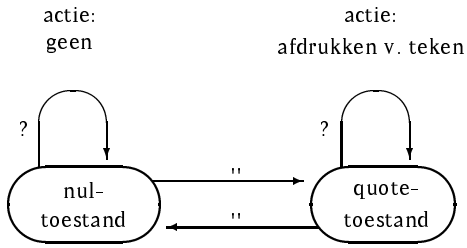
zorgen ervoor dat als in de nul-toestand een dubbel aanhalingsteken gelezen wordt de boodschap *String:* verschijnt en dat vervolgens de lexical analyzer in de quote-toestand terecht komt. De expressie en de actie op regel 12 zorgen ervoor dat als in de nul-toestand een ander teken of een newline-teken gelezen wordt geen actie wordt ondernomen.

De quote-toestand wordt met behulp van de expressies en de acties op de regels 9 en 10 verwerkt. De expressie op regel 9:

```
<quote>\"      { putchar ('\n'); BEGIN 0; }
```

zorgt ervoor dat als de lexical analyzer in de quote-toestand is, en als een dubbel aanhalingsteken gelezen wordt, de lexical analyzer naar de nul-toestand overgaat en een '\n' naar het scherm schrijft. De expressie op regel 10 zorgt ervoor dat als de lexical analyzer in de quote-toestand is en als een willekeurig ander teken dan het dubbele aanhalingsteken gelezen wordt, dit teken naar het scherm geschreven wordt. Het gelezen teken wordt bepaald aan de hand van de buffer *yytext*.

Overigens moet opgemerkt worden dat de lexical analyzer in listing 138 op geen enkele plaats expliciet retourneert. Een impliciete *return* vindt pas plaats als alle invoer gelezen is. Een functie *main()* die deze lexical



Figuur 18.3: De quote-miniscanner als eindige toestandsautomaat.

analyzer zou activeren, zou dus alleen stream *stdin* openen en zou vervolgens slechts één maal de functie *yylex()* aanroepen.

De lexical analyzer die in listing 138 is weergegeven, kan ook als een eindige toestandsautomaat worden weergegeven (zie figuur 18.3). De overgang van de nul-toestand naar de quote-toestand en terug wordt door een dubbel aanhalingsteken veroorzaakt. Daarnaast blijft elke toestand gehandhaafd als geen dubbel aanhalingsteken wordt gelezen; bij deze default transitie is alleen in de quote-toestand een actie.

De implementatie van een miniscanner

In dit onderdeel worden alle facetten van de implementatie van een miniscanner nog eens op een rijtje gezet:

- Allereerst moet bekend zijn *welke* miniscanners er nodig zijn. Bovendien moet vaststaan aan de hand van *welke invoer* welke miniscanner actief moet worden. Een plaatje van een eindige toestandsautomaat, zoals weergegeven in figuur 18.3, kan hierbij zeer nuttig zijn.
- Vervolgens wordt in de *flex*-specificatie een *%Start* regel opgenomen (zie listing 138). In deze regel worden de gebruikte miniscanners genoemd.
- Bij de acties van de reguliere expressies worden de overgangen van de ene naar de andere miniscanner vastgelegd. Dat een expressie alleen voor één bepaalde miniscanner geldt wordt aangegeven door vóór de expressie de naam van de miniscanner tussen < en > te zetten.
  Merk op dat reguliere expressies die *niet* voorafgegaan zijn door de naam van een miniscanner voor *alle* miniscanners, dus niet alleen voor de nul-scanner, gelden.

De overgangen van de ene naar de andere miniscanner worden met behulp van de *BEGIN*-actie weergegeven. Deze actie wordt bij de vertaling van een specificatie door *flex* omgezet naar **C**-code. Achter het woord *BEGIN* wordt de naam van de betreffende miniscanner gezet. Om een overgang naar de nul-toestand (de starttoestand van de functie *yylex()*) te bewerkstelligen, wordt *BEGIN 0* gebruikt.

De volgorde waarin reguliere expressies in de *flex*-specificatie staan is zeer belangrijk. Ten eerste geldt (zoals eerder opgemerkt) dat, als de functie *yylex()* kan kiezen voor twee expressies, de eerste van de twee de voorkeur krijgt.

Ten tweede moet in het achterhoofd gehouden worden dat een reguliere expressie, die *niet* door de naam van een miniscanner wordt voorafgegaan, voor *alle* miniscanners van toepassing is. Dat betekent dat als bijvoorbeeld in listing 138 regel 11 als eerste reguliere expressie wordt opgenomen, de lexical scanner zich heel anders gaat gedragen. In dat geval wordt de quote-miniscanner altijd actief als een dubbel aanhalingsteken gelezen wordt; de actie die de terugkeer naar de nul-miniscanner weergeeft wordt nooit gekozen. In het algemeen moeten dus reguliere expressies die voor miniscanners van toepassing zijn vóór andere reguliere expressies staan.

18.2.5 De opdracht flex

In dit onderdeel wordt de DOS-opdracht waarmee *flex* geactiveerd wordt onder de loep genomen. Vaak wordt bij deze opdracht alleen de bestandsnaam opgegeven, waarin de specificatie voor een lexical analyzer zich bevindt. Het programma *flex* beschikt echter over een aantal interessante opties, waarvan we er twee aan de orde laten komen. Deze opties worden vóór de bestandsnaam van de specificatie voor een lexical analyzer gegeven:

De –i optie wordt gebruikt om *flex* een interactieve lexical analyzer te laten maken. Als deze optie niet wordt gegeven, dan gaat *flex* ervan uit dat de gegenereerde lexer uit een stream zal lezen. In dat geval kan de invoer op een eenvoudige manier plaatsvinden, omdat de functie *yylex()* in het bestand vooruit kan kijken.

Als de –i optie wèl is meegegeven, dan gaat *flex* ervan uit dat de gegenereerde lexer van *stdin* zal moeten lezen. Bijvoorbeeld, bij het *scalc* programma uit hoofdstuk 16 zou een interactieve lexer geconstrueerd kunnen worden.

De –d optie kan gebruikt worden om een lexical analyzer te maken die weergeeft welke reguliere expressie bij welke invoer gekozen is. Deze optie is met name handig als de gemaakte lexical analyzer zich niet gedraagt zoals verwacht.

18.3 Bison en parsers

De codegenerator *bison* vormt een perfecte aanvulling op *flex* voor die programma's die gecompliceerde invoer moeten verwerken. We hebben in hoofdstuk 16 al een programma geconstrueerd (het *scalc* programma), waarbij duidelijk sprake is van een LALR-1 grammatica en waarbij aan de hand van deze grammatica de invoer wordt verwerkt. Bij het *scalc* programma was het echter zo dat alle grammaticale regels en de bijbehorende acties 'met de hand' werden geconstrueerd. In dit deel zullen we laten zien hoe de verwerking van een LALR-1 grammatica met behulp van *bison* gerealiseerd kan worden.

18.3.1 De specificatie van een parser

De grammaticale regels aan de hand waarvan *bison* een parser bouwt, worden als een specificatiebestand aan *bison* aangeboden. Deze specificatie lijkt enigszins op de specificatie die wordt gebruikt om *flex* een lexical analyzer te laten bouwen. De *bison*-specificatie bestaat gewoonlijk uit drie delen:

- Het eerste deel bevat letterlijke **C**-code die in de uiteindelijk gegenereerde brontekst van functie *yyparse()* komt. In dit deel kunnen bijvoorbeeld *#include* opdrachten, declaraties van externe variabelen of functies, enzovoort staan. Dit deel wordt voorafgegaan door %{ en wordt afgesloten door %}.
- Het tweede deel bevat de symbolische namen die *bison* zal gebruiken om terminale symbolen aan te geven. Zoals in hoofdstuk 16 al werd vermeld, analyseert een LALR-1 parser de invoer aan de hand van de terminale symbolen die in de invoer worden aangetroffen.
- Het derde deel bevat de grammaticale regels die de structuur van de invoer weergeven. Zoals we zullen laten zien, lijken deze grammaticale regels zeer sterk op de regels zoals die in hoofdstuk 16 zijn beschreven. In dit deel zijn tevens de *acties* vastgelegd die de parser moet ondernemen als een grammaticale regel gekozen is.

```
1     /* Deel 1: letterlijke C code */
2     %{
3     #include <stdio.h>
4     extern void
5         yyerror (char *);
6     %}
7
8     /* Deel 2: terminale symbolen */
9     %token NUMBER PLUS MINUS TIMES DIVIDE OPENPAR CLOSEPAR
10
11    %%
12    /* Deel 3: nonterminale regels */
13    expression:
14            expression PLUS term
15        |
16            expression MINUS term
17        |
18            term
```

```
19          ;
20
21      term:
22              term TIMES factor
23          |
24              term DIVIDE factor
25          |
26              factor
27          ;
28
29      factor:
30              NUMBER
31          |
32              MINUS factor
33          |
34              PLUS factor
35          |
36              OPENPAR expression CLOSEPAR
37          ;
```

Listing 139: Een eenvoudige *bison*-specificatie.

Een eenvoudig voorbeeld van een *bison*-specificatie is als listing 139 weergegeven. Deze specificatie is een herschrijving van de grammaticale regels voor een expressie die in hoofdstuk 16 zijn beschreven (zie ook figuur 16.9, pagina 349). Over de verschillende onderdelen in een *bison*-specificatie merken we het volgende op:

- In het eerste deel (de letterlijke **C**-code) is een externe functie *yyerror()* gedeclareerd. Deze functie is bij het gebruik van *bison* verplicht: als de parser die door *bison* wordt gegenereerd een fout vindt, dan wordt *yyerror()* aangeroepen. De functie krijgt één argument: een string die meedeelt wat er fout is.
  Een zeer eenvoudige versie van de functie *yyerror()* zou bijvoorbeeld de string die als argument is meegegeven naar het scherm schrijven en zou vervolgens het programma beëindigen.
- In het tweede deel zijn de terminale symbolen van de grammatica genoemd. We suggereren hier een layout waarbij de namen van terminale symbolen in hoofdletters worden gegeven. Dit voorkomt dat terminale symbolen met nonterminale symbolen verward worden.
- In het derde deel zijn de nonterminale regels van de grammatica gedefinieerd. Voor elke regel geldt dat de naam van de regel op de eerste kolom van de *bison*-specificatie begint. Achter de naam volgt een ':'. Hieropvolgend komen de alternatieven voor de regel, gescheiden door '|'. De hele regel wordt door ';' afgesloten.

De *bison*-specificatie zoals in listing 139 weergegeven 'doet niets' behalve het nakijken van de invoer op syntactische fouten. In de specificatie zijn nog geen acties opgenomen. Het opnemen van acties in een *bison*-specificatie wordt in 18.3.2 behandeld.

18.3.2 De acties in een bison-specificatie

De acties die tijdens verwerking van de invoer moeten plaatsvinden, worden in de *bison*-specificatie *achter* de betreffende regels geplaatst. In het volgende voorbeeld wordt bij de verwerking van de invoer de boodschap "Hoera, een getal" getoond als in de invoer een getal staat:

```
factor:
        NUMBER          { puts ("Hoera, een getal!"); }
    |
        MINUS
        factor
    |
        PLUS
        factor
    |
        OPENPAR
        expression
        CLOSEPAR
    ;
```

In dit voorbeeld is alleen de nonterminale regel voor een factor weergegeven. Merk op:

- dat de acties, die moeten worden ondernomen als een grammaticale regel gekozen is, *achter* het laatste terminale of nonterminale symbool van die regel staan,
- dat de acties als één compound statement, dus tussen '{' en '}', gedefinieerd worden.

Een actie zoals in het vorige voorbeeld is weergegeven, is echter doorgaans weinig zinvol. Vaak is het veel handiger als een grammaticale regel iets *oplevert*, met andere woorden retourneert. Om die reden werd bij het *scalc* programma (zie hoofdstuk 16) elke grammaticale regel als één functie gedefinieerd; waarbij elke functie een *double* retourneerde.

Een grammaticale regel kan bij een *bison*-specificatie niet feitelijk retourneren met een *return*-statement, omdat *bison* alle grammaticale regels in één functie *yyparse()* plaatst. Een *return*-statement als actie zou de parser onderbreken. Er is echter wel een symbolische manier waarmee aangegeven kan worden wat een grammaticale regel moet opleveren. Een voorbeeld van deze manier is in listing 140 weergegeven. Bij deze symbolische return-waarden wordt gewerkt met de notaties $$, $1, $2 enzovoort. Hierbij geldt het volgende:

- De waarde die een grammaticale regel moet opleveren wordt met $$ aangeduid.
- De waarde die een grammaticale sub-regel oplevert wordt aangeduid met $ en een getal. Dit getal is het volgnummer van deze subregel. Bijvoorbeeld wordt in de berekening van de waarde die een term moet opleveren (zie regel 4 van listing 140) gebruik gemaakt van $1 en van $3. Het product van deze waarden wordt toegekend aan $$: de waarde die de regel term zelf moet opleveren.

- Als een grammaticale regel meer dan één alternatief heeft, dan begint de $-nummering bij een volgend alternatief weer bij één.

```
1     term:
2             term
3             TIMES
4             factor          { $$ = $1 * $3; }
5         |
6             term
7             DIVIDE
8             factor          {
9                                 if (! $3)
10                                    yyerror ("Division by zero.\n");
11                                $$ = $1 / $3;
12                            }
13        |
14            factor          { $$ = $1; }
15        ;
16
17    factor:
18            NUMBER          { $$ = atoi (yytext); }
19        |
20            MINUS
21            factor          { $$ = -$2; }
22        |
23            PLUS
24            factor          { $$ = $2; }
25        |
26            OPENPAR
27            expression
28            CLOSEPAR        { $$ = $2; }
29        ;
```

Listing 140: De return-waarde van grammaticale regels.

De functie *yyparse()* hanteert een eigen stack waarop de waarden die grammaticale regels opleveren worden bijgehouden. Het type van de elementen van die stack bepaalt dus het type van de opgeleverde waarden. Bij verstek is dit type *int*; de software-ontwikkelaar kan echter het stacktype van *yyparse()* wijzigen door middel van de *#define* opdracht:

#define YYSTYPE new-type

Deze opdracht wordt in het eerste deel van een *bison*-specificatie, dus in het deel waar de letterlijke **C**-code staat, opgenomen. In het geval van een rekenmachine zoals *scalc* zou het stack type *double* zijn.

18.3.3 Hoe yyparse() met yylex() communiceert

De communicatie tussen de parser (de functie *yyparse()*) en de lexical analyzer (de functie *yylex()*) begint bij de *bison*-specificatie. In deze specificatie zijn immers alle terminale symbolen, die bij een grammatica gebruikt worden, vastgelegd. In het geval van de grammatica voor een eenvoudige rekenmachine zijn dit de rekenkundige operatoren, de haakjes en het symbool *NUMBER*.

Bij de verwerking van een grammaticaspecificatie kent *bison* aan elk terminaal symbool een getal toe. Het is dus zaak om de functie *yylex()*, die deze symbolen moet gaan herkennen, van deze getallen op de hoogte te stellen. Dit kan gerealiseerd worden door bij de bewerking van de *bison*-specificatie door *bison* de **–d** vlag op te geven. Een specificatie in een bestand *parser* wordt zo door *bison* bewerkt na de opdracht:

bison –d parser

Bison maakt bij de aanwezigheid van de *–d* vlag een header file aan, waarin de namen van de eindsymbolen en hun numerieke waarden worden gedefinieerd.

Opgave 118.

Voer de *bison*-specificatie van listing 139 in als bestand *parser*. Geef vervolgens de opdracht *bison –d parser*.
1. Welke bestanden zijn hierdoor ontstaan?
2. Hoe nummert *bison* de terminale symbolen die in de *%token*-declaratie van listing 139 zijn genoemd?

Opgave 119.

1. Breid bestand *parser* zodanig uit dat alle grammaticale regels uit figuur 16.9 (zie pagina 349) zijn opgenomen. Gebruik de listings 139 en 140 als voorbeelden.
2. Breid de parser uit met een nieuwe startregel *input*, die specificeert dat de invoer uit één of meer expressies bestaat. Het opgeleverde resultaat van die expressies (opgeleverd via '$1' enzovoort) moet hier afgedrukt worden.
3. Geef vervolgens de opdracht *bison –d parser*. Vertaal de door *bison* gegenereerde C brontekst. Merk op dat in de brontekst van de functie *yyparse()* bepaalde declaraties, zoals van de string *yytext*, kunnen ontbreken. In dat geval moeten die declaraties in het eerste deel van de *bison*-specificatie alsnog worden opgenomen, moet de specificatie nogmaals door *bison* bewerkt worden en moet weer een vertaalslag plaatsvinden.

Aan de hand van de lijst van terminale symbolen en de header file die door *bison* is gemaakt, kan met behulp van *flex* een lexical analyzer worden geconstrueerd. Doorgaans gebeurt dit op de volgende manier:

- In het eerste deel van de *flex*-specificatie, dat is de letterlijke **C**-code, wordt een *#include*-opdracht opgenomen waarbij een lokale header file wordt aangesloten. De naam van deze header file is de door *bison* gegenereerde lijst van terminale symbolen en hun numerieke representatie.
- In de regels-sectie van de *flex*-specificatie worden reguliere expressies opgenomen die de terminale symbolen herkennen. In het geval van de expressiekraker zijn dit getallen, haakjes en rekenkundige operatoren. Als acties bij de reguliere expressies worden *return*-statements opgenomen. De waarden die worden geretourneerd, worden symbolisch aangeduid met behulp van de namen die in de door *bison* gegenereerde header file voorkomen.

Als voorbeeld van een *flex*-specificatie is in listing 141 de lexical analyzer voor de expressiekraker weergegeven. In deze listing is ervan uitgegaan dat de lokale header file die door *bison* is aangemaakt *parser.h* heet.

```
1     /* deel 1: letterlijke C code */
2     #include <stdio.h>
3     #include "parser.h"
4
5         extern int
6             yyerror (char *);
7
8     /* deel 2: reguliere expressies en acties */
9     %%
10    [0-9]+(\.[0-9]+)?          return (NUMBER);
11    "+"                        return (PLUS);
12    "-"                        return (MINUS);
13    "*"                        return (TIMES);
14    "/"                        return (DIVIDE);
15    "("                        return (OPENPAR);
16    ")"                        return (CLOSEPAR);
17    [ \t\n]                    ;
18    .                          yyerror ("Illegal input.\n");
```

Listing 141: De lexical analyzer bij de expressiekraker.

Opgave 120.

1. Voer de *flex*-specificatie van listing 141 in als bestand *lexer*. Let erop dat de header file die in opgave 119 is ontstaan in de *flex*-specificatie wordt aangesloten. Laat *flex* dit bestand naar C-code omzetten door de opdracht *flex –i lexer* te geven. Vertaal vervolgens de onstane C-brontekst.
2. Stel een functie *main()* op waarin de stream *yyin*, waarvan de lexical analyzer leest, *stdin* wordt. Vervolgens moet de functie *main()* de functie *yyparse()* aanroepen.
3. Maak nu de complete expressiekraker met behulp van de functie *main()* en met behulp van *bison* en *flex*.

18.3.4 ASCII-waarden als terminale symbolen

In hoofdstuk 16 werd een grammatica ontwikkeld waarbij ASCII-waarden van tekens als nummers van terminale symbolen gebruikt werden. Analoog hieraan werd een lexical analyzer geconstrueerd die (in sommige gevallen) het ASCII-nummer van een gelezen teken retourneerde.

In tegenstelling tot deze opzet is in listing 139 een *bison*-specificatie gegeven waarin *alle* terminale tekens van een symbolische naam zijn voorzien. In dit onderdeel wordt behandeld hoe in een *flex*- en *bison*-specificatie ASCII-tekens als terminale symbolen gebruikt kunnen worden. De achterliggende idee hierbij is dat bij losse characters in de invoer (bijvoorbeeld bij de + operator) de ASCII-waarde van de characters als tekenwaarde kan dienen.

- In een *bison*-specificatie worden ASCII-tekens in grammaticale regels opgenomen. De tekens worden tussen enkele aanhalingstekens weergegeven. Een voorbeeld is hieronder gegeven:

```
expression:
```

```
        expression '+' term
    |
        expression '-' term
    |
        term
    ;
```

- De lexical analyzer wordt zodanig aangepast dat soms een ASCII-waarde van een gelezen character wordt geretourneerd. In het voorbeeld hierboven zou in de *flex*-specificatie een regel als:

  "+" return ('+');

  kunnen voorkomen. Een andere mogelijkheid is om bij sommige tekens *yytext[0]* te retourneren:

  "+"|-" return (yytext [0]);

  Daarnaast kan in de *flex*-specificatie een vangnet-expressie worden opgenomen waarbij bij *alle* niet herkende invoer *yytext[0]* wordt geretourneerd.

Of losse characters met behulp van hun eigen ASCII-waarde of met behulp van een symbolische naam moeten worden aangeduid, is vaak een kwestie van smaak. Aan de ene kant wordt een *flex*-specificatie korter bij het gebruik van een gecombineerde expressie of een vangnet-expressie, waarbij *yytext[0]* wordt geretourneerd. Aan de andere kant ontstaat bij het gebruik van vangnet-expressies het gevaar dat characters in de invoer die hier niet thuis horen over het hoofd worden gezien.

18.3.5 Bison en expressies

Bij complexe grammatica's is het opstellen van de regels voor expressies één van de lastigste taken. De complexiteit van de regels neemt toe met het aantal operatoren die de grammatica moet herkennen.

In 18.3.1 is een eenvoudige *bison*-specificatie weergegeven die vier operatoren herkent. Bij deze grammatica is elk prioriteitsniveau van de operatoren in één grammaticale regel weergegeven. Hoewel er op deze aanpak van het opstellen van een grammatica niets aan te merken is, bestaat bij *bison* een eenvoudigere manier om prioriteit en associativiteit van operatoren te definiëren. Deze eenvoudigere manier is in *bison* geïmplementeerd om de software-ontwikkelaar te ontlasten bij het opstellen van een grammatica.

```
1    /* Deel 1: letterlijke C code */
2    %{
3    #include <stdio.h>
4    extern void
5        yyerror (char *);
6    %}
7
8    /* Deel 2: terminale symbolen */
9    %token NUMBER
```

```
10    %left '+' '-'
11    %left '*' '/'
12    %right UMINUS UPLUS
13
14    %%
15    /* Deel 3: nonterminale regels */
16    expression:
17            NUMBER
18        |
19            expression '+' expression
20        |
21            expression '-' expression
22        |
23            expression '*' expression
24        |
25            expression '/' expression
26        |
27            '-' expression %prec UMINUS
28        |
29            '+' expression %prec UPLUS
30        ;
```

Listing 142: Een alternatieve *bison*-specificatie voor een expressiekraker.

De prioriteit en associativiteit van operatoren kan al in het tweede deel van een *bison*-specificatie, waar de terminale symbolen worden vermeld, gedefinieerd worden. De terminale symbolen die een zekere prioriteit of associativiteit kennen, worden in dat geval niet met *%token* opgegeven, maar met *%left* en *%right*. In listing 142 is een *bison*-specificatie weergegeven waarin de parser uit 18.3.1 herschreven is. Over deze listing merken we het volgende op:

- In het tweede deel van de *bison*-specificatie worden de terminale symbolen van de grammatica bekend gemaakt. Bij de operatoren wordt bovendien de associativiteit en de prioriteit vastgelegd. Dit gebeurt op de volgende manier:
  - De operatoren bij *%left*- of *%right*-specificaties die als eerste genoemd zijn, hebben de laagste prioriteit. De *%left*- of *%right*-specificaties die het laatst voorkomen, definiëren de hoogste prioriteit.
  - Operatoren met dezelfde prioriteit zijn in één *%left*- of in één *%right*-specificatie opgenomen.
  - Rechts-associativiteit wordt aangeduid met *%right*. Links-associativiteit wordt aangeduid met *%left*.
- Als een operator voor meerdere doeleinden gebruikt wordt (zoals de + operator in optelling en unaire plus) dan wordt in de definitie van de terminale symbolen een schijnoperator opgenomen. In listing 142 komen zo twee schijnoperatoren voor: *UMINUS* en *UPLUS*. Met behulp van deze symbolische namen wordt de associativiteit en prioriteit van respectievelijk de unaire min en de unaire plus gedefinieerd; de – en + tekens kunnen hiervoor niet gebruikt worden omdat zij al in de *%left*-specificaties voor het binair gebruik voorkomen.

In de grammaticale regels zelf kunnen expressies sterk vereenvoudigd weergegeven worden. Zo komt in listing 142 de regel voor:

expression: expression '+' expression

Dankzij het feit dat de prioriteit en de associativiteit van de + operator al is gedefinieerd, is deze grammaticale regel valide. Merk op dat als de prioriteit en de associativiteit van de + operator niet gedefineerd zou zijn, deze regel een onduidelijke grammatica zou definiëren. Het zou dan immers niet duidelijk zijn hoe een expressie als:

$3 + 5 + 4 * 6$

geëvalueerd moest worden: het zou onduidelijk zijn of de + een hogere of lagere prioriteit dan de $*$ had, bovendien zou onduidelijk zijn of de + links- of rechts-associatief was.

Het gebruik van schijnoperatoren komt in listing 142 naar voren op de regels 27 en 29, waar de unaire min- en plusoperaties gedefinieerd worden. De regel:

expression: '–' expression %prec UMINUS

stelt dat als een – teken gevonden wordt er een expressie moet volgen. In dit geval moet de – niet als de binaire min opgevat worden, maar moet de prioriteit en associativiteit van de (schijnoperator) *UMINUS* toegepast worden. De schijnoperatoren *UMINUS* en *UPLUS* komen dus alleen in de *bison*-specificatie voor om de prioriteit en de associativiteit van operatoren vast te leggen.

De acties die de parser moet uitvoeren zijn in listing 142 niet weergegeven. Het implementeren van een rekenmachine met behulp van deze parser zou analoog aan de manier die in 18.3.2 is geschetst kunnen plaatsvinden: de stack van de parser zou op een ander type dan *int* gezet kunnen worden, er zou een super-regel opgenomen kunnen worden die berekende waarden afdrukt en het opleveren van waarden zou in actieblokken met behulp van *$$*, *$1*, *$2* enzovoort gedefiniëerd kunnen worden. Deze implementatie laten we aan de lezer over.

18.3.6 Fouten bij het gebruik van bison

Bij de verwerking van een *bison*-specificatie door het programma *bison* kunnen fouten optreden. Afgezien van fouten in de specificatie zelf, zoals het opgeven van een verkeerde naam door een typfout, kan het gebeuren dat de gespecificeerde grammatica niet voldoet aan de LALR-1 eisen, waardoor *bison* niet in staat is een conflictvrije parser te bouwen. We behandelen in dit onderdeel twee vaak voorkomende fouten in de grammaticaspecificatie en wat eraan gedaan kan worden.

Reduce-reduce conflicten

Conflicten die door *bison* aangeduid worden als *reduce-reduce* conflicten zijn de keuze-conflicten die al in hoofdstuk 16 aan de orde zijn gekomen.

Reduce-reduce conflicten komen er op neer dat de grammatica onduidelijk is naar welk alternatief een grammaticale regel gereduceerd moet worden. Dit wordt geïllustreerd door het volgende fragment van een *bison*-specificatie:

```
input:          rule_a  |  rule_b  |  rule_c ;
rule_a:         'a' ;
rule_b:         'b' ;
rule_c:         'b' ;
```

In dit voorbeeld is het duidelijk dat als in de invoer een *a* wordt gevonden, de *input*-regel gereduceerd kan worden tot *rule_a*. De *input*-regel is daarmee succesvol voltooid. Het keuzeconflict treedt op als in de invoer een *b* gevonden wordt: op dat moment is het onduidelijk of voor *rule_b* of *rule_c* gekozen moet worden.

Zoals al in hoofdstuk 16 is vermeld zijn keuzeconflicten altijd te vermijden.

Shift-reduce conflicten

Conflicten die *bison* aanduidt als *shift-reduce* conflicten, betreffen onvolkomenheden in de grammatica waardoor de parser op een gegeven moment niet kan beslissen of een regel al voltooid is of niet. Dit type conflicten is in hoofdstuk 16 al aan de orde geweest als het onvoltooide regelconflict.

Zoals al vermeld is, zijn onvoltooide regelconflicten niet altijd te vermijden. De software-constructeur moet echter goed op de hoogte zijn van de omstandigheden die de conflicten veroorzaken. Zo kan bij een **C**-achtige grammatica verwacht worden dat één shift-reduce conflict optreedt, namelijk bij het *if-else* statement. Bij een grammaticale specificatie zal de door *bison* gemaakte parser elke *else* bij de dichtstbijzijnde *if* groeperen, zoals dat bij de taal **C** het geval is. Een **C**-achtige grammatica die meer dan één shift-reduce conflict bevat, is niet correct.

Als van tevoren bekend is dat één shift-reduce conflict verwacht kan worden, dan kan dit aan *bison* meegedeeld worden. In zo'n geval kan de regel:

%expect 1

bij het tweede deel van de *bison*-specificatie, waar ook de terminale symbolen worden bekendgemaakt, opgenomen worden. Het effect van de regel is dat *bison* de melding dat een shift-reduce conflict is gevonden niet meer geeft. De parser functioneert er niet slechter of beter door.

18.4 Een applicatie: rasic

In dit onderdeel laten we de in dit hoofdstuk behandelde stof over lexical analyzers en over parsers nog eens de revue passeren. We doen dit aan de hand van een spelletje: *rasic*.

De opzet van het spelletje rasic *is dat een speler een automatische aardappelrooimachine maakt. Het opstellen van deze machine gebeurt met behulp van een speciale stuurtaal die* rasic *heet:* Robotics All-purpose Symbolic Instruction Code. *Zoals we zullen zien, lijkt deze taal enigszins op* **basic** *(Beginners All-purpose Symbolic Instruction Code).*
Een aardappelrooimachine die in rasic *geprogrammeerd is, kan in een veld met aardappelen aan de slag. Gewoonlijk is het veld 20 bij 20 eenheden groot (waarbij op elke eenheid één aardappel staat), maar de veldgrootte kan ook in de* rasic*-code bijgesteld worden. Het doel van de robot bestaat eruit dat over elke aardappel op het veld gereden moet worden, zodat de aardappel wordt gerooid. Als geen enkele aardappel meer op het veld staat, dan stopt het spel.*
Het kan echter ook fout gaan. Ten eerste kan de robot van het veld af in de sloot rijden. Dan is de aardappelrooimachine stuk. Ten tweede kunnen zich op het veld stenen bevinden (het aantal stenen op het veld is door een rasic*-instructie bij te stellen). Als de robot over een steen rijdt, dan gaat het rooimechanisme kapot. Zodra de aardappelrooimachine kapot is, stopt het spel (de speler heeft dan verloren).*

De instructies van rasic

De instructies in de *rasic*-taal bestaan uit een regelnummer, waarachter de feitelijke code staat. Een regelnummer wordt als label gebruikt om vanuit andere programmadelen een sprong naar deze regel te kunnen maken. Een instructie wordt afgesloten door een newline-teken; dat wil zeggen dat per regel één instructie mag voorkomen. Om de leesbaarheid van een *rasic*-programma te vergroten, mogen ook lege regels voorkomen.

De instructies van de *rasic*-taal zijn hieronder opgesomd:

drive De *drive*-opdracht heeft één argument: de richting waarin gereden moet gaan worden. Na het uitvoeren van de opdracht staat de motor van de robot aan; dat wil zeggen dat doorgereden wordt totdat een andere *drive*-opdracht uitgevoerd wordt of totdat de *stop*-opdracht (zie later) uitgevoerd wordt.
Het argument van de *drive*-opdracht is een richting. Dit kan *north*, *east*, *south* of *west* zijn. Evengoed kan de richting als een getal worden opgegeven, waarbij 0 voor *north*, 1 voor *east*, 2 voor *south* en 3 voor *west* staat. Als het argument een getal hoger dan 3 is, dan wordt deze waarde geconverteerd naar 0, 1, 2 of 3: richting 4 is weer *north*.
Daarnaast mag het argument *random* zijn. In dat geval wordt een toevalsrichting gekozen. Tenslotte mag de richting door *direction* worden gegeven; in dat geval wordt gereden in de richting waarin het laatst gekeken werd (zie de *look*-opdracht).

De end opdracht. De *end*-opdracht beëindigt het spel.

De field opracht. De *field*-opdracht heeft één argument dat een getal moet zijn. Bij het uitvoeren van de opdracht wordt de veldgrootte op het aantal eenheden gezet. Zo zorgt de opdracht:

10 field 5

ervoor dat het aardappelveld vijf bij vijf eenheden groot gemaakt wordt.
De *field*-opdracht mag alleen voorkomen als nog geen acties in het programma (zoals het rijden) zijn geweest.

De goto opdracht. De *goto*-opdracht heeft één argument: een getal. Bij het uitvoeren van de opdracht wordt een sprong uitgevoerd naar de regel waarvan het nummer door het getal wordt weergegeven.

De if opdracht. De *if*-opdracht wordt gevolgd door een vergelijkende expressie, gevolgd door *then* en een regelnummer. De vergelijkende expressie moet bestaan uit twee argumenten, die door <, >, <=, >=, == of != worden vergeleken. De argumenten kunnen zijn:

- een richting, zoals *north*, *east*, *south* of *west*,
- een getal,
- *random*, dat voor een toevalswaarde tussen 0 tot en met 3 staat,
- *direction*,
- *distance*.

Argument *direction* geeft de richting aan waarin gereden wordt. Argument *distance* geeft de afstand aan tot de rand van het veld of tot de dichtstbijzijnde steen die bij de laatste *look*-opdracht is bepaald.
Als de vergelijkende expressie waar is, dan wordt een sprong uitgevoerd naar het regelnummer dat achter *then* is gegeven.
De *if*-opdracht mag gevolgd worden door *else* en een regelnummer. Als de vergelijkende expressie onwaar is, dan wordt in dit geval een sprong naar deze regel uitgevoerd.

De look opdracht. De *look*-opdracht wordt gevolgd door een argument dat een richting aangeeft. Het argument mag hetzelfde zijn als bij de *drive*-opdracht. Bij de *look*-opdracht wordt de afstand bepaald in de bedoelde richting tot de rand van het veld of tot de dichtstbijzijnde steen. De afstand kan als *distance* in een *if*-opdracht worden opgevraagd.

De object opdracht. De *object*-opdracht mag, evenals de *field*-opdracht, alleen voorkomen als nog geen opdrachten zijn geweest die een actie tot gevolg hebben (zoals de *drive*-opdracht). De *object*-opdracht wordt gevolgd door een getal.
Bij het uitvoeren van de *object*-opdracht wordt het bedoelde aantal stenen over het aardappelveld verspreid.

De randomize opdracht. De *randomize*-opdracht heeft geen argumenten. Bij het uitvoeren van de opdracht wordt de random-generator van het spel geïnitialiseerd, zodat eropvolgende *random* waarden meer

gerandomiseerd zijn[2].

De sleep opdracht. De *sleep*-opdracht wordt gevolgd door één argument dat een getal is en zorgt ervoor dat de aardappelrooimachine een tijdlang geen instructies uitvoert. Het rijden en rooien kan echter doorgaan als de motor aan is. Zo betekent:

20 sleep 1

dat één loze opdracht wordt uitgevoerd.

De stop opdracht. De *stop*-opdracht heeft geen argumenten en heeft tot gevolg dat de motor van de robot wordt uitgeschakeld.

```
1  object 30

10 look random
20 if distance >= 5 then 1000
30 if distance == 4 then 2000
40 if distance == 3 then 3000
50 if distance == 2 then 4000
60 goto 10

1000 if random < 1 then 4000
1010 drive direction
1020 sleep 3
1030 stop
1040 goto 10

2000 drive direction
2010 sleep 2
2030 stop
2040 goto 10

3000 drive direction
3010 sleep 1
3020 stop
3030 goto 10

4000 drive direction
4010 stop
4020 goto 10
```

Listing 143: Een voorbeeld van een *rasic*-programma.

Een voorbeeld van een *rasic*-programma dat een aardappelrooimachine aanstuurt, is in listing 143 weergegeven. In deze specificatie worden op regel 1 dertig stenen op het aardappelveld geplaatst. Op regel 10 wordt vervolgens een willekeurige richting bepaald en wordt de afstand tot de rand van het veld of tot een steen in die richting bepaald. Afhankelijk van de afstand wordt dan op de regels 20, 30, 40 en 50 een stuk code geactiveerd. De code die bijvoorbeeld geactiveerd wordt als de bepaalde afstand 4 bedraagt, staat op regel 2000. Op deze regel wordt de motor aangezet om in de bepaalde richting te rijden. Vervolgens worden twee loze instructies uitgevoerd (terwijl het rijden doorgaat) en wordt de motor gestopt.

2. Het randomiseren van getallen is eerder in opgave 30 (pagina 117) aan de orde geweest. We zullen er in dit hoofdstuk nog op terugkomen.

Daarna wordt een sprong naar regel 10 uitgevoerd, zodat het proces weer van voorafaan begint.

18.4.1 Implementatie van rasic

Het implementeren van een spelletje zoals dat hierboven beschreven is, houdt in dat een kleine vertaler voor een specifieke taal geschreven moet worden. Zulke kleine vertalers zijn gangbaarder dan op het eerste gezicht zou lijken: zo hebben veel editors macro-faciliteiten waarbij een serie acties van de editor aan een toets gekoppeld wordt. De macro's kunnen gewoonlijk als een bestand bewaard worden. Bij het opstarten van de editor wordt het bestand met de macro's ingelezen[3], zodat alle macro's weer beschikbaar zijn. Voor dit inlezen wordt van een kleine vertaler gebruik gemaakt omdat de invoer volgens een zekere grammatica is opgebouwd; het vertalen is in deze context de omzetting van tekstvorm (in een bestand) naar een interne representatie (waar een editor mee kan werken).

Bij de implementatie van de *rasic*-vertaler zullen we een *interpreter* bouwen, analoog aan de **basic**-interpreter[4]. Dit houdt in dat robotprogramma's in tekstvorm in een bestand staan en bij het spelen door de interpreter worden ingelezen. De interpreter voert vervolgens de instructies van het robotprogramma één voor één uit. Het grote voordeel van een interpreter is dat het inlezen en het uitvoeren van een (robot-) programma eenvoudig gerealiseerd kan worden. Het nadeel van een interpreter ligt in de traagheid waarmee een programma uitgevoerd wordt.

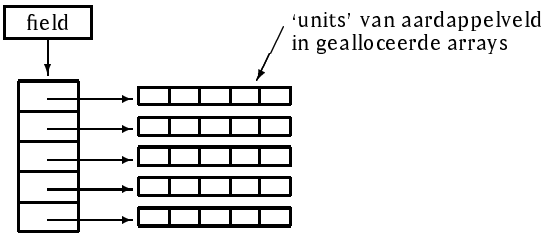
Bij het maken van het *rasic*-spel zullen de volgende onderwerpen de revue passeren:

- Allereerst zullen de datatypen die bij het spel een rol spelen, gedefiniëerd worden. Zoals goede software-ontwikkelaars betaamt, zullen we de constructie van een gecompliceerd programma beginnen met de data.
- Vervolgens zal het inlezen van een robotprogramma aan de orde komen. De grammatica van het robotprogramma zullen we met behulp van *bison* realiseren; het feitelijke inlezen van een bestand met een robotprogramma zullen we aan *flex* overlaten.
- Vervolgens zal het uitvoeren van een ingelezen robotprogramma aan de orde komen, waarbij we enige zorg aan de 'buitenkant' van het spel zullen besteden.

18.4.2 De datatypen en de data van rasic

Het speelbord. De eerste vraag bij het vastleggen van de data voor het *rasic* programma betreft het speelbord. We zullen op het speelbord vastleg-

3. Een voorbeeld van zulke editors zijn het GNU MicroEmacs programma en de bij het ICCE ontwikkelde editor *oe!*.
4. Tegenwoordig zijn ook **basic**-compilers op de markt. Het oorspronkelijke idee van **basic** is echter met behulp van een interpreter gerealiseerd.



Figuur 18.4: Het aardappelveld bij het *rasic* spel.

gen of een eenheid van het aardappelveld gerooid is, of er een aardappel op staat, danwel of er een steen ligt. Hetgene dat op het aardappelveld aangetroffen kan worden, wordt gedefinieerd door een *enum* type:

```
typedef enum
{
    obj_empty,
    obj_potato,
    obj_stone,
} FIELD_;
```

De staat van het hele veld kan uiteraard bewaard worden in een array. Omdat echter de omvang van het veld door een robotprogramma gewijzigd kan worden, kiezen we voor een andere aanpak waarbij het aardappelveld realloceerbaar is. We laten een pointer *field* wijzen naar een array van pointers, die elk naar een array van *FIELD_* elementen wijzen. Dit is geïllustreerd in figuur 18.4. Deze opzet van de opslag van een spelbord is overigens ook gebruikt in het *horses* programma (zie hoofdstuk 15).

De instructies. Een andere vraag is hoe de instructies van een robotprogramma opgeslagen zullen worden. Als we naar de instructieset van de *rasic* taal kijken, dan zien we dat de *if*-instructie de meeste informatie bevat. Een regel met een *if*-instructie kan er bijvoorbeeld als volgt uitzien:

10 if distance <= 6 then 100 else 200

In deze instructie komen de volgende elementen voor:

- een regelnummer (in dit voorbeeld 10),
- het type van de instructie (*if*),
- de linker operand (*distance*),
- een operator (<=),
- de rechter operand (6),
- een *goto*-regelnummer (100),
- een *else*-regelnummer (200).

Het type van een instructie kan met behulp van een *enum* type opgeslagen worden. Op die manier kan in de brontekst en met symbolische namen aan instructies gerefereerd worden. De *enum* voor het instructietype is hieronder gegeven:

```
typedef enum
{
    i_drive,
    i_end,
    i_field,
    i_goto,
    i_if,
    i_look,
    i_object,
    i_randomize,
    i_sleep,
    i_stop,
} INSTRUCTION_TYPE_;
```

Bij het opslaan van een operand moet het type van een operand en mogelijk een numerieke waarde opgeslagen worden. Een operand kan de volgende waarden aannemen:

- *direction*, de richting waarin gereden wordt of de richting waarin het laatst een afstandsbepaling werd gedaan,
- *distance*, de laatst bepaalde afstand bij een *look*-instructie,
- *random*, een toevalswaarde tussen 0 tot en met 3,
- een getal.

Het type van een operand wordt vastgelegd met behulp van een *enum* type *OPERAND_TYPE_*. De operand zelf wordt vastgelegd als het type van de operand en als een *int* waarde die een getal representeert:

```
typedef enum
{
    o_direction,
    o_distance,
    o_random,
    o_number,
} OPERAND_TYPE_;

typedef struct
{
    OPERAND_TYPE_
        o_type;
    int
        val;
} OPERAND_;
```

De operator die twee operanden vergelijkt kan ook met behulp van een *enum* type weergegeven worden:

```
typedef enum
{
    o_smaller,
    o_greater,
    o_smallerequal,
    o_greaterequal,
    o_equal,
```

```
    o_notequal,
} OPERATOR_;
```

Met behulp van de gedefinieerde *structs* en *enums* kan nu het type *INSTRUCTION_* gedefinieerd worden:

```
typedef struct
{
    int
        line_no,
        goto_no,
        else_no;
    INSTRUCTION_TYPE_
        i_type;
    OPERAND_
        o_left,
        o_right;
    OPERATOR_
        op;
} INSTRUCTION_;
```

18.4.3 Het inlezen van een robotprogramma

Bij de implementatie van het inlezen van een robotprogramma beginnen we met het opstellen van de grammatica. De grammatica van een robotprogramma moet de volgende specificaties bevatten:

- Een robotprogramma is een serie van één of meer regels.
- Elke regel bestaat uit een getal (het regelnummer), gevolgd door een instructie. De regel wordt afgesloten door een newline-teken. Hiermee leggen we vast dat per regel één instructie mag voorkomen.
  Een regel kan ook uit één newline-teken bestaan. Hiermee staan we toe dat in een robotprogramma lege regels voorkomen.
- Een instructie kan vele vormen aannemen. Een voorbeeld is het keyword *drive* gevolgd door één operand. Een ander voorbeeld is het keyword *stop*, dat geen operand heeft.
- De *if*-instructie bestaat uit het keyword *if*, gevolgd door een conditie, gevolgd door het keyword *then*, gevolgd door een regelnummer. Eventueel kan achter dit regelnummer het keyword *else* en nogmaals een regelnummer volgen. De conditie bestaat uit een operand, een operator en een tweede operand.
- Een operand is één van de keywords *distance*, *direction*, *random* of een getal. Een operator is één van de tekens $<$, $>$, $<=$, $>=$, $==$ of $!=$.
- Een getal kan ofwel een numerieke waarde zijn, of het kan één van de keywords *north*, *east*, *south* of *west* zijn. Hiermee wordt bewerkstelligd dat bij het inlezen van een robotprogramma de keywords *north*, *south*, *east* en *west* meteen naar een interne getalrepresentatie geconverteerd worden.

Opgave 121.
Geef de hierboven omschreven grammatica weer in een LALR-1 vorm.

De bison-specificatie van de parser

De conversie van de grammatica die in de vorige opgave is opgesteld naar een *bison*-specificatie is niet moeilijk. Het enige punt dat aandacht verdient is hoe de *acties* van de parser gestalte moeten krijgen. Het moge duidelijk zijn dat na de ontleding door de parser er een serie van *INSTRUCTION_ structs* moet bestaan, zodat bij het uitvoeren van een robotprogramma de informatie die in deze *structs* is vastgelegd, uitgevoerd kan worden. Het stack-type van de parser, dus het datatype dat met behulp van $$, $1, enzovoort wordt doorgegeven, definiëren we daarom als *INSTRUCTION_*.

```
1     %{
2     #include "rasic.h"
3
4     #define YYSTYPE INSTRUCTION_
5     %}
6
7     %token DRIVE ELSE END FIELD IF GOTO LOOK OBJECT RANDOMIZE SLEEP STOP THEN
8            DIRECTION DISTANCE RANDOM
9            NORTH EAST SOUTH WEST
10           SMALLER GREATER SMALLEREQUAL GREATEREQUAL EQUAL
11           NUMBER
12
13    %%
14    program:
15            program
16            line
17        |
18            line
19        ;
20
21    line:
22            number
23            instruction
24            '\n'                    {
25                                        $$ = $2;
26                                        $$.line_no = $1.o_left.val;
27                                        assert (&$$);
28                                    }
29        |
30            '\n'
31        ;
32
33    instruction:
34            DRIVE
35            operand                 {
36                                        $$.i_type = i_drive;
37                                        $$.o_left = $2.o_left;
38                                    }
39        |
40
41    .
42    .
43    .
44
45    number:
46            NUMBER                  {
```

```
47                                      $$.o_left.o_type = o_number;
48                                      $$.o_left.val = atoi (yytext);
49                                  }
50          |
51              NORTH               {
52                                      $$.o_left.o_type = o_number;
53                                      $$.o_left.val = d_north;
54                                  }
55          |
56              EAST                {
57                                      $$.o_left.o_type = o_number;
58                                      $$.o_left.val = d_east;
59                                  }
60          |
61              SOUTH               {
62                                      $$.o_left.o_type = o_number;
63                                      $$.o_left.val = d_south;
64                                  }
65          |
66              WEST                {
67                                      $$.o_left.o_type = o_number;
68                                      $$.o_left.val = d_west;
69                                  }
70          ;
```

Listing 144: De *bison*-specificatie voor de robottaal.

Het eerste deel van de *bison*-specificatie voor de parser is in listing 144 gegeven. In deze listing wordt in het eerste deel van de *bison*-specificatie (de letterlijke **C** code) de lokale header file *rasic.h* aangesloten. In deze header file worden de variabelen en de functies die voor het *rasic* programma van belang zijn (zoals de buffer van de lexical analyzer *yytext*) gedeclareerd. Op dit bestand komen we in listing 147 terug. Daarnaast wordt het stack-type van de parser als het type *INSTRUCTION_* gedefinieerd.

In het tweede deel van de *bison*-specificatie worden alle eindsymbolen, waarvan de grammatica gebruik maakt, gedefinieerd. In overeenstemming met onze suggestie zijn deze namen in hoofdletters gegeven.

In het derde deel van de specificatie tenslotte worden de grammaticale regels voor een robotprogramma gegeven. Een aantal grammaticale regels voor de instructies zijn weggelaten (op de plaats van de puntjes). Merk op dat de opslag van een instructie in de grammaticale regel voor een *line* wordt overgelaten aan de functie *assert()*. Op deze functie komen we later, in listing 146, terug.

Het doorgeven van *INSTRUCTION_ structs* van de ene grammaticale regel naar de andere grammaticale regel vereist enige uitleg. Het is nodig dat een eenduidige afspraak wordt gemaakt over *welke* velden van de *INSTRUCTION_ structs waarvoor* gebruikt worden. Bijvoorbeeld is in listing 144 te zien dat de grammaticale regel voor een *number* zijn informatie in het *o_left* veld van de opgeleverde *struct* zet (dit veld is zelf een *struct* en bestaat uit twee velden: *o_type*, waarin het type van een operand wordt bewaard en *val*, waarin de numerieke waarde van een operand wordt bewaard). Dat betekent dat een grammaticale regel die gebruik maakt van

de *number*-regel de informatie over een getal uit het *o_left* veld van de opgeleverde *struct* moet halen.

Voor het doorgeven van *INSTRUCTION_ structs* van de ene grammaticale regel naar de andere grammaticale regel spreken we een aantal conventies af. In de onderstaande opsomming staan deze afspraken, waarbij aan grammaticale regels wordt gerefereerd zoals deze bij het antwoord op opgave 121 zijn genoemd:

- De grammaticale regel *number* vult het *o_left* veld van de opgeleverde *struct*. Hierbij wordt het *o_type* veld van de operand op *o_number* gezet, hetgeen betekent dat de operand een getal is. Veld *val* van de operand wordt op de numerieke waarde van de gelezen invoer gezet. De terminale symbolen *north* enzovoort worden hier naar een numerieke waarde geconverteerd.
- De grammaticale regel *operator* vult het *op* veld van de opgeleverde *struct*. Dit veld krijgt de waarde van één van de constanten *o_smaller*, *o_greater*, *o_smallerequal*, *o_greaterequal*, *o_equal* en *o_notequal*.
- De grammaticale regel *operand* vult het *o_left* veld van de opgeleverde *struct*. Hier worden de eindsymbolen *distance*, *direction*, *random* en getallen verwerkt. Als de gelezen invoer een getal is (zie de grammaticale regel *number*), dan kan de *struct* die door de grammaticale regel *number* is opgeleverd als '$$' doorgegeven worden. Anders wordt het *o_type* veld van de *struct* die de linker operand weergeeft (*o_left*) op het betreffende operandtype gezet.
- De grammaticale regel die een conditie weergeeft (*condition*), verwerkt eerst de ene operand, vervolgens een operator en vervolgens de rechter operand. Merk op dat de *operand*-regel een *INSTRUCTION_ struct* oplevert, waarin een operand in het *o_left* is weergegeven. In de *condition* regel moet deze informatie één keer in het *o_left* veld komen te staan en één keer in het *o_right* veld.
- Bij alle regels die instructies van het robotprogramma weergeven, geldt dat het *i_type* veld van de opgeleverde *INSTRUCTION_ struct* de soort van de instructie moet weergeven. Daarnaast is het zo dat de *if*- en *goto*-instructies het *goto_nr* veld van de opgeleverde *struct* vullen. Dit veld geeft een sprong weer (bij de *goto*-instructie is deze sprong onvoorwaardelijk).
  De *if*-instructie vult bovendien het *else_nr* veld. Dit veld bevat het regelnummer waar heengesprongen wordt als de conditie in de *if*-instructie onwaar is. Om te kunnen onderscheiden of een *if*-instructie wèl of niet een *else*-deel heeft, spreken we af dat bij afwezigheid van het *else*-deel het *else_nr* veld de waarde -9999 krijgt (zie ook de constante *NO_ELSE* in de header file *rasic.h*, listing 147).
- De instructies *drive* en *look* moeten door één argument gevolgd worden (zie de grammaticale regel voor *argument*). Dit argument

wordt als het veld *o_left* opgeleverd. De instructies *field*, *object* en *sleep* moeten door één nummer gevolgd worden. Dit nummer wordt in het *o_left* veld opgeleverd.

- De *if*-instructie is de enige instructie die twee argumenten verwacht en dus tweemaal de grammaticale regel *operand* activeert. De twee operanden worden als *o_left* en *o_right* opgeleverd.

Opgave 122.

Maak aan de hand van de bovenstaande beschrijving en met behulp van listing 144 de parser voor het *rasic* programma af. Gebruik de constante *NO_ELSE* in veld *else_nr* om aan te geven dat een *if*-instructie geen *else*-deel heeft.

De flex-specificatie van de lexical analyzer

Aan de hand van de *bison*-specificatie waar alle eindsymbolen genoemd zijn, kan nu een *flex*-specificatie voor de lexical analyzer opgesteld worden. Een lexical analyzer voor het *rasic*-programma is in listing 145 weergegeven. Bij deze specificatie maken we de volgende opmerkingen:

- De lexer moet, naast alle eindsymbolen, het *n*-teken kunnen herkennen. Dit teken is immers een scheidingsteken dat door de grammatica wordt gebruikt.
- Als de lexer een *n*-teken leest dan wordt de (globale) variabele *line_number* opgehoogd. Deze variabele kan worden gebruikt om het regelnummer in de invoer aan te duiden bij foutmeldingen. Sommige *flex*-implementaties verzorgen deze taak automatisch. De regelteller heet dan *yylineno*. In dit voorbeeld is voor dit doel een eigen teller gebruikt, die in de header file *rasic.h* (zie listing 147) is gedeclareerd.
- Als de lexical analyzer een deel van de invoer niet kan begrijpen, dan wordt het programma met een foutmelding afgebroken. Dit gebeurt met behulp van functie *error()*. Deze functie is in de header file *rasic.h* gedeclareerd.

```
1     #include "rasic.h"
2     #include "parser.h"
3
4     %%
5     direction           return (DIRECTION);
6     distance            return (DISTANCE);
7     drive               return (DRIVE);
8     east                return (EAST);
9     else                return (ELSE);
10    end                 return (END);
11    field               return (FIELD);
12    goto                return (GOTO);
13    if                  return (IF);
14    look                return (LOOK);
15    north               return (NORTH);
16    object              return (OBJECT);
17    randomize           return (RANDOMIZE);
18    random              return (RANDOM);
```

```
19    sleep                return (SLEEP);
20    south                return (SOUTH);
21    stop                 return (STOP);
22    then                 return (THEN);
23    west                 return (WEST);
24    "<"                  return (SMALLER);
25    ">"                  return (GREATER);
26    "<="                 return (SMALLEREQUAL);
27    ">="                 return (GREATEREQUAL);
28    "=="                 return (EQUAL);
29    "!="                 return (NOTEQUAL);
30    [0-9]+               return (NUMBER);
31    [ \t]                ;
32    \n                   { line_number++; return ('\n'); };
33    [a-zA-Z0-9]+ |
34    .                    { error ("Unrecognized input '%s' at line %d.\n",
35                                  yytext, line_number);
36                         }
```

Listing 145: De *flex*-specificatie voor de lexical analyzer van het *rasic*-programma.

Opgave 123.

1. Welke van de eindsymbolen zouden door hun ASCII-waarde gerepresenteerd kunnen worden?
2. Waarom zijn dubbele aanhalingstekens wel nodig rond '<=', maar niet nodig rond *west* ?
3. Wat gebeurt er in de lexical analyzer met spaties?

De opslag van een instructie

Zoals uit listing 144 blijkt geeft de functie *yyparse()* een adres van een *INSTRUCTION_* struct door aan de functie *assert()*, die geacht wordt deze informatie op de een of andere manier op te slaan.

Een voor de hand liggende manier om instructies van een robotprogramma op te slaan, is gebruik te maken van een realloceerbaar array. Als een instructie opgeslagen moet worden, dan realloceert de functie *assert()* dit array waarbij één extra plaats wordt gemaakt. De instructie wordt op die plaats bewaard.

De functie *assert()* is in listing 146 weergegeven. De functie *error()*, die een foutmelding afdrukt en het programma afbreekt als het geheugen op is, is een oude bekende. Variabele *out_of_memory* is een globaal bekende string.

Op de regels 9 en 10 van listing 146 wordt, afhankelijk van het opstaan van de bit-vlag *flag_list*, de functie *listline()* aangeroepen. Deze functie drukt de instructie die aan *assert()* als argument is meegegeven in tekstvorm af. De macro's die bit-vlaggen manipuleren, worden in de header file *rasic.h* (zie listing 147) gedefinieerd.

```
1     #include "rasic.h"
2
3     void assert (INSTRUCTION_ *i)
```

```
4     {
5         if (! (inst = realloc (inst, (maxinst + 1) * sizeof (INSTRUCTION_))) )
6             error (out_of_memory);
7         inst [maxinst++] = *i;
8
9         if (flag_test (flag_list))
10            listline (i);
11    }
```

Listing 146: De functie *assert()*.

Losse eindjes

In dit onderdeel zullen we nog een aantal losse eindjes behandelen die voor het inlezen van een *rasic*-programma van belang zijn.

De functie yyerror(). Zoals al vermeld is, heeft elke parser die met behulp van *bison* gemaakt is een functie *yyerror()* nodig. Het prototype van deze functie is:

*int yyerror (char *)*

Het argument dat de functie *yyparse()* aan *yyerror()* doorgeeft, duidt aan welke soort fout is opgetreden. In het geval van een ontleedfout is dit de string "parse error".

Opgave 124.

Construeer functie *yyerror()*. Deze functie moet *error()* aanroepen, waarbij het regelnummer in de invoer (bekend als de variabele *line_number*) en de foutmelding die als argument is meegegeven afgedrukt worden. Daarnaast wordt afgedrukt welke invoer op dat moment gelezen werd. De laatst gelezen invoer staat in de buffer *yytext*.

De lokale header file rasic.h. De lokale header file *rasic.h*, die in alle bronteksten van functies wordt aangesloten, is in listing 147 weergegeven. Over deze header file maken we de volgende opmerkingen:

- Bovenaan het bestand worden alle header files die voor het *rasic*-programma nodig zijn (zoals *stdio.h*) aangesloten. Een alternatieve aanpak bestaat eruit om in elke losse brontekst de benodigde header files aan te sluiten.
- Op de regels 10 tot en met 12 worden drie macro's gedefinieerd waarmee bit-vlaggen gemanipuleerd worden. De bit-vlaggen zijn in één *int flags* verzameld.
- Op regel 14 is de constante *NO_ELSE* gedefinieerd. Deze constante wordt gebruikt om aan te geven dat een *if*-instructie van een robotprogramma geen *else*-deel heeft.
- Op de regels 17 tot en met 92 zijn de *typedef*s van alle structs en enums die in het *rasic*-programma gebruikt worden, weergegeven. Deze datatypen zijn al in onderdeel 18.4.2 besproken.

- Op de regels 95 tot en met 121 zijn alle globale variabelen gedeclareerd. Van een aantal van deze variabelen geven we hier de betekenis:
  - *char **field* is het speelveld (zie 18.4.2).
  - *char fubar[]* en *char out_of_memory[]* zijn foutmeldingen.
  - *char * instruction_name[]*, ** operand_name[]* en ** operator_name []* zijn drie arrays van strings waarmee namen van respectievelijk instructies, operanden en operatoren worden aangeduid. Deze namen worden in de functie *listline()* (zie pagina 466) gebruikt.
  - *INSTRUCTION_ *inst* is een pointer naar een array waar instructies bewaard worden. Dit is in listing 146 behandeld.
  - *int curline* is een teller die gebruikt wordt om aan te geven welke instructie op het moment uitgevoerd wordt. Het is een index in het *inst* array.
  - *int curx* en *cury* geven de positie weer van de robot op het aardappelveld.
  - *int distance*, *drive_dir* en *look_dir* geven variabelen weer die in een *rasic*-programma met behulp van de operanden *distance* en *direction* opgevraagd kunnen worden.
  - *int maxinst* geeft het aantal instructies weer die in het array *inst* zijn opgeslagen. De eerste instructie is dus *inst[0]*; de laatste instructie is *inst[maxinst–1]*.
  - *int objects* geeft het aantal stenen op het aardappelveld weer.
  - *int potatoes* geeft het aantal gerooide aardappelen weer.
  - *long turns* geeft het aantal beurten weer. Elke *rasic*-instructie telt als één beurt.
- Vanaf regel 124 zijn in het bestand *rasic.h* alle externe functies gedeclareerd. Op deze functies komen we later terug.

```
1     #include <conio.h>
2     #include <graph.h>
3     #include <process.h>
4     #include <stdarg.h>
5     #include <stdio.h>
6     #include <stdlib.h>
7     #include <time.h>
8
9     // ———————————————————————————— defines
10    #define flag_test(x)        (flags & (x))
11    #define flag_up(x)          (flags |= (x))
12    #define flag_down(x)        (flags &= ~(x))
13    #define MAXNUM              9999
14    #define NO_ELSE             (-MAXNUM)
15
16    // ———————————————————————————— types
17    typedef enum
18    {
19        d_north,
20        d_east,
21        d_south,
22        d_west,
```

```
23    } DIRECTION_;
24
25    typedef enum
26    {
27        obj_empty,
28        obj_potato,
29        obj_stone,
30    } FIELD_;
31
32    typedef enum
33    {
34        flag_list           =   1,
35        flag_fieldbuilt     =   2,
36        flag_stepwise       =   4,
37    } FLAG_;
38
39    typedef enum
40    {
41        o_direction,
42        o_distance,
43        o_random,
44        o_number,
45    } OPERAND_TYPE_;
46
47    typedef enum
48    {
49        i_drive,
50        i_end,
51        i_field,
52        i_goto,
53        i_if,
54        i_look,
55        i_object,
56        i_randomize,
57        i_sleep,
58        i_stop,
59    } INSTRUCTION_TYPE_;
60
61    typedef struct
62    {
63        OPERAND_TYPE_
64            o_type;
65        int
66            val;
67    } OPERAND_;
68
69    typedef enum
70    {
71        o_smaller,
72        o_greater,
73        o_smallerequal,
74        o_greaterequal,
75        o_equal,
76        o_notequal,
77    } OPERATOR_;
78
79    typedef struct
80    {
81        int
82            line_no,
83            goto_no,
84            else_no;
85        INSTRUCTION_TYPE_
86            i_type;
87        OPERAND_
88            o_left,
89            o_right;
90        OPERATOR_
91            op;
```

```
92    } INSTRUCTION_;
93
94    // ———————————————————————————— global variables
95    extern char
96        **field,
97        fubar [],
98        *instruction_name [],
99        *operand_name [],
100       *operator_name [],
101       out_of_memory [],
102       yytext [];
103   extern FILE
104       *yyin;
105   extern INSTRUCTION_
106       *inst;
107   extern int
108       curline,
109       curx,
110       cury,
111       distance,
112       drive_dir,
113       fieldsize,
114       flags,
115       line_number,
116       look_dir,
117       maxinst,
118       objects,
119       potatoes;
120   extern long
121       turns;
122
123   // ———————————————————————————— functions
124   extern int
125       findline (int),
126       if_instruction (int),
127       operand (OPERAND_),
128       yyerror (char *),
129       yylex (void),
130       yyparse (void);
131
132   extern void
133       assert (INSTRUCTION_ *),
134       check (void),
135       error (char *, ...),
136       flag (char *),
137       initfield (void),
138       listline (INSTRUCTION_ *),
139       look_instruction (int),
140       object_instruction (void),
141       print_operand (OPERAND_),
142       process (void),
143       show (int, int, int);
```

Listing 147: De lokale header file *rasic.h*.

De globale data zijn gedefinieerd in bestand *data.c*. De listing van dat bestand is hieronder weergegeven:

```
1     #include "rasic.h"
2
3     char
4         **field,
5         fubar []                =       "Internal fubar error in function %s.\n",
6         *instruction_name []    =       { "drive", "end", "field", "goto",
7                                           "if", "look", "object", "randomize",
8                                           "sleep", "stop", },
9         *operator_name []       =       { "<", ">", "<=", ">=", "==", },
```

```
10          *operand_name []        =       { "direction", "distance", "random", },
11          out_of_memory []        =       "Out of memory, sorry.\n";
12
13      INSTRUCTION_
14          *inst                   =       NULL;
15
16      int
17          curline,
18          curx,
19          cury,
20          distance                =       MAXNUM,
21          drive_dir               =      -1,
22          flags                   =       0,
23          fieldsize               =       0,
24          line_number             =       1,
25          look_dir                =      -1,
26          maxinst                 =       0,
27          objects                 =       0,
28          potatoes                =       0;
29
30      long
31          turns                   =       0L;
```

Listing 148: De globale data in bestand *data.c*.

18.4.4 Het uitvoeren van een robotprogramma

In dit onderdeel zullen we het uitvoeren van een robotprogramma toelichten. Veel functies die hiervoor nodig zijn, worden hier niet besproken; de implementatie ervan laten we aan de lezer over omdat dit hoofdstuk tenslotte over *bison* en *flex* gaat. Dit onderdeel is eerder een schets van de manier waarop de uitvoering van een robotprogramma zou kunnen plaatsvinden.

De functie main(). De functie *main()* is in listing 149 weergegeven. In deze functie wordt door middel van *#pragma*-opdrachten aangegeven dat het *rasic* programma van twee speciale functiebibliotheken gebruik maakt. Ten eerste is dit *rasic.lib*: de (lokale) bibliotheek waarin alle functies die bij het *rasic* programma horen verzameld worden. Het aanmaken en onderhouden van deze bibliotheek kan gebeuren met behulp van het *movelib* programma (zie hoofdstuk 11). Ten tweede wordt in het *rasic*-programma de functiebibliotheek *graphics.lib* gebruikt. Deze bibliotheek bevat een aantal schermuitvoerfuncties waarmee op *ibm-pc*-achtige computers de schermuitvoer verfraaid kan worden.

Op de regels 13 tot en met 18 worden eventuele opties bekeken die bij de aanroep van het *rasic* programma zijn meegegeven. Als het eerste argument van het programma, *argv[1]*, met een – begint, dan wordt functie *flag()* aangeroepen om te kijken wat voor optie dit kan zijn. Bovendien wordt *argv* één positie verschoven zodat *argv[1]* nu op het volgende argument wijst en wordt *argc* verlaagd. Op deze manier worden alle opties van het programma afgelopen.

De functie *flag()* krijgt één argument: een *char** die op een argument wijst. Als dit argument -s of -S is, dan moet de bit-vlag *flag_stepwise* opgezet worden. Als dit argument -l of -L is, dan moet de bit-vlag *flag_list* opgezet worden. Bij alle andere opties wordt het programma met een passende foutmelding beëindigd. De implementatie van de functie *flag()* laten we aan de lezer over.

Op de regels 33 tot en met 36 worden de functies aangeroepen die het feitelijke werk doen. Als eerste wordt de invoer ontleed door de functie *yyparse()*. Vervolgens wordt de gelezen invoer nagekeken op semantische fouten door de functie *check()*. Daarna wordt het veld geïnitialiseerd door de functie *initfield()*. Tenslotte wordt de functie *process()* aangeroepen om het spel te spelen.

```
1     /*
2       RASIC.C
3         Main function of "rasic" program.
4         Compile for COMPACT memory model.
5     */
6
7     #include "rasic.h"
8     #pragma comment (lib, "rasic")
9     #pragma comment (lib, "graphics")
10
11    void main (int argc, char **argv)
12    {
13        while (*argv [1] == '-')
14        {
15            flag (argv [1]);
16            argv++;
17            argc--;
18        }
19
20        if (argc != 2)
21            error ("Usage: %s [flags] robot-file\n"
22                   "where:\n"
23                   "\t flags are optional flags;\n"
24                   "\t\t -s or -S : stepwise execution of robot program\n"
25                   "\t\t -l or -L : a listing of the source is given\n"
26                   "\t robot-file is the file containing the specification\n"
27                   "\t of the game robot.\n"
28                 , argv [0]);
29
30        if (! (yyin = fopen (argv [1], "r")) )
31            error ("Cannot open file %s for reading.\n", argv [1]);
32
33        yyparse ();
34        check ();
35        initfield ();
36        process ();
37    }
```

Listing 149: De functie *main()* van het *rasic* programma.

De functie check(). De ontleder die met behulp van *bison* geconstrueerd is, analyseert de invoer aan de hand van syntactische regels. In een robotprogramma kunnen echter ook semantische fouten optreden; bijvoorbeeld

als een *goto*-instructie voorkomt met een bestemming die niet bestaat. Deze semantische fouten worden door de functie *check()* nagekeken.

De functie *check()* is als listing 150 weergegeven. De acties van deze functie zijn de volgende:

- Alle instructies, opgeslagen in array *inst*, worden nagekeken.
- Als het regelnummer van die instructie (zoals opgeslagen in het veld *line_no*) kleiner is dan één, dan treedt een fout op.
- Als de betreffende instructie *i_object* is, dan moet de instructie in het begindeel van het robotprogramma staan; dat wil zeggen dat nog geen instructie geweest mag zijn die een actie weergeeft. Een globale *int objects* krijgt hier de waarde van de linker operand van de instructie.
- Als de betreffende instructie *i_field* is, dan moet (evenals bij *i_object*) de instructie in het begindeel van het robotprogramma staan. Een *int fieldsize* krijgt de waarde van de linker operand van de instructie.
  Merk op dat het verwerken van instructies die in een bepaald deel van het robotprogramma mogen voorkomen ook in de grammatica vastgelegd kan worden. Zo zou een robotprogramma uit twee delen kunnen bestaan, een preambule-deel en een actiedeel. We hanteren hier echer een opzet waarbij de semantische eisen gescheiden van de syntactische eisen worden verwerkt.
- Als de betreffende instructie *i_goto* of *i_if* is, dan moet een instructie bestaan met een regelnummer dat als het *goto_no*-veld van de *i_goto* of *i_if* instructie bekend staat.
  Het opzoeken van een bepaalde instructie aan de hand van een regelnummer gebeurt door de functie *findline()*. Deze functie krijgt een *int* argument dat een regelnummer weergeeft. De functie *findline()* retourneert een index in het *inst* array waar de instructie met dit regelnummer staat, of –1 als geen instructie met dit regelnummer voorkomt. De implementatie van de functie *findline()* laten we aan de lezer over.
- Als de betreffende instructie *i_if* is, en als het *else_no* veld van deze instructie niet *NO_ELSE* is, dan moet een instructie bestaan met het regelnummer dat door het *else_no* veld wordt aangegeven.
- Aan het einde van het nakijken van alle instructies wordt gekeken of de veldgrootte, bewaard in de variabele *fieldsize*, door het robotprogramma gezet is. Zo niet, dan wordt *fieldsize* op 20 gezet.

```
1     #include "rasic.h"
2
3     void check ()
4     {
5         register int
6             line,
7             preamble = 1;
```

```
8
9
10        for (line = 0; line < maxinst; line++)
11        {
12            if (inst [line].line_no < 1)
13                error ("Instruction line %d: negative line number.\n",
14                    inst [line].line_no);
15
16            if (inst [line].i_type == i_object)
17            {
18                if (! preamble)
19                    error ("Instruction line %d: may appear only before"
20                           "actions.\n", inst [line].line_no);
21                if (objects)
22                    error ("Instruction line %d: attempt to replace objects.\n",
23                        inst [line].line_no);
24                objects = inst [line].o_left.val;
25            }
26            else if (inst [line].i_type == i_field)
27            {
28                if (! preamble)
29                    error ("Instruction line %d: may appear only before"
30                           "actions.\n", inst [line].line_no);
31                if (fieldsize)
32                    error ("Instruction line %d: attempt to resize the field.\n",
33                        inst [line].line_no);
34                fieldsize = inst [line].o_left.val;
35            }
36            else
37                preamble = 0;
38
39            if (inst [line].i_type == i_goto ||
40                inst [line].i_type == i_if
41               )
42                if (findline (inst [line].goto_no) == -1     ||
43                    ( inst [line].i_type == i_if             &&
44                      inst [line].else_no != NO_ELSE         &&
45                      findline (inst [line].else_no) == -1
46                    )
47                   )
48                    error ("Instruction line %d: no such line number for"
49                           "goto/if/else known.\n" , inst [line].line_no);
50        }
51
52        if (! fieldsize)
53            fieldsize = 20;
54        else if (fieldsize > 22 || fieldsize < 5)
55            error ("Incorrect field size, must be between 5 and 22.\n");
56    }
```

Listing 150: De functie *check()*.

De functie initfield(). Na het doorlopen van de *check()* functie is de veldgrootte bepaald. Deze waarde is opgeslagen als de globale *int fieldsize*. De functie *initfield()* is in listing 151 weergegeven. In de functie *initfield()* worden de volgende acties ondernomen:

- Allereerst wordt het scherm gewist. Dit gebeurt door middel van een aanroep van de functie *_clearscreen()*. Deze functie is in de header file *graph.h* gedeclareerd en bevindt zich in de functiebibliotheek *graphics.lib*. Het argument *_GCLEARSCREEN* geeft aan dat het gehele scherm gewist moet worden.

- Vervolgens wordt het speelveld gealloceerd. Dit veld wordt door de pointer *field* aangewezen (zie ook figuur 18.4). Op elk veldje wordt een aardappel geplaatst door de waarde van dit veld de constante *obj_potato* te laten zijn. Met behulp van de functie *show()* (zie verderop) wordt de eenheid van het speelbord op het scherm getoond.
- De aardappelrooimachine wordt midden op het speelveld gezet. Dit wordt bereikt door de coördinaten die de plaats van de robot aangeven (*curx* en *cury*) de waarde *fieldsize / 2* te geven.
- Tenslotte wordt de bit-vlag *flag_fieldbuilt* opgezet. Deze vlag wordt in de functie *error()* gebruikt.

```
1     #include "rasic.h"
2
3     void initfield ()
4     {
5         register int
6             y,
7             x;
8
9         _clearscreen (_GCLEARSCREEN);
10        curx = cury = fieldsize / 2;
11
12        if (! (field = malloc (fieldsize * sizeof (char *))) )
13            error (out_of_memory);
14        for (y = 0; y < fieldsize; y++)
15        {
16            if (! (field [y] = malloc (fieldsize * sizeof (char))) )
17                error (out_of_memory);
18            for (x = 0; x < fieldsize; x++)
19            {
20                field [y][x] = obj_potato;
21                show (y, x, 1);
22            }
23        }
24        flag_up (flag_fieldbuilt);
25    }
```

Listing 151: De functie *initfield()*.

De functie error(). De functie *error()* is een oude bekende. Deze functie verwacht een variabel aantal argumenten, waarbij het eerste argument een *char** moet zijn (de format string). Met behulp van *va_start* en *vprintf()* wordt een melding naar het scherm geschreven. Vervolgens wordt het programma beëindigd.

Zoals we later zullen zien is de positie van de cursor op het scherm niet constant: doordat de positie van de aardappelrooimachine in het veld verandert, wordt deze positie op het scherm weergegeven. Hierbij verspringt de cursor. We maken bij de functie *error()* daarom de volgende uitbreiding: als de bit-vlag *flag_fieldbuilt* opstaat (deze vlag wordt in de functie *initfield()* gezet) dan wordt vóór het afdrukken van de foutmelding de cursor naar de eerste kolom op rij 23 van het scherm gezet. Hiermee wordt bereikt dat foutmeldingen onderaan het scherm verschijnen en niet dwars door andere informatie heen.

De implementatie van de functie *error()* laten we aan de lezer over. Het bewegen van de cursor over het scherm kan met behulp van de functie *_settextposition()* gebeuren.

De functie show(). Het tonen van informatie op het scherm gebeurt door de functie *show()*. Deze functie krijgt drie argumenten:

- *int y* geeft de verticale (rij-) coördinaat aan van het veldje dat getoond moet worden;
- *int x* geeft de horizontale (kolom-) coördinaat aan van het veldje dat getoond moet worden;
- *int on* heeft alleen een betekenis als de meegegeven argumenten gelijk zijn aan de coördinaten van de robot. In dat geval wordt als *on* niet-nul is een 'smile' symbooltje op het scherm getekend. Als *on* nul is, dan wordt het veldje getekend alsof de robot er niet op stond.

Met behulp van het *on* argument wordt bereikt dat als de robot rijdt, de oude positie van de robot op het scherm 'gewist' wordt voordat op de nieuwe positie de robot weer getoond wordt.

```
1     #include "rasic.h"
2
3     void show (int y, int x, int on)
4     {
5         if (y < 0 || y >= fieldsize ||
6             x < 0 || x >= fieldsize
7            )
8             return;
9
10        _settextposition (y + 1, x + 1);
11        if (x == curx && y == cury && on)
12            putchar (2);
13        else switch (field [y] [x])
14        {
15            case obj_empty:
16                putchar (' ');
17                break;
18            case obj_potato:
19                putchar (4);
20                break;
21            case obj_stone:
22                putchar (1);
23                break;
24            default:
25                error (fubar, "show()");
26        }
27    }
```

Listing 152: De functie *show()*.

Overigens gebeurt **alle** schermuitvoer van het *rasic*-programma met behulp van de functie *show()*, voorzover dit het aardappelveld betreft. Dankzij deze aanpak is het gemakkelijker wijzigingen (zoals uitvoer waarbij

kleuren worden gebruikt) door te voeren: als de schermuitvoer over meerdere plaatsen verspreid zou zijn, dan zouden aanpassingen meer moeite vergen. De functie *show()* is in listing 152 weergegeven.

De functies listline() en print_operand(). De functies *listline()* en *print_operand ()* worden gebruikt om de interne representatie van een instructie in tekstvorm af te drukken. Volgens de conventie die in de functie *assert()* is gevolgd, wordt *listline()* aangeroepen met één argument dat een pointer is naar een *INSTRUCTION_ struct*. Uiteraard bestaan andere mogelijkheden voor het doorgeven van de informatie die ertoe moet leiden dat de functie *listline()* een instructie afdrukt.

Het afdrukken van een operand is in onze opzet overgelaten aan de functie *print_operand()*. Deze functie krijgt, zoals blijkt uit de listing van de header file *rasic.h*, één argument van het type *OPERAND_*. Het doel van de functie *print_operand()* bestaat eruit dat één van de strings "direction", "distance", "random" of een getal wordt afgedrukt; afhankelijk van het type van de operand. Wat de communicatie tussen de functies *listline()* en *print_operand()* betreft, bestaan uiteraard andere mogelijkheden.

De implementatie van de functies *listline()* en *print_operand()* laten we aan de lezer over. Overigens zijn de namen van alle instructies opgeslagen als het array van strings *instruction_name*. Het veld *i_type* van een *INSTRUCTION_ struct* kan gebruikt worden om een passende string uit dit array te selecteren. Analoog hieraan zijn de strings die de operatoren representeren, opgeslagen als het array van strings *operator_name* en zijn namen van operanden opgeslagen als het array van strings *operand_name*.

De functie process()

De functie *process()*, die door *main()* wordt aangeroepen, is het hart van de *rasic*-interpreter. Hier wordt aan de hand van opgeslagen instructies besloten welke verdere actie ondernomen moet worden.

De functie *process()* is in feite een software-chip die de instructies van de *rasic*-taal uitvoert; net zoals een INTEL 80836 processor de instructies van de assembler-taal uitvoert[5].

In listing 153 is de functie *process()* weergegeven. We zullen hier niet alle acties van de chip uitvoerig beschrijven, maar zullen een globale schets van de implementatie geven:

- Allereerst wordt door middel van de functie *show()* de robot op het veld getoond. Vervolgens wordt variabele *curline* op nul gezet: het uitvoeren van het robotprogramma begint immers bij instructie *instr[0]*. (Zie hiervoor de regels 10 en 11 van listing 153.)
- Vervolgens worden in een *while*-statement instructies uitgevoerd (regel 13). Het uitvoeren van robotinstructies gaat door totdat een

5. Een software-chip moet niet verward worden met een aardappelchip.

end instructie wordt tegengekomen, of totdat er geen instructies meer uit te voeren zijn.

- Op de regels 15 tot en met 33 wordt, als de bit-vlag *flag_stepwise* opstaat, informatie over de huidige stand van zaken getoond. Dit is voor de ontwikkelaar van een perfecte aardappelrooimachine buitengewoon handig (dit stelden wij proefondervindelijk vast). De 22^e regel van het scherm wordt gewist en vervolgens wordt de functie *listline()* aangeroepen om hier de instructie af te drukken die uitgevoerd gaat worden. Daarna wordt op de rechter schermhelft een aantal variabelen afgedrukt.
- Als de laatst uitgevoerde instructie een *sleep*-instructie was (regel 37) dan wordt de slaaptijd verlaagd. Anders wordt in een *switch*-statement (regel 42) de huidige instructie uitgevoerd.
- Het uitvoeren van een instructie als *goto* ligt voor de hand en wordt hier niet verder uitgewerkt. Een drietal gecompliceerder instructies, *if*, *look* en *object*, zijn aan hulpfuncties overgelaten.
- Het uitvoeren van de *randomize*-instructie behoeft enige toelichting. In een **C**-programma kunnen pseudo-randomgetallen bepaald worden met behulp van de standaardfunctie *rand()*. Deze getallen worden *pseudo*-willekeurig genoemd, omdat de randomgenerator van de functie *rand()* steeds bij hetzelfde getal begint. Concreet betekent dit dat een programma als *rasic* zich bij herhaaldelijke aanroepen op dezelfde wijze gedraagt: de randomgenerator maakt steeds dezelfde getallen.
  Om de randomgenerator te initialiseren kan de standaardfunctie *srand()* gebruikt worden. Deze functie verwacht één argument dat de initialisatiewaarde van de generator is. Om pseudo-randomgetallen zo willekeurig mogelijk te maken, kan de functie *srand()* aangeroepen worden met als argument de return-waarde van de functie *time()*. Deze functie retourneert het aantal verstreken seconden sinds 1 januari 1970. Dit aantal is een redelijke basis voor de initialisatie van de randomgenerator.
- Na het doorlopen van de *switch* (regels 80 tot en met 91) wordt gekeken of een aardappel gerooid is, en zo ja, of daarmee alle aardappelen binnen zijn. Bovendien wordt gekeken of de robot tegen een steen danwel van het aardappelveld afgereden is.
- Op de regels 93 tot en met 114 wordt de robot verplaatst. Dat gebeurt alleen als de variabele *drive_dir* niet –1 is.

```
1    #include "rasic.h"
2
3    void process ()
4    {
5        register int
6            i,
7            sleep_time = 0,
8            thisline;
```

```
9
10          show (cury, curx, 1);
11          curline = 0;
12
13          while (inst [curline].i_type != i_end && curline < maxinst)
14          {
15              if (flag_test (flag_stepwise))
16              {
17                  _settextposition (22, 1);
18                  for (i = 0; i < 79; i++)
19                      putchar (' ');
20                  _settextposition (22, 1);
21                  listline (&inst [curline]);
22                  _settextposition (2, 40);
23                  printf ("Coordinates      : y=%2.2d, x=%2.2d", cury, curx);
24                  _settextposition (3, 40);
25                  printf ("Distance         : %2d"  , distance);
26                  _settextposition (4, 40);
27                  printf ("Driving direction: %2d"  , drive_dir);
28                  _settextposition (5, 40);
29                  printf ("Looking direction: %2d"  , look_dir);
30                  _settextposition (6, 40);
31                  printf ("Potatoes so far  : %3d"  , potatoes);
32                  getch ();
33              }
34
35              thisline = curline;
36              turns++;
37              if (sleep_time)
38                  sleep_time--;
39              else
40              {
41                  curline++;
42                  switch (inst [thisline].i_type)
43                  {
44                      case i_drive:
45                          drive_dir = operand (inst [thisline].o_left) % 4;
46                          if (drive_dir == -1)
47                              drive_dir = look_dir;
48                          break;
49
50                      case i_goto:
51                          curline = findline (inst [thisline].goto_no);
52                          break;
53
54                      case i_if:
55                          curline = if_instruction (thisline);
56                          break;
57
58                      case i_look:
59                          look_instruction (thisline);
60                          break;
61
62                      case i_object:
63                          object_instruction ();
64                          break;
65
66                      case i_randomize:
67                          srand ( (int) time (NULL) );
68                          break;
69
70                      case i_sleep:
71                          sleep_time = operand (inst [thisline].o_left) - 1;
72                          break;
73
74                      case i_stop:
75                          drive_dir = look_dir = -1;
76                          break;
77                  }
```

```
78              }
79
80              if (field [cury] [curx] == obj_potato)
81              {
82                  field [cury] [curx] = obj_empty;
83                  potatoes++;
84                  if (potatoes >= fieldsize * fieldsize - objects)
85                      error ("You win in %ld turns.\n", turns);
86              }
87              else if (field [cury] [curx] == obj_stone   ||
88                       cury < 0 || cury >= fieldsize      ||
89                       curx < 0 || curx >= fieldsize
90                      )
91                  error ("You're dead. You collected %d potatoes.\n", potatoes);
92
93              if (drive_dir != -1)
94              {
95                  show (cury, curx, 0);
96                  switch (drive_dir)
97                  {
98                      case d_north:
99                          cury--;
100                         break;
101                     case d_east:
102                         curx++;
103                         break;
104                     case d_south:
105                         cury++;
106                         break;
107                     case d_west:
108                         curx--;
109                         break;
110                     default:
111                         error (fubar, "process()");
112                 }
113                 show (cury, curx, 1);
114             }
115         }
116
117         if (curline >= maxinst)
118             error ("Program ran out of instructions.\n");
119         else
120             error ("Program stopped. You collected %d potatoes.\n", potatoes);
121     }
```

Listing 153: Functie *process()*: de software-chip

Hulpfuncties voor de chip

In dit onderdeel worden de hulpfuncties die bij het uitvoeren van instructies betrokken zijn besproken.

De functie if_instruction(). De brontekst van de functie *if_instruction()* is in listing 154 weergegeven. Deze functie krijgt één argument: een *int* waarde die de index in het *inst* array weergeeft waar de uit te voeren *if*-instructie te vinden is. De functie *if_instruction()* retourneert de index van de volgende uit te voeren instructie.

Voor de return-waarde van de functie *if_instruction()* zijn drie mogelijkheden:

- Als de conditie van de *if*-instructie waar is, dan moet de uitvoering van het robotprogramma doorgaan bij de instructie die door het regelnummer wordt weergegeven dat als veld *goto_no* is opgeslagen.
- Als de conditie van de *if*-instructie onwaar is en als er geen *else*-deel in de *if*-instructie voorkomt, dan moet de uitvoering van het robotprogramma doorgaan bij de instructie die op de *if*-instructie volgt.
- Als de conditie van de *if*-instructie onwaar is en als er wèl een *else*-deel in de *if*-instructie voorkomt, dan moet de uitvoering van het robotprogramma doorgaan bij de instructie die door het regelnummer wordt weergegeven dat als veld *else_no* is opgeslagen.

Deze drie mogelijkheden zijn in listing 154 terug te vinden. Een regelnummer dat in het veld *goto_no* of in het veld *else_no* is opgeslagen, wordt tot een index in het *inst* array geconverteerd door een aanroep van de functie *findline()*.

```
1     #include "rasic.h"
2
3     int if_instruction (int line)
4     {
5         register int
6             leftop,
7             rightop,
8             true;
9
10        leftop  = operand (inst [line].o_left);
11        rightop = operand (inst [line].o_right);
12
13        switch (inst [line].op)
14        {
15            case o_smaller:
16                true = leftop < rightop;
17                break;
18            case o_greater:
19                true = leftop > rightop;
20                break;
21            case o_smallerequal:
22                true = leftop <= rightop;
23                break;
24            case o_greaterequal:
25                true = leftop >= rightop;
26                break;
27            case o_equal:
28                true = leftop == rightop;
29                break;
30            case o_notequal:
31                true = leftop != rightop;
32                break;
33            default:
34                error (fubar, "if_instruction()");
35        }
36
37        if (true)
38            return (findline (inst [line].goto_no));
39
40        if (inst [line].else_no != NO_ELSE)
41            return (findline (inst [line].else_no));
```

```
42
43          return (line + 1);
44      }
```

Listing 154: De functie *if_instruction()*.

De functie operand(). Zoals uit listing 154 blijkt, worden waarden van operanden bepaald door de functie *operand()*. Deze functie heeft één parameter: een *OPERAND_ struct*, aan de hand waarvan een getal wordt bepaald. Dit getal wordt geretourneerd.

Als het type van de operand (veld *o_type* van de *OPERAND_ struct*) *o_direction* is, dan retourneert de functie *operand()* de waarde van de variabele *drive_dir*. Als het type van de operand *o_distance* is, dan retourneert de functie *operand()* de waarde van de variabele *distance* (zie de functie *look_instruction()*). Als het type van de operand *o_number* is, dan retourneert de functie *operand()* de waarde van veld *val* van de operand.

De implementatie van de functie *operand()* laten we aan de lezer over.

De functie look_instruction(). De functie *look_instruction()* is weergegeven in listing 155. In deze functie worden, afhankelijk van de richting waarin gekeken wordt, alle veldjes van het aardappelveld afgelopen. Zolang als geen steen gevonden wordt en zolang als de rand van het aardappelveld niet bereikt is, wordt de globale variabele *distance* met één opgehoogd.

```
1       #include "rasic.h"
2
3       void look_instruction (int line)
4       {
5           register int
6               i;
7
8           look_dir = operand (inst [line].o_left) % 4;
9
10          distance = 0;
11
12          switch (look_dir)
13          {
14              case d_north:
15                  for (i = cury; i >= 0 ; i--, distance++)
16                      if (field [i][curx] == obj_stone)
17                          break;
18                  break;
19
20              case d_east:
21                  for (i = curx; i <= fieldsize - 1; i++, distance++)
22                      if (field [cury][i] == obj_stone)
23                          break;
24                  break;
25
26              case d_south:
27                  for (i = cury; i <= fieldsize - 1; i++, distance++)
28                      if (field [i][curx] == obj_stone)
29                          break;
30                  break;
```

```
31
32              case d_west:
33                  for (i = curx; i >= 0; i--, distance++)
34                      if (field [cury][i] == obj_stone)
35                          break;
36                  break;
37          }
38      }
```

Listing 155: De functie *look_instruction()*.

De functie object_instruction(). De functie *object_instruction()* wordt aangeroepen door de functie *process()* om een aantal stenen over het aardappelveld te verspreiden. Het aantal te verspreiden stenen is opgeslagen in de globale variabele *objects*.

De functie *object_instruction()* bepaalt met behulp van de standaardfunctie *rand()* een willekeurige x-coördinaat en een willekeurige y-coördinaat. Deze waarden worden teruggebracht tot de omvang van het veld. Als op *field[y][x]* nog geen steen staat en als de coördinaten niet de standplaats van de robot aangeven, dan wordt op het veldje dat door deze twee coördinaten wordt aangewezen een steen geplaatst. Vervolgens wordt de functie *show()* aangeroepen om de geplaatste steen op het scherm weer te geven.

Het bepalen van coördinaten, het plaatsen van een steen en het tonen van de steen op het scherm wordt herhaald totdat *objects* stenen geplaatst zijn. De implementatie van de functie *object_instruction()* laten we aan de lezer over.

18.5 Speciale kenmerken van bison en flex

In dit onderdeel wordt een aantal speciale kenmerken van de codegeneratoren *bison* en *flex* behandeld. Gewoonlijk wordt van deze kenmerken geen gebruik gemaakt, maar in bijzondere omstandigheden kunnen deze extra's toch zeer handig zijn.

18.5.1 De functie yylex() en meerdere invoerbestanden

Bij verstek genereert *flex* een lexical analyzer, die slechts één bestand hoeft te verwerken. Dit bestand wordt, zoals eerder vermeld, aangeduid door de stream *yyin*.

Als de functie *yylex()* meer dan één bestand moet kunnen verwerken, dan kan dit op de volgende manier gerealiseerd worden:

- De stream *yyin* wordt voor een te verwerken bestand geopend.
- De globale *int yy_init* wordt op de waarde 1 gezet. De functie *yylex* inspecteert deze waarde om te beoordelen of *yyin* op een nieuw te verwerken bestand wijst.

- Vervolgens wordt *yylex()* aangeroepen om de invoer te lezen. Deze aanroep kan uiteraard indirect gebeuren door de functie *yyparse()* aan te roepen: de functie *yyparse()* roept immers zelf *yylex()* aan.
- Tot slot wordt de stream *yyin* gesloten.

De hierboven gegeven punten worden herhaald voor elk te verwerken bestand. Samengevat is het verschil met de werkwijze waarbij slechts één bestand verwerkt wordt het zetten van *int yy_init* op de waarde 1 bij elk nieuw te verwerken bestand en het sluiten van de stream *yyin* na afloop van het lezen.

18.5.2 De bison-specificatie en het aangeven van fouten

Zoals eerder is vermeld, activeert de functie *yyparse()* de functie *yyerror()* als een zodanige fout optreedt dat de invoer niet meer aan de hand van de gespecificeerde grammatica ontleed kan worden. Bij deze actie wordt de string *"parse error"* als foutmelding meegegeven aan de functie *yyerror()*.

Vaak is deze foutmelding een beetje aan de magere kant. Een gebruiker is veel meer gediend met een melding die zegt wat gelezen werd en wat gelezen had moeten worden. In dit onderdeel behandelen we een opzet waarmee foutmeldingen context-gevoelig worden gemaakt en waarmee meerdere fouten kunnen worden herkend.

In de opzet van context-gevoelige foutmeldingen staat de grammatica, zoals weergegeven in een *bison*-specificatie, centraal. Dit is niet zo vreemd, want de software-ontwikkelaar die de *bison*-specificatie opstelt is de enige die foutcondities kan voorspellen. De procedure die bij het opstellen van context-gevoelige foutmeldingen gevolgd kan worden, is hieronder weergegeven:

- In die grammaticale regels waar eindsymbolen verwacht worden, wordt een vlag gezet die aangeeft wat verwacht wordt.
- Als de functie *yyparse()* de functie *yyerror()* aanroept en als de meegegeven string *"parse error"* is, dan wordt de eerder gezette vlag geïnspecteerd en wordt een passende foutmelding afgedrukt.

Deze opzet is in de onderstaande listing 156 weergegeven. De hier getoonde grammatica maakt deel uit van de robottaal *rasic*, die eerder in onderdeel 18.4 is behandeld. Als vlagwaarde wordt hier een *char** gebruikt, die een foutmelding aangeeft.

```
1     %%
2     operator:
3         set_operator_expected
4         terminal_operator           { $$ = $2; }
5         ;
6
7     terminal_operator:
8             SMALLER                 { $$.op = o_smaller; }
9         |
```

```
10          GREATER                 { $$.op = o_greater; }
11      |
12          SMALLEREQUAL            { $$.op = o_smallerequal; }
13      |
14          GREATEREQUAL            { $$.op = o_greaterequal; }
15      |
16          EQUAL                   { $$.op = o_equal; }
17      |
18          NOTEQUAL                { $$.op = o_notequal; }
19      ;
20
21  set_operator_expected:
22                                  { errormessage = "operator expected"; }
23      ;
```

Listing 156: Context-gevoelige foutmeldingen in een *bison*-specificatie.

In deze *bison*-specificatie is een regel *operand* te vinden waar *vóór* het feitelijke ophalen van een operand de vlag *errormessage* gezet wordt. Dit gebeurt door de activering van de regel *set_operator_expected*. Vervolgens wordt de regel *terminal_operand* geactiveerd om een operator op te halen. Merk op dat in de *operand*-regel de doorgegeven waarde daardoor *$2* is.

Een bijbehorende functie *yyerror()*, die de gezette vlag *errormessage* inspecteert, is in listing 157 gegeven.

```
1   int yyerror (char *msg)
2   {
3       if (! strcmp (msg, "parse error") )
4           printf ("Error (%s was read) : %s\n",
5               yytext, errormessage);
6       else
7           puts (msg);
8
9       exit (1);
10  }
```

Listing 157: Context-gevoelige foutmeldingen in de functie *yyerror()*.

18.5.3 Het herstel van de parser na foutcondities

De parsers die in dit hoofdstuk geconstrueerd zijn, reageren op een foutconditie op een zeer eenvoudige wijze: het programma wordt, na een passende melding, beëindigd. *Bison* geeft de software-ontwikkelaar echter een mogelijkheid om op een vriendelijker wijze op fouten in te spelen. De gedachte hierbij is dat als een fout optreedt, welliswaar een melding gegeven wordt, maar dat vervolgens de rest van de invoer nog geanalyseerd wordt. Hierdoor kan een gebruiker in één keer op meerdere fouten gewezen worden.

Het genereren van foutmeldingen waarbij een programma na één fout niet afgebroken hoeft te worden is gebaseerd op de *error*-regel van *bison*. Deze regel wordt op cruciale plaatsen in de grammatica opgenomen. De *error*-regel is in *bison* 'ingebakken': de software-ontwikkelaar

hoeft zich niet bezig te houden met vragen als *hoe* de *error*-regel geactiveerd wordt. Het voorkomen van een *error*-regel in een *bison*-specificatie houdt in dat als de functie *yyparse()* de invoer niet volgens de opgegeven grammatica kan ontleden, de functie *yyerror()* aangeroepen wordt en dat vervolgens dit deel van de invoer als afgesloten geacht wordt.

```
1     %{
2     #include "rasic.h"
3
4     #define YYSTYPE INSTRUCTION_
5     %}
6
7     %token DRIVE ELSE END FIELD IF GOTO LOOK OBJECT RANDOMIZE SLEEP STOP THEN
8            DIRECTION DISTANCE RANDOM
9            NORTH EAST SOUTH WEST
10           SMALLER GREATER SMALLEREQUAL GREATEREQUAL EQUAL NOTEQUAL
11           NUMBER
12
13    %%
14    program:
15            program
16            line
17        |
18            line
19        ;
20
21    line:
22            number
23            instruction
24            '\n'                    {
25                                        $$.line_no = $1.o_left.val;
26                                        $$ = $2;
27                                        assert (&$$);
28                                    }
29        |
30            '\n'
31        |
32            error
33            '\n'
34        ;
35
36    instruction:
37            ...
```

Listing 158: Een gemodificeerde *bison*-specificatie voor het *rasic*-programma.

Een voor de hand liggende plaats om een *error*-regel op te nemen in de *bison*-specificatie voor het programma *rasic* is wellicht in de regel *line*. Een deel van een dergelijke *bison*-specificatie is in listing 158 gegeven. In deze listing is te zien dat een *line* onder andere een *instruction* kan zijn, maar ook een *error* kan zijn. De functie *yyparse()* die door *bison* gegenereerd wordt, kiest voor de *error*-regel als een instructie niet volgens de regels van de grammatica geëvalueerd kan worden. In de praktijk betekent dat, dat bij een syntactisch niet correcte instructie de *instruction*-regel welliswaar faalt, maar de *line*-regel toch slaagt vanwege het voorkomen van het *error* alternatief. Hierdoor negeert de regel *line* alle invoer die eigenlijk bij een *instruction* had moeten horen en activeert vervolgens na de

eerstvolgende *n* opnieuw de *instruction*-regel zodat een nieuwe instructie verwerkt wordt. Essentieel hierbij is dat het *error*-alternatief door een terminaal symbool gevolgd wordt. In dit voorbeeld is dat het newline-teken: hierdoor zal de *error*-regel alle invoer tot aan de eerstvolgende newline negeren en dat is in deze grammatica precies de scheiding tussen instructies. De vuistregel voor het plaatsen van een *error*-regel in een grammatica luidt dat de *error*-regel als een alternatief wordt opgenomen, gevolgd door een terminaal symbool.

Om het programma bij een syntactische fout niet te laten stoppen, moet de functie *yyerror()*, die door de functie *yyparse()* bij een foutconditie wordt aangeroepen, gewijzigd worden. De functie *yyerror()* moet nu na het afdrukken van een foutmelding de waarde nul retourneren. Dit signaal wordt door de functie *yyparse()* geïnterpreteerd als een boodschap dat de foutconditie verwerkt is en dat het ontleden kan doorgaan. Uiteraard kan de functie *yyerror()* voor een passende foutmelding zorgen op de manier die op pagina 474 is aangegeven.

In de praktijk wordt de hier geschetste opzet van foutherkenning met nog twee kenmerken uitgebreid:

- Meestal wordt door middel van een vlag bijgehouden of een fout is opgetreden. Na het ontleden van de invoer wordt de vlag geïnspecteerd om te bepalen of acties moeten volgen. In het voorbeeld van het *rasic*-programma zou bij het optreden van één of meerdere fouten geen robotsimulatie (meer) plaats kunnen vinden.
- Vaak wordt het aantal gevonden fouten bijgehouden met behulp van een teller. Als het aantal fouten een bepaald maximum overschrijdt, wordt het ontleedproces alsnog gestopt. In het voorbeeld van het *rasic*-programma zou in de functie *yyerror()* een *static int* gebruikt kunnen worden om bij te houden hoeveel fouten gevonden zijn. Als dit aantal bijvoorbeeld 20 zou overschrijden, zou de functie *yyerror()* het programma beëindigen.

18.6 Een fout in flex

Sommige implementaties van *flex* bevatten een serieuze en irritante, doch niet onoverkomelijke fout. Deze fout is veroorzaakt doordat *flex* oorspronkelijk op een computer is gemaakt waar geen hoge ASCII-tekens voorkwamen. De hoge ASCII-tekens zijn de lettertekens die een ASCII-waarde tussen 128 en 255 hebben en representeren de speciale symbolen zoals de griekse letters, de hokjestekens, enzovoort.

Het probleem wordt veroorzaakt doordat de functie *yylex()* die door *flex* geconstrueerd wordt niet berekend is op het voorkomen van die hoge ASCII-tekens. Als zo'n ASCII-teken gelezen wordt, dan 'blokkeert' de functie *yylex()* en lijkt het programma te 'hangen'.

Dit probleem kan door de software-ontwikkelaar verholpen worden door de uitvoer van *flex* met de hand bij te werken. In de door *flex* gegenereerde brontekst komt een array van *static int*s voor, dat *e* genaamd is. Dit array wordt door de functie *yylex()* als een transitiematrix gebruikt, waarbij per letterteken een actie gedefinieerd is. Het probleem ligt uiteraard aan het feit dat het array *e* niet 256 elementen maar 128 elementen bevat. Door dit array te vergroten tot 256 elementen is het probleem verholpen. De *int* waarde voor de hoge ASCII-tekens in het array *e* moet op de waarde 1 gezet worden: dit is voor de functie *yylex()* een signaal om de 'default' actie te verrichten. De default actie is dezelfde actie als die door een punt in een reguliere expressie wordt aangeduid.

Hoofdstuk 19

Heuristieken

19.1 Inleiding

In dit hoofdstuk worden heuristieken geïntroduceerd. Een heuristiek is een methode om een probleem op acceptabele wijze op te lossen in de wetenschap dat de oplossing suboptimaal is. De heuristiek wordt toegepast omdat de methode om de optimale oplossing te vinden niet bekend is, of omdat de methode om de optimale oplossing te vinden praktisch niet realiseerbaar is. Anders geformuleerd: wanneer de beste oplossing voor een probleem niet bekend of realiseerbaar is, gebruik dan een (voorlopig) acceptabele oplossing en noem dat een heuristiek. Zo is een heuristiek voor de weersverwachting voor morgen dat het morgen hetzelfde weer zal zijn als vandaag. De heuristiek is suboptimaal in de zin dat het KNMI tot betere weersverwachtingen komt. Toch is de heuristiek acceptabel omdat de verwachtingen die ermee worden gemaakt opmerkelijk vaak uitkomen en vermoedelijk een gunstige kosten-baten verhouding met zich mee brengen: er zijn geen super-computers nodig en er is geen geschoold personeel nodig om de heuristiek te kunnen hanteren.

Opgave 125.
Merk op dat de heuristiek in Nederland al heel aardig werkt. Probeer de heuristiek ook eens uit voor de Noordpool of de Sahara.

Een concrete situatie waarbij heuristieken een rol spelen, wordt gevonden bij het maken van dienstroosters. In dit hoofdstuk zal deze situatie nader worden uitgewerkt tot een programma, waarin de in dit hoofdstuk gepresenteerde heuristieken worden toegepast om roosters (dienstroosters voor het verplegend personeel in een ziekenhuis) samen te stellen.

Het probleem bij het maken van personeelsroosters is dat er eigenlijk teveel variabelen een rol spelen om een wiskundig èn personeelstechnisch acceptabele oplossing te vinden. Wiskundige methoden kunnen weliswaar worden gebruikt om een oplossing te vinden, maar die blijken voor de persoon die verantwoordelijk is voor het rooster in het algemeen geen acceptabele oplossingen te leveren. Omgekeerd is de roostermaker doorgaans niet in staat om precies te specificeren hoe het rooster wordt samengesteld, zodat het lastig blijft een sluitend algoritme voor het roosteren te construeren. Een andere benadering om het roosterprobleem aan te pakken bestaat eruit dat door de roostermaker een aantal voorwaarden worden gespecificeerd, waarna het roosteralgoritme op basis van de voorwaarden een aantal mogelijke roosters genereert. De roostermaker kan een voorgesteld rooster accepteren, of nieuwe voorwaarden specificeren, waarna het proces zich herhaalt, todat een uiteindelijk acceptabel rooster is ontstaan.

Wanneer op deze wijze een rooster wordt geconstrueerd, ontstaat er een programma dat in hoge mate interactief is. Het programma wordt tot een stuk gereedschap waarmee de roostermaker roosters kan maken

op de manier waarop iemand met behulp van een boormachine gaten in de muur kan maken voor de bevestiging van boekenplanken. De boormachine kiest de planken niet uit en bevestigt de planken evenmin, maar de constructie van de boekenkast wordt door het gebruik van de boormachine wel veel gemakkelijker.

De situatie waarvoor het roosterprogramma zal worden ontwikkeld heeft betrekking op de afdeling van een ziekenhuis waar 30 verpleegkundigen werkzaam zijn. Uiteraard werken niet alle 30 verpleegkundigen tegelijkertijd, en is een werkrooster noodzakelijk. Hierbij spelen de volgende overwegingen een rol:

- Er is een vaste staf van 30 personen. Het roosterprogramma zal deze personen bij hun volgnummer noemen (1 t/m 30). Een verfraaiing van het programma kan er uiteindelijk toe leiden dat de verpleegkundigen bij naam en niet via een nummer worden aangeduid.
- Elke verpleegkundige heeft een bepaalde ervaring. Deze factor zal bij inroostering een grote rol gaan spelen: er moet vermeden worden dat uitsluitend onervaren verpleegkundigen moeten samenwerken. Maar ook uitsluitend ervaren verpleegkundigen hoeven niet samen te werken, omdat dat extra hoge personeelskosten met zich mee brengt: de salarisschalen van de verpleegkundigen zijn gerelateerd aan hun ervaring.
- Aangenomen wordt dat per dag slechts een deel (bijvoorbeeld de helft) van de staf beschikbaar is om te werken.
- De verpleegkundigen werken in paren. Het roosterprogramma zal, uitgaande van de beschikbare mensen, paren moeten samenstellen.
- Het is wenselijk om in één paar een ervaren persoon samen met een onervaren persoon onder te brengen; dat verdient de voorkeur boven twee ervaren personen (en boven twee onervaren personen).
- Er wordt uitgegaan van blokken van vijf dagen. Wanneer een verpleegkundige (als deel van een paar) voor een bepaalde dag ingeroosterd is, dan mag die verpleegkundige niet worden ingeroosterd op een andere dag van het blok van vijf dagen.
- Het programma dient meerdere oplossingen voor de inroostering van tweetallen over een periode van vijf dagen te genereren. De gegenereerde oplossingen moeten naar wenselijkheid zijn gesorteerd, zodat de roostermaker uit bijvoorbeeld 10 suggesties kan kiezen.

Hier volgt de globale beschrijving van het algoritme dat in dit hoofdstuk zal worden gebruikt om roosters te construeren:

- Er wordt uitgegaan van een lijst van per dag beschikbaar personeel. Voor vijf dagen zijn er dus vijf lijsten beschikbaar.
- Per dag worden op zodanige wijze paren samengesteld dat de paren in principe acceptabel zijn. Dat houdt bijvoorbeeld in, dat twee

ervaren of onervaren verpleegkundigen niet in paren worden samengevoegd. In deze fase van het algoritme zal een heuristiek worden gebruikt: niet alle mogelijke paren worden samengesteld, maar slechts een veelbelovend deel. Er wordt verondersteld dat, wanneer voor elke dag een 'goed' paar in een rooster wordt opgenomen, het rooster voor een blok van vijf dagen acceptabel zal zijn.

- Uitgaande van elke dag wordt een aantal mogelijke roosters samengesteld. Hierbij wordt de heuristiek gebruikt dat de kwaliteit van de roosters gerelateerd is aan de kwaliteit van de paren, die per dag zijn ingeroosterd: hoe beter de paren, hoe beter de roosters. Daarom worden per dag niet alle mogelijke roosters gegenereerd.
- Uit alle in de vorige stap gegenereerde roosters worden tenslotte de beste roosters overgehouden en gepresenteerd. Merk op dat hier geen sprake is van een heuristiek: nieuw gegenereerde roosters worden geaccepteerd wanneer er nog niet voldoende alternatieve roosters zijn, of wanneer een nieuw rooster beter is dan het rooster dat op dat moment het minst wordt gewaardeerd.

19.2 De invoer voor het roosterprogramma

De invoer voor het roosterprogramma bestaat uit vijf tabellen (één tabel per dag), waarin vermeld staat welke personen beschikbaar zijn en wat hun respectievelijke ervaring is. Deze invoer wordt door de roostermaker op dagbasis geconstrueerd op grond van overwegingen als ziekte, vakantie, aanstellingspercentages, personele verhoudigen, etcetera. De ervaring van de verpleegkundigen is voor de roostermaker irrelevant, omdat het algoritme al met de ervaring van de verpleegkundigen rekening zal houden. De ervaring van de verpleegkundigen wordt weergegeven als het 'ervaringsgetal'. Dit 'ervaringsgetal' geeft de roostermaker in deze opzet overigens ook de mogelijkheid de voorkeur van de personen in de roostering te betrekken: verpleegkundigen zullen bijvoorbeeld op verjaardagen liever niet willen werken. In het roosterprogramma kan dat worden verwerkt door het ervaringsgetal per dag aan te passen aan bijzondere omstandigheden.

Een illustratie

In figuur 19.1 is een voorbeeld van de invoer voor het roosteralgoritme afgedrukt: per dag zijn 15 verpleegkundigen beschikbaar, waarbij de tabel per dag naar ervaring is gesorteerd. Merk op, dat niet alle verpleegkundigen alle dagen beschikbaar zijn. Zo zijn verpleegkundigen 3 en 30 op de eerste dag niet beschikbaar. Bij elke verpleegkundige is een ervaringsgetal vermeld. In figuur 19.1 is (arbitrair) besloten dat persoon 1 de hoogste ervaring krijgt (waarde 120) en persoon 30 de laagste ervaring (waarde 4). De

dag 1		dag 2		dag 3		dag 4		dag 5	
pers	exp	pers	exp	pers	exp	pers	exp	pers	exp
1	116	2	112	1	116	2	112	3	108
2	112	3	108	2	112	3	108	4	104
4	104	5	100	3	108	4	104	6	96
5	100	6	96	5	100	6	96	8	88
6	96	8	88	14	64	8	88	9	84
10	80	9	84	16	56	9	84	11	76
11	76	11	76	17	52	10	80	12	72
14	64	17	52	19	44	12	72	13	68
17	52	18	48	20	40	15	60	16	56
18	48	19	44	21	36	16	56	17	52
21	36	21	36	22	32	18	48	20	40
22	32	22	32	23	28	20	40	23	28
25	20	24	24	24	24	21	36	24	24
27	12	25	20	27	12	26	16	26	16
29	4	26	16	28	8	28	8	28	8

Figuur 19.1: Voorbeeld van de invoer.

ervaring van een verpleegkundige met een ander volgnummer kan bepaald worden met de functie:

$$f(x) = -4 * x + 120$$

Uitgaande van de gegevens in figuur 19.1 kunnen paren worden samengesteld. Wanneer voorlopig afgesproken wordt dat per dag de meest ervaren en minst ervaren verpleegkundigen worden gecombineerd tot een paar, dan is meteen al te zien dat dag 1 en dag 3 een conflict opleveren: als op dag 1 verpleegkundige 1 zal werken, dan zal die persoon op dag 3 niet opnieuw moeten worden ingeroosterd en omgekeerd. Een voor de hand liggende manier om het conflict op te lossen, is de volgende:

- Op de eerste dag kunnen de verpleegkundigen 1 en 29 (de meest en minst ervaren) worden ingeroosterd.
- Op de tweede dag kunnen de verpleegkundigen 2 en 26 worden gekozen.
- Op de derde dag zijn de verpleegkundigen 3 en 28 beschikbaar. De zeer ervaren verpleegkundigen 1 en 2 zijn immers al ingeroosterd, zodat verpleegkundige 3 als beste mogelijkheid overblijft, terwijl de minst ervaren verpleegkundige op dag 3 (verpleegkundige 28) nog niet is ingeroosterd en dus met verpleegkundige 3 tot een paar kan worden gecombineerd.
- Voor de vierde dag zijn de verpleegkundigen 4 en 21 overgebleven. Zowel de verpleegkundigen met meer ervaring dan verpleegkundige 4 als verpleegkundigen met minder ervaren verpleegkundigen dan verpleegkundige 21 kunnen dan niet meer worden ingeroosterd.
- Tenslotte kunnen voor de vijfde dag de verpleegkundigen 6 en 24 worden ingeroosterd.

	1	2	4	5	6	10	11	14	17	18	21	22	25	27	29
1	o	•	•	•	•	•	•	•	•	•	•	•	•	•	•
2		o	•	•	•	•	•	•	•	•	•	•	•	•	•
4			o	•	•	•	•	•	•	•	•	•	•	•	•
5				o	•	•	•	•	•	•	•	•	•	•	•
6					o	•	•	•	•	•	•	•	•	•	•
10						o	•	•	•	•	•	•	•	•	•
11							o	•	•	•	•	•	•	•	•
14								o	•	•	•	•	•	•	•
17									o	•	•	•	•	•	•
18										o	•	•	•	•	•
21											o	•	•	•	•
22												o	•	•	•
25													o	•	•
27														o	•
29															o

Figuur 19.2: Kruistabel voor paarvorming.

De hierboven gegenereerde methode is gebaseerd op een heuristiek: op een bepaalde dag wordt het paar geselecteerd dat bestaat uit de meest ervaren en de minst ervaren verpleegkundigen, die op dat moment nog beschikbaar zijn. Zoals bij de globale beschrijving van het algoritme al is opgemerkt, kan de methode worden verbeterd door per dag een aantal roosters te genereren, waaruit tenslotte de beste roosters worden geselecteerd.

19.3 Decompositie van het probleem

Nadat de lijsten met per dag beschikbaar personeel zijn opgesteld, zijn er twee momenten in het algoritme waarbij heuristieken een rol spelen. Er wordt gebruik gemaakt van een heuristiek om per dag een aantal mogelijke paren samen te stellen en er wordt gebruik gemaakt van een heuristiek bij de uiteindelijke constructie van de dienstroosters.

19.3.1 Opstellen van tweetallen

Het opstellen van tweetallen heeft tot doel de beschikbare personen van een dag te combineren tot paren. Er zijn vijftien personen per dag beschikbaar.

Om de paren die uit een aantal personen kunnen worden gevormd te vinden kan een kruistabel worden geconstrueerd, waarin alle mogelijke paren voorkomen. In figuur 19.2 zijn op deze wijze alle mogelijke paren voor de eerste dag (dag 1) weergegeven. In deze figuur zijn de mogelijke paren met een • weergegeven. Op de diagonaal van de tabel zijn niet-mogelijke combinaties weergegeven met een '*o*'. Dat het hier niet-mogelijke combinaties betreft zal duidelijk zijn: personen kunnen niet met zichzelf een paar vormen. Verder is de onderste helft van de tabel niet ingevuld omdat de onderdriehoek van de tabel dezelfde paren oplevert als de bovendriehoek. Zo is het paar dat bestaat uit de verpleegkundigen 1 en 2 (bovendriehoek) hetzelfde als het paar dat bestaat uit de verpleegkundigen 2 en 1 (onderdriehoek). Het aantal paren dat uit N elementen kan worden gevormd is dus:

$$(N^2 - N) / 2$$

Voor een staf van vijftien personen per dag is dit 105. Voor een tweemaal zo grote staf is dat al 435 en voor een driemaal zo grote staf 990. Bij een verdubbeling wordt het aantal paren dus ongeveer vier keer zo groot, bij een verdrievoudiging wordt het aantal paren ongeveer negen keer zo groot. Er is een kwadratisch verband tussen het aantal personen en het aantal paren dat eruit kan worden samengesteld. Het effect van een vergroting van het aantal personen wordt nog dramatischer wanneer drietallen, viertallen, enzovoort zouden moeten worden gevormd. Het aantal combinaties van k-personen dat uit een groep van N personen kan worden gevormd, wordt gegeven door de formule (zie bijvoorbeeld Selby, 1972)[1]:

$$\binom{N}{k} = \frac{N!}{k! * (N-k)!}$$

Wanneer drietallen moeten worden ingeroosterd in plaats van paren neemt het aantal paren nog sneller toe: bij 15 personen zijn er 455 drietallen, bij 30 personen zijn er 4060 drietallen, bij 45 personen 14190. Deze sterke toename van het aantal combinaties bij toename van het aantal wordt de 'combinatorische explosie' genoemd. Algoritmes waarbij de combinatorische explosie een rol speelt (zie in dit verband ook wat in hoofdstuk 15 is opgemerkt bij het programma *subset),* kunnen in de praktijk al snel niet meer worden gebruikt, omdat de bepaling van alle mogelijke combinaties te veel tijd gaat kosten. Om aan de combinatorische explosie te ontsnappen, wordt daarom de heuristiek toegepast dat alleen de beste N paren zullen worden onthouden voor verdere bewerking, bijvoorbeeld de beste 10 paren. De gedachte achter de heuristiek is dat het weinig zinvol is om de slechtste paren te onthouden, omdat die in de praktijk toch altijd zullen worden geweigerd. De reden dat er tien paren zullen worden bewaard is arbitrair: het aantal paren zal in deze context voldoende groot blijken te zijn om uiteindelijk oplossingen te kunnen genereren. In een algemener

1. De faculteitsoperator '!' is in dit boek genoemd in hoofdstuk 15.

roosterprogramma zou dit aantal tijdens de verwerking van het programma aangepast moeten kunnen worden.

	dag 1		dag 2		dag 3		dag 4		dag 5	
paarnr.	pers.	exp.	pers.	exp.	pers.	exp.	pers.	exp.	pers.	exp.
1	1, 29	112	2, 26	96	1, 28	108	2, 28	104	3, 28	100
2	2, 29	108	3, 26	92	2, 28	104	3, 28	100	4, 28	96
3	1, 27	104	2, 25	92	1, 27	104	2, 26	96	3, 26	92
4	4, 29	100	2, 24	88	2, 27	100	4, 28	96	6, 28	88
5	2, 27	100	3, 25	88	3, 28	100	3, 26	92	4, 26	88
6	1, 25	96	3, 24	84	3, 27	96	6, 28	88	3, 24	84
7	5, 29	96	5, 26	84	1, 24	92	4, 26	88	4, 24	80
8	4, 27	92	5, 25	80	5, 28	92	8, 28	80	8, 28	80
9	2, 25	92	2, 22	80	2, 24	88	6, 26	80	3, 23	80
10	6, 29	92	6, 26	80	5, 27	88	2, 21	76	6, 26	80

Figuur 19.3: De beste paren voor vijf dagen.

De 'beste paren' worden bepaald met behulp van een functie waarmee een waardering kan worden toegekend aan een willekeurig paar. De 'beste paren' worden eenvoudig gedefinieerd als die paren waarvoor de waarderingsfunctie de grootste waarde oplevert. Een mogelijke (maar niet noodzakelijk optimale) waarderingsfunctie gaat uit van de redenering dat het beste wat kan worden bereikt twee maximaal ervaren verpleegkundigen is. De functie kan dan worden gedefinieerd als:

$$waardering = ervaring_1 + ervaring_2.$$

Andere functies leiden tot een andere selectie van paren. In het voorbeeld van de verpleegkundigen houdt de waarderingsfunctie die de gecombineerde ervaring maximaliseert geen rekening met de eis dat zeer ervaren verpleegkundigen juist niet moeten worden gecombineerd. Een functie die een premie stelt op diversiteit in ervaring doet dat wel. Voor de eerste dag zou dat betekenen dat persoon 1 en 29 het beste team vormen. Wanneer als waarderingsfuntie de functie:

$$waardering = | ervaring_1 - ervaring_2 |$$

wordt gekozen, wordt juist een diversiteit in ervaring gewaardeerd. Met deze functie wordt echter geen aandacht geschonken aan het niveau van de ervaring: twee verpleegkundigen met ervaring 30 en 80 krijgen eenzelfde waardering als twee verpleegkundigen met ervaring 70 en 120. Om te bereiken dat zowel verschillen als niveau in de beoordeling een rol spelen, kan de ervaring van de verpleegkundigen eerst worden gekwadrateerd. Daardoor wordt bereikt dat verschillen worden geprononceerd. Zo'n functie is bijvoorbeeld:

$$waardering = | ervaring_1^2 - ervaring_2^2 |$$

De verpleegkundigen met waardering 30 en 80 krijgen volgens deze functie een waardering 5500, terwijl de verpleegkundigen met ervaring 120 en 70 een waardering 9500 krijgen.

De gebruikte waarderingsfunctie is dus duidelijk van invloed op de paren die als 'beste' door het programma worden gekozen. In het voorbeeld van de verpleegkundigen beperken we ons tot de functie waarbij het verschil in ervaring tussen verpleegkundigen de bepalende factor is bij de paarvorming. Uiteraard staat het de lezer vrij (en raden we aan) te experimenteren met andere waarderingsfuncties dan de functies die hier zijn genoemd. Wanneer de waarderingsfunctie, waarmee het verschil in ervaring tussen verpleegkundigen wordt gewaardeerd, wordt toegepast op de gegevens uit figuur 19.1, dan ontstaat figuur 19.3. Een dergelijke tabel is het uitgangspunt voor de tweede stap in de oplossing van het probleem, waarbij de uiteindelijke tweetallen worden geselecteerd voor alle dagen van een blok.

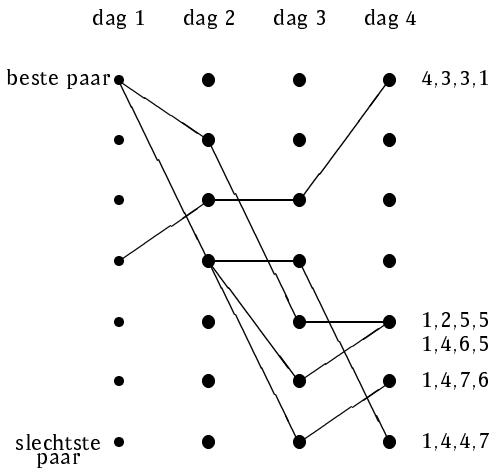
19.3.2 De constructie van de roosters

Het combineren van tweetallen moet leiden tot een aantal oplossingen voor het rooster. We spreken af dat het roosterprogramma tien mogelijke roosters moet genereren, waaruit de roostermaker de meest geschikte kan kiezen.

Een intuïtieve methode om de roosters te vormen, bestaat eruit dat alle paren met alle paren worden vergeleken. In dat geval moeten alle paren van dag 1 tot en met dag 5 met elkaar worden vergeleken. Bij deze vergelijking zouden alle paren waarin personen minstens twee keer voorkomen, moeten worden verwijderd. Een rooster voor vijf dagen dat begint bij een paar uit dag 1 bestaat bijvoorbeeld uit de vijf paren 2, 29 (dag 1); 5, 25 (dag 2); 1, 28 (dag 3); 4, 26 (dag 4) en 3, 23 (dag 5). Bij tien paren per dag en vijf dagen zijn er 10^5 potentiële roosters. Uiteraard zijn er daarvan een groot aantal niet mogelijk. Uit de roosters die uiteindelijk mogelijk zijn, moeten dan de beste tien roosters worden gezocht.

De intuïtieve benadering leidt tot de beste tien roosters, maar lijdt wederom onder de combinatorische explosie, omdat het aantal vergelijkingen exponentieel toeneemt met het aantal dagen waarvoor een rooster moet worden samengesteld. Opnieuw wordt een heuristiek gebruikt om de combinatorische explosie te omzeilen. De heuristiek is de volgende: per dag worden de beste tien oplossingen gegenereerd; dus voor dag 1, voor dag 2, enzovoort. Elke oplossing voor elke dag is dus een potentieel rooster. Aan elk rooster wordt een score toegekend die de wenselijkheid van dat rooster aangeeft. De score die zal worden toegekend is eenvoudig de som van de afzonderlijke wenselijkheidsscores van elk paar van het rooster. Voor vijf dagen kunnen dus in totaal 50 roosters worden gegenereerd en uit de gegenereerde roosters worden de tien roosters met de hoogste score uiteindelijk geselecteerd.

De vraag rijst wellicht waarom de beste oplossingen voor èlke dag moeten worden gegenereerd en waarom het niet voldoende is alleen de beste tien oplossingen voor dag 1 te genereren. Het antwoord op deze



Figuur 19.4: Wegen langs paren.

vraag is gebaseerd op een vereenvoudiging van het probleem, waarbij voor vier dagen een rooster wordt geconstrueerd, waarbij de beste zeven paren in een tabel worden verzameld, terwijl per dag de beste vier roosters moeten worden geconstrueerd. Voor deze situatie is de constructie van de roosters weergegeven in figuur 19.4. In deze figuur is het linker bovenste deel van figuur 19.3 schematisch weergegeven: de paren uit die figuur zijn als cirkels gerepresenteerd, voor elk van de vier dagen één kolom. De bovenste cirkel correspondeert met het beste paar voor die dag. Mogelijke oplossingen zijn als 'wegen' langs de cirkels getekend: beginnend bij de kolom voor dag 1 en eindigend bij de kolom voor dag 4. De paren zelf zijn ook genummerd: het bovenste paar krijgt nummer 1, en het onderste paar krijgt nummer 7.

Wanneer wordt volstaan met de bepaling van de vier roosters voor dag 1 dan worden de volgende paren in de roosters opgenomen:

- rooster 1: de paren 1,2,5,5.
- rooster 2: de paren 1,4,6,5.
- rooster 3: de paren 1,4,7,6.
- rooster 4: de paren 1,4,4,7.

Op dat moment zijn er vier oplossingen gevonden, en de roosters zijn geconstrueerd. Echter, een veel beter rooster zou zijn gevonden wanneer het zoekproces bij dag 4 zou zijn begonnen. In dat geval zou onder andere het rooster 4,3,3,1 zijn gevonden. Het voordeel van zoeken waarbij ook andere dagen als startpunt worden genomen, wordt nu duidelijk: als het zoekpro-

ces voor elke dag afzonderlijk wordt uitgevoerd hoeft nog steeds maar een deel van alle mogelijke roosters te worden gegenereerd, maar kan uiteindelijk winst worden geboekt doordat bij elke dag opnieuw de beste roosters worden samengesteld.

Uitgaande van een bepaalde startdag kunnen de mogelijke oplossingen als volgt worden gegenereerd:

- Uit de selectie van de paren voor de startdag wordt een paar gekozen, te beginnen bij het eerste paar.
- Vervolgens wordt voor de eerstvolgende dag een paar gekozen, opnieuw te beginnen bij het eerste paar van die dag.
- Als de geselecteerde paren geen overlap in personen vertonen (dat wil zeggen: wanneer het nieuwe paar uit unieke personen bestaat), dan kan dit proces voor de volgende dag worden herhaald, totdat paren voor alle beoogde dagen zijn gevonden.
- Als gedurende dit proces voor een willekeurige dag géén uniek paar kan worden gevonden, dan wordt het proces herhaald bij het volgende paar van de huidige dag.
- Wanneer alle paren voor een dag zijn geprobeerd, kan bij die dag geen uniek paar meer worden gevonden en wordt de zoekfunctie beëindigd.

poging nummer	paar nr. voor dag 1	paar nr. voor dag 2	paar nr. voor dag 3	paar nr. voor dag 4	paar nr. voor dag 5
1:	1:ok	1:ok	1:faalt	-	-
2:	1:ok	1:ok	2:faalt	-	-
3:	1:ok	1:ok	3:faalt	-	-
4:	1:ok	1:ok	4:faalt	-	-
5:	1:ok	1:ok	5:ok	1:faalt	-
6:	1:ok	1:ok	5:ok	2:faalt	-
7:	1:ok	1:ok	5:ok	3:faalt	-
8:	1:ok	1:ok	5:ok	4:faalt	-
9:	1:ok	1:ok	5:ok	5:faalt	-
etc.					

Figuur 19.5: De eerste negen pogingen om een oplossing te vinden.

Bij de paren die zijn vermeld in figuur 19.3 zal, wanneer de startdag dag 1 is, een serie mogelijkheden worden afgelopen die ten dele zijn afgebeeld in figuur 19.5. In deze figuur geeft elke regel een poging aan om een rooster te vinden.

- Poging 1: Begonnen wordt met paar 1 (de verpleegkundigen 1 en 29) op dag 1. Dit paar bevat unieke personen en is dus acceptabel. Het paar 1 van dag 2 kan daaraan worden toegevoegd, omdat de verpleegkundigen 2 en 26 een uniek paar vormen. Op dag 3 echter is paar 1 niet uniek: verpleegkundige 1 was al eerder opgenomen.
- Poging 2: De gevonden paren voor de dagen 1 en 2 blijven geaccepteerd, en geprobeerd wordt paar 2 van dag 3 toe te voegen. Dit

mislukt, omdat verpleegkundige 2 al was opgenomen.
- Poging 3: De gevonden paren voor de dagen 1 en 2 blijven geaccepteerd, en geprobeerd wordt paar 3 van dag 3 toe te voegen. Dit mislukt, omdat verpleegkundige 1 al was opgenomen.
- Poging 4: De gevonden paren voor de dagen 1 en 2 blijven geaccepteerd, en geprobeerd wordt paar 4 van dag 3 toe te voegen. Dit mislukt, omdat verpleegkundige 2 al was opgenomen.
- Poging 5: De gevonden paren voor de dagen 1 en 2 blijven geaccepteerd, en geprobeerd wordt paar 5 van dag 3 toe te voegen. Dit lukt, en de verpleegkundigen 3 en 28 worden opgenomen in het rooster.
- ...

poging nummer	paar nr. voor dag 2	paar nr. voor dag 3	paar nr. voor dag 4	paar nr. voor dag 5	paar nr. voor dag 1
1:	1:ok	1:faalt	-	-	-
etc.					

Figuur 19.6: Poging om een oplossing te vinden, uitgaande van dag 1.

Uiteindelijk zal echter blijken dat wanneer wordt begonnen bij paar 1 van dag 1 er geen enkel rooster kan worden gevormd. Pas bij paar 2 van dag 1 wordt een oplossing gevonden. Wanneer een andere dag (bijvoorbeeld dag 2) als startdag wordt genomen, dan worden paren gezocht voor achtereenvolgens dag 2, 3, 4, 5 en 1. De zoektabel krijgt dan de gedaante die is afgebeeld in figuur 19.6, waarbij alleen de eerste poging is afgebeeld. Let in deze figuur op de volgorde van de dagen: de eerste dag is niet dag 1, maar dag 2, terwijl de laatste dag niet dag 5 is, maar dag 1.

19.4 Implementatie van het roosterprogramma

Bij de implementatie wordt met één lokale header file *(schedule.h)* gewerkt, die in alle bronteksten gebruikt wordt. Elke functie staat in een eigen bestand (uitgezonderd static functies voor lokaal gebruik). Alle objectmodules worden in één bibliotheek *schedule.lib* verzameld, met uitzondering van de functie *main(),* die als losstaand bestand blijft bestaan ten behoeve van de linker.

19.4.1 De header file

```
1   // ———————————————————————————— typedefs, defines
2   #define MAXPERSON                30  // totaal aantal personen
3   #define AVAILABLE                15  // aantal beschikbare personen
4   #define NDUOS                    10  // aantal te genereren paren
5   #define NDAYS                     5  // aantal dagen in het rooster
```

```
6     #define NSOLS                        10  // aantal te genereren roosters
7     #define MAXINT                    0x7fff  // maximale 'int' waarde
8
9     typedef struct                            // informatie over een persoon
10    {
11        int
12            person,                           // persoonsnummer
13            value;                            // ervaringsgetal
14    } PERSON_;
15
16    typedef struct                            // informatie over een paar:
17    {
18        int
19            pers1,                            // nr. van 1-e persoon
20            pers2,                            // nr. van 2-e persoon
21            total_val;                        // gecombineerde ervaring
22    } DUO_;
23
24    typedef struct                            // een rooster:
25    {
26        int
27            dayindex[NDAYS],                  // nummers van de paren
28            total_val;                        // roosterwaarde
29    } SOLUTION_;
30
31    // ———————————————————————————————— data
32    extern PERSON_                            // personeel: NDAYS dagen
33        staff[NDAYS][AVAILABLE];              // van AVAILABLE personen.
34    extern DUO_                               // NDAYS dagen met NDUOS
35        daystaff[NDAYS][NDUOS];               // paren
36    extern SOLUTION_                          // de gegenereerde
37        solution[NSOLS];                      // roosters
38    extern int
39        total_sol,                            // aantal gegenereerde roosters
40        dayindex[NDAYS],                      // vector met paarnummers
41        nsolutions,                           // aantal oplossingen per dag
42        startday;
43    extern char
44        occupied[MAXPERSON];                  // vector met geselecteerde
45                                              // personen
46
47    // ———————————————————————————— functies
48    extern void
49        addsol (void),                        // voegt rooster toe
50        buildtab (void),                      // maakt tabel van personeel
51        printsug (void),                      // drukt suggesties af
52        schedule_day (int),                   // maakt paren voor een dag
53        showstaff (void),                     // vermeldt beschikbaar personeel
54        showsol (void),                       // geeft de roosters
55        solve (void),                         // maakt alle roosters
56        solveday (int);                       // maakt roosters voor 1 dag
```

Listing 159: De header file *schedule.h*

De lokale header file *schedule.h* is gegeven in listing 159. In de header file wordt een aantal datatypen gedefinieerd.

De informatie over de beschikbare personen wordt verzameld in een array *staff*, dat bestaat uit vijf rijen van vijftien elementen (er zijn per dag vijftien personen beschikbaar, en er zijn vijf dagen in een blok). Per persoon wordt informatie in *structs* van het type *PERSON_* bewaard. Dit datatype bestaat uit twee *ints:* de index bij het volgnummer[2] van de persoon en diens 'ervaringsgetal'.

Het datatype *DUO_* bestaat uit twee *ints,* die de indexen van twee verpleegkundigen in een paar representeren en uit een *int,* die de gecombineerde ervaringswaarde van die twee verpleegkundigen weergeeft. De paren die per dag worden samengesteld, worden opgenomen in het tweedimensionale array *daystaff,* dat uit vijf rijen (één rij per dag) van tien *DUO_ structs* bestaat.

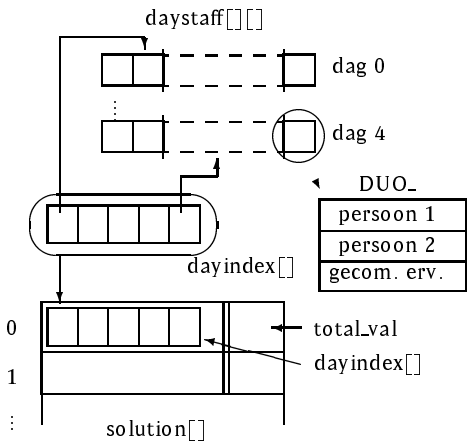
Bij het samenstellen van een rooster wordt de index van het paar, dat voor een bepaalde dag wordt ingeroosterd, onthouden in het array *dayindex,* dat vijf *int*-elementen heeft. Per dag is er één element. Zodra een rooster is gegenereerd, wordt het (zo nodig) opgeslagen in het array *solution* van het type *SOLUTION_*. Dit laatste datatype is een *struct* waarin een array *dayindex* van vijf *ints* is gedefinieerd, dat een kopie is van het globale array *dayindex,* dat is gebruikt bij het samenstellen van het rooster. Daarnaast bevat de *struct SOLUTION_* een *int* veld *total_val,* waarin de gecombineerde ervaring van de paren uit het rooster wordt onthouden. De manier waarop de informatie in de arrays *daystaff, dayindex* en *solution* aan elkaar is gerelateerd, wordt geïllustreerd in figuur 19.7.

In de header file worden de functies die in het programma worden gebruikt aan het einde gedeclareerd. Bij de ontwikkeling van het programma dienen de hulpfuncties in aparte brontekstbestanden te worden gedefinieerd. De bijbehorende objectmodules kunnen in een bibliotheek *schedule.lib* worden opgegenomen. Bij het linken moet de linker, naast de standaard bibliotheken, ook de bibliotheek *schedule.lib* gebruiken om het uiteindelijke programma te creëren.

19.4.2 De hulpfuncties

In de blauwdrukversie van het roosterprogramma wordt het array *staff* niet door de roostermaker opgeleverd, maar wordt een hulpfunctie *buildtab()* gebruikt om de invoertabellen met toevalsgetallen te vullen. De informatie die in het array *staff* is opgeslagen, kan vervolgens worden afgedrukt met behulp van een andere hulpfunctie, de functie *showstaff()*.

2. Het te ontwikkelen programma zal intern met 0-gebaseerde indexen werken. Omdat het voor mensen natuurlijker is om met 1-gebaseerde nummers te werken zal bij de uitvoer ervoor worden gezorgd dat indexwaarden worden opgehoogd. In de uitvoer zal dus 'persoon 1' en 'dag 1' verschijnen, terwijl door het programma met 'persoon 0' en 'dag 0' zal worden gerekend.



Figuur 19.7: De organisatie van de data.

Met behulp van deze twee hulpfuncties kan een zeer primitieve versie van het roosterprogramma worden geconstrueerd, waarmee de invoergegevens voor het roosteralgoritme kunnen worden afgedrukt.

De functie buildtab()

De functie *buildtab()* vult het array *staff* met toevalsgetallen. De functie bepaalt voor elk van de vijf dagen van een blok de indexen van vijftien verpleegkundigen, die vervolgens in het array *staff* worden opgenomen.

Wanneer de indexen van de verpleegkundigen worden gegenereerd, moet worden bijgehouden welke verpleegkundigen al zijn geselecteerd: deze personen kunnen op die dag niet nogmaals worden gekozen. Of een verpleegkundige al is geselecteerd wordt bijgehouden in een *char* array *occupied[]*. Dit array heeft evenveel elementen als er personen zijn, zodat een persoonsindex kan worden gebruikt als index in het array. Wanneer een element uit het array de waarde 0 heeft, dan is de overeenkomstige persoon nog niet geselecteerd Wanneer de waarde niet-nul is, dan is de overeenkomstige persoon voor die dag al wel geselecteerd.

De functie *buildtab()* bevat verder geen eigenaardigheden. De listing van de functie is gegeven in listing 160.

```
1         #include <memory.h>
2         #include <stdlib.h>
3         #include "schedule.h"
4
5         void buildtab ()
6         {
```

```
7           register int
8               tmp,                                // hulpvariabele
9               day,                                // dagteller
10              pers,                               // persoonsteller
11              prev;                               // vorige persoon
12
13          for (day = 0; day < NDAYS; day++)
14          {
15              for (pers = 0; pers < AVAILABLE; pers++)
16              {
17                  do
18                  {
19                      tmp = rand() % MAXPERSON;   // bepaal toevalsgetal
20                      for (prev = 0; prev < pers; prev++)
21                          if (occupied[tmp])      // is de persoon al aanwezig ?
22                              break;
23                  }
24                  while (occupied[tmp]++);        // blijf proberen tot een
25                                                  // nieuwe persoon
26
27                  staff[day][pers].person =       // neem de persoon en diens
28                      tmp;                        // ervaringsgetal op in staff[]
29                  staff [day][pers].value =
30                      (MAXPERSON * 4) - 4 * tmp;
31              }
32              memset(occupied, 0, sizeof(occupied));
33              putchar('\n');
34          }
35      }
36
```

Listing 160: De functie *buildtab()*.

De functie showstaff()

De functie *showstaff()* schrijft de inhoud van het array *staff* naar het beeldscherm. Elke rij (de informatie over een enkele dag) van het array *staff* wordt ten behoeve van de leesbaarheid gesorteerd op het ervaringsgetal. Merk op dat de sortering van het hoogste ervaringsgetal naar het laagste ervaringsgetal is uitgevoerd. Dit wordt bereikt door in de hulpfunctie *compare()* voor *qsort()* het verschil tussen de *value*-velden van het tweede en het eerste argument te retourneren. Bij de uitvoer wordt een tabel van 30 rijen (personen) bij 5 kolommen (dagen) afgedrukt, waarbij bij elk element van de tabel een persoonsnummer en een ervaringsgetal wordt afgedrukt. De functie *showstaff()* is gegeven in listing 161. Bij de afdruk worden alle indexwaarden omgezet naar natuurlijke getallen: in plaats van 'dag 0' verschijnt 'dag 1', enzovoort.

```
1       #include <stdio.h>
2       #include <search.h>
3       #include "schedule.h"
```

```
4
5         static void separator()
6         {
7             register int
8                 daynr;
9
10            for (daynr = 0; daynr < NDAYS; daynr++)
11                printf ("----------");
12            putchar ('\n');
13        }
14        static int compare (PERSON_ *a, PERSON_ *b)
15        {
16            return (b->value - a->value);
17        }
18
19        void showstaff ()
20        {
21            register int
22                day,                                // dagteller
23                pers;                               // persoonsteller
24
25            for (day = 0; day < NDAYS; day++)       // sorteer per dag
26                qsort (staff[day], AVAILABLE, sizeof(PERSON_), compare);
27
28            for (day = 1; day <= NDAYS; day++)      // vermeld "day 1" etc.
29                printf ("   day %1d"  , day);
30            putchar ('\n');
31
32            separator();                            // scheidingsregel
33
34            for (pers = 0; pers < NDAYS; pers++)    // vermeld
35                printf (" pers exp" );              // kolombetekenis
36            putchar ('\n');
37
38            for (pers = 0; pers < AVAILABLE; pers++)
39            {                                       // vermeld per persoon
40                for (day = 0; day < NDAYS; day++)   // en per dag:
41                    printf ("%4d %4d" ,
42                        staff[day][pers].person + 1, // het persoonsnummer
43                        staff[day][pers].value);    // en de ervaring
44                putchar ('\n');
45            }
46            separator();
47        }
48
```

Listing 161: De functie *showstaff()*.

19.4.3 De functie main()

```
1     #include <stdio.h>
2     #include "schedule.h"
3
```

```
4     #pragma comment (lib, "schedule")
5
6
7     PERSON_                                    // beschikbaar personeel:
8         staff[NDAYS][AVAILABLE];               // dagen bij personen
9     DUO_                                       // paren:
10        daystaff[NDAYS][NDUOS];                // dagen bij paren
11    SOLUTION_                                  // gegenereerde oplossingen
12        solution[NSOLS];
13    int
14        total_sol,                             // totaal aantal oplossingen
15        dayindex[NDAYS] =                      // indexen in daystaff[]
16            {-1, -1, -1, -1, -1},
17        nsolutions,                            // aantal oplossingen per dag
18        startday;
19    char
20        occupied[MAXPERSON];                   // geselecteerde personen
21
22
23    void main ()
24    {
25        register int
26            tryday;
27
28        buildtab ();
29        showstaff ();
30
31        puts ("\nSuggesting staff per day...");  // suggereer paren
32        for (tryday = 0; tryday < NDAYS; tryday++)// voor elke dag apart
33            schedule_day (tryday);
34        puts ("Suggested day staff:");
35        printsug ();
36
37        puts ("\nSolving...");                   // maak roosters
38        solve ();
39
40        puts ("Suggested solution:");            // vermeld de resultaten
41        showsol ();
42    }
```

Listing 162: De functie *main()*.

In listing 162 zijn de globale variabelen en de functie *main()* van het roosterprogramma gedefinieerd. De functies *buildtab()* en *showstaff(),* die in de secties 19.4.2 en 160 zijn gedefinieerd, worden aangeroepen om de invoergegevens voor het roosteralgoritme te genereren en af te drukken. Vervolgens wordt de functie *schedule_day()* aangeroepen om voor elke dag tweetallen samen te stellen. De gegenereerde tweetallen worden door de functie *printsug()* getoond. De functie *solve()* wordt geactiveerd om het roosterprobleem op te lossen. Tenslotte worden de gegenereerde oplossingen door de functie *showsol()* getoond.

De elementen van de variabele *dayindex* worden door de vertaler geïnitialiseerd op de waarde -1. De reden van deze initialisatie wordt toegelicht bij de beschrijving van de functie *solveday()* in 19.4.6.

19.4.4 De functie schedule_day()

De functie *schedule_day()* heeft tot taak om *NDUOS* paren samen te stellen uit de beschikbare staf (bewaard in het array *staff*) voor één bepaalde dag. De functie krijgt één argument: de dag, waarvoor paren samengesteld dienen te worden.

Het algoritme voor het samenstellen van paren werkt als volgt. Er zijn twee indexen nodig: één voor de eerste persoon en één voor de tweede. Deze indexen worden gebruikt om *staff[d]* af te lopen, waarbij *d* het dagnummer (0 tot en met 4) is. Als de index voor de ene persoon *one* heet, dan staat dus op *staff[d][one]* informatie over die persoon: veld *person* is het persoonsnummer en veld *value* is het ervaringsgetal van die persoon.

De index voor de eerste persoon, *one*, wordt gebruikt om vanaf 0 tot en met 13 te lopen (er zijn 15 beschikbare personen per dag). De index voor de tweede persoon loopt dan van *one + 1* tot en met 14, zodat alle paren worden geïnspecteerd. De volgorde van gegenereerde paren is dus:

- *staff[d][0]* met *staff[d][1]*;
- *staff[d][0]* met *staff[d][2]*;
- *staff[d][0]* met *staff[d][3]*;
- ....
- *staff[d][1]* met *staff[d][2]*;
- *staff[d][1]* met *staff[d][3]*;
- *staff[d][1]* met *staff[d][4]*;
- ....
- *staff[d][13]* met *staff[d][14]*;

```
1       void schedule_day(int d)
2       {
3           register int
4               one,
5               two;
6
7           for (one = 0; one < AVAILABLE - 1; one++)
8               for (two = one + 1; two < AVAILABLE; two++)
9               {
10                  // doe wat gedaan moet worden met het paar
11                  // staff[d][one] en staff[d][two]
12              }
13      }
```

Listing 163: Het code-frame voor de verwerking van paren.

Het code-frame dat kan worden gebruikt om alle paren te inspecteren, is gegeven in listing 163.

De functie *schedule_day()* moet de gegenereerde paren van een rooster in de variabele *daystaff* opnemen. Dit is (wederom) een tweedimensionaal array, met *NDAYS* rijen van *NDUOS* elementen. Elk element is van het type *DUO_*. Er zijn twee redenen waarom de functie *schedule_day()* kan besluiten de paren van een rooster op te nemen in het array *daystaff*. Ten eerste worden de paren uit een rooster opgenomen wanneer er minder dan *NDUOS* paren zijn gegenereerd. Daarna worden de paren uit een rooster opgenomen wanneer de gecombineerde ervaring van het nieuwe paar hoger is dan de gecombineerde ervaring van het paar met de laagste gecombineerde ervaring. In dat geval moet het nieuwe paar de plaats van het paar met de laagste gecombineerde ervaring innemen en wordt opnieuw het paar met de laagste gecombineerde ervaring bepaald.

```
1        // bepaal de gecombineerde ervaring van het nieuwe paar:
2        combined_exp = combine (&staff[d][one], &staff[d][two]);
3        // zijn er minder dan NDUOS paren ?
4        if (...)
5        {
6            // onthoud dan de laagste ervaring tot nu toe
7            // en het element van daystaff[d] waarbij dat zo is.
8            // vul het eerstvolgende vrije element van daystaff[d].
9            // klaar.
10       }
11
12       // doe niets wanneer de nieuwe gecombineerde ervaring lager
13       // is dan de laagste gecombineerde ervaring tot nu toe.
14
15       // vervang het element met de laagste ervaring tot nu toe
16       // door de informatie over het nieuwe paar.
17
18       // bepaal de index van het paar met de laagste ervaring,
19       // en onthoud de gevonden laagste ervaring.
```

Listing 164: Het code-frame voor het testen van een paar.

Het code-frame dat kan worden gebruikt om de functie te construeren waarmee een nieuw paar wordt getest, is gegeven in listing 164. De aanroep van de te construeren functie komt op de plaats van het commentaar in listing 163, dus binnen het dubbele *for*-statement. In het code-frame uit listing 164 worden de volgende variabelen, alle van het type *int,* gebruikt:

- De lokale variabele *combined_exp:* deze variabele wordt gebruikt om de gecombineerde ervaring van twee personen op te slaan. Deze waarde wordt bepaald door de functie *combine()* (zie hieronder), die twee argumenten van het type *PERSON_* $*$ krijgt.
- De globale statische variabele *nfound:* deze variabele bevat het aantal al gevonden paren. Deze variabele moet bij elke aanroep van *schedule_day()* op 0 worden geïnitialiseerd.

- De lokale variabele *target:* deze variabele wordt gebruikt om aan te geven waar in het array *daystaff* een nieuw paar geplaatst moet worden. De variabele wijst op de eerstvolgende vrije plaats in *daystaff* zolang *nfound* kleiner is dan *NDUOS*. Daarna wijst de variabele op het paar met de laagste gecombineerde ervaring.
- De globale statische variabele *worst:* de variabele *worst* houdt de laagst gevonden ervaringswaarde van alle gevonden paren bij. Deze variabele wordt bij elke aanroep van *schedule_day()* geïnitialiseerd op *MAXINT*.
- De globale statische variabele *worst_index:* deze variabele houdt de indexpositie bij in de vector *staff[day]* waarbij de gevonden ervaringswaarde gelijk is aan *worst*.

De functie die uit code-frame 164 wordt geconstrueerd, kan een statische functie *test()* zijn met drie *int* parameters: een parameter voor de dag (het argument waarmee *schedule_day()* wordt aangeroepen) en twee parameters voor de indexen van de personen van een paar dat moet worden getest. Deze functie kan op zijn beurt gebruik maken van een statische functie *add()* om een nieuw element in het array *daystaff* op te nemen. Deze functie krijgt vijf parameters: de twee indexen voor het array *daystaff,* de twee indexen van de personen die een paar vormen in het rooster en hun gecombineerde ervaring. Tenslotte kan een statische functie *combine()* worden gedefinieerd die de gecombineerde ervaring als het absolute verschil tussen de ervaringswaarden van twee personen retourneert.

Nadat de functie *schedule_day()* een aantal paren voor dag *d* heeft samengesteld, moeten deze paren nog op volgorde van hun gecombineerde ervaring worden gesorteerd. Hiervoor wordt de functie *qsort()* gebruikt. De functie *compare(),* die door *qsort()* wordt aangeroepen om twee elementen te vergelijken, kan ook als statische functie worden gedefinieerd. Deze functie krijgt twee argumenten die pointers naar *DUO_structs* zijn en retourneert het verschil tussen de gecombineerde ervaringswaarde van het tweede paar en het eerste paar, omdat de sortering in afnemende volgorde moet worden uitgevoerd.

Opgave 126.

Construeer functie *schedule_day()* op grond van de beschreven code-frames en specificaties.

Nadat *schedule_day()* paren heeft gegenereerd voor de vijf dagen van een werkweek is het prettig wanneer de paren per dag zichtbaar worden gemaakt. In een latere versie van het programma kan een roostermaker op dit punt ingrijpen door naar eigen inzicht bepaalde paren te verwijderen, of juist toe te voegen. De functie *printsug()* kan de gesuggereerde paren afdrukken op de manier waarop dat is gedaan in figuur 19.3.

Opgave 127.
Construeer de functie *printsug()* waarmee de door *schedule_day()* gegenereerde paren worden vermeld in een tabel die is opgebouwd als figuur 19.3

19.4.5 De functie solve()

```
1       #include <memory.h>
2       #include <search.h>
3       #include "schedule.h"
4
5       int compare (SOLUTION_ *a, SOLUTION_ *b)
6       {
7           return (b->total_val - a->total_val);
8       }
9
10      void solve ()
11      {
12          for (startday = 0; startday < NDAYS; // genereer roosters voor
13               startday++)                     // elke dag.
14          {
15              nsolutions = 0;                  // nog geen roosters gemaakt
16              solveday (startday);             // maak roosters voor deze dag
17          }
18
19          qsort (solution, total_sol,          // sorteer de gemaakte
20                 sizeof(SOLUTION_), compare);  // roosters
21      }
22
```

Listing 165: De functie *solve()*.

De roosters worden, zoals hiervoor beschreven, in twee stappen gegenereerd. Het genereren van een rooster houdt in dat bijgehouden moet worden welke personen al 'bezet' zijn, doordat zij in een eerder gekozen paar voorkomen: deze personen kunnen in geen geval bij een ander paar van dezelfde oplossing betrokken worden. Hiervoor wordt wederom gebruik gemaakt van het array *occupied0,* zoals dat ook bij de functie *buildtab()* in 19.4.2 is gedaan. Als een element uit het array de waarde 0 heeft, dan is de overeenkomstige persoon nog nergens ingeroosterd. Als de waarde niet-nul is, dan is de overeenkomstige persoon al wel ergens ingeroosterd.

Om een paar, dat is gedefinieerd in een element van *daystaff,* te kunnen accepteren, moet dus in ieder geval gelden dat de bij de personen van het paar horende elementen van het array *occupied* nog nul zijn. Wanneer een paar eenmaal is geaccepteerd, moeten deze elementen een waarde ongelijk aan 0 krijgen.

Zodra een rooster is gevonden, moet de gevonden oplossing nog bekend zijn om te kunnen worden opgenomen in de verzameling gegenereerde roosters. Een gevonden rooster wordt bewaard in het *int* array

dayindex. De index van de elementen in dit array correspondeert met een dagindex in het array *daystaff* en het element zelf correspondeert met een index van een paar (zie ook 19.3.1). De elementen kunnen bijvoorbeeld 0, 7, 2, 1, 4 zijn, waarmee wordt aangegeven dat paar 0 van dag 0, paar 7 van dag 1, paar 2 van dag 2, paar 1 van dag 3 en paar 4 van dag 3 een mogelijk rooster vormen. Initieel krijgen de elementen van *dayindex* de waarde -1, zodat bij een verhoging van hun waarde indexwaarden ontstaan.

De functie *solve()* genereert oplossingen voor elk van de vijf dagen. Deze oplossingen worden verzameld in een *SOLUTION_* array *solution,* dat aan het einde van de functie *solve()* wordt gesorteerd op de *total_val* velden van de *SOLUTION_structs*. De listing van de functie *solve()* is gegeven in listing 165.

19.4.6 De functie solveday()

De functie *solveday()*, aangeroepen door *solve()*, heeft één argument: de dag waarvoor een paar moet worden gezocht. De globale variabele *startday* bevat hierbij de dagindex van waaruit het zoeken begint.

De functie *solveday()* werkt als volgt:

1. Doe het volgende zolang er voor dag *day* nog niet-geïnspecteerde paren zijn en zolang er nog niet voldoende roosters zijn gegenereerd:
    2. Doe het volgende wanneer geen van beide personen van het paar al eerder werd geselecteerd:
        3. Rooster deze personen in voor de huidige dag.
        4. Als dag *day* de laatste dag van een blok is, onthoud dan de gegenereerde oplossing.
        5. Als dag *day* nog niet de laatste dag van een blok is, zoek dan (recursief) een paar voor de volgende dag.
        6. Beschouw de personen van het paar niet langer als ingeroosterd.
7. Zet de index van het eerstvolgende paar weer op -1.

Voor een bepaalde dag *day* worden alle paren getest. De index van het laatst geteste paar staat in *dayindex[day]*. Aangezien *dayindex[]* op –1 is geïnitialiseerd, heeft het eerstvolgende paar dat in aanmerking komt de index ++*dayindex[day]*. Als deze waarde gelijk is aan *NDUOS* dan is *solveday()* met dag *day* klaar, omdat er dan geen nieuwe paren meer zijn die moeten worden gecontroleerd. Daarnaast kan de situatie zich voordoen dat er, uitgaande van een bepaalde dag, al genoeg roosters zijn geconstrueerd. In beide gevallen kan de functie *solveday()* worden beëindigd. Onderdeel 1 kan daarom als een *while*-statement met een dubbele test worden geïmplementeerd: bij de eerste test wordt de waarde van *dayindex[day]* na verhoging vergeleken met de constante *NDUOS,* bij de tweede

test wordt gecontroleerd of er wellicht al voldoende roosters zijn geconstrueerd voor deze dag.

Voor een mogelijk paar geldt dat de gegevens van dat paar in element *daystaff[day][dayindex[day]]* (zie ook figuur 19.7) worden gevonden. De gegevens zijn de persoonsindexen van de personen van het paar en hun gecombineerde ervaring. De persoonsindexen worden in twee lokale variabelen (bijvoorbeeld *p1* en *p2)* van de functie *solveday()* bewaard.

In onderdeel 2 moet de functie *solveday()* nagaan of de twee personen van een paar al niet eerder waren ingeroosterd. Die informatie of dat het geval is, is opgeslagen in het array *occupied[]* op indexposities *p1* en *p2*. Beide waarden moeten nul zijn om een paar als mogelijke kandidaat te accepteren. Als één van beide of beide personen al eerder ingeroosterd was, dan kan *solveday()* dit paar verder negeren en kan het volgende paar worden getest. Als beide personen nog niet waren ingeroosterd, dan kan dat nu alsnog gebeuren.

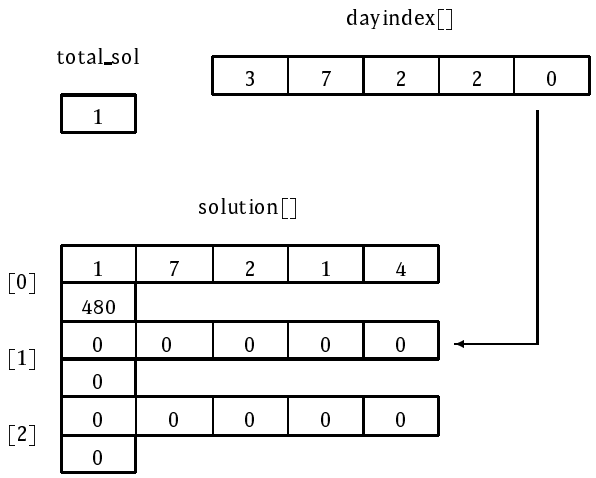
In onderdeel 3 worden onder andere de elementen *p1* en *p2* van het array *occupied[]* op 1 gezet. Hiermee wordt aangegeven dat de personen zijn ingeroosterd.

In onderdeel 4 wordt de vraag gesteld of een rooster nu klaar is, of dat er slechts een deel van een rooster is geconstrueerd. Dat kan worden bepaald uit de vergelijking tussen de eerstvolgende dag die op dag *day* volgt en de (globale) variabele *startday*. Als de eertvolgende dag gelijk is aan *startday*, dan is de oplossing klaar. Zoniet, dan moeten nog meer dagen worden ingeroosterd. Bij onderdeel 4 wordt de hulp ingeroepen van een functie *nextday()* die de eerstvolgende dag in de reeks bepaalt, uitgaande van het meegegeven argument. Wanneer er vijf dagen in een blok zijn, volgt dag 1 op dag 0, dag 2 op dag 1, enzovoort, totdat op dag 4 weer dag 0 volgt. Wanneer *nextday()* de waarde *startday* retourneert, is het rooster klaar. Wanneer het rooster klaar is, wordt de functie *addsol()* aangeroepen om het gevonden rooster op te slaan in het array *solution*.

In onderdeel 5 is geconstateerd dat er nog meer dagen zijn waarvoor een rooster moet worden opgesteld. Om de roosters voor de overige dagen te bepalen, wordt de functie *solveday()* recursief aangeroepen om de roosters voor de volgende dagen af te werken.

Bij onderdeel 6 is de functie *solveday()* klaar met de personen van een paar. De personen kunnen in volgende roosters opnieuw worden ingeroosterd. Dit wordt kenbaaar gemaakt door de elementen *p1* en *p2* van het array *occupied* weer de waarde 0 te geven.

In onderdeel 7 wordt ervoor gezorgd dat bij een nieuwe aanroep van de functie *solveday()* weer bij het eerste paar wordt begonnen. Hiertoe wordt *dayindex[day]* weer de waarde -1 gegeven.



Figuur 19.8: De organisatie van het array *solution*.

Opgave 128.

Stel functie *solveday()* samen op basis van bovenstaande beschrijving. De functie maakt gebruik van een statische functie *nextday()* waarmee de volgende dagindex wordt bepaald, en een *void* functie *addsol()* die bij onderdeel 4 wordt aangeroepen. De functie *addsol()* hoeft niet als onderdeel van deze opgave te worden gedefinieerd.

19.4.7 De functie addsol()

De functie *addsol()* wordt vanuit *solveday()* aangeroepen als een volledig rooster is bepaald. De indexen van de paren die in het rooster moeten worden opgenomen, zijn dan opgeslagen in het array *dayindex[]*.

De gevonden roosters worden bewaard in een array van het type *SOLUTION_*. Dit type bestaat uit een array van *NDAYS ints,* die de indexen van de paren in het array *dayindex[]* zullen bevatten en uit een *int* die de som van de ervaringswaarden van de paren van een rooster bevat. De organisatie van het array *solution* is weergegeven in figuur 19.8. In deze figuur is in array *solution[]* al één oplossing opgeslagen. Deze oplossing wijst paar 1 aan voor de eerste dag, paar 7 aan voor de tweede dag, enzovoort. De gecombineerde ervaringswaarde van alle vijf paren is 480: 108 + 80 + 104 + 100 + 88 (vergelijk ook figuur 19.3). Het aantal reeds bekende oplossingen is opgeslagen in de globale integer *total_sol*. Een nieuwe oplossing,

opgeslagen in vector *dayindex[]*, zal in het tweede element van *solution[]* worden opgeslagen.

Daarnaast wordt bijgehouden hoeveel oplossingen er zijn gegenereerd in de globale variabele *nsolutions*. Deze variabele krijgt voor de aanroep van de functie *solveday()* vanuit de functie *solve()* de waarde 0, ten teken dat er voor de desbetreffende dag nog geen oplossingen zijn gevonden. De variabele *nsolutions* wordt echter bij elke oplossing die aan het array *solutions* wordt toegevoegd opgehoogd, zodat in de functie *solveday()* kan worden gestopt met het genereren van oplossingen wanneer er voor een dag *NSOLS* oplossingen zijn gevonden.

Het (mogelijk) toevoegen van een oplossing gaat volgens het volgende schema:

- Eerst wordt de gecombineerde waarde bepaald van de paren die door *dayindex[]* aangewezen worden. Deze waarde wordt, wanneer het rooster wordt geaccepteerd, als veld *total_val* van één van de elementen van *solution[]* bewaard.
- Wanneer *total_sol* kleiner is dan *NSOLS*, dan kan het rooster onvoorwaardelijk worden geaccepteerd. Het rooster wordt toegevoegd door de statische functie *add()*. Bovendien wordt gecontroleerd of de waarde van het veld *total_val* van het nieuwe rooster lager is dan de laagste tot nu toe gevonden waarde van een veld *total_val*. Als dat het geval is, dan worden de variabelen aangepast, waarin de index en de waarde van de laagst gevonden waarden worden onthouden. Hierna kan de functie worden verlaten.
- Wanneer de waarde van de gecombineerde ervaring van het zojuist gevonden rooster lager is dan de laagste tot nu toe gevonden gecombineerde ervaring, dan wordt het nieuwe rooster niet opgenomen in de verzameling roosters, en kan de functie *addsol()* weer worden verlaten.
- Op dit punt kan worden geconstateerd dat het nieuwe rooster beter is dan het slechtste rooster dat tot nu toe werd gevonden. De functie *add()* kan worden aangeroepen om het nieuwe rooster op de plaats van het slechtste rooster dat tot nu toe werd gevonden te plaatsen.
- Opnieuw wordt de plaats en de gecombineerde ervaring van het rooster met de laagste gecombineerde ervaring bepaald.

Merk op dat de procedure om nieuwe oplossingen aan het array *solutions* toe te voegen sterk lijkt op de procedure die bij de functie *test()* van de functie *schedule_day()* is gevolgd om een paar voor een bepaalde dag toe te voegen. De functies *addsol()* en *add()* kunnen dan ook overeenkomstig worden geconstrueerd.

Opgave 129.
Construeer de functie *addsol()* op grond van bovenstaande specificaties analoog aan de functie *test()* van de functie *schedule_day()*.

Tenslotte moeten de gevonden roosters worden vermeld. De functie *showsol()* wordt vanuit *main()* aangeroepen om de door *solve()* gegenereerde oplossingen af te drukken. De functie *showsol()* kan eenvoudig worden geconstrueerd en dient de volgende informatie te geven:

- Het volgordenummer van het rooster en de totale ervaring van de paren die in het rooster zijn opgenomen.
- Per dag de persoonsnummers en de gecombineerde ervaring van de verpleegkundigen die als paar op de desbetreffende dag zijn ingeroosterd.

Opgave 130.
Construeer de functie *showsol()* aan de hand van de gegeven specificaties.

DEEL III

BIJLAGEN

Bijlage A

Syntaxis van C

A.1 De grammatica van C

Onderstaande beschrijving geeft de syntaxis van de **C** taal weer gedefinieerd zijn de terminale symbolen voor een integer-constante, voor een character-constante, voor een floating-point-constante, voor een identifier, voor een string, en voor een enumeration constante. **Vet** afgedrukte namen zijn letterlijk op te vatten. Alternatieven in een regel worden aangegeven door een || teken; optionele elementen worden aangegeven door *_opt*.

translation-unit:
    external-declaration || translation-unit external-declaration

external-declaration:
    function-definition || declaration

function-definition:
    declaration-specifiers*_opt* declarator declaration-list*_opt* compound-statement

declaration:
    declaration-specifiers init-declaration-list*_opt* ;

declaration-list:
    declaration || declaration-list declaration

declaration-specifiers:
    storage-class-specifier declaration-specifiers*_opt* ||
    type-specifier declaration-specifiers*_opt* ||
    type-qualifier declaration-specifiers*_opt*

storage-class-specifier:
    auto || **register** || **static** || **extern** || **typedef**

type-specifier:
    void || **char** || **short** || **int** || **long** || **float** ||
    double || **signed** || **unsigned** ||

```
    struct-or-union-specifier   ||   enum-specifier   ||   typedef-name

type-qualifier:
    const   ||   volatile

struct-or-union-specifier:
    struct-or-union-specifier_opt { struct-declaration-list }   ||
    struct-or-union-identifier

struct-or-union:
    struct   ||   union

struct-declaration-list:
    struct-declaration   ||   struct-declaration-list struct-declaration

init-declarator-list:
    init-declarator   ||   init-declarator-list , init-declarator

init-declarator:
    declarator   ||   declarator = initializer

struct-declaration:
    specifier-qualifier-list struct-declarator-list ;

specifier-qualifier-list:
    type-specifier specifier-qualifier-list_opt   ||
    type-qualifier specifier-qualifier-list

struct-declarator-list:
    struct-declarator   ||   struct-declarator-list , struct-declarator

struct-declarator:
    declarator   ||
    declarator_opt : constant-expression

enum-specifier:
    enum identifier_opt { enumerator-list }   ||
    enum identifier

enumerator-list:
    enumerator   ||   enumerator-list , enumerator

enumerator:
    identifier   ||   identifier = constant-expression

declarator:
    pointer_opt direct-declarator

direct-declarator:
    identifier   ||
    ( declarator )   ||
    direct-declarator [ constant-expression_opt ]   ||
    direct-declarator ( parameter-type-list )   ||
    direct-declarator ( identifier-list_opt )
```

pointer:
  ∗ type-qualifier-list_*opt* ||
  ∗ type-qualifier-list_*opt* pointer

type-qualifier-list:
  type-qualifier ||
  type-qualifier-list type-qualifier

parameter-type-list:
  parameter-list ||
  parameter-list , ...

parameter-list:
  parameter-declaration || parameter-list , parameter-declaration

parameter-declaration:
  declaration-specifiers declarator ||
  declaration-specifiers abstract-declarator_*opt*

identifier-list:
  identifier || identifier-list , identifier

initializer:
  assignment-expression ||
  { initializer-list } ||
  { initializer-list , }

initializer-list:
  initializer || initializer-list , initializer

type-name:
  specifier-qualifier-list abstract-declarator_*opt*

abstract-declarator:
  pointer ||
  pointer_*opt* direct-abstract-declarator

direct-abstract-declarator:
  (abstract-declarator) ||
  direct-abstract-declarator_*opt* [constant-expression_*opt*] ||
  direct-abstract-declarator_*opt* (parameter-type-list_*opt*)

typedef-name:
  identifier

statement:
  labeled-statement || expression-statement ||
  compound-statement || selection-statement ||
  iteration-statement || jump-statement

labeled-statement:
  identifier : statement ||
  case constant-expression : statement ||
  default : statement

```
expression-statement:
    expression_opt ;

compound-statement:
    { declaration-list_opt statement-list_opt }

statement-list:
    statement    ||    statement-list statement

selection-statement:
    if ( expression ) statement    ||
    if ( expression ) statement else statement    ||
    switch ( expression ) statement

iteration-statement:
    while ( expression ) statement    ||
    do statement while ( expression ) ;    ||
    for ( expression_opt ; expression_opt ;
        expression_opt ) statement

jump-statement:
    goto identifier ;    ||    continue ;    ||
    break ;    ||    return expression_opt ;

expression:
    assignment-expression    ||    expression , assignment-expression

assignment-expression:
    conditional-expression    ||
    unary-expression assignment-operator assingment-expression

assignment-operator:
    =    ||    *=    ||    /=    ||    %=    ||    +=    ||
        -=    ||    <<=    ||    >>=    ||    &=    ||
        ^=    ||    |=

conditional-expression:
    logical-or-expression    ||
    logical-or-expression ? expression : conditional-expression

constant-expression:
    conditional-expression

logical-or-expression:
    logical-and-expression    ||
    logical-or-expression || logical-and-expression

logical-and-expression:
    inclusive-or-expression    ||
    logical-and-expression && inclusive-or-expression

inclusive-or-expression:
    exclusive-or-expression    ||
    inclusive-or-expression | exclusive-or-expression
```

```
exclusive-or-expression:
    and-expression   ||
    exclusive-or-expression ^ and-expression

and-expression:
    equality-expression   ||
    and-expression & equality-expression

equality-expression:
    relational-expression   ||
    equality-expression == relational-expression   ||
    equality-expression != relational-expression

relational-expression:
    shift-expression   ||
    relational-expression < shift-expression   ||
    relational-expression > shift-expression   ||
    relational-expression <= shift-expression   ||
    relational-expression >= shift-expression

shift-expression:
    additive-expression   ||
    shift-expression << additive-expression   ||
    shift-expression >> additive-expression

additive-expression:
    multiplicative-expression   ||
    additive-expression + multiplicative-expression   ||
    additive-expression - multiplicative-expression

multiplicative-expression:
    cast-expression   ||
    multiplicative-expression * cast-expression   ||
    multiplicative-expression / cast-expression   ||
    multiplicative-expression % cast-expression

cast-expression:
    unary-expression   ||   ( type-name ) cast-expression

unary-expression:
    postfix-expression   ||
    ++ unary-expression   ||
    -- unary-expression   ||
    unary-operator cast-expression   ||
    sizeof unary-expression   ||
    sizeof type-name

unary-operator:
    &   ||   *   ||   +   ||   -   ||   ~   ||   !

postfix-expression:
    primary-expression   ||
    postfix-expression [ expression ]   ||
    postfix-expression ( argument-expression-list_opt )   ||
    postfix-expression . identifier   ||
```

```
    postfix-expression -> identifier    ||
    postfix-expression ++    ||
    postfix-expression --

primary-expression:
    identifier    ||    constant    ||    string    ||    ( expression )

argument-expression-list:
    assignment-expression    ||    assignment-expression-list , assignment-expression

constant:
    integer-constant    ||    character-constant    ||
    floating-constant    ||    enumeration-constant
```

Onderstaande grammatica geeft de structuur van preprocessor-controleregels weer. Symbool text is hierin niet gespecificeerd en staat voor gewone brontekst.

```
control-line:
    #define identifier token-sequence    ||
    #define identifier ( identifier , ... ,
        identifier ) token-sequence    ||
    #undef identifier    ||
    #include < filename >    ||
    #include "filename "    ||
    #include token-sequence    ||
    #line constant "filename "    ||
    #line constant    ||
    #error token-sequence_opt    ||
    #pragma token-sequence_opt    ||
    #    ||
    preprocessor-conditional

preprocessor-conditional:
    if-line text elif-parts else-part_opt #endif

if-line:
    #if constant-expression    ||
    #ifdef identifier    ||
    #ifndef identifier

elif-parts:
    elif-line text    ||
    elif-parts_opt

elif-line:
    #elif constant-expression

else-part:
    else-line text

else-line:
    #else
```

A.2 De keywords van C

In deze bijlage worden alle keywords van **C** die in versie 6.00 van de Microsoft **C** vertaler zijn gedefinieerd genoemd. Keywords zijn gereserveerde namen en mogen door de programmeur alleen voor hun specifieke doel worden gebruikt.

Keywords die met een underscore (_) beginnen, zijn specifiek voor Microsoft's implementatie van **C** en horen niet tot de ANSI-versie, maar zijn er een uitbreiding op.

Keywords:				
_asm	*do*	*for*	*_pascal*	*static*
auto	*double*	*_fortran*	*register*	*struct*
_based	*else*	*goto*	*return*	*switch*
break	*_emit*	*_huge*	*_saveregs*	*typedef*
case	*enum*	*if*	*_segment*	*union*
_cdecl	*_export*	*int*	*_segname*	*unsigned*
char	*extern*	*_interrupt*	*_self*	*void*
const	*_far*	*_loadds*	*short*	*volatile*
continue	*_fastcall*	*long*	*signed*	*while*
default	*float*	*_near*	*sizeof*	

A.3 Escape-sequences

In deze bijlage worden de escape-sequences besproken. Escape-sequences beginnen met de \, en kunnen in strings en character constanten worden gebruikt om specifieke tekens aan te duiden.

Escape-sequences zijn gedefinieerd als speciale ASCII tekens. Het doel van een escape-sequence is een implementatie-onafhankelijke definitie te krijgen van speciale handelingen (zoals de backspace). In de praktijk is de gebruikte definitie echter gebaseerd op de ASCII tekens. Daarom geven we het gebruikte ASCII teken in de onderstaande tabel louter ter informatie, zonder te willen impliceren dat de gegeven escape-sequences noodzakelijk gekoppeld zijn aan de gegeven ASCII tekens. De aanduiding van de betekenis van de escape-sequences zijn onvertaald gelaten, en spreken vermoedelijk voor zichzelf. Opgemerkt wordt nog dat een nieuwe regel in een string wordt aangeduid door *n*, en het indrukken van de enter-toets wordt ingelezen als *r*. Bij het bedrijfssysteem DOS worden de *n* tekens in strings over en weer geconverteerd naar *r* *n* tekencombinaties in tekstbestanden.

Sequence:	*Betekenis:*	*ASCII waarde*
\a	*Alert (bell)*	*7*
\b	*Backspace*	*8*
\f	*Form feed*	*12*
\n	*Newline*	*10*
\r	*Carriage return*	*13*
\t	*Horizontal tab*	*9*
\v	*Vertical tab*	*11*
\0ddd	*Ascii teken 'ddd', octaal genoteerd*	
\xdd	*Ascii teken 'dd', hexadecimaal genoteerd*	

Alle overige tekens die door een \worden voorafgegaan, worden letterlijk, zonder de ervoor staande \, overgenomen. Zo wordt apostrof (de quote) aangegeven als \', en de \zelf als \\.

A.4 Operatoren, associativiteit en prioriteit

Deze bijlage toont de operatoren van C en hun volgorde van prioriteit. Bovenaan de tabel staan operatoren met de hoogste prioriteit; operatoren met lagere prioriteit zijn gescheiden door een lijn. Naast de prioriteit is het type van de operatie weergegeven. Alle operatoren, behalve de unaire, ternaire en assignment operatoren, associëren van links naar rechts; de unaire, ternaire en assignment operatoren associëren van rechts naar links.

Operator	Beschrijving
( )	(sub) expression
[ ]	indexing
.	struct field
–>	struct field using pointer
–	unary min (toggle sign)
~	bitwise not (toggle bits)
!	logical not
*	unary pointer operator
&	unary adress-of operator
+ +	unary increment
- -	unary decrement
sizeof	unary size macro
type cast	unary type casting
*	multiplication
/	division
%	modulo operation
+	addition
-	subtraction
<<	shift left
>>	shift right

Operator	Beschrijving
<	smaller than
>	greater than
<=	smaller or equal
>=	greater or equal
==	equal
!=	not equal
&	bitwise and
^	bitwise xor
|	bitwise or
&&	logical and
||	logical or
? :	ternary operator
=	assignment
*=	multiplication assignment
/=	division assignment
%=	modulo assignment
+=	addition assignment
-=	subtraction assignment
<<=	leftshift assignment
>>=	rightshift assignment
&=	binary and assignment
|=	binary or assignment
^=	binary xor assignment
,	sequential evaluation

Bijlage B

Standaard-C versus ANSI-C

In deze bijlage worden een aantal verschillen tussen standaard-**C** en ANSI-**C** geïllustreerd.

Commentaar in een brontekst wordt in standaard-**C** voorafgegaan door /* en afgesloten door */. Dit commentaar kan meerdere regels beslaan. In ANSI-**C** bestaat daarnaast het end-of-line comment, dat voorafgegaan wordt door // en tot het einde van de regel loopt.

Argumenten van een functie worden in standaard-**C** bij de functiedefinitie in de argumentenlijst alleen bij naam genoemd. Het type van de argumenten wordt na de functienaam en de argumentenlijst gespecificeerd. Een voorbeeld hiervan is functie *add()* in listing 166.

```
1     /*
2       Function add(): takes two integers, and
3       adds them
4     */
5
6     add (a, b)
7     int a;
8     int b;
9     {
10        ...                          // code of function
11        ...                          // goes here
12    }
```

Listing 166: De functie *add()* volgens standaard-**C**.

Bij ANSI-**C** kan het type van de argumenten van een functie in de argumentenlijst worden opgenomen. Een voorbeeld hiervan is dezelfde functie *add()* in listing 167.

Declaraties van externe functies kennen in standaard-**C** geen vermelding van het aantal en het soort van argumenten. Zo stelt een declaratie:

```
extern f();
```

```
1     /*
2        Function add(): takes two integers, and
3        adds them
4     */
5
6     add (int a, int b)
7     {
8         ...                                  // code of function
9         ...                                  // goes here
10    }
```

Listing 167: De functie *add()* volgens ANSI-**C**.

dat er een externe[1] functie *f()* bestaat. Of *f()* voorzien moet worden van argumenten, en zo ja hoeveel en van welk type, wordt niet bekend gemaakt.
Bij ANSI-**C** mag een functiedeclaratie voorzien worden van argumentspecificaties. Zo stelt de declaratie:

extern f(**int**, **int**)

dat er een externe functie *f()* bestaat en dat bij de aanroep van deze functie twee argumenten moeten worden meegegeven, beide van het type *int*[2].

Strings (stukken tekst) mogen in standaard-**C** over meerdere regels verdeeld worden door het \ teken te gebruiken. Zo is de string:
*"de C *
compiler"
dezelfde als:
"de C compiler" .
ANSI-**C** staat daarnaast toe dat strings in stukken opgegeven mogen worden. Twee fragmenten (bijvoorbeeld *"de C"* en *" compiler"*) worden als één string opgevat; deze stukken mogen op dezelfde of verschillende regels staan (als ertussen geen andere informatie staat).

1. Een externe functie of variabele is in een andere brontekst aangemaakt. Met de declaratie van een externe functie of variabele wordt de vertaler over het bestaan van die functie of variabele geïnformeerd.
2. Type *int* staat voor integer, oftewel een geheel getal.

Bijlage C

Variabele aantallen argumenten

In deze bijlage wordt nader ingegaan op het gebruik van variabele aantallen argumenten. Aan het einde van hoofdstuk 5 werd een vuistregel gegeven voor de ontwikkeling van functies met variabele aantallen argumenten. In deze bijlage wordt het hoe en waarom van deze vuistregel toegelicht en wordt behandeld hoe die vuistregel kan worden versoepeld.

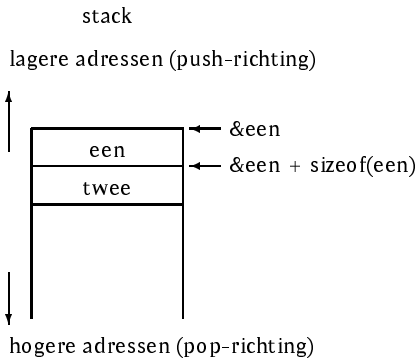
Om te beginnen geven we hier de macro's *va_start()* en *va_arg()*:

```
typedef char *va_list;
#define va_start(ap,v)    ap = (va_list)&v + sizeof(v)
#define va_arg(ap,t)      ( (t *)  (ap += sizeof(t)) ) [-1]
```

Het type `va_list` is een *char* pointer. *va_start* zet variabele *ap* op het adres van variabele *v* plus de omvang van het datatype van *v*. Aangezien *v* een parameter van een functie is en argumenten van functies via de stack worden doorgegeven, is het adres van *v* dus een locatie op de stack. In **C** worden argumenten in omgekeerde volgorde op de stack gezet, en bij de INTEL 80x86 processors wordt de stack van het hoogste adres naar het laagste adres gevuld. Wanneer een functie twee *int* argumenten *een* en *twee* krijgt, zoals in:

fun(een, twee);

dan staat de waarde van variabele *twee* dus op een hoger stack-adres dan de waarde van variabele *een*. Variabele *twee* bevindt zich dus op het adres van variabele *een* + de omvang van een *int* waarde (zie ook figuur C.1). Wanneer echter de variabelen *een* en *twee* in de functie gedefinieerd zijn als *char* parameters, of (in het algemeen) als parameters die uit een oneven aantal bytes bestaan, ontstaat er een conflict tussen de manier waarop de stack wordt behandeld en de werking van de *va_start()* macro. Op de stack worden alleen *int* waarden of een veelvoud van *int* waarden geplaatst. Ook wanneer een *char* waarde op de stack wordt gezet, wordt de *char* waarde



Figuur C.1: Twee *int* argumenten op de stack.

als *int* waarde op de stack gezet. Wanneer de laatste parameter van een functie met variabel aantal argumenten dus een *char* parameter is (zoals bij de volgende functie) dan zal de variabele *list* na het statement *va_start(list, ch)* dus niet op het eerstvolgende argument wijzen, maar op een locatie in de stack die één byte verder is dan het adres van *ch*. Deze waarde is het hoge byte van de *int* waarde op de stack waar *ch* staat. De variabele *list* wordt dan door *va_start()* verkeerd geïnitialiseerd.

Het probleem treedt ook op bij parameters van een array-type. Array-argumenten worden als het adres van het eerste byte van het array aan een functie doorgegeven. Wanneer de laatste parameter van een functie met variabele aantallen argumenten bijvoorbeeld van het type *int [20]* is, dan zal *va_start()* de variabele *list* initialiseren op een stack-locatie die zich 20 *ints* hoger op de stack bevindt, terwijl in feite slechts een array pointer op de stack wordt gezet bij de aanroep van de functie.

De moraal van het verhaal is dat bij de ontwikkeling van een functie met variabele aantallen argumenten, rekening moet worden gehouden met de eigenaardigheden van *va_start()*. Dat kan door ervoor te zorgen dat de laatste parameter van een functie met variabele aantallen argumenten van een type is dat een even *sizeof()* waarde retourneert, terwijl de *sizeof()* van de parameter gelijk moet zijn aan de *sizeof()* waarde van het type van de parameter. Een dergelijke functie die een *char* parameter als laatste parameter zou moeten hebben, moet worden ontwikkeld als een functie die een *int* als laatste parameter heeft. Evenzo moet een dergelijke functie geen laatste parameter hebben die een array-parameter is, maar moeten array-parameters worden gedefinieerd als array pointers.

De *va_arg()* macro leidt aan hetzelfde euvel. Deze macro zet de pointer op de variabele argumenten één argument positie verder, en re-

tourneert dan de waarde van het vorige argument. Ook hier wordt met behulp van de *sizeof()* operator de plaats van het volgende argument bepaald. Wanneer het type *char* wordt gebruikt bij *va_start()* zal de argument pointer steeds één byte verder worden gezet. Echter, *char* argumenten worden als *int* argumenten op de stack gezet, waarbij het hoge byte van de *int* waarde niet gedefinieerd is, en zeker niet de waarde van het volgende argument zal bevatten. Opnieuw moet bij de ontwikkeling van de functie ervoor worden gezorgd dat de types van de variabele argumenten een even *sizeof()* waarde retourneren, terwijl de *sizeof()* van het type gelijk moet zijn aan de *sizeof()* waarde van de waarde die wordt geretourneerd. Wederom dus geen arrays als argumenten maar array-pointers, en geen *chars* als argumenten maar *ints,* ook al worden bij de aanroep van de functie *char* waarden aan de functie doorgegeven.

Bijlage D

Layout van C brontekst

Deze bijlage illustreert de in dit boek aanghouden layout-conventies.

Preprocessor directives. Preprocessor directives (*#include, #define*, etc.) beginnen op de eerste kolom van de tekst.

Tabulatie. De brontekst is op een standaard manier geïndenteerd. Gewoonlijk zijn tabulatiestops 4 of 8 spaties. Belangrijk is, dat het inspringen *consistent* is.

Accolades. De accolades, welke het begin en eind van een blok markeren, staan elk op een *aparte* regel, en worden eventueel gevolgd door commentaar. Het accoladepaar dat het eerste niveau van een blok weergeeft (zoals het begin en eind van een functie) staat op de eerste kolom van een regel. Accolades zijn altijd geïndenteerd op het niveau van de erboven staande code.

Functies zonder argumenten. Functies zonder argumenten worden gedefinieerd en gedeclareerd met een *void* argumentenlijst, conform de ANSI-**C** conventie.

Met ingang van versie 6.00 van Microsoft's **C** vertaler worden ook waarschuwingen gegeven wanneer *void* niet is vermeld, waar dat eigenlijk wèl zou moeten.

Indentering van code. De code die op een open accolade volgt, staat op een andere regel dan de accolade zelf en is geïndenteerd over één extra tabulatiestop.

Bij conditioneel uitgevoerde code (bij voorbeeld de code na een *if*-statement) geldt ook dat één niveau extra geïndenteerd wordt. Deze code wordt dus geïndenteerd alsof deze door accolades omgeven zou zijn. Dezelfde indentering wordt ook voor een *null* (leeg) statement gebruikt.

De *if-else ladder* (een gegeneraliseerd *switch*-statement) wijkt hier enigszins vanaf: de *else if* keywords komen op één regel voor.

Complexe *for*-statements. Wanneer de drie code onderdelen van een *for*-statement complex (of lang) worden, verdient het aanbeveling om

elk onderdeel op een nieuwe regel te laten beginnen en extra te indenteren. Een voorbeeld van een geïndenteerd *for*-statement is in listing 168 gegeven.
Bij een geïndenteerd *for*-statement worden de haakjes als accolades behandeld (geïndenteerd op het niveau van het *for* keyword).

```
1     for
2     (
3         x = *argv++;                    // initialisatie
4             x && ! strcmp (x, "end");   // lus controle
5                 x++;                    // verhoging
6     )
```

Listing 168: Een geïndenteerd *for*-statement.

Het switch-statement. Het *switch*-statement zelf is geïndenteerd als elke andere code. De labels van de *switch* zijn over één extra positie geïndenteerd. De code die op de labels volgt is verder geïndenteerd.
Het *default* label van een *switch*-statement staat als laatste in de opsomming, tenzij de uitvoering van een *default*-statement in een alternatief moet doorvallen.
Op het *break*-statement kan een lege regel volgen, hierdoor is het einde van de *case* beter zichtbaar.

Goto labels. *Goto* labels staan op een aparte regel, *niet* geïndenteerd. Overigens is het gebruik van *goto* af te raden.

Gebruik van spaties en haakjes. Spaties worden gebruikt om de leesbaarheid van code te verhogen. Relationele, binaire en assignment operatoren worden omgeven door spaties. Unaire operatoren zijn *niet* door spaties van hun argument gescheiden.
In expressies kunnen haakjes gebruikt worden om de leesbaarheid te vergroten.
Wanneer de expressie complex wordt, raden we aan de afzonderlijke delen van de expressie op aparte regels te zetten en de omhullende haakjes wederom als accolades te behandelen (ook genest).
Een voorbeeld van een geïndenteerde expressie is in listing 169 gegeven. Let op de indentering van de expressieëlementen en op de indentering van het *puts()*-statement na de expressie (de *puts()* is verder geïndenteerd dan de *if*).

Variabelen-declaraties en macro's. Het type van variabelen moet bij declaraties altijd gegeven zijn, ook als het om het default type *int* gaat (zie ook onderdeel 2.2.3).
Macronamen (*define*-s) worden in hoofdletters gegeven. De *define*-s worden bovenaan de brontekst genoemd om de zichtbaarheid ervan te vergroten.

```
1     if                                    // als bestand niet geopend
2     (                                     // kan worden, of als de
3         !                                 // buffer niet gealloceerd
4         (                                 // kan worden ....
5             (file = fopen("demo", "r"))
// termineren!
6             &&
7             (buffer = malloc(10000))
8         )
9     )
10        puts("Can't open file or allocate memory");
```

Listing 169: Een geïndenteerde expressie.

Aangeraden wordt om zelf gedefiniëerde *struct, union* en *enum typedef*s een naam in hoofdletters te geven, waaraan een *underscore* teken wordt toegevoegd. Daarnaast krijgen zelf gedefinieerde structs en unions geen *struct tags*, tenzij ze 'self referential' zijn.

Bijlage E

Unix System V Flex Description

De hieronder gegeven beschrijving van de codegenerator *flex* is de originele documentatie bij de *flex* implementatie onder UNIX system V.

NAME

flex - fast lexical analyzer generator

SYNOPSIS

flex [-dfirstvFILT -c[efmF] -Sskeleton_file] filename

DESCRIPTION

flex is a rewrite of lex intended to right some of that tool's deficiencies: in particular, flex generates lexical analyzers much faster, and the analyzers use smaller tables and run faster.

OPTIONS

In addition to lex's **-t** flag, flex has the following options:

-d makes the generated scanner run in debug mode. Whenever a pattern is recognized the scanner will write to stderr a line of the form:
–accepting rule #n
Rules are numbered sequentially with the first one being 1.

-f has the same effect as lex's **-f** flag (do not compress the scanner tables); the mnemonic changes from **fast compilation** to (take your pick) **full table** or **fast scanner**. The actual compilation takes longer, since flex is I/O bound writing out the big table.
This option is equivalent to **-cf** (see below).

-i instructs flex to generate a **case-insensitive** scanner. The case of letters given in the flex input patterns will be ignored, and the rules will be matched regardless of case. The matched text given in **yytext** will have the preserved case (i.e., it will not be folded).

-r specifies that the scanner uses the **REJECT** action.

-s causes the **default rule** (that unmatched scanner input is echoed to **stdout**) to be suppressed. If the scanner encounters input that does not match any of its rules, it aborts with an error. This option is useful for finding holes in a scanner's rule set.

-v has the same meaning as for lex (print to **stderr** a summary of statistics of the generated scanner). Many more statistics are printed, though, and the summary spans several lines. Most of the statistics are meaningless to the casual flex user.

-F specifies that the **fast** scanner table representation should be used. This representation is about as fast as the full table representation (**-f**), and for some sets of patterns will be considerably smaller (and for others, larger). In general, if

the pattern set contains both "keywordsänd a catch-all, ïdentifier"rule, such as in the set:

```
"case"      return ( TOK_CASE );
"switch"    return ( TOK_SWITCH );
...
"default"   return ( TOK_DEFAULT );
[a-z]+      return ( TOK_ID );
```

then you're better off using the full table representation. If only the ïdentifier"rule is present and you then use a hash table or some such to detect the keywords, you're better off using **-F**.
This option is equivalent to **-cF** (see below).

-I instructs flex to generate an **interactive** scanner. Normally, scanners generated by flex always look ahead one character before deciding that a rule has been matched. At the possible cost of some scanning overhead (it's not clear that more overhead is involved), flex will generate a scanner which only looks ahead when needed. Such scanners are called **interactive** because if you want to write a scanner for an interactive system such as a command shell, you will probably want the user's input to be terminated with a newline, and without **-I** the user will have to type a character in addition to the newline in order to have the newline recognized. This leads to dreadful interactive performance.
If all this seems to confusing, here's the general rule: if a human will be typing in input to your scanner, use **-I**, otherwise don't; if you don't care about how fast your scanners run and don't want to make any assumptions about the input to your scanner, always use **-I**.
Note, **-I** cannot be used in conjunction with **full** or **fast tables**, i.e., the -f, -F, -cf, or -cF flags.

-L instructs flex to not generate #line directives (see below).

-T makes flex run in **trace** mode. It will generate a lot of messages to standard out concerning the form of the input and the resultant non-deterministic and deterministic finite automatons. This option is mostly for use in maintaining flex.

-c[**efmF**] controls the degree of table compression. **-ce** directs flex to construct **equivalence classes**, i.e., sets of characters which have identical lexical properties (for example, if the only appearance of digits in the flex input is in the character class "[0-9]"then the digits '0', '1', ..., '9' will all be put in the same equivalence class). **-cf** specifies that the **full** scanner tables should be generated - flex should not compress the tables by taking advantages of similar transition functions for different states. **-cF** specifies that the alternate fast scanner representation (described above under the **-F** flag) should be used. **-cm** directs flex to construct **meta-equivalence classes**, which are sets of equivalence classes (or characters, if equivalence classes are not being used) that are commonly used together. A lone **-c** specifies that the scanner tables should be compressed but neither equivalence classes nor meta-equivalence classes should be used. The options **-cf** or **-cF** and **-cm** do not make sense together - there is no opportunity for meta-equivalence classes if the table is not being compressed. Otherwise the options may be freely mixed.
The default setting is **-cem** which specifies that flex should generate equivalence classes and meta-equivalence classes. This setting provides the highest degree of table compression. You can trade off faster-executing scanners at the cost of larger tables with the following generally being true:

slowest	smallest
-cem	
-ce	
-cm	
-c	
-c{f,F}e	
-c{f,F}	
fastest	largest

-Sskeleton_file overrides the default skeleton file from which flex constructs its scanners. You'll never need this option unless you are doing flex maintenance or development.

INCOMPATIBILITIES WITH LEX

flex is fully compatible with **lex** with the following exceptions:

- There is no run-time library to link with. You needn't specify **-ll** when linking, and you must supply a main program. (Hacker's note: since the lex library contains a main() which simply calls yylex(), you actually **can** be lazy and not supply your own main program and link with **-ll**.)
- lex's **%r** (Ratfor scanners) and **%t** (translation table) options are not supported.
- The do-nothing **-n** flag is not supported.
- When definitions are expanded, flex encloses them in parentheses. With lex, the following

```
NAME        [A-Z][A-Z0-9]*
%%
foo{NAME}?      printf( "Found it\n");
%%
```

  will not match the string "foo"because when the macro is expanded the rule is equivalent to "foo[A-Z][A-Z0-9]*?änd the precedence is such that the '?' is associated with "[A-Z0-9]*". With flex, the rule will be expanded to "foo([A-z][A-Z0-9]*)?änd so the string "foo"will match.
- **yymore()** is not supported.
- The undocumented lex-scanner internal variable **yylineno** is not supported.
- If your input uses **REJECT**, you must run flex with the **-r** flag. If you leave out the flag, the scanner will abort at run-time with a message that the scanner was compiled without the flag being specified.
- The **input()** routine is not redefinable, though may be called to read characters following whatever has been matched by a rule. If **input()** encounters and end-of-file the normal **yywrap()** processing is done. A "real" end-of-file is returned as **EOF**.

  Input can be controlled by redefining the **YY_INPUT** macro. **YY_INPUT**'s calling sequence is IJY_INPUT (buf, result, maxsize)". Its action is to place up to max_size characters in the character buffer "bufänd return in the integer variable "resultëither the number of characters read or the constant YY_NULL (0 on Unix systems) systems) to indicate EOF. The default YY_INPUT reads from the file-pointer ijyin"(which is by default **stdin**), so if you just want to change the input file, you needn't redefine YY_INPUT - just point yyin at the input file.

  A sample redefinition of YY_INPUT (in the first section of the input file):

```
%{
#undef YY_INPUT
#define YY_INPUT(buf,result,max_size) \
        result = (buf[0] = getchar()) == EOF ? YY_NULL : 1;
%}
```

You also can add in things like counting keeping track of the input line number this way; but don't expect your scanner to go very fast.

- **output()** is not supported. Output from the ECHO macro is done to the file-pointer ijyout"(default **stdout**).
- Trailing context is restricted to patterns which have either a fixed-sized leading part or a fixed-sized trailing part. For example, ä∗/bänd
  ä/b∗äre okay, but not ä∗/b∗". This restriction is due to a bug in the trailing context algorithm given in **Principles of Compiler Design** (and **Compilers - Principles, Techniques, and Tools**) which can result in mismatches. Try the following lex program

```
%%
x+/xy        printf( "I found %s\n", yytext );
```

  on the input "xxy". (If anyone knows of a fast algorithm for finding the beginning of trailing context for an arbitrary pair of regular expressions, please let me know!) If you must have arbitrary trailing context, you can use **yyless()** to effect it.
- flex reads only one input file, while lex's input is made up of the concatenation of its input files.

ENHANCEMENTS

- **Exclusive start-conditions** can be declared by using **%x** instead of **%s**. These start-conditions have the property that when they are active, **no other rules are active**. Thus a set of rules governed by the same exclusive start condition describe a scanner which is independent of any of the other rules in the flex input. This feature makes it easy to specify "mini-scanners"which scan portions of the input that are syntactically different from the rest (e.g., comments).
- flex dynamically resizes its internal tables, so directives like "%a 3000äre not needed when specifying large scanners.
- The scanning routine generated by flex is declared using the macro **YY_DECL**. By redefining this macro you can change the routine's name and its calling sequence. For example, you could use:

```
#undef YY_DECL
#define YY_DECL float lexscan( a, b ) float a, b;
```

  to give it the name **lexscan**, returning a float, and taking two floats as arguments.
- flex generates **#line** directives mapping lines in the output to their origin in the input file.
- You can put multiple actions on the same line, separated with semi-colons. With lex, the following

```
foo       handle_foo(); return 1;
```

  is truncated to

```
foo       handle_foo();
```

  flex does not truncate the action. Actions that are not enclosed in braces are terminated at the end of the line.
- Actions can be begun with %{ and terminated with %}. In this case, flex does not count braces to figure out where the action ends - actions are terminated by the closing %}. This feature is useful when the enclosed action has extraneous braces in it (usually in comments or inside inactive #ifdef's) that throw off the brace-count.
- All of the scanner actions (e.g., **ECHO**, **yywrap**) except the **unput()** and **input()** routines, are written as macros, so they can be redefined if necessary without requiring a separate library to link to.

FILES

- **flex.sk**
  skeleton scanner
- **flex.fsk**
  skeleton scanner for -f and -F
- **flexscom.h**
  common definitions for skeleton files
- **flexsdef.h** definitions for compressed skeleton file
- **fastsdef.h** definitions for -f, -F skeleton file

SEE ALSO

lex(1)
M. E. Lesk and E. Schmidt, **LEX - Lexical Analyzer Generator**

AUTHOR

Vern Paxson, with the help of many ideas and much inspiration from Van Jacobson. Original version by Jef Poskanzer. Fast table representation is a partial implementation of a design done by Van Jacobson. The implementation was done by Kevin Gong and Vern Paxson.
Thanks to the many flex beta-testers, especially Casey Leedom, Nick Christopher, Chris Faylor, Eric Goldman, Craig Leres, Mohamed el Lozy, Esmond Pitt, Jef Poskanzer, and Dave Tallman. Thanks to John Gilmore, Bob Mulcahy, Rich Salz, and Richard Stallman for help with various distribution headaches.
Send comments to:
Vern Paxson
Real Time Systems
Bldg. 46A
Lawrence Berkeley Laboratory
1 Cyclotron Rd.
Berkeley, CA 94720
(415) 486-6411
vern@lbl-{csam,rtsg}.arpa
ucbvax!lbl-csam.arpa!vern

DIAGNOSTICS

flex scanner jammed - a scanner compiled with **-s** has encountered an input string which wasn't matched by any of its rules.
flex input buffer overflowed - a scanner rule matched a string long enough to overflow the scanner's internal input buffer (as large as **BUFSIZ** in "/usr/include/stdio.h"). You can edit **flexscom.h** and increase **YY_BUF_SIZE** and **YY_MAX_LINE** to increase this limit.
REJECT used and scanner was not generated using -r - just like it sounds. Your scanner uses **REJECT**. You must run flex on the scanner description using the **-r** flag.
old-style lex command ignored - the flex input contains a lex command (e.g., "%n 1000") which is being ignored.

BUGS

Use of unput() or input() trashes the current yytext and yyleng.
Use of unput() to push back more text than was matched can result in the pushed-back text matching a beginning-of-line ('^') rule even though it didn't come at the beginning of the line.
Nulls are not allowed in flex inputs or in the inputs to scanners generated by flex. Their presence generates fatal errors.
Do not mix trailing context with the '|' operator used to specify that multiple rules use the same action. That is, avoid constructs like:

```
foo/bar |
bletch |
bugprone    { ... }
```

They can result in subtle mismatches. This is actually not a problem if there is only one rule using trailing context and it is the first in the list (so the above example will actually work okay). The problem is due to fall-through in the action switch statement, causing non-trailing-context rules to execute the trailing-context code of their fellow rules. This should be fixed, as it's a nasty bug and not obvious. The proper fix is for flex to spit out a
FLEX_TRAILING_CONTEXT_USED #define and then have the backup logic in a separate table which is consulted for each rule-match, rather than as part of the rule action. The place to do the tweaking is in add_accept() - any kind soul want to be a hero?
The pattern:
x{3}
is considered to be variable-length for the purposes of trailing context, even though it has a clear fixed length.
Due to both buffering of input and read-ahead, you cannot intermix calls to, for example, **getchar()** with flex rules and expect it to work. Call **input()** instead.
The total table entries listed by the -**v** flag excludes the number of table entries needed to determine what rule has been matched. The number of entries is equal to the number of DFA states if the scanner was not compiled with -**r**, and greater than the number of states if it was.
The scanner run-time speeds have not been optimized as much as they deserve. Van Jacobson's work shows that the can go quite a bit faster still.

Bijlage F

Antwoorden op opgaven

Opgave 1:
1. Alles behalve het sign-bit: 7f (hex), 127 (dec), 127 (pos. decimaal)
2. Alleen sign-bit: 80 (hex), -128 (dec), 128 (pos. decimaal)
3. -(-128) is -80(hex), is 7f + 1: 80 (hex), -128 (dec), 128 (pos. decimaal)
4. 78 (hex), 120 (decimaal), 120 (pos. decimaal)
5. 87 (hex), -121 (decimaal), 135 (pos. decimaal)
6. ff (hex), -1 (decimaal), 255 (pos. decimaal).

Opgave 2:
Ontstane bestanden zijn *hello.obj* en *hello.exe*.

Opgave 3:
Na de *hello* wordt een nieuwe regel gestart.

Opgave 4:
Programma's werken identiek. Het tweede *.exe* bestand is groter dan het eerste, omdat informatie voor *codeview* erin is opgenomen.

Opgave 6:
1. Hierdoor wordt de header file *stdio.h* door de preprocessor gelezen. Dit bestand bevat o.a. declaraties van functies, zoals *getchar()* en *putchar()*.
2. De haakjes geven aan dat *main* een functie is. De term *void* geeft aan dat *main()* niets aan de aanroeper 'teruggeeft'.
3. Functie *getchar()* leest een character van de standaard-invoerstroom (*stdin*), gewoonlijk het toetsenbord. Functie *putchar()* schrijft een character naar de standaarduitvoerstroom.
4. Functie *printf()* schrijft de tekst die als argument meegegeven wordt naar de standaarduitvoerstroom (het scherm). De *\n* is voor *printf()* een signaal om een nieuwe regel te beginnen. Op het scherm zal dus een nieuwe regel begonnen worden, dan zal 'End of file.' verschijnen, en daarna zal weer een nieuwe regel begonnen worden.
5. Het programma zal tekens lezen en naar het scherm schrijven, totdat end-of-file bereikt wordt. Door het '<' teken (het redirectieteken voor DOS) zal het programma geen tekens van het toetsenbord lezen, maar zal bestand *repeat.c* afwerken.

Opgave 7:
De volgende namen kunnen niet als identifier worden gebruikt.

3 *18_* Begint niet met een letter of underscore.
6 *is_dit_nog_wel_een_identifier_of_is_dit_geen_identifier_?* Het vraagteken is geen letter, cijfer, of underscore.

Opgave 8:

1.
```
unsigned char
    n_spaces,
    n_letters;
```

(*unsigned* kan vervallen wanneer het standaard type *char* al *unsigned* is).

2.
```
unsigned int
    n_spaces,
    n_letters;
```

(*int* kan desgewenst vervallen omdat *int* het default type is.

3.
```
unsigned long int
    n_spaces,
    n_letters;
```

(*int* kan desgewenst vervallen omdat *int* het default type is.

Opgave 9:

1. int index = 3 * 331;
2. static int statvar;
3. Kan niet. De waarde van *index* is niet compile time te bepalen.
4. int locin = 3 * 331;
5. static int statloc = 0;
6. register int locreg = index;

Opgave 10:

j = 10, i = 11

Opgave 11:

(-i) is geen lvalue, maar een tussenresultaat. De – – operator kan alleen bij een lvalue gebruikt worden.

Opgave 12:

```
1     #include <stdio.h>
2
3     void main ()
4     {
5         printf ("De omvang van char   is %d \n", sizeof (char) * 8);
6         printf ("De omvang van int    is %d \n", sizeof (int) * 8);
7         printf ("De omvang van float  is %d \n", sizeof (float) * 8);
8         printf ("De omvang van double is %d \n", sizeof (double) * 8);
9     }
```

Opgave 13:

```
#include <stdio.h>

void timescalled ()
{
    static int                                  // = 0 initialisatie
        ncalls = 0;                             // kan vervallen
```

```
    ncalls++;                                   // verhoog ncalls

    printf ("Functie is %d keer aangeroepen.\n", ncalls);
}

void main ()
{
    timescalled ();                             // eerste aanroep
    timescalled ();                             // tweede aanroep
    timescalled ();                             // derde aanroep
}
```

Opgave 14:
j = 13, i = 7

Opgave 15:
j = -42, i = 7

Opgave 16:
j = 2, i = 9

Opgave 17:
Van rechts naar links.

Opgave 18:
a = 4

Opgave 19:
i = 5, j = 2. Volgens de layout conventie moeten om de – in –j spaties staan.

Opgave 20:
De *while*-lus wordt 20 keer doorlopen. Kennelijk bepaalt bij de Microsoft vertaler de laatste expressie van een sequentiele evaluatie de lusconditie. Dit is *niet anders* dan proefondervindelijk te bepalen! Bij andere vertalers kunnen andere regels gelden.

Opgave 21:
double

Opgave 22:
C. Te achterhalen op de volgende manier: 707651 is in hexadecimale notatie 0xacc43. Het minst significante byte is dan 0x43, oftewel decimaal 67. In een ASCII tabel kan opgezocht worden dat dit de letter 'C' is.

Opgave 23:
1. De functie *puts()* schrijft een string naar *stdout* (meestal het beeldscherm) en schrijft dan zelf nog een \n-teken naar *stdout*. De functie *fprintf()* is een functie die net zo werkt als *printf()* maar de informatie naar een gespecificeerd bestand stuurt, in plaats van naar *stdout*.
2. Met *stdprn* wordt de printer van de computer bedoeld. *stdprn* is één van de standaardstreams die in C beschikbaar zijn. Informatie die naar *stdprn* wordt geschreven, wordt naar de printer-aansluiting van de computer gestuurd (en zal dus, normaliter, worden geprint).

Opgave 24:
1.

```
    void a()
    {
    }
```

2. De functie *a()* doet niets.

Opgave 25:

1. *int* is het default type, en wordt door de vertaler verondersteld wanneer geen expliciet type wordt vermeld.
2. *extern* mag worden weggelaten omdat een functieprototype al een functiedeclaratie is.

Opgave 26:

1. char *proto_1* (int, int);
2. float *proto_2* (int [], char []);
2. void *proto_3* (char []);

Opgave 27:

```
#include <stdio.h>
#include <stdarg.h>
#include <process.h>

void error(char *fmt, ...)
{
    va_list
        list;

    // zet list op eerste argument na fmt:
    va_start(list, fmt);

    // forceer nieuwe regel
    putchar('\n');

    // print de foutmelding
    vprintf(fmt, list);

    // stop het programma
    exit(1);
}
```

Opgave 28:

Gebruik na vertaling de link opdracht `l wild+\c\lib\setargv/noe`. Het programma is:

```
void main(int argc, char *argv[])
{
    int
        index;

    for (index = 0; index < argc; index++)
        puts(argv[index]);
}
```

Opgave 29:

2. Gebruik om te linken `l ccount+c\lib\setargv/noe`.
3. Verander `unsigned count[256]` in `long count[256]` en vertaal en link het gewijzigde programma opnieuw.
4. Verander `if (count[index] && isprint(index))`
in `if (count[index] && (index > 127 || isprint(index)))`
5. Bij Microsoft: *printf()* in `stdio.h`, *memset()* in `string.h` en *memory.h, isprint()* in `ctype.h`.

Opgave 30:

2. Steeds worden dezelfde toevalskeuzes gedaan: bij herhaalde aanroep levert *rand()* steeds dezelfde waarden.

3. Het statement

srand((unsigned)clock());

is vereist. Het programma levert nu na vertaling en linken niet steeds dezelfde toevalskeuzes, maar steeds andere.
4. Definieer de *int* variabelen *smalluser, biguser* en *new_roll* in plaats van de overeenkomstige *#defines* en vervang de in hoofdletters geschreven namen van constanten door de namen van de nieuwe variabelen (in kleine letters). De essentiele wijzigingen van de functie *main()* worden:

```
int
    new_roll,
    biguser,
    smalluser,
    roll[2];

void main(int argc, char *argv[])
{
    int
        spilled = 0,
        refresh = 0;

    new_roll  = atoi(argv[1]);
    biguser   = atoi(argv[2]);
    smalluser = atoi(argv[3]);

    // overige 'statements'
}
```

Opgave 31:

In fragment 1 komt de string twee maal voor; in fragment 2 slechts één maal. Fragment 2 is dus te prefereren. Eén en ander kan d.m.v. de /E optie bij de *cl* opdracht gecontroleerd worden.

Opgave 32:

```
#define MIN(a,b,c)      ( (a) < (b) ?                   \
                            (a) < (c) ? (a) : (c)       \
                        :                               \
                            (b) < (c) ? (b) : (c)       \
                        )
```

Opgave 33:

```
#define ERROREXIT(msg)          \
    { printf ("%s\n", msg); exit (1); }
```

Opgave 34:

```
/*
   fcopy (in, out)
   Kopieert per teken bestand 'in' naar 'out'.
*/
void fcopy (FILE *infile, FILE *outfile)
{
    char
        buffer [200];                           // databuffer
    unsigned
        result;                                 // aantal gelezen items

    while (1)                                   // herhaal:
    {
        result = fread (buffer,                 // lees data van 'infile'
```

```
            sizeof (char), 200, infile);
        if (! result)                                   // niets gelezen?
            break;                                      // verlaat de lus
        fwrite (buffer, sizeof (char),                  // schrijf data naar
            result, outfile);                           // 'outfile'
    }
}
/*
   compact alternatief voor fread():
     if (! (result = fread (buffer, sizeof (char), 200, infile)) )
         break;
*/
```

Opgave 35:

(a) Number 80
(b) Same number 50, now as hex

Opgave 36:

void process (char [], char []) ;

Opgave 37:

Het aantal geconverteerde waarden. In dit voorbeeld is dit 1 als de conversie slaagt, of 0 als de conversie niet slaagt.

Opgave 38:

```
1     void process (char line [], char formatstring [])
2     {
3         int
4             number;
5
6         if (sscanf (line, formatstring, &number) == 0)
7             puts ("Bad input, try again.");
8         else
9             printf ("The number: 0x%x hexadecimal, %d decimal.\n",
10                number, number);
11    }
```

Opgave 39:

```
#include <stdio.h>                                      // voor FILE, fprintf()
#include <process.h>                                    // voor exit()

FILE *xfopen (char name [], char mode [])
{
    FILE
        *return_file;                                   // return-waarde

    if (!                                               // open het bestand
        ( return_file = fopen (name, mode) )
      )
    {                                                   // het mislukt:
        fprintf (stderr,                                // geef informatie
            "Cannot open %s for %s\n",
                name,
                mode [0] == 'r'
                    ?
                        "reading"
                    :
                        "writing" );
        exit (1);                                       // einde van 't programma
    }

    return (return_file);                               // het lukt: retourneer
}                                                       // de FILE pointer
```

Opgave 40:

```
1     void compare (FILE *in1, FILE *in2)
2     {
3         register int
4             ch1,
5             ch2;
6         unsigned
7             line = 1;
8         long
9             pos1 = 0,
10            pos2 = 0;
11
12        while (1)
13        {
14            ch1 = fgetc (in1);
15            ch2 = fgetc (in2);
16            if (ch1 != ch2)
17            {
18                showline (1, line, in1, pos1);
19                showline (2, line, in2, pos2);
20                pos1 = ftell (in1);
21                pos2 = ftell (in2);
22                line++;
23            }
24            else if (ch1 == '\n')
25            {
26                line++;
27                pos1 = ftell (in1);
28                pos2 = ftell (in2);
29            }
30            if (feof (in1) || feof (in2))
31                break;
32        }
33    }
```

Opgave 41:

```
#include <io.h>
#include <fcntl.h>
#include <types.h>
#include <stat.h>

int str2flag (char *mode)
{
    register int
        flag = 0;

    switch (*mode++)                    // bekijk eerste teken:
    {                                   // 'r', 'a', or 'w'
        case 'r':                       // bewerking is lezen
            flag = O_RDONLY;
        break;

        case 'a':                       // bewerking is bijschrijven
            flag = O_WRONLY | O_APPEND; // aan het einde
        break;

        case 'w':                       // bewerking is schrijven
            flag = O_WRONLY | O_TRUNC | O_BINARY;
        break;
    }

    while (*mode++)                     // bekijk volgende tekens:
    {                                   // '+', 't', 'b'
        switch (*mode)
        {
            case '+':                   // lezen EN schrijven
                flag &= ~(O_RDONLY | O_WRONLY);
                flag |= O_RDWR;
```

```
            break;

            case 't':                   // geen binaire maar
                flag &= ~O_BINARY;      // tekstmodus
                flag |= O_TEXT;
            break;

            case 'b':                   // geen tekstmodus maar
                flag &= ~O_TEXT;        // binaire modus
                flag |= O_BINARY;
            break;
        }
    }

    return (flag);
}
```

Opgave 42:

De vertaler geeft een waarschuwing dat de types van argument *argument_2* en parameter *parameter_2* niet overeenkomen, maar vertaalt het programma verder wel. Wanneer het programma wordt gestart, dan zal de waarde van *parameter_2* 5 zijn, in plaats van het adres van *argument_2*. De functie *change()* zal keurig een increment uitvoeren van de waarde waarop door *parameter_2* wordt gewezen. De increment zal dus worden uitgevoerd op de inhoud van geheugenlocatie 5, met onvoorspelbare gevolgen.

Opgave 43:

```
#include <stdio.h>
#include <malloc.h>
#include <process.h>

char
    out_of_memory[] = "\nOut of memory";

void *xrealloc(void *ptr, unsigned size)
{
    if
    (
        !(ptr = realloc (ptr, size))
        &&
        size
    )
    {
        puts (out_of_memory);
        exit(1);
    }

    return (ptr);
}
```

Opgave 44:

De functie *readint()* zou hebben gewerkt in geval 3: De inhoud van *countptr* wordt bereikt doordat de haakjes de normale evaluatievolgorde doorbreken. De waarde is een *int,* waarop de increment operator wordt toegepast.
Bij 1 Wordt op grond van de prefix-increment *countptr* op het het eerstvolgende *int* adres gezet, met de inhoud-van operator wordt verder niets gedaan. Wat er dus verandert is het pointerargument *countptr* zelf, in plaats van de *int* waarde waar *countptr* op wijst.
Bij 2 wordt op grond van de postfix-increment na afloop van de expressie de waarde van *countptr* zelf opgehoogd, zodat *countptr* na afloop van het statement steeds één *int* waarde verder zal gaan wijzen. Met de inhoud-van operator wordt verder niets gedaan. Bij 4 gebeurt hetzelfde als bij 2. De haakjes bij 4 wijzigen de normale evaluatievolgorde niet.

Opgave 45:

```
1     #include <stdio.h>
2     #include <string.h>
3
4     // de functie strsort() komt hier
5
6     void main(int argc, char **argv)
7     {
8         int
9             i;
10
11        strsort(argc, argv);
12        for (i = 0; i < argc; i++)
13            puts(argv[i]);
14    }
```

Opgave 46:

```
1     char *progname(char *program)
2     {
3         while (*program++)
4             ;                                   // Vind einde van de string
5
6         while (*--program != '.')               // Vind extensie
7             ;
8
9         *program = 0;                           // Vernietig de extensie
10
11        while (*--program != '\\')              // Vind laatste '\'
12            ;
13
14        return (program + 1);                   // Retourneer programma naam
15    }
```

Opgave 47:

1. "Argument '1' is kleiner dan argument '2' "
2. "Argument 'een' komt in het alfabet na argument '2' "
3. "Argument '1' komt in het alfabet voor argument 'twee' "
4. "Argument 'een' komt in het alfabet voor argument 'twee' "
5. "Argument 'Een' komt in het alfabet voor argument 'twee' "
6. "Argument 'string' is gelijk aan argument 'STRING' "

Opgave 48:

De vereiste hulpfunctie is

```
int indirect_strcmp(char **el_1, char **el_2)
{
    return (strcmp(*el_2, *el_1));
}
```

Opgave 49:

Bij de uitwerking van de opgave is aangenomen dat:

argv[1]: naam subject-1
argv[2]: leeftijd subject-1
argv[3]: naam subject-2
argv[4]: leeftijd subject-2

```
#include <stdio.h>
#include <string.h>

typedef struct
{
    char
        *naam;
    int
        leeftijd;
} PERSON_;

PERSON_
    subject_1,
    subject_2;

void main(int argc, char **argv)
{
    subject_1.naam = strdup(argv[1]);
    sscanf(argv[2], "%d", &subject_1.leeftijd);
    subject_2.naam = strdup(argv[3]);
    sscanf(argv[4], "%d", &subject_2.leeftijd);

    printf(
        "persoon 1: naam: %s, leeftijd: %d\n"
        "persoon 1: naam: %s, leeftijd: %d\n",
            subject_1.naam, subject_1.leeftijd,
            subject_2.naam, subject_2.leeftijd);
}
```

Opgave 50:

```
union                                                   // union definitie
{
    char
        str[11];
    long
        long_nr;
    int
        int_nr;
    unsigned char
        byte_nr;
}
    unionvar =                                          // variabele definitie
    {
        "I love it!"                // initialisatie
    };
```

De eerste vier tekens van de string *"I love it!" zijn* ascii-*tekens 0x49, 0x20, 0x6c* en *0x6f*. De *long* value wordt dus *0x6f6c2049,* hetgeen 1.869.357.129 is. De *int* waarde maakt gebruik van de eerste twee bytes en is dus *0x6f6c,* hetgeen 28.524 is. Het *byte_nr* veld is de waarde van het eerste byte *(0x6f),* en is dus 111.
Veld *str* is gedefinieerd als een *char [11]* omdat de string uit 10 tekens + een *ascii-z* bestaat.

Opgave 51:

```
void findname()
{
    char
```

```
        target[81],
        source[81];
    int
        found = 0,
        index;

    printf("\nEnter the name to search:" );          // 1.
    gets(target);                                     // 2.
    strlwr(target);                                   // 3.
    for (index = 0; index < maxadr; index++)          // 4.
    {
        strcpy(source, address[index].name);          // 5.
        strlwr(source);                               // 6.
        if (strstr(source, target))                   // 7.
        {
            found++;                                  // 8.
            printf( "\n"                              // 8.
                    "Name:   %s\n"
                    "Street: %s\n"
                    "City:   %s\n"
                    "Phone:  %s\n",
                        address[index].name,
                        address[index].street,
                        address[index].city,
                        address[index].phone);
        }
    }
    if (!found)                                       // 9.
        printf("\nNot found: '%s'\n", target);
}
```

Opgave 52:

```
// aanpassing van de CHOICE_ enum:
typedef enum
{
    choice_add,
    choice_list,
    choice_quit,
    choice_search,
} CHOICE_;

// aanpassing van het keys[] array:
    keys [] = "alqs"

// aanpassing van de functie mainmenu():
CHOICE_ mainmenu ()
{
    ...
    puts ("\n"                                        // aanpassing menu-string
          "(a) Add entry\n"
          "(l) List base\n"
          "(q) Quit\n"
          "(s) Search for a name\n"
         );
    ...
}

// Voeg de functie findname() toe voor de functie main()

// aanpassing van de functie main():
int main (int argc, char **argv)
{
    ...
        switch (choice)
        {
            ...

            case choice_search:                       // extra case entry
                findname ();
```

```
            break;
        }
    ...
}
```

Opgave 53:

```
// aanpassing van de CHOICE_ enum:
typedef enum
{
    choice_add,
    choice_delete,
    choice_list,
    choice_quit,
    choice_search,
} CHOICE_;

// aanpassing van het keys[] array:
    keys [] = "adlqs"

// aanpassing van de functie mainmenu():
CHOICE_ mainmenu ()
{
    ...
    puts ("\n"                                          // aanpassing menu-string
          "(a) Add entry\n"
          "(d) Delete a record\n"
          "(l) List base\n"
          "(s) Search for a name\n"
          "(q) Quit\n"
         );
    ...
}

// Voeg de functie delete() toe voor de functie findname()
int delete(int index)
{
    puts("\nPress 'd' to delete the record,\n"
         "press any other key to keep it ...");
    if (getch() == 'd')
    {
        maxadr--;

        free(address[index].name);
        free(address[index].street);
        free(address[index].city);
        free(address[index].phone);

        memcpy(&address[index], &address[index + 1],
                (maxadr - index) * sizeof (ADDRESS_));

        address = xrealloc(address, maxadr * sizeof(ADDRESS_));

        index--;
    }
    return (index);
}

// aanpassing van de functie findname()
void findname(CHOICE_ function)
{
    ...
        if (strstr(source, target))
        {
            ...
            if (function == choice_delete)              // optioneel:
                index = delete(index);                  // wis record
        }
    }
    ...
```

```
}

// aanpassing van de functie main():
int main (int argc, char **argv)
{
    ...
        switch (choice)
        {
            ...

            case choice_search:                     // dubbele case
            case choice_delete:
                findname (choice);                  // argument voor findname
            break;
        }
    ...
}
```

Opgave 54:

```
// Extra globale variabele (0 by default):
int
    changed;

// aanpassing van de functie delete()
int delete(int index)
{
    ...
    if (getch() == 'd')
    {
        changed = 1;
        ...
    }
}

// aanpassing van de functie addentry()
void addentry ()
{
    changed = 1;
    ...
}
```

Opgave 55:

```
// Plaats functie writebase() voor main()
void writebase(char *name)
{
    int
        index;
    FILE
        *f;

    if (!maxadr)                                    // geen records ?
    {
        unlink(name);                               // verwijder bestand
        return;
    }

    if (!(f = fopen(name, "w")))                    // open om te schrijven
    {
        printf("\nCannot save database '%s'\n",
                name);
        exit(1);                                    // stop indien mislukt.
    }

    // verwerk alle records:
    for (index = 0; index < maxadr; index++)
        fprintf(f,                                  // schrijf naar bestand
            "%s\n"
```

```
            "%s\n"
            "%s\n"
            "%s\n",
                address[index].name,
                address[index].street,
                address[index].city,
                address[index].phone);
    fclose(f);                                          // sluit bestand
}

//aanpassing van main():
int main(int argc, char **argv)
{
    ...
    if (changed)
        writebase(argv[1]);
    return (0);
}
```

Opgave 56:

```
#include <stdio.h>

void main(int argc, char **argv)
{
    long
        last,
        current,
        index;

    sscanf(argv[1], "%ld", &last);
    for (current = 2; current <= last; current++)
    {
        for (index = 2; index < current; index++)
            if (!(current % index))
                break;
        if (index == current)
            printf("prime: %ld\n", current);
    }
}
```

Opgave 57:

```
#include <stdio.h>
#include <malloc.h>

int
    nprimes;
long
    *prime;

void main(int argc, char **argv)
{
    long
        last,
        current;
    int
        index;

    sscanf(argv[1], "%ld", &last);
    for (current = 2; current <= last; current++)
    {
        for (index = 0; index < nprimes; index++)
        {
            if (!(current % prime[index]))
```

```
                break;
            }
            if (index == nprimes)
            {
                printf("prime: %ld\n", current);
                if (!(prime = realloc(prime, ++nprimes * sizeof(long))))
                {
                    puts("\nNot enough memory");
                    return;
                }
                prime[index] = current;
            }
        }
}
```

Opgave 59:

Merk op dat in de implementatie van de macro's om het argument *x* haken zijn geplaatst. Die haken zijn essentieel.

```
#define clearflag(x)    (flags &= ~(1 << (x)))
#define setflag(x)      (flags |=  (1 << (x)))
#define testflag(x)     (flags &   (1 << (x)))
#define toggleflag(x)   (flags ^=  (1 << (x)))
```

Opgave 60:

```
if (! tryout)
    for (i = 2; i < argc; i++)
        if (unlink (argv [i]))
            printf ("%s: warning - cannot delete file %s\n",
                        progname, argv [i]);
```

Opgave 62:

lib movelib +data +process +command

Opgave 63:

l movelib

Opgave 65:

char ****progname;*
char ****argv0;*

Opgave 66:

```
#include <stdio.h>
#include <stdarg.h>
#include <process.h>
#include "help.h"

void error (char *fmt, ...)
{
    va_list
        marker;

    fprintf (stderr, "Fatal error:\n");
    va_start (marker, fmt);
    vfprintf (stderr, fmt, marker);
    fprintf (stderr, "\nAborting.\n");
```

```
    exit (2);
}
```

Opgave 67:

```
#include <malloc.h>
#include <string.h>
#include <stdio.h>
#include "help.h"

void assert (char *name, long pos)
{
    register unsigned
        i;

    name [15] = '\0';

    for (i = 0; i < nsym; i++)
        if (! strcmp (symtab [i].name, name))
            error ("Line %d: subject already defined", lineno);

    if ( (! (symtab = realloc (symtab, (nsym + 1) * sizeof (SYMTAB_))) ) ||
         (! (symtab [nsym].name = strdup (name)) )
       )
        error ("Out of memory.");

    nsym++;
}
```

Opgave 68:

```
#include <stdio.h>
#include <string.h>
#include "help.h"

static LEXER_TOKEN_ process_subject ()
{
    // code omitted in this listing
}

LEXER_TOKEN_ lexer ()
{
    while (1)
    {
        if (! fgets (lexbuf, 200, fdata))
            return (EOF);
        if (strchr (lexbuf, '\n'))
            lineno++;
        if (lexbuf [0] == '[')
            return (process_subject ());
        if (lexbuf [0] != '#')
            return (textline);
    }
}
```

Opgave 69:

```
#include <stdio.h>
#include <string.h>
#include "help.h"

int symcomp (SYMTAB_ *a, SYMTAB_ *b)
{
    return (strcmp (a->name, b->name));
}
```

Opgave 70:

```
#include <stdio.h>
```

```
#include <search.h>
#include "help.h"

unsigned lookup (char *name)
{
    SYMTAB_
        *ret,
        tmp;

    tmp.name = name;
    if (! (ret = bsearch (&tmp, symtab, nsym, sizeof (SYMTAB_), symcomp)) )
        return (-1);
    return (ret - symtab);
}
```

Opgave 72:

In het programma kunnen bijvoorbeeld far data worden gedefinieerd, of far functies worden aangeroepen. De data en functies bevinden zich dan buiten de default-segmenten. De default-segmenten kunnen dan zelf nog steeds 128 Kbytes innemen. In een dergelijk geval is er in feite geen sprake meer van een zuiver small geheugenmodel, maar van een gemengd model.

Opgave 73:

1. *var* is:
   - een near pointer naar
   - een near functie die
   - een far pointer naar
   - een double oplevert.
2. *var* is:
   - een far pointer naar
   - een far functie die
   - een near pointer naar
   - een double oplevert.
3. *var* is:
   - een far array van
   - near pointers naar
   - near functies die
   - far pointers naar
   - characters retourneren
4. *fun* is:
   - een static near functie die
   - en far pointer naar
   - een char retourneert

Opgave 74:

int _huge *(_far *ptr)();

Opgave 75:

Voordat alle bronteksten worden vertaald moet de opdracht

- set mm=C

worden gegeven. Indexbestanden hoeven niet opnieuw te worden aangemaakt: het programma is geschikt gemaakt voor het verwerken van grotere databestanden, niet voor het verwerken van kleinere databestanden, terwijl de eisen waaraan een index- of helpinformatie-bestand moeten voldoen niet zijn veranderd.

Opgave 76:

```
int count_sentence ()
```

```
{
    register int
        ret,
        i;
    unsigned
        local_freq [256];

    memset (local_freq, 0, sizeof (local_freq));
    for (i = 0; i < strlen (sentence); i++)
        local_freq [sentence[i]]++;
    ret = 1;
    for (i = 0; i < strlen (relevant); i++)
    {
        if (local_freq [relevant [i]] != freq [relevant [i]])
        {
            ret = 0;
            freq [relevant [i]] = local_freq [relevant [i]];
        }
    }
    return (ret);
}
```

Opgave 78:

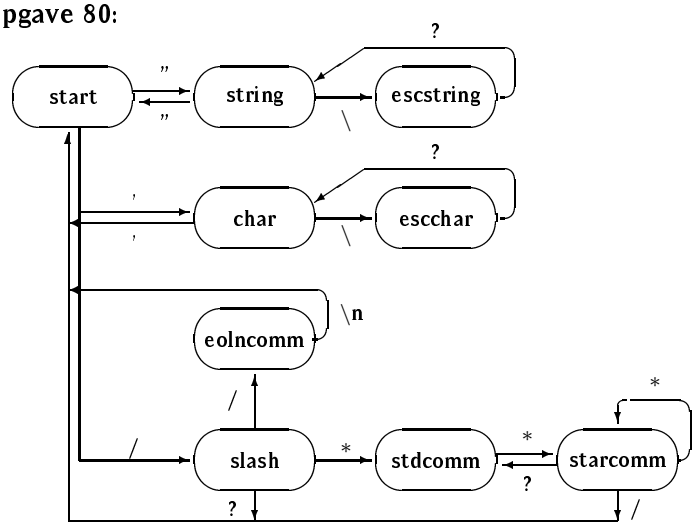
Alleen de uitbreiding op *count_sentence()* is afgebeeld:

```
if (local_freq [relevant [i]] != freq [relevant [i]])
{
    printf ("Frequentie voor '%c' bijgesteld.\n", relevant [i]);
    freq [i] = local_freq [i];
}
```

Opgave 79:

Het programma kan de aantallen niet juist bepalen. De aantallen 'convergeren niet naar een ruststand'.

Opgave 80:



Opgave 83:

Een implementatie van de functie *error()* is:

```
#include <process.h>

void error()
{
    printf("Syntactische fout in de brontekst op regel %d\n", linenr);
    exit(1);
}
```

De definitie van de transitiematrix moet op de volgende punten worden gewijzigd:

```
STATE_
    transition[][] =
    {
        ...
        {                                   // string toestand
            ...
            {string,  error},               // input element: regeleinde
            ...
        },
        {                                   // character toestand
            ...
            {character, error},             // input element: regeleinde
            ...
        },
        ...
    };
```

Opgave 84:

```
#include <stdio.h>
#include <malloc.h>
#include <process.h>

void nomem()
{
    puts("Out of memory"):
    exit(1);
}

initialize_board()
{
    if (!(board = malloc((max + 4) * sizeof(COORD_ *))))
        nomem();

    for (x = 0; x < max + 4; x++)
    {
        if (!(board[x] = calloc((max + 4), sizeof(COORD_))))
            nomem();
        board[x] += 2;
    }

    board += 2;
    for (x = 0; x < max; x++)
        memset(board[x], -1, max * sizeof(COORD_));
}
```

Opgave 85:

```
void show_trace(int starty, int startx)
{
    register int
        newy,
        newx;
    int
        horse_count = 1;

    // Vermeld de route:
    printf("Startveld:   [%d][%d]\n", starty, startx);

    while ((newx = board[starty][startx].x) != -1)
```

```
    {
        newy = board[starty][startx].y;
        printf("Sprong naar: [%d][%d]\n", newy, newx);
        board[starty][startx].x = horse_count++;
        starty = newy;
        startx = newx;
    }
    board[starty][startx].x = horse_count;
    puts("Alle velden bereikt.\n");

    // Vermeld de sprongnummers:
    printf("''   );
    for (newx = 0; newx < max; newx++)                  // rij met kolomnummers
        printf("%5d", newx);

    for (newy = 0; newy < max; newy++)                  // afdruk per rij
    {
        printf("\n%-3d", newy);                         // rijnummer
        for (newx = 0; newx < max; newx++)              // sprongnummers per kolom
            printf("%5d", board[newy][newx].x);
    }

    putchar('\n');
}
```

Opgave 86:

Een sprongmatrix voor een 5 x 5 bord is:

1	12	3	18	21
4	17	20	13	8
11	2	7	22	19
16	5	24	9	14
25	10	15	6	23

Opgave 87:

```
void quicksort(int lower, int upper)
{
    char
        *change,
        *item;
    int
        left,
        right;

    if (lower >= upper)                                 // array van hoogstens
        return;                                         // 1 element: klaar

    item = gargv[upper];                                // selectie-element

    left = lower;                                       // sorteergrenzen
    right = upper;

    do
    {
        while (strcmp(gargv[left], item) < 0)
            left++;
        while (strcmp(item, gargv[right]) < 0)
            right--;
        if (left <= right)
        {
            change = gargv[left];
            gargv[left] = gargv[right];
            gargv[right] = change;

            left++;
            right--;
```

```
        }
    }
    while (left <= right);

    quicksort(lower, right);
    quicksort(left, upper);
}
```

Opgave 88:

```
char
    target[80];

int binsearch(int lower, int upper)
{
    int
        ret,
        middle;

    if (lower > upper)
        return (-1);

    middle = lower + (upper - lower + 1) / 2;

    ret = strcmp(target, gargv[middle]);
    if (!ret)
        return (middle);

    if (ret < 0)
        return (binsearch(lower, middle - 1));

    return (binsearch(middle + 1, upper));
}

void main(int argc, char **argv)
{
    int
        index;

    // quicsort() gerelateerde statements

    printf("Zoekstring ?" );
    gets(target);

    if ((index = binsearch(0, argc - 1)) != -1)
        printf("%s heeft indexwaarde %d\n", target, index);
    else
        printf("%s is niet gevonden\n", target);
}
```

Opgave 89:

compound-statement:
{ **opt-statements** }
opt-statements:
opt-statements statement ||
ϵ
statement:
if-statement || **while-statement** || **compound-statement**

Opgave 90:

N.B.: Bij onderstaande brontekst wordt bij een deling niet getest op het delen door nul. Dit is met behulp van de functie *error()* eenvoudig te implementeren.

```
double term (void)
{
    double
```

```
        ret;

    ret = factor ();

    while (token == '*' || token == '/')
        if (token == '*')
        {
            lexer ();
            ret *= term ();
        }
        else
        {
            lexer ();
            ret /= term ();
        }

    return (ret);
}
```

Opgave 92:

(a) Deling door nul is een semantische fout; expressie *12 / (3 – 2 – 1)* is syntactisch correct.
(b) Ondervangen van deze fout kan in *term()* gebeuren; hier wordt immers de deling uitgevoerd.
(c) De functie *term()* wordt:

```
double term (void)
{
    double
        ret,
        divisor;

    ret = factor ();

    while (token == '*' || token == '/')
        if (token == '*')
        {
            lexer ();
            ret *= term ();
        }
        else
        {
            lexer ();
            divisor = term ();
            if (! divisor)
                error ("Division by zero.");
            ret /= divisor;
        }

    return (ret);
}
```

Opgave 93:

```
double unit (void)
{
    double
        val;

    switch (token)
    {
        case NUM:
            val = value;
            lexer ();
            break;

        case '(':
            lexer ();
            val = expr ();
            if (token != ')')
                error ("Closing ')' missing.");
```

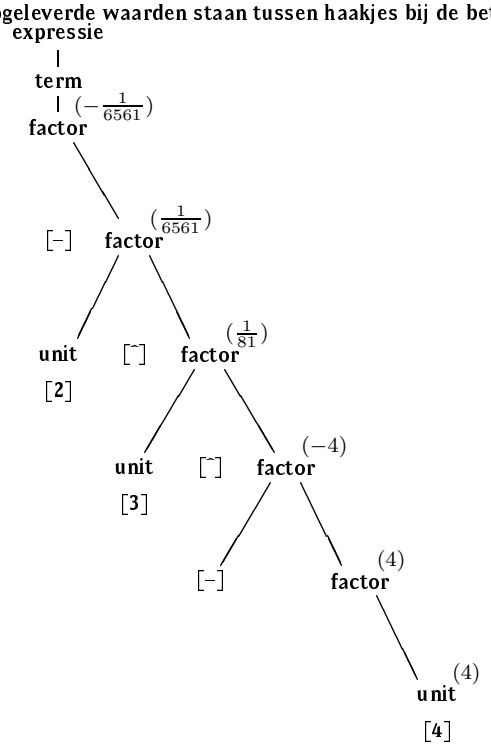
```
            lexer ();
            break;

        default:
            error ("Number, -, +, or ) expected.");
    }

    return (val);
}
```

Opgave 94:

factor:
- **– factor** ||
- **+ factor** ||
- **unit ¯ factor** ||
- **unit**

Opgave 95:

In onderstaande figuur is de ontleedboom weergegeven. De verwerkte invoer staat tussen vierkante haken; de opgeleverde waarden staan tussen haakjes bij de betreffende regels.



Opgave 96:

```
double factor (void)
{
    double
        val;
```

```
    switch (token)
    {
        case '-':
            lexer ();
            val = -factor ();
            break;

        case '+':
            lexer ();
            val = factor ();
            break;

        default:
            val = unit ();
            if (token == '^')
            {
                lexer ();
                val = pow (val, factor ());
            }
    }

    return (val);
}
```

Opgave 98:

De functie moet worden aangeroepen vóór het *for*-statement. Bij voorkeur wordt *checkargs()* aangeroepen als eerste statement. Een implementatie van de functie *checkargs()* is hieronder gegeven.

```
void checkargs(int argc, char **argv)
{
    int
        value,
        index;

    for (value = 1; value < argc; value++)
    {
        for (index = 1; index < argc; index++)
        {
            if (value == atoi(argv[index]))
                break;
        }
        if (index == argc)
        {
            printf ("Nummer %d" , value);
            error ("ontbreekt.");
        }
    }
}
```

Opgave 99:

```
#include <stdio.h>
#include <string.h>
#include <process.h>

static char
    *queue[MAX_QUEUE_SIZE];
static int
    head,
    tail;

int read_queue(char **target)
{
    if (head == tail)
        return (1);
    *target = queue[head];
```

```
    if (++head == MAX_QUEUE_SIZE)
        head = 0;
    return (0);
}

int write_queue(char *value)
{
    int
        next_tail;

    next_tail = (tail + 1) % MAX_QUEUE_SIZE;
    if (next_tail == head)
        return (1);
    if (!(queue[tail] = strdup(value)))
    {
        puts("Out of memory");
        exit (1);
    }
    tail = next_tail;
    return (0);
}
```

Opgave 101:

```
typedef struct
{
    int
        x, y;
} INFO_;

typedef struct DLIST_
{
    struct DLIST_
        *next,
        *previous;

    INFO_
        info;
} DLIST_;

DLIST_
    first =
    {
        &first,
        &first,
    };

DLIST_ *add_dlist(DLIST_ *current)
{
    if (new = malloc(sizeof(DLIST_)))
    {
        new->next = current->next;
        new->previous = current;
        current->next = new;
        new->next->previous = new;
    }
    return (new);
}

void delete_dlist(DLIST_ *current)
{
    if (current->next == current->previous)
        return;
    current->previous->next = current->next;
    curent->next->previous  = current->previous;
    free(current);
}
```

Opgave 102:

```
int lagged_listsearch(LIST_ **previous, LIST_ *head, char *info)
{
    int
        ret;

    *previous = NULL;
    while (head)
    {
        if ((ret = strcmp(head->info, info)) >= 0)
            return(ret);

        *previous = head;
        head = head->next;
    }
    return (1);
}
```

Opgave 103:

```
void delete_name(LIST_ **link)
{
    printf("'%s' is verwijderd\n", (*link)->info);
    free((*link)->info);
    *link = delete_list(*link);
}
```

Opgave 104:

```
void display_names(LIST_ *head)
{
    for (; head; head = head->next)
        puts(head->info);
}
```

Opgave 105:

De functies *add_list()* en *delete_list()* zijn:

```
LIST_ *add_list(LIST_ *current)
{
    LIST_
        *new;

    if (new = malloc(sizeof(LIST_)))
        new->next     = current;
    return (new);
}

LIST_ *delete_list(LIST_ *current)
{
    LIST_
        *aux;

    if (!current)                                   // Stap 1
        return (NULL);

    aux = current->next;
    free(current);

    return (aux);
}
```

Opgave 106:

```
CHAR_INFO_
    count[257];

count_frequencies()
{
    int
```

```
        c;

    count[256].freq = 1;
    while ((c = fgetc(source)) != EOF)
        count[c].freq++;
}
```

Opgave 107:

```
#include <malloc.h>

void define_node()
{
    register int
        node_index,
        index;

    for (index = 0; index < 257; index++)
        n_nodes += (count[index].freq != 0);

    if (!(node = calloc(n_nodes, sizeof(NODE_))))
        error(out_of_memory);

    for (node_index = 0, index = 0; index < 257; index++)
    {
        if (count[index].freq)
        {
            node[node_index].index = index;
            node[node_index].freq = count[index].freq;
            node_index++;
        }
    }
}
```

Opgave 108:

```
int compare_node(NODE_ *el1, NODE_ *el2)
{
    long
        diff;

    diff = el1->freq - el2->freq;

    if (diff < 0)
        return (-1);
    if (diff > 0)
        return (1);

    return (0);
}
```

Opgave 110:

```
int read_bit()
{
    register int
        ret;

    if (!bitindex)
        bitbuffer = fgetc(source);
    ret = bitbuffer & 0x80;
    bitbuffer <<= 1;
    if (++bitindex == 8)
        bitindex = 0;
    return (ret);
}

int read_byte()
{
```

```
    register int
        byte,
        index;

    byte = 0;
    for (index = 0; index < 8; index++)
    {
        byte >>= 1;
        if (read_bit())
            byte |= 0x80;
    }
    return (byte);
}

NODE_ *read_tree()
{
    register int
        bit;
    register NODE_
        *new;

    if (!(new = makenode()))                        // Tekst: 5.1.1.
        error("Out of memory");

    if (!read_bit())                                // Tekst: 5.1.2.
    {
        if (read_bit())                             // Tekst: 5.1.3.
            new->index = -1;
        else
            new->index = read_byte();               // Tekst: 5.1.4.
        return (new);                               // Tekst: 5.1.5.
    }
    new->left = read_tree();                        // Tekst: 5.1.6.
    new->right = read_tree();                       // Tekst: 5.1.7.
    return (new);                                   // Tekst: 5.1.8.
}

void decompress_source()
{
    register NODE_
        *branch;

    while (1)
    {
        branch = node;                              // Tekst: 5.2.1
        while (branch->left)                        // Tekst: 5.2.2
            branch = read_bit() ?                   // Tekst: 5.2.3
                        branch->left
                    :
                        branch->right;
        if (branch->index == -1)                    // Tekst: 5.2.4
                return;
        fputc(branch->index, dest);                 // Tekst: 5.2.5
    }
}

void decompress()
{
    node = read_tree();                             // Tekst: 5.1
    decompress_source();                            // Tekst: 5.2
}
```

Opgave 111:

```
#include <stdio.h>
#include <string.h>

FILE
    *source;
```

```
void main (int argc, char **argv)
{
    register unsigned
        index;
    HASH_
        element;

    element.freq = 1;

    if (!(source = fopen(argv[1], "r")))
    {
        puts("Missing source file");
        return;
    }

    while (element.key = getword())
    {
        if ((index = hash_lookup(element.key)) != HASH_PRIME)
            hash[index].freq++;
        else if ((index = hash_store(&element)) == HASH_PRIME)
        {
            puts("Full hash table.");
            return;
        }
    }
    printtable ();
}
```

Opgave 112:

```
#include <ctype.h>

char *getword()
{
    char
        buffer[100];
    register int
        index,
        c;

    while (1)
    {
        c = fgetc(source);
        if (c == EOF)
            return (NULL);
        if (isalpha(c))
            break;
    }

    index = 0;
    while (1)
    {
        if (index < 99)
            buffer[index++] = c;
        else
            return (NULL);
        c = fgetc(source);
        if (!isalpha(c))
            break;
    }
    buffer[index] = 0;
    return (strdup(buffer));
}
```

Opgave 113:

```
void printtable ()
{
    register unsigned
```

```
        index;

    for (index = 0; index < HASH_PRIME; index++)
    {
        if (hash[index].key)
            printf ("%5d %s\n", hash[index].freq,
                                hash[index].key);
    }
}
```

Opgave 114:

```
#include <malloc.h>
#include <search.h>

int compare(HASH_ **el1, HASH_ **el2)
{                                                   // vergelijk twee woorden
    return (strcmp((*el1)->key, (*el2)->key));
}

void printtable ()
{
    register unsigned
        v_index,
        index;
    HASH_
        **vector;

    if (!(vector =                                  // alloceer extra vector
                  malloc(n_entries * sizeof(HASH_ *))))
    {
        puts("Out of memory");
        return;
    }

    for (index = 0, v_index = 0; index < HASH_PRIME; index++)
        if (hash[index].key)                        // vul vector
            vector[v_index++] = &hash[index];

    qsort(vector, n_entries, sizeof(HASH_ *),      // sorteer vector
          compare);

    for (v_index = 0; v_index < n_entries; v_index++) // afdruk informatie
        printf ("%5d %s\n", vector[v_index]->freq,
                            vector[v_index]->key);

    free(vector);                                   // vrijgeven van geheugen

}
```

Opgave 115:

lexer.c. Merk op dat bepaalde *flex*-versies altijd hetzelfde bestand, vaak *yylex.c* genaamd, aanmaken.

Opgave 116:

1. Invoer die aan geen van de reguliere expressies voldoet, wordt ge-echood naar *stdout*. Op het beeldscherm verschijnt dus de inhoud van het tekstbestand behalve de woorden "de", "het" of "een".
2. Nee. Bijvoorbeeld, invoer *de waarde* levert twee "de"-s op.

Opgave 117:

```
de              return (1);
het             return (2);
een             return (3);
```

```
[a-zA-Z0-9]+      ;
.|\n              ;
```

Opgave 118:

1. *parser.c* (de code van functie *yyparse()*) en *parser.h* (de header file). Sommige *bison*-implementaties gebruiken één vaststaande bestandsnaam *yytab*, in dat geval ontstaan de bestanden *yytab.c* en *yytab.h*.
2. De nummering is oplopend vanaf een bepaalde waarde.

Opgave 121:

Terminale symbolen zijn hieronder vet gedrukt weergegeven. Alternatieven worden door || gescheiden. De robotgrammatica kan als volgt worden omschreven:

program:
  program line || **line**
line:
  number instruction \n || **\n**
instruction:
  drive operand || **end** || **field number** ||
  goto number || **if-instruction** || **object number** ||
  sleep operand || **stop**
if-instruction:
  if condition then number else-tail
else-tail:
  else number || ϵ
condition:
  operand operator operand
operand:
  distance || **direction** || **random** ||
  number
operator:
  < || > || <= || <= || == || !=
number:
  numeric-value || **north** || **east** || **south** || **west**

Het terminale symbool *numeric-value* is hier niet verder gedefinieerd. Dit symbool bestaat uit een serie van één of meer tekens **0** tot en met **9**.

Opgave 122:

Alleen de delen die in listing 144 zijn weggelaten zijn hier weergegeven:

```
instruction:
        DRIVE
        operand                 {
                                    $$.i_type = i_drive;
                                    $$.o_left = $2.o_left;
                                }
    |
        END                     { $$.i_type = i_end; }
    |
        FIELD
        number                  {
                                    $$.i_type = i_field;
                                    $$.o_left = $2.o_left;
                                }
    |
        GOTO
        number                  {
                                    $$.i_type = i_goto;
                                    $$.o_left = $2.o_left;
```

```
                                        $$.goto_no = $2.o_left.val;
                                    }
    |
        if_instr                    { $$ = $1; }
    |
        LOOK
        operand                     {
                                        $$.i_type = i_look;
                                        $$.o_left = $2.o_left;
                                    }
    |
        OBJECT
        number                      {
                                        $$.i_type = i_object;
                                        $$.o_left = $2.o_left;
                                    }
    |
        RANDOMIZE                   { $$.i_type = i_randomize; }
    |
        SLEEP
        number                      {
                                        $$.i_type = i_sleep;
                                        $$.o_left = $2.o_left;
                                    }
    |
        STOP                        { $$.i_type = i_stop; }
    ;

if_instr:
        IF
        condition
        THEN
        number
        else_tail                   {
                                        $$.i_type = i_if;
                                        $$.o_left = $2.o_left;
                                        $$.o_right = $2.o_right;
                                        $$.op = $2.op;
                                        $$.goto_no = $4.o_left.val;
                                        $$.else_no = $5.else_no;
                                    }
    ;

else_tail:
        ELSE
        number                      { $$.else_no = $2.o_left.val; }
    |
        /* empty */                 { $$.else_no = NO_ELSE; }
    ;

condition:
        operand
        operator
        operand                     {
                                        $$.o_left = $1.o_left;
                                        $$.op = $2.op;
                                        $$.o_right = $3.o_left;
                                    }
    ;

operand:
        DISTANCE                    { $$.o_left.o_type = o_distance; }
    |
        DIRECTION                   { $$.o_left.o_type = o_direction; }
    |
        RANDOM                      { $$.o_left.o_type = o_random; }
    |
        number                      { $$ = $1; }
    ;
```

```
operator:
        SMALLER                 { $$.op = o_smaller; }
    |
        GREATER                 { $$.op = o_greater; }
    |
        SMALLEREQUAL            { $$.op = o_smallerequal; }
    |
        GREATEREQUAL            { $$.op = o_greaterequal; }
    |
        EQUAL                   { $$.op = o_equal; }
    |
        NOTEQUAL                { $$.op = o_notequal; }
    ;
```

Opgave 123:

1. Alleen de < en >; alle andere eindsymbolen bestaan uit meer dan één teken.
2. In <= komt een speciaal *flex*-teken voor; het < teken (zie miniscanners). Door de dubbele aanhalingstekens wordt het < teken opgevat als een letterlijk teken in plaats van als een stuurteken. In *west* komen geen speciale tekens voor; rond *west* mogen dubbele aanhalingstekens staan maar het is niet nodig.
3. Spaties worden door de lexer gelezen en verder genegeerd.

Opgave 124:

```
1     #include "rasic.h"
2
3     int yyerror (char *msg)
4     {
5         error ("%s at line %d, unexpected input '%s'.\n",
6             msg, line_number, yytext);
7     }
```

Opgave 126:

```
#include <search.h>
#include "schedule.h"

static int
    worst,
    nfound,
    worst_index;

static void add(int day, int target, int one, int two, int total)
{
    daystaff[day][target].pers1     = staff[day][one].person;
    daystaff[day][target].pers2     = staff[day][two].person;
    daystaff[day][target].total_val = total;
}

static int combine(int v1, int v2)
{
    return (v1 > v2 ? v1 - v2 : v2 - v1);
}

static void test(int day, int one, int two)
{
    int
        target,
        combined_exp;

    combined_exp = combine(staff[day][one].value, staff[day][two].value);

    if (nfound < NDUOS)
    {
        if (combined_exp < worst)
        {
```

```
            worst = combined_exp;
            worst_index = nfound;
        }
        add(day, nfound++, one, two, combined_exp);
        return;
    }

    if (combined_exp <= worst)
        return;

    add(day, worst_index, one, two, combined_exp);

    worst = MAXINT;
    for (target = 0; target < NDUOS; target++)
        if (daystaff[day][target].total_val < worst)
        {
            worst_index = target;
            worst = daystaff[day][target].total_val;
        }
}

static int compare (DUO_ *a, DUO_ *b)
{
    return (b->total_val - a->total_val);
}

void schedule_day (int day)
{
    register int
        one,
        two;

    worst = MAXINT;
    nfound = 0;

    for (one = 0; one < AVAILABLE - 1; one++)
        for (two = one + 1; two < AVAILABLE; two++)
            test(day, one, two);
    qsort (daystaff[day], NDUOS, sizeof(DUO_), compare);
}
```

Opgave 127:

```
#include <stdio.h>
#include "schedule.h"

void printsug ()
{
    register int
        d,
        duo;

    for (d = 1; d <= NDAYS; d++)                    // vermeld "day xx" voor elke
        printf ("   day %1d"  , d);                 // afzonderlijke dag.
    putchar ('\n');

    for (d = 0; d < NDAYS; d++)                     // een scheidingslijn
        printf ("----------");
    putchar ('\n');

    for (d = 0; d < NDAYS; d++)                     // kolomtitels: de twee
        printf (" pers exp" );                      // persoonsnummers en hun
    putchar ('\n');                                 // gecomb. ervaring

    for (duo = 0; duo < NDUOS; duo++)               // vermeld voor elk paar
    {                                               // de persoonsnummers en
        for (d = 0; d < NDAYS; d++)                 // de gecomb. ervaring.
            printf ("%2d %2d %3d" ,
                daystaff[d][duo].pers1 + 1,
```

```
                daystaff[d][duo].pers2 + 1,
                daystaff[d][duo].total_val);
        putchar ('\n');
    }

    for (d = 0; d < NDAYS; d++)                         // afsluitende lijn
        printf ("----------");
    putchar ('\n');

}
```

Opgave 128:

```
#include "schedule.h"

int nextday(int day)
{
    return (day == NDAYS - 1 ? 0 : day + 1);
}

void solveday (int day)
{
    int
        nextd,
        p1,
        p2;

    while (++dayindex[day] < NDUOS && nsolutons != NSOLS)// Tekst: 1
    {
        p1 = daystaff[day][dayindex[day]].pers1;
        p2 = daystaff[day][dayindex[day]].pers2;

        if (!(occupied[p1] || occupied[p2]))            // Tekst: 2
        {
            occupied[p1]++;                             // Tekst: 3
            occupied[p2]++;

            if ((nextd = nextday(day)) == startday)     // Tekst: 4
                addsol ();
            else                                        // Tekst: 5
                solveday (nextd);

            occupied[p1]--;                             // Tekst: 6
            occupied[p2]--;
        }
    }
    dayindex[day] = -1;                                 // Tekst: 7
}
```

Opgave 129:

```
#include <stdio.h>
#include <memory.h>
#include "schedule.h"

static int
    worst = MAXINT,
    worst_index;

static void add(int target, int total)
{
    memcpy(&solutions[target].dayindex, dayindex, NDAYS * sizeof(int));
    solutions[target].total_val = total;
    nsolutions++;
    total_sol++;
}
```

```
static int combine()
{
    int
        ret = 0,
        index;

    for (index = 0; index < NDAYS; index++)
        ret += daystaff[index][dayindex[index]].total_val;
    return (ret);
}

void addsol()
{
    int
        target,
        combined_exp;

    combined_exp = combine();

    if (total_sol < NSOLS)
    {
        if (combined_exp < worst)
        {
            worst = combined_exp;
            worstindex = total_sol;
        }
        add(total_sol++, combined_exp);
        return;
    }

    if (combined_exp <= worst)
        return;

    add(worstindex, combined_exp);
    total_sol--;

    worst = MAXINT;


    for (target = 0; target < NSOLS; target++)
        if (solutions[target].total_val < worst)
        {
            worstindex = target;
            worst = solutions[target].total_val;
        }
}
```

Opgave 130:

```
#include <stdio.h>
#include "schedule.h"

void showsol ()
{
    register int
        n,                                          // oplossingsindex
        d;                                          // dagindex
    DUO_
        *duop;

    for (n = 0; n < total_sol; n++)                 // vermeld alle oplossingen
    {
        printf("\n\tROOSTER %d: totale ervaring: %5d\n\n",
               n + 1, solution[n].total_val);

        for (d = 0; d < NDAYS; d++)                 // vermeld de personen per dag
        {
            duop = &daystaff[d][solution[n].dayindex[d]];
            printf ("dag %d: %2d %2d %3d\n",
```

```
                d + 1,
                duop->pers1 + 1,
                duop->pers2 + 1,
                duop->total_val);                    // inclusief gecomb. ervaring
        }
    }
}
```

Bijlage G

Literatuurverwijzingen

Aho, A.V., Sethi, R., Ullman, J.D., (1986) *Compilers; Principles, Techniques and Tools*. Addison-Wesley Publishing Company, ISBN 0-201-10194-7

Aho, A.V., Hopcroft, J.E., Ullman, J.D., (1987) *Data Structures and Algorithms*. Addison-Wesley Publishing Company, ISBN 0-201-00023-7

Bishir, J.W., Drewes, D.W., (1970) *Mathematics in the behavioral and social sciences*. Harcourt, Brace & World, ISBN 0-15-555251-1

Brokken, F.B., Jedema, J., Kubat, K., (1990) *The ΩCΩ Editor and Support Programs*. ICCE Technical Report No. 19. Universiteit van Groningen

Dahl, O.-J., Dijkstra, E.W., Hoare, C.A.R., (1972) *Structured Programming*. Academic Press, ISBN 0-12-200550-3

The Editors of Dr. Dobb's Journal, (1986) *Dr. Dobb's Toolbook of C*. Prentice Hall, ISBN 0-89303-615-3

Geus, A.F. de, Jongejan, J.H., Koelmans, A.M., (1985) *Adventure Description Language*. Sigma Press, ISBN 1-85058-011-1

Hoare, C.A.R., (1962) "Quicksort", *Comp. Journal* 5, *no. 1,* 10-15.

Hofstadter, D.R., (1985) *Metamagical Themas*. Penguin Books Ltd. ISBN 0-14-008534-3

Huffman, D.A., (1952) "A method for the construction of minimum redundancy codes", *Proc. IRE* 40, pp. 1098-1101.

IBM, (1983) *Personal Computer Technical Reference Manual*. IBM, publicatie nummer 6936844

Jensen, K., Wirth, N., (1985) *PASCAL User Manual and Report*. Springer Verlag, ISBN 0-387-96048-1

Johnson, S.C., (1975) *Yacc – Yet another compiler compiler*. Computing Science Technical Report 32, AT&T Bell Laboratories, Murray Hill, N. J.

Kernighan, B.W., Ritchie, D.M., (1988) *The C Programming Language (second edition).* Prentice Hall, ISBN 0-13-110370-9

Kidder, J.T., (1983) *The soul of a new machine.* Penguin books Ltd.

Knuth, D.E., (1979) *The Art of Computer Programming, Vol. 1: Fundamental Algorithms.* Addison-Wesley, 5^e druk, LIBRARY OF CONGRES CARD NO. 73-1830

Knuth, D.E., (1973) *The Art of Computer Programming, Vol. 3: Sorting and Searching.* Addison-Wesley,ISBN 0-201-03803-X

Knuth, D.E., (1988) "The toilet paper problem", *Amer. Math. Monthly* 91, *no. 8,* 465-470.

Kruyskamp, C., (1976) *Van Dale Groot Woordenboek der Nederlandse Taal.* Martinus Nijhof, ISBN 90-247-1830-9

Kubat, K., Brokken, F.B., (1989) *The OCOSGRAB Environment.* ICCE Technical Report No. 9. Universiteit van Groningen

Kubat, K., Brokken, F.B., (1990) *The COW Environment.* ICCE Technical Report No. 21. Universiteit van Groningen

Microsoft Press, General editor Ray Duncan, (1988) *The MS-DOS Encyclopedia.* Microsoft Press, Redmond Washington, ISBN 1-55615-049-0

Minsky, M.L., (1972) *Computation: Finite and infinite macines.* Prentice Hall, London, ISBN 0-13-165449-7

Murray, J.D., (1989) *Mathematical Biology.* Springer Verlag, ISBN 3-540-19460-6

Press, W.H., Flannery, B.P., Teukolsky, S.A., Vetterling, W.T., (1988, 1989) *Numerical Recipes in C.* Cambridge University Press, ISBN 0-521-35465-X

Rollins, D., (1985) *IBM PC 8088 Macro Assembler Programming.* Macmillan Publishing Company, ISBN 0-02-403210-7

Selby, S., (1972) *Standard Mathematical Tables.* The Chemical Rubber Company Press, LIBRARY OF CONGRES CARD NO. 30-4052

Tannenbaum, A.S., (1987) *Operating Systems: Design and Implementation.* Prentice-Hall ISBN 0-13-637331-3

Tremblay, J.P, Sorenson, P.G., (1985) *Compiler Writing.* McGraw-Hill, ISBN 0-07-065161-2

Tolkien, J.R.R., (1976) *The Lord of the Rings.* Unwin Books, London. ISBN 0-04-823112-6

Toonder, M., (1975) *En daar houd ik mij aan.* De Bezige Bij, Amsterdam, ISBN 902340503x

Wirth, N., (1976) *Algorithms + Data Structures = Programs.* Prentice-Hall, ISBN 0-13-022418-9

Index

! 59
    omkering van samengestelde expressie 72
!= 63
'' 35
    bij #include 120, 224
$$ 436
$
    bij reguliere expressies 427
%%
    bij flex 425
% 62
    in bison-specificatie 434
&& 64
    volgorde van evaluatie 70
& 59, 63, 150
    en scanf() 133
(59
    bij reguliere expressie 428
    in #define 123
) 59
    bij reguliere expressie 428
* 59, 62, 150
    bij library manager 227
    bij reguliere expressie 428
*/ 19
++ 59
+= 64
+ 62
    bij library manager 227
    bij reguliere expressie 428
, 67
    bij for-statement 83
    in #define 122
- 59, 62
    bij library manager 226
    bij pointers 156
    bij reguliere expressie 428
-- 60
-> 187
. 187
    bij reguliere expressie 427
/* 19
/ 62
    bij reguliere expressie 429
// 19
:
    bij bison 435
;
    bij bison 435
< 63
    bij DOS redirectie 137
    bij #include 120
    bij miniscanner 432
<< 62
<= 63
= 64
== 63
> 63
    bij DOS redirectie 137, 235
    bij #include 120
    bij miniscanner 432
>= 63
>> 63
?
    bij reguliere expressie 428
?: 66, 77
[59
    bij reguliere expressie 428
\
    in #define 123
    in escape-sequence 515
] 59
    bij reguliere expressie 428
^ 63
    bij reguliere expressie 428
    bij reguliere expressies 428
^= 64
| 63
    bij bison 435
    bij open() 142
    bij reguliere expressie 428
||
    volgorde van evaluatie 70
~ 59

'\0' 70
0x 21, 134

$1 436

64Kb 260

/A 265
aaargh 428
aangeroepen functie 87
aanroep van een functie 93
aanroepende functie 87
access modifier 42
accolade 20, 34
acties
    bij lexical analyzer 430
    bij parser 353, 436
adres 47, 150
    berekening
        forceren 269
    in een segment 260
    nomenclatuur 264
    ssss:oooo notatie 264
    van een functie 166
    van een segment 260
/AH 268
Aho 92, 297, 346, 397, 411, 422
/AL 267
algoritme 89, 211
    iteratief proces 280
    om te hashen 411
    prey-predator model 281
allocatie 157
    van geheugen 87
    bij onvoldoende geheugen 158
    deallocatie 158
    dynamisch 158
    op de stack 47
    reallocatie 159
    van structs 193
    van unions 193
    vereiste omvang 159
/AM 266
annuïteitsberekening 280
ANSI-C 18
    commentaar 19
    en standaard-C 18, 519
apostrof 516
argc 102
argument
    array als argument 97
    by value 96
    counter (argc) 103
    van #define 122
    van een programma (argv) 103
    variabel aantal 100
argv 102
Aristothenes 212
array 52
    als argument/parameter 97
    als pointer-parameter 153
    argumenten op de stack 521
    definitie 53
    dimensie 52
    element 52
    huge 268
    index 52
    initialisatie van 54
    maximale grootte 218
    meerdimensionaal 52
        indiceren 53
    sizeof 60
    van pointers 162
/AS 266
ascii 23, 41
    hoge ascii-tekens bij flex 476
    waarde 51
        als terminaal symbool 439
    -z 52
.asm 28
assembler 28
    code 27
assignment 58
    aan pointervariabelen 150
    informatieverlies 68
    operatoren 64
    type conversie 68
associativiteit van operatoren 58, 516
    bison 441
asterix 62, 103, 150, 179, 186, 227, 428
atoi() 117
auto 46
autoexec.bat 31

B 16
backtracking 317
basic 16, 444
.bat 31, 32
BCPL 16
beëindigen van een programma 87
Been
    rangeermeester 363
begintoestand 293
bereik
    van types 41, 42
bestand
    decompressie 408
binair
    boom 320
    operatoren 58, 62
    stream modus 140, 238
    talstelsel 22
    zoeken 254, 333

Bishir 72
bison 30, 422, 434
    context-gevoelige foutmelding 473
    error regel 474
    foutmeldingen 442
    grammatica van expressie 440
    herstel na foutconditie 474
    speciale kenmerken 472
bit 23, 210
    bit-velden bij structs 188
    bitwise operatoren 63
    masker 216
    meest significante bit 24
    omzetten via ^= 64
    sign bit 24
    teken-bit 24
    testen van een bit 215
    vlag 214, 456
    wissen 217
    zetten 217
blad
    van een boom 391
blok 20, 45
    geheugenblok 157
    nesting 20
    openen en sluiten 34
Bommel, O.B. 22
boolean 70
boom 390
    bewerkingen bij bomen 391
    binair 320, 391
    de lege boom 393
    knooppunt 391
    wortel en blad 391
boormachine 481
booting 16, 172
bordenbak 47
botsing
    bij het hashen 411
boundary
    checking 16
    error 174
break 85
    uit een switch 79
    uit een while 82
    uit eeuwige lus 85
Brokken 267
bsearch() 254
buffer
    circulair 370
buffered I/O 128
busy waiting 370
byte 23

C 26, 32, 111, 132
    ANSI-C en standaard-C 18, 519
    filosofie 16
    geschiedenis 16
    installatie van vertaler 31
    layout van brontekst 18, 525
    opdracht 32
    operator 516
    preprocessor 26
    syntaxis 509
    vertaler 26
    vertaler en functiebibliotheek 17
%c 69
call by value 152
calloc() 218
cannabis 410
case 79
    doorvallen 79
case sensitive 40
c.bat 32
cc opdracht 32
cc.bat 32
ccount 106
char 41
    argumenten op de stack 521
character
    constante 21, 55
        escape-sequence 21
    printable 110
chip
    Intel 80x86 serie 260
    software-chip 466
circulaire buffer 370
_clearscreen() 463
clock() 117
close() 142
clustering
    bij hashen 414
code 47
    analyzer 30
    default-segment 261
    en data 19
    far 261
    frame 89
    generatoren 30, 422
    near 261
    segment 47, 261
    spaghetti 89
    splitsen 224
    vervangen door data 216
codeview 35
    en #defines 122
collisie 411, 413
combat 131
combinatie 485
    combinatorische explosie 485

commentaar 19
  end-of-line commentaar 19
  in flex specificatie 425
  standaardcommentaar 19
compact geheugenmodel 267
compile-time evaluation 51
  en `sizeof` resultaat 60
compound
  assignment operatoren 64
  statement 20
compressie
  van data 397
condities 70, 74
conflict in grammatica 343
  keuze 344, 442
  onvoltooide regel 343, 443
`const` 42
constante 20
  character constante 21, 55
  decimale notatie 21
  escape-sequence 21
  hexadecimale notatie 21
  `long` 42
  notatie 21
  numerieke constante 21
  `sizeof` resultaat 60
  string constante 21
`continue` 86
  bij het `for`-statement 87
controle variabele 77, 79
CS 261
cv opdracht 35

`%d` 61, 132, 135
/D optie 124
  van bison 438
  van flex 433
dambord 323
dangling `else` 76
data 47
  compressie 397
  drempel 263
  far 262
  hiding 154, 362, 372
  in plaats van code 216
  near 262
  nieuwe datatypes 184
  recursieve definitie 187
  samengestelde datatypes 184
  segment 47, 261, 262
  self referential 377
  structuur
    recursief 316
  threshold 263
database 197
  record 197
  van help faciliteit 236
datamaskine 19
dbase 197
De Morgan 72
deallocatie 158
debugger 29
dechex 133
decimale notatie 21
  conversie naar hexadecimaal 133
declaratie
  bij standaard C 519
  en definitie 44, 50
  van functies 97
    bij standaard-C 99
  van pointers 150
decompressie 408
decrement 60
default
  actie van `yylex()` 430
  codesegment 261
  datatype 43
  library search 227
    `#pragma` 228
  plaatsing in de `switch` 80
  transitie 295
  type van functies 93
`#define` 121
  ABS 122
  expansie 121
  leeg 260
  met argumenten 122
definitie
  array 53
  dichtstbijzijnde 46
  en declaratie 44, 50
    bij header files 120
  `struct`
    genest 190
  `union`
    genest 190
  van een functie 92
  van pointers 150
dekhengst 133
deterministische eindige automaat 297
dichtsbijzijnde definitie 46
Dijkstra 92, 106
dimensie 52
doorvallen 79
`double` 41
double linked list 384
double word 23
`do-while` 83
Drewes 72
Droste 314

DS 262
dummy statement 79
dword 23

/E 310
/E optie 125
editor 30, 267
eeuwige lus 82
eindige toestandsautomaat 292
eindsymbool 337
else 74
    dangling 76
emmer 93
#endif 124
entry point
    in een switch 79
enum 194
    en #defines 195
    initialisatie 195
    komma achter de enumerators 195
environment
    envp 103
EOF 82
error 474
errorlevel 87
escape sequence 55
escape-sequence 21, 515
Esther 314
evil 106
.exe 28
    afkappen van argv[0] 230
exit()
    en errorlevel 87
    functie 87
    status 87
expansie van #define 121
%expect 443
expliciete initialisatie 51
explosie
    combinatorisch 485
expressie 58
    bij for-statement 83
    bison-grammatica 440
    grammatica 357
    kraker 345
    voorbeeld van ontleding 349
extended memory 293
extensie
    .asm 28
    .bat 32
    .c 26, 32
    .exe 28
    .h 31, 34
    .lib 31, 226
    .obj 28, 226
extern 50

faculteit 317
falling through 79
false 70
_far 260, 269
    call 261
    code 261
    data 262
F.B.B. 5
fclose() 138
Febojan 5
feof() 129
fgetc() 129
fgets() 129
    regelgewijs lezen 145
field
    pointer 187
    selector 187
fifo 369
FILE 137, 138
    openen 137
    sluiten 138
filo 361
filosofie achter C 16
First In, First Out 369
First In, Last Out 361
flex 30, 422
    beschrijving 529
    en hoge ascii-tekens 476
    fout in programma 476
    miniscanner 430, 432
    opdracht 433
    reguliere expressies 427
    speciale kenmerken 472
    specificatie 424
    verwerking van meerdere bestanden 472
    yytext 426
float 41
floating point 41
flow control 74
foo_bar() 85
fopen() 138
    binaire en tekst modus 140
    falen 139
for 83
    en continue 87
    verschillen met pascal 83
format string 61, 111, 131, 132, 135, 264
formated I/O 131
    functies 131
formule
    aantal ints bij bits 215
    bit-index bij ints 215

combinaties van k uit N 485
Data Structures + Algorithms 360
element-index in een geheugen-element 215
geheugen bij data-elementen 215
geheugen-index bij data-elementen 215
int-index bij bits 215
lineair adres en segment:offset 264
fout
bij bison 442
yyerror() 435
boundary error 174
context-gevoelige fout bij bison 473
herstel na foutconditie in parser 474
in flex-programma 476
memory control blocks destroyed 174
parse error 456
wild pointer 172
%Fp 264
fprintf() 131
fputc() 129
fputs() 129
fread() 153
free() 157
fscanf() 131
fseek() 144, 253
fstat() 242
ftell() 144
fucked up 85
functie 92
aangeroepen 87
aanroep 93
via pointervariabele 167
via transitiematrix 303
aanroepende 87
als tabel 216
declaratie 98
definiëren 92
die een pointer retourneert 154
en geheugenmodellen 265
en gemengd model programma's 273
extern 97, 99
far 273
far parameters bij small/medium modellen 271
functienaam 93
hashpjw() 412
in expressies 155
interface 274, 362, 372
main() 102
met variabel aantal argumenten 100
near 273
parameter 93, 95
pointer naar functies 166
prototype 97
bij standaard-C 99
recursief 314
indirect 316
return-waarde 93, 99, 275
type 99
zonder return-waarde 93

_GCLEARSCREEN 463
geheugen
allocatie 87, 157
extended 293
lineair 264
mal 189
masker 189
model 260, 262, 263
compact 267
en functie-ontwikkeling 265
gemengd 268
huge 268
large 267
medium 266
small 266
reallocatie 159
segment 260
vrijgeven 158
vrijgeven in een lijst 382
geheugenmodel 265
gemengd geheugenmodel 268
gereedschap 92
gereserveerde naam 40, 515
geschiedenis van C 16
gestructureerd programmeren 92
getallenlijn 212
getchar() 129
getche() 374
gets() 97, 129
Geus, De 397
globale variabele 46
goto 88
grammatica 336
alternatieven 339
bij bison-specificatie 435
bison-specificatie 435
conflict 343
reduce-reduce 442
shift-reduce 443
factor 348
infix 394
LALR-1 338
machtsverheffing in expressie 357
nesting 342
ontleding 346
postfix 393
prefix 409
serie van één of meer 339

taal 336
term 347
terminal symbol 337
grondtal 22
Groningen 363
/Gt 263

.h 31, 34
handen en voeten 293
handle 142
hangende computer 158
door flex 476
hash 410
add the hash 414
algoritme 411
clustering van indexwaarden 414
en collisies 413
functie 412
met botsingen 411
rehashing 413
sorteren van de hash-tabel 419
waarde 411
head
van een queue 370
header file 31, 120
Hello World 34
help faciliteit 235
herformuleren
van problemen 106
heuristiek 480
hexadecimaal
conversie naar decimaal 133
notatie 21
talstelsel 22
hexdump 237
Hoare 331
Hofstadter 285
hoger-lager 254
Hopcroft 92, 397
horses 325
Huffman 397, 401
_huge 269
huge geheugenmodel 268

–i optie van flex 433
ICCE 267
identifier 40
constante 40
reguliere expressie 428
if
genest 75
if-else ladder 76
statement 74
#ifdef 124
#ifndef 125
impliciete initialisatie 50
#include 34, 120
en #ifndef 125
van lokale header file 224
INCLUDE systeemvariabele 31, 120
increment 59
bij for-statement 83
index 52
bij meerdimensionaal array 53
negatief 52
organisatie van indexbestand 237
index bestand 236
indirection 161
infix 394
ingang in een switch 79
initialisatie
van structs 192
van unions 193
van arrays 54
van een dubbele lijst 385
van een enum 195
van een lijst 379
van for-statement 83
van variabelen 50
inorde 393
input en output 128
installatie van C vertaler 31
int 41
Intel 80x86 chips 260
interface
functie 274, 362, 372
intermediate codegenerator 26
interrupt 370
hardware 370
invoer
bij een eindige toestandsautomaat 293
I/O 128
buffered, unbuffered 128
formated en unformated 131
unbuffered 141
iteratief proces 280
itoa() 288

/J optie 41
Jensen 360
Johnson 422
Jongejan 397

kbhit() 114, 370
Kernighan 3, 16, 18, 19, 34, 80, 207, 509
keuzemenu 373
key 410
keyword 40
_far 269
_huge 269

_near 269
uitschakelen 260
verwerking door lexical analyzer 249
keywords 515
Kidder 295
K.K. 5
klontering
bij hashen 414
KNMI 480
Knuth 106, 360
Koelman 397
koffiezetter 293
komma
bij for-statement 83
in #define 122
kop
van een queue 370
Koster 5
Kruyskamp 3
Kubat 274, 292

L
bij int constanten 42
suffix 42
l opdracht 33
label 88
labyrint 317
LALR-1 grammatica 338
bison 434
Last In, First Out 361
latest possible binding 106
layout van C brontekst 18, 525
l.bat 33
lc opdracht 33
lc.bat 33
leaf 391
leeg statement 79
%left 441
left shift 62
lege verzameling 319
lengte van een string 53
levensduur
bij recursieve functies 315
levensduur van variabelen 45
lexical analyzer 249, 336, 353
yytext 426
lfind() 169, 170
.lib 31, 226
LIB systeemvariabele 31
library manager 30, 226
bewerkingen 226
wildcard expansie 228
lifo 361
lijst 377
dubbele lijst 384
initialisatie 379
vorige element 383
linked list
double 384
linker 28, 226, 227
en bibliotheken 227
/st: optie 263
lint 30
lisp 317
literatuur verwijzingen 573
logisch
expressie 69
onwaar 70
operator 64, 69
volgorde van evaluatie 70
wetten van De Morgan 72
waar 70
lokale variabele 45
long 42
L-suffix 42
lust
rum 3
lvalue 60

machine-afhankelijke code-omzetter 27
machinewoord 23
main() 34, 88, 102
en yyparse() 424
parameters 102
return-waarde 102
make 30
malloc() 87, 157
masker 216
medium geheugenmodel 266
meerdimensionaal 52
meest negatieve waarde 25
meest significant 22
bit 24
byte, verlies van 69
cijfer 22
nibble 23
memcpy() 207
memset() 109
menu 373
miniscanner 430
implementatie 432
Minsky 292
MM 266
en compact geheugenmodel 267
en huge geheugenmodel 268
en large geheugenmodel 267
en medium geheugenmodel 267
en small geheugenmodel 266
MM=C 267
MM=H 268

MM=L 267
MM=M 267
MM=S 266
systeemvariabele 32
modifier
access 42
const 42
volatile 42
type 42
short 42
module 224
modulo 62
movelib 228
multi
tasking 369
Murray 284

\n 35, 55
en lexical analyzer 430
naam
gereserveerd 40, 515
van codesegment 274
van een functie 93
van parameters bij functieprototypes 98
_near 260, 269
bij data 262
bij functies 273
call 261
code 261
data 262
negatieve
getallen 24
index 52
nesting
van if-statements 75
newline 55
node 391
/noe 105
Noordpool 480
%Np 264
/Nt 275
NULL 70
argument bij realloc() 159
NULL-pointer 172
pointer assignment 158, 172
numerieke constante 21

.obj 28, 226
object code 28
ocosgrab 292
octaal stelsel 22
oe! 267
offset 264
bij %p uitvoer 264
Olivier B. Bommel 22
oma
zie opa 173
Omgekeerde Poolse Notatie 393
one's complement 24
operator 63
ontleding
boom 346
ontwikkelomgeving 26
Oostum 363
opa 171
eet een boterham 336
en oma 173, 249
in een reguliere expressie 427
wordt gegeten door fiets 338
opdracht
c 32
cc 32
cv 35
l 33
lc 33
open() 142, 147
open accolade 20, 34
operand 58
operator
adres-van 59, 150, 151
aftrekking 62
array-index 59
assignment 64
binair en unair 58
associativiteit 58, 346, 516
binair 62
bitwise 210
and, or, xor 63
compound assignment 64
decrement 60
deling 62
field pointer ('->') 187
field selector ('.') 187
(geneste) ternaire operator 66
groter dan (of gelijk aan) 63
haakjes 59
increment 59
inhoud-van 59, 150
kleiner dan (of gelijk aan) 63
left shift 62
logical and, or 64
logical not 59
min 59
modulo 62
one's complement 63
(on)gelijk aan 63
optelling 62
prioriteit 58, 516
right shift 63

sequential evaluation 67
sizeof 60
ternair 77
bij ABS #define 122
ternaire operator 66
unair 58
vermenigvuldiging 62
volgorde van evaluatie 70
optie
/A 265
/AC 267
/AH 268
/AL 267
/AM 266
/AS 266
/D 124
bij bison 438
bij flex 433
/E 125, 310
/Gt 263
–i bij flex 433
/Nt 275
/st: 263
verwerking 460
/Zs 310
optimalisatie
prematuur 106
optioneel
else 74, 124
organisatie van software 31

%p 264
paardesprong 322
parameter 95
array als parameter 97
lijst 93
van een functie 93
van main() 102
parse error 456
context-gevoelige foutmelding 473
parsing tree 346
PATH systeemvariabele 31
PeeWee 5
Plomp 5
B.V. 363
pointer 150
naar pointers 161
als array 157
als array-element 162
arithmetic
bij bsearch() 255
declaratie 150
definitie 150
far
bij small en medium modellen 271
fouten bij pointers 171
geretourneerd door een functie 154
in gemengd geheugenmodel programma's 270
naar een struct 186
naar functies 166
niet geïnitialiseerde pointers 172
NULL-pointer assignment 158
NULL-pointer bij realloc() 159
NULL-pointer 172
pointer arithmetic 155
rekenen met pointers 155
self referential 377
swappen 165
variabele
waarde 150
verschil van pointers 156
void * 158
voorbij gealloceerd geheugen 172
wijzend op een variabele 156
wild pointers 172
portabiliteit 159
postfix 59, 393
postorde 393
#pragma 228, 230, 460
%prec 442
prefix 59, 409
preorde 392
preprocessor 26, 120
directive 120
syntaxis 514
Press 280
prey-predator model 281
priemgetal 210
printable character 110
printf() 35, 131
%c 111
%c 69
%d 61, 135
%Fp 264
%Np 264
%p 264
%x 135
prioriteit van operatoren 58, 516
grammatica 345
bison 441
producent-consument proces 369
profiler 30
programma
adressenbestand 197
adventure 295
beëindigen 87
bison 422
ccount 106
database 197

dechex 133
einde van een programma 102
flex 422
opdracht 433
gemengd model 269
functie 273
Hello World 34
help faciliteit 235
hexdump 237
horses 325
huffman 397
interactief 198, 480
met argumenten 103
movelib 228
ontwikkeling 106, 198
portabiliteit 159
rangeer 365
rasic 443
scalc 345, 353
schedule 490
simdiff 144
start van een programma 102
subset 319
toilet 112
Variable Definer (vd) 174
vd 175
versienummer van 240
Zeef van Aristothenes 218
programmeren
gestructureerd 92
project manager 30
prolog 317
prototype 97
bij indirecte recursie 316
bij standaard-C 99
pseudo-code 89
pseudo-random 467
putchar() 129
puts() 129
en emmer 93

qsort() 169, 170, 252
queue 369
head 370
tail 370
quicksort 332

rand() 115, 467
random generator 467
rangeer 365
rasic 443
instructies 444
read() 142
realloc() 159, 247
als free() 160
als malloc() 159
reallocatie van geheugen 159
record
van een database 197
recursie 314, 585
bij een struct 187
grammaticale regel 339
in datastructuren 316
indirect 316
reduce-reduce conflict 442
register 49
segment register 261
snelheid 49
reguliere expressie
paren met invoer 429
vangnet-expressie 430
reguliere expressies bij flex 427
Reitdiep 363
rekenen met pointers 155
rekenmachine 345
relocatable object code 28
return 87
adres 314
bij main() 88, 102
return-waarde van functies 93
type van functies 99
waarde 275
return-waarde 87
Reversed Polish Notation 393
%right 441
right shift 63
Ritchie 3, 16, 18, 19, 34, 80
run-time
geopende streams 137
run-time evaluation 51

%s 132
Sahara 480
scalc 345, 353
scanf() 131, 153, 353
en & 133
schaakbord 323
schedule 490
segment
bij %p uitvoer 264
code 261
naam 274
codesegment 261
data 262
datasegment 261
default 261
override 269
register 261
stack 263
stacksegment 261

segmentering van geheugen 260
Selby 210, 485
self referential data 187, 377
semantische waarde 249
sentinel 398
sequential evaluation 67
set 103
setargv.obj 105
    en l.bat 105
Sethi 346, 412, 422
_settextposition() 464
shift-reduce conflict 443
short 42
side effect 71
sign bit 24
signed 42
simdiff 144
simulatie 112
sizeof 60
    array 60
    string 61
sluit accolade 20, 34
small geheugenmodel 266, 267
Smit Duyzentkunst 112
Smith's 260, 466
software-chip 466
sorteren 169
    bij een hash-tabel 419
    qsort() voorbeeld 252
spaghetti 89
spatie
    en layout regels 526
    in #define 123
specificatie
    voor flex 424
spieken in de invoer 353
sprintf() 131
srand() 117, 467
SS 263
sscanf() 131, 168
/st: 263
staart
    van een queue 370
stack
    bewerking via de vertaler 361
    bewerkingen 361
    en array argumenten 521
    en char argumenten 521
    en default datasegment 263
    groei van de stack 363
    grootte 263
    overflow 315, 361
    pointer 362
    segment 261, 263
    systeem 361
    underflow 361
    van yyparse() 437
standaard invoerstroom 82
statement 20
    break 85
    continue 86
    do-while 83
    dummy 79
    for 83
    goto 88
    if 74
        genest 75
    leeg 79
    return 87
    switch 79
    while 81
static 48
st_atime 242
stderr 137
stdin 82, 137
stdio.h 34
    en FILE 138
stdout 137
stdrdup() 176
storage class
    auto 46
    extern 50
    register 49
    static 48
strcat() 233
strcmp() 165
strcpy() 163, 233
strdup() 247
stream 128
    binaire en tekst modus 140, 238
    falen van openen 139
    openen 137
    positioneren 143
    terugduwen van een teken 353
    vs. handle 142
    yyin 426, 472
stricmp() 169
string 35, 52
    concatenatie 136
    constante 21
    lengte 53
    parse error 456
    sizeof 61
strlen() 180, 233
strlwr() 207
strstr() 207
struct 185
    allocatie 193
    bit-velden 188
    definitie 185, 186

via typedef 186
genest 190
naamloos 395
initialisatie 192
operatoren 187
pointer naar een struct 186
self-referential 378
stat 242
tag 187, 378
veld 185
structuren
recursief 377
subset 319
suffix
_text 275
Sulkers 366
swappen
van pointers 165
switch 79
een bekende fout 80
ingang 79
symbolische debugger 29
symbool
naam 40
gereserveerd 40
syntactische waarde 249
syntaxis
ambiguïteit 76
dangling else 76
grammaticaal correct 338
van C 509
van #define 121
van preprocessor 514
systeemvariabele
autoexec.bat 31
envp 103
INCLUDE 31, 120
LIB 31
MM 32, 266
PATH 31
zetten 31
system() 234
sytaxis
lexical analyzer 249

tabel
in plaats van functie 216
tag 378
van een struct 187
vergeleken met typedef 187
tail
van een queue 370
talstelsel 22
binair 22
hexadecimaal 22
octaal 22
Tannenbaum 369
teken bit 24
term 347
terminaal symbool 337
ascii-waarde 439
prioriteit en associativiteit in bison 441
ternaire operator 66, 77
absolute waarde 122
genest 66
_text
suffix 275
time() 467
time-space tradeoff 217
tmpfile() 141
toename
bij for-statement 83
toestand
automaat 292
diagram 293
matrix 295
toetsenbordbuffer 370
toevalswaarde 115
toilet 112
toiletpapier 112
Tolkien 295
tool 92
Toonder 22
transitie 296
default 295
diagram
constructie 298
matrix 296
en acties 303
Tremblay 338
true 70
two's complement 25
type
cast 68
informatieverlies 69
char 41
controle 67
conversie 67
default 43
double 41
float 41
int 41
modifier 42
long 42
signed 42
unsigned 42
nieuwe datatypes 184
nieuwe types 184
samengestelde types 184

void 41
typedef 184
bij een struct 186
en enums 195
gebruik 185
syntaxis 184

%u 132
uitvoering
conditioneel 74
sequentieel 74
Ullman 92, 346, 397, 412, 422
unaire operatoren 58
unbuffered I/O 128, 141
unformated I/O 131
functies 131
ungetc() 353
union 189
allocatie 193
genest 190
naamloos 395
initialisatie 193
omvang in bytes 189
UNIX 128
unlink() 233
unsigned 42

va_arg() 521
en array argumenten 522
en char parameters 522
vangnet-expressie van yylex() 430
variabel aantal argumenten 100, 521
variabele 19
adres van een 47, 150
auto 46
controle 77, 79
declaratie 50
definitie 43, 50
extern 50
geheugenplaats 43
globaal 46
het verbergen van 49
het wijzen op een variabele 156
in het default datasegment 262
in welk datasegment? 263
initiële waarde 50
levensduur 45
lokaal 45
en globaal 20
naam 43
pointervariabele 150
register 49
sizeof 60
static 48
toegestane definities 175
type 43
van type struct 185
wijzigen
vanuit een functie 152
via return-waarde 152
zichtbaarheid 45
Variable Definer 174
va_start() 521
en array argumenten 521
en char parameters 521
vd.exe 174
vector 52
veld
pointer 187
notatie 187
selector 187
van een struct 185
van een union 189
Vens 5
verborgen variabelen 49
Vermeer 5
Veroveraar, Willem de 16
verpleegkundige 481
verpleegster 314
versienummer 240
vertaler 26
en functiebibliotheek 17
en de stack 361
optie
/A 265
/AC 267
/AH 268
/AL 267
/AM 266
/AS 266
/D 124
/E 125, 310
/Gt 263
/J 41
/Nt 275
/Zs 310
verzameling 319
vlag 400
bit-vlag 214, 455
van programma-aanroep 460
void 41
pointer 158
volatile 42
vuistregel
bij een variabel aantal argumenten 102
bij gemengd model programma's 269
bij hashing 412
bij het hashen 415
bij lijstmodificaties 382
bij _near en _far 275

code splitsen 224
error regel in grammatica 476
voor recursieve functies 317
Vunzo 5

waardenbereik 41
waardetoekenning
aan bit-velden 188
aan struct-velden 187
aan variabelen buiten functies 152
via argumenten 152
wachtrij 369
Weinberger 412
werkrooster 480
Weterings 5
wetten van De Morgan 72
while 81
wijzen op een variabele 156
wildcards 104
bij de library manager 228
Willem de Veroveraar 16
Wirth 360
word 23
worst 499
wortel
van een boom 391
write() 142

%x 132, 135
xfopen() 140

yyerror() 456
context-gevoelige foutmelding 473
yyin 426, 472
yy_init 472
yylex() 423
communicatie
met andere functies 426
met yyparse() 437
default actie 430
verwerking van meerdere bestanden 472
yyparse() 423
communicatie met yylex() 437
error regel 474
herstel na foutconditie 474
stack type 437
YYSTYPE 437
yytext 426, 431

Zeef
van Aristothenes 212
implementatie 212, 218
zichtbaarheid van variabelen 45
zoeken 169, 410
binair 333
zie bsearch() 254
lineair 306
via een sleutel 410
/Zs 310